# basic-usage: Basic Usage
URL: /docs/basic-usage
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/basic-usage.mdx

Getting started with Better Auth

***

title: Basic Usage
description: Getting started with Better Auth

Better Auth provides built-in authentication support for:

* **Email and password**
* **Social provider (Google, GitHub, Apple, and more)**

But also can easily be extended using plugins, such as: [username](/docs/plugins/username), [magic link](/docs/plugins/magic-link), [passkey](/docs/plugins/passkey), [email-otp](/docs/plugins/email-otp), and more.

## Email & Password

To enable email and password authentication:

```ts title="auth.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    emailAndPassword: {    // [!code highlight]
        enabled: true // [!code highlight]
    } // [!code highlight]
})
```

### Sign Up

To sign up a user you need to call the client method `signUp.email` with the user's information.

```ts title="sign-up.ts"
import { authClient } from "@/lib/auth-client"; //import the auth client // [!code highlight]

const { data, error } = await authClient.signUp.email({
        email, // user email address
        password, // user password -> min 8 characters by default
        name, // user display name
        image, // User image URL (optional)
        callbackURL: "/dashboard" // A URL to redirect to after the user verifies their email (optional)
    }, {
        onRequest: (ctx) => {
            //show loading
        },
        onSuccess: (ctx) => {
            //redirect to the dashboard or sign in page
        },
        onError: (ctx) => {
            // display the error message
            alert(ctx.error.message);
        },
});
```

By default, the users are automatically signed in after they successfully sign up. To disable this behavior you can set `autoSignIn` to `false`.

```ts title="auth.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    emailAndPassword: {
        enabled: true,
        autoSignIn: false //defaults to true // [!code highlight]
  },
})
```

### Sign In

To sign a user in, you can use the `signIn.email` function provided by the client.

```ts title="sign-in"
const { data, error } = await authClient.signIn.email({
        /**
         * The user email
         */
        email,
        /**
         * The user password
         */
        password,
        /**
         * A URL to redirect to after the user verifies their email (optional)
         */
        callbackURL: "/dashboard",
        /**
         * remember the user session after the browser is closed.
         * @default true
         */
        rememberMe: false
}, {
    //callbacks
})
```

<Callout type="warn">
  Always invoke client methods from the client side. Don't call them from the server.
</Callout>

### Server-Side Authentication

To authenticate a user on the server, you can use the `auth.api` methods.

```ts title="server.ts"
import { auth } from "./auth"; // path to your Better Auth server instance

const response = await auth.api.signInEmail({
    body: {
        email,
        password
    },
    asResponse: true // returns a response object instead of data
});
```

<Callout>
  If the server cannot return a response object, you'll need to manually parse and set cookies. But for frameworks like Next.js we provide [a plugin](/docs/integrations/next#server-action-cookies) to handle this automatically
</Callout>

## Social Sign-On

Better Auth supports multiple social providers, including Google, GitHub, Apple, Discord, and more. To use a social provider, you need to configure the ones you need in the `socialProviders` option on your `auth` object.

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
    socialProviders: { // [!code highlight]
        github: { // [!code highlight]
            clientId: process.env.GITHUB_CLIENT_ID!, // [!code highlight]
            clientSecret: process.env.GITHUB_CLIENT_SECRET!, // [!code highlight]
        } // [!code highlight]
    }, // [!code highlight]
})
```

### Sign in with social providers

To sign in using a social provider you need to call `signIn.social`. It takes an object with the following properties:

```ts title="sign-in.ts"
import { authClient } from "@/lib/auth-client"; //import the auth client // [!code highlight]

await authClient.signIn.social({
    /**
     * The social provider ID
     * @example "github", "google", "apple"
     */
    provider: "github",
    /**
     * A URL to redirect after the user authenticates with the provider
     * @default "/"
     */
    callbackURL: "/dashboard",
    /**
     * A URL to redirect if an error occurs during the sign in process
     */
    errorCallbackURL: "/error",
    /**
     * A URL to redirect if the user is newly registered
     */
    newUserCallbackURL: "/welcome",
    /**
     * disable the automatic redirect to the provider.
     * @default false
     */
    disableRedirect: true,
});
```

You can also authenticate using `idToken` or `accessToken` from the social provider instead of redirecting the user to the provider's site. See social providers documentation for more details.

## Signout

To signout a user, you can use the `signOut` function provided by the client.

```ts title="user-card.tsx"
await authClient.signOut();
```

you can pass `fetchOptions` to redirect onSuccess

```ts title="user-card.tsx"
await authClient.signOut({
  fetchOptions: {
    onSuccess: () => {
      router.push("/login"); // redirect to login page
    },
  },
});
```

## Session

Once a user is signed in, you'll want to access the user session. Better Auth allows you to easily access the session data from both the server and client sides.

### Client Side

#### Use Session

Better Auth provides a `useSession` hook to easily access session data on the client side. This hook is implemented using nanostore and has support for each supported framework and vanilla client, ensuring that any changes to the session (such as signing out) are immediately reflected in your UI.

<Tabs items={["React", "Vue","Svelte", "Solid", "Vanilla"]} defaultValue="react">
  <Tab value="React">
    ```tsx title="user.tsx"
    import { authClient } from "@/lib/auth-client" // import the auth client // [!code highlight]

    export function User(){

        const { // [!code highlight]
            data: session, // [!code highlight]
            isPending, //loading state // [!code highlight]
            error, //error object // [!code highlight]
            refetch //refetch the session
        } = authClient.useSession() // [!code highlight]

        return (
            //...
        )
    }
    ```
  </Tab>

  <Tab value="Vue">
    ```vue title="index.vue"
    <script setup lang="ts">
    import { authClient } from "~/lib/auth-client" // [!code highlight]

    const session = authClient.useSession() // [!code highlight]
    </script>

    <template>
        <div>
            <div>
                <pre>{{ session.data }}</pre>
                <button v-if="session.data" @click="authClient.signOut()">
                    Sign out
                </button>
            </div>
        </div>
    </template>
    ```
  </Tab>

  <Tab value="Svelte">
    ```svelte title="user.svelte"
    <script lang="ts">
    import { authClient } from "$lib/auth-client"; // [!code highlight]

````mdx
    const session = authClient.useSession(); // [!code highlight]
    </script>
    <p>
        {$session.data?.user.email}
    </p>
    ```
  </Tab>

  <Tab value="Vanilla">
    ```ts title="user.svelte"
    import { authClient } from "~/lib/auth-client"; //import the auth client

    authClient.useSession.subscribe((value)=>{
        //do something with the session //
    })
    ```
  </Tab>

  <Tab value="Solid">
    ```tsx title="user.tsx"
    import { authClient } from "~/lib/auth-client"; // [!code highlight]

    export default function Home() {
        const session = authClient.useSession() // [!code highlight]
        return (
            <pre>{JSON.stringify(session(), null, 2)}</pre>
        );
    }
    ```
  </Tab>
</Tabs>

#### Get Session

If you prefer not to use the hook, you can use the `getSession` method provided by the client.

```ts title="user.tsx"
import { authClient } from "@/lib/auth-client" // import the auth client // [!code highlight]

const { data: session, error } = await authClient.getSession()
```

You can also use it with client-side data-fetching libraries like [TanStack Query](https://tanstack.com/query/latest).

### Server Side

The server provides a `session` object that you can use to access the session data. It requires request headers object to be passed to the `getSession` method.

Example: Using some popular frameworks

<Tabs items={["Next.js", "Nuxt", "Svelte", "Astro", "Hono", "TanStack"]}>
  <Tab value="Next.js">
    ```ts title="server.ts"
    import { auth } from "./auth"; // path to your Better Auth server instance
    import { headers } from "next/headers";

    const session = await auth.api.getSession({
        headers: await headers() // you need to pass the headers object.
    })
    ```
  </Tab>

  <Tab value="Remix">
    ```ts title="route.ts"
    import { auth } from "lib/auth"; // path to your Better Auth server instance

    export async function loader({ request }: LoaderFunctionArgs) {
        const session = await auth.api.getSession({
            headers: request.headers
        })

        return json({ session })
    }
    ```
  </Tab>

  <Tab value="Astro">
    ```astro title="index.astro"
    ---
    import { auth } from "./auth";

    const session = await auth.api.getSession({
        headers: Astro.request.headers,
    });
    ---
    <!-- Your Astro Template -->
    ```
  </Tab>

  <Tab value="Svelte">
    ```ts title="+page.ts"
    import { auth } from "./auth";

    export async function load({ request }) {
        const session = await auth.api.getSession({
            headers: request.headers
        })
        return {
            props: {
                session
            }
        }
    }
    ```
  </Tab>

  <Tab value="Hono">
    ```ts title="index.ts"
    import { auth } from "./auth";

    const app = new Hono();

    app.get("/path", async (c) => {
        const session = await auth.api.getSession({
            headers: c.req.raw.headers
        })
    });
    ```
  </Tab>

  <Tab value="Nuxt">
    ```ts title="server/session.ts"
    import { auth } from "~/utils/auth";

    export default defineEventHandler((event) => {
        const session = await auth.api.getSession({
            headers: event.headers,
        })
    });
    ```
  </Tab>
````

````
  <Tab value="TanStack">
    ```ts title="app/routes/api/index.ts"
    import { auth } from "./auth";
    import { createAPIFileRoute } from "@tanstack/start/api";

    export const APIRoute = createAPIFileRoute("/api/$")({
        GET: async ({ request }) => {
            const session = await auth.api.getSession({
                headers: request.headers
            })
        },
    });
    ```
  </Tab>
</Tabs>

<Callout>
  For more details check [session-management](/docs/concepts/session-management) documentation.
</Callout>

## Using Plugins

One of the unique features of Better Auth is a plugins ecosystem. It allows you to add complex auth related functionality with small lines of code.

Below is an example of how to add two factor authentication using two factor plugin.

<Steps>
  <Step>
    ### Server Configuration

    To add a plugin, you need to import the plugin and pass it to the `plugins` option of the auth instance. For example, to add two factor authentication, you can use the following code:

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { twoFactor } from "better-auth/plugins" // [!code highlight]

    export const auth = betterAuth({
        //...rest of the options
        plugins: [ // [!code highlight]
            twoFactor() // [!code highlight]
        ] // [!code highlight]
    })
    ```

    now two factor related routes and method will be available on the server.
  </Step>

  <Step>
    ### Migrate Database

    After adding the plugin, you'll need to add the required tables to your database. You can do this by running the `migrate` command, or by using the `generate` command to create the schema and handle the migration manually.

    generating the schema:

    ```bash title="terminal"
    npx @better-auth/cli generate
    ```

    using the `migrate` command:

    ```bash title="terminal"
    npx @better-auth/cli migrate
    ```

    <Callout>
      If you prefer adding the schema manually, you can check the schema required on the [two factor plugin](/docs/plugins/2fa#schema) documentation.
    </Callout>
  </Step>

  <Step>
    ### Client Configuration

    Once we're done with the server, we need to add the plugin to the client. To do this, you need to import the plugin and pass it to the `plugins` option of the auth client. For example, to add two factor authentication, you can use the following code:

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client";
    import { twoFactorClient } from "better-auth/client/plugins"; // [!code highlight]

    const authClient = createAuthClient({
        plugins: [ // [!code highlight]
            twoFactorClient({ // [!code highlight]
                twoFactorPage: "/two-factor" // the page to redirect if a user needs to verify 2nd factor // [!code highlight]
            }) // [!code highlight]
        ] // [!code highlight]
    })
    ```

    now two factor related methods will be available on the client.

    ```ts title="profile.ts"
    import { authClient } from "./auth-client"

    const enableTwoFactor = async() => {
        const data = await authClient.twoFactor.enable({
            password // the user password is required
        }) // this will enable two factor
    }

    const disableTwoFactor = async() => {
        const data = await authClient.twoFactor.disable({
            password // the user password is required
        }) // this will disable two factor
    }

    const signInWith2Factor = async() => {
        const data = await authClient.signIn.email({
            //...
        })
        //if the user has two factor enabled, it will redirect to the two factor page
    }

    const verifyTOTP = async() => {
        const data = await authClient.twoFactor.verifyTOTP({
            code: "123456", // the code entered by the user
            /**
             * If the device is trusted, the user won't
             * need to pass 2FA again on the same device
             */
            trustDevice: true
        })
    }
    ```
  </Step>
````

````
  <Step>
    Next step: See the <Link href="/docs/plugins/2fa">two factor plugin documentation</Link>.
  </Step>
</Steps>

# comparison: Comparison
URL: /docs/comparison
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/comparison.mdx

Comparison of Better Auth versus over other auth libraries and services.

***

title: Comparison
description: Comparison of Better Auth versus over other auth libraries and services.
-------------------------------------------------------------------------------------

> <p className="text-orange-200">Comparison is the thief of joy.</p>

Here are non detailed reasons why you may want to use Better Auth over other auth libraries and services.

### vs Other Auth Libraries

* **Framework agnostic** - Works with any framework, not just specific ones
* **Advanced features built-in** - 2FA, multi-tenancy, multi-session, rate limiting, and many more
* **Plugin system** - Extend functionality without forking or complex workarounds
* **Full control** - Customize auth flows exactly how you want

### vs Self-Hosted Auth Servers

* **No separate infrastructure** - Runs in your app, users stay in your database
* **Zero server maintenance** - No auth servers to deploy, monitor, or update
* **Complete feature set** - Everything you need without the operational overhead

### vs Managed Auth Services

* **Keep your data** - Users stay in your database, not a third-party service
* **No per-user costs** - Scale without worrying about auth billing
* **Single source of truth** - All user data in one place

### vs Rolling Your Own

* **Security handled** - Battle-tested auth flows and security practices
* **Focus on your product** - Spend time on features that matter to your business
* **Plugin extensibility** - Add custom features without starting from scratch

# installation: Installation
URL: /docs/installation
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/installation.mdx

Learn how to configure Better Auth in your project.

***

title: Installation
description: Learn how to configure Better Auth in your project.
----------------------------------------------------------------

<Steps>
  <Step>
    ### Install the Package

    Let's start by adding Better Auth to your project:

    <CodeBlockTabs defaultValue="npm">
      <CodeBlockTabsList>
        <CodeBlockTabsTrigger value="npm">
          npm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="pnpm">
          pnpm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="yarn">
          yarn
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="bun">
          bun
        </CodeBlockTabsTrigger>
      </CodeBlockTabsList>

      <CodeBlockTab value="npm">
        ```bash
        npm install better-auth
        ```
      </CodeBlockTab>

      <CodeBlockTab value="pnpm">
        ```bash
        pnpm add better-auth
        ```
      </CodeBlockTab>

      <CodeBlockTab value="yarn">
        ```bash
        yarn add better-auth
        ```
      </CodeBlockTab>

      <CodeBlockTab value="bun">
        ```bash
        bun add better-auth
        ```
      </CodeBlockTab>
    </CodeBlockTabs>

    <Callout type="info">
      If you're using a separate client and server setup, make sure to install Better Auth in both parts of your project.
    </Callout>
  </Step>

  <Step>
    ### Set Environment Variables

    Create a `.env` file in the root of your project and add the following environment variables:

    1. **Secret Key**

    Random value used by the library for encryption and generating hashes. **You can generate one using the button below** or you can use something like openssl.

    ```txt title=".env"
    BETTER_AUTH_SECRET=
    ```

    <GenerateSecret />
~~~

````mdx
  2. **Set Base URL**

  ```txt title=".env"
  BETTER_AUTH_URL=http://localhost:3000 # Base URL of your app
  ```
</Step>

<Step>
  ### Create A Better Auth Instance

  Create a file named `auth.ts` in one of these locations:

  * Project root
  * `lib/` folder
  * `utils/` folder

  You can also nest any of these folders under `src/`, `app/` or `server/` folder. (e.g. `src/lib/auth.ts`, `app/lib/auth.ts`).

  And in this file, import Better Auth and create your auth instance. Make sure to export the auth instance with the variable name `auth` or as a `default` export.

  ```ts title="auth.ts"
  import { betterAuth } from "better-auth";

  export const auth = betterAuth({
    //...
  });
  ```
</Step>

<Step>
  ### Configure Database

  Better Auth requires a database to store user data.
  You can easily configure Better Auth to use SQLite, PostgreSQL, or MySQL, and more!

  <Tabs items={["sqlite", "postgres", "mysql"]}>
    <Tab value="sqlite">
      ```ts title="auth.ts"
      import { betterAuth } from "better-auth";
      import Database from "better-sqlite3";

      export const auth = betterAuth({
          database: new Database("./sqlite.db"),
      })
      ```
    </Tab>

    <Tab value="postgres">
      ```ts title="auth.ts"
      import { betterAuth } from "better-auth";
      import { Pool } from "pg";

      export const auth = betterAuth({
          database: new Pool({
              // connection options
          }),
      })
      ```
    </Tab>

    <Tab value="mysql">
      ```ts title="auth.ts"
      import { betterAuth } from "better-auth";
      import { createPool } from "mysql2/promise";

      export const auth = betterAuth({
          database: createPool({
              // connection options
          }),
      })
      ```
    </Tab>
  </Tabs>

  Alternatively, if you prefer to use an ORM, you can use one of the built-in adapters.

  <Tabs items={["drizzle", "prisma", "mongodb"]}>
    <Tab value="drizzle">
      ```ts title="auth.ts"
      import { betterAuth } from "better-auth";
      import { drizzleAdapter } from "better-auth/adapters/drizzle";
      import { db } from "@/db"; // your drizzle instance

      export const auth = betterAuth({
          database: drizzleAdapter(db, {
              provider: "pg", // or "mysql", "sqlite"
          }),
      });
      ```
    </Tab>

    <Tab value="prisma">
      ```ts title="auth.ts"
      import { betterAuth } from "better-auth";
      import { prismaAdapter } from "better-auth/adapters/prisma";
      // If your Prisma file is located elsewhere, you can change the path
      import { PrismaClient } from "@/generated/prisma";

      const prisma = new PrismaClient();
      export const auth = betterAuth({
          database: prismaAdapter(prisma, {
              provider: "sqlite", // or "mysql", "postgresql", ...etc
          }),
      });
      ```
    </Tab>

    <Tab value="mongodb">
      ```ts title="auth.ts"
      import { betterAuth } from "better-auth";
      import { mongodbAdapter } from "better-auth/adapters/mongodb";
      import { client } from "@/db"; // your mongodb client

      export const auth = betterAuth({
          database: mongodbAdapter(client),
      });
      ```
    </Tab>
  </Tabs>

  <Callout>
    If your database is not listed above, check out our other supported
    [databases](/docs/adapters/other-relational-databases) for more information,
    or use one of the supported ORMs.
  </Callout>
</Step>

<Step>
  ### Create Database Tables
~~~

~~~mdx
    Better Auth includes a CLI tool to help manage the schema required by the library.

    * **Generate**: This command generates an ORM schema or SQL migration file.

    <Callout>
      If you're using Kysely, you can apply the migration directly with `migrate` command below. Use `generate` only if you plan to apply the migration manually.
    </Callout>

    ```bash title="Terminal"
    npx @better-auth/cli generate
    ```

    * **Migrate**: This command creates the required tables directly in the database. (Available only for the built-in Kysely adapter)

    ```bash title="Terminal"
    npx @better-auth/cli migrate
    ```

    see the [CLI documentation](/docs/concepts/cli) for more information.

    <Callout>
      If you instead want to create the schema manually, you can find the core schema required in the [database section](/docs/concepts/database#core-schema).
    </Callout>
  </Step>

  <Step>
    ### Authentication Methods

    Configure the authentication methods you want to use. Better Auth comes with built-in support for email/password, and social sign-on providers.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth";

    export const auth = betterAuth({
      //...other options // [!code highlight]
      emailAndPassword: { // [!code highlight]
        enabled: true, // [!code highlight]
      }, // [!code highlight]
      socialProviders: { // [!code highlight]
        github: { // [!code highlight]
          clientId: process.env.GITHUB_CLIENT_ID as string, // [!code highlight]
          clientSecret: process.env.GITHUB_CLIENT_SECRET as string, // [!code highlight]
        }, // [!code highlight]
      }, // [!code highlight]
    });
    ```

    <Callout type="info">
      You can use even more authentication methods like [passkey](/docs/plugins/passkey), [username](/docs/plugins/username), [magic link](/docs/plugins/magic-link) and more through plugins.
    </Callout>
  </Step>

  <Step>
    ### Mount Handler

    To handle API requests, you need to set up a route handler on your server.

    Create a new file or route in your framework's designated catch-all route handler. This route should handle requests for the path `/api/auth/*` (unless you've configured a different base path).

    <Callout>
      Better Auth supports any backend framework with standard Request and Response
      objects and offers helper functions for popular frameworks.
    </Callout>

    <Tabs items={["next-js", "nuxt", "svelte-kit", "remix", "solid-start", "hono", "express", "elysia", "tanstack-start", "expo"]} defaultValue="react">
      <Tab value="next-js">
        ```ts title="/app/api/auth/[...all]/route.ts"
        import { auth } from "@/lib/auth"; // path to your auth file
        import { toNextJsHandler } from "better-auth/next-js";

        export const { POST, GET } = toNextJsHandler(auth);
        ```
      </Tab>

      <Tab value="nuxt">
        ```ts title="/server/api/auth/[...all].ts"
        import { auth } from "~/utils/auth"; // path to your auth file

        export default defineEventHandler((event) => {
            return auth.handler(toWebRequest(event));
        });
        ```
      </Tab>

      <Tab value="svelte-kit">
        ```ts title="hooks.server.ts"
        import { auth } from "$lib/auth"; // path to your auth file
        import { svelteKitHandler } from "better-auth/svelte-kit";
        import { building } from '$app/environment'

        export async function handle({ event, resolve }) {
            return svelteKitHandler({ event, resolve, auth, building });
        }
        ```
      </Tab>

      <Tab value="remix">
        ```ts title="/app/routes/api.auth.$.ts"
        import { auth } from '~/lib/auth.server' // Adjust the path as necessary
        import type { LoaderFunctionArgs, ActionFunctionArgs } from "@remix-run/node"

        export async function loader({ request }: LoaderFunctionArgs) {
            return auth.handler(request)
        }

        export async function action({ request }: ActionFunctionArgs) {
            return auth.handler(request)
        }
        ```
      </Tab>

      <Tab value="solid-start">
        ```ts title="/routes/api/auth/*all.ts"
        import { auth } from "~/lib/auth"; // path to your auth file
        import { toSolidStartHandler } from "better-auth/solid-start";

        export const { GET, POST } = toSolidStartHandler(auth);
        ```
      </Tab>

      <Tab value="hono">
        ```ts title="src/index.ts"
        import { Hono } from "hono";
        import { auth } from "./auth"; // path to your auth file
        import { serve } from "@hono/node-server";
        import { cors } from "hono/cors";

        const app = new Hono();
~~~

~~~
    app.on(["POST", "GET"], "/api/auth/*", (c) => auth.handler(c.req.raw));

    serve(app);
    ```
   </Tab>

   <Tab value="express">
    <Callout type="warn">
     ExpressJS v5 introduced breaking changes to route path matching by switching to `path-to-regexp@6`. Wildcard routes like `*` should now be written using the new named
syntax, e.g. `/{*any}`, to properly capture catch-all patterns. This ensures compatibility and predictable behavior across routing scenarios.
     See the [Express v5 migration guide](https://expressjs.com/en/guide/migrating-5.html) for details.

     As a result, the implementation in ExpressJS v5 should look like this:

     ```ts
     app.all('/api/auth/{*any}', toNodeHandler(auth));
     ```

     *The name any is arbitrary and can be replaced with any identifier you prefer.*
    </Callout>

    ```ts title="server.ts"
    import express from "express";
    import { toNodeHandler } from "better-auth/node";
    import { auth } from "./auth";

    const app = express();
    const port = 8000;

    app.all("/api/auth/*", toNodeHandler(auth));

    // Mount express json middleware after Better Auth handler
    // or only apply it to routes that don't interact with Better Auth
    app.use(express.json());

    app.listen(port, () => {
      console.log(`Better Auth app listening on port ${port}`);
    });
    ```

    This will also work for any other node server framework like express, fastify, hapi, etc., but may require some modifications. See [fastify guide]
(/docs/integrations/fastify). Note that CommonJS (cjs) isn't supported.
   </Tab>

   <Tab value="astro">
    ```ts title="/pages/api/auth/[...all].ts"
    import type { APIRoute } from "astro";
    import { auth } from "@/auth"; // path to your auth file

    export const GET: APIRoute = async (ctx) => {
      return auth.handler(ctx.request);
    };

    export const POST: APIRoute = async (ctx) => {
      return auth.handler(ctx.request);
    };
    ```
   </Tab>

   <Tab value="elysia">
    ```ts
    import { Elysia, Context } from "elysia";
    import { auth } from "./auth";

    const betterAuthView = (context: Context) => {
      const BETTER_AUTH_ACCEPT_METHODS = ["POST", "GET"]
      // validate request method
      if(BETTER_AUTH_ACCEPT_METHODS.includes(context.request.method)) {
        return auth.handler(context.request);
      } else {
        context.error(405)
      }
    }

    const app = new Elysia().all("/api/auth/*", betterAuthView).listen(3000);

    console.log(
    `ğŸ¦Š Elysia is running at ${app.server?.hostname}:${app.server?.port}`
    );
    ```
   </Tab>

   <Tab value="tanstack-start">
    ```ts title="src/routes/api/auth/$.ts"
    import { createFileRoute } from '@tanstack/react-router'
    import { auth } from '@/lib/auth/auth'

    export const Route = createFileRoute('/api/auth/$')({
    server: {
      handlers: {
        GET: async ({ request }:{ request: Request }) => {
          return await auth.handler(request)
        },
        POST: async ({ request }:{ request: Request }) => {
          return await auth.handler(request)
        },
      },
    },
    })
    ```
   </Tab>

   <Tab value="expo">
    ```ts title="app/api/auth/[...all]+api.ts"
    import { auth } from '@/lib/server/auth'; // path to your auth file

    const handler = auth.handler;
    export { handler as GET, handler as POST };

   </Tab>
    ```
  </Tabs>
 </Step>

 <Step>
  ### Create Client Instance

  The client-side library helps you interact with the auth server. Better Auth comes with a client for all the popular web frameworks, including vanilla JavaScript.

  1. Import `createAuthClient` from the package for your framework (e.g., "better-auth/react" for React).
  2. Call the function to create your client.
  3. Pass the base URL of your auth server. (If the auth server is running on the same domain as your client, you can skip this step.)

  <Callout type="info">
   If you're using a different base path other than `/api/auth` make sure to pass
   the whole URL including the path. (e.g.
   `http://localhost:3000/custom-path/auth`)
  </Callout>
~~~

~~~
    <Tabs
      items={["react", "vue", "svelte", "solid",
"vanilla"]}
      defaultValue="react"
    >
      <Tab value="vanilla">
        ```ts title="lib/auth-client.ts"
        import { createAuthClient } from "better-auth/client"
        export const authClient = createAuthClient({
            /** The base URL of the server (optional if you're using the same domain) */ // [!code highlight]
            baseURL: "http://localhost:3000" // [!code highlight]
        })
        ```
      </Tab>

      <Tab value="react" title="lib/auth-client.ts">
        ```ts title="lib/auth-client.ts"
        import { createAuthClient } from "better-auth/react"
        export const authClient = createAuthClient({
            /** The base URL of the server (optional if you're using the same domain) */ // [!code highlight]
            baseURL: "http://localhost:3000" // [!code highlight]
        })
        ```
      </Tab>

      <Tab value="vue" title="lib/auth-client.ts">
        ```ts title="lib/auth-client.ts"
        import { createAuthClient } from "better-auth/vue"
        export const authClient = createAuthClient({
            /** The base URL of the server (optional if you're using the same domain) */ // [!code highlight]
            baseURL: "http://localhost:3000" // [!code highlight]
        })
        ```
      </Tab>

      <Tab value="svelte" title="lib/auth-client.ts">
        ```ts title="lib/auth-client.ts"
        import { createAuthClient } from "better-auth/svelte"
        export const authClient = createAuthClient({
            /** The base URL of the server (optional if you're using the same domain) */ // [!code highlight]
            baseURL: "http://localhost:3000" // [!code highlight]
        })
        ```
      </Tab>

      <Tab value="solid" title="lib/auth-client.ts">
        ```ts title="lib/auth-client.ts"
        import { createAuthClient } from "better-auth/solid"
        export const authClient = createAuthClient({
            /** The base URL of the server (optional if you're using the same domain) */ // [!code highlight]
            baseURL: "http://localhost:3000" // [!code highlight]
        })
        ```
      </Tab>
    </Tabs>

    <Callout type="info">
      Tip: You can also export specific methods if you prefer:
    </Callout>

    ```ts
    export const { signIn, signUp, useSession } = createAuthClient()
    ```
  </Step>

  <Step>
    ### ğŸ ‰ That's it!

    That's it! You're now ready to use better-auth in your application. Continue to [basic usage](/docs/basic-usage) to learn how to use the auth instance to sign in users.
  </Step>
</Steps>



# introduction: Introduction
URL: /docs/introduction
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/introduction.mdx

Introduction to Better Auth.

***

title: Introduction
description: Introduction to Better Auth.
-----------------------------------------

Better Auth is a framework-agnostic, universal authentication and authorization framework for TypeScript. It provides a comprehensive set of features out of the box and includes
a plugin ecosystem that simplifies adding advanced functionalities. Whether you need 2FA, passkey, multi-tenancy, multi-session support, or even enterprise features like SSO,
creating your own IDP, it lets you focus on building your application instead of reinventing the wheel.

## Features

Better Auth aims to be the most comprehensive auth library. It provides a wide range of features out of the box and allows you to extend it with plugins. Here are some of the
features:

<Features />

...and much more!

***

## AI tooling

### LLMs.txt

Better Auth exposes an `LLMs.txt` that helps AI models understand how to integrate and interact with your authentication system. See it at [https://better-auth.com/llms.txt]
(https://better-auth.com/llms.txt).

### MCP

Better Auth provides an MCP server so you can use it with any AI model that supports the Model Context Protocol (MCP).

<AddToCursor />

#### CLI Options

Use the Better Auth CLI to easily add the MCP server to your preferred client:

<Tabs items={["Cursor", "Claude Code", "Open Code", "Manual"]}>
  <Tab value="Cursor">
    ```bash title="terminal"
    pnpm @better-auth/cli mcp --cursor
    ```
  </Tab>

  <Tab value="Claude Code">
    ```bash title="terminal"
    pnpm @better-auth/cli mcp --claude-code
    ```
  </Tab>
~~~

~~~text
  <Tab value="Open Code">
    ```bash title="terminal"
    pnpm @better-auth/cli mcp --open-code
    ```
  </Tab>

  <Tab value="Manual">
    ```bash title="terminal"
    pnpm @better-auth/cli mcp --manual
    ```
  </Tab>
</Tabs>

#### Manual Configuration

Alternatively, you can manually configure the MCP server for each client:

<Tabs items={["Claude Code", "Open Code", "Manual"]}>
  <Tab value="Claude Code">
    ```bash title="terminal"
    claude mcp add --transport http better-auth https://mcp.chonkie.ai/better-auth/better-auth-builder/mcp
    ```
  </Tab>

  <Tab value="Open Code">
    ```json title="opencode.json"
      {
          "$schema": "https://opencode.ai/config.json",
          "mcp": {
              "Better Auth": {
                  "type": "remote",
                  "url": "https://mcp.chonkie.ai/better-auth/better-auth-builder/mcp",
                  "enabled": true,
              }
          }
      }
    ```
  </Tab>

  <Tab value="Manual">
    ```json title="mcp.json"
    {
       "Better Auth": {
           "url": "https://mcp.chonkie.ai/better-auth/better-auth-builder/mcp"
       }
    }
    ```
  </Tab>
</Tabs>

<Callout>
  We provide a firstâ€‘party MCP, powered by [Chonkie](https://chonkie.ai). You can alternatively use [`context7`](https://context7.com/) and other MCP providers.
</Callout>



# adapters: Community Adapters
URL: /docs/adapters/community-adapters
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/adapters/community-adapters.mdx

Integrate Better Auth with community made database adapters.

***

title: Community Adapters
description: Integrate Better Auth with community made database adapters.
-------------------------------------------------------------------------

This page showcases a list of recommended community made database adapters.
We encourage you to create any missing database adapters and maybe get added to the list!

| Adapter                                                                                   | Database Dialect                            | Author
|
| ----------------------------------------------------------------------------------------- | ------------------------------------------- | ----------------------------
----------------------------------------------------------------------------------------------------------------------------------------------------------------- |
| [convex-better-auth](https://github.com/get-convex/better-auth)                           | [Convex Database](https://www.convex.dev/) | <img
src="https://github.com/erquhart.png" className="rounded-full w-6 h-6 border opacity-70 m-0 inline mr-1" /> [erquhart](https://github.com/erquhart)
|
| [surreal-better-auth](https://github.com/oskar-gmerek/surreal-better-auth)                 | [SurrealDB](https://surrealdb.com/)        | <img
src="https://github.com/oskar-gmerek.png" className="rounded-full w-6 h-6 border opacity-70 m-0 inline mr-1" /> <a href="https://oskargmerek.com" alt="Web Developer UK">Oskar
Gmerek</a> |
| [surrealdb-better-auth](https://github.com/Necmttn/surrealdb-better-auth)                  | [Surreal Database](https://surrealdb.com/) | <img
src="https://github.com/Necmttn.png" className="rounded-full w-6 h-6 border opacity-70 m-0 inline mr-1" /> [Necmttn](https://github.com/Necmttn)
|
| [better-auth-surrealdb](https://github.com/msanchezdev/better-auth-surrealdb)             | [Surreal Database](https://surrealdb.com/) | <img
src="https://github.com/msanchezdev.png" className="rounded-full w-6 h-6 border opacity-70 m-0 inline mr-1" /> [msanchezdev](https://github.com/msanchezdev)
|
| [payload-better-auth](https://github.com/ForrestDevs/payload-better-auth/tree/main/packages/db-adapter) | [Payload CMS](https://payloadcms.com/)     | <img
src="https://github.com/forrestdevs.png" className="rounded-full w-6 h-6 border opacity-70 m-0 inline mr-1" /> [forrestdevs](https://github.com/forrestdevs)
|
| [@ronin/better-auth](https://github.com/ronin-co/better-auth)                             | [RONIN](https://ronin.co)                  | <img
src="https://github.com/ronin-co.png" className="rounded-full w-6 h-6 border opacity-70 m-0 inline mr-1" /> [ronin-co](https://github.com/ronin-co)
|
| [better-auth-instantdb](https://github.com/daveyplate/better-auth-instantdb)              | [InstantDB](https://www.instantdb.com/)    | <img
src="https://github.com/daveycodez.png" className="rounded-full w-6 h-6 border opacity-70 m-0 inline mr-1" /> [daveycodez](https://github.com/daveycodez)
|
| [@nerdfolio/remult-better-auth](https://github.com/nerdfolio/remult-better-auth)          | [Remult](https://remult.dev/)              | <img
src="https://github.com/taivo.png" className="rounded-full w-6 h-6 border opacity-70 m-0 inline mr-1" /> [Tai Vo](https://github.com/taivo)
|



# adapters: Drizzle ORM Adapter
URL: /docs/adapters/drizzle
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/adapters/drizzle.mdx

Integrate Better Auth with Drizzle ORM.

***

title: Drizzle ORM Adapter
description: Integrate Better Auth with Drizzle ORM.
----------------------------------------------------

Drizzle ORM is a powerful and flexible ORM for Node.js and TypeScript. It provides a simple and intuitive API for working with databases, and supports a wide range of databases
including MySQL, PostgreSQL, SQLite, and more.
Read more here: [Drizzle ORM](https://orm.drizzle.team/).

## Example Usage

Make sure you have Drizzle installed and configured.
Then, you can use the Drizzle adapter to connect to your database.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { drizzleAdapter } from "better-auth/adapters/drizzle";
import { db } from "./database.ts";
~~~

~~~
export const auth = betterAuth({
  database: drizzleAdapter(db, {
    // [!code highlight]
    provider: "sqlite", // or "pg" or "mysql" // [!code highlight]
  }), // [!code highlight]
  //... the rest of your config
});
```

## Schema generation & migration

The [Better Auth CLI](/docs/concepts/cli) allows you to generate or migrate
your database schema based on your Better Auth configuration and plugins.

To generate the schema required by Better Auth, run the following command:

```bash title="Schema Generation"
npx @better-auth/cli@latest generate
```

To generate and apply the migration, run the following commands:

```bash title="Schema Migration"
npx drizzle-kit generate # generate the migration file
npx drizzle-kit migrate # apply the migration
```

## Additional Information

The Drizzle adapter expects the schema you define to match the table names. For example, if your Drizzle schema maps the `user` table to `users`, you need to manually pass the
schema and map it to the user table.

```ts
import { betterAuth } from "better-auth";
import { db } from "./drizzle";
import { drizzleAdapter } from "better-auth/adapters/drizzle";
import { schema } from "./schema";

export const auth = betterAuth({
  database: drizzleAdapter(db, {
    provider: "sqlite", // or "pg" or "mysql"
    schema: {
      ...schema,
      user: schema.users,
    },
  }),
});
```

If all your tables are using plural form, you can just pass the `usePlural` option:

```ts
export const auth = betterAuth({
  database: drizzleAdapter(db, {
    ...
    usePlural: true,
  }),
});
```

If you're looking for performance improvements or tips, take a look at our guide to <Link href="/docs/guides/optimizing-for-performance">performance optimizations</Link>.



# adapters: MongoDB Adapter
URL: /docs/adapters/mongo
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/adapters/mongo.mdx

Integrate Better Auth with MongoDB.

***

title: MongoDB Adapter
description: Integrate Better Auth with MongoDB.
------------------------------------------------

MongoDB is a popular NoSQL database that is widely used for building scalable and flexible applications. It provides a flexible schema that allows for easy data modeling and
querying.
Read more here: [MongoDB](https://www.mongodb.com/).

## Example Usage

Make sure you have MongoDB installed and configured.
Then, you can use the mongodb adapter.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { MongoClient } from "mongodb";
import { mongodbAdapter } from "better-auth/adapters/mongodb";

const client = new MongoClient("mongodb://localhost:27017/database");
const db = client.db();

export const auth = betterAuth({
  database: mongodbAdapter(db, {
    // Optional: if you don't provide a client, database transactions won't be enabled.
    client
  }),
});
```

## Schema generation & migration

For MongoDB, we don't need to generate or migrate the schema.



# adapters: MS SQL
URL: /docs/adapters/mssql
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/adapters/mssql.mdx

Integrate Better Auth with MS SQL.

***

title: MS SQL
description: Integrate Better Auth with MS SQL.
-----------------------------------------------

Microsoft SQL Server is a relational database management system developed by Microsoft, designed for enterprise-level data storage, management, and analytics with robust security
and scalability features.
Read more [here](https://en.wikipedia.org/wiki/Microsoft_SQL_Server).

## Example Usage

Make sure you have MS SQL installed and configured.
Then, you can connect it straight into Better Auth.
~~~

~~~
```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { MssqlDialect } from "kysely";
import * as Tedious from 'tedious'
import * as Tarn from 'tarn'

const dialect = new MssqlDialect({
  tarn: {
    ...Tarn,
    options: {
      min: 0,
      max: 10,
    },
  },
  tedious: {
    ...Tedious,
    connectionFactory: () => new Tedious.Connection({
      authentication: {
        options: {
          password: 'password',
          userName: 'username',
        },
        type: 'default',
      },
      options: {
        database: 'some_db',
        port: 1433,
        trustServerCertificate: true,
      },
      server: 'localhost',
    }),
  },
  TYPES: {
                ...Tedious.TYPES,
                DateTime: Tedious.TYPES.DateTime2,
        },
})

export const auth = betterAuth({
  database: {
    dialect,
    type: "mssql"
  }
});


```

<Callout>
  For more information, read Kysely's documentation to the [MssqlDialect](https://kysely-org.github.io/kysely-apidoc/classes/MssqlDialect.html).
</Callout>

## Schema generation & migration

The [Better Auth CLI](/docs/concepts/cli) allows you to generate or migrate
your database schema based on your Better Auth configuration and plugins.

<table>
  <thead>
    <tr className="border-b">
      <th>
        <p className="font-bold text-[16px] mb-1">MS SQL Schema Generation</p>
      </th>

      <th>
        <p className="font-bold text-[16px] mb-1">MS SQL Schema Migration</p>
      </th>
    </tr>
  </thead>

  <tbody>
    <tr className="h-10">
      <td>âœ… Supported</td>
      <td>âœ… Supported</td>
    </tr>
  </tbody>
</table>

```bash title="Schema Generation"
npx @better-auth/cli@latest generate
```

```bash title="Schema Migration"
npx @better-auth/cli@latest migrate
```

## Additional Information

MS SQL is supported under the hood via the [Kysely](https://kysely.dev/) adapter, any database supported by Kysely would also be supported. (<Link href="/docs/adapters/other-
relational-databases">Read more here</Link>)

If you're looking for performance improvements or tips, take a look at our guide to <Link href="/docs/guides/optimizing-for-performance">performance optimizations</Link>.



# adapters: MySQL
URL: /docs/adapters/mysql
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/adapters/mysql.mdx

Integrate Better Auth with MySQL.

***

title: MySQL
description: Integrate Better Auth with MySQL.
----------------------------------------------

MySQL is a popular open-source relational database management system (RDBMS) that is widely used for building web applications and other types of software. It provides a flexible
and scalable database solution that allows for efficient storage and retrieval of data.
Read more here: [MySQL](https://www.mysql.com/).

## Example Usage

Make sure you have MySQL installed and configured.
Then, you can connect it straight into Better Auth.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { createPool } from "mysql2/promise";

export const auth = betterAuth({
  database: createPool({
    host: "localhost",
    user: "root",
    password: "password",
    database: "database",
    timezone: "Z", // Important to ensure consistent timezone values
  }),
});
```
~~~

<Callout>
  For more information, read Kysely's documentation to the
  [MySQLDialect](https://kysely-org.github.io/kysely-apidoc/classes/MysqlDialect.html).
</Callout>

## Schema generation & migration

The [Better Auth CLI](/docs/concepts/cli) allows you to generate or migrate
your database schema based on your Better Auth configuration and plugins.

<table>
  <thead>
    <tr className="border-b">
      <th>
        <p className="font-bold text-[16px] mb-1">MySQL Schema Generation</p>
      </th>

      <th>
        <p className="font-bold text-[16px] mb-1">MySQL Schema Migration</p>
      </th>
    </tr>
  </thead>

  <tbody>
    <tr className="h-10">
      <td>âœ… Supported</td>
      <td>âœ… Supported</td>
    </tr>
  </tbody>
</table>

```bash title="Schema Generation"
npx @better-auth/cli@latest generate
```

```bash title="Schema Migration"
npx @better-auth/cli@latest migrate
```

## Additional Information

MySQL is supported under the hood via the [Kysely](https://kysely.dev/) adapter, any database supported by Kysely would also be supported. (<Link href="/docs/adapters/other-relational-databases">Read more here</Link>)

If you're looking for performance improvements or tips, take a look at our guide to <Link href="/docs/guides/optimizing-for-performance">performance optimizations</Link>.

# adapters: Other Relational Databases
URL: /docs/adapters/other-relational-databases
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/adapters/other-relational-databases.mdx

Integrate Better Auth with other relational databases.

***

title: Other Relational Databases
description: Integrate Better Auth with other relational databases.
-------------------------------------------------------------------

Better Auth supports a wide range of database dialects out of the box thanks to <Link href="https://kysely.dev/">Kysely</Link>.

Any dialect supported by Kysely can be utilized with Better Auth, including capabilities for generating and migrating database schemas through the <Link href="/docs/concepts/cli">CLI</Link>.

## Core Dialects

* [MySQL](/docs/adapters/mysql)
* [SQLite](/docs/adapters/sqlite)
* [PostgreSQL](/docs/adapters/postgresql)
* [MS SQL](/docs/adapters/mssql)

## Kysely Organization Dialects

* [Postgres.js](https://github.com/kysely-org/kysely-postgres-js)
* [SingleStore Data API](https://github.com/kysely-org/kysely-singlestore)
* [Supabase](https://github.com/kysely-org/kysely-supabase)

## Kysely Community dialects

* [PlanetScale Serverless Driver](https://github.com/depot/kysely-planetscale)
* [Cloudflare D1](https://github.com/aidenwallis/kysely-d1)
* [AWS RDS Data API](https://github.com/serverless-stack/kysely-data-api)
* [SurrealDB](https://github.com/igalklebanov/kysely-surrealdb)
* [Neon](https://github.com/seveibar/kysely-neon)
* [Xata](https://github.com/xataio/client-ts/tree/main/packages/plugin-client-kysely)
* [AWS S3 Select](https://github.com/igalklebanov/kysely-s3-select)
* [libSQL/sqld](https://github.com/libsql/kysely-libsql)
* [Fetch driver](https://github.com/andersgee/kysely-fetch-driver)
* [SQLite WASM](https://github.com/DallasHoff/sqlocal)
* [Deno SQLite](https://gitlab.com/soapbox-pub/kysely-deno-sqlite)
* [TiDB Cloud Serverless Driver](https://github.com/tidbcloud/kysely)
* [Capacitor SQLite Kysely](https://github.com/DawidWetzler/capacitor-sqlite-kysely)
* [BigQuery](https://github.com/maktouch/kysely-bigquery)
* [Clickhouse](https://github.com/founderpathcom/kysely-clickhouse)
* [PGLite](https://github.com/czeidler/kysely-pglite-dialect)

<Callout>
  You can see the full list of supported Kysely dialects{" "}
  <Link href="https://kysely.dev/docs/dialects">here</Link>.
</Callout>

# adapters: PostgreSQL
URL: /docs/adapters/postgresql
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/adapters/postgresql.mdx

Integrate Better Auth with PostgreSQL.

***

title: PostgreSQL
description: Integrate Better Auth with PostgreSQL.
---------------------------------------------------

PostgreSQL is a powerful, open-source relational database management system known for its advanced features, extensibility, and support for complex queries and large datasets.
Read more [here](https://www.postgresql.org/).

## Example Usage

Make sure you have PostgreSQL installed and configured.
Then, you can connect it straight into Better Auth.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { Pool } from "pg";

export const auth = betterAuth({
  database: new Pool({
    connectionString: "postgres://user:password@localhost:5432/database",
  }),
});
```

<Callout>
  For more information, read Kysely's documentation to the
  [PostgresDialect](https://kysely-org.github.io/kysely-apidoc/classes/PostgresDialect.html).
</Callout>

## Schema generation & migration

The [Better Auth CLI](/docs/concepts/cli) allows you to generate or migrate
your database schema based on your Better Auth configuration and plugins.

<table>
  <thead>
    <tr className="border-b">
      <th>
        <p className="font-bold text-[16px] mb-1">PostgreSQL Schema Generation</p>
      </th>

      <th>
        <p className="font-bold text-[16px] mb-1">PostgreSQL Schema Migration</p>
      </th>
    </tr>
  </thead>

  <tbody>
    <tr className="h-10">
      <td>âœ… Supported</td>
      <td>âœ… Supported</td>
    </tr>
  </tbody>
</table>

```bash title="Schema Generation"
npx @better-auth/cli@latest generate
```

```bash title="Schema Migration"
npx @better-auth/cli@latest migrate
```

## Use a non-default schema

In most cases, the default schema is `public`. To have Better Auth use a
non-default schema (e.g., `auth`) for its tables,
set the PostgreSQL user's default schema before generating or migrating:

```sql
ALTER USER authuser SET SEARCH_PATH TO auth;
```

alternatively, append the option to your connection URI, for example:

```
postgres://<DATABASE_URL>?option=-c search_path=auth
```

URL-encode if needed: `?option=-c%20search_path%3Dauth`.

Ensure the target schema exists and the database user has the required permissions.

## Additional Information

PostgreSQL is supported under the hood via the [Kysely](https://kysely.dev/) adapter, any database supported by Kysely would also be supported. (<Link href="/docs/adapters/other-relational-databases">Read more here</Link>)

If you're looking for performance improvements or tips, take a look at our guide to <Link href="/docs/guides/optimizing-for-performance">performance optimizations</Link>.

# adapters: Prisma
URL: /docs/adapters/prisma
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/adapters/prisma.mdx

Integrate Better Auth with Prisma.

***

title: Prisma
description: Integrate Better Auth with Prisma.
-----------------------------------------------

Prisma ORM is an open-source database toolkit that simplifies database access and management in applications by providing a type-safe query builder and an intuitive data modeling interface.
Read more [here](https://www.prisma.io/).

## Example Usage

Make sure you have Prisma installed and configured.
Then, you can use the Prisma adapter to connect to your database.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { prismaAdapter } from "better-auth/adapters/prisma";
import { PrismaClient } from "@prisma/client";

const prisma = new PrismaClient();

export const auth = betterAuth({
  database: prismaAdapter(prisma, {
    provider: "sqlite",
  }),
});
```

<Callout type="warning">
  If you have configured a custom output directory in your `schema.prisma` file (e.g., `output = "../src/generated/prisma"`), make sure to import the Prisma client from that location instead of `@prisma/client`. Learn more about custom output directories in the [Prisma documentation](https://www.prisma.io/docs/guides/nextjs#21-install-prisma-orm-and-create-your-first-models).
</Callout>

## Schema generation & migration

The [Better Auth CLI](/docs/concepts/cli) allows you to generate or migrate
your database schema based on your Better Auth configuration and plugins.

<table>
  <thead>
    <tr className="border-b">
      <th>
        <p className="font-bold text-[16px] mb-1">Prisma Schema Generation</p>
      </th>

      <th>

~~~text
        <p className="font-bold text-[16px] mb-1">Prisma Schema Migration</p>
      </th>
    </tr>
  </thead>

  <tbody>
    <tr className="h-10">
      <td>âœ… Supported</td>
      <td>â Œ Not Supported</td>
    </tr>
  </tbody>
</table>

```bash title="Schema Generation"
npx @better-auth/cli@latest generate
```

## Additional Information

If you're looking for performance improvements or tips, take a look at our guide to <Link href="/docs/guides/optimizing-for-performance">performance optimizations</Link>.



# adapters: SQLite
URL: /docs/adapters/sqlite
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/adapters/sqlite.mdx

Integrate Better Auth with SQLite.

***

title: SQLite
description: Integrate Better Auth with SQLite.
-----------------------------------------------

SQLite is a lightweight, serverless, self-contained SQL database engine that is widely used for local data storage in applications.
Read more [here.](https://www.sqlite.org/)

## Example Usage

Better Auth supports multiple SQLite drivers. Choose the one that best fits your environment:

### Better-SQLite3 (Recommended)

The most popular and stable SQLite driver for Node.js:

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import Database from "better-sqlite3";

export const auth = betterAuth({
  database: new Database("database.sqlite"),
});
```

<Callout>
  For more information, read Kysely's documentation to the
  [SqliteDialect](https://kysely-org.github.io/kysely-apidoc/classes/SqliteDialect.html).
</Callout>

### Node.js Built-in SQLite (Experimental)

<Callout type="warning">
  The `node:sqlite` module is still experimental and may change at any time. It requires Node.js 22.5.0 or later.
</Callout>

Starting from Node.js 22.5.0, you can use the built-in [SQLite](https://nodejs.org/api/sqlite.html) module:

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { DatabaseSync } from "node:sqlite";

export const auth = betterAuth({
  database: new DatabaseSync("database.sqlite"),
});
```

To run your application with Node.js SQLite:

```bash
node your-app.js
```

### Bun Built-in SQLite

You can also use the built-in [SQLite](https://bun.com/docs/api/sqlite) module in Bun, which is similar to the Node.js version:

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { Database } from "bun:sqlite";
export const auth = betterAuth({
  database: new Database("database.sqlite"),
});
```

## Schema generation & migration

The [Better Auth CLI](/docs/concepts/cli) allows you to generate or migrate
your database schema based on your Better Auth configuration and plugins.

<table>
  <thead>
    <tr className="border-b">
      <th>
        <p className="font-bold text-[16px] mb-1">SQLite Schema Generation</p>
      </th>

      <th>
        <p className="font-bold text-[16px] mb-1">SQLite Schema Migration</p>
      </th>
    </tr>
  </thead>

  <tbody>
    <tr className="h-10">
      <td>âœ… Supported</td>
      <td>âœ… Supported</td>
    </tr>
  </tbody>
</table>

```bash title="Schema Generation"
npx @better-auth/cli@latest generate
```

```bash title="Schema Migration"
npx @better-auth/cli@latest migrate
```

## Additional Information
~~~

~~~
SQLite is supported under the hood via the [Kysely](https://kysely.dev/) adapter, any database supported by Kysely would also be supported. (<Link href="/docs/adapters/other-relational-databases">Read more here</Link>)

If you're looking for performance improvements or tips, take a look at our guide to <Link href="/docs/guides/optimizing-for-performance">performance optimizations</Link>.



# authentication: Apple
URL: /docs/authentication/apple
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/apple.mdx

Apple provider setup and usage.

***

title: Apple
description: Apple provider setup and usage.
--------------------------------------------

<Steps>
  <Step>
    ### Get your OAuth credentials

    To use Apple sign in, you need a client ID and client secret. You can get them from the [Apple Developer Portal](https://developer.apple.com/account/resources/authkeys/list).

    You will need an active **Apple Developer account** to access the developer portal and generate these credentials.

    Follow these steps to set up your App ID, Service ID, and generate the key needed for your client secret:

    1. **Navigate to Certificates, Identifiers & Profiles:**
       In the Apple Developer Portal, go to the "Certificates, Identifiers & Profiles" section.

    2. **Create an App ID:**
       * Go to the `Identifiers` tab.
       * Click the `+` icon next to Identifiers.
       * Select `App IDs`, then click `Continue`.
       * Select `App` as the type, then click `Continue`.
       * **Description:** Enter a name for your app (e.g., "My Awesome App"). This name may be displayed to users when they sign in.
       * **Bundle ID:** Set a bundle ID. The recommended format is a reverse domain name (e.g., `com.yourcompany.yourapp`). Using a suffix like `.ai` (for app identifier) can help with organization but is not required (e.g., `com.yourcompany.yourapp.ai`).
       * Scroll down to **Capabilities**. Select the checkbox for `Sign In with Apple`.
       * Click `Continue`, then `Register`.

    3. **Create a Service ID:**
       * Go back to the `Identifiers` tab.
       * Click the `+` icon.
       * Select `Service IDs`, then click `Continue`.
       * **Description:** Enter a description for this service (e.g., your app name again).
       * **Identifier:** Set a unique identifier for the service. Use a reverse domain format, distinct from your App ID (e.g., `com.yourcompany.yourapp.si`, where `.si` indicates service identifier - this is for your organization and not required). **This Service ID will be your `clientId`.**
       * Click `Continue`, then `Register`.

    4. **Configure the Service ID:**
       * Find the Service ID you just created in the `Identifiers` list and click on it.
       * Check the `Sign In with Apple` capability, then click `Configure`.
       * Under **Primary App ID**, select the App ID you created earlier (e.g., `com.yourcompany.yourapp.ai`).
       * Under **Domains and Subdomains**, list all the root domains you will use for Sign In with Apple (e.g., `example.com`, `anotherdomain.com`).
       * Under **Return URLs**, enter the callback URL. `https://yourdomain.com/api/auth/callback/apple`. Add all necessary return URLs.
       * Click `Next`, then `Done`.
       * Click `Continue`, then `Save`.

    5. **Create a Client Secret Key:**
       * Go to the `Keys` tab.
       * Click the `+` icon to create a new key.
       * **Key Name:** Enter a name for the key (e.g., "Sign In with Apple Key").
       * Scroll down and select the checkbox for `Sign In with Apple`.
       * Click the `Configure` button next to `Sign In with Apple`.
       * Select the **Primary App ID** you created earlier.
       * Click `Save`, then `Continue`, then `Register`.
       * **Download the Key:** Immediately download the `.p8` key file. **This file is only available for download once.** Note the Key ID (available on the Keys page after creation) and your Team ID (available in your Apple Developer Account settings).

    6. **Generate the Client Secret (JWT):**
       Apple requires a JSON Web Token (JWT) to be generated dynamically using the downloaded `.p8` key, the Key ID, and your Team ID. This JWT serves as your `clientSecret`.

       You can use the guide below from [Apple's documentation](https://developer.apple.com/documentation/accountorganizationaldatasharing/creating-a-client-secret) to understand how to generate this client secret. You can also use our built in generator [below](#generate-apple-client-secret-jwt) to generate the client secret JWT required for 'Sign in with Apple'.

       **Note:** Apple allows a maximum expiration of 6 months (180 days) for the client secret JWT. You will need to regenerate the client secret before it expires to maintain uninterrupted authentication.
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to add it to the `socialProviders` option of the auth instance.

    You also need to add `https://appleid.apple.com` to the `trustedOrigins` array in your auth instance configuration to allow communication with Apple's authentication servers.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            apple: { // [!code highlight]
                clientId: process.env.APPLE_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.APPLE_CLIENT_SECRET as string, // [!code highlight]
                // Optional
                appBundleIdentifier: process.env.APPLE_APP_BUNDLE_IDENTIFIER as string, // [!code highlight]
            }, // [!code highlight]
        },
        // Add appleid.apple.com to trustedOrigins for Sign In with Apple flows
        trustedOrigins: ["https://appleid.apple.com"], // [!code highlight]
    })
    ```

    On native iOS, it doesn't use the service ID but the app ID (bundle ID) as client ID, so if using the service ID as `clientId` in `signIn.social` with `idToken`, it throws an error: `JWTClaimValidationFailed: unexpected "aud" claim value`. So you need to provide the `appBundleIdentifier` when you want to sign in with Apple using the ID Token.
  </Step>
</Steps>

## Usage

### Sign In with Apple

To sign in with Apple, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

* `provider`: The provider to use. It should be set to `apple`.

```ts title="auth-client.ts"  /
import { createAuthClient } from "better-auth/client"
const authClient =  createAuthClient()

const signIn = async () => {
    const data = await authClient.signIn.social({
        provider: "apple"
    })
}
```
~~~

~~~
### Sign In with Apple With ID Token

To sign in with Apple using the ID Token, you can use the `signIn.social` function to pass the ID Token.

This is useful when you have the ID Token from Apple on the client-side and want to use it to sign in on the server.

<Callout>
  If ID token is provided no redirection will happen, and the user will be signed in directly.
</Callout>

```ts title="auth-client.ts"
await authClient.signIn.social({
    provider: "apple",
    idToken: {
        token: // Apple ID Token,
        nonce: // Nonce (optional)
        accessToken: // Access Token (optional)
    }
})
```

## Generate Apple Client Secret (JWT)

<GenerateAppleJwt />



# authentication: Atlassian
URL: /docs/authentication/atlassian
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/atlassian.mdx

Atlassian provider setup and usage.

***

title: Atlassian
description: Atlassian provider setup and usage.
------------------------------------------------

<Steps>
  <Step>
    ### Get your Credentials

    1. Sign in to your Atlassian account and go to the [Atlassian Developer Console](https://developer.atlassian.com/console/myapps/)
    2. Click "Create new app"
    3. Fill out the app details
    4. Configure your redirect URI (e.g., `https://yourdomain.com/api/auth/callback/atlassian`)
    5. Note your Client ID and Client Secret

    <Callout type="info">
      * The default scope is `read:jira-user` and `offline_access`. For additional scopes, refer to the [Atlassian OAuth documentation]
(https://developer.atlassian.com/cloud/confluence/oauth-2-3lo-apps/).
    </Callout>

    Make sure to set the redirect URI to match your application's callback URL. If you change the base path of the auth routes, you should update the redirect URI accordingly.
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            atlassian: { // [!code highlight]
                clientId: process.env.ATLASSIAN_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.ATLASSIAN_CLIENT_SECRET as string, // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```
  </Step>

  <Step>
    ### Sign In with Atlassian

    To sign in with Atlassian, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `atlassian`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient =  createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "atlassian"
        })
    }
    ```

    <Callout type="info">
      For more information about Atlassian's OAuth scopes and API capabilities, refer to the [official Atlassian OAuth 2.0 (3LO) apps documentation]
(https://developer.atlassian.com/cloud/confluence/oauth-2-3lo-apps/).
    </Callout>
  </Step>
</Steps>



# authentication: Cognito
URL: /docs/authentication/cognito
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/cognito.mdx

Amazon Cognito provider setup and usage.

***

title: Cognito
description: Amazon Cognito provider setup and usage.
-----------------------------------------------------

<Steps>
  <Step>
    ### Get your Cognito Credentials

    To integrate with Cognito, you need to set up a **User Pool** and an **App client** in the [Amazon Cognito Console](https://console.aws.amazon.com/cognito/).

    Follow these steps:

    1. Go to the **Cognito Console** and create a **User Pool**.
    2. Under **App clients**, create a new **App client** (note the Client ID and Client Secret if enabled).
    3. Go to **Domain** and set a Cognito Hosted UI domain (e.g., `your-app.auth.us-east-1.amazoncognito.com`).
    4. In **App client settings**, enable:
       * Allowed OAuth flows: `Authorization code grant`
       * Allowed OAuth scopes: `openid`, `profile`, `email`
    5. Add your callback URL (e.g., `http://localhost:3000/api/auth/callback/cognito`).
~~~

~~~~
    <Callout type="info">
      * **User Pool is required** for Cognito authentication.
      * Make sure the callback URL matches exactly what you configure in Cognito.
    </Callout>
  </Step>

  <Step>
    ### Configure the provider

    Configure the `cognito` key in `socialProviders` key of your `auth` instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth";

    export const auth = betterAuth({
      socialProviders: {
        cognito: {
          clientId: process.env.COGNITO_CLIENT_ID as string, // [!code highlight]
          clientSecret: process.env.COGNITO_CLIENT_SECRET as string, // [!code highlight]
          domain: process.env.COGNITO_DOMAIN as string, // e.g. "your-app.auth.us-east-1.amazoncognito.com" [!code highlight]
          region: process.env.COGNITO_REGION as string, // e.g. "us-east-1" [!code highlight]
          userPoolId: process.env.COGNITO_USERPOOL_ID as string, // [!code highlight]
        },
      },
    })
    ```
  </Step>

  <Step>
    ### Sign In with Cognito

    To sign in with Cognito, use the `signIn.social` function from the client.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"

    const authClient = createAuthClient()

    const signIn = async () => {
      const data = await authClient.signIn.social({
        provider: "cognito"
      })
    }
    ```

    ### Additional Options:

    * `scope`: Additional OAuth2 scopes to request (combined with default permissions).
      * Default: `"openid" "profile" "email"`
      * Common Cognito scopes:
        * `openid`: Required for OpenID Connect authentication
        * `profile`: Access to basic profile info
        * `email`: Access to userâ€™s email
        * `phone`: Access to userâ€™s phone number
        * `aws.cognito.signin.user.admin`: Grants access to Cognito-specific APIs
    * Note: You must configure the scopes in your Cognito App Client settings. [available scopes](https://docs.aws.amazon.com/cognito/latest/developerguide/token-endpoint.html#token-endpoint-userinfo)
    * `getUserInfo`: Custom function to retrieve user information from the Cognito UserInfo endpoint.

    <Callout type="info">
      For more information about Amazon Cognito's scopes and API capabilities, refer to the [official documentation](https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pools-define-resource-servers.html?utm_source).
    </Callout>
  </Step>
</Steps>



# authentication: Discord
URL: /docs/authentication/discord
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/discord.mdx

Discord provider setup and usage.

***

title: Discord
description: Discord provider setup and usage.
----------------------------------------------

<Steps>
  <Step>
    ### Get your Discord credentials

    To use Discord sign in, you need a client ID and client secret. You can get them from the [Discord Developer Portal](https://discord.com/developers/applications).

    Make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/discord` for local development. For production, you should set it to the URL of your application. If you change the base path of the auth routes, you should update the redirect URL accordingly.
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            discord: { // [!code highlight]
                clientId: process.env.DISCORD_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.DISCORD_CLIENT_SECRET as string, // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```
  </Step>
</Steps>

## Usage

### Sign In with Discord

To sign in with Discord, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

* `provider`: The provider to use. It should be set to `discord`.

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client"
const authClient =  createAuthClient()

const signIn = async () => {
    const data = await authClient.signIn.social({
        provider: "discord"
    })
}
```

## Options
~~~~

For the full list of options supported by all social providers, check the [Provider Options](/docs/concepts/oauth#provider-options).

### Bot Permissions (Optional)

If you're using the `bot` scope with Discord OAuth, you can specify bot permissions using the `permissions` option. It can either be a bitwise value (e.g `2048 | 16384` for Send Messages and Embed Links) or a specific permission value (e.g `16384` for Embed Links).

```ts title="auth.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    socialProviders: {
        discord: {
            clientId: process.env.DISCORD_CLIENT_ID as string,
            clientSecret: process.env.DISCORD_CLIENT_SECRET as string,
            permissions: 2048 | 16384, // Send Messages + Embed Links // [!code highlight]
        },
    },
})
```

**Note:** The `permissions` parameter only works when the `bot` scope is included in your OAuth2 scopes. Read more about [Discord bot permissions](https://discord.com/developers/docs/topics/permissions).

# authentication: Dropbox
URL: /docs/authentication/dropbox
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/dropbox.mdx

Dropbox provider setup and usage.

***

title: Dropbox
description: Dropbox provider setup and usage.
----------------------------------------------

<Steps>
  <Step>
    ### Get your Dropbox credentials

    To use Dropbox sign in, you need a client ID and client secret. You can get them from the [Dropbox Developer Portal](https://www.dropbox.com/developers). You can Allow "Implicit Grant & PKCE" for the application in the App Console.

    Make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/dropbox` for local development. For production, you should set it to the URL of your application. If you change the base path of the auth routes, you should update the redirect URL accordingly.
  </Step>

  If you need deeper dive into Dropbox Authentication, you can check out the [official documentation](https://developers.dropbox.com/oauth-guide).

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            dropbox: { // [!code highlight]
                clientId: process.env.DROPBOX_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.DROPBOX_CLIENT_SECRET as string, // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```
  </Step>

  <Step>
    ### Sign In with Dropbox

    To sign in with Dropbox, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `dropbox`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient =  createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "dropbox"
        })
    }
    ```
  </Step>
</Steps>

# authentication: Email & Password
URL: /docs/authentication/email-password
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/email-password.mdx

Implementing email and password authentication with Better Auth.

***

title: Email & Password
description: Implementing email and password authentication with Better Auth.
-----------------------------------------------------------------------------

Email and password authentication is a common method used by many applications. Better Auth provides a built-in email and password authenticator that you can easily integrate into your project.

<Callout type="info">
  If you prefer username-based authentication, check out the{" "}
  <Link href="/docs/plugins/username">username plugin</Link>. It extends the
  email and password authenticator with username support.
</Callout>

## Enable Email and Password

To enable email and password authentication, you need to set the `emailAndPassword.enabled` option to `true` in the `auth` configuration.

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
  emailAndPassword: { // [!code highlight]
    enabled: true, // [!code highlight]
  }, // [!code highlight]
});
```

<Callout type="info">
  If it's not enabled, it'll not allow you to sign in or sign up with email and
  password.
</Callout>

## Usage

### Sign Up

To sign a user up, you can use the `signUp.email` function provided by the client.

### Client Side

```ts
const { data, error } = await authClient.signUp.email({
    name: John Doe,
    email: john.doe@example.com,
    password: password1234,
    image: https://example.com/image.png, // required
    callbackURL: https://example.com/callback, // required
});
```

### Server Side

```ts
const data = await auth.api.signUpEmail({
    body: {
        name: John Doe,
        email: john.doe@example.com,
        password: password1234,
        image: https://example.com/image.png, // required
        callbackURL: https://example.com/callback, // required
    }
});
```

### Type Definition

```ts
type signUpEmail = {
    /**
     * The name of the user.
     */
    name: string = "John Doe"
    /**
     * The email address of the user.
     */
    email: string = "john.doe@example.com"
    /**
     * The password of the user. It should be at least 8 characters long and max 128 by default.
     */
    password: string = "password1234"
    /**
     * An optional profile image of the user.
     */
    image?: string = "https://example.com/image.png"
    /**
     * An optional URL to redirect to after the user signs up.
     */
    callbackURL?: string = "https://example.com/callback"

}
```

<Callout>
  These are the default properties for the sign up email endpoint, however it's possible that with [additional fields](/docs/concepts/typescript#additional-fields) or special plugins you can pass more properties to the endpoint.
</Callout>

### Sign In

To sign a user in, you can use the `signIn.email` function provided by the client.

### Client Side

```ts
const { data, error } = await authClient.signIn.email({
    email: john.doe@example.com,
    password: password1234,
    rememberMe, // required
    callbackURL: https://example.com/callback, // required
});
```

### Server Side

```ts
const data = await auth.api.signInEmail({
    body: {
        email: john.doe@example.com,
        password: password1234,
        rememberMe, // required
        callbackURL: https://example.com/callback, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type signInEmail = {
    /**
     * The email address of the user.
     */
    email: string = "john.doe@example.com"
    /**
     * The password of the user. It should be at least 8 characters long and max 128 by default.
     */
    password: string = "password1234"
    /**
     * If false, the user will be signed out when the browser is closed. (optional) (default: true)
     */
    rememberMe?: boolean = true
    /**
     * An optional URL to redirect to after the user signs in. (optional)
     */
    callbackURL?: string = "https://example.com/callback"

}
```

<Callout>
  These are the default properties for the sign in email endpoint, however it's possible that with [additional fields](/docs/concepts/typescript#additional-fields) or special plugins you can pass different properties to the endpoint.
</Callout>

### Sign Out

To sign a user out, you can use the `signOut` function provided by the client.

### Client Side

```ts
const { data, error } = await authClient.signOut({});
```

### Server Side

```ts
await auth.api.signOut({

    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type signOut = {

}
```

you can pass `fetchOptions` to redirect onSuccess

```ts title="auth-client.ts"
await authClient.signOut({
  fetchOptions: {
    onSuccess: () => {
      router.push("/login"); // redirect to login page
    },
  },
});
```

### Email Verification

To enable email verification, you need to pass a function that sends a verification email with a link. The `sendVerificationEmail` function takes a data object with the following properties:

* `user`: The user object.
* `url`: The URL to send to the user which contains the token.
* `token`: A verification token used to complete the email verification.

and a `request` object as the second parameter.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { sendEmail } from "./email"; // your email sending function

export const auth = betterAuth({
  emailVerification: {
    sendVerificationEmail: async ( { user, url, token }, request) => {
      await sendEmail({
        to: user.email,
        subject: "Verify your email address",
        text: `Click the link to verify your email: ${url}`,
      });
    },
  },
});
```

On the client side you can use `sendVerificationEmail` function to send verification link to user. This will trigger the `sendVerificationEmail` function you provided in the `auth` configuration.

Once the user clicks on the link in the email, if the token is valid, the user will be redirected to the URL provided in the `callbackURL` parameter. If the token is invalid, the user will be redirected to the URL provided in the `callbackURL` parameter with an error message in the query string `?error=invalid_token`.

#### Require Email Verification

If you enable require email verification, users must verify their email before they can log in. And every time a user tries to sign in, sendVerificationEmail is called.

<Callout>
  This only works if you have sendVerificationEmail implemented and if the user
  is trying to sign in with email and password.
</Callout>

```ts title="auth.ts"
export const auth = betterAuth({
  emailAndPassword: {
    requireEmailVerification: true,
  },
});
```

If a user tries to sign in without verifying their email, you can handle the error and show a message to the user.

```ts title="auth-client.ts"
await authClient.signIn.email(
  {
    email: "email@example.com",
    password: "password",
  },
  {
    onError: (ctx) => {
      // Handle the error
      if (ctx.error.status === 403) {
        alert("Please verify your email address");
      }
      //you can also show the original error message
      alert(ctx.error.message);
    },
  }
);
```

#### Triggering manually Email Verification

You can trigger the email verification manually by calling the `sendVerificationEmail` function.

```ts
await authClient.sendVerificationEmail({
  email: "user@email.com",
  callbackURL: "/", // The redirect URL after verification
});
```

### Request Password Reset

To allow users to reset a password first you need to provide `sendResetPassword` function to the email and password authenticator. The `sendResetPassword` function takes a data object with the following properties:

* `user`: The user object.
* `url`: The URL to send to the user which contains the token.
* `token`: A verification token used to complete the password reset.

and a `request` object as the second parameter.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { sendEmail } from "./email"; // your email sending function
```

```ts
export const auth = betterAuth({
  emailAndPassword: {
    enabled: true,
    sendResetPassword: async ({user, url, token}, request) => {
      await sendEmail({
        to: user.email,
        subject: "Reset your password",
        text: `Click the link to reset your password: ${url}`,
      });
    },
    onPasswordReset: async ({ user }, request) => {
      // your logic here
      console.log(`Password for user ${user.email} has been reset.`);
    },
  },
});
```

Additionally, you can provide an `onPasswordReset` callback to execute logic after a password has been successfully reset.

Once you configured your server you can call `requestPasswordReset` function to send reset password link to user. If the user exists, it will trigger the `sendResetPassword` function you provided in the auth config.

### Client Side

```ts
const { data, error } = await authClient.requestPasswordReset({
    email: john.doe@example.com,
    redirectTo: https://example.com/reset-password, // required
});
```

### Server Side

```ts
const data = await auth.api.requestPasswordReset({
    body: {
        email: john.doe@example.com,
        redirectTo: https://example.com/reset-password, // required
    }
});
```

### Type Definition

```ts
type requestPasswordReset = {
    /**
     * The email address of the user to send a password reset email to
     */
    email: string = "john.doe@example.com"
    /**
     * The URL to redirect the user to reset their password. If the token isn't valid or expired, it'll be redirected with a query parameter `?error=INVALID_TOKEN`. If the token is valid, it'll be redirected with a query parameter `?token=VALID_TOKEN
     */
    redirectTo?: string = "https://example.com/reset-password"

}
```

When a user clicks on the link in the email, they will be redirected to the reset password page. You can add the reset password page to your app. Then you can use `resetPassword` function to reset the password. It takes an object with the following properties:

* `newPassword`: The new password of the user.

```ts title="auth-client.ts"
const { data, error } = await authClient.resetPassword({
  newPassword: "password1234",
  token,
});
```

### Client Side

```ts
const { data, error } = await authClient.resetPassword({
    newPassword: password1234,
    token,
});
```

### Server Side

```ts
const data = await auth.api.resetPassword({
    body: {
        newPassword: password1234,
        token,
    }
});
```

### Type Definition

```ts
type resetPassword = {
    /**
     * The new password to set
     */
    newPassword: string = "password1234"
    /**
     * The token to reset the password
     */
    token: string

}
```

### Update password

A user's password isn't stored in the user table. Instead, it's stored in the account table. To change the password of a user, you can use one of the following approaches:

### Client Side

```ts
const { data, error } = await authClient.changePassword({
    newPassword: newpassword1234,
    currentPassword: oldpassword1234,
    revokeOtherSessions, // required
});
```

### Server Side

```ts
const data = await auth.api.changePassword({
    body: {
        newPassword: newpassword1234,
        currentPassword: oldpassword1234,
        revokeOtherSessions, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type changePassword = {
    /**
     * The new password to set
     */
    newPassword: string = "newpassword1234"
    /**
     * The current user password
     */
    currentPassword: string = "oldpassword1234"
    /**
     * When set to true, all other active sessions for this user will be invalidated
     */
    revokeOtherSessions?: boolean = true

}
```

### Configuration

**Password**

Better Auth stores passwords inside the `account` table with `providerId` set to `credential`.

**Password Hashing**: Better Auth uses `scrypt` to hash passwords. The `scrypt` algorithm is designed to be slow and memory-intensive to make it difficult for attackers to brute force passwords. OWASP recommends using `scrypt` if `argon2id` is not available. We decided to use `scrypt` because it's natively supported by Node.js.

You can pass custom password hashing algorithm by setting `passwordHasher` option in the `auth` configuration.

```ts title="auth.ts"
import { betterAuth } from "better-auth"
import { scrypt } from "scrypt"

export const auth = betterAuth({
    //...rest of the options
    emailAndPassword: {
        password: {
            hash: // your custom password hashing function
            verify: // your custom password verification function
        }
    }
})
```

```
<TypeTable
  type={{
  enabled: {
    description: "Enable email and password authentication.",
    type: "boolean",
    default: "false",
  },
  disableSignUp: {
    description: "Disable email and password sign up.",
    type: "boolean",
    default: "false"
  },
  minPasswordLength: {
    description: "The minimum length of a password.",
    type: "number",
    default: 8,
  },
  maxPasswordLength: {
    description: "The maximum length of a password.",
    type: "number",
    default: 128,
  },
  sendResetPassword: {
    description:
      "Sends a password reset email. It takes a function that takes two parameters: token and user.",
    type: "function",
  },
  onPasswordReset: {
    description:
      "A callback function that is triggered when a user's password is changed successfully.",
    type: "function",
  },
  resetPasswordTokenExpiresIn: {
    description:
      "Number of seconds the reset password token is valid for.",
    type: "number",
    default: 3600
  },
  password: {
    description: "Password configuration.",
    type: "object",
    properties: {
      hash: {
        description: "custom password hashing function",
        type: "function",
      },
      verify: {
        description: "custom password verification function",
        type: "function",
      },
    },
  },
}}
/>
```

# authentication: Facebook
URL: /docs/authentication/facebook
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/facebook.mdx

Facebook provider setup and usage.

***

title: Facebook
description: Facebook provider setup and usage.
-----------------------------------------------

```
<Steps>
  <Step>
    ### Get your Facebook credentials
```

~~~
    To use Facebook sign in, you need a client ID and client Secret. You can get them from the [Facebook Developer Portal](https://developers.facebook.com/).
    Select your app, navigate to **App Settings > Basic**, locate the following:

    * **App ID**: This is your `clientId`
    * **App Secret**: This is your `clientSecret`.

    <Callout type="warn">
      Avoid exposing the `clientSecret` in client-side code (e.g., frontend apps) because itâ€™s sensitive information.
    </Callout>

    Make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/facebook` for local development. For production, you should set it to the URL of your
application. If you change the base path of the auth routes, you should update the redirect URL accordingly.
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            facebook: { // [!code highlight]
                clientId: process.env.FACEBOOK_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.FACEBOOK_CLIENT_SECRET as string, // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```

    <Callout>
      BetterAuth also supports Facebook Login for Business, all you need
      to do is provide the `configId` as listed in **Facebook Login For Business > Configurations** alongside your `clientId` and `clientSecret`. Note that the app must be a
Business app and, since BetterAuth expects to have an email address and account id, the configuration must be of the "User access token" type. "System-user access token" is not
supported.
    </Callout>
  </Step>

  <Step>
    ### Sign In with Facebook

    To sign in with Facebook, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `facebook`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/auth-client"
    const authClient = createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "facebook"
        })
    }
    ```
  </Step>
</Steps>

## Additional Configuration

### Scopes

By default, Facebook provides basic user information. If you need additional permissions, you can specify scopes in your auth configuration:

```ts title="auth.ts"
export const auth = betterAuth({
    socialProviders: {
        facebook: {
            clientId: process.env.FACEBOOK_CLIENT_ID as string,
            clientSecret: process.env.FACEBOOK_CLIENT_SECRET as string,
            scopes: ["email", "public_profile", "user_friends"], // Overwrites permissions
            fields: ["user_friends"], // Extending list of fields
        },
    },
})
```

Additional options:

* `scopes`: Access basic account information (overwrites).
  * Default: `"email", "public_profile"`
* `fields`: Extend list of fields to retrieve from the Facebook user profile (assignment).
  * Default: `"id", "name", "email", "picture"`

### Sign In with Facebook With ID or Access Token

To sign in with Facebook using the ID Token, you can use the `signIn.social` function to pass the ID Token.

This is useful when you have the ID Token from Facebook on the client-side and want to use it to sign in on the server.

<Callout>
  If ID token is provided no redirection will happen, and the user will be signed in directly.
</Callout>

For limited login, you need to pass `idToken.token`, for only `accessToken` you need to pass `idToken.accessToken` and `idToken.token` together because of (#1183)\
[[https://github.com/better-auth/better-auth/issues/1183](https://github.com/better-auth/better-auth/issues/1183)].

```ts title="auth-client.ts"
const data = await authClient.signIn.social({
    provider: "facebook",
    idToken: {  // [!code highlight]
        ...(platform === 'ios' ?  // [!code highlight]
            { token: idToken }  // [!code highlight]
            : { token: accessToken, accessToken: accessToken }), // [!code highlight]
    },
})
```

For a complete list of available permissions, refer to the [Permissions Reference](https://developers.facebook.com/docs/permissions).



# authentication: Figma
URL: /docs/authentication/figma
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/figma.mdx

Figma provider setup and usage.

***

title: Figma
description: Figma provider setup and usage.
--------------------------------------------

<Steps>
  <Step>
    ### Get your Credentials
~~~

~~~
    1. Sign in to your Figma account and go to the [Developer Apps page](https://www.figma.com/developers/apps)
    2. Click "Create new app"
    3. Fill out the app details (name, description, etc.)
    4. Configure your redirect URI (e.g., `https://yourdomain.com/api/auth/callback/figma`)
    5. Note your Client ID and Client Secret

    <Callout type="info">
      * The default scope is `file_read`. For additional scopes like `file_write`, refer to the [Figma OAuth documentation](https://www.figma.com/developers/api#oauth2).
    </Callout>

    Make sure to set the redirect URI to match your application's callback URL. If you change the base path of the auth routes, you should update the redirect URI accordingly.
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            figma: { // [!code highlight]
                clientId: process.env.FIGMA_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.FIGMA_CLIENT_SECRET as string, // [!code highlight]
                clientKey: process.env.FIGMA_CLIENT_KEY as string, // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```
  </Step>

  <Step>
    ### Sign In with Figma

    To sign in with Figma, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `figma`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient =  createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "figma"
        })
    }
    ```

    <Callout type="info">
      For more information about Figma's OAuth scopes and API capabilities, refer to the [official Figma API documentation](https://www.figma.com/developers/api).
    </Callout>
  </Step>
</Steps>



# authentication: GitHub
URL: /docs/authentication/github
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/github.mdx

GitHub provider setup and usage.

***

title: GitHub
description: GitHub provider setup and usage.
---------------------------------------------

<Steps>
  <Step>
    ### Get your GitHub credentials

    To use GitHub sign in, you need a client ID and client secret. You can get them from the [GitHub Developer Portal](https://github.com/settings/developers).

    Make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/github` for local development. For production, you should set it to the URL of your application. If you change the base path of the auth routes, you should update the redirect URL accordingly.

    Important: You MUST include the user:email scope in your GitHub app. See details below.
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            github: { // [!code highlight]
                clientId: process.env.GITHUB_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.GITHUB_CLIENT_SECRET as string, // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```
  </Step>

  <Step>
    ### Sign In with GitHub

    To sign in with GitHub, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `github`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient =  createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "github"
        })
    }
    ```
  </Step>
</Steps>

## Usage

### Setting up your Github app

Github has two types of apps: Github apps and OAuth apps.

For OAuth apps, you don't have to do anything special (just follow the steps above). For Github apps, you DO have to add one more thing, which is enable it to read the user's email:
~~~

1. After creating your app, go to *Permissions and Events* > *Account Permissions* > *Email Addresses* and select "Read-Only"

2. Save changes.

That's all! Now you can copy the Client ID and Client Secret of your app!

<Callout>
  If you get "email_not_found" error, it's because you selected a Github app & did not configure this part!
</Callout>

### Why don't I have a refresh token?

Github doesn't issue refresh tokens for OAuth apps. For regular OAuth apps,
GitHub issues access tokens that remain valid indefinitely unless the user revokes them,
the app revokes them, or they go unused for a year.
There's no need for a refresh token because the access token doesn't expire on a short interval like Google or Discord.

# authentication: GitLab
URL: /docs/authentication/gitlab
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/gitlab.mdx

GitLab provider setup and usage.

***

title: GitLab
description: GitLab provider setup and usage.
---------------------------------------------

<Steps>
  <Step>
    ### Get your GitLab credentials

    To use GitLab sign in, you need a client ID and client secret. [GitLab OAuth documentation](https://docs.gitlab.com/ee/api/oauth2.html).

    Make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/gitlab` for local development. For production, you should set it to the URL of your application. If you change the base path of the auth routes, you should update the redirect URL accordingly.
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            gitlab: { // [!code highlight]
                clientId: process.env.GITLAB_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.GITLAB_CLIENT_SECRET as string, // [!code highlight]
                issuer: process.env.GITLAB_ISSUER as string, // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```

    #### Configuration Options

    * `clientId`: Your GitLab application's Client ID
    * `clientSecret`: Your GitLab application's Client Secret
    * `issuer`: (Optional) The URL of your GitLab instance. Use this for self-hosted GitLab servers.
      * Default: `"https://gitlab.com"` (GitLab.com)
      * Example: `"https://gitlab.company.com"`

    <Callout type="info">
      The `issuer` option is useful when using a self-hosted GitLab instance. If you're using GitLab.com, you can omit this option as it defaults to `https://gitlab.com`.
    </Callout>

    #### Example with self-hosted GitLab

    ```ts title="auth.ts"
    export const auth = betterAuth({
        socialProviders: {
            gitlab: {
                clientId: process.env.GITLAB_CLIENT_ID as string,
                clientSecret: process.env.GITLAB_CLIENT_SECRET as string,
                issuer: "https://gitlab.company.com", // Your self-hosted GitLab URL // [!code highlight]
            },
        },
    })
    ```
  </Step>

  <Step>
    ### Sign In with GitLab

    To sign in with GitLab, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `gitlab`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient =  createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "gitlab"
        })
    }
    ```
  </Step>
</Steps>

# authentication: Google
URL: /docs/authentication/google
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/google.mdx

Google provider setup and usage.

***

title: Google
description: Google provider setup and usage.
---------------------------------------------

<Steps>
  <Step>
    ### Get your Google credentials

    To use Google as a social provider, you need to get your Google credentials. You can get them by creating a new project in the [Google Cloud Console](https://console.cloud.google.com/apis/dashboard).

~~~
    In the Google Cloud Console > Credentials > Authorized redirect URIs, make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/google` for local development. For production, make sure to set the redirect URL as your application domain, e.g. `https://example.com/api/auth/callback/google`. If you change the base path of the auth routes, you should update the redirect URL accordingly.
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to pass the `clientId` and `clientSecret` to `socialProviders.google` in your auth configuration.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            google: { // [!code highlight]
                clientId: process.env.GOOGLE_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.GOOGLE_CLIENT_SECRET as string, // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```
  </Step>
</Steps>

## Usage

### Sign In with Google

To sign in with Google, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

* `provider`: The provider to use. It should be set to `google`.

```ts title="auth-client.ts"  /
import { createAuthClient } from "better-auth/client";
const authClient = createAuthClient();

const signIn = async () => {
  const data = await authClient.signIn.social({
    provider: "google",
  });
};
```

### Sign In with Google With ID Token

To sign in with Google using the ID Token, you can use the `signIn.social` function to pass the ID Token.

This is useful when you have the ID Token from Google on the client-side and want to use it to sign in on the server.

<Callout>
  If ID token is provided no redirection will happen, and the user will be
  signed in directly.
</Callout>

```ts title="auth-client.ts"
const data = await authClient.signIn.social({
    provider: "google",
    idToken: {
        token: // Google ID Token,
        accessToken: // Google Access Token
    }
})
```

<Callout>
  If you want to use google one tap, you can use the [One Tap
  Plugin](/docs/plugins/one-tap) guide.
</Callout>

### Always ask to select an account

If you want to always ask the user to select an account, you pass the `prompt` parameter to the provider, setting it to `select_account`.

```ts
socialProviders: {
    google: {
        prompt: "select_account", // [!code highlight]
        clientId: process.env.GOOGLE_CLIENT_ID as string,
        clientSecret: process.env.GOOGLE_CLIENT_SECRET as string,
    },
}
```

### Requesting Additional Google Scopes

If your application needs additional Google scopes after the user has already signed up (e.g., for Google Drive, Gmail, or other Google services), you can request them using the `linkSocial` method with the same Google provider.

```tsx title="auth-client.ts"
const requestGoogleDriveAccess = async () => {
  await authClient.linkSocial({
    provider: "google",
    scopes: ["https://www.googleapis.com/auth/drive.file"],
  });
};

// Example usage in a React component
return (
  <button onClick={requestGoogleDriveAccess}>
    Add Google Drive Permissions
  </button>
);
```

This will trigger a new OAuth flow that requests the additional scopes. After completion, your account will have the new scope in the database, and the access token will give you access to the requested Google APIs.

<Callout>
  Ensure you're using Better Auth version 1.2.7 or later to avoid "Social
  account already linked" errors when requesting additional scopes from the same
  provider.
</Callout>

### Always get refresh token

Google only issues a refresh token the first time a user consents to your app.
If the user has already authorized your app, subsequent OAuth flows will only return an access token, not a refresh token.

To always get a refresh token, you can set the `accessType` to `offline`, and `prompt` to `select_account consent` in the provider options.

```ts
socialProviders: {
    google: {
        clientId: process.env.GOOGLE_CLIENT_ID as string,
        clientSecret: process.env.GOOGLE_CLIENT_SECRET as string,
        accessType: "offline", // [!code highlight]
        prompt: "select_account consent", // [!code highlight]
    },
}
```
~~~

~~~
<Callout>
  **Revoking Access:** If you want to get a new refresh token for a user who has
  already authorized your app, you must have them revoke your app's access in
  their Google account settings, then re-authorize.
</Callout>



# authentication: Hugging Face
URL: /docs/authentication/huggingface
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/huggingface.mdx

Hugging Face provider setup and usage.

***

title: Hugging Face
description: Hugging Face provider setup and usage.
---------------------------------------------------

<Steps>
  <Step>
    ### Get your Hugging Face credentials

    To use Hugging Face sign in, you need a client ID and client secret. [Hugging Face OAuth documentation](https://huggingface.co/docs/hub/oauth). Make sure the created oauth app on Hugging Face has the "email" scope.

    Make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/huggingface` for local development. For production, you should set it to the URL of your application. If you change the base path of the auth routes, you should update the redirect URL accordingly.
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            huggingface: { // [!code highlight]
                clientId: process.env.HUGGINGFACE_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.HUGGINGFACE_CLIENT_SECRET as string, // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```
  </Step>

  <Step>
    ### Sign In with Hugging Face

    To sign in with Hugging Face, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `huggingface`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient =  createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "huggingface"
        })
    }
    ```
  </Step>
</Steps>



# authentication: Kakao
URL: /docs/authentication/kakao
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/kakao.mdx

Kakao provider setup and usage.

***

title: Kakao
description: Kakao provider setup and usage.
--------------------------------------------

<Steps>
  <Step>
    ### Get your Kakao Credentials

    To use Kakao sign in, you need a client ID and client secret. You can get them from the [Kakao Developer Portal](https://developers.kakao.com).

    Make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/kakao` for local development. For production, you should set it to the URL of your application. If you change the base path of the auth routes, you should update the redirect URL accordingly.
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            kakao: { // [!code highlight]
                clientId: process.env.KAKAO_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.KAKAO_CLIENT_SECRET as string, // [!code highlight]
            }, // [!code highlight]
        }
    })
    ```
  </Step>

  <Step>
    ### Sign In with Kakao

    To sign in with Kakao, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `kakao`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient =  createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "kakao"
        })
    }
    ```
~~~

  </Step>
</Steps>

# authentication: Kick
URL: /docs/authentication/kick
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/kick.mdx

Kick provider setup and usage.

***

title: Kick
description: Kick provider setup and usage.
-------------------------------------------

<Steps>
  <Step>
    ### Get your Kick Credentials

    To use Kick sign in, you need a client ID and client secret. You can get them from the [Kick Developer Portal](https://kick.com/settings/developer).

    Make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/kick` for local development. For production, you should set it to the URL of your application. If you change the base path of the auth routes, you should update the redirect URL accordingly.
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            kick: { // [!code highlight]
                clientId: process.env.KICK_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.KICK_CLIENT_SECRET as string, // [!code highlight]
            }, // [!code highlight]
        }
    })
    ```
  </Step>

  <Step>
    ### Sign In with Kick

    To sign in with Kick, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `kick`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient =  createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "kick"
        })
    }
    ```
  </Step>
</Steps>

# authentication: LINE
URL: /docs/authentication/line
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/line.mdx

LINE provider setup and usage.

***

title: LINE
description: LINE provider setup and usage.
-------------------------------------------

<Steps>
  <Step>
    ### Get your LINE credentials

    1. Create a channel in the LINE Developers Console.
    2. Note your Channel ID (client_id) and Channel secret (client_secret).
    3. In the channel settings, add your Redirect URI, e.g. `http://localhost:3000/api/auth/callback/line` for local development.
    4. Enable required scopes (at least `openid`; add `profile`, `email` if you need name, avatar, email).

    See LINE Login v2.1 reference for details: \[`https://developers.line.biz/en/reference/line-login/#issue-access-token`]
  </Step>

  <Step>
    ### Configure the provider

    Add your LINE credentials to `socialProviders.line` in your auth configuration.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth";

    export const auth = betterAuth({
      socialProviders: {
        line: {
          clientId: process.env.LINE_CLIENT_ID as string,
          clientSecret: process.env.LINE_CLIENT_SECRET as string,
          // Optional: override redirect if needed
          // redirectURI: "https://your.app/api/auth/callback/line",
          // scopes are prefilled: ["openid","profile","email"]. Append if needed
        },
      },
    });
    ```
  </Step>
</Steps>

## Usage

### Sign In with LINE

Use the client `signIn.social` with `provider: "line"`.

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client";
const authClient = createAuthClient();

async function signInWithLINE() {
  const res = await authClient.signIn.social({ provider: "line" });
}
```

### Sign In with LINE using ID Token (optional)

If you obtain the LINE ID token on the client, you can sign in directly without redirection.

```ts title="auth-client.ts"
await authClient.signIn.social({
  provider: "line",
  idToken: {
    token: "<LINE_ID_TOKEN>",
    accessToken: "<LINE_ACCESS_TOKEN>",
  },
});
```

### Notes

* Default scopes include `openid profile email`. Adjust as needed via provider options.
* Verify redirect URI exactly matches the value configured in LINE Developers Console.
* LINE ID token verification uses the official endpoint and checks audience and optional nonce per spec.

Designing a login button? Follow LINE's button [guidelines](https://developers.line.biz/en/docs/line-login/login-button/).

# authentication: Linear
URL: /docs/authentication/linear
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/linear.mdx

Linear provider setup and usage.

***

title: Linear
description: Linear provider setup and usage.
---------------------------------------------

<Steps>
  <Step>
    ### Get your Linear credentials

    To use Linear sign in, you need a client ID and client secret. You can get them from the [Linear Developer Portal](https://linear.app/settings/api).

    Make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/linear` for local development. For production, you should set it to the URL of your application. If you change the base path of the auth routes, you should update the redirect URL accordingly.

    When creating your OAuth application in Linear, you'll need to specify the required scopes. The default scope is `read`, but you can also request additional scopes like `write` if needed.
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            linear: { // [!code highlight]
                clientId: process.env.LINEAR_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.LINEAR_CLIENT_SECRET as string, // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```
  </Step>

  <Step>
    ### Sign In with Linear

    To sign in with Linear, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `linear`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient = createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "linear"
        })
    }
    ```
  </Step>

  <Step>
    ### Available scopes

    Linear OAuth supports the following scopes:

    * `read` (default): Read access for the user's account
    * `write`: Write access for the user's account
    * `issues:create`: Allows creating new issues and their attachments
    * `comments:create`: Allows creating new issue comments
    * `timeSchedule:write`: Allows creating and modifying time schedules
    * `admin`: Full access to admin level endpoints (use with caution)

    You can specify additional scopes when configuring the provider:

    ```ts title="auth.ts"
    export const auth = betterAuth({
        socialProviders: {
            linear: {
                clientId: process.env.LINEAR_CLIENT_ID as string,
                clientSecret: process.env.LINEAR_CLIENT_SECRET as string,
                scope: ["read", "write"] // [!code highlight]
            },
        },
    })
    ```
  </Step>
</Steps>

# authentication: LinkedIn
URL: /docs/authentication/linkedin
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/linkedin.mdx

LinkedIn Provider

***

title: LinkedIn
description: LinkedIn Provider
------------------------------

<Steps>
  <Step>
    ### Get your LinkedIn credentials

~~~
    To use LinkedIn sign in, you need a client ID and client secret. You can get them from the [LinkedIn Developer Portal](https://www.linkedin.com/developers/).

    Make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/linkedin` for local development. For production, you should set it to the URL of your application. If you change the base path of the auth routes, you should update the redirect URL accordingly.
  </Step>

  <Callout type="info">
    In the LinkedIn portal under products you need the **Sign In with LinkedIn using OpenID Connect** product.
  </Callout>

  There are some different Guides here:
  [Authorization Code Flow (3-legged OAuth) (Outdated)](https://learn.microsoft.com/en-us/linkedin/shared/authentication/authorization-code-flow)
  [Sign In with LinkedIn using OpenID Connect](https://learn.microsoft.com/en-us/linkedin/consumer/integrations/self-serve/sign-in-with-linkedin-v2?context=linkedin%2Fconsumer%2Fcontext)

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            linkedin: { // [!code highlight]
                clientId: process.env.LINKEDIN_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.LINKEDIN_CLIENT_SECRET as string, // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```
  </Step>

  <Step>
    ### Sign In with LinkedIn

    To sign in with LinkedIn, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `linkedin`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient =  createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "linkedin"
        })
    }
    ```
  </Step>
</Steps>



# authentication: Microsoft
URL: /docs/authentication/microsoft
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/microsoft.mdx

Microsoft provider setup and usage.

***

title: Microsoft
description: Microsoft provider setup and usage.
------------------------------------------------

Enabling OAuth with Microsoft Azure Entra ID (formerly Active Directory) allows your users to sign in and sign up to your application with their Microsoft account.

<Steps>
  <Step>
    ### Get your Microsoft credentials

    To use Microsoft as a social provider, you need to get your Microsoft credentials. Which involves generating your own Client ID and Client Secret using your Microsoft Entra ID dashboard account.

    Make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/microsoft` for local development. For production, you should change it to the URL of your application. If you change the base path of the auth routes, you should update the redirect URL accordingly.

    see the [Microsoft Entra ID documentation](https://docs.microsoft.com/en-us/azure/active-directory/develop/quickstart-register-app) for more information.
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to pass the `clientId` and `clientSecret` to `socialProviders.microsoft` in your auth configuration.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            microsoft: { // [!code highlight]
                clientId: process.env.MICROSOFT_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.MICROSOFT_CLIENT_SECRET as string, // [!code highlight]
                // Optional
                tenantId: 'common', // [!code highlight]
                authority: "https://login.microsoftonline.com", // Authentication authority URL // [!code highlight]
                prompt: "select_account", // Forces account selection // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```

    **Authority URL**: Use the default `https://login.microsoftonline.com` for standard Entra ID scenarios or `https://<tenant-id>.ciamlogin.com` for CIAM (Customer Identity and Access Management) scenarios.
  </Step>
</Steps>

## Sign In with Microsoft

To sign in with Microsoft, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

* `provider`: The provider to use. It should be set to `microsoft`.

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client";

const authClient = createAuthClient();

const signIn = async () => {
  const data = await authClient.signIn.social({
    provider: "microsoft",
    callbackURL: "/dashboard", // The URL to redirect to after the sign in
  });
};
```
~~~

# authentication: Naver
URL: /docs/authentication/naver
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/naver.mdx

Naver provider setup and usage.

***

title: Naver
description: Naver provider setup and usage.
-----------------------------------------------

<Steps>
  <Step>
    ### Get your Naver Credentials

    To use Naver sign in, you need a client ID and client secret. You can get them from the [Naver Developers](https://developers.naver.com/).

    Make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/naver` for local development. For production, you should set it to the URL of your application. If you change the base path of the auth routes, you should update the redirect URL accordingly.
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            naver: { // [!code highlight]
                clientId: process.env.NAVER_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.NAVER_CLIENT_SECRET as string, // [!code highlight]
            }, // [!code highlight]
        }
    })
```
  </Step>

  <Step>
    ### Sign In with Naver

    To sign in with Naver, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `naver`.

```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient =  createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "naver"
        })
    }
```
  </Step>
</Steps>

---

# authentication: Notion
URL: /docs/authentication/notion
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/notion.mdx

Notion provider setup and usage.

***

title: Notion
description: Notion provider setup and usage.
-------------------------------------------------

<Steps>
  <Step>
    ### Get your Notion credentials

    To use Notion as a social provider, you need to get your Notion OAuth credentials. You can get them by creating a new integration in the [Notion Developers Portal](https://www.notion.so/my-integrations).

    In the Notion integration settings > OAuth Domain & URIs, make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/notion` for local development. For production, make sure to set the redirect URL as your application domain, e.g. `https://example.com/api/auth/callback/notion`. If you change the base path of the auth routes, you should update the redirect URL accordingly.

    <Callout>
      Make sure your Notion integration has the appropriate capabilities enabled. For user authentication, you'll need the "Read user information including email addresses" capability.
    </Callout>
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to pass the `clientId` and `clientSecret` to `socialProviders.notion` in your auth configuration.

```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            notion: { // [!code highlight]
                clientId: process.env.NOTION_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.NOTION_CLIENT_SECRET as string, // [!code highlight]
            }, // [!code highlight]
        },
    })
```
  </Step>
</Steps>

## Usage

### Sign In with Notion

To sign in with Notion, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

* `provider`: The provider to use. It should be set to `notion`.

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client"
const authClient =  createAuthClient()

const signIn = async () => {
    const data = await authClient.signIn.social({
        provider: "notion"
    })
}
```

### Notion Integration Types

Notion supports different integration types. When creating your integration, you can choose between:

* **Public integrations**: Can be installed by any Notion workspace
* **Internal integrations**: Limited to your own workspace

For most authentication use cases, you'll want to create a public integration to allow users from different workspaces to sign in.

### Requesting Additional Notion Scopes

If your application needs additional Notion capabilities after the user has already signed up, you can request them using the `linkSocial` method with the same Notion provider and additional scopes.

```ts title="auth-client.ts"
const requestNotionAccess = async () => {
    await authClient.linkSocial({
        provider: "notion",
        // Notion automatically provides access based on integration capabilities
    });
};

// Example usage in a React component
return <button onClick={requestNotionAccess}>Connect Notion Workspace</button>;
```

<Callout>
  After authentication, you can use the access token to interact with the Notion API to read and write pages, databases, and other content that the user has granted access to.
</Callout>

# authentication: Other Social Providers
URL: /docs/authentication/other-social-providers
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/other-social-providers.mdx

Other social providers setup and usage.

***

title: Other Social Providers
description: Other social providers setup and usage.
----------------------------------------------------

Better Auth provides out of the box support for a [Generic OAuth Plugin](/docs/plugins/generic-oauth) which allows you to use any social provider that implements the OAuth2 protocol or OpenID Connect (OIDC) flows.

To use a provider that is not supported out of the box, you can use the [Generic OAuth Plugin](/docs/plugins/generic-oauth).

## Installation

<Steps>
  <Step>
    ### Add the plugin to your auth config

    To use the Generic OAuth plugin, add it to your auth config.

```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { genericOAuth } from "better-auth/plugins" // [!code highlight]

    export const auth = betterAuth({
        // ... other config options
        plugins: [
            genericOAuth({ // [!code highlight]
                config: [ // [!code highlight]
                    { // [!code highlight]
                        providerId: "provider-id", // [!code highlight]
                        clientId: "test-client-id", // [!code highlight]
                        clientSecret: "test-client-secret", // [!code highlight]
                        discoveryUrl: "https://auth.example.com/.well-known/openid-configuration", // [!code highlight]
                        // ... other config options // [!code highlight]
                    }, // [!code highlight]
                    // Add more providers as needed // [!code highlight]
                ] // [!code highlight]
            }) // [!code highlight]
        ]
    })
```
  </Step>

  <Step>
    ### Add the client plugin

    Include the Generic OAuth client plugin in your authentication client instance.

```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    import { genericOAuthClient } from "better-auth/client/plugins"

    const authClient = createAuthClient({
        plugins: [
            genericOAuthClient()
        ]
    })
```
  </Step>
</Steps>

<Callout>
  Read more about installation and usage of the Generic Oauth plugin
  [here](/docs/plugins/generic-oauth#usage).
</Callout>

## Example usage

### Instagram Example

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { genericOAuth } from "better-auth/plugins";

export const auth = betterAuth({
  // ... other config options
  plugins: [
    genericOAuth({
      config: [
        {
          providerId: "instagram",
          clientId: process.env.INSTAGRAM_CLIENT_ID as string,
          clientSecret: process.env.INSTAGRAM_CLIENT_SECRET as string,
          authorizationUrl: "https://api.instagram.com/oauth/authorize",
          tokenUrl: "https://api.instagram.com/oauth/access_token",
          scopes: ["user_profile", "user_media"],
        },
      ],
    }),
  ],
});
```

```ts title="sign-in.ts"
const response = await authClient.signIn.oauth2({
  providerId: "instagram",
  callbackURL: "/dashboard", // the path to redirect to after the user is authenticated
});
```

### Coinbase Example

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { genericOAuth } from "better-auth/plugins";

export const auth = betterAuth({
  // ... other config options
  plugins: [
    genericOAuth({
      config: [
        {
          providerId: "coinbase",
          clientId: process.env.COINBASE_CLIENT_ID as string,
          clientSecret: process.env.COINBASE_CLIENT_SECRET as string,
          authorizationUrl: "https://www.coinbase.com/oauth/authorize",
          tokenUrl: "https://api.coinbase.com/oauth/token",
          scopes: ["wallet:user:read"], // and more...
        },
      ],
    }),
  ],
});
```

```ts title="sign-in.ts"
const response = await authClient.signIn.oauth2({
  providerId: "coinbase",
  callbackURL: "/dashboard", // the path to redirect to after the user is authenticated
});
```

# authentication: PayPal
URL: /docs/authentication/paypal
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/paypal.mdx

Paypal provider setup and usage.

***

title: PayPal
description: Paypal provider setup and usage.
---------------------------------------------

<Steps>
  <Step>
    ### Get your PayPal Credentials

    To integrate with PayPal, you need to obtain API credentials by creating an application in the [PayPal Developer Portal](https://developer.paypal.com/dashboard).

    Follow these steps:

    1. Create an account on the PayPal Developer Portal
    2. Create a new application, [official docs](https://developer.paypal.com/developer/applications/)
    3. Configure Log in with PayPal under "Other features"
    4. Set up your Return URL (redirect URL)
    5. Configure user information permissions
    6. Note your Client ID and Client Secret

    <Callout type="info">
      * PayPal has two environments: Sandbox (for testing) and Live (for production)
      * For testing, create sandbox test accounts in the Developer Dashboard under "Sandbox" â†’ "Accounts"
      * You cannot use your real PayPal account to test in sandbox mode - you must use the generated test accounts
      * The Return URL in your PayPal app settings must exactly match your redirect URI
      * The PayPal API does not work with localhost. You need to use a public domain for the redirect URL and HTTPS for local testing. You can use [NGROK](https://ngrok.com/) or another similar tool for this.
    </Callout>

    Make sure to configure "Log in with PayPal" in your app settings:

    1. Go to your app in the Developer Dashboard
    2. Under "Other features", check "Log in with PayPal"
    3. Click "Advanced Settings"
    4. Enter your Return URL
    5. Select the user information you want to access (email, name, etc.)
    6. Enter Privacy Policy and User Agreement URLs

    <Callout type="info">
      * PayPal doesn't use traditional OAuth2 scopes in the authorization URL. Instead, you configure permissions directly in the Developer Dashboard
      * For live apps, PayPal must review and approve your application before it can go live, which typically takes a few weeks
    </Callout>
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            paypal: { // [!code highlight]
                clientId: process.env.PAYPAL_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.PAYPAL_CLIENT_SECRET as string, // [!code highlight]
                environment: "sandbox", // or "live" for production //, // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```

    #### Options

    The PayPal provider accepts the following options:

    * `environment`: `'sandbox' | 'live'` - PayPal environment to use (default: `'sandbox'`)
    * `requestShippingAddress`: `boolean` - Whether to request shipping address information (default: `false`)

    ```ts title="auth.ts"
    export const auth = betterAuth({
        socialProviders: {
            paypal: {
                clientId: process.env.PAYPAL_CLIENT_ID as string,
                clientSecret: process.env.PAYPAL_CLIENT_SECRET as string,
                environment: "live", // Use "live" for production
                requestShippingAddress: true, // Request address info
            },
        },
    })
    ```
  </Step>

~~~
  <Step>
    ### Sign In with PayPal

    To sign in with PayPal, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `paypal`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient =  createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "paypal"
        })
    }
    ```

    ### Additional Options:

    * `environment`: PayPal environment to use.
      * Default: `"sandbox"`
      * Options: `"sandbox"` | `"live"`
    * `requestShippingAddress`: Whether to request shipping address information.
      * Default: `false`
    * `scope`: Additional scopes to request (combined with default permissions).
      * Default: Configured in PayPal Developer Dashboard
      * Note: PayPal doesn't use traditional OAuth2 scopes - permissions are set in the Dashboard
        For more details refer to the [Scopes Reference](https://developer.paypal.com/docs/log-in-with-paypal/integrate/reference/#scope-attributes)
    * `mapProfileToUser`: Custom function to map PayPal profile data to user object.
    * `getUserInfo`: Custom function to retrieve user information.
      For more details refer to the [User Reference](https://developer.paypal.com/docs/api/identity/v1/#userinfo_get)
    * `verifyIdToken`: Custom ID token verification function.
  </Step>
</Steps>



# authentication: Reddit
URL: /docs/authentication/reddit
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/reddit.mdx

Reddit provider setup and usage.

***

title: Reddit
description: Reddit provider setup and usage.
---------------------------------------------

<Steps>
  <Step>
    ### Get your Reddit Credentials

    To use Reddit sign in, you need a client ID and client secret. You can get them from the [Reddit Developer Portal](https://www.reddit.com/prefs/apps).

    1. Click "Create App" or "Create Another App"
    2. Select "web app" as the application type
    3. Set the redirect URL to `http://localhost:3000/api/auth/callback/reddit` for local development
    4. For production, set it to your application's domain (e.g. `https://example.com/api/auth/callback/reddit`)
    5. After creating the app, you'll get the client ID (under the app name) and client secret

    If you change the base path of the auth routes, make sure to update the redirect URL accordingly.
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            reddit: {
                clientId: process.env.REDDIT_CLIENT_ID as string,
                clientSecret: process.env.REDDIT_CLIENT_SECRET as string,
            },
        },
    })
    ```
  </Step>

  <Step>
    ### Sign In with Reddit

    To sign in with Reddit, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `reddit`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient = createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "reddit"
        })
    }
    ```
  </Step>
</Steps>

## Additional Configuration

### Scopes

By default, Reddit provides basic user information. If you need additional permissions, you can specify scopes in your auth configuration:

```ts title="auth.ts"
export const auth = betterAuth({
    socialProviders: {
        reddit: {
            clientId: process.env.REDDIT_CLIENT_ID as string,
            clientSecret: process.env.REDDIT_CLIENT_SECRET as string,
            duration: "permanent",
            scope: ["read", "submit"] // Add required scopes
        },
    },
})
```

Common Reddit scopes include:

* `identity`: Access basic account information
* `read`: Access posts and comments
* `submit`: Submit posts and comments
* `subscribe`: Manage subreddit subscriptions
* `history`: Access voting history
~~~

~~~
For a complete list of available scopes, refer to the [Reddit OAuth2 documentation](https://www.reddit.com/dev/api/oauth).



# authentication: Roblox
URL: /docs/authentication/roblox
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/roblox.mdx

Roblox provider setup and usage.

***

title: Roblox
description: Roblox provider setup and usage.
----------------------------------------------

<Steps>
  <Step>
    ### Get your Roblox Credentials

    Get your Roblox credentials from the [Roblox Creator Hub](https://create.roblox.com/dashboard/credentials?activeTab=OAuthTab).

    Make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/roblox` for local development. For production, you should set it to the URL of your application.
If you change the base path of the auth routes, you should update the redirect URL accordingly.

    <Callout type="info">
      The Roblox API does not provide email addresses. As a workaround, the user's `email` field uses the `preferred_username` value instead.
    </Callout>
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            roblox: { // [!code highlight]
                clientId: process.env.ROBLOX_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.ROBLOX_CLIENT_SECRET as string, // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```
  </Step>

  <Step>
    ### Sign In with Roblox

    To sign in with Roblox, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `roblox`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient =  createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "roblox"
        })
    }
    ```
  </Step>
</Steps>



# authentication: Salesforce
URL: /docs/authentication/salesforce
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/salesforce.mdx

Salesforce provider setup and usage.

***

title: Salesforce
description: Salesforce provider setup and usage.
--------------------------------------------------

<Steps>
  <Step>
    ### Get your Salesforce Credentials

    1. Log into your Salesforce org (Production or Developer Edition)
    2. Navigate to **Setup > App Manager**
    3. Click **New Connected App**
    4. Fill in the basic information:
       * Connected App Name: Your app name
       * API Name: Auto-generated from app name
       * Contact Email: Your email address
    5. Enable OAuth Settings:
       * Check **Enable OAuth Settings**
       * Set **Callback URL** to your redirect URI (e.g., `http://localhost:3000/api/auth/callback/salesforce` for development)
       * Select Required OAuth Scopes:
         * Access your basic information (id)
         * Access your identity URL service (openid)
         * Access your email address (email)
         * Perform requests on your behalf at any time (refresh\_token, offline\_access)
    6. Enable **Require Proof Key for Code Exchange (PKCE)** (required)
    7. Save and note your **Consumer Key** (Client ID) and **Consumer Secret** (Client Secret)

    <Callout type="info">
      * For development, you can use `http://localhost:3000` URLs, but production requires HTTPS
      * The callback URL must exactly match what's configured in Better Auth
      * PKCE (Proof Key for Code Exchange) is required by Salesforce and is automatically handled by the provider
    </Callout>

    <Callout type="warning">
      For sandbox testing, you can create the Connected App in your sandbox org, or use the same Connected App but specify `environment: "sandbox"` in the provider configuration.
    </Callout>
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
~~~

~~~
    export const auth = betterAuth({
        socialProviders: {
            salesforce: { // [!code highlight]
                clientId: process.env.SALESFORCE_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.SALESFORCE_CLIENT_SECRET as string, // [!code highlight]
                environment: "production", // or "sandbox" // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```

    #### Configuration Options

    * `clientId`: Your Connected App's Consumer Key
    * `clientSecret`: Your Connected App's Consumer Secret
    * `environment`: `"production"` (default) or `"sandbox"`
    * `loginUrl`: Custom My Domain URL (without `https://`) - overrides environment setting
    * `redirectURI`: Override the auto-generated redirect URI if needed

    #### Advanced Configuration

    ```ts title="auth.ts"
    export const auth = betterAuth({
        socialProviders: {
            salesforce: {
                clientId: process.env.SALESFORCE_CLIENT_ID as string,
                clientSecret: process.env.SALESFORCE_CLIENT_SECRET as string,
                environment: "sandbox", // [!code highlight]
                loginUrl: "mycompany.my.salesforce.com", // Custom My Domain // [!code highlight]
                redirectURI: "http://localhost:3000/api/auth/callback/salesforce", // Override if needed // [!code highlight]
            },
        },
    })
    ```

    <Callout type="info">
      * Use `environment: "sandbox"` for testing with Salesforce sandbox orgs
      * The `loginUrl` option is useful for organizations with My Domain enabled
      * The `redirectURI` option helps resolve redirect URI mismatch errors
    </Callout>
  </Step>

  <Step>
    ### Environment Variables

    Add the following environment variables to your `.env.local` file:

    ```bash title=".env.local"
    SALESFORCE_CLIENT_ID=your_consumer_key_here
    SALESFORCE_CLIENT_SECRET=your_consumer_secret_here
    BETTER_AUTH_URL=http://localhost:3000 # Important for redirect URI generation
    ```

    For production:

    ```bash title=".env"
    SALESFORCE_CLIENT_ID=your_consumer_key_here
    SALESFORCE_CLIENT_SECRET=your_consumer_secret_here
    BETTER_AUTH_URL=https://yourdomain.com
    ```
  </Step>

  <Step>
    ### Sign In with Salesforce

    To sign in with Salesforce, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `salesforce`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient = createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "salesforce"
        })
    }
    ```
  </Step>

  <Step>
    ### Troubleshooting

    #### Redirect URI Mismatch Error

    If you encounter a `redirect_uri_mismatch` error:

    1. **Check Callback URL**: Ensure the Callback URL in your Salesforce Connected App exactly matches your Better Auth callback URL
    2. **Protocol**: Make sure you're using the same protocol (`http://` vs `https://`)
    3. **Port**: Verify the port number matches (e.g., `:3000`)
    4. **Override if needed**: Use the `redirectURI` option to explicitly set the redirect URI

    ```ts
    salesforce: {
        clientId: process.env.SALESFORCE_CLIENT_ID as string,
        clientSecret: process.env.SALESFORCE_CLIENT_SECRET as string,
        redirectURI: "http://localhost:3000/api/auth/callback/salesforce", // [!code highlight]
    }
    ```

    #### Environment Issues

    * **Production**: Use `environment: "production"` (default) with `login.salesforce.com`
    * **Sandbox**: Use `environment: "sandbox"` with `test.salesforce.com`
    * **My Domain**: Use `loginUrl: "yourcompany.my.salesforce.com"` for custom domains

    #### PKCE Requirements

    Salesforce requires PKCE (Proof Key for Code Exchange) which is automatically handled by this provider. Make sure PKCE is enabled in your Connected App settings.

    <Callout type="info">
      The default scopes requested are `openid`, `email`, and `profile`. The provider will automatically include the `id` scope for accessing basic user information.
    </Callout>
  </Step>
</Steps>



# authentication: Slack
URL: /docs/authentication/slack
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/slack.mdx

Slack provider setup and usage.

***

title: Slack
description: Slack provider setup and usage.
--------------------------------------------
~~~

~~~
<Steps>
  <Step>
    ### Get your Slack credentials

    To use Slack as a social provider, you need to create a Slack app and get your credentials.

    1. Go to [Your Apps on Slack API](https://api.slack.com/apps) and click "Create New App"
    2. Choose "From scratch" and give your app a name and select a development workspace
    3. In your app settings, navigate to "OAuth & Permissions"
    4. Under "Redirect URLs", add your redirect URL:
       * For local development: `http://localhost:3000/api/auth/callback/slack`
       * For production: `https://yourdomain.com/api/auth/callback/slack`
    5. Copy your Client ID and Client Secret from the "Basic Information" page

    <Callout>
      Slack requires HTTPS for redirect URLs in production. For local development, you can use tools like [ngrok](https://ngrok.com/) to create a secure tunnel.
    </Callout>
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to pass the `clientId` and `clientSecret` to `socialProviders.slack` in your auth configuration.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            slack: { // [!code highlight]
                clientId: process.env.SLACK_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.SLACK_CLIENT_SECRET as string, // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```
  </Step>
</Steps>

## Usage

### Sign In with Slack

To sign in with Slack, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

* `provider`: The provider to use. It should be set to `slack`.

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client";
const authClient = createAuthClient();

const signIn = async () => {
  const data = await authClient.signIn.social({ provider: "slack" });
};
```

### Requesting Additional Scopes

By default, Slack uses OpenID Connect scopes: `openid`, `profile`, and `email`. You can request additional Slack scopes during sign-in:

```ts title="auth-client.ts"
const signInWithSlack = async () => {
  await authClient.signIn.social({
    provider: "slack",
    scopes: ["channels:read", "chat:write"], // Additional Slack API scopes
  });
};
```

### Workspace-Specific Sign In

If you want to restrict sign-in to a specific Slack workspace, you can pass the `team` parameter:

```ts title="auth.ts"
socialProviders: {
    slack: {
        clientId: process.env.SLACK_CLIENT_ID as string,
        clientSecret: process.env.SLACK_CLIENT_SECRET as string,
        team: "T1234567890", // Your Slack workspace ID
    },
}
```

### Using Slack API After Sign In

After successful authentication, you can access the user's Slack information through the session. The access token can be used to make requests to the Slack API:

```ts
const session = await authClient.getSession();
if (session?.user) {
  // Access Slack-specific data
  const slackUserId = session.user.id; // This is the Slack user ID
  // The access token is stored securely on the server
}
```

<Callout>
  The Slack provider uses OpenID Connect by default, which provides basic user
  information. If you need to access other Slack APIs, make sure to request the
  appropriate scopes during sign-in.
</Callout>



# authentication: Spotify
URL: /docs/authentication/spotify
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/spotify.mdx

Spotify provider setup and usage.

***

title: Spotify
description: Spotify provider setup and usage.
----------------------------------------------

<Steps>
  <Step>
    ### Get your Spotify Credentials

    To use Spotify sign in, you need a client ID and client secret. You can get them from the [Spotify Developer Portal](https://developer.spotify.com/dashboard/applications).

    Make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/spotify` for local development. For production, you should set it to the URL of your
application. If you change the base path of the auth routes, you should update the redirect URL accordingly.
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.
~~~

~~~text
    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({

        socialProviders: {
            spotify: { // [!code highlight]
                clientId: process.env.SPOTIFY_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.SPOTIFY_CLIENT_SECRET as string, // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```
  </Step>

  <Step>
    ### Sign In with Spotify

    To sign in with Spotify, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `spotify`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient =  createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "spotify"
        })
    }
    ```
  </Step>
</Steps>



# authentication: TikTok
URL: /docs/authentication/tiktok
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/tiktok.mdx

TikTok provider setup and usage.

***

title: TikTok
description: TikTok provider setup and usage.
---------------------------------------------

<Steps>
  <Step>
    ### Get your TikTok Credentials

    To integrate with TikTok, you need to obtain API credentials by creating an application in the [TikTok Developer Portal](https://developers.tiktok.com/apps).

    Follow these steps:

    1. Create an account on the TikTok Developer Portal
    2. Create a new application
    3. Set up a sandbox environment for testing
    4. Configure your redirect URL (must be HTTPS)
    5. Note your Client Secret and Client Key

    <Callout type="info">
      * The TikTok API does not work with localhost. You need to use a public domain for the redirect URL and HTTPS for local testing. You can use [NGROK](https://ngrok.com/) or another similar tool for this.
      * For testing, you will need to use the [Sandbox mode](https://developers.tiktok.com/blog/introducing-sandbox), which you can enable in the TikTok Developer Portal.
      * The default scope is `user.info.profile`. For additional scopes, refer to the [Available Scopes](https://developers.tiktok.com/doc/tiktok-api-scopes/) documentation.
    </Callout>

    Make sure to set the redirect URL to a valid HTTPS domain for local development. For production, you should set it to the URL of your application. If you change the base path of the auth routes, you should update the redirect URL accordingly.

    <Callout type="info">
      * The TikTok API does not provide email addresses. As a workaround, this implementation uses the user's `username` value for the `email` field, which is why it requires the `user.info.profile` scope instead of just `user.info.basic`.
      * For production use, you will need to request approval from TikTok for the scopes you intend to use.
    </Callout>
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            tiktok: { // [!code highlight]
                clientSecret: process.env.TIKTOK_CLIENT_SECRET as string, // [!code highlight]
                clientKey: process.env.TIKTOK_CLIENT_KEY as string, // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```
  </Step>

  <Step>
    ### Sign In with TikTok

    To sign in with TikTok, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `tiktok`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient =  createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "tiktok"
        })
    }
    ```
  </Step>
</Steps>



# authentication: Twitch
URL: /docs/authentication/twitch
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/twitch.mdx

Twitch provider setup and usage.

***

title: Twitch
description: Twitch provider setup and usage.
~~~

~~~
---------------------------------------------

<Steps>
  <Step>
    ### Get your Twitch Credentials

    To use Twitch sign in, you need a client ID and client secret. You can get them from the [Twitch Developer Portal](https://dev.twitch.tv/console/apps).

    Make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/twitch` for local development. For production, you should set it to the URL of your application. If you change the base path of the auth routes, you should update the redirect URL accordingly.
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            twitch: { // [!code highlight]
                clientId: process.env.TWITCH_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.TWITCH_CLIENT_SECRET as string, // [!code highlight]
            }, // [!code highlight]
        }
    })
    ```
  </Step>

  <Step>
    ### Sign In with Twitch

    To sign in with Twitch, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `twitch`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient =  createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "twitch"
        })
    }
    ```
  </Step>
</Steps>



# authentication: Twitter (X)
URL: /docs/authentication/twitter
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/twitter.mdx

Twitter provider setup and usage.

***

title: Twitter (X)
description: Twitter provider setup and usage.
----------------------------------------------

<Steps>
  <Step>
    ### Get your Twitter Credentials

    Get your Twitter credentials from the [Twitter Developer Portal](https://developer.twitter.com/en/portal/dashboard).

    Make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/twitter` for local development. For production, you should set it to the URL of your application. If you change the base path of the auth routes, you should update the redirect URL accordingly.

    <Callout type="info">
      Twitter API v2 now supports email address retrieval. Make sure to request the `user.email` scope when configuring your Twitter app to enable this feature.
    </Callout>
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
        socialProviders: {
            twitter: { // [!code highlight]
                clientId: process.env.TWITTER_CLIENT_ID as string, // [!code highlight]
                clientSecret: process.env.TWITTER_CLIENT_SECRET as string, // [!code highlight]
            }, // [!code highlight]
        },
    })
    ```
  </Step>

  <Step>
    ### Sign In with Twitter

    To sign in with Twitter, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `twitter`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient =  createAuthClient()

    const signIn = async () => {
        const data = await authClient.signIn.social({
            provider: "twitter"
        })
    }
    ```
  </Step>
</Steps>



# authentication: VK
URL: /docs/authentication/vk
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/vk.mdx

VK ID Provider

***

title: VK
description: VK ID Provider
---------------------------
~~~

~~~
<Steps>
  <Step>
    ### Get your VK ID credentials

    To use VK ID sign in, you need a client ID and client secret. You can get them from the [VK ID Developer Portal](https://id.vk.com/about/business/go/docs).

    Make sure to set the redirect URL to `http://localhost:3000/api/auth/callback/vk` for local development. For production, you should set it to the URL of your application. If you change the base path of the auth routes, you should update the redirect URL accordingly.
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth";

    export const auth = betterAuth({
      socialProviders: {
        vk: { // [!code highlight]
          clientId: process.env.VK_CLIENT_ID as string, // [!code highlight]
          clientSecret: process.env.VK_CLIENT_SECRET as string, // [!code highlight]
        },
      },
    });
    ```
  </Step>

  <Step>
    ### Sign In with VK

    To sign in with VK, you can use the `signIn.social` function provided by the client. The `signIn` function takes an object with the following properties:

    * `provider`: The provider to use. It should be set to `vk`.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client";
    const authClient = createAuthClient();

    const signIn = async () => {
      const data = await authClient.signIn.social({
        provider: "vk",
      });
    };
    ```
  </Step>
</Steps>



# authentication: Zoom
URL: /docs/authentication/zoom
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/authentication/zoom.mdx

Zoom provider setup and usage.

***

title: Zoom
description: Zoom provider setup and usage.
-------------------------------------------

<Steps>
  <Step>
    ### Create a Zoom App from Marketplace

    1. Visit [Zoom Marketplace](https://marketplace.zoom.us).

    2. Hover on the `Develop` button and select `Build App`

    3. Select `General App` and click `Create`
  </Step>

  <Step>
    ### Configure your Zoom App

    Ensure that you are in the `Basic Information` of your app settings.

    1. Under `Select how the app is managed`, choose `User-managed`

    2. Under `App Credentials`, copy your `Client ID` and `Client Secret` and store them in a safe location

    3. Under `OAuth Information` -> `OAuth Redirect URL`, add your Callback URL. For example,

       ```
       http://localhost:3000/api/auth/callback/zoom
       ```

       <Callout>
         For production, you should set it to the URL of your application. If you change the base
         path of the auth routes, you should update the redirect URL accordingly.
       </Callout>

    Skip to the `Scopes` section, then

    1. Click the `Add Scopes` button
    2. Search for `user:read:user` (View a user) and select it
    3. Add any other scopes your applications needs and click `Done`
  </Step>

  <Step>
    ### Configure the provider

    To configure the provider, you need to import the provider and pass it to the `socialProviders` option of the auth instance.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"

    export const auth = betterAuth({
      socialProviders: {
        zoom: { // [!code highlight]
          clientId: process.env.ZOOM_CLIENT_ID as string, // [!code highlight]
          clientSecret: process.env.ZOOM_CLIENT_SECRET as string, // [!code highlight]
        }, // [!code highlight]
      },
    })
    ```
  </Step>

  <Step>
    ### Sign In with Zoom

    To sign in with Zoom, you can use the `signIn.social` function provided by the client.
    You will need to specify `zoom` as the provider.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    const authClient =  createAuthClient()
~~~

~~~~
    const signIn = async () => {
      const data = await authClient.signIn.social({
        provider: "zoom"
      })
    }
    ```
  </Step>
</Steps>
~~~~

# concepts: API
URL: /docs/concepts/api
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/concepts/api.mdx

Better Auth API.

***

title: API
description: Better Auth API.
-----------------------------

When you create a new Better Auth instance, it provides you with an `api` object. This object exposes every endpoint that exists in your Better Auth instance. And you can use this to interact with Better Auth server side.

Any endpoint added to Better Auth, whether from plugins or the core, will be accessible through the `api` object.

## Calling API Endpoints on the Server

To call an API endpoint on the server, import your `auth` instance and call the endpoint using the `api` object.

```ts title="server.ts"
import { betterAuth } from "better-auth";
import { headers } from "next/headers";

export const auth = betterAuth({
    //...
})

// calling get session on the server
await auth.api.getSession({
    headers: await headers() // some endpoints might require headers
})
```

### Body, Headers, Query

Unlike the client, the server needs the values to be passed as an object with the key `body` for the body, `headers` for the headers, and `query` for query parameters.

```ts title="server.ts"
await auth.api.getSession({
    headers: await headers()
})

await auth.api.signInEmail({
    body: {
        email: "john@doe.com",
        password: "password"
    },
    headers: await headers() // optional but would be useful to get the user IP, user agent, etc.
})

await auth.api.verifyEmail({
    query: {
        token: "my_token"
    }
})
```

<Callout>
  Better Auth API endpoints are built on top of [better-call](https://github.com/bekacru/better-call), a tiny web framework that lets you call REST API endpoints as if they were regular functions and allows us to easily infer client types from the server.
</Callout>

### Getting `headers` and `Response` Object

When you invoke an API endpoint on the server, it will return a standard JavaScript object or array directly as it's just a regular function call.

But there are times when you might want to get the `headers` or the `Response` object instead. For example, if you need to get the cookies or the headers.

#### Getting `headers`

To get the `headers`, you can pass the `returnHeaders` option to the endpoint.

```ts
const { headers, response } = await auth.api.signUpEmail({
        returnHeaders: true,
        body: {
                email: "john@doe.com",
                password: "password",
                name: "John Doe",
        },
});
```

The `headers` will be a `Headers` object, which you can use to get the cookies or the headers.

```ts
const cookies = headers.get("set-cookie");
const headers = headers.get("x-custom-header");
```

#### Getting `Response` Object

To get the `Response` object, you can pass the `asResponse` option to the endpoint.

```ts title="server.ts"
const response = await auth.api.signInEmail({
    body: {
        email: "",
        password: ""
    },
    asResponse: true
})
```

### Error Handling

When you call an API endpoint on the server, it will throw an error if the request fails. You can catch the error and handle it as you see fit. The error instance is an instance of `APIError`.

```ts title="server.ts"
import { APIError } from "better-auth/api";
```

```
try {
    await auth.api.signInEmail({
        body: {
            email: "",
            password: ""
        }
    })
} catch (error) {
    if (error instanceof APIError) {
        console.log(error.message, error.status)
    }
}
```

# concepts: CLI
URL: /docs/concepts/cli
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/concepts/cli.mdx

Built-in CLI for managing your project.

***

title: CLI
description: Built-in CLI for managing your project.
-----------------------------------------------------

Better Auth comes with a built-in CLI to help you manage the database schemas, initialize your project, generate a secret key for your application, and gather diagnostic information about your setup.

## Generate

The `generate` command creates the schema required by Better Auth. If you're using a database adapter like Prisma or Drizzle, this command will generate the right schema for your ORM. If you're using the built-in Kysely adapter, it will generate an SQL file you can run directly on your database.

```bash title="Terminal"
npx @better-auth/cli@latest generate
```

### Options

* `--output` - Where to save the generated schema. For Prisma, it will be saved in prisma/schema.prisma. For Drizzle, it goes to schema.ts in your project root. For Kysely, it's an SQL file saved as schema.sql in your project root.
* `--config` - The path to your Better Auth config file. By default, the CLI will search for an auth.ts file in **./**, **./utils**, **./lib**, or any of these directories under the `src` directory.
* `--yes` - Skip the confirmation prompt and generate the schema directly.

## Migrate

The migrate command applies the Better Auth schema directly to your database. This is available if you're using the built-in Kysely adapter. For other adapters, you'll need to apply the schema using your ORM's migration tool.

```bash title="Terminal"
npx @better-auth/cli@latest migrate
```

### Options

* `--config` - The path to your Better Auth config file. By default, the CLI will search for an auth.ts file in **./**, **./utils**, **./lib**, or any of these directories under the `src` directory.
* `--yes` - Skip the confirmation prompt and apply the schema directly.

## Init

The `init` command allows you to initialize Better Auth in your project.

```bash title="Terminal"
npx @better-auth/cli@latest init
```

### Options

* `--name` - The name of your application. (defaults to the `name` property in your `package.json`).
* `--framework` - The framework your codebase is using. Currently, the only supported framework is `Next.js`.
* `--plugins` - The plugins you want to use. You can specify multiple plugins by separating them with a comma.
* `--database` - The database you want to use. Currently, the only supported database is `SQLite`.
* `--package-manager` - The package manager you want to use. Currently, the only supported package managers are `npm`, `pnpm`, `yarn`, `bun` (defaults to the manager you used to initialize the CLI).

## Info

The `info` command provides diagnostic information about your Better Auth setup and environment. Useful for debugging and sharing when seeking support.

```bash title="Terminal"
npx @better-auth/cli@latest info
```

### Output

The command displays:

* **System**: OS, CPU, memory, Node.js version
* **Package Manager**: Detected manager and version
* **Better Auth**: Version and configuration (sensitive data auto-redacted)
* **Frameworks**: Detected frameworks (Next.js, React, Vue, etc.)
* **Databases**: Database clients and ORMs (Prisma, Drizzle, etc.)

### Options

* `--config` - Path to your Better Auth config file
* `--json` - Output as JSON for sharing or programmatic use

### Examples

```bash
# Basic usage
npx @better-auth/cli@latest info

# Custom config path
npx @better-auth/cli@latest info --config ./config/auth.ts

# JSON output
npx @better-auth/cli@latest info --json > auth-info.json
```

Sensitive data like secrets, API keys, and database URLs are automatically replaced with `[REDACTED]` for safe sharing.

## Secret

The CLI also provides a way to generate a secret key for your Better Auth instance.

```bash title="Terminal"
npx @better-auth/cli@latest secret
```

## Common Issues

**Error: Cannot find module X**

If you see this error, it means the CLI can't resolve imported modules in your Better Auth config file. We are working on a fix for many of these issues, but in the meantime, you can try the following:

* Remove any import aliases in your config file and use relative paths instead. After running the CLI, you can revert to using aliases.

# concepts: Client
URL: /docs/concepts/client
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/concepts/client.mdx

Better Auth client library for authentication.

***

title: Client
description: Better Auth client library for authentication.
-----------------------------------------------------------

Better Auth offers a client library compatible with popular frontend frameworks like React, Vue, Svelte, and more. This client library includes a set of functions for interacting with the Better Auth server. Each framework's client library is built on top of a core client library that is framework-agnostic, so that all methods and hooks are consistently available across all client libraries.

## Installation

If you haven't already, install better-auth.

<CodeBlockTabs defaultValue="npm">
  <CodeBlockTabsList>
    <CodeBlockTabsTrigger value="npm">
      npm
    </CodeBlockTabsTrigger>

    <CodeBlockTabsTrigger value="pnpm">
      pnpm
    </CodeBlockTabsTrigger>

    <CodeBlockTabsTrigger value="yarn">
      yarn
    </CodeBlockTabsTrigger>

    <CodeBlockTabsTrigger value="bun">
      bun
    </CodeBlockTabsTrigger>
  </CodeBlockTabsList>

  <CodeBlockTab value="npm">
    ```bash
    npm i better-auth
    ```
  </CodeBlockTab>

  <CodeBlockTab value="pnpm">
    ```bash
    pnpm add better-auth
    ```
  </CodeBlockTab>

  <CodeBlockTab value="yarn">
    ```bash
    yarn add better-auth
    ```
  </CodeBlockTab>

  <CodeBlockTab value="bun">
    ```bash
    bun add better-auth
    ```
  </CodeBlockTab>
</CodeBlockTabs>

## Create Client Instance

Import `createAuthClient` from the package for your framework (e.g., "better-auth/react" for React). Call the function to create your client. Pass the base URL of your auth server. If the auth server is running on the same domain as your client, you can skip this step.

<Callout type="info">
  If you're using a different base path other than `/api/auth`, make sure to pass the whole URL, including the path. (e.g., `http://localhost:3000/custom-path/auth`)
</Callout>

<Tabs
  items={["react", "vue", "svelte", "solid",
"vanilla"]}
  defaultValue="react"
>
  <Tab value="vanilla">
    ```ts title="lib/auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    export const authClient = createAuthClient({
        baseURL: "http://localhost:3000" // The base URL of your auth server // [!code highlight]
    })
    ```
  </Tab>

  <Tab value="react" title="lib/auth-client.ts">
    ```ts title="lib/auth-client.ts"
    import { createAuthClient } from "better-auth/react"
    export const authClient = createAuthClient({
        baseURL: "http://localhost:3000" // The base URL of your auth server // [!code highlight]
    })
    ```
  </Tab>

  <Tab value="vue" title="lib/auth-client.ts">
    ```ts title="lib/auth-client.ts"
    import { createAuthClient } from "better-auth/vue"
    export const authClient = createAuthClient({
        baseURL: "http://localhost:3000" // The base URL of your auth server // [!code highlight]
    })
    ```
  </Tab>

  <Tab value="svelte" title="lib/auth-client.ts">
    ```ts title="lib/auth-client.ts"
    import { createAuthClient } from "better-auth/svelte"
    export const authClient = createAuthClient({
        baseURL: "http://localhost:3000" // The base URL of your auth server // [!code highlight]
    })
    ```
  </Tab>

  <Tab value="solid" title="lib/auth-client.ts">
    ```ts title="lib/auth-client.ts"
    import { createAuthClient } from "better-auth/solid"
    export const authClient = createAuthClient({
        baseURL: "http://localhost:3000" // The base URL of your auth server // [!code highlight]
    })
    ```
  </Tab>
</Tabs>

## Usage

Once you've created your client instance, you can use the client to interact with the Better Auth server. The client provides a set of functions by default and they can be extended with plugins.

**Example: Sign In**

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client"
const authClient = createAuthClient()

await authClient.signIn.email({
    email: "test@user.com",
    password: "password1234"
})
```

### Hooks

In addition to the standard methods, the client provides hooks to easily access different reactive data. Every hook is available in the root object of the client and they all start with `use`.

**Example: useSession**

<Tabs items={["React", "Vue","Svelte", "Solid"]} defaultValue="react">
  <Tab value="React">
    ```tsx title="user.tsx"
    //make sure you're using the react client
    import { createAuthClient } from "better-auth/react"
    const { useSession } = createAuthClient() // [!code highlight]

    export function User() {
        const {
            data: session,
            isPending, //loading state
            error, //error object
            refetch //refetch the session
        } = useSession()
        return (
            //...
        )
    }
    ```
  </Tab>

  <Tab value="Vue">
    ```vue title="user.vue"
    <script lang="ts" setup>
    import { authClient } from '@/lib/auth-client'
    const session = authClient.useSession()
    </script>
    <template>
        <div>
            <button v-if="!session.data" @click="() => authClient.signIn.social({
                provider: 'github'
            })">
                Continue with GitHub
            </button>
            <div>
                <pre>{{ session.data }}</pre>
                <button v-if="session.data" @click="authClient.signOut()">
                    Sign out
                </button>
            </div>
        </div>
    </template>
    ```
  </Tab>

  <Tab value="Svelte">
    ```svelte title="user.svelte"
    <script lang="ts">
    import { client } from "$lib/client";
    const session = client.useSession();
    </script>

    <div
        style="display: flex; flex-direction: column; gap: 10px; border-radius: 10px; border: 1px solid #4B453F; padding: 20px; margin-top: 10px;"
    >
        <div>
        {#if $session.data}
            <div>
            <p>
                {$session.data.user.name}
            </p>
            <p>
                {$session.data.user.email}
            </p>
            <button
                onclick={async () => {
                await authClient.signOut();
                }}
            >
                Signout
            </button>
            </div>
        {:else}
            <button
            onclick={async () => {
                await authClient.signIn.social({
                provider: "github",
                });
            }}
            >
            Continue with GitHub
            </button>
        {/if}
        </div>
    </div>
    ```
  </Tab>

  <Tab value="Solid">
    ```tsx title="user.tsx"
    import { client } from "~/lib/client";
    import { Show } from 'solid-js';

    export default function Home() {
        const session = client.useSession()
        return (
            <Show
                when={session()}
                fallback={<button onClick={toggle}>Log in</button>}
            >
                <button onClick={toggle}>Log out</button>
            </Show>
        );
    }
    ```
  </Tab>
</Tabs>

### Fetch Options

The client uses a library called [better fetch](https://better-fetch.vercel.app) to make requests to the server.

Better fetch is a wrapper around the native fetch API that provides a more convenient way to make requests. It's created by the same team behind Better Auth and is designed to work seamlessly with it.

You can pass any default fetch options to the client by passing `fetchOptions` object to the `createAuthClient`.

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client"

const authClient = createAuthClient({
    fetchOptions: {
        //any better-fetch options
    },
})
```

You can also pass fetch options to most of the client functions. Either as the second argument or as a property in the object.

```ts title="auth-client.ts"
await authClient.signIn.email({
    email: "email@email.com",
    password: "password1234",
}, {
    onSuccess(ctx) {
            //
    }
})

//or

await authClient.signIn.email({
    email: "email@email.com",
    password: "password1234",
    fetchOptions: {
        onSuccess(ctx) {
            //
        }
    },
})
```

### Handling Errors

Most of the client functions return a response object with the following properties:

* `data`: The response data.
* `error`: The error object if there was an error.

The error object contains the following properties:

* `message`: The error message. (e.g., "Invalid email or password")
* `status`: The HTTP status code.
* `statusText`: The HTTP status text.

```ts title="auth-client.ts"
const { data, error } = await authClient.signIn.email({
    email: "email@email.com",
    password: "password1234"
})
if (error) {
    //handle error
}
```

If the action accepts a `fetchOptions` option, you can pass an `onError` callback to handle errors.

```ts title="auth-client.ts"

await authClient.signIn.email({
    email: "email@email.com",
    password: "password1234",
}, {
    onError(ctx) {
        //handle error
    }
})

//or
await authClient.signIn.email({
    email: "email@email.com",
    password: "password1234",
    fetchOptions: {
        onError(ctx) {
            //handle error
        }
    }
})
```

Hooks like `useSession` also return an error object if there was an error fetching the session. On top of that, they also return an `isPending` property to indicate if the request is still pending.

```ts title="auth-client.ts"
const { data, error, isPending } = useSession()
if (error) {
    //handle error
}
```

#### Error Codes

The client instance contains $ERROR_CODES object that contains all the error codes returned by the server. You can use this to handle error translations or custom error messages.

```ts title="auth-client.ts"
const authClient = createAuthClient();

type ErrorTypes = Partial<
        Record<
                keyof typeof authClient.$ERROR_CODES,
                {
                        en: string;
                        es: string;
                }
        >
>;

const errorCodes = {
        USER_ALREADY_EXISTS: {
                en: "user already registered",
                es: "usuario ya registrada",
        },
} satisfies ErrorTypes;
```

```
const getErrorMessage = (code: string, lang: "en" | "es") => {
        if (code in errorCodes) {
                return errorCodes[code as keyof typeof errorCodes][lang];
        }
        return "";
};


const { error } = await authClient.signUp.email({
        email: "user@email.com",
        password: "password",
        name: "User",
});
if(error?.code){
    alert(getErrorMessage(error.code, "en"));
}
```

### Plugins

You can extend the client with plugins to add more functionality. Plugins can add new functions to the client or modify existing ones.

**Example: Magic Link Plugin**

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client"
import { magicLinkClient } from "better-auth/client/plugins"

const authClient = createAuthClient({
    plugins: [
        magicLinkClient()
    ]
})
```

once you've added the plugin, you can use the new functions provided by the plugin.

```ts title="auth-client.ts"
await authClient.signIn.magicLink({
    email: "test@email.com"
})
```

# concepts: Cookies

URL: /docs/concepts/cookies
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/concepts/cookies.mdx

Learn how cookies are used in Better Auth.

***

title: Cookies
description: Learn how cookies are used in Better Auth.

---

Cookies are used to store data such as session tokens, OAuth state, and more. All cookies are signed using the `secret` key provided in the auth options.

### Cookie Prefix

By default, Better Auth cookies follow the format `${prefix}.${cookie_name}`. The default prefix is "better-auth". You can change the prefix by setting `cookiePrefix` in the `advanced` object of the auth options.

```ts title="auth.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    advanced: {
        cookiePrefix: "my-app"
    }
})
```

### Custom Cookies

All cookies are `httpOnly` and `secure` when the server is running in production mode.

If you want to set custom cookie names and attributes, you can do so by setting `cookieOptions` in the `advanced` object of the auth options.

By default, Better Auth uses the following cookies:

* `session_token` to store the session token
* `session_data` to store the session data if cookie cache is enabled
* `dont_remember` to store the flag when `rememberMe` is disabled

Plugins may also use cookies to store data. For example, the Two Factor Authentication plugin uses the `two_factor` cookie to store the two-factor authentication state.

```ts title="auth.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    advanced: {
        cookies: {
            session_token: {
                name: "custom_session_token",
                attributes: {
                    // Set custom cookie attributes
                }
            },
        }
    }
})
```

### Cross Subdomain Cookies

Sometimes you may need to share cookies across subdomains.
For example, if you authenticate on `auth.example.com`, you may also want to access the same session on `app.example.com`.

<Callout type="warn">
  The `domain` attribute controls which domains can access the cookie. Setting it to your root domain (e.g. `example.com`) makes the cookie accessible across all subdomains. For security, follow these guidelines:

  1. Only enable cross-subdomain cookies if it's necessary
  2. Set the domain to the most specific scope needed (e.g. `app.example.com` instead of `.example.com`)
  3. Be cautious of untrusted subdomains that could potentially access these cookies
  4. Consider using separate domains for untrusted services (e.g. `status.company.com` vs `app.company.com`)
</Callout>

```ts title="auth.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    advanced: {
        crossSubDomainCookies: {
            enabled: true,
            domain: "app.example.com", // your domain
        },
    },
    trustedOrigins: [
        'https://example.com',
        'https://app1.example.com',
        'https://app2.example.com',
    ],
})
```

### Secure Cookies

By default, cookies are secure only when the server is running in production mode. You can force cookies to be always secure by setting `useSecureCookies` to `true` in the `advanced` object in the auth options.

```ts title="auth.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    advanced: {
        useSecureCookies: true
    }
})
```

# concepts: Database
URL: /docs/concepts/database
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/concepts/database.mdx

Learn how to use a database with Better Auth.

***

title: Database
description: Learn how to use a database with Better Auth.
-----------------------------------------------------------

## Adapters

Better Auth requires a database connection to store data. The database will be used to store data such as users, sessions, and more. Plugins can also define their own database tables to store data.

You can pass a database connection to Better Auth by passing a supported database instance in the database options. You can learn more about supported database adapters in the [Other relational databases](/docs/adapters/other-relational-databases) documentation.

## CLI

Better Auth comes with a CLI tool to manage database migrations and generate schema.

### Running Migrations

The cli checks your database and prompts you to add missing tables or update existing ones with new columns. This is only supported for the built-in Kysely adapter. For other adapters, you can use the `generate` command to create the schema and handle the migration through your ORM.

```bash
npx @better-auth/cli migrate
```

### Generating Schema

Better Auth also provides a `generate` command to generate the schema required by Better Auth. The `generate` command creates the schema required by Better Auth. If you're using a database adapter like Prisma or Drizzle, this command will generate the right schema for your ORM. If you're using the built-in Kysely adapter, it will generate an SQL file you can run directly on your database.

```bash
npx @better-auth/cli generate
```

See the [CLI](/docs/concepts/cli) documentation for more information on the CLI.

<Callout>
  If you prefer adding tables manually, you can do that as well. The core schema
  required by Better Auth is described below and you can find additional schema
  required by plugins in the plugin documentation.
</Callout>

## Secondary Storage

Secondary storage in Better Auth allows you to use key-value stores for managing session data, rate limiting counters, etc. This can be useful when you want to offload the storage of this intensive records to a high performance storage or even RAM.

### Implementation

To use secondary storage, implement the `SecondaryStorage` interface:

```typescript
interface SecondaryStorage {
  get: (key: string) => Promise<unknown>;
  set: (key: string, value: string, ttl?: number) => Promise<void>;
  delete: (key: string) => Promise<void>;
}
```

Then, provide your implementation to the `betterAuth` function:

```typescript
betterAuth({
  // ... other options
  secondaryStorage: {
    // Your implementation here
  },
});
```

<Callout type="warn">
  Better Auth uses **seconds** for the TTL value in `set()`.
  If your storage expects milliseconds, multiply by 1000 when passing the TTL (`ttl * 1000`).
</Callout>

**Example: Redis Implementation**

Here's a basic example using Redis:

```typescript
import { createClient } from "redis";
import { betterAuth } from "better-auth";

const redis = createClient();
await redis.connect();

````
export const auth = betterAuth({
        // ... other options
        secondaryStorage: {
                get: async (key) => {
                        return await redis.get(key);
                },
                set: async (key, value, ttl) => {
                        // TTL in seconds â€” convert ms with ttl * 1000.
                        if (ttl) await redis.set(key, value, { EX: ttl });
                        // or for ioredis:
                        // if (ttl) await redis.set(key, value, 'EX', ttl)
                        else await redis.set(key, value);
                },
                delete: async (key) => {
                        await redis.del(key);
                }
        }
});
```

This implementation allows Better Auth to use Redis for storing session data and rate limiting counters. You can also add prefixes to the keys names.

## Core Schema

Better Auth requires the following tables to be present in the database. The types are in `typescript` format. You can use corresponding types in your database.

### User

Table Name: `user`

<DatabaseTable
  fields={[
  {
    name: "id",
    type: "string",
    description: "Unique identifier for each user",
    isPrimaryKey: true,
  },
  {
    name: "name",
    type: "string",
    description: "User's chosen display name",
  },
  {
    name: "email",
    type: "string",
    description: "User's email address for communication and login",
  },
  {
    name: "emailVerified",
    type: "boolean",
    description: "Whether the user's email is verified",
  },
  {
    name: "image",
    type: "string",
    description: "User's image url",
    isOptional: true,
  },
  {
    name: "createdAt",
    type: "Date",
    description: "Timestamp of when the user account was created",
  },
  {
    name: "updatedAt",
    type: "Date",
    description: "Timestamp of the last update to the user's information",
  },
]}
/>

### Session

Table Name: `session`

<DatabaseTable
  fields={[
  {
    name: "id",
    type: "string",
    description: "Unique identifier for each session",
    isPrimaryKey: true,
  },
  {
    name: "userId",
    type: "string",
    description: "The ID of the user",
    isForeignKey: true,
  },
  {
    name: "token",
    type: "string",
    description: "The unique session token",
    isUnique: true,
  },
  {
    name: "expiresAt",
    type: "Date",
    description: "The time when the session expires",
  },
  {
    name: "ipAddress",
    type: "string",
    description: "The IP address of the device",
    isOptional: true,
  },
  {
    name: "userAgent",
    type: "string",
    description: "The user agent information of the device",
    isOptional: true,
  },
  {
    name: "createdAt",
    type: "Date",
    description: "Timestamp of when the session was created",
  },
  {
    name: "updatedAt",
    type: "Date",
    description: "Timestamp of when the session was updated",
  },
]}
/>

### Account

Table Name: `account`
````

```
<DatabaseTable
  fields={[
  {
    name: "id",
    type: "string",
    description: "Unique identifier for each account",
    isPrimaryKey: true,
  },
  {
    name: "userId",
    type: "string",
    description: "The ID of the user",
    isForeignKey: true,
  },
  {
    name: "accountId",
    type: "string",
    description:
      "The ID of the account as provided by the SSO or equal to userId for credential accounts",
  },
  {
    name: "providerId",
    type: "string",
    description: "The ID of the provider",
  },
  {
    name: "accessToken",
    type: "string",
    description: "The access token of the account. Returned by the provider",
    isOptional: true,
  },
  {
    name: "refreshToken",
    type: "string",
    description: "The refresh token of the account. Returned by the provider",
    isOptional: true,
  },
  {
    name: "accessTokenExpiresAt",
    type: "Date",
    description: "The time when the access token expires",
    isOptional: true,
  },
  {
    name: "refreshTokenExpiresAt",
    type: "Date",
    description: "The time when the refresh token expires",
    isOptional: true,
  },
  {
    name: "scope",
    type: "string",
    description: "The scope of the account. Returned by the provider",
    isOptional: true,
  },
  {
    name: "idToken",
    type: "string",
    description: "The ID token returned from the provider",
    isOptional: true,
  },
  {
    name: "password",
    type: "string",
    description:
      "The password of the account. Mainly used for email and password authentication",
    isOptional: true,
  },
  {
    name: "createdAt",
    type: "Date",
    description: "Timestamp of when the account was created",
  },
  {
    name: "updatedAt",
    type: "Date",
    description: "Timestamp of when the account was updated",
  },
]}
/>
```

### Verification

Table Name: `verification`

```
<DatabaseTable
  fields={[
  {
    name: "id",
    type: "string",
    description: "Unique identifier for each verification",
    isPrimaryKey: true,
  },
  {
    name: "identifier",
    type: "string",
    description: "The identifier for the verification request",
  },
  {
    name: "value",
    type: "string",
    description: "The value to be verified",
  },
  {
    name: "expiresAt",
    type: "Date",
    description: "The time when the verification request expires",
  },
  {
    name: "createdAt",
    type: "Date",
    description: "Timestamp of when the verification request was created",
  },
  {
    name: "updatedAt",
    type: "Date",
    description: "Timestamp of when the verification request was updated",
  },
]}
/>
```

## Custom Tables

Better Auth allows you to customize the table names and column names for the core schema. You can also extend the core schema by adding additional fields to the user and session tables.

### Custom Table Names

You can customize the table names and column names for the core schema by using the `modelName` and `fields` properties in your auth config:

```ts title="auth.ts"
export const auth = betterAuth({
  user: {
    modelName: "users",
    fields: {
      name: "full_name",
      email: "email_address",
    },
  },
  session: {
    modelName: "user_sessions",
    fields: {
      userId: "user_id",
    },
  },
});
```

<Callout>
  Type inference in your code will still use the original field names (e.g.,
  `user.name`, not `user.full_name`).
</Callout>

To customize table names and column name for plugins, you can use the `schema` property in the plugin config:

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { twoFactor } from "better-auth/plugins";

export const auth = betterAuth({
  plugins: [
    twoFactor({
      schema: {
        user: {
          fields: {
            twoFactorEnabled: "two_factor_enabled",
            secret: "two_factor_secret",
          },
        },
      },
    }),
  ],
});
```

### Extending Core Schema

Better Auth provides a type-safe way to extend the `user` and `session` schemas. You can add custom fields to your auth config, and the CLI will automatically update the database schema. These additional fields will be properly inferred in functions like `useSession`, `signUp.email`, and other endpoints that work with user or session objects.

To add custom fields, use the `additionalFields` property in the `user` or `session` object of your auth config. The `additionalFields` object uses field names as keys, with each value being a `FieldAttributes` object containing:

* `type`: The data type of the field (e.g., "string", "number", "boolean").
* `required`: A boolean indicating if the field is mandatory.
* `defaultValue`: The default value for the field (note: this only applies in the JavaScript layer; in the database, the field will be optional).
* `input`: This determines whether a value can be provided when creating a new record (default: `true`). If there are additional fields, like `role`, that should not be provided by the user during signup, you can set this to `false`.

Here's an example of how to extend the user schema with additional fields:

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
  user: {
    additionalFields: {
      role: {
        type: "string",
        required: false,
        defaultValue: "user",
        input: false, // don't allow user to set role
      },
      lang: {
        type: "string",
        required: false,
        defaultValue: "en",
      },
    },
  },
});
```

Now you can access the additional fields in your application logic.

```ts
//on signup
const res = await auth.api.signUpEmail({
  email: "test@example.com",
  password: "password",
  name: "John Doe",
  lang: "fr",
});

//user object
res.user.role; // > "admin"
res.user.lang; // > "fr"
```

<Callout>
  See the
  [TypeScript](/docs/concepts/typescript#inferring-additional-fields-on-client)
  documentation for more information on how to infer additional fields on the
  client side.
</Callout>

If you're using social / OAuth providers, you may want to provide `mapProfileToUser` to map the profile data to the user object. So, you can populate additional fields from the provider's profile.

Example: Mapping Profile to User For `firstName` and `lastName`

```ts title="auth.ts"
import { betterAuth } from "better-auth";

````
export const auth = betterAuth({
  socialProviders: {
    github: {
      clientId: "YOUR_GITHUB_CLIENT_ID",
      clientSecret: "YOUR_GITHUB_CLIENT_SECRET",
      mapProfileToUser: (profile) => {
        return {
          firstName: profile.name.split(" ")[0],
          lastName: profile.name.split(" ")[1],
        };
      },
    },
    google: {
      clientId: "YOUR_GOOGLE_CLIENT_ID",
      clientSecret: "YOUR_GOOGLE_CLIENT_SECRET",
      mapProfileToUser: (profile) => {
        return {
          firstName: profile.given_name,
          lastName: profile.family_name,
        };
      },
    },
  },
});

### ID Generation

Better Auth by default will generate unique IDs for users, sessions, and other entities. If you want to customize how IDs are generated, you can configure this in the
`advanced.database.generateId` option in your auth config.

You can also disable ID generation by setting the `advanced.database.generateId` option to `false`. This will assume your database will generate the ID automatically.

Example: Automatic Database IDs

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { db } from "./db";

export const auth = betterAuth({
  database: db,
  advanced: {
    database: {
      generateId: false,
    },
  },
});
```

Example: Using a Custom ID Generator

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { db } from "./db";

export const auth = betterAuth({
  database: db,
  advanced: {
    database: {
      generateId: () => crypto.randomUUID(),
    },
  },
});
```

### Numeric IDs

If you prefer auto-incrementing numeric IDs, you can set the `advanced.database.useNumberId` option to `true`.
Doing this will disable Better-Auth from generating IDs for any table, and will assume your
database will generate the numeric ID automatically.

When enabled, the Better-Auth CLI will generate or migrate the schema with the `id` field as a numeric type for your database
with auto-incrementing attributes associated with it.

```ts
import { betterAuth } from "better-auth";
import { db } from "./db";

export const auth = betterAuth({
  database: db,
  advanced: {
    database: {
      useNumberId: true,
    },
  },
});
```

<Callout type="info">
  Better-Auth will continue to infer the type of the `id` field as a `string` for the database, but will
  automatically convert it to a numeric type when fetching or inserting data from the database.

  It's likely when grabbing `id` values returned from Better-Auth that you'll receive a string version of a number,
  this is normal. It's also expected that all id values passed to Better-Auth (eg via an endpoint body) is expected to be a string.
</Callout>

### Database Hooks

Database hooks allow you to define custom logic that can be executed during the lifecycle of core database operations in Better Auth. You can create hooks for the following
models: **user**, **session**, and **account**.

<Callout type="warn">
  Additional fields are supported, however full type inference for these fields isn't yet supported.
  Improved type support is planned.
</Callout>

There are two types of hooks you can define:

#### 1. Before Hook

* **Purpose**: This hook is called before the respective entity (user, session, or account) is created or updated.
* **Behavior**: If the hook returns `false`, the operation will be aborted. And If it returns a data object, it'll replace the original payload.

#### 2. After Hook

* **Purpose**: This hook is called after the respective entity is created or updated.
* **Behavior**: You can perform additional actions or modifications after the entity has been successfully created or updated.

Example Usage

```typescript title="auth.ts"
import { betterAuth } from "better-auth";
````

```typescript
export const auth = betterAuth({
  databaseHooks: {
    user: {
      create: {
        before: async (user, ctx) => {
          // Modify the user object before it is created
          return {
            data: {
              // Ensure to return Better-Auth named fields, not the original field names in your database.
              ...user,
              firstName: user.name.split(" ")[0],
              lastName: user.name.split(" ")[1],
            },
          };
        },
        after: async (user) => {
          //perform additional actions, like creating a stripe customer
        },
      },
    },
  },
});
```

#### Throwing Errors

If you want to stop the database hook from proceeding, you can throw errors using the `APIError` class imported from `better-auth/api`.

```typescript title="auth.ts"
import { betterAuth } from "better-auth";
import { APIError } from "better-auth/api";

export const auth = betterAuth({
  databaseHooks: {
    user: {
      create: {
        before: async (user, ctx) => {
          if (user.isAgreedToTerms === false) {
            // Your special condition.
            // Send the API error.
            throw new APIError("BAD_REQUEST", {
              message: "User must agree to the TOS before signing up.",
            });
          }
          return {
            data: user,
          };
        },
      },
    },
  },
});
```

#### Using the Context Object

The context object (`ctx`), passed as the second argument to the hook, contains useful information. For `update` hooks, this includes the current `session`, which you can use to access the logged-in user's details.

```typescript title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
  databaseHooks: {
    user: {
      update: {
        before: async (data, ctx) => {
          // You can access the session from the context object.
          if (ctx.context.session) {
            console.log("User update initiated by:", ctx.context.session.userId);
          }
          return { data };
        },
      },
    },
  },
});
```

Much like standard hooks, database hooks also provide a `ctx` object that offers a variety of useful properties. Learn more in the [Hooks Documentation](/docs/concepts/hooks#ctx).

## Plugins Schema

Plugins can define their own tables in the database to store additional data. They can also add columns to the core tables to store additional data. For example, the two factor authentication plugin adds the following columns to the `user` table:

* `twoFactorEnabled`: Whether two factor authentication is enabled for the user.
* `twoFactorSecret`: The secret key used to generate TOTP codes.
* `twoFactorBackupCodes`: Encrypted backup codes for account recovery.

To add new tables and columns to your database, you have two options:

`CLI`: Use the migrate or generate command. These commands will scan your database and guide you through adding any missing tables or columns.
`Manual Method`: Follow the instructions in the plugin documentation to manually add tables and columns.

Both methods ensure your database schema stays up to date with your plugins' requirements.

# concepts: Email

URL: /docs/concepts/email
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/concepts/email.mdx

Learn how to use email with Better Auth.

***

title: Email
description: Learn how to use email with Better Auth.

Email is a key part of Better Auth, required for all users regardless of their authentication method. Better Auth provides email and password authentication out of the box, and a lot of utilities to help you manage email verification, password reset, and more.

## Email Verification

Email verification is a security feature that ensures users provide a valid email address. It helps prevent spam and abuse by confirming that the email address belongs to the user. In this guide, you'll get a walk through of how to implement token based email verification in your app.
To use otp based email verification, check out the [OTP Verification](/docs/plugins/email-otp) guide.

### Adding Email Verification to Your App

To enable email verification, you need to pass a function that sends a verification email with a link.

* **sendVerificationEmail**: This function is triggered when email verification starts. It accepts a data object with the following properties:
  * `user`: The user object containing the email address.
  * `url`: The verification URL the user must click to verify their email.
  * `token`: The verification token used to complete the email verification to be used when implementing a custom verification URL.

and a `request` object as the second parameter.

```ts title="auth.ts"
import { betterAuth } from 'better-auth';
import { sendEmail } from './email'; // your email sending function

export const auth = betterAuth({
    emailVerification: {
        sendVerificationEmail: async ({ user, url, token }, request) => {
            await sendEmail({
                to: user.email,
                subject: 'Verify your email address',
                text: `Click the link to verify your email: ${url}`
            })
        }
    }
})
```

### Triggering Email Verification

You can initiate email verification in several ways:

#### 1. During Sign-up

To automatically send a verification email at signup, set `emailVerification.sendOnSignUp` to `true`.

```ts title="auth.ts"
import { betterAuth } from 'better-auth';

export const auth = betterAuth({
    emailVerification: {
        sendOnSignUp: true
    }
})
```

This sends a verification email when a user signs up. For social logins, email verification status is read from the SSO.

<Callout>
  With `sendOnSignUp` enabled, when the user logs in with an SSO that does not claim the email as verified, Better Auth will dispatch a verification email, but the verification is not required to login even when `requireEmailVerification` is enabled.
</Callout>

#### 2. Require Email Verification

If you enable require email verification, users must verify their email before they can log in. And every time a user tries to sign in, `sendVerificationEmail` is called.

<Callout>
  This only works if you have `sendVerificationEmail` implemented and if the user is trying to sign in with email and password.
</Callout>

```ts title="auth.ts"
export const auth = betterAuth({
    emailAndPassword: {
        requireEmailVerification: true
    }
})
```

if a user tries to sign in without verifying their email, you can handle the error and show a message to the user.

```ts title="auth-client.ts"
await authClient.signIn.email({
    email: "email@example.com",
    password: "password"
}, {
    onError: (ctx) => {
        // Handle the error
        if(ctx.error.status === 403) {
            alert("Please verify your email address")
        }
        //you can also show the original error message
        alert(ctx.error.message)
    }
})
```

#### 3. Manually

You can also manually trigger email verification by calling `sendVerificationEmail`.

```ts
await authClient.sendVerificationEmail({
    email: "user@email.com",
    callbackURL: "/" // The redirect URL after verification
})
```

### Verifying the Email

If the user clicks the provided verification URL, their email is automatically verified, and they are redirected to the `callbackURL`.

For manual verification, you can send the user a custom link with the `token` and call the `verifyEmail` function.

```ts
await authClient.verifyEmail({
    query: {
        token: "" // Pass the token here
    }
})
```

### Auto Sign In After Verification

To sign in the user automatically after they successfully verify their email, set the `autoSignInAfterVerification` option to `true`:

```ts
const auth = betterAuth({
    //...your other options
    emailVerification: {
        autoSignInAfterVerification: true
    }
})
```

### Callback after successful email verification

You can run custom code immediately after a user verifies their email using the `afterEmailVerification` callback. This is useful for any side-effects you want to trigger, like granting access to special features or logging the event.

The `afterEmailVerification` function runs automatically when a user's email is confirmed, receiving the `user` object and `request` details so you can perform actions for that specific user.

Here's how you can set it up:

```ts title="auth.ts"
import { betterAuth } from 'better-auth';

~~~~
export const auth = betterAuth({
    emailVerification: {
        async afterEmailVerification(user, request) {
            // Your custom logic here, e.g., grant access to premium features
            console.log(`${user.email} has been successfully verified!`);
        }
    }
})
```

## Password Reset Email

Password reset allows users to reset their password if they forget it. Better Auth provides a simple way to implement password reset functionality.

You can enable password reset by passing a function that sends a password reset email with a link.

```ts title="auth.ts"
import { betterAuth } from 'better-auth';
import { sendEmail } from './email'; // your email sending function

export const auth = betterAuth({
    emailAndPassword: {
        enabled: true,
        sendResetPassword: async ({ user, url, token }, request) => {
            await sendEmail({
                to: user.email,
                subject: 'Reset your password',
                text: `Click the link to reset your password: ${url}`
            })
        }
    }
})
```

Check out the [Email and Password](/docs/authentication/email-password#forget-password) guide for more details on how to implement password reset in your app.
Also you can check out the [Otp verification](/docs/plugins/email-otp#reset-password) guide for how to implement password reset with OTP in your app.

# concepts: Hooks
URL: /docs/concepts/hooks
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/concepts/hooks.mdx

Better Auth Hooks let you customize BetterAuth's behavior

***

title: Hooks
description: Better Auth Hooks let you customize BetterAuth's behavior
----

Hooks in Better Auth let you "hook into" the lifecycle and execute custom logic. They provide a way to customize Better Auth's behavior without writing a full plugin.

<Callout>
  We highly recommend using hooks if you need to make custom adjustments to an endpoint rather than making another endpoint outside of Better Auth.
</Callout>

## Before Hooks

Before hooks run *before* an endpoint is executed. Use them to modify requests, pre validate data, or return early.

### Example: Enforce Email Domain Restriction

This hook ensures that users can only sign up if their email ends with `@example.com`:

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { createAuthMiddleware, APIError } from "better-auth/api";

export const auth = betterAuth({
    hooks: {
        before: createAuthMiddleware(async (ctx) => {
            if (ctx.path !== "/sign-up/email") {
                return;
            }
            if (!ctx.body?.email.endsWith("@example.com")) {
                throw new APIError("BAD_REQUEST", {
                    message: "Email must end with @example.com",
                });
            }
        }),
    },
});
```

### Example: Modify Request Context

To adjust the request context before proceeding:

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { createAuthMiddleware } from "better-auth/api";

export const auth = betterAuth({
    hooks: {
        before: createAuthMiddleware(async (ctx) => {
            if (ctx.path === "/sign-up/email") {
                return {
                    context: {
                        ...ctx,
                        body: {
                            ...ctx.body,
                            name: "John Doe",
                        },
                    }
                };
            }
        }),
    },
});
```

## After Hooks

After hooks run *after* an endpoint is executed. Use them to modify responses.

### Example: Send a notification to your channel when a new user is registered

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { createAuthMiddleware } from "better-auth/api";
import { sendMessage } from "@/lib/notification"
~~~~

````
export const auth = betterAuth({
    hooks: {
        after: createAuthMiddleware(async (ctx) => {
            if(ctx.path.startsWith("/sign-up")){
                const newSession = ctx.context.newSession;
                if(newSession){
                    sendMessage({
                        type: "user-register",
                        name: newSession.user.name,
                    })
                }
            }
        }),
    },
});

## Ctx

When you call `createAuthMiddleware` a `ctx` object is passed that provides a lot of useful properties. Including:

* **Path:** `ctx.path` to get the current endpoint path.
* **Body:** `ctx.body` for parsed request body (available for POST requests).
* **Headers:** `ctx.headers` to access request headers.
* **Request:** `ctx.request` to access the request object (may not exist in server-only endpoints).
* **Query Parameters:** `ctx.query` to access query parameters.
* **Context**: `ctx.context` auth related context, useful for accessing new session, auth cookies configuration, password hashing, config...

and more.

### Request Response

This utilities allows you to get request information and to send response from a hook.

#### JSON Responses

Use `ctx.json` to send JSON responses:

```ts
const hook = createAuthMiddleware(async (ctx) => {
    return ctx.json({
        message: "Hello World",
    });
});
```

#### Redirects

Use `ctx.redirect` to redirect users:

```ts
import { createAuthMiddleware } from "better-auth/api";

const hook = createAuthMiddleware(async (ctx) => {
    throw ctx.redirect("/sign-up/name");
});
```

#### Cookies

* Set cookies: `ctx.setCookies` or `ctx.setSignedCookie`.
* Get cookies: `ctx.getCookies` or `ctx.getSignedCookie`.

Example:

```ts
import { createAuthMiddleware } from "better-auth/api";

const hook = createAuthMiddleware(async (ctx) => {
    ctx.setCookies("my-cookie", "value");
    await ctx.setSignedCookie("my-signed-cookie", "value", ctx.context.secret, {
        maxAge: 1000,
    });

    const cookie = ctx.getCookies("my-cookie");
    const signedCookie = await ctx.getSignedCookie("my-signed-cookie");
});
```

#### Errors

Throw errors with `APIError` for a specific status code and message:

```ts
import { createAuthMiddleware, APIError } from "better-auth/api";

const hook = createAuthMiddleware(async (ctx) => {
    throw new APIError("BAD_REQUEST", {
        message: "Invalid request",
    });
});
```

### Context

The `ctx` object contains another `context` object inside that's meant to hold contexts related to auth. Including a newly created session on after hook, cookies configuration, password hasher and so on.

#### New Session

The newly created session after an endpoint is run. This only exist in after hook.

```ts title="auth.ts"
createAuthMiddleware(async (ctx) => {
    const newSession = ctx.context.newSession
});
```

#### Returned

The returned value from the hook is passed to the next hook in the chain.

```ts title="auth.ts"
createAuthMiddleware(async (ctx) => {
    const returned = ctx.context.returned; //this could be a successful response or an APIError
});
```

#### Response Headers

The response headers added by endpoints and hooks that run before this hook.

```ts title="auth.ts"
createAuthMiddleware(async (ctx) => {
    const responseHeaders = ctx.context.responseHeaders;
});
```

#### Predefined Auth Cookies

Access BetterAuthâ€™s predefined cookie properties:
~~~

```ts title="auth.ts"
createAuthMiddleware(async (ctx) => {
    const cookieName = ctx.context.authCookies.sessionToken.name;
});
```

Secret

You can access the `secret` for your auth instance on `ctx.context.secret`

Password

The password object provider `hash` and `verify`

* `ctx.context.password.hash`: let's you hash a given password.
* `ctx.context.password.verify`: let's you verify given `password` and a `hash`.

Adapter

Adapter exposes the adapter methods used by Better Auth. Including `findOne`, `findMany`, `create`, `delete`, `update` and `updateMany`. You generally should use your actually `db` instance from your orm rather than this adapter.

Internal Adapter

These are calls to your db that perform specific actions. `createUser`, `createSession`, `updateSession`...

This may be useful to use instead of using your db directly to get access to `databaseHooks`, proper `secondaryStorage` support and so on. If you're make a query similar to what exist in this internal adapter actions it's worth a look.

generateId

You can use `ctx.context.generateId` to generate Id for various reasons.

Reusable Hooks

If you need to reuse a hook across multiple endpoints, consider creating a plugin. Learn more in the [Plugins Documentation](/docs/concepts/plugins).

concepts: OAuth

URL: /docs/concepts/oauth
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/concepts/oauth.mdx

How Better Auth handles OAuth

***

title: OAuth
description: How Better Auth handles OAuth

Better Auth comes with built-in support for OAuth 2.0 and OpenID Connect. This allows you to authenticate users via popular OAuth providers like Google, Facebook, GitHub, and more.

If your desired provider isn't directly supported, you can use the [Generic OAuth Plugin](/docs/plugins/generic-oauth) for custom integrations.

Configuring Social Providers

To enable a social provider, you need to provide `clientId` and `clientSecret` for the provider.

Here's an example of how to configure Google as a provider:

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
  // Other configurations...
  socialProviders: {
    google: {
      clientId: "YOUR_GOOGLE_CLIENT_ID",
      clientSecret: "YOUR_GOOGLE_CLIENT_SECRET",
    },
  },
});
```

Usage

Sign In

To sign in with a social provider, you can use the `signIn.social` function with the `authClient` or `auth.api` for server-side usage.

```ts
// client-side usage
await authClient.signIn.social({
  provider: "google", // or any other provider id
})
```

```ts
// server-side usage
await auth.api.signInSocial({
  body: {
    provider: "google", // or any other provider id
  },
});
```

Link account

To link an account to a social provider, you can use the `linkAccount` function with the `authClient` or `auth.api` for server-side usage.

```ts
await authClient.linkSocial({
  provider: "google", // or any other provider id
})
```

server-side usage:

```ts
await auth.api.linkSocialAccount({
  body: {
    provider: "google", // or any other provider id
  },
  headers: // pass headers with authenticated token
});
```

Get Access Token

To get the access token for a social provider, you can use the `getAccessToken` function with the `authClient` or `auth.api` for server-side usage. When you use this endpoint, if the access token is expired, it will be refreshed.

```ts
const { accessToken } = await authClient.getAccessToken({
  providerId: "google", // or any other provider id
  accountId: "accountId", // optional, if you want to get the access token for a specific account
})
```

server-side usage:

```ts
await auth.api.getAccessToken({
  body: {
    providerId: "google", // or any other provider id
    accountId: "accountId", // optional, if you want to get the access token for a specific account
    userId: "userId", // optional, if you don't provide headers with authenticated token
  },
  headers: // pass headers with authenticated token
});
```

Get Account Info Provided by the provider

To get provider specific account info you can use the `accountInfo` function with the `authClient` or `auth.api` for server-side usage.

```ts
const info = await authClient.accountInfo({
  accountId: "accountId", // here you pass in the provider given account id, the provider is automatically detected from the account id
})
```

server-side usage:

```ts
await auth.api.accountInfo({
  body: { accountId: "accountId" },
  headers: // pass headers with authenticated token
});
```

Requesting Additional Scopes

Sometimes your application may need additional OAuth scopes after the user has already signed up (e.g., for accessing GitHub repositories or Google Drive). Users may not want to grant extensive permissions initially, preferring to start with minimal permissions and grant additional access as needed.

You can request additional scopes by using the `linkSocial` method with the same provider. This will trigger a new OAuth flow that requests the additional scopes while maintaining the existing account connection.

```ts
const requestAdditionalScopes = async () => {
    await authClient.linkSocial({
        provider: "google",
        scopes: ["https://www.googleapis.com/auth/drive.file"],
    });
};
```

<Callout>
  Make sure you're running Better Auth version 1.2.7 or later. Earlier versions (like 1.2.2) may show a "Social account already linked" error when trying to link with an existing provider for additional scopes.
</Callout>

Provider Options

scope

The scope of the access request. For example, `email` or `profile`.

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
  // Other configurations...
  socialProviders: {
    google: {
      clientId: "YOUR_GOOGLE_CLIENT_ID",
      clientSecret: "YOUR_GOOGLE_CLIENT_SECRET",
      scope: ["email", "profile"],
    },
  },
});
```

redirectURI

Custom redirect URI for the provider. By default, it uses `/api/auth/callback/${providerName}`

```ts title="auth.ts"

export const auth = betterAuth({
  // Other configurations...
  socialProviders: {
    google: {
      clientId: "YOUR_GOOGLE_CLIENT_ID",
      clientSecret: "YOUR_GOOGLE_CLIENT_SECRET",
      redirectURI: "https://your-app.com/auth/callback",
    },
  },
});
```

disableSignUp

Disables sign-up for new users.

disableIdTokenSignIn

Disables the use of the ID token for sign-in. By default, it's enabled for some providers like Google and Apple.

verifyIdToken

A custom function to verify the ID token.

overrideUserInfoOnSignIn

A boolean value that determines whether to override the user information in the database when signing in. By default, it is set to `false`, meaning that the user information will not be overridden during sign-in. If you want to update the user information every time they sign in, set this to `true`.

mapProfileToUser

A custom function to map the user profile returned from the provider to the user object in your database.

Useful, if you have additional fields in your user object you want to populate from the provider's profile. Or if you want to change how by default the user object is mapped.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
```

~~~
export const auth = betterAuth({
  // Other configurations...
  socialProviders: {
    google: {
      clientId: "YOUR_GOOGLE_CLIENT_ID",
      clientSecret: "YOUR_GOOGLE_CLIENT_SECRET",
      mapProfileToUser: (profile) => {
        return {
          firstName: profile.given_name,
          lastName: profile.family_name,
        };
      },
    },
  },
});

### refreshAccessToken

A custom function to refresh the token. This feature is only supported for built-in social providers (Google, Facebook, GitHub, etc.) and is not currently supported for custom
OAuth providers configured through the Generic OAuth Plugin. For built-in providers, you can provide a custom function to refresh the token if needed.

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
  // Other configurations...
  socialProviders: {
    google: {
      clientId: "YOUR_GOOGLE_CLIENT_ID",
      clientSecret: "YOUR_GOOGLE_CLIENT_SECRET",
      refreshAccessToken: async (token) => {
        return {
          accessToken: "new-access-token",
          refreshToken: "new-refresh-token",
        };
      },
    },
  },
});
```

### clientKey

The client key of your application. This is used by TikTok Social Provider instead of `clientId`.

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
  // Other configurations...
  socialProviders: {
    tiktok: {
      clientKey: "YOUR_TIKTOK_CLIENT_KEY",
      clientSecret: "YOUR_TIKTOK_CLIENT_SECRET",
    },
  },
});
```

### getUserInfo

A custom function to get user info from the provider. This allows you to override the default user info retrieval process.

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
  // Other configurations...
  socialProviders: {
    google: {
      clientId: "YOUR_GOOGLE_CLIENT_ID",
      clientSecret: "YOUR_GOOGLE_CLIENT_SECRET",
      getUserInfo: async (token) => {
        // Custom implementation to get user info
        const response = await fetch("https://www.googleapis.com/oauth2/v2/userinfo", {
          headers: {
            Authorization: `Bearer ${token.accessToken}`,
          },
        });
        const profile = await response.json();
        return {
          user: {
            id: profile.id,
            name: profile.name,
            email: profile.email,
            image: profile.picture,
            emailVerified: profile.verified_email,
          },
          data: profile,
        };
      },
    },
  },
});
```

### disableImplicitSignUp

Disables implicit sign up for new users. When set to true for the provider, sign-in needs to be called with `requestSignUp` as true to create new users.

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
  // Other configurations...
  socialProviders: {
    google: {
      clientId: "YOUR_GOOGLE_CLIENT_ID",
      clientSecret: "YOUR_GOOGLE_CLIENT_SECRET",
      disableImplicitSignUp: true,
    },
  },
});
```

### prompt

The prompt to use for the authorization code request. This controls the authentication flow behavior.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
````

```ts
export const auth = betterAuth({
  // Other configurations...
  socialProviders: {
    google: {
      clientId: "YOUR_GOOGLE_CLIENT_ID",
      clientSecret: "YOUR_GOOGLE_CLIENT_SECRET",
      prompt: "select_account", // or "consent", "login", "none", "select_account+consent"
    },
  },
});
```

### responseMode

The response mode to use for the authorization code request. This determines how the authorization response is returned.

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
  // Other configurations...
  socialProviders: {
    google: {
      clientId: "YOUR_GOOGLE_CLIENT_ID",
      clientSecret: "YOUR_GOOGLE_CLIENT_SECRET",
      responseMode: "query", // or "form_post"
    },
  },
});
```

### disableDefaultScope

Removes the default scopes of the provider. By default, providers include certain scopes like `email` and `profile`. Set this to `true` to remove these default scopes and use only the scopes you specify.

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
  // Other configurations...
  socialProviders: {
    google: {
      clientId: "YOUR_GOOGLE_CLIENT_ID",
      clientSecret: "YOUR_GOOGLE_CLIENT_SECRET",
      disableDefaultScope: true,
      scope: ["https://www.googleapis.com/auth/userinfo.email"], // Only this scope will be used
    },
  },
});
```

### Other Provider Configurations

Each provider may have additional options, check the specific provider documentation for more details.

# concepts: Plugins
URL: /docs/concepts/plugins
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/concepts/plugins.mdx

Learn how to use plugins with Better Auth.

***

title: Plugins
description: Learn how to use plugins with Better Auth.

Plugins are a key part of Better Auth, they let you extend the base functionalities. You can use them to add new authentication methods, features, or customize behaviors.

Better Auth comes with many built-in plugins ready to use. Check the plugins section for details. You can also create your own plugins.

## Using a Plugin

Plugins can be a server-side plugin, a client-side plugin, or both.

To add a plugin on the server, include it in the `plugins` array in your auth configuration. The plugin will initialize with the provided options.

```ts title="server.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
    plugins: [
        // Add your plugins here
    ]
});
```

Client plugins are added when creating the client. Most plugin require both server and client plugins to work correctly.
The Better Auth auth client on the frontend uses the `createAuthClient` function provided by `better-auth/client`.

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client";

const authClient =  createAuthClient({
    plugins: [
        // Add your client plugins here
    ]
});
```

We recommend keeping the auth-client and your normal auth instance in separate files.

```
<Files>
  <Folder name="auth" defaultOpen>
    <File name="server.ts" />

    <File name="auth-client.ts" />
  </Folder>
</Files>
```

## Creating a Plugin

To get started, you'll need a server plugin.
Server plugins are the backbone of all plugins, and client plugins are there to provide an interface with frontend APIs to easily work with your server plugins.

<Callout type="info">
  If your server plugins has endpoints that needs to be called from the client, you'll also need to create a client plugin.
</Callout>

### What can a plugin do?

* Create custom `endpoint`s to perform any action you want.
* Extend database tables with custom `schemas`.
* Use a `middleware` to target a group of routes using it's route matcher, and run only when those routes are called through a request.
* Use `hooks` to target a specific route or request. And if you want to run the hook even if the endpoint is called directly.
* Use `onRequest` or `onResponse` if you want to do something that affects all requests or responses.
* Create custom `rate-limit` rule.

## Create a Server plugin

To create a server plugin you need to pass an object that satisfies the `BetterAuthPlugin` interface.

The only required property is `id`, which is a unique identifier for the plugin.
Both server and client plugins can use the same `id`.

```ts title="plugin.ts"
import type { BetterAuthPlugin } from "better-auth";

export const myPlugin = ()=>{
    return {
        id: "my-plugin",
    } satisfies BetterAuthPlugin
}
```

<Callout>
  You don't have to make the plugin a function, but it's recommended to do so. This way you can pass options to the plugin and it's consistent with the built-in plugins.
</Callout>

### Endpoints

To add endpoints to the server, you can pass `endpoints` which requires an object with the key being any `string` and the value being an `AuthEndpoint`.

To create an Auth Endpoint you'll need to import `createAuthEndpoint` from `better-auth`.

Better Auth uses wraps around another library called <Link href="https://github.com/bekacru/better-call"> Better Call </Link> to create endpoints. Better call is a simple ts web framework made by the same team behind Better Auth.

```ts title="plugin.ts"
import { createAuthEndpoint } from "better-auth/api";

const myPlugin = ()=> {
    return {
        id: "my-plugin",
        endpoints: {
            getHelloWorld: createAuthEndpoint("/my-plugin/hello-world", {
                method: "GET",
            }, async(ctx) => {
                return ctx.json({
                    message: "Hello World"
                })
            })
        }
    } satisfies BetterAuthPlugin
}
```

Create Auth endpoints wraps around `createEndpoint` from Better Call. Inside the `ctx` object, it'll provide another object called `context` that give you access better-auth specific contexts including `options`, `db`, `baseURL` and more.

Context Object

* `appName`: The name of the application. Defaults to "Better Auth".
* `options`: The options passed to the Better Auth instance.
* `tables`: Core tables definition. It is an object which has the table name as the key and the schema definition as the value.
* `baseURL`: the baseURL of the auth server. This includes the path. For example, if the server is running on `http://localhost:3000`, the baseURL will be `http://localhost:3000/api/auth` by default unless changed by the user.
* `session`: The session configuration. Includes `updateAge` and `expiresIn` values.
* `secret`: The secret key used for various purposes. This is defined by the user.
* `authCookie`: The default cookie configuration for core auth cookies.
* `logger`: The logger instance used by Better Auth.
* `db`: The Kysely instance used by Better Auth to interact with the database.
* `adapter`: This is the same as db but it give you `orm` like functions to interact with the database. (we recommend using this over `db` unless you need raw sql queries or for performance reasons)
* `internalAdapter`: These are internal db calls that are used by Better Auth. For example, you can use these calls to create a session instead of using `adapter` directly. `internalAdapter.createSession(userId)`
* `createAuthCookie`: This is a helper function that let's you get a cookie `name` and `options` for either to `set` or `get` cookies. It implements things like `__secure` prefix and `__host` prefix for cookies based on

For other properties, you can check the <Link href="https://github.com/bekacru/better-call">Better Call</Link> documentation and the <Link href="https://github.com/better-auth/better-auth/blob/main/packages/better-auth/src/init.ts">source code </Link>.

Rules for Endpoints

* Makes sure you use kebab-case for the endpoint path
* Make sure to only use `POST` or `GET` methods for the endpoints.
* Any function that modifies a data should be a `POST` method.
* Any function that fetches data should be a `GET` method.
* Make sure to use the `createAuthEndpoint` function to create API endpoints.
* Make sure your paths are unique to avoid conflicts with other plugins. If you're using a common path, add the plugin name as a prefix to the path. (`/my-plugin/hello-world` instead of `/hello-world`.)

### Schema

You can define a database schema for your plugin by passing a `schema` object. The schema object should have the table name as the key and the schema definition as the value.

```ts title="plugin.ts"
import { BetterAuthPlugin } from "better-auth/plugins";

const myPlugin = ()=> {
    return {
        id: "my-plugin",
        schema: {
            myTable: {
                fields: {
                    name: {
                        type: "string"
                    }
                },
                modelName: "myTable" // optional if you want to use a different name than the key
            }
        }
    } satisfies BetterAuthPlugin
}
```

Fields

By default better-auth will create an `id` field for each table. You can add additional fields to the table by adding them to the `fields` object.

The key is the column name and the value is the column definition. The column definition can have the following properties:

`type`: The type of the field. It can be `string`, `number`, `boolean`, `date`.

`required`: if the field should be required on a new record. (default: `false`)

`unique`: if the field should be unique. (default: `false`)

`reference`: if the field is a reference to another table. (default: `null`) It takes an object with the following properties:

* `model`: The table name to reference.
* `field`: The field name to reference.
* `onDelete`: The action to take when the referenced record is deleted. (default: `null`)

Other Schema Properties

`disableMigration`: if the table should not be migrated. (default: `false`)

```ts title="plugin.ts"
const myPlugin = (opts: PluginOptions)=>{
    return {
        id: "my-plugin",
        schema: {
            rateLimit: {
                fields: {
                    key: {
                        type: "string",
                    },
                },
                disableMigration: opts.storage.provider !== "database", // [!code highlight]
            },
        },
    } satisfies BetterAuthPlugin
}
```

if you add additional fields to a `user` or `session` table, the types will be inferred automatically on `getSession` and `signUpEmail` calls.

```ts title="plugin.ts"

const myPlugin = ()=>{
    return {
        id: "my-plugin",
        schema: {
            user: {
                fields: {
                    age: {
                        type: "number",
                    },
                },
            },
        },
    } satisfies BetterAuthPlugin
}
```

This will add an `age` field to the `user` table and all `user` returning endpoints will include the `age` field and it'll be inferred properly by typescript.

<Callout type="warn">
  Don't store sensitive information in `user` or `session` table. Crate a new table if you need to store sensitive information.
</Callout>

### Hooks

Hooks are used to run code before or after an action is performed, either from a client or directly on the server. You can add hooks to the server by passing a `hooks` object, which should contain `before` and `after` properties.

```ts title="plugin.ts"
import {  createAuthMiddleware } from "better-auth/plugins";

const myPlugin = ()=>{
    return {
        id: "my-plugin",
        hooks: {
            before: [{
                    matcher: (context)=>{
                        return context.headers.get("x-my-header") === "my-value"
                    },
                    handler: createAuthMiddleware(async (ctx)=>{
                        //do something before the request
                        return  {
                            context: ctx // if you want to modify the context
                        }
                    })
                }],
            after: [{
                matcher: (context)=>{
                    return context.path === "/sign-up/email"
                },
                handler: createAuthMiddleware(async (ctx)=>{
                    return ctx.json({
                        message: "Hello World"
                    }) // if you want to modify the response
                })
            }]
        }
    } satisfies BetterAuthPlugin
}
```

### Middleware

You can add middleware to the server by passing a `middlewares` array. This array should contain middleware objects, each with a `path` and a `middleware` property. Unlike hooks, middleware only runs on `api` requests from a client. If the endpoint is invoked directly, the middleware will not run.

The `path` can be either a string or a path matcher, using the same path-matching system as `better-call`.

If you throw an `APIError` from the middleware or returned a `Response` object, the request will be stopped and the response will be sent to the client.

```ts title="plugin.ts"
const myPlugin = ()=>{
    return {
        id: "my-plugin",
        middlewares: [
            {
                path: "/my-plugin/hello-world",
                middleware: createAuthMiddleware(async(ctx)=>{
                    //do something
                })
            }
        ]
    } satisfies BetterAuthPlugin
}
```

### On Request & On Response

Additional to middlewares, you can also hook into right before a request is made and right after a response is returned. This is mostly useful if you want to do something that affects all requests or responses.

#### On Request

The `onRequest` function is called right before the request is made. It takes two parameters: the `request` and the `context` object.

Hereâ€™s how it works:

* **Continue as Normal**: If you don't return anything, the request will proceed as usual.
* **Interrupt the Request**: To stop the request and send a response, return an object with a `response` property that contains a `Response` object.
* **Modify the Request**: You can also return a modified `request` object to change the request before it's sent.

```ts title="plugin.ts"
const myPlugin = ()=> {
    return  {
        id: "my-plugin",
        onRequest: async (request, context) => {
            //do something
        },
    } satisfies BetterAuthPlugin
}
```

#### On Response

The `onResponse` function is executed immediately after a response is returned. It takes two parameters: the `response` and the `context` object.

Hereâ€™s how to use it:

* **Modify the Response**: You can return a modified response object to change the response before it is sent to the client.
* **Continue Normally**: If you don't return anything, the response will be sent as is.

```ts title="plugin.ts"
const myPlugin = ()=>{
    return {
        id: "my-plugin",
        onResponse: async (response, context) => {
            //do something
        },
    } satisfies BetterAuthPlugin
}
```

### Rate Limit

You can define custom rate limit rules for your plugin by passing a `rateLimit` array. The rate limit array should contain an array of rate limit objects.

```ts title="plugin.ts"
const myPlugin = ()=>{
    return {
        id: "my-plugin",
        rateLimit: [
            {
                pathMatcher: (path)=>{
                    return path === "/my-plugin/hello-world"
                },
                limit: 10,
                window: 60,
            }
        ]
    } satisfies BetterAuthPlugin
}
```

### Server-plugin helper functions

Some additional helper functions for creating server plugins.

#### `getSessionFromCtx`

Allows you to get the client's session data by passing the auth middleware's `context`.

```ts title="plugin.ts"
import {  createAuthMiddleware } from "better-auth/plugins";
import { getSessionFromCtx } from "better-auth/api";

const myPlugin = {
    id: "my-plugin",
    hooks: {
        before: [{
                matcher: (context)=>{
                    return context.headers.get("x-my-header") === "my-value"
                },
                handler: createAuthMiddleware(async (ctx) => {
                    const session = await getSessionFromCtx(ctx);
                    //do something with the client's session.

                    return  {
                        context: ctx
                    }
                })
            }],
    }
} satisfies BetterAuthPlugin
```

#### `sessionMiddleware`

A middleware that checks if the client has a valid session. If the client has a valid session, it'll add the session data to the context object.

```ts title="plugin.ts"
import { createAuthMiddleware } from "better-auth/plugins";
import { sessionMiddleware } from "better-auth/api";

const myPlugin = ()=>{
    return {
        id: "my-plugin",
        endpoints: {
            getHelloWorld: createAuthEndpoint("/my-plugin/hello-world", {
                method: "GET",
                use: [sessionMiddleware], // [!code highlight]
            }, async(ctx) => {
                const session = ctx.context.session;
                return ctx.json({
                    message: "Hello World"
                })
            })
        }
    } satisfies BetterAuthPlugin
}
```

## Creating a client plugin

If your endpoints needs to be called from the client, you'll need to also create a client plugin. Better Auth clients can infer the endpoints from the server plugins. You can also add additional client side logic.

```ts title="client-plugin.ts"
import type { BetterAuthClientPlugin } from "better-auth";

export const myPluginClient = ()=>{
    return {
        id: "my-plugin",
    } satisfies BetterAuthClientPlugin
}
```

### Endpoint Interface

Endpoints are inferred from the server plugin by adding a `$InferServerPlugin` key to the client plugin.

The client infers the `path` as an object and converts kebab-case to camelCase. For example, `/my-plugin/hello-world` becomes `myPlugin.helloWorld`.

```ts title="client-plugin.ts"
import type { BetterAuthClientPlugin } from "better-auth/client";
import type { myPlugin } from "./plugin";

const myPluginClient = ()=> {
    return  {
        id: "my-plugin",
        $InferServerPlugin: {} as ReturnType<typeof myPlugin>,
    } satisfies BetterAuthClientPlugin
}
```

### Get actions

If you need to add additional methods or what not to the client you can use the `getActions` function. This function is called with the `fetch` function from the client.

Better Auth uses <Link href="https://better-fetch.vercel.app"> Better fetch </Link> to make requests. Better fetch is a simple fetch wrapper made by the same author of Better Auth.

```ts title="client-plugin.ts"
import type { BetterAuthClientPlugin } from "better-auth/client";
import type { myPlugin } from "./plugin";
import type { BetterFetchOption } from "@better-fetch/fetch";

const myPluginClient = {
    id: "my-plugin",
    $InferServerPlugin: {} as ReturnType<typeof myPlugin>,
    getActions: ($fetch)=>{
        return {
            myCustomAction: async (data: {
                foo: string,
            }, fetchOptions?: BetterFetchOption)=>{
                const res = $fetch("/custom/action", {
                    method: "POST",
                    body: {
                        foo: data.foo
                    },
                    ...fetchOptions
                })
                return res
            }
        }
    }
} satisfies BetterAuthClientPlugin
```

<Callout>
  As a general guideline, ensure that each function accepts only one argument, with an optional second argument for fetchOptions to allow users to pass additional options to the fetch call. The function should return an object containing data and error keys.

  If your use case involves actions beyond API calls, feel free to deviate from this rule.
</Callout>

### Get Atoms

This is only useful if you want to provide `hooks` like `useSession`.

Get atoms is called with the `fetch` function from better fetch and it should return an object with the atoms. The atoms should be created using <Link href="https://github.com/nanostores/nanostores">nanostores</Link>. The atoms will be resolved by each framework `useStore` hook provided by nanostores.

```ts title="client-plugin.ts"
import { atom } from "nanostores";
import type { BetterAuthClientPlugin } from "better-auth/client";

const myPluginClient = {
    id: "my-plugin",
    $InferServerPlugin: {} as ReturnType<typeof myPlugin>,
    getAtoms: ($fetch)=>{
        const myAtom = atom<null>()
        return {
            myAtom
        }
    }
} satisfies BetterAuthClientPlugin
```

See built-in plugins for examples of how to use atoms properly.

### Path methods

By default, inferred paths use `GET` method if they don't require a body and `POST` if they do. You can override this by passing a `pathMethods` object. The key should be the path and the value should be the method ("POST" | "GET").

```ts title="client-plugin.ts"
import type { BetterAuthClientPlugin } from "better-auth/client";
import type { myPlugin } from "./plugin";

const myPluginClient = {
    id: "my-plugin",
    $InferServerPlugin: {} as ReturnType<typeof myPlugin>,
    pathMethods: {
        "/my-plugin/hello-world": "POST"
    }
} satisfies BetterAuthClientPlugin
```

### Fetch plugins

If you need to use better fetch plugins you can pass them to the `fetchPlugins` array. You can read more about better fetch plugins in the <Link href="https://better-fetch.vercel.app/docs/plugins">better fetch documentation</Link>.

### Atom Listeners

This is only useful if you want to provide `hooks` like `useSession` and you want to listen to atoms and re-evaluate them when they change.

You can see how this is used in the built-in plugins.

# concepts: Rate Limit
URL: /docs/concepts/rate-limit
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/concepts/rate-limit.mdx

How to limit the number of requests a user can make to the server in a given time period.

***

title: Rate Limit
description: How to limit the number of requests a user can make to the server in a given time period.

Better Auth includes a built-in rate limiter to help manage traffic and prevent abuse. By default, in production mode, the rate limiter is set to:

* Window: 60 seconds
* Max Requests: 100 requests

<Callout type="warning">
  Server-side requests made using `auth.api` aren't affected by rate limiting. Rate limits only apply to client-initiated requests.
</Callout>

You can easily customize these settings by passing the rateLimit object to the betterAuth function.

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
    rateLimit: {
        window: 10, // time window in seconds
        max: 100, // max requests in the window
    },
})
```

Rate limiting is disabled in development mode by default. In order to enable it, set `enabled` to `true`:

```ts title="auth.ts"
export const auth = betterAuth({
    rateLimit: {
        enabled: true,
        //...other options
    },
})
```

In addition to the default settings, Better Auth provides custom rules for specific paths. For example:

* `/sign-in/email`: Is limited to 3 requests within 10 seconds.

In addition, plugins also define custom rules for specific paths. For example, `twoFactor` plugin has custom rules:

* `/two-factor/verify`: Is limited to 3 requests within 10 seconds.

These custom rules ensure that sensitive operations are protected with stricter limits.

## Configuring Rate Limit

### Connecting IP Address

Rate limiting uses the connecting IP address to track the number of requests made by a user. The default header checked is `x-forwarded-for`, which is commonly used in production environments. If you are using a different header to track the user's IP address, you'll need to specify it.

```ts title="auth.ts"
export const auth = betterAuth({
    //...other options
    advanced: {
        ipAddress: {
          ipAddressHeaders: ["cf-connecting-ip"], // Cloudflare specific header example
      },
    },
    rateLimit: {
        enabled: true,
        window: 60, // time window in seconds
        max: 100, // max requests in the window
    },
})
```

### Rate Limit Window

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
    //...other options
    rateLimit: {
        window: 60, // time window in seconds
        max: 100, // max requests in the window
    },
})
```

You can also pass custom rules for specific paths.

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
    //...other options
    rateLimit: {
        window: 60, // time window in seconds
        max: 100, // max requests in the window
        customRules: {
            "/sign-in/email": {
                window: 10,
                max: 3,
            },
            "/two-factor/*": async (request)=> {
                // custom function to return rate limit window and max
                return {
                    window: 10,
                    max: 3,
                }
            }
        },
    },
})
```

If you like to disable rate limiting for a specific path, you can set it to `false` or return `false` from the custom rule function.

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
    //...other options
    rateLimit: {
        customRules: {
            "/get-session": false,
        },
    },
})
```

### Storage

By default, rate limit data is stored in memory, which may not be suitable for many use cases, particularly in serverless environments. To address this, you can use a database, secondary storage, or custom storage for storing rate limit data.

Using Database

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
    //...other options
    rateLimit: {
        storage: "database",
        modelName: "rateLimit", //optional by default "rateLimit" is used
    },
})
```

Make sure to run `migrate` to create the rate limit table in your database.

```bash
npx @better-auth/cli migrate
```

Using Secondary Storage

If a [Secondary Storage](/docs/concepts/database#secondary-storage) has been configured you can use that to store rate limit data.

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
    //...other options
    rateLimit: {
		storage: "secondary-storage"
    },
})
```

Custom Storage

If none of the above solutions suits your use case you can implement a `customStorage`.

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
    //...other options
    rateLimit: {
        customStorage: {
            get: async (key) => {
                // get rate limit data
            },
            set: async (key, value) => {
                // set rate limit data
            },
        },
    },
})
```

## Handling Rate Limit Errors

When a request exceeds the rate limit, Better Auth returns the following header:

* `X-Retry-After`: The number of seconds until the user can make another request.

To handle rate limit errors on the client side, you can manage them either globally or on a per-request basis. Since Better Auth clients wrap over Better Fetch, you can pass `fetchOptions` to handle rate limit errors

Global Handling

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client";

export const authClient = createAuthClient({
    fetchOptions: {
        onError: async (context) => {
            const { response } = context;
            if (response.status === 429) {
                const retryAfter = response.headers.get("X-Retry-After");
                console.log(`Rate limit exceeded. Retry after ${retryAfter} seconds`);
            }
        },
    }
})
```

Per Request Handling

```ts title="auth-client.ts"
import { authClient } from "./auth-client";

await authClient.signIn.email({
    fetchOptions: {
        onError: async (context) => {
            const { response } = context;
            if (response.status === 429) {
                const retryAfter = response.headers.get("X-Retry-After");
                console.log(`Rate limit exceeded. Retry after ${retryAfter} seconds`);
            }
        },
    }
})
```

### Schema

If you are using a database to store rate limit data you need this schema:

Table Name: `rateLimit`

```
<DatabaseTable
  fields={[
      {
      name: "id",
      type: "string",
      description: "Database ID",
      isPrimaryKey: true
      },
      {
      name: "key",
      type: "string",
      description: "Unique identifier for each rate limit key",
      },
      {
      name: "count",
      type: "integer",
      description: "Time window in seconds"
      },
      {
      name: "lastRequest",
      type: "bigint",
      description: "Max requests in the window"
      }]}
/>
```

# concepts: Session Management
URL: /docs/concepts/session-management
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/concepts/session-management.mdx

Better Auth session management.

***

title: Session Management
description: Better Auth session management.

~~~
---------------------------------------------

Better Auth manages session using a traditional cookie-based session management. The session is stored in a cookie and is sent to the server on every request. The server then verifies the session and returns the user data if the session is valid.

## Session table

The session table stores the session data. The session table has the following fields:

* `id`: The session token. Which is also used as the session cookie.
* `userId`: The user ID of the user.
* `expiresAt`: The expiration date of the session.
* `ipAddress`: The IP address of the user.
* `userAgent`: The user agent of the user. It stores the user agent header from the request.

## Session Expiration

The session expires after 7 days by default. But whenever the session is used and the `updateAge` is reached, the session expiration is updated to the current time plus the `expiresIn` value.

You can change both the `expiresIn` and `updateAge` values by passing the `session` object to the `auth` configuration.

```ts title="auth.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    //... other config options
    session: {
        expiresIn: 60 * 60 * 24 * 7, // 7 days
        updateAge: 60 * 60 * 24 // 1 day (every 1 day the session expiration is updated)
    }
})
```

### Disable Session Refresh

You can disable session refresh so that the session is not updated regardless of the `updateAge` option.

```ts title="auth.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    //... other config options
    session: {
        disableSessionRefresh: true
    }
})
```

## Session Freshness

Some endpoints in Better Auth require the session to be **fresh**. A session is considered fresh if its `createdAt` is within the `freshAge` limit. By default, the `freshAge` is set to **1 day** (60 \* 60 \* 24).

You can customize the `freshAge` value by passing a `session` object in the `auth` configuration:

```ts title="auth.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    //... other config options
    session: {
        freshAge: 60 * 5 // 5 minutes (the session is fresh if created within the last 5 minutes)
    }
})
```

To **disable the freshness check**, set `freshAge` to `0`:

```ts title="auth.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    //... other config options
    session: {
        freshAge: 0 // Disable freshness check
    }
})
```

## Session Management

Better Auth provides a set of functions to manage sessions.

### Get Session

The `getSession` function retrieves the current active session.

```ts client="client.ts"
import { authClient } from "@/lib/client"

const { data: session } = await authClient.getSession()
```

To learn how to customize the session response check the [Customizing Session Response](#customizing-session-response) section.

### Use Session

The `useSession` action provides a reactive way to access the current session.

```ts client="client.ts"
import { authClient } from "@/lib/client"

const { data: session } = authClient.useSession()
```

### List Sessions

The `listSessions` function returns a list of sessions that are active for the user.

```ts title="auth-client.ts"
import { authClient } from "@/lib/client"

const sessions = await authClient.listSessions()
```

### Revoke Session

When a user signs out of a device, the session is automatically ended. However, you can also end a session manually from any device the user is signed into.

To end a session, use the `revokeSession` function. Just pass the session token as a parameter.

```ts title="auth-client.ts"
await authClient.revokeSession({
    token: "session-token"
})
```

### Revoke Other Sessions

To revoke all other sessions except the current session, you can use the `revokeOtherSessions` function.
~~~

```ts title="auth-client.ts"
await authClient.revokeOtherSessions()
```

### Revoke All Sessions

To revoke all sessions, you can use the `revokeSessions` function.

```ts title="auth-client.ts"
await authClient.revokeSessions()
```

### Revoking Sessions on Password Change

You can revoke all sessions when the user changes their password by passing `revokeOtherSessions` as true on `changePassword` function.

```ts title="auth.ts"
await authClient.changePassword({
    newPassword: newPassword,
    currentPassword: currentPassword,
    revokeOtherSessions: true,
})
```

## Session Caching

### Cookie Cache

Calling your database every time `useSession` or `getSession` invoked isnâ€™t ideal, especially if sessions donâ€™t change frequently. Cookie caching handles this by storing session data in a short-lived, signed cookieâ€”similar to how JWT access tokens are used with refresh tokens.

When cookie caching is enabled, the server can check session validity from the cookie itself instead of hitting the database each time. The cookie is signed to prevent tampering, and a short `maxAge` ensures that the session data gets refreshed regularly. If a session is revoked or expires, the cookie will be invalidated automatically.

To turn on cookie caching, just set `session.cookieCache` in your auth config:

```ts title="auth.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    session: {
        cookieCache: {
            enabled: true,
            maxAge: 5 * 60 // Cache duration in seconds
        }
    }
});
```

If you want to disable returning from the cookie cache when fetching the session, you can pass `disableCookieCache:true` this will force the server to fetch the session from the database and also refresh the cookie cache.

```ts title="auth-client.ts"
const session = await authClient.getSession({ query: {
    disableCookieCache: true
}})
```

or on the server

```ts title="server.ts"
await auth.api.getSession({
    query: {
        disableCookieCache: true,
    },
    headers: req.headers, // pass the headers
});
```

## Customizing Session Response

When you call `getSession` or `useSession`, the session data is returned as a `user` and `session` object. You can customize this response using the `customSession` plugin.

```ts title="auth.ts"
import { customSession } from "better-auth/plugins";

export const auth = betterAuth({
    plugins: [
        customSession(async ({ user, session }) => {
            const roles = findUserRoles(session.session.userId);
            return {
                roles,
                user: {
                    ...user,
                    newField: "newField",
                },
                session
            };
        }),
    ],
});
```

This will add `roles` and `user.newField` to the session response.

Infer on the Client

```ts title="auth-client.ts"
import { customSessionClient } from "better-auth/client/plugins";
import type { auth } from "@/lib/auth"; // Import the auth instance as a type

const authClient = createAuthClient({
    plugins: [customSessionClient<typeof auth>()],
});

const { data } = authClient.useSession();
const { data: sessionData } = await authClient.getSession();
// data.roles
// data.user.newField
```

### Caveats on Customizing Session Response

1. The passed `session` object to the callback does not infer fields added by plugins.

However, as a workaround, you can pull up your auth options and pass it to the plugin to infer the fields.

```ts
import { betterAuth, BetterAuthOptions } from "better-auth";

const options = {
  //...config options
  plugins: [
    //...plugins
  ]
} satisfies BetterAuthOptions;

```
export const auth = betterAuth({
    ...options,
    plugins: [
        ...(options.plugins ?? []),
        customSession(async ({ user, session }, ctx) => {
            // now both user and session will infer the fields added by plugins and your custom fields
            return {
                user,
                session
            }
        }, options), // pass options here // [!code highlight]
    ]
})

2. When your server and client code are in separate projects or repositories, and you cannot import the `auth` instance as a type reference, type inference for custom session fields will not work on the client side.
3. Session caching, including secondary storage or cookie cache, does not include custom fields. Each time the session is fetched, your custom session function will be called.

Mutating the list-device-sessions endpoint
The `/multi-session/list-device-sessions` endpoint from the [multi-session](/docs/plugins/multi-session) plugin is used to list the devices that the user is signed into.

You can mutate the response of this endpoint by passing the `shouldMutateListDeviceSessionsEndpoint` option to the `customSession` plugin.

By default, we do not mutate the response of this endpoint.

```ts title="auth.ts"
import { customSession } from "better-auth/plugins";

export const auth = betterAuth({
    plugins: [
        customSession(async ({ user, session }, ctx) => {
            return {
                user,
                session
            }
        }, {}, { shouldMutateListDeviceSessionsEndpoint: true }), // [!code highlight]
    ],
});
```

# concepts: TypeScript
URL: /docs/concepts/typescript
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/concepts/typescript.mdx

Better Auth TypeScript integration.

***

title: TypeScript
description: Better Auth TypeScript integration.

Better Auth is designed to be type-safe. Both the client and server are built with TypeScript, allowing you to easily infer types.

## TypeScript Config

### Strict Mode

Better Auth is designed to work with TypeScript's strict mode. We recommend enabling strict mode in your TypeScript config file:

```json title="tsconfig.json"
{
  "compilerOptions": {
    "strict": true
  }
}
```

if you can't set `strict` to `true`, you can enable `strictNullChecks`:

```json title="tsconfig.json"
{
  "compilerOptions": {
    "strictNullChecks": true,
  }
}
```

<Callout type="warn">
  If you're running into issues with TypeScript inference exceeding maximum length the compiler will serialize,
  then please make sure you're following the instructions above, as well as ensuring that both `declaration` and `composite` are not enabled.
</Callout>

## Inferring Types

Both the client SDK and the server offer types that can be inferred using the `$Infer` property. Plugins can extend base types like `User` and `Session`, and you can use `$Infer` to infer these types. Additionally, plugins can provide extra types that can also be inferred through `$Infer`.

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client"

const authClient = createAuthClient()

export type Session = typeof authClient.$Infer.Session
```

The `Session` type includes both `session` and `user` properties. The user property represents the user object type, and the `session` property represents the `session` object type.

You can also infer types on the server side.

```ts title="auth.ts"
import { betterAuth } from "better-auth"
import Database from "better-sqlite3"

export const auth = betterAuth({
    database: new Database("database.db")
})

type Session = typeof auth.$Infer.Session
```

## Additional Fields

Better Auth allows you to add additional fields to the user and session objects. All additional fields are properly inferred and available on the server and client side.

~~~ts
import { betterAuth } from "better-auth"
import Database from "better-sqlite3"

```ts
export const auth = betterAuth({
    database: new Database("database.db"),
    user: {
       additionalFields: {
          role: {
              type: "string",
              input: false
            }
        }
    }

})

type Session = typeof auth.$Infer.Session
~~~

In the example above, we added a `role` field to the user object. This field is now available on the `Session` type.

### The `input` property

The `input` property in an additional field configuration determines whether the field should be included in the user input. This property defaults to `true`, meaning the field will be part of the user input during operations like registration.

To prevent a field from being part of the user input, you must explicitly set `input: false`:

```ts
additionalFields: {
    role: {
        type: "string",
        input: false
    }
}
```

When `input` is set to `false`, the field will be excluded from user input, preventing users from passing a value for it.

By default, additional fields are included in the user input, which can lead to security vulnerabilities if not handled carefully. For fields that should not be set by the user, like a `role`, it is crucial to set `input: false` in the configuration.

### Inferring Additional Fields on Client

To make sure proper type inference for additional fields on the client side, you need to inform the client about these fields. There are two approaches to achieve this, depending on your project structure:

1. For Monorepo or Single-Project Setups

If your server and client code reside in the same project, you can use the `inferAdditionalFields` plugin to automatically infer the additional fields from your server configuration.

```ts
import { inferAdditionalFields } from "better-auth/client/plugins";
import { createAuthClient } from "better-auth/react";
import type { auth } from "./auth";

export const authClient = createAuthClient({
  plugins: [inferAdditionalFields<typeof auth>()],
});
```

2. For Separate Client-Server Projects

If your client and server are in separate projects, you'll need to manually specify the additional fields when creating the auth client.

```ts
import { inferAdditionalFields } from "better-auth/client/plugins";

export const authClient = createAuthClient({
  plugins: [inferAdditionalFields({
      user: {
        role: {
          type: "string"
        }
      }
  })],
});
```

# concepts: User & Accounts

URL: /docs/concepts/users-accounts
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/concepts/users-accounts.mdx

User and account management.

***

title: User & Accounts
description: User and account management.

Beyond authenticating users, Better Auth also provides a set of methods to manage users. This includes, updating user information, changing passwords, and more.

The user table stores the authentication data of the user [Click here to view the schema](/docs/concepts/database#user).

The user table can be extended using [additional fields](/docs/concepts/database#extending-core-schema) or by plugins to store additional data.

## Update User

### Update User Information

To update user information, you can use the `updateUser` function provided by the client. The `updateUser` function takes an object with the following properties:

```ts
await authClient.updateUser({
    image: "https://example.com/image.jpg",
    name: "John Doe",
})
```

### Change Email

To allow users to change their email, first enable the `changeEmail` feature, which is disabled by default. Set `changeEmail.enabled` to `true`:

```ts
export const auth = betterAuth({
    user: {
        changeEmail: {
            enabled: true,
        }
    }
})
```

For users with a verified email, provide the `sendChangeEmailVerification` function. This function triggers when a user changes their email, sending a verification email with a URL and token. If the current email isn't verified, the change happens immediately without verification.

```ts
export const auth = betterAuth({
    user: {
        changeEmail: {
            enabled: true,
            sendChangeEmailVerification: async ({ user, newEmail, url, token }, request) => {
                await sendEmail({
                    to: user.email, // verification email must be sent to the current user email to approve the change
                    subject: 'Approve email change',
                    text: `Click the link to approve the change: ${url}`
                })
            }
        }
    }
})
```

Once enabled, use the `changeEmail` function on the client to update a userâ€™s email. The user must verify their current email before changing it.

```ts
await authClient.changeEmail({
    newEmail: "new-email@email.com",
    callbackURL: "/dashboard", //to redirect after verification
});
```

After verification, the new email is updated in the user table, and a confirmation is sent to the new address.

<Callout type="warn">
  If the current email is unverified, the new email is updated without the verification step.
</Callout>

### Change Password

A user's password isn't stored in the user table. Instead, it's stored in the account table. To change the password of a user, you can use one of the following approaches:

### Client Side

```ts
const { data, error } = await authClient.changePassword({
    newPassword: newpassword1234,
    currentPassword: oldpassword1234,
    revokeOtherSessions, // required
});
```

### Server Side

```ts
const data = await auth.api.changePassword({
    body: {
        newPassword: newpassword1234,
        currentPassword: oldpassword1234,
        revokeOtherSessions, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type changePassword = {
    /**
     * The new password to set
     */
    newPassword: string = "newpassword1234"
    /**
     * The current user password
     */
    currentPassword: string = "oldpassword1234"
    /**
     * When set to true, all other active sessions for this user will be invalidated
     */
    revokeOtherSessions?: boolean = true

}
```

### Set Password

If a user was registered using OAuth or other providers, they won't have a password or a credential account. In this case, you can use the `setPassword` action to set a password for the user. For security reasons, this function can only be called from the server. We recommend having users go through a 'forgot password' flow to set a password for their account.

```ts
await auth.api.setPassword({
    body: { newPassword: "password" },
    headers: // headers containing the user's session token
});
```

## Delete User

Better Auth provides a utility to hard delete a user from your database. It's disabled by default, but you can enable it easily by passing `enabled:true`

```ts
export const auth = betterAuth({
    //...other config
    user: {
        deleteUser: { // [!code highlight]
            enabled: true // [!code highlight]
        } // [!code highlight]
    }
})
```

Once enabled, you can call `authClient.deleteUser` to permanently delete user data from your database.

### Adding Verification Before Deletion

For added security, youâ€™ll likely want to confirm the userâ€™s intent before deleting their account. A common approach is to send a verification email. Better Auth provides a `sendDeleteAccountVerification` utility for this purpose.
This is especially needed if you have OAuth setup and want them to be able to delete their account without forcing them to login again for a fresh session.

Hereâ€™s how you can set it up:

```ts
export const auth = betterAuth({
    user: {
        deleteUser: {
            enabled: true,
            sendDeleteAccountVerification: async (
                {
                    user,   // The user object
                    url, // The auto-generated URL for deletion
                    token  // The verification token  (can be used to generate custom URL)
                },
                request  // The original request object (optional)
            ) => {
                // Your email sending logic here
                // Example: sendEmail(data.user.email, "Verify Deletion", data.url);
            },
        },
    },
});
```

How callback verification works:

* **Callback URL**: The URL provided in `sendDeleteAccountVerification` is a pre-generated link that deletes the user data when accessed.

```ts title="delete-user.ts"
await authClient.deleteUser({
    callbackURL: "/goodbye" // you can provide a callback URL to redirect after deletion
});
```

* **Authentication Check**: The user must be signed in to the account theyâ€™re attempting to delete.
  If they arenâ€™t signed in, the deletion process will fail.

If you have sent a custom URL, you can use the `deleteUser` method with the token to delete the user.

```ts title="delete-user.ts"
await authClient.deleteUser({
    token
});
```

### Authentication Requirements

To delete a user, the user must meet one of the following requirements:

1. A valid password

if the user has a password, they can delete their account by providing the password.

```ts title="delete-user.ts"
await authClient.deleteUser({
    password: "password"
});
```

2. Fresh session

The user must have a `fresh` session token, meaning the user must have signed in recently. This is checked if the password is not provided.

<Callout type="warn">
  By default `session.freshAge` is set to `60 * 60 * 24` (1 day). You can change this value by passing the `session` object to the `auth` configuration. If it is set to `0`, the freshness check is disabled. It is recommended not to disable this check if you are not using email verification for deleting the account.
</Callout>

```ts title="delete-user.ts"
await authClient.deleteUser();
```

3. Enabled email verification (needed for OAuth users)

As OAuth users don't have a password, we need to send a verification email to confirm the user's intent to delete their account. If you have already added the `sendDeleteAccountVerification` callback, you can just call the `deleteUser` method without providing any other information.

```ts title="delete-user.ts"
await authClient.deleteUser();
```

4. If you have a custom delete account page and sent that url via the `sendDeleteAccountVerification` callback.
   Then you need to call the `deleteUser` method with the token to complete the deletion.

```ts title="delete-user.ts"
await authClient.deleteUser({
    token
});
```

### Callbacks

beforeDelete: This callback is called before the user is deleted. You can use this callback to perform any cleanup or additional checks before deleting the user.

```ts title="auth.ts"
export const auth = betterAuth({
    user: {
        deleteUser: {
            enabled: true,
            beforeDelete: async (user) => {
                // Perform any cleanup or additional checks here
            },
        },
    },
});
```

you can also throw `APIError` to interrupt the deletion process.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { APIError } from "better-auth/api";

export const auth = betterAuth({
    user: {
        deleteUser: {
            enabled: true,
            beforeDelete: async (user, request) => {
                if (user.email.includes("admin")) {
                    throw new APIError("BAD_REQUEST", {
                        message: "Admin accounts can't be deleted",
                    });
                }
            },
        },
    },
});
```

afterDelete: This callback is called after the user is deleted. You can use this callback to perform any cleanup or additional actions after the user is deleted.

```ts title="auth.ts"
export const auth = betterAuth({
    user: {
        deleteUser: {
            enabled: true,
            afterDelete: async (user, request) => {
                // Perform any cleanup or additional actions here
            },
        },
    },
});
```

## Accounts

Better Auth supports multiple authentication methods. Each authentication method is called a provider. For example, email and password authentication is a provider, Google authentication is a provider, etc.

When a user signs in using a provider, an account is created for the user. The account stores the authentication data returned by the provider. This data includes the access token, refresh token, and other information returned by the provider.

The account table stores the authentication data of the user [Click here to view the schema](/docs/concepts/database#account)

### List User Accounts

To list user accounts you can use `client.user.listAccounts` method. Which will return all accounts associated with a user.

```ts
const accounts = await authClient.listAccounts();
```

### Token Encryption

Better Auth doesnâ€™t encrypt tokens by default and thatâ€™s intentional. We want you to have full control over how encryption and decryption are handled, rather than baking in behavior that could be confusing or limiting. If you need to store encrypted tokens (like accessToken or refreshToken), you can use databaseHooks to encrypt them before theyâ€™re saved to your database.

```ts
export const auth = betterAuth({
    databaseHooks: {
        account: {
            create: {
                before(account, context) {
                    const withEncryptedTokens = { ...account };
                    if (account.accessToken) {
                        const encryptedAccessToken = encrypt(account.accessToken)  // [!code highlight]
                        withEncryptedTokens.accessToken = encryptedAccessToken;
                    }
                    if (account.refreshToken) {
                        const encryptedRefreshToken = encrypt(account.refreshToken); // [!code highlight]
                        withEncryptedTokens.refreshToken = encryptedRefreshToken;
                    }
                    return {
                        data: withEncryptedTokens
                    }
                },
            }
        }
    }
})
```

Then whenever you retrieve back the account make sure to decrypt the tokens before using them.

### Account Linking

Account linking enables users to associate multiple authentication methods with a single account. With Better Auth, users can connect additional social sign-ons or OAuth providers to their existing accounts if the provider confirms the user's email as verified.

If account linking is disabled, no accounts can be linked, regardless of the provider or email verification status.

```ts title="auth.ts"
export const auth = betterAuth({
    account: {
        accountLinking: {
            enabled: true,
        }
    },
});
```

#### Forced Linking

You can specify a list of "trusted providers." When a user logs in using a trusted provider, their account will be automatically linked even if the provider doesnâ€™t confirm the email verification status. Use this with caution as it may increase the risk of account takeover.

```ts title="auth.ts"
export const auth = betterAuth({
    account: {
        accountLinking: {
            enabled: true,
            trustedProviders: ["google", "github"]
        }
    },
});
```

#### Manually Linking Accounts

Users already signed in can manually link their account to additional social providers or credential-based accounts.

* **Linking Social Accounts:** Use the `linkSocial` method on the client to link a social provider to the user's account.

  ```ts
  await authClient.linkSocial({
      provider: "google", // Provider to link
      callbackURL: "/callback" // Callback URL after linking completes
  });
  ```

  You can also request specific scopes when linking a social account, which can be different from the scopes used during the initial authentication:

  ```ts
  await authClient.linkSocial({
      provider: "google",
      callbackURL: "/callback",
      scopes: ["https://www.googleapis.com/auth/drive.readonly"] // Request additional scopes
  });
  ```

  You can also link accounts using ID tokens directly, without redirecting to the provider's OAuth flow:

  ```ts
  await authClient.linkSocial({
      provider: "google",
      idToken: {
          token: "id_token_from_provider",
          nonce: "nonce_used_for_token", // Optional
          accessToken: "access_token", // Optional, may be required by some providers
          refreshToken: "refresh_token" // Optional
      }
  });
  ```

  This is useful when you already have valid tokens from the provider, for example:

  * After signing in with a native SDK
  * When using a mobile app that handles authentication
  * When implementing custom OAuth flows

  The ID token must be valid and the provider must support ID token verification.

  If you want your users to be able to link a social account with a different email address than the user, or if you want to use a provider that does not return email addresses, you will need to enable this in the account linking settings.

  ```ts title="auth.ts"
  export const auth = betterAuth({
      account: {
          accountLinking: {
              allowDifferentEmails: true
          }
      },
  });
  ```

  If you want the newly linked accounts to update the user information, you need to enable this in the account linking settings.

  ```ts title="auth.ts"
  export const auth = betterAuth({
      account: {
          accountLinking: {
              updateUserInfoOnLink: true
          }
      },
  });
  ```

* **Linking Credential-Based Accounts:** To link a credential-based account (e.g., email and password), users can initiate a "forgot password" flow, or you can call the `setPassword` method on the server.

  ```ts
  await auth.api.setPassword({
      headers: /* headers containing the user's session token */,
      password: /* new password */
  });
  ```

<Callout>
  `setPassword` can't be called from the client for security reasons.
</Callout>

### Account Unlinking

You can unlink a user account by providing a `providerId`.

```ts
await authClient.unlinkAccount({
    providerId: "google"
});

// Unlink a specific account
await authClient.unlinkAccount({
    providerId: "google",
    accountId: "123"
});
```

If the account doesn't exist, it will throw an error. Additionally, if the user only has one account, unlinking will be prevented to stop account lockout (unless `allowUnlinkingAll` is set to `true`).

```ts title="auth.ts"
export const auth = betterAuth({
    account: {
        accountLinking: {
            allowUnlinkingAll: true
        }
    },
});
```

# examples: Astro Example
URL: /docs/examples/astro
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/examples/astro.mdx

Better Auth Astro example.

***

title: Astro Example
description: Better Auth Astro example.

This is an example of how to use Better Auth with Astro. It uses Solid for building the components.

Implements the following features:
Email & Password . Social Sign-in with Google . Passkeys . Email Verification . Password Reset . Two Factor Authentication . Profile Update . Session Management

<ForkButton url="https://github.com/better-auth/examples/tree/main/astro-example" />

<iframe
  src="https://stackblitz.com/github/better-auth/examples/tree/main/astro-example?codemirror=1&fontsize=14&hidenavigation=1&runonclick=1&hidedevtools=1"
  style={{
    width: "100%",
    height: "500px",
    border: 0,
    borderRadius: "4px",
    overflow: "hidden"
 }}
  title="Better Auth Astro+Solid Example"
  allow="accelerometer; ambient-light-sensor; camera; encrypted-media; geolocation; gyroscope; hid; microphone; midi; payment; usb; vr; xr-spatial-tracking"
  sandbox="allow-forms allow-modals allow-popups allow-presentation allow-same-origin allow-scripts"
/>

## How to run

1. Clone the code sandbox (or the repo) and open it in your code editor

2. Provide .env file with the following variables
   ```txt

```
   GOOGLE_CLIENT_ID=
   GOOGLE_CLIENT_SECRET=
   BETTER_AUTH_SECRET=

//if you don't have these, you can get them from the google developer console. If you don't want to use google sign-in, you can remove the google config from the `auth.ts` file.

3. Run the following commands
   ```bash
   pnpm install
   pnpm run dev
   ```

4. Open the browser and navigate to `http://localhost:3000`

# examples: Next.js Example
URL: /docs/examples/next-js
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/examples/next-js.mdx

Better Auth Next.js example.

***

title: Next.js Example
description: Better Auth Next.js example.

This is an example of how to use Better Auth with Next.

Implements the following features:
Email & Password . Social Sign-in . Passkeys . Email Verification . Password Reset . Two Factor Authentication . Profile Update . Session Management . Organization, Members and Roles

See [Demo](https://demo.better-auth.com)

```
<ForkButton url="better-auth/better-auth/tree/main/demo/nextjs" />

<iframe
  src="https://stackblitz.com/github/better-auth/better-auth/tree/main/demo/nextjs?codemirror=1&fontsize=14&hidenavigation=1&runonclick=1&hidedevtools=1"
  style={{
    width: "100%",
    height: "500px",
    border: 0,
    borderRadius: "4px",
    overflow: "hidden"
 }}
  title="Better Auth Next.js Example"
  allow="accelerometer; ambient-light-sensor; camera; encrypted-media; geolocation; gyroscope; hid; microphone; midi; payment; usb; vr; xr-spatial-tracking"
  sandbox="allow-forms allow-modals allow-popups allow-presentation allow-same-origin allow-scripts"
/>
```

## How to run

1. Clone the code sandbox (or the repo) and open it in your code editor
2. Move .env.example to .env and provide necessary variables
3. Run the following commands
   ```bash
   pnpm install
   pnpm dev
   ```
4. Open the browser and navigate to `http://localhost:3000`

### SSO Login Example

For this example, we utilize DummyIDP. Initiate the login from the [DummyIDP login](https://dummyidp.com/apps/app_01k16v4vb5yytywqjjvv2b3435/login), click "Proceed", and from here it will direct you to user's dashboard.

# examples: Nuxt Example
URL: /docs/examples/nuxt
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/examples/nuxt.mdx

Better Auth Nuxt example.

***

title: Nuxt Example
description: Better Auth Nuxt example.

This is an example of how to use Better Auth with Nuxt.

Implements the following features:
Email & Password . Social Sign-in with Google

```
<ForkButton url="https://github.com/better-auth/examples/tree/main/nuxt-example" />

<iframe
  src="https://stackblitz.com/github/better-auth/examples/tree/main/nuxt-example?codemirror=1&fontsize=14&hidenavigation=1&runonclick=1&hidedevtools=1"
  style={{
    width: "100%",
    height: "500px",
    border: 0,
    borderRadius: "4px",
    overflow: "hidden"
 }}
  title="Better Auth Nuxt Example"
  allow="accelerometer; ambient-light-sensor; camera; encrypted-media; geolocation; gyroscope; hid; microphone; midi; payment; usb; vr; xr-spatial-tracking"
  sandbox="allow-forms allow-modals allow-popups allow-presentation allow-same-origin allow-scripts"
/>
```

## How to run

1. Clone the code sandbox (or the repo) and open it in your code editor
2. Move .env.example to .env and provide necessary variables
3. Run the following commands
   ```bash
   pnpm install
   pnpm dev
   ```
4. Open the browser and navigate to `http://localhost:3000`

# examples: Remix Example
URL: /docs/examples/remix
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/examples/remix.mdx

Better Auth Remix example.

***

title: Remix Example
description: Better Auth Remix example.

This is an example of how to use Better Auth with Remix.

Implements the following features:
Email & Password . Social Sign-in with Google . Passkeys . Email Verification . Password Reset . Two Factor Authentication . Profile Update . Session Management

<ForkButton url="https://github.com/better-auth/examples/tree/main/remix-example" />

<iframe
  src="https://stackblitz.com/github/better-auth/examples/tree/main/remix-example?codemirror=1&fontsize=14&hidedevtools=1&hidenavigation=1&runonclick=1"
  style={{
    width: "100%",
    height: "500px",
    border: 0,
    borderRadius: "4px",
    overflow: "hidden"
 }}
  title="Better Auth Remix Example"
  allow="accelerometer; ambient-light-sensor; camera; encrypted-media; geolocation; gyroscope; hid; microphone; midi; payment; usb; vr; xr-spatial-tracking"
  sandbox="allow-forms allow-modals allow-popups allow-presentation allow-same-origin allow-scripts"
/>

## How to run

1. Clone the code sandbox (or the repo) and open it in your code editor
2. Provide .env file with by copying the `.env.example` file and adding the variables
3. Run the following commands
   ```bash
   pnpm install
   pnpm run dev
   ```
4. Open the browser and navigate to `http://localhost:3000`

# examples: SvelteKit Example
URL: /docs/examples/svelte-kit
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/examples/svelte-kit.mdx

Better Auth SvelteKit example.

***

title: SvelteKit Example
description: Better Auth SvelteKit example.

This is an example of how to use Better Auth with SvelteKit.

Implements the following features:
Email & Password . <u>Social Sign-in with Google</u> . Passkeys . Email Verification . Password Reset . Two Factor Authentication . Profile Update . Session Management

<ForkButton url="https://github.com/better-auth/examples/tree/main/svelte-kit-example" />

<iframe
  src="https://stackblitz.com/github/better-auth/examples/tree/main/svelte-kit-example?codemirror=1&fontsize=14&hidenavigation=1&runonclick=1&hidedevtools=1"
  style={{
    width: "100%",
    height: "500px",
    border: 0,
    borderRadius: "4px",
    overflow: "hidden"
 }}
  title="Better Auth SvelteKit Example"
  allow="accelerometer; ambient-light-sensor; camera; encrypted-media; geolocation; gyroscope; hid; microphone; midi; payment; usb; vr; xr-spatial-tracking"
  sandbox="allow-forms allow-modals allow-popups allow-presentation allow-same-origin allow-scripts"
/>

## How to run

1. Clone the code sandbox (or the repo) and open it in your code editor
2. Move .env.example to .env and provide necessary variables
3. Run the following commands
   ```bash
   pnpm install
   pnpm dev
   ```
4. Open the browser and navigate to `http://localhost:3000`

# guides: Migrating from Auth0 to Better Auth
URL: /docs/guides/auth0-migration-guide
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/guides/auth0-migration-guide.mdx

A step-by-step guide to transitioning from Auth0 to Better Auth.

***

title: Migrating from Auth0 to Better Auth
description: A step-by-step guide to transitioning from Auth0 to Better Auth.

In this guide, we'll walk through the steps to migrate a project from Auth0 to Better Auth â€” including email/password with proper hashing, social/external accounts, two-factor authentication, and more.

<Callout type="warn">
  This migration will invalidate all active sessions. This guide doesn't currently show you how to migrate Organizations but it should be possible with additional steps and the [Organization](/docs/plugins/organization) Plugin.
</Callout>

## Before You Begin

Before starting the migration process, set up Better Auth in your project. Follow the [installation guide](/docs/installation) to get started.

<Steps>
  <Step>
    ### Connect to your database

    You'll need to connect to your database to migrate the users and accounts. You can use any database you want, but for this example, we'll use PostgreSQL.

    <CodeBlockTabs defaultValue="npm">
      <CodeBlockTabsList>
        <CodeBlockTabsTrigger value="npm">
          npm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="pnpm">
          pnpm
        </CodeBlockTabsTrigger>

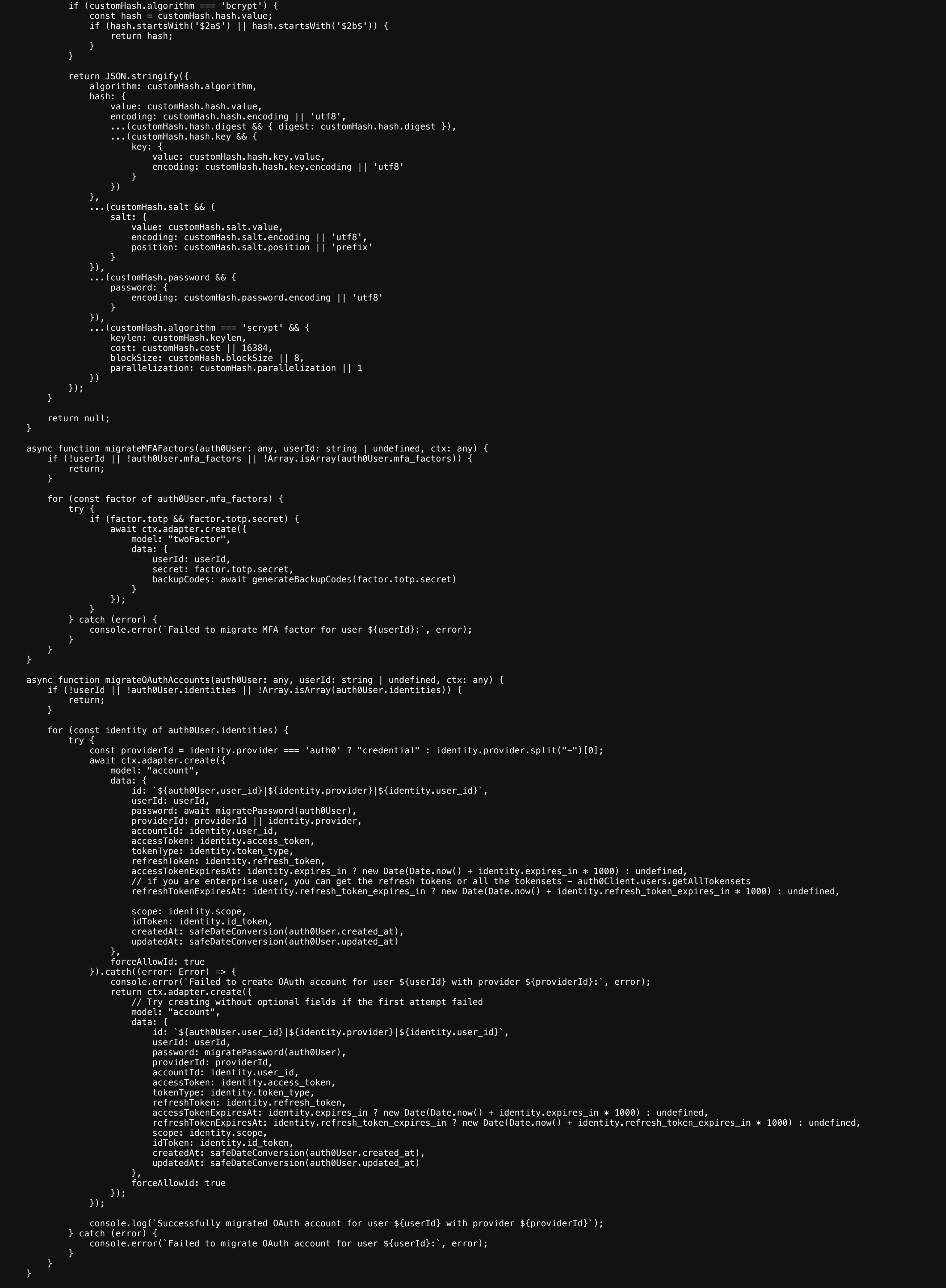
        <CodeBlockTabsTrigger value="yarn">
          yarn
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="bun">
          bun
        </CodeBlockTabsTrigger>
      </CodeBlockTabsList>

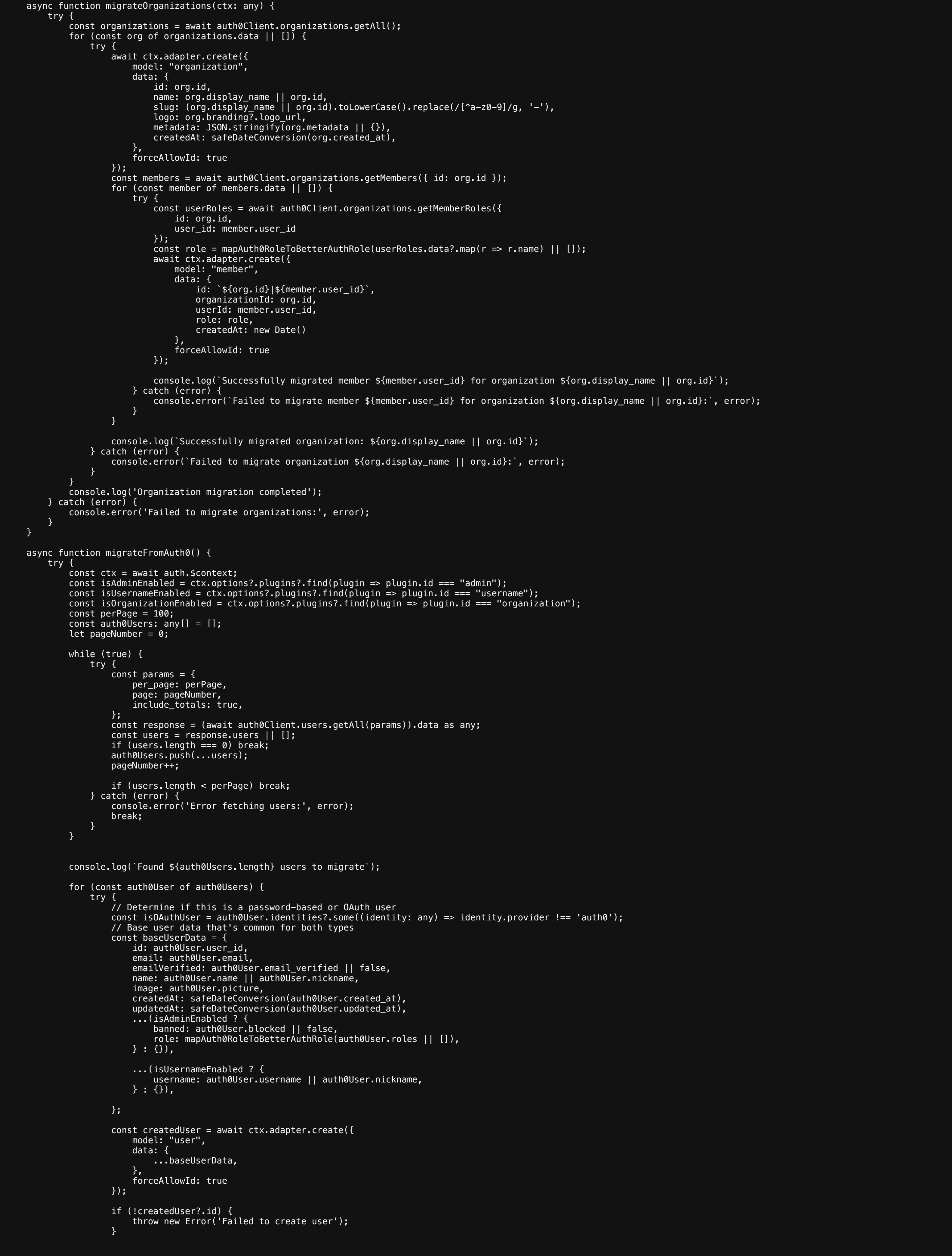
      <CodeBlockTab value="npm">
        ```bash
        npm install pg
        ```

~~~
    </CodeBlockTab>

    <CodeBlockTab value="pnpm">
      ```bash
      pnpm add pg
      ```
    </CodeBlockTab>

    <CodeBlockTab value="yarn">
      ```bash
      yarn add pg
      ```
    </CodeBlockTab>

    <CodeBlockTab value="bun">
      ```bash
      bun add pg
      ```
    </CodeBlockTab>
  </CodeBlockTabs>

  And then you can use the following code to connect to your database.

  ```ts title="auth.ts"
  import { Pool } from "pg";

  export const auth = betterAuth({
      database: new Pool({
          connectionString: process.env.DATABASE_URL
      }),
  })
  ```
</Step>

<Step>
  ### Enable Email and Password (Optional)

  Enable the email and password in your auth config and implement your own logic for sending verification emails, reset password emails, etc.

  ```ts title="auth.ts"
  import { betterAuth } from "better-auth";

  export const auth = betterAuth({
      database: new Pool({
          connectionString: process.env.DATABASE_URL
      }),
      emailAndPassword: { // [!code highlight]
          enabled: true, // [!code highlight]
      }, // [!code highlight]
      emailVerification: {
        sendVerificationEmail: async({ user, url })=>{
          // implement your logic here to send email verification
        }
      },
  })
  ```

  See [Email and Password](/docs/authentication/email-password) for more configuration options.
</Step>

<Step>
  ### Setup Social Providers (Optional)

  Add social providers you have enabled in your Auth0 project in your auth config.

  ```ts title="auth.ts"
  import { betterAuth } from "better-auth";

  export const auth = betterAuth({
      database: new Pool({
          connectionString: process.env.DATABASE_URL
      }),
      emailAndPassword: {
          enabled: true,
      },
      socialProviders: { // [!code highlight]
          google: { // [!code highlight]
              clientId: process.env.GOOGLE_CLIENT_ID, // [!code highlight]
              clientSecret: process.env.GOOGLE_CLIENT_SECRET, // [!code highlight]
          }, // [!code highlight]
          github: { // [!code highlight]
              clientId: process.env.GITHUB_CLIENT_ID, // [!code highlight]
              clientSecret: process.env.GITHUB_CLIENT_SECRET, // [!code highlight]
          } // [!code highlight]
      } // [!code highlight]
  })
  ```
</Step>

<Step>
  ### Add Plugins (Optional)

  You can add the following plugins to your auth config based on your needs.

  [Admin](/docs/plugins/admin) Plugin will allow you to manage users, user impersonations and app level roles and permissions.

  [Two Factor](/docs/plugins/2fa) Plugin will allow you to add two-factor authentication to your application.

  [Username](/docs/plugins/username) Plugin will allow you to add username authentication to your application.

  ```ts title="auth.ts"
  import { Pool } from "pg";
  import { betterAuth } from "better-auth";
  import { admin, twoFactor, username } from "better-auth/plugins";

  export const auth = betterAuth({
      database: new Pool({
          connectionString: process.env.DATABASE_URL
      }),
      emailAndPassword: {
          enabled: true,
          password: {
              verify: (data) => {
                  // this for an edgecase that you might run in to on verifying the password
              }
          }
      },
      socialProviders: {
          google: {
              clientId: process.env.GOOGLE_CLIENT_ID!,
              clientSecret: process.env.GOOGLE_CLIENT_SECRET!,
          },
          github: {
              clientId: process.env.GITHUB_CLIENT_ID!,
              clientSecret: process.env.GITHUB_CLIENT_SECRET!,
          }
      },
      plugins: [admin(), twoFactor(), username()], // [!code highlight]
  })
  ```
</Step>
~~~

~~~
  <Step>
    ### Generate Schema

    If you're using a custom database adapter, generate the schema:

    ```sh
    npx @better-auth/cli generate
    ```

    or if you're using the default adapter, you can use the following command:

    ```sh
    npx @better-auth/cli migrate
    ```
  </Step>

  <Step>
    ### Install Dependencies

    Install the required dependencies for the migration:

    ```bash
    npm install auth0
    ```
  </Step>

  <Step>
    ### Create the migration script

    Create a new file called `migrate-auth0.ts` in the `scripts` folder and add the following code:

    <Callout type="info">
      Instead of using the Management API, you can use Auth0's bulk user export functionality and pass the exported JSON data directly to the `auth0Users` array. This is especially useful if you need to migrate password hashes and complete user data, which are not available through the Management API.

      **Important Notes:**

      * Password hashes export is only available for Auth0 Enterprise users
      * Free plan users cannot export password hashes and will need to request a support ticket
      * For detailed information about bulk user exports, see the [Auth0 Bulk User Export Documentation](https://auth0.com/docs/manage-users/user-migration/bulk-user-exports)
      * For password hash export details, refer to [Exporting Password Hashes](https://auth0.com/docs/troubleshoot/customer-support/manage-subscriptions/export-data#user-passwords)

      Example:

      ```ts
      // Replace this with your exported users JSON data
      const auth0Users = [
        {
          "email": "helloworld@gmail.com",
          "email_verified": false,
          "name": "Hello world",
          // Note: password_hash is only available for Enterprise users
          "password_hash": "$2b$10$w4kfaZVjrcQ6ZOMiG.M8JeNvnVQkPKZV03pbDUHbxy9Ug0h/McDXi",
          // ... other user data
        }
      ];
      ```
    </Callout>

    ```ts title="scripts/migrate-auth0.ts"
    import { ManagementClient } from 'auth0';
    import { generateRandomString, symmetricEncrypt } from "better-auth/crypto";
    import { auth } from '@/lib/auth';

    const auth0Client = new ManagementClient({
        domain: process.env.AUTH0_DOMAIN!,
        clientId: process.env.AUTH0_CLIENT_ID!,
        clientSecret: process.env.AUTH0_SECRET!,
    });



    function safeDateConversion(timestamp?: string | number): Date {
        if (!timestamp) return new Date();

        const numericTimestamp = typeof timestamp === 'string' ? Date.parse(timestamp) : timestamp;

        const milliseconds = numericTimestamp < 1000000000000 ? numericTimestamp * 1000 : numericTimestamp;

        const date = new Date(milliseconds);

        if (isNaN(date.getTime())) {
            console.warn(`Invalid timestamp: ${timestamp}, falling back to current date`);
            return new Date();
        }

        // Check for unreasonable dates (before 2000 or after 2100)
        const year = date.getFullYear();
        if (year < 2000 || year > 2100) {
            console.warn(`Suspicious date year: ${year}, falling back to current date`);
            return new Date();
        }

        return date;
    }

    // Helper function to generate backup codes for 2FA
    async function generateBackupCodes(secret: string) {
        const key = secret;
        const backupCodes = Array.from({ length: 10 })
            .fill(null)
            .map(() => generateRandomString(10, "a-z", "0-9", "A-Z"))
            .map((code) => `${code.slice(0, 5)}-${code.slice(5)}`);

        const encCodes = await symmetricEncrypt({
            data: JSON.stringify(backupCodes),
            key: key,
        });
        return encCodes;
    }

    function mapAuth0RoleToBetterAuthRole(auth0Roles: string[]) {
        if (typeof auth0Roles === 'string') return auth0Roles;
        if (Array.isArray(auth0Roles)) return auth0Roles.join(',');
    }
    // helper function to migrate password from auth0 to better auth for custom hashes and algs
    async function migratePassword(auth0User: any) {
        if (auth0User.password_hash) {
            if (auth0User.password_hash.startsWith('$2a$') || auth0User.password_hash.startsWith('$2b$')) {
                return auth0User.password_hash;
            }
        }

        if (auth0User.custom_password_hash) {
            const customHash = auth0User.custom_password_hash;
~~~



```
        if (customHash.algorithm === 'bcrypt') {
            const hash = customHash.hash.value;
            if (hash.startsWith('$2a$') || hash.startsWith('$2b$')) {
                return hash;
            }
        }

        return JSON.stringify({
            algorithm: customHash.algorithm,
            hash: {
                value: customHash.hash.value,
                encoding: customHash.hash.encoding || 'utf8',
                ...(customHash.hash.digest && { digest: customHash.hash.digest }),
                ...(customHash.hash.key && {
                    key: {
                        value: customHash.hash.key.value,
                        encoding: customHash.hash.key.encoding || 'utf8'
                    }
                })
            },
            ...(customHash.salt && {
                salt: {
                    value: customHash.salt.value,
                    encoding: customHash.salt.encoding || 'utf8',
                    position: customHash.salt.position || 'prefix'
                }
            }),
            ...(customHash.password && {
                password: {
                    encoding: customHash.password.encoding || 'utf8'
                }
            }),
            ...(customHash.algorithm === 'scrypt' && {
                keylen: customHash.keylen,
                cost: customHash.cost || 16384,
                blockSize: customHash.blockSize || 8,
                parallelization: customHash.parallelization || 1
            })
        });
    }

    return null;
}

async function migrateMFAFactors(auth0User: any, userId: string | undefined, ctx: any) {
    if (!userId || !auth0User.mfa_factors || !Array.isArray(auth0User.mfa_factors)) {
        return;
    }

    for (const factor of auth0User.mfa_factors) {
        try {
            if (factor.totp && factor.totp.secret) {
                await ctx.adapter.create({
                    model: "twoFactor",
                    data: {
                        userId: userId,
                        secret: factor.totp.secret,
                        backupCodes: await generateBackupCodes(factor.totp.secret)
                    }
                });
            }
        } catch (error) {
            console.error(`Failed to migrate MFA factor for user ${userId}:`, error);
        }
    }
}

async function migrateOAuthAccounts(auth0User: any, userId: string | undefined, ctx: any) {
    if (!userId || !auth0User.identities || !Array.isArray(auth0User.identities)) {
        return;
    }

    for (const identity of auth0User.identities) {
        try {
            const providerId = identity.provider === 'auth0' ? "credential" : identity.provider.split("-")[0];
            await ctx.adapter.create({
                model: "account",
                data: {
                    id: `${auth0User.user_id}|${identity.provider}|${identity.user_id}`,
                    userId: userId,
                    password: await migratePassword(auth0User),
                    providerId: providerId || identity.provider,
                    accountId: identity.user_id,
                    accessToken: identity.access_token,
                    tokenType: identity.token_type,
                    refreshToken: identity.refresh_token,
                    accessTokenExpiresAt: identity.expires_in ? new Date(Date.now() + identity.expires_in * 1000) : undefined,
                    // if you are enterprise user, you can get the refresh tokens or all the tokensets - auth0Client.users.getAllTokensets
                    refreshTokenExpiresAt: identity.refresh_token_expires_in ? new Date(Date.now() + identity.refresh_token_expires_in * 1000) : undefined,

                    scope: identity.scope,
                    idToken: identity.id_token,
                    createdAt: safeDateConversion(auth0User.created_at),
                    updatedAt: safeDateConversion(auth0User.updated_at)
                },
                forceAllowId: true
            }).catch((error: Error) => {
                console.error(`Failed to create OAuth account for user ${userId} with provider ${providerId}:`, error);
                return ctx.adapter.create({
                    // Try creating without optional fields if the first attempt failed
                    model: "account",
                    data: {
                        id: `${auth0User.user_id}|${identity.provider}|${identity.user_id}`,
                        userId: userId,
                        password: migratePassword(auth0User),
                        providerId: providerId,
                        accountId: identity.user_id,
                        accessToken: identity.access_token,
                        tokenType: identity.token_type,
                        refreshToken: identity.refresh_token,
                        accessTokenExpiresAt: identity.expires_in ? new Date(Date.now() + identity.expires_in * 1000) : undefined,
                        refreshTokenExpiresAt: identity.refresh_token_expires_in ? new Date(Date.now() + identity.refresh_token_expires_in * 1000) : undefined,
                        scope: identity.scope,
                        idToken: identity.id_token,
                        createdAt: safeDateConversion(auth0User.created_at),
                        updatedAt: safeDateConversion(auth0User.updated_at)
                    },
                    forceAllowId: true
                });
            });

            console.log(`Successfully migrated OAuth account for user ${userId} with provider ${providerId}`);
        } catch (error) {
            console.error(`Failed to migrate OAuth account for user ${userId}:`, error);
        }
    }
}
```



```
async function migrateOrganizations(ctx: any) {
    try {
        const organizations = await auth0Client.organizations.getAll();
        for (const org of organizations.data || []) {
            try {
                await ctx.adapter.create({
                    model: "organization",
                    data: {
                        id: org.id,
                        name: org.display_name || org.id,
                        slug: (org.display_name || org.id).toLowerCase().replace(/[^a-z0-9]/g, '-'),
                        logo: org.branding?.logo_url,
                        metadata: JSON.stringify(org.metadata || {}),
                        createdAt: safeDateConversion(org.created_at),
                    },
                    forceAllowId: true
                });
                const members = await auth0Client.organizations.getMembers({ id: org.id });
                for (const member of members.data || []) {
                    try {
                        const userRoles = await auth0Client.organizations.getMemberRoles({
                            id: org.id,
                            user_id: member.user_id
                        });
                        const role = mapAuth0RoleToBetterAuthRole(userRoles.data?.map(r => r.name) || []);
                        await ctx.adapter.create({
                            model: "member",
                            data: {
                                id: `${org.id}|${member.user_id}`,
                                organizationId: org.id,
                                userId: member.user_id,
                                role: role,
                                createdAt: new Date()
                            },
                            forceAllowId: true
                        });

                        console.log(`Successfully migrated member ${member.user_id} for organization ${org.display_name || org.id}`);
                    } catch (error) {
                        console.error(`Failed to migrate member ${member.user_id} for organization ${org.display_name || org.id}:`, error);
                    }
                }

                console.log(`Successfully migrated organization: ${org.display_name || org.id}`);
            } catch (error) {
                console.error(`Failed to migrate organization ${org.display_name || org.id}:`, error);
            }
        }
        console.log('Organization migration completed');
    } catch (error) {
        console.error('Failed to migrate organizations:', error);
    }
}

async function migrateFromAuth0() {
    try {
        const ctx = await auth.$context;
        const isAdminEnabled = ctx.options?.plugins?.find(plugin => plugin.id === "admin");
        const isUsernameEnabled = ctx.options?.plugins?.find(plugin => plugin.id === "username");
        const isOrganizationEnabled = ctx.options?.plugins?.find(plugin => plugin.id === "organization");
        const perPage = 100;
        const auth0Users: any[] = [];
        let pageNumber = 0;

        while (true) {
            try {
                const params = {
                    per_page: perPage,
                    page: pageNumber,
                    include_totals: true,
                };
                const response = (await auth0Client.users.getAll(params)).data as any;
                const users = response.users || [];
                if (users.length === 0) break;
                auth0Users.push(...users);
                pageNumber++;

                if (users.length < perPage) break;
            } catch (error) {
                console.error('Error fetching users:', error);
                break;
            }
        }


        console.log(`Found ${auth0Users.length} users to migrate`);

        for (const auth0User of auth0Users) {
            try {
                // Determine if this is a password-based or OAuth user
                const isOAuthUser = auth0User.identities?.some((identity: any) => identity.provider !== 'auth0');
                // Base user data that's common for both types
                const baseUserData = {
                    id: auth0User.user_id,
                    email: auth0User.email,
                    emailVerified: auth0User.email_verified || false,
                    name: auth0User.name || auth0User.nickname,
                    image: auth0User.picture,
                    createdAt: safeDateConversion(auth0User.created_at),
                    updatedAt: safeDateConversion(auth0User.updated_at),
                    ...(isAdminEnabled ? {
                        banned: auth0User.blocked || false,
                        role: mapAuth0RoleToBetterAuthRole(auth0User.roles || []),
                    } : {}),

                    ...(isUsernameEnabled ? {
                        username: auth0User.username || auth0User.nickname,
                    } : {}),

                };

                const createdUser = await ctx.adapter.create({
                    model: "user",
                    data: {
                        ...baseUserData,
                    },
                    forceAllowId: true
                });

                if (!createdUser?.id) {
                    throw new Error('Failed to create user');
                }
```

~~~
                    await migrateOAuthAccounts(auth0User, createdUser.id, ctx)
                    console.log(`Successfully migrated user: ${auth0User.email}`);
                } catch (error) {
                    console.error(`Failed to migrate user ${auth0User.email}:`, error);
                }
            }
            if (isOrganizationEnabled) {
                await migrateOrganizations(ctx);
            }
            // the reset of migration will be here.
            console.log('Migration completed successfully');
        } catch (error) {
            console.error('Migration failed:', error);
            throw error;
        }
    }

    migrateFromAuth0()
        .then(() => {
            console.log('Migration completed');
            process.exit(0);
        })
        .catch((error) => {
            console.error('Migration failed:', error);
            process.exit(1);
        });
    ```

    Make sure to replace the Auth0 environment variables with your own values:

    * `AUTH0_DOMAIN`
    * `AUTH0_CLIENT_ID`
    * `AUTH0_SECRET`
  </Step>

  <Step>
    ### Run the migration

    Run the migration script:

    ```sh
    bun run scripts/migrate-auth0.ts # or use your preferred runtime
    ```

    <Callout type="warning">
      Important considerations:

      1. Test the migration in a development environment first
      2. Monitor the migration process for any errors
      3. Verify the migrated data in Better Auth before proceeding
      4. Keep Auth0 installed and configured until the migration is complete
      5. The script handles bcrypt password hashes by default. For custom password hashing algorithms, you'll need to modify the `migratePassword` function
    </Callout>
  </Step>

  <Step>
    ### Change password hashing algorithm

    By default, Better Auth uses the `scrypt` algorithm to hash passwords. Since Auth0 uses `bcrypt`, you'll need to configure Better Auth to use bcrypt for password verification.

    First, install bcrypt:

    ```bash
    npm install bcrypt
    npm install -D @types/bcrypt
    ```

    Then update your auth configuration:

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth";
    import bcrypt from "bcrypt";

    export const auth = betterAuth({
       emailAndPassword: {
           password: {
               hash: async (password) => {
                   return await bcrypt.hash(password, 10);
               },
               verify: async ({ hash, password }) => {
                   return await bcrypt.compare(password, hash);
               }
           }
       }
    })
    ```
  </Step>

  <Step>
    ### Verify the migration

    After running the migration, verify that:

    1. All users have been properly migrated
    2. Social connections are working
    3. Password-based authentication is working
    4. Two-factor authentication settings are preserved (if enabled)
    5. User roles and permissions are correctly mapped
  </Step>

  <Step>
    ### Update your components

    Now that the data is migrated, update your components to use Better Auth. Here's an example for the sign-in component:

    ```tsx title="components/auth/sign-in.tsx"
    import { authClient } from "better-auth/client";

    export const SignIn = () => {
      const handleSignIn = async () => {
        const { data, error } = await authClient.signIn.email({
          email: "helloworld@gmail.com",
          password: "helloworld",
        });

        if (error) {
          console.error(error);
          return;
        }
        // Handle successful sign in
      };

      return (
        <form onSubmit={handleSignIn}>
          <button type="submit">Sign in</button>
        </form>
      );
    };
    ```
  </Step>
~~~

  <Step>
    ### Update the middleware

    Replace your Auth0 middleware with Better Auth's middleware:

    ```ts title="middleware.ts"
    import { NextRequest, NextResponse } from "next/server";
    import { getSessionCookie } from "better-auth/cookies";

    export async function middleware(request: NextRequest) {
      const sessionCookie = getSessionCookie(request);
      const { pathname } = request.nextUrl;

      if (sessionCookie && ["/login", "/signup"].includes(pathname)) {
        return NextResponse.redirect(new URL("/dashboard", request.url));
      }

      if (!sessionCookie && pathname.startsWith("/dashboard")) {
        return NextResponse.redirect(new URL("/login", request.url));
      }

      return NextResponse.next();
    }

    export const config = {
      matcher: ["/dashboard", "/login", "/signup"],
    };
    ```
  </Step>

  <Step>
    ### Remove Auth0 Dependencies

    Once you've verified everything is working correctly with Better Auth, remove Auth0:

    ```bash
    npm remove @auth0/auth0-react @auth0/auth0-spa-js @auth0/nextjs-auth0
    ```
  </Step>
</Steps>

## Additional Considerations

### Password Migration

The migration script handles bcrypt password hashes by default. If you're using custom password hashing algorithms in Auth0, you'll need to modify the `migratePassword` function in the migration script to handle your specific case.

### Role Mapping

The script includes a basic role mapping function (`mapAuth0RoleToBetterAuthRole`). Customize this function based on your Auth0 roles and Better Auth role requirements.

### Rate Limiting

The migration script includes pagination to handle large numbers of users. Adjust the `perPage` value based on your needs and Auth0's rate limits.

## Wrapping Up

Now! You've successfully migrated from Auth0 to Better Auth.

Better Auth offers greater flexibility and more featuresâ€”be sure to explore the [documentation](/docs) to unlock its full potential.

# guides: Browser Extension Guide
URL: /docs/guides/browser-extension-guide
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/guides/browser-extension-guide.mdx

A step-by-step guide to creating a browser extension with Better Auth.

***

title: Browser Extension Guide
description: A step-by-step guide to creating a browser extension with Better Auth.

In this guide, we'll walk you through the steps of creating a browser extension using <Link href="https://docs.plasmo.com/">Plasmo</Link> with Better Auth for authentication.

If you would like to view a completed example, you can check out the <Link href="https://github.com/better-auth/examples/tree/main/browser-extension-example">browser extension example</Link>.

<Callout type="warn">
  The Plasmo framework does not provide a backend for the browser extension.
  This guide assumes you have{" "}
  <Link href="/docs/integrations/hono">a backend setup</Link> of Better Auth and
  are ready to create a browser extension to connect to it.
</Callout>

<Steps>
  <Step>
    ## Setup & Installations

    Initialize a new Plasmo project with TailwindCSS and a src directory.

    ```bash
    pnpm create plasmo --with-tailwindcss --with-src
    ```

    Then, install the Better Auth package.

    ```bash
    pnpm add better-auth
    ```

    To start the Plasmo development server, run the following command.

    ```bash
    pnpm dev
    ```
  </Step>

  <Step>
    ## Configure tsconfig

    Configure the `tsconfig.json` file to include `strict` mode.

    For this demo, we have also changed the import alias from `~` to `@` and set it to the `src` directory.

    ```json title="tsconfig.json"
    {
        "compilerOptions": {
            "paths": {
                "@/_": [
                    "./src/_"
                ]
            },
            "strict": true,
            "baseUrl": "."
        }
    }
    ```

~~~
</Step>

<Step>
  ## Create the client auth instance

  Create a new file at `src/auth/auth-client.ts` and add the following code.

  <Files>
    <Folder name="src" defaultOpen>
      <Folder name="auth" defaultOpen>
        <File name="auth-client.ts" />
      </Folder>
    </Folder>
  </Files>

  ```ts title="auth-client.ts"
  import { createAuthClient } from "better-auth/react"

  export const authClient = createAuthClient({
      baseURL: "http://localhost:3000" /* Base URL of your Better Auth backend. */,
      plugins: [],
  });
  ```
</Step>

<Step>
  ## Configure the manifest

  We must ensure the extension knows the URL to the Better Auth backend.

  Head to your package.json file, and add the following code.

  ```json title="package.json"
  {
      //...
      "manifest": {
          "host_permissions": [
              "https://URL_TO_YOUR_BACKEND" // localhost works too (e.g. http://localhost:3000)
          ]
      }
  }
  ```
</Step>

<Step>
  ## You're now ready!

  You have now set up Better Auth for your browser extension.

  Add your desired UI and create your dream extension!

  To learn more about the client Better Auth API, check out the <Link href="/docs/concepts/client">client documentation</Link>.

  Here's a quick example ğŸ˜

  ```tsx title="src/popup.tsx"
  import { authClient } from "./auth/auth-client"


  function IndexPopup() {
      const {data, isPending, error} = authClient.useSession();
      if(isPending){
          return <>Loading...</>
      }
      if(error){
          return <>Error: {error.message}</>
      }
      if(data){
          return <>Signed in as {data.user.name}</>
      }
  }

  export default IndexPopup;
  ```
</Step>

<Step>
  ## Bundle your extension

  To get a production build, run the following command.

  ```bash
  pnpm build
  ```

  Head over to <Link href="chrome://extensions" target="_blank">chrome://extensions</Link> and enable developer mode.

  <img src="https://docs.plasmo.com/_next/image?url=%2F_next%2Fstatic%2Fmedia%2Fdeveloper_mode.76f090f7.png&w=1920&q=75" />

  Click on "Load Unpacked" and navigate to your extension's `build/chrome-mv3-dev` (or `build/chrome-mv3-prod`) directory.

  To see your popup, click on the puzzle piece icon on the Chrome toolbar, and click on your extension.

  Learn more about <Link href="https://docs.plasmo.com/framework#loading-the-extension-in-chrome">bundling your extension here.</Link>
</Step>

<Step>
  ## Configure the server auth instance

  First, we will need your extension URL.

  An extension URL formed like this: `chrome-extension://YOUR_EXTENSION_ID`.

  You can find your extension ID at <Link href="chrome://extensions" target="_blank">chrome://extensions</Link>.

  <img src="/extension-id.png" width={500} />

  Head to your server's auth file, and make sure that your extension's URL is added to the `trustedOrigins` list.

  ```ts title="server.ts"
  import { betterAuth } from "better-auth"
  import { auth } from "@/auth/auth"

  export const auth = betterAuth({
      trustedOrigins: ["chrome-extension://YOUR_EXTENSION_ID"],
  })
  ```

  If you're developing multiple extensions or need to support different browser extensions with different IDs, you can use wildcard patterns:

  ```ts title="server.ts"
  export const auth = betterAuth({
      trustedOrigins: [
          // Support a specific extension ID
          "chrome-extension://YOUR_EXTENSION_ID",

          // Or support multiple extensions with wildcard (less secure)
          "chrome-extension://*"
      ],
  })
  ```
~~~

~~~
    <Callout type="warn">
      Using wildcards for extension origins (`chrome-extension://*`) reduces security by trusting all extensions.
      It's safer to explicitly list each extension ID you trust. Only use wildcards for development and testing.
    </Callout>
  </Step>

  <Step>
    ## That's it!

    Everything is set up! You can now start developing your extension. ğŸ ‰
  </Step>
</Steps>

## Wrapping Up

Congratulations! You've successfully created a browser extension using Better Auth and Plasmo.
We highly recommend you visit the <Link href="https://docs.plasmo.com/">Plasmo documentation</Link> to learn more about the framework.

If you would like to view a completed example, you can check out the <Link href="https://github.com/better-auth/examples/tree/main/browser-extension-example">browser extension
example</Link>.

If you have any questions, feel free to open an issue on our <Link href="https://github.com/better-auth/better-auth/issues">GitHub repo</Link>, or join our <Link
href="https://discord.gg/better-auth">Discord server</Link> for support.



# guides: Migrating from Clerk to Better Auth
URL: /docs/guides/clerk-migration-guide
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/guides/clerk-migration-guide.mdx

A step-by-step guide to transitioning from Clerk to Better Auth.

***

title: Migrating from Clerk to Better Auth
description: A step-by-step guide to transitioning from Clerk to Better Auth.
-----------------------------------------------------------------------------

In this guide, we'll walk through the steps to migrate a project from Clerk to Better Auth â€” including email/password with proper hashing, social/external accounts, phone
number, two-factor data, and more.

<Callout type="warn">
  This migration will invalidate all active sessions. This guide doesn't currently show you how to migrate Organization but it should be possible with additional steps and the
[Organization](/docs/plugins/organization) Plugin.
</Callout>

## Before You Begin

Before starting the migration process, set up Better Auth in your project. Follow the [installation guide](/docs/installation) to get started. And go to

<Steps>
  <Step>
    ### Connect to your database

    You'll need to connect to your database to migrate the users and accounts. You can use any database you want, but for this example, we'll use PostgreSQL.

    <CodeBlockTabs defaultValue="npm">
      <CodeBlockTabsList>
        <CodeBlockTabsTrigger value="npm">
          npm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="pnpm">
          pnpm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="yarn">
          yarn
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="bun">
          bun
        </CodeBlockTabsTrigger>
      </CodeBlockTabsList>

      <CodeBlockTab value="npm">
        ```bash
        npm install pg
        ```
      </CodeBlockTab>

      <CodeBlockTab value="pnpm">
        ```bash
        pnpm add pg
        ```
      </CodeBlockTab>

      <CodeBlockTab value="yarn">
        ```bash
        yarn add pg
        ```
      </CodeBlockTab>

      <CodeBlockTab value="bun">
        ```bash
        bun add pg
        ```
      </CodeBlockTab>
    </CodeBlockTabs>

    And then you can use the following code to connect to your database.

    ```ts title="auth.ts"
    import { Pool } from "pg";

    export const auth = betterAuth({
        database: new Pool({
            connectionString: process.env.DATABASE_URL
        }),
    })
    ```
  </Step>

  <Step>
    ### Enable Email and Password (Optional)

    Enable the email and password in your auth config and implement your own logic for sending verification emails, reset password emails, etc.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth";
~~~

~~~mdx
    export const auth = betterAuth({
        database: new Pool({
            connectionString: process.env.DATABASE_URL
        }),
        emailAndPassword: { // [!code highlight]
            enabled: true, // [!code highlight]
        }, // [!code highlight]
        emailVerification: {
          sendVerificationEmail: async({ user, url })=>{
            // implement your logic here to send email verification
          }
        },
    })
    ```

    See [Email and Password](/docs/authentication/email-password) for more configuration options.
  </Step>

  <Step>
    ### Setup Social Providers (Optional)

    Add social providers you have enabled in your Clerk project in your auth config.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth";

    export const auth = betterAuth({
        database: new Pool({
            connectionString: process.env.DATABASE_URL
        }),
        emailAndPassword: {
            enabled: true,
        },
        socialProviders: { // [!code highlight]
            github: { // [!code highlight]
                clientId: process.env.GITHUB_CLIENT_ID, // [!code highlight]
                clientSecret: process.env.GITHUB_CLIENT_SECRET, // [!code highlight]
            } // [!code highlight]
        } // [!code highlight]
    })
    ```
  </Step>

  <Step>
    ### Add Plugins (Optional)

    You can add the following plugins to your auth config based on your needs.

    [Admin](/docs/plugins/admin) Plugin will allow you to manage users, user impersonations and app level roles and permissions.

    [Two Factor](/docs/plugins/2fa) Plugin will allow you to add two-factor authentication to your application.

    [Phone Number](/docs/plugins/phone-number) Plugin will allow you to add phone number authentication to your application.

    [Username](/docs/plugins/username) Plugin will allow you to add username authentication to your application.

    ```ts title="auth.ts"
    import { Pool } from "pg";
    import { betterAuth } from "better-auth";
    import { admin, twoFactor, phoneNumber, username } from "better-auth/plugins";

    export const auth = betterAuth({
        database: new Pool({
            connectionString: process.env.DATABASE_URL
        }),
        emailAndPassword: {
            enabled: true,
        },
        socialProviders: {
            github: {
                clientId: process.env.GITHUB_CLIENT_ID!,
                clientSecret: process.env.GITHUB_CLIENT_SECRET!,
            }
        },
        plugins: [admin(), twoFactor(), phoneNumber(), username()], // [!code highlight]
    })
    ```
  </Step>

  <Step>
    ### Generate Schema

    If you're using a custom database adapter, generate the schema:

    ```sh
    npx @better-auth/cli generate
    ```

    or if you're using the default adapter, you can use the following command:

    ```sh
    npx @better-auth/cli migrate
    ```
  </Step>

  <Step>
    ### Export Clerk Users

    Go to the Clerk dashboard and export the users. Check how to do it [here](https://clerk.com/docs/deployments/exporting-users#export-your-users-data-from-the-clerk-dashboard). It will download a CSV file with the users data. You need to save it as `exported_users.csv` and put it in the root of your project.
  </Step>

  <Step>
    ### Create the migration script

    Create a new file called `migrate-clerk.ts` in the `scripts` folder and add the following code:

    ```ts title="scripts/migrate-clerk.ts"
    import { generateRandomString, symmetricEncrypt } from "better-auth/crypto";

    import { auth } from "@/lib/auth"; // import your auth instance

    function getCSVData(csv: string) {
      const lines = csv.split('\n').filter(line => line.trim());
      const headers = lines[0]?.split(',').map(header => header.trim()) || [];
      const jsonData = lines.slice(1).map(line => {
          const values = line.split(',').map(value => value.trim());
          return headers.reduce((obj, header, index) => {
              obj[header] = values[index] || '';
              return obj;
          }, {} as Record<string, string>);
      });
~~~

```
      return jsonData as Array<{
          id: string;
          first_name: string;
          last_name: string;
          username: string;
          primary_email_address: string;
          primary_phone_number: string;
          verified_email_addresses: string;
          unverified_email_addresses: string;
          verified_phone_numbers: string;
          unverified_phone_numbers: string;
          totp_secret: string;
          password_digest: string;
          password_hasher: string;
      }>;
    }

    const exportedUserCSV = await Bun.file("exported_users.csv").text(); // this is the file you downloaded from Clerk

    async function getClerkUsers(totalUsers: number) {
      const clerkUsers: {
          id: string;
          first_name: string;
          last_name: string;
          username: string;
          image_url: string;
          password_enabled: boolean;
          two_factor_enabled: boolean;
          totp_enabled: boolean;
          backup_code_enabled: boolean;
          banned: boolean;
          locked: boolean;
          lockout_expires_in_seconds: number;
          created_at: number;
          updated_at: number;
          external_accounts: {
              id: string;
              provider: string;
              identification_id: string;
              provider_user_id: string;
              approved_scopes: string;
              email_address: string;
              first_name: string;
              last_name: string;
              image_url: string;
              created_at: number;
              updated_at: number;
          }[]
      }[] = [];
      for (let i = 0; i < totalUsers; i += 500) {
          const response = await fetch(`https://api.clerk.com/v1/users?offset=${i}&limit=${500}`, {
              headers: {
                  'Authorization': `Bearer ${process.env.CLERK_SECRET_KEY}`
              }
          });
          if (!response.ok) {
              throw new Error(`Failed to fetch users: ${response.statusText}`);
          }
          const clerkUsersData = await response.json();
          // biome-ignore lint/suspicious/noExplicitAny: <explanation>
          clerkUsers.push(...clerkUsersData as any);
      }
      return clerkUsers;
    }


    export async function generateBackupCodes(
      secret: string,
    ) {
      const key = secret;
      const backupCodes = Array.from({ length: 10 })
          .fill(null)
          .map(() => generateRandomString(10, "a-z", "0-9", "A-Z"))
          .map((code) => `${code.slice(0, 5)}-${code.slice(5)}`);
      const encCodes = await symmetricEncrypt({
          data: JSON.stringify(backupCodes),
          key: key,
      });
      return encCodes
    }

    // Helper function to safely convert timestamp to Date
    function safeDateConversion(timestamp?: number): Date {
      if (!timestamp) return new Date();

      // Convert seconds to milliseconds
      const date = new Date(timestamp * 1000);

      // Check if the date is valid
      if (isNaN(date.getTime())) {
          console.warn(`Invalid timestamp: ${timestamp}, falling back to current date`);
          return new Date();
      }

      // Check for unreasonable dates (before 2000 or after 2100)
      const year = date.getFullYear();
      if (year < 2000 || year > 2100) {
          console.warn(`Suspicious date year: ${year}, falling back to current date`);
          return new Date();
      }

      return date;
    }

    async function migrateFromClerk() {
      const jsonData = getCSVData(exportedUserCSV);
      const clerkUsers = await getClerkUsers(jsonData.length);
      const ctx = await auth.$context
      const isAdminEnabled = ctx.options?.plugins?.find(plugin => plugin.id === "admin");
      const isTwoFactorEnabled = ctx.options?.plugins?.find(plugin => plugin.id === "two-factor");
      const isUsernameEnabled = ctx.options?.plugins?.find(plugin => plugin.id === "username");
      const isPhoneNumberEnabled = ctx.options?.plugins?.find(plugin => plugin.id === "phone-number");
      for (const user of jsonData) {
          const { id, first_name, last_name, username, primary_email_address, primary_phone_number, verified_email_addresses, unverified_email_addresses, verified_phone_numbers,
unverified_phone_numbers, totp_secret, password_digest, password_hasher } = user;
          const clerkUser = clerkUsers.find(clerkUser => clerkUser?.id === id);
```

~~~
          // create user
          const createdUser = await ctx.adapter.create<{
              id: string;
          }>({
              model: "user",
              data: {
                  id,
                  email: primary_email_address,
                  emailVerified: verified_email_addresses.length > 0,
                  name: `${first_name} ${last_name}`,
                  image: clerkUser?.image_url,
                  createdAt: safeDateConversion(clerkUser?.created_at),
                  updatedAt: safeDateConversion(clerkUser?.updated_at),
                  // # Two Factor (if you enabled two factor plugin)
                  ...(isTwoFactorEnabled ? {
                      twoFactorEnabled: clerkUser?.two_factor_enabled
                  } : {}),
                  // # Admin (if you enabled admin plugin)
                  ...(isAdminEnabled ? {
                      banned: clerkUser?.banned,
                      banExpiresAt: clerkUser?.lockout_expires_in_seconds,
                      role: "user"
                  } : {}),
                  // # Username (if you enabled username plugin)
                  ...(isUsernameEnabled ? {
                      username: username,
                  } : {}),
                  // # Phone Number (if you enabled phone number plugin)
                  ...(isPhoneNumberEnabled ? {
                      phoneNumber: primary_phone_number,
                      phoneNumberVerified: verified_phone_numbers.length > 0,
                  } : {}),
              },
              forceAllowId: true
          }).catch(async e => {
              return await ctx.adapter.findOne<{
                  id: string;
              }>({
                  model: "user",
                  where: [{
                      field: "id",
                      value: id
                  }]
              })
          })
          // create external account
          const externalAccounts = clerkUser?.external_accounts;
          if (externalAccounts) {
              for (const externalAccount of externalAccounts) {
                  const { id, provider, identification_id, provider_user_id, approved_scopes, email_address, first_name, last_name, image_url, created_at, updated_at } = externalAccount;
                  if (externalAccount.provider === "credential") {
                      await ctx.adapter.create({
                          model: "account",
                          data: {
                              id,
                              providerId: provider,
                              accountId: externalAccount.provider_user_id,
                              scope: approved_scopes,
                              userId: createdUser?.id,
                              createdAt: safeDateConversion(created_at),
                              updatedAt: safeDateConversion(updated_at),
                              password: password_digest,
                          }
                      })
                  } else {
                      await ctx.adapter.create({
                          model: "account",
                          data: {
                              id,
                              providerId: provider.replace("oauth_", ""),
                              accountId: externalAccount.provider_user_id,
                              scope: approved_scopes,
                              userId: createdUser?.id,
                              createdAt: safeDateConversion(created_at),
                              updatedAt: safeDateConversion(updated_at),
                          },
                          forceAllowId: true
                      })
                  }
              }
          }

          //two factor
          if (isTwoFactorEnabled) {
              await ctx.adapter.create({
                  model: "twoFactor",
                  data: {
                      userId: createdUser?.id,
                      secret: totp_secret,
                      backupCodes: await generateBackupCodes(totp_secret)
                  }
              })
          }
      }
    }

    migrateFromClerk()
      .then(() => {
          console.log('Migration completed');
          process.exit(0);
      })
      .catch((error) => {
          console.error('Migration failed:', error);
          process.exit(1);
      });
    ```

    Make sure to replace the `process.env.CLERK_SECRET_KEY` with your own Clerk secret key. Feel free to customize the script to your needs.
  </Step>

  <Step>
    ### Run the migration

    Run the migration:

    ```sh
    bun run script/migrate-clerk.ts # you can use any thing you like to run the script
    ```

    <Callout type="warning">
      Make sure to:

      1. Test the migration in a development environment first
      2. Monitor the migration process for any errors
      3. Verify the migrated data in Better Auth before proceeding
      4. Keep Clerk installed and configured until the migration is complete
    </Callout>
  </Step>
~~~

~~~
  <Step>
    ### Verify the migration

    After running the migration, verify that all users have been properly migrated by checking the database.
  </Step>

  <Step>
    ### Update your components

    Now that the data is migrated, you can start updating your components to use Better Auth. Here's an example for the sign-in component:

    ```tsx title="components/auth/sign-in.tsx"
    import { authClient } from "better-auth/client";

    export const SignIn = () => {
      const handleSignIn = async () => {
        const { data, error } = await authClient.signIn.email({
          email: "user@example.com",
          password: "password",
        });

        if (error) {
          console.error(error);
          return;
        }
        // Handle successful sign in
      };

      return (
        <form onSubmit={handleSignIn}>
          <button type="submit">Sign in</button>
        </form>
      );
    };
    ```
  </Step>

  <Step>
    ### Update the middleware

    Replace your Clerk middleware with Better Auth's middleware:

    ```ts title="middleware.ts"

    import { NextRequest, NextResponse } from "next/server";
    import { getSessionCookie } from "better-auth/cookies";
    export async function middleware(request: NextRequest) {
      const sessionCookie = getSessionCookie(request);
      const { pathname } = request.nextUrl;
      if (sessionCookie && ["/login", "/signup"].includes(pathname)) {
        return NextResponse.redirect(new URL("/dashboard", request.url));
      }
      if (!sessionCookie && pathname.startsWith("/dashboard")) {
        return NextResponse.redirect(new URL("/login", request.url));
      }
      return NextResponse.next();
    }

    export const config = {
      matcher: ["/dashboard", "/login", "/signup"],
    };
    ```
  </Step>

  <Step>
    ### Remove Clerk Dependencies

    Once you've verified that everything is working correctly with Better Auth, you can remove Clerk:

    ```bash title="Remove Clerk"
    pnpm remove @clerk/nextjs @clerk/themes @clerk/types
    ```
  </Step>
</Steps>

## Additional Resources

[Goodbye Clerk, Hello Better Auth â€“ Full Migration Guide!](https://www.youtube.com/watch?v=Za_QihbDSuk)

## Wrapping Up

Congratulations! You've successfully migrated from Clerk to Better Auth.

Better Auth offers greater flexibility and more featuresâ€”be sure to explore the [documentation](/docs) to unlock its full potential.



# guides: Create a Database Adapter
URL: /docs/guides/create-a-db-adapter
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/guides/create-a-db-adapter.mdx

Learn how to create a custom database adapter for Better-Auth

***

title: Create a Database Adapter
description: Learn how to create a custom database adapter for Better-Auth
--------------------------------------------------------------------------

Learn how to create a custom database adapter for Better-Auth using `createAdapter`.

Our `createAdapter` function is designed to be very flexible, and we've done our best to make it easy to understand and use.
Our hope is to allow you to focus on writing database logic, and not have to worry about how the adapter is working with Better-Auth.

Anything from custom schema configurations, custom ID generation, safe JSON parsing, and more is handled by the `createAdapter` function.
All you need to do is provide the database logic, and the `createAdapter` function will handle the rest.

## Quick Start

<Steps>
  <Step>
    ### Get things ready

    1. Import `createAdapter`.
    2. Create `CustomAdapterConfig` interface that represents your adapter config options.
    3. Create the adapter!

    ```ts
    import { createAdapter, type DBAdapterDebugLogOption } from "better-auth/adapters";

    // Your custom adapter config options
    interface CustomAdapterConfig {
      /**
       * Helps you debug issues with the adapter.
       */
      debugLogs?: DBAdapterDebugLogOption;
      /**
       * If the table names in the schema are plural.
       */
      usePlural?: boolean;
    }
~~~

~~~mdx
    export const myAdapter = (config: CustomAdapterConfig = {}) =>
      createAdapter({
        // ...
      });
    ```
  </Step>

  <Step>
    ### Configure the adapter

    The `config` object is mostly used to provide information about the adapter to Better-Auth.
    We try to minimize the amount of code you need to write in your adapter functions, and these `config` options are used to help us do that.

    ```ts
    // ...
    export const myAdapter = (config: CustomAdapterConfig = {}) =>
      createAdapter({
        config: {
          adapterId: "custom-adapter", // A unique identifier for the adapter.
          adapterName: "Custom Adapter", // The name of the adapter.
          usePlural: config.usePlural ?? false, // Whether the table names in the schema are plural.
          debugLogs: config.debugLogs ?? false, // Whether to enable debug logs.
          supportsJSON: false, // Whether the database supports JSON. (Default: false)
          supportsDates: true, // Whether the database supports dates. (Default: true)
          supportsBooleans: true, // Whether the database supports booleans. (Default: true)
          supportsNumericIds: true, // Whether the database supports auto-incrementing numeric IDs. (Default: true)
        },
        // ...
      });
    ```
  </Step>

  <Step>
    ### Create the adapter

    The `adapter` function is where you write the code that interacts with your database.

    ```ts
    // ...
    export const myAdapter = (config: CustomAdapterConfig = {}) =>
      createAdapter({
        config: {
          // ...
        },
        adapter: ({}) => {
          return {
            create: async ({ data, model, select }) => {
              // ...
            },
            update: async ({ data, model, select }) => {
              // ...
            },
            updateMany: async ({ data, model, select }) => {
              // ...
            },
            delete: async ({ data, model, select }) => {
              // ...
            },
            // ...
          };
        },
      });
    ```

    <Callout>
      Learn more about the `adapter` here [here](/docs/concepts/database#adapters).
    </Callout>
  </Step>
</Steps>

## Adapter

The `adapter` function is where you write the code that interacts with your database.

If you haven't already, check out the `options` object in the [config section](#config), as it can be useful for your adapter.

Before we get into the adapter function, let's go over the parameters that are available to you.

* `options`: The Better Auth options.
* `schema`: The schema from the user's Better Auth instance.
* `debugLog`: The debug log function.
* `getField`: The get field function.
* `getDefaultModelName`: The get default model name function.
* `getDefaultFieldName`: The get default field name function.
* `getFieldAttributes`: The get field attributes function.

```ts title="Example"
adapter: ({
  options,
  schema,
  debugLog,
  getField,
  getDefaultModelName,
  getDefaultFieldName,
}) => {
  return {
    // ...
  };
};
```

### Adapter Methods

* All `model` values are already transformed into the correct model name for the database based on the end-user's schema configuration.
  * This also means that if you need access to the `schema` version of a given model, you can't use this exact `model` value, you'll need to use the `getDefaultModelName` function provided in the options to convert the `model` to the `schema` version.
* We will automatically fill in any missing fields you return based on the user's `schema` configuration.
* Any method that includes a `select` parameter, is only for the purpose of getting data from your database more efficiently. You do not need to worry about only returning what the `select` parameter states, as we will handle that for you.

### `create` method

The `create` method is used to create a new record in the database.

<Callout>
  Note:
  If the user has enabled the `useNumberId` option, or if `generateId` is `false` in the user's Better Auth config,
  then it's expected that the `id` is provided in the `data` object. Otherwise, the `id` will be automatically generated.

  Additionally, it's possible to pass `forceAllowId` as a parameter to the `create` method, which allows `id` to be provided in the `data` object.
  We handle `forceAllowId` internally, so you don't need to worry about it.
</Callout>

parameters:

* `model`: The model/table name that new data will be inserted into.
* `data`: The data to insert into the database.
* `select`: An array of fields to return from the database.

<Callout>
~~~

~~~mdx
  Make sure to return the data that is inserted into the database.
</Callout>

```ts title="Example"
create: async ({ model, data, select }) => {
  // Example of inserting data into the database.
  return await db.insert(model).values(data);
};
```

### `update` method

The `update` method is used to update a record in the database.

parameters:

* `model`: The model/table name that the record will be updated in.
* `where`: The `where` clause to update the record by.
* `update`: The data to update the record with.

<Callout>
  Make sure to return the data in the row which is updated. This includes any
  fields that were not updated.
</Callout>

```ts title="Example"
update: async ({ model, where, update }) => {
  // Example of updating data in the database.
  return await db.update(model).set(update).where(where);
};
```

### `updateMany` method

The `updateMany` method is used to update multiple records in the database.

parameters:

* `model`: The model/table name that the records will be updated in.
* `where`: The `where` clause to update the records by.
* `update`: The data to update the records with.

<Callout>Make sure to return the number of records that were updated.</Callout>

```ts title="Example"
updateMany: async ({ model, where, update }) => {
  // Example of updating multiple records in the database.
  return await db.update(model).set(update).where(where);
};
```

### `delete` method

The `delete` method is used to delete a record from the database.

parameters:

* `model`: The model/table name that the record will be deleted from.
* `where`: The `where` clause to delete the record by.

```ts title="Example"
delete: async ({ model, where }) => {
  // Example of deleting a record from the database.
  await db.delete(model).where(where);
}
```

### `deleteMany` method

The `deleteMany` method is used to delete multiple records from the database.

parameters:

* `model`: The model/table name that the records will be deleted from.
* `where`: The `where` clause to delete the records by.

<Callout>Make sure to return the number of records that were deleted.</Callout>

```ts title="Example"
deleteMany: async ({ model, where }) => {
  // Example of deleting multiple records from the database.
  return await db.delete(model).where(where);
};
```

### `findOne` method

The `findOne` method is used to find a single record in the database.

parameters:

* `model`: The model/table name that the record will be found in.
* `where`: The `where` clause to find the record by.
* `select`: The `select` clause to return.

<Callout>Make sure to return the data that is found in the database.</Callout>

```ts title="Example"
findOne: async ({ model, where, select }) => {
  // Example of finding a single record in the database.
  return await db.select().from(model).where(where).limit(1);
};
```

### `findMany` method

The `findMany` method is used to find multiple records in the database.

parameters:

* `model`: The model/table name that the records will be found in.
* `where`: The `where` clause to find the records by.
* `limit`: The limit of records to return.
* `sortBy`: The `sortBy` clause to sort the records by.
* `offset`: The offset of records to return.

<Callout>
  Make sure to return the array of data that is found in the database.
</Callout>

```ts title="Example"
findMany: async ({ model, where, limit, sortBy, offset }) => {
  // Example of finding multiple records in the database.
  return await db
    .select()
    .from(model)
    .where(where)
    .limit(limit)
    .offset(offset)
    .orderBy(sortBy);
};
```
~~~

### `count` method

The `count` method is used to count the number of records in the database.

parameters:

* `model`: The model/table name that the records will be counted in.
* `where`: The `where` clause to count the records by.

<Callout>Make sure to return the number of records that were counted.</Callout>

```ts title="Example"
count: async ({ model, where }) => {
  // Example of counting the number of records in the database.
  return await db.select().from(model).where(where).count();
};
```

### `options` (optional)

The `options` object is for any potential config that you got from your custom adapter options.

```ts title="Example"
const myAdapter = (config: CustomAdapterConfig) =>
  createAdapter({
    config: {
      // ...
    },
    adapter: ({ options }) => {
      return {
        options: config,
      };
    },
  });
```

### `createSchema` (optional)

The `createSchema` method allows the [Better Auth CLI](/docs/concepts/cli) to [generate](/docs/concepts/cli/#generate) a schema for the database.

parameters:

* `tables`: The tables from the user's Better-Auth instance schema; which is expected to be generated into the schema file.
* `file`: The file the user may have passed in to the `generate` command as the expected schema file output path.

```ts title="Example"
createSchema: async ({ file, tables }) => {
  // ... Custom logic to create a schema for the database.
};
```

## Test your adapter

We've provided a test suite that you can use to test your adapter. It requires you to use `vitest`.

```ts title="my-adapter.test.ts"
import { expect, test, describe } from "vitest";
import { runAdapterTest } from "better-auth/adapters/test";
import { myAdapter } from "./my-adapter";

describe("My Adapter Tests", async () => {
  afterAll(async () => {
    // Run DB cleanup here...
  });
  const adapter = myAdapter({
    debugLogs: {
      // If your adapter config allows passing in debug logs, then pass this here.
      isRunningAdapterTests: true, // This is our super secret flag to let us know to only log debug logs if a test fails.
    },
  });

  await runAdapterTest({
    getAdapter: async (betterAuthOptions = {}) => {
      return adapter(betterAuthOptions);
    },
  });
});
```

### Numeric ID tests

If your database supports numeric IDs, then you should run this test as well:

```ts title="my-adapter.number-id.test.ts"
import { expect, test, describe } from "vitest";
import { runNumberIdAdapterTest } from "better-auth/adapters/test";
import { myAdapter } from "./my-adapter";

describe("My Adapter Numeric ID Tests", async () => {
  afterAll(async () => {
    // Run DB cleanup here...
  });
  const adapter = myAdapter({
    debugLogs: {
      // If your adapter config allows passing in debug logs, then pass this here.
      isRunningAdapterTests: true, // This is our super secret flag to let us know to only log debug logs if a test fails.
    },
  });

  await runNumberIdAdapterTest({
    getAdapter: async (betterAuthOptions = {}) => {
      return adapter(betterAuthOptions);
    },
  });
});
```

## Config

The `config` object is used to provide information about the adapter to Better-Auth.

We **highly recommend** going through and reading each provided option below, as it will help you understand how to properly configure your adapter.

### Required Config

### `adapterId`

A unique identifier for the adapter.

### `adapterName`

The name of the adapter.

### Optional Config

### `supportsNumericIds`

Whether the database supports numeric IDs. If this is set to `false` and the user's config has enabled `useNumberId`, then we will throw an error.

### `supportsJSON`

Whether the database supports JSON. If the database doesn't support JSON, we will use a `string` to save the JSON data.And when we retrieve the data, we will safely parse the `string` back into a JSON object.

### `supportsDates`

Whether the database supports dates. If the database doesn't support dates, we will use a `string` to save the date. (ISO string) When we retrieve the data, we will safely parse the `string` back into a `Date` object.

### `supportsBooleans`

Whether the database supports booleans. If the database doesn't support booleans, we will use a `0` or `1` to save the boolean value. When we retrieve the data, we will safely parse the `0` or `1` back into a boolean value.

### `usePlural`

Whether the table names in the schema are plural. This is often defined by the user, and passed down through your custom adapter options. If you do not intend to allow the user to customize the table names, you can ignore this option, or set this to `false`.

```ts title="Example"
const adapter = myAdapter({
  // This value then gets passed into the `usePlural`
  // option in the createAdapter `config` object.
  usePlural: true,
});
```

### `transaction`

Whether the adapter supports transactions. If `false`, operations run sequentially; otherwise provide a function that executes a callback with a `TransactionAdapter`.

<Callout type="warn">
  If your database does not support transactions, the error handling and rollback
  will not be as robust. We recommend using a database that supports transactions
  for better data integrity.
</Callout>

### `debugLogs`

Used to enable debug logs for the adapter. You can pass in a boolean, or an object with the following keys: `create`, `update`, `updateMany`, `findOne`, `findMany`, `delete`, `deleteMany`, `count`.
If any of the keys are `true`, the debug logs will be enabled for that method.

```ts title="Example"
// Will log debug logs for all methods.
const adapter = myAdapter({
  debugLogs: true,
});
```

```ts title="Example"
// Will only log debug logs for the `create` and `update` methods.
const adapter = myAdapter({
  debugLogs: {
    create: true,
    update: true,
  },
});
```

### `disableIdGeneration`

Whether to disable ID generation. If this is set to `true`, then the user's `generateId` option will be ignored.

### `customIdGenerator`

If your database only supports a specific custom ID generation, then you can use this option to generate your own IDs.

### `mapKeysTransformInput`

If your database uses a different key name for a given situation, you can use this option to map the keys. This is useful for databases that expect a different key name for a given situation.
For example, MongoDB uses `_id` while in Better-Auth we use `id`.

Each key in the returned object represents the old key to replace.
The value represents the new key.

This can be a partial object that only transforms some keys.

```ts title="Example"
mapKeysTransformInput: () => {
  return {
    id: "_id", // We want to replace `id` to `_id` to save into MongoDB
  };
},
```

### `mapKeysTransformOutput`

If your database uses a different key name for a given situation, you can use this option to map the keys. This is useful for databases that use a different key name for a given situation.
For example, MongoDB uses `_id` while in Better-Auth we use `id`.

Each key in the returned object represents the old key to replace.
The value represents the new key.

This can be a partial object that only transforms some keys.

```ts title="Example"
mapKeysTransformOutput: () => {
  return {
    _id: "id", // We want to replace `_id` (from MongoDB) to `id` (for Better-Auth)
  };
},
```

### `customTransformInput`

If you need to transform the input data before it is saved to the database, you can use this option to transform the data.

<Callout type="warn">
  If you're using `supportsJSON`, `supportsDates`, or `supportsBooleans`, then
  the transformations will be applied before your `customTransformInput`
  function is called.
</Callout>

The `customTransformInput` function receives the following arguments:

* `data`: The data to transform.
* `field`: The field that is being transformed.
* `fieldAttributes`: The field attributes of the field that is being transformed.
* `select`: The `select` values which the query expects to return.
* `model`: The model that is being transformed.
* `schema`: The schema that is being transformed.
* `options`: Better Auth options.

The `customTransformInput` function runs at every key in the data object of a given action.

```ts title="Example"
customTransformInput: ({ field, data }) => {
  if (field === "id") {
    return "123"; // Force the ID to be "123"
  }

  return data;
};
```

### `customTransformOutput`

If you need to transform the output data before it is returned to the user, you can use this option to transform the data. The `customTransformOutput` function is used to transform the output data.
Similar to the `customTransformInput` function, it runs at every key in the data object of a given action, but it runs after the data is retrieved from the database.

```ts title="Example"
customTransformOutput: ({ field, data }) => {
  if (field === "name") {
    return "Bob"; // Force the name to be "Bob"
  }

  return data;
};
```

```ts
const some_data = await adapter.create({
  model: "user",
  data: {
    name: "John",
  },
});

// The name will be "Bob"
console.log(some_data.name);
```

# guides: Migrating from NextAuth.js to Better Auth
URL: /docs/guides/next-auth-migration-guide
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/guides/next-auth-migration-guide.mdx

A step-by-step guide to transitioning from NextAuth.js to Better Auth.

***

title: Migrating from NextAuth.js to Better Auth
description: A step-by-step guide to transitioning from NextAuth.js to Better Auth.

In this guide, weâ€™ll walk through the steps to migrate a project from [NextAuth.js](https://authjs.dev/) to Better Auth, ensuring no loss of data or functionality. While this guide focuses on Next.js, it can be adapted for other frameworks as well.

***

## Before You Begin

Before starting the migration process, set up Better Auth in your project. Follow the [installation guide](/docs/installation) to get started.

***

<Steps>
  <Step>
    ### Mapping Existing Columns

    Instead of altering your existing database column names, you can map them to match Better Auth's expected structure. This allows you to retain your current database schema.

    #### User Schema

    Map the following fields in the user schema:

    * (next-auth v4) `emailVerified`: datetime â†’ boolean

    #### Session Schema

    Map the following fields in the session schema:

    * `expires` â†’ `expiresAt`
    * `sessionToken` â†’ `token`
    * (next-auth v4) add `createdAt` with datetime type
    * (next-auth v4) add `updatedAt` with datetime type

    ```typescript title="auth.ts"
    export const auth = betterAuth({
        // Other configs
        session: {
            fields: {
                expiresAt: "expires", // Map your existing `expires` field to Better Auth's `expiresAt`
                token: "sessionToken" // Map your existing `sessionToken` field to Better Auth's `token`
            }
        },
    });
    ```

    Make sure to have `createdAt` and `updatedAt` fields on your session schema.

    #### Account Schema

    Map these fields in the account schema:

    * (next-auth v4) `provider` â†’ `providerId`
    * `providerAccountId` â†’ `accountId`
    * `refresh_token` â†’ `refreshToken`
    * `access_token` â†’ `accessToken`
    * (next-auth v3) `access_token_expires` â†’ `accessTokenExpiresAt` and int â†’ datetime
    * (next-auth v4) `expires_at` â†’ `accessTokenExpiresAt` and int â†’ datetime
    * `id_token` â†’ `idToken`
    * (next-auth v4) add `createdAt` with datetime type
    * (next-auth v4) add `updatedAt` with datetime type

    Remove the `session_state`, `type`, and `token_type` fields, as they are not required by Better Auth.

    ```typescript title="auth.ts"
    export const auth = betterAuth({
        // Other configs
        account: {
            fields: {
                accountId: "providerAccountId",
                refreshToken: "refresh_token",
                accessToken: "access_token",
                accessTokenExpiresAt: "access_token_expires",
                idToken: "id_token",
            }
        },
    });
    ```

    Note: If you use ORM adapters, you can map these fields in your schema file.

    Example with Prisma:

    ```prisma title="schema.prisma"
    model Session {
        id          String   @id @default(cuid())
        expiresAt   DateTime @map("expires") // Map your existing `expires` field to Better Auth's `expiresAt`
        token       String   @map("sessionToken") // Map your existing `sessionToken` field to Better Auth's `token`
        userId      String
        user        User     @relation(fields: [userId], references: [id])
    }
    ```

    Make sure to have `createdAt` and `updatedAt` fields on your account schema.
  </Step>

  <Step>
    ### Update the Route Handler

    In the `app/api/auth` folder, rename the `[...nextauth]` file to `[...all]` to avoid confusion. Then, update the `route.ts` file as follows:

    ```typescript title="app/api/auth/[...all]/route.ts"
    import { toNextJsHandler } from "better-auth/next-js";
    import { auth } from "~/server/auth";

    export const { POST, GET } = toNextJsHandler(auth);
    ```
  </Step>

  <Step>
    ### Update the Client

    Create a file named `auth-client.ts` in the `lib` folder. Add the following code:

    ```typescript title="auth-client.ts"
    import { createAuthClient } from "better-auth/react";

    export const authClient = createAuthClient({
        baseURL: process.env.BASE_URL! // Optional if the API base URL matches the frontend
    });

    export const { signIn, signOut, useSession } = authClient;
    ```

    #### Social Login Functions

    Update your social login functions to use Better Auth. For example, for Discord:

    ```typescript
    import { signIn } from "~/lib/auth-client";

    export const signInDiscord = async () => {
        const data = await signIn.social({
            provider: "discord"
        });
        return data;
    };
    ```

    #### Update `useSession` Calls

    Replace `useSession` calls with Better Authâ€™s version. Example:

    ```typescript title="Profile.tsx"
    import { useSession } from "~/lib/auth-client";

    export const Profile = () => {
        const { data } = useSession();
        return (
            <div>
                <pre>
                    {JSON.stringify(data, null, 2)}
                </pre>
            </div>
        );
    };
    ```
  </Step>

  <Step>
    ### Server-Side Session Handling

    Use the `auth` instance to get session data on the server:

    ```typescript title="actions.ts"
    "use server";

    import { auth } from "~/server/auth";
    import { headers } from "next/headers";

    export const protectedAction = async () => {
        const session = await auth.api.getSession({
            headers: await headers(),
        });
    };
    ```
  </Step>

  <Step>
    ### Middleware

    To protect routes with middleware, refer to the [Next.js middleware guide](/docs/integrations/next#middleware).
  </Step>
</Steps>

## Wrapping Up

Congratulations! Youâ€™ve successfully migrated from NextAuth.js to Better Auth. For a complete implementation with multiple authentication methods, check out the [demo repository](https://github.com/Bekacru/t3-app-better-auth).

Better Auth offers greater flexibility and more featuresâ€”be sure to explore the [documentation](/docs) to unlock its full potential.

# guides: Optimizing for Performance
URL: /docs/guides/optimizing-for-performance
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/guides/optimizing-for-performance.mdx

A guide to optimizing your Better Auth application for performance.

***

title: Optimizing for Performance
description: A guide to optimizing your Better Auth application for performance.

In this guide, weâ€™ll go over some of the ways you can optimize your application for a more performant Better Auth app.

## Caching

Caching is a powerful technique that can significantly improve the performance of your Better Auth application by reducing the number of database queries and speeding up response times.

### Cookie Cache

Calling your database every time `useSession` or `getSession` is invoked isnâ€™t ideal, especially if sessions donâ€™t change frequently. Cookie caching handles this by storing session data in a short-lived, signed cookie similar to how JWT access tokens are used with refresh tokens.

To turn on cookie caching, just set `session.cookieCache` in your auth config:

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
  session: {
    cookieCache: {
      enabled: true,
      maxAge: 5 * 60, // Cache duration in seconds
    },
  },
});
```

Read more about [cookie caching](/docs/concepts/session-management#cookie-cache).

### Framework Caching

Here are examples of how you can do caching in different frameworks and environments:

<Tabs items={["Next", "Remix", "SolidStart", "React Query"]}>
  <Tab value="Next">
    Since Next v15, we can use the `"use cache"` directive to cache the response of a server function.

    ```ts
    export async function getUsers() {
        'use cache' // [!code highlight]
        const { users } = await auth.api.listUsers();
        return users
    }
    ```

    Learn more about NextJS use cache directive <Link href="https://nextjs.org/docs/app/api-reference/directives/use-cache">here</Link>.
  </Tab>

  <Tab value="Remix">
    In Remix, you can use the `cache` option in the `loader` function to cache responses on the server. Hereâ€™s an example:

    ```ts
    import { json } from '@remix-run/node';

    export const loader = async () => {
    const { users } = await auth.api.listUsers();
    return json(users, {
        headers: {
        'Cache-Control': 'max-age=3600', // Cache for 1 hour
        },
    });
    };
    ```

    You can read a nice guide on Loader vs Route Cache Headers in Remix <Link href="https://sergiodxa.com/articles/loader-vs-route-cache-headers-in-remix">here</Link>.
  </Tab>

  <Tab value="SolidStart">
    In SolidStart, you can use the `query` function to cache data. Hereâ€™s an example:

    ```tsx
    const getUsers = query(
        async () => (await auth.api.listUsers()).users,
        "getUsers"
    );
    ```

    Learn more about SolidStart `query` function <Link href="https://docs.solidjs.com/solid-router/reference/data-apis/query">here</Link>.
  </Tab>

  <Tab value="React Query">
    With React Query you can use the `useQuery` hook to cache data. Hereâ€™s an example:

    ```ts
    import { useQuery } from '@tanstack/react-query';

    const fetchUsers = async () => {
        const { users } = await auth.api.listUsers();
        return users;
    };

    export default function Users() {
        const { data: users, isLoading } = useQuery('users', fetchUsers, {
            staleTime: 1000 * 60 * 15, // Cache for 15 minutes
        });

        if (isLoading) return <div>Loading...</div>;

        return (
            <ul>
                {users.map(user => (
                    <li key={user.id}>{user.name}</li>
                ))}
            </ul>
        );
    }
    ```

    Learn more about React Query use cache directive <Link href="https://react-query.tanstack.com/reference/useQuery#usecache">here</Link>.
  </Tab>
</Tabs>

## SSR Optimizations

If you're using a framework that supports server-side rendering, it's usually best to pre-fetch the user session on the server and use it as a fallback on the client.

```ts
const session = await auth.api.getSession({
  headers: await headers(),
});
//then pass the session to the client
```

## Database optimizations

Optimizing database performance is essential to get the best out of Better Auth.

#### Recommended fields to index

Table	Fields	Plugin
users	`email`	
accounts	`userId`	
sessions	`userId`, `token`	
verifications	`identifier`	
invitations	`email`, `organizationId`	organization
members	`userId`, `organizationId`	organization
organizations	`slug`	organization
passkey	`userId`	passkey
twoFactor	`secret`	twoFactor

<Callout>
We intend to add indexing support in our schema generation tool in the future.
</Callout>

# guides: SAML SSO with Okta

URL: /docs/guides/saml-sso-with-okta
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/guides/saml-sso-with-okta.mdx

A guide to integrating SAML Single Sign-On (SSO) with Better Auth, featuring Okta

***

title: SAML SSO with Okta
description: A guide to integrating SAML Single Sign-On (SSO) with Better Auth, featuring Okta

---

This guide walks you through setting up SAML Single Sign-On (SSO) with your Identity Provider (IdP), using Okta as an example. For advanced configuration details and the full API reference, check out the [SSO Plugin Documentation](/docs/plugins/sso).

## What is SAML?

SAML (Security Assertion Markup Language) is an XML-based standard for exchanging authentication and authorization data between an Identity Provider (IdP) (e.g., Okta, Azure AD, OneLogin) and a Service Provider (SP) (in this case, Better Auth).

In this setup:

- **IdP (Okta)**: Authenticates users and sends assertions about their identity.
- **SP (Better Auth)**: Validates assertions and logs the user in.up.

### Step 1: Create a SAML Application in Okta

1. Log in to your Okta Admin Console

2. Navigate to Applications > Applications

3. Click "Create App Integration"

4. Select "SAML 2.0" as the Sign-in method

5. Configure the following settings:

   - **Single Sign-on URL**: Your Better Auth ACS endpoint (e.g., `http://localhost:3000/api/auth/sso/saml2/sp/acs/sso`). while `sso` being your providerId
   - **Audience URI (SP Entity ID)**: Your Better Auth metadata URL (e.g., `http://localhost:3000/api/auth/sso/saml2/sp/metadata`)
   - **Name ID format**: Email Address or any of your choice.

6. Download the IdP metadata XML file and certificate

### Step 2: Configure Better Auth

Hereâ€™s an example configuration for Okta in a dev environment:

```typescript
const ssoConfig = {
  defaultSSO: [{
    domain: "localhost:3000", // Your domain
    providerId: "sso",
    samlConfig: {
      // SP Configuration
      issuer: "http://localhost:3000/api/auth/sso/saml2/sp/metadata",
      entryPoint: "https://trial-1076874.okta.com/app/trial-1076874_samltest_1/exktofb0a62hqLAUL697/sso/saml",
      callbackUrl: "/dashboard", // Redirect after successful authentication

      // IdP Configuration
      idpMetadata: {
        entityID: "https://trial-1076874.okta.com/app/exktofb0a62hqLAUL697/sso/saml/metadata",
        singleSignOnService: [{
          Binding: "urn:oasis:names:tc:SAML:2.0:bindings:HTTP-Redirect",
          Location: "https://trial-1076874.okta.com/app/trial-1076874_samltest_1/exktofb0a62hqLAUL697/sso/saml"
        }],
        cert: `-----BEGIN CERTIFICATE-----
MIIDqjCCApKgAwIBAgIGAZhVGMeUMA0GCSqGSIb3DQEBCwUAMIGVMQswCQYDVQQGEwJVUzETMBEG
...
[Your Okta Certificate]
...
-----END CERTIFICATE-----`
      },

      // SP Metadata
      spMetadata: {
        metadata: `<md:EntityDescriptor xmlns:md="urn:oasis:names:tc:SAML:2.0:metadata"
          entityID="http://localhost:3000/api/sso/saml2/sp/metadata">
          ...
          [Your SP Metadata XML]
          ...
        </md:EntityDescriptor>`
      }
    }
  }]
}
```

### Step 3: Multiple Default Providers (Optional)

You can configure multiple SAML providers for different domains:

```typescript
const ssoConfig = {
  defaultSSO: [
    {
      domain: "company.com",
      providerId: "company-okta",
      samlConfig: {
        // Okta SAML configuration for company.com
      }
    },
    {
      domain: "partner.com",
      providerId: "partner-adfs",
      samlConfig: {
        // ADFS SAML configuration for partner.com
      }
    },
    {
      domain: "contractor.org",
      providerId: "contractor-azure",
      samlConfig: {
        // Azure AD SAML configuration for contractor.org
      }
    }
  ]
}
```

<Callout type="info">
  Explicit: Pass providerId directly when signing in.
  Domain fallback: Matches based on the userâ€™s email domain. e.g. user@company.com â†’ matches `company-okta` provider.
</Callout>

### Step 4: Initiating Sign-In

You can start an SSO flow in three ways:

1. Explicitly by `providerId` (recommended):

```typescript
// Explicitly specify which provider to use
await authClient.signIn.sso({
  providerId: "company-okta",
  callbackURL: "/dashboard"
});
```

2. By email domain matching:

```typescript
// Automatically matches provider based on email domain
await authClient.signIn.sso({
  email: "user@company.com",
  callbackURL: "/dashboard"
});
```

3. By specifying domain:

```typescript
// Explicitly specify domain for matching
await authClient.signIn.sso({
  domain: "partner.com",
  callbackURL: "/dashboard"
});
```

Important Notes:

* DummyIDP should ONLY be used for development and testing
* Never use these certificates in production
* The example uses `localhost:3000` - adjust URLs for your environment
* For production, always use proper IdP providers like Okta, Azure AD, or OneLogin

### Step 5: Dynamically Registering SAML Providers

For dynamic registration, you should register SAML providers using the API. See the [SSO Plugin Documentation](/docs/plugins/sso#register-a-saml-provider) for detailed registration instructions.

Example registration:

```typescript
await authClient.sso.register({
  providerId: "okta-prod",
  issuer: "https://your-domain.com",
  domain: "your-domain.com",
  samlConfig: {
    // Your production SAML configuration
  }
});
```

## Additional Resources

* [SSO Plugin Documentation](/docs/plugins/sso)
* [Okta SAML Documentation](https://developer.okta.com/docs/concepts/saml/)
* [SAML 2.0 Specification](https://docs.oasis-open.org/security/saml/v2.0/saml-core-2.0-os.pdf)

# guides: Migrating from Supabase Auth to Better Auth
URL: /docs/guides/supabase-migration-guide
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/guides/supabase-migration-guide.mdx

A step-by-step guide to transitioning from Supabase Auth to Better Auth.

***

title: Migrating from Supabase Auth to Better Auth
description: A step-by-step guide to transitioning from Supabase Auth to Better Auth.

In this guide, we'll walk through the steps to migrate a project from Supabase Auth to Better Auth.

<Callout type="warn">
  This migration will invalidate all active sessions. While this guide doesn't currently cover migrating two-factor (2FA) or Row Level Security (RLS) configurations, both should be possible with additional steps.
</Callout>

## Before You Begin

Before starting the migration process, set up Better Auth in your project. Follow the [installation guide](/docs/installation) to get started.

<Steps>
  <Step>
    ### Connect to your database

    You'll need to connect to your database to migrate the users and accounts. Copy your `DATABASE_URL` from your Supabase project and use it to connect to your database. And for this example, we'll need to install `pg` to connect to the database.

~~~
  <CodeBlockTabs defaultValue="npm">
    <CodeBlockTabsList>
      <CodeBlockTabsTrigger value="npm">
        npm
      </CodeBlockTabsTrigger>

      <CodeBlockTabsTrigger value="pnpm">
        pnpm
      </CodeBlockTabsTrigger>

      <CodeBlockTabsTrigger value="yarn">
        yarn
      </CodeBlockTabsTrigger>

      <CodeBlockTabsTrigger value="bun">
        bun
      </CodeBlockTabsTrigger>
    </CodeBlockTabsList>

    <CodeBlockTab value="npm">
      ```bash
      npm install pg
      ```
    </CodeBlockTab>

    <CodeBlockTab value="pnpm">
      ```bash
      pnpm add pg
      ```
    </CodeBlockTab>

    <CodeBlockTab value="yarn">
      ```bash
      yarn add pg
      ```
    </CodeBlockTab>

    <CodeBlockTab value="bun">
      ```bash
      bun add pg
      ```
    </CodeBlockTab>
  </CodeBlockTabs>

  And then you can use the following code to connect to your database.

  ```ts title="auth.ts"
  import { Pool } from "pg";

  export const auth = betterAuth({
      database: new Pool({
          connectionString: process.env.DATABASE_URL
      }),
  })
  ```
</Step>

<Step>
  ### Enable Email and Password (Optional)

  Enable the email and password in your auth config.

  ```ts title="auth.ts"
  import { admin, anonymous } from "better-auth/plugins";

  export const auth = betterAuth({
      database: new Pool({
          connectionString: process.env.DATABASE_URL
      }),
      emailVerification: {
              sendEmailVerification: async(user)=>{
                      // send email verification email
                      // implement your own logic here
              }
      },
      emailAndPassword: { // [!code highlight]
          enabled: true, // [!code highlight]
      } // [!code highlight]
  })
  ```
</Step>

<Step>
  ### Setup Social Providers (Optional)

  Add social providers you have enabled in your Supabase project in your auth config.

  ```ts title="auth.ts"
  import { admin, anonymous } from "better-auth/plugins";

  export const auth = betterAuth({
      database: new Pool({
          connectionString: process.env.DATABASE_URL
      }),
      emailAndPassword: {
          enabled: true,
      },
      socialProviders: { // [!code highlight]
          github: { // [!code highlight]
              clientId: process.env.GITHUB_CLIENT_ID, // [!code highlight]
              clientSecret: process.env.GITHUB_CLIENT_SECRET, // [!code highlight]
          } // [!code highlight]
      } // [!code highlight]
  })
  ```
</Step>

<Step>
  ### Add admin and anonymous plugins (Optional)

  Add the [admin](/docs/plugins/admin) and [anonymous](/docs/plugins/anonymous) plugins to your auth config.

  ```ts title="auth.ts"
  import { admin, anonymous } from "better-auth/plugins";

  export const auth = betterAuth({
      database: new Pool({
          connectionString: process.env.DATABASE_URL
      }),
      emailAndPassword: {
          enabled: true,
      },
      socialProviders: {
          github: {
              clientId: process.env.GITHUB_CLIENT_ID!,
              clientSecret: process.env.GITHUB_CLIENT_SECRET!,
          }
      },
      plugins: [admin(), anonymous()], // [!code highlight]
  })
  ```
~~~

~~~
</Step>

<Step>
  ### Run the migration

  Run the migration to create the necessary tables in your database.

  ```bash title="Terminal"
  npx @better-auth/cli migrate
  ```

  This will create the following tables in your database:

  * [`user`](/docs/concepts/database#user)
  * [`account`](/docs/concepts/database#account)
  * [`session`](/docs/concepts/database#session)
  * [`verification`](/docs/concepts/database#verification)

  This tables will be created on the `public` schema.
</Step>

<Step>
  ### Copy the migration script

  Now that we have the necessary tables in our database, we can run the migration script to migrate the users and accounts from Supabase to Better Auth.

  Start by creating a `.ts` file in your project.

  ```bash title="Terminal"
  touch migration.ts
  ```

  And then copy and paste the following code into the file.

  ```ts title="migration.ts"
  import { Pool } from "pg";
  import { auth } from "./auth";
  import { User as SupabaseUser } from "@supabase/supabase-js";

  type User = SupabaseUser & {
      is_super_admin: boolean;
      raw_user_meta_data: {
              avatar_url: string;
      };
      encrypted_password: string;
      email_confirmed_at: string;
      created_at: string;
      updated_at: string;
      is_anonymous: boolean;
      identities: {
              provider: string;
              identity_data: {
                      sub: string;
                      email: string;
              };
              created_at: string;
              updated_at: string;
      };
  };

  const migrateFromSupabase = async () => {
      const ctx = await auth.$context;
      const db = ctx.options.database as Pool;
      const users = await db
              .query(`
                      SELECT
                              u.*,
                              COALESCE(
                                      json_agg(
                                              i.* ORDER BY i.id
                                      ) FILTER (WHERE i.id IS NOT NULL),
                                      '[]'::json
                              ) as identities
                      FROM auth.users u
                      LEFT JOIN auth.identities i ON u.id = i.user_id
                      GROUP BY u.id
              `)
              .then((res) => res.rows as User[]);
      for (const user of users) {
              if (!user.email) {
                      continue;
              }
              await ctx.adapter
                      .create({
                              model: "user",
                              data: {
                                      id: user.id,
                                      email: user.email,
                                      name: user.email,
                                      role: user.is_super_admin ? "admin" : user.role,
                                      emailVerified: !!user.email_confirmed_at,
                                      image: user.raw_user_meta_data.avatar_url,
                                      createdAt: new Date(user.created_at),
                                      updatedAt: new Date(user.updated_at),
                                      isAnonymous: user.is_anonymous,
                              },
                      })
                      .catch(() => {});
              for (const identity of user.identities) {
                      const existingAccounts = await ctx.internalAdapter.findAccounts(user.id);

                      if (identity.provider === "email") {
                              const hasCredential = existingAccounts.find(
                                      (account) => account.providerId === "credential",
                              );
                              if (!hasCredential) {
                                      await ctx.adapter
                                              .create({
                                                      model: "account",
                                                      data: {
~~~

~~~
                                                                userId: user.id,
                                                                providerId: "credential",
                                                                accountId: user.id,
                                                                password: user.encrypted_password,
                                                                createdAt: new Date(user.created_at),
                                                                updatedAt: new Date(user.updated_at),
                                                        },
                                                })
                                                .catch(() => {});
                                }
                        }
                        const supportedProviders = Object.keys(ctx.options.socialProviders || {})
                        if (supportedProviders.includes(identity.provider)) {
                                const hasAccount = existingAccounts.find(
                                        (account) => account.providerId === identity.provider,
                                );
                                if (!hasAccount) {
                                        await ctx.adapter.create({
                                                model: "account",
                                                data: {
                                                        userId: user.id,
                                                        providerId: identity.provider,
                                                        accountId: identity.identity_data?.sub,
                                                        createdAt: new Date(identity.created_at ?? user.created_at),
                                                        updatedAt: new Date(identity.updated_at ?? user.updated_at),
                                                },
                                        });
                                }
                        }
                }
        }
    };
    migrateFromSupabase();
    ```
  </Step>

  <Step>
    ### Customize the migration script (Optional)

    * `name`: the migration script will use the user's email as the name. You might want to customize it if you have the user display name in your database.
    * `socialProviderList`: the migration script will use the social providers you have enabled in your auth config. You might want to customize it if you have additional social providers that you haven't enabled in your auth config.
    * `role`: remove `role` if you're not using the `admin` plugin
    * `isAnonymous`: remove `isAnonymous` if you're not using the `anonymous` plugin.
    * update other tables that reference the `users` table to use the `id` field.
  </Step>

  <Step>
    ### Run the migration script

    Run the migration script to migrate the users and accounts from Supabase to Better Auth.

    ```bash title="Terminal"
    bun migration.ts # or use node, ts-node, etc.
    ```
  </Step>

  <Step>
    ### Change password hashing algorithm

    By default, Better Auth uses the `scrypt` algorithm to hash passwords. Since Supabase uses `bcrypt`, you'll need to configure Better Auth to use bcrypt for password verification.

    First, install bcrypt:

    ```bash
    npm install bcrypt
    npm install -D @types/bcrypt
    ```

    Then update your auth configuration:

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth";
    import bcrypt from "bcrypt";

    export const auth = betterAuth({
       emailAndPassword: {
           password: {
               hash: async (password) => {
                   return await bcrypt.hash(password, 10);
               },
               verify: async ({ hash, password }) => {
                   return await bcrypt.compare(password, hash);
               }
           }
       }
    })
    ```
  </Step>

  <Step>
    ### Update your code

    Update your codebase from Supabase auth calls to Better Auth API.

    Here's a list of the Supabase auth API calls and their Better Auth counterparts.

    * `supabase.auth.signUp` -> `authClient.signUp.email`
    * `supabase.auth.signInWithPassword` -> `authClient.signIn.email`
    * `supabase.auth.signInWithOAuth` -> `authClient.signIn.social`
    * `supabase.auth.signInAnonymously` -> `authClient.signIn.anonymous`
    * `supabase.auth.signOut` -> `authClient.signOut`
    * `supabase.auth.getSession` -> `authClient.getSession` -  you can also use `authClient.useSession` for reactive state

    Learn more:

    * [Basic Usage](/docs/basic-usage): Learn how to use the auth client to sign up, sign in, and sign out.
    * [Email and Password](/docs/authentication/email-and-password): Learn how to add email and password authentication to your project.
    * [Anonymous](/docs/plugins/anonymous): Learn how to add anonymous authentication to your project.
    * [Admin](/docs/plugins/admin): Learn how to add admin authentication to your project.
    * [Email OTP](/docs/authentication/email-otp): Learn how to add email OTP authentication to your project.
    * [Hooks](/docs/concepts/hooks): Learn how to use the hooks to listen for events.
    * [Next.js](/docs/integrations/next): Learn how to use the auth client in a Next.js project.
  </Step>
</Steps>

### Middleware

To protect routes with middleware, refer to the [Next.js middleware guide](/docs/integrations/next#middleware) or your framework's documentation.

## Wrapping Up

Congratulations! You've successfully migrated from Supabase Auth to Better Auth.

Better Auth offers greater flexibility and more featuresâ€”be sure to explore the [documentation](/docs) to unlock its full potential.
~~~

~~~
# guides: Create your first plugin
URL: /docs/guides/your-first-plugin
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/guides/your-first-plugin.mdx

A step-by-step guide to creating your first Better Auth plugin.

***

title: Create your first plugin
description: A step-by-step guide to creating your first Better Auth plugin.
----------------------------------------------------------------------------

In this guide, weâ€™ll walk you through the steps of creating your first Better Auth plugin.

<Callout type="warn">
  This guide assumes you have <Link href="/docs/installation">setup the basics</Link> of Better Auth and are ready to create your first plugin.
</Callout>

<Steps>
  <Step>
    ## Plan your idea

    Before beginning, you must know what plugin you intend to create.

    In this guide, weâ€™ll create a **birthday plugin** to keep track of user birth dates.
  </Step>

  <Step>
    ## Server plugin first

    Better Auth plugins operate as a pair: a <Link href="/docs/concepts/plugins#create-a-server-plugin">server plugin</Link> and a <Link href="/docs/concepts/plugins#create-a-
client-plugin">client plugin</Link>.
    The server plugin forms the foundation of your authentication system, while the client plugin provides convenient frontend APIs to interact with your server implementation.

    <Callout>
      You can read more about server/client plugins in our <Link href="/docs/concepts/plugins#creating-a-plugin">documentation</Link>.
    </Callout>

    ### Creating the server plugin

    Go ahead and find a suitable location to create your birthday plugin folder, with an `index.ts` file within.

    <Files>
      <Folder name="birthday-plugin" defaultOpen>
        <File name="index.ts" />
      </Folder>
    </Files>

    In the `index.ts` file, weâ€™ll export a function that represents our server plugin.
    This will be what we will later add to our plugin list in the `auth.ts` file.

    ```ts title="index.ts"
    import { createAuthClient } from "better-auth/client";
    import type { BetterAuthPlugin } from "better-auth";

    export const birthdayPlugin = () =>
      ({
        id: "birthdayPlugin",
      } satisfies BetterAuthPlugin);

    ```

    Although this does nothing, you have technically just made yourself your first plugin, congratulations! ğŸ  ‰
  </Step>

  <Step>
    ### Defining a schema

    In order to save each userâ€™s birthday data, we must create a schema on top of the `user` model.

    By creating a schema here, this also allows <Link href="/docs/concepts/cli">Better Authâ€™s CLI</Link> to generate the schemas required to update your database.

    <Callout type="info">
      You can learn more about <Link href="/docs/concepts/plugins#schema">plugin schemas here</Link>.
    </Callout>

    ```ts title="index.ts"
    //...
    export const birthdayPlugin = () =>
      ({
        id: "birthdayPlugin",
        schema: {// [!code highlight]
          user: {// [!code highlight]
            fields: {// [!code highlight]
              birthday: {// [!code highlight]
                type: "date", // string, number, boolean, date // [!code highlight]
                required: true, // if the field should be required on a new record. (default: false) // [!code highlight]
                unique: false, // if the field should be unique. (default: false) // [!code highlight]
                references: null // if the field is a reference to another table. (default: null) // [!code highlight]
              },// [!code highlight]
            },// [!code highlight]
          },// [!code highlight]
        },
      } satisfies BetterAuthPlugin);
    ```
  </Step>

  <Step>
    ### Authorization logic

    For this example guide, weâ€™ll set up authentication logic to check and ensure that the user who signs-up is older than 5.
    But the same concept could be applied for something like verifying users agreeing to the TOS or anything alike.

    To do this, weâ€™ll utilize <Link href="/docs/concepts/plugins#hooks">Hooks</Link>, which allows us to run code `before` or `after` an action is performed.

    ```ts title="index.ts"
    export const birthdayPlugin = () => ({
        //...
        // In our case, we want to write authorization logic,
        // meaning we want to intercept it `before` hand.
        hooks: {
          before: [
            {
              matcher: (context) => /* ... */,
              handler: createAuthMiddleware(async (ctx) => {
                //...
              }),
            },
          ],
        },
    } satisfies BetterAuthPlugin)
    ```

    In our case we want to match any requests going to the signup path:

    ```ts title="Before hook"
    {
      matcher: (context) => context.path.startsWith("/sign-up/email"),
      //...
    }
    ```
~~~

~~~
    And for our logic, weâ€™ll write the following code to check the if userâ€™s birthday makes them above 5 years old.

    ```ts title="Imports"
    import { APIError } from "better-auth/api";
    import { createAuthMiddleware } from "better-auth/plugins";
    ```

    ```ts title="Before hook"
    {
      //...
      handler: createAuthMiddleware(async (ctx) => {
        const { birthday } = ctx.body;
        if(!(birthday instanceof Date)) {
          throw new APIError("BAD_REQUEST", { message: "Birthday must be of type Date." });
        }

        const today = new Date();
        const fiveYearsAgo = new Date(today.setFullYear(today.getFullYear() - 5));

        if(birthday >= fiveYearsAgo) {
          throw new APIError("BAD_REQUEST", { message: "User must be above 5 years old." });
        }

        return { context: ctx };
      }),
    }
    ```

    **Authorized!** ğŸ”’

    Weâ€™ve now successfully written code to ensure authorization for users above 5!
  </Step>

  <Step>
    ## Client Plugin

    Weâ€™re close to the finish line! ğŸ

    Now that we have created our server plugin, the next step is to develop our client plugin.
    Since there isnâ€™t much frontend APIs going on for this plugin, there isnâ€™t much to do!

    First, letâ€™s create our `client.ts` file first:

    <Files>
      <Folder name="birthday-plugin" defaultOpen>
        <File name="index.ts" />

        <File name="client.ts" />
      </Folder>
    </Files>

    Then, add the following code:

    ```ts title="client.ts"
    import { BetterAuthClientPlugin } from "better-auth";
    import type { birthdayPlugin } from "./index"; // make sure to import the server plugin as a type // [!code highlight]

    type BirthdayPlugin = typeof birthdayPlugin;

    export const birthdayClientPlugin = () => {
      return {
        id: "birthdayPlugin",
        $InferServerPlugin: {} as ReturnType<BirthdayPlugin>,
      } satisfies BetterAuthClientPlugin;
    };
    ```

    What weâ€™ve done is allow the client plugin to infer the types defined by our schema from the server plugin.

    And thatâ€™s it!  This is all it takes for the birthday client plugin. ğŸ  ,
  </Step>

  <Step>
    ## Initiate your plugin!

    Both the `client` and `server` plugins are now ready, the last step is to import them to both your `auth-client.ts` and your `server.ts` files respectively to initiate the plugin.

    ### Server initiation

    ```ts title="server.ts"
    import { betterAuth } from "better-auth";
    import { birthdayPlugin } from "./birthday-plugin";// [!code highlight]

    export const auth = betterAuth({
        plugins: [
          birthdayPlugin(),// [!code highlight]
        ]
    });
    ```

    ### Client initiation

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client";
    import { birthdayClientPlugin } from "./birthday-plugin/client";// [!code highlight]

    const authClient = createAuthClient({
        plugins: [
          birthdayClientPlugin()// [!code highlight]
        ]
    });
    ```

    ### Oh yeah, the schemas!

    Donâ€™t forget to add your `birthday` field to your `user` table model!

    Or, use the `generate` <Link href="/docs/concepts/cli#generate">CLI command</Link>:

    ```bash
    npx @better-auth/cli@latest generate
    ```
  </Step>
</Steps>

## Wrapping Up

Congratulations! Youâ€™ve successfully created your first ever Better Auth plugin.
We highly recommend you visit our <Link href="/docs/concepts/plugins">plugins documentation</Link> to learn more information.

If you have a plugin youâ€™d like to share with the community, feel free to let us know through
our <Link href="https://discord.gg/better-auth">Discord server</Link>,
or through a <Link href="https://github.com/better-auth/better-auth/pulls">pull-request</Link>
and we may add it to the <Link href="/docs/plugins/community-plugins">community-plugins</Link> list!
~~~

# integrations: Astro Integration
URL: /docs/integrations/astro
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/integrations/astro.mdx

Integrate Better Auth with Astro.

***

title: Astro Integration
description: Integrate Better Auth with Astro.

Better Auth comes with first class support for Astro. This guide will show you how to integrate Better Auth with Astro.

Before you start, make sure you have a Better Auth instance configured. If you haven't done that yet, check out the [installation](/docs/installation).

### Mount the handler

To enable Better Auth to handle requests, we need to mount the handler to a catch all API route. Create a file inside `/pages/api/auth` called `[...all].ts` and add the following code:

```ts title="pages/api/auth/[...all].ts"
import { auth } from "~/auth";
import type { APIRoute } from "astro";

export const ALL: APIRoute = async (ctx) => {
	// If you want to use rate limiting, make sure to set the 'x-forwarded-for' header to the request headers from the context
	// ctx.request.headers.set("x-forwarded-for", ctx.clientAddress);
	return auth.handler(ctx.request);
};
```

<Callout>
  You can change the path on your better-auth configuration but it's recommended to keep it as `/api/auth/[...all]`
</Callout>

## Create a client

Astro supports multiple frontend frameworks, so you can easily import your client based on the framework you're using.

If you're not using a frontend framework, you can still import the vanilla client.

<Tabs
  items={["vanilla", "react", "vue", "svelte", "solid",
]}
  defaultValue="react"
>
  <Tab value="vanilla">
    ```ts title="lib/auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    export const authClient =  createAuthClient()
    ```
  </Tab>

  <Tab value="react" title="lib/auth-client.ts">
    ```ts title="lib/auth-client.ts"
    import { createAuthClient } from "better-auth/react"
    export const authClient =  createAuthClient()
    ```
  </Tab>

  <Tab value="vue" title="lib/auth-client.ts">
    ```ts title="lib/auth-client.ts"
    import { createAuthClient } from "better-auth/vue"
    export const authClient =  createAuthClient()
    ```
  </Tab>

  <Tab value="svelte" title="lib/auth-client.ts">
    ```ts title="lib/auth-client.ts"
    import { createAuthClient } from "better-auth/svelte"
    export const authClient =  createAuthClient()
    ```
  </Tab>

  <Tab value="solid" title="lib/auth-client.ts">
    ```ts title="lib/auth-client.ts"
    import { createAuthClient } from "better-auth/solid"
    export const authClient =  createAuthClient()
    ```
  </Tab>
</Tabs>

## Auth Middleware

### Astro Locals types

To have types for your Astro locals, you need to set it inside the `env.d.ts` file.

```ts title="env.d.ts"

/// <reference path="../.astro/types.d.ts" />

declare namespace App {
    // Note: 'import {} from ""' syntax does not work in .d.ts files.
    interface Locals {
        user: import("better-auth").User | null;
        session: import("better-auth").Session | null;
    }
}
```

### Middleware

To protect your routes, you can check if the user is authenticated using the `getSession` method in middleware and set the user and session data using the Astro locals with the types we set before. Start by creating a `middleware.ts` file in the root of your project and follow the example below:

```ts title="middleware.ts"
import { auth } from "@/auth";
import { defineMiddleware } from "astro:middleware";

export const onRequest = defineMiddleware(async (context, next) => {
    const isAuthed = await auth.api
        .getSession({
            headers: context.request.headers,
        })

    if (isAuthed) {
        context.locals.user = isAuthed.user;
        context.locals.session = isAuthed.session;
    } else {
        context.locals.user = null;
        context.locals.session = null;
    }

    return next();
});
```

### Getting session on the server inside `.astro` file

~~~
You can use `Astro.locals` to check if the user has session and get the user data from the server side. Here is an example of how you can get the session inside an `.astro` file:

```astro
---
import { UserCard } from "@/components/user-card";

const session = () => {
    if (Astro.locals.session) {
        return Astro.locals.session;
    } else {
        // Redirect to login page if the user is not authenticated
        return Astro.redirect("/login");
    }
}

---

<UserCard initialSession={session} />
```



# integrations: Convex Integration
URL: /docs/integrations/convex
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/integrations/convex.mdx

Integrate Better Auth with Convex.

***

title: Convex Integration
description: Integrate Better Auth with Convex.
-----------------------------------------------

<Callout>
  This documentation comes from the [Convex documentation](https://convex-better-auth.netlify.app/),
  for more information, please refer to their documentation.
</Callout>

## Prerequisites

<Steps>
  <Step>
    ### Create a Convex project

    To use Convex + Better Auth, you'll first need a [Convex](https://www.convex.dev/) project.
    If you don't have one, run the following command to get started.

    <CodeBlockTabs defaultValue="npm">
      <CodeBlockTabsList>
        <CodeBlockTabsTrigger value="npm">
          npm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="pnpm">
          pnpm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="yarn">
          yarn
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="bun">
          bun
        </CodeBlockTabsTrigger>
      </CodeBlockTabsList>

      <CodeBlockTab value="npm">
        ```bash
        npm create convex@latest
        ```
      </CodeBlockTab>

      <CodeBlockTab value="pnpm">
        ```bash
        pnpm create convex
        ```
      </CodeBlockTab>

      <CodeBlockTab value="yarn">
        ```bash
        yarn create convex
        ```
      </CodeBlockTab>

      <CodeBlockTab value="bun">
        ```bash
        bunx create-convex
        ```
      </CodeBlockTab>
    </CodeBlockTabs>

    Check out the [Convex docs](https://docs.convex.dev/home) to learn more about Convex.
  </Step>

  <Step>
    ### Run `convex dev`

    Running the CLI during setup will initialize your Convex deployment
    if it doesn't already exist, and keeps generated types current through the process. Keep it running.

    <CodeBlockTabs defaultValue="npm">
      <CodeBlockTabsList>
        <CodeBlockTabsTrigger value="npm">
          npm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="pnpm">
          pnpm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="yarn">
          yarn
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="bun">
          bun
        </CodeBlockTabsTrigger>
      </CodeBlockTabsList>

      <CodeBlockTab value="npm">
        ```bash
        npx convex dev
        ```
      </CodeBlockTab>

      <CodeBlockTab value="pnpm">
        ```bash
        pnpm dlx convex dev
        ```
      </CodeBlockTab>
~~~

~~~mdx
      <CodeBlockTab value="yarn">
        ```bash
        yarn dlx convex dev
        ```
      </CodeBlockTab>

      <CodeBlockTab value="bun">
        ```bash
        bun x convex dev
        ```
      </CodeBlockTab>
    </CodeBlockTabs>
  </Step>
</Steps>

## Installation of Convex + Better Auth

The following documentation assumes you're using Next.js.

If you're not using Next.js, please refer to the [installation guide by Convex](https://convex-better-auth.netlify.app/#select-your-framework).

<Callout>
  For a complete example, check out Convex + Better Auth example with Next.js in this [Github repository](https://github.com/get-convex/better-auth/tree/main/examples/next).
</Callout>

### Installation

<Steps>
  <Step>
    #### Install packages

    Install the component, a pinned version of Better Auth, and ensure the latest version of Convex.

    <Callout>
      This component requires Convex `1.25.0` or later.
    </Callout>

    <CodeBlockTabs defaultValue="npm">
      <CodeBlockTabsList>
        <CodeBlockTabsTrigger value="npm">
          npm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="pnpm">
          pnpm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="yarn">
          yarn
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="bun">
          bun
        </CodeBlockTabsTrigger>
      </CodeBlockTabsList>

      <CodeBlockTab value="npm">
        ```bash
        npm install better-auth@1.3.8 --save-exact
        npm install convex@latest @convex-dev/better-auth
        ```
      </CodeBlockTab>

      <CodeBlockTab value="pnpm">
        ```bash
        pnpm add better-auth@1.3.8 --save-exact
        pnpm add convex@latest @convex-dev/better-auth
        ```
      </CodeBlockTab>

      <CodeBlockTab value="yarn">
        ```bash
        yarn add better-auth@1.3.8 --exact
        yarn add convex@latest @convex-dev/better-auth
        ```
      </CodeBlockTab>

      <CodeBlockTab value="bun">
        ```bash
        bun add better-auth@1.3.8 --exact
        bun add convex@latest @convex-dev/better-auth
        ```
      </CodeBlockTab>
    </CodeBlockTabs>
  </Step>

  <Step>
    #### Register the component

    Register the Better Auth component in your Convex project.

    ```ts title="convex/convex.config.ts"
    import { defineApp } from "convex/server";
    import betterAuth from "@convex-dev/better-auth/convex.config";

    const app = defineApp();
    app.use(betterAuth);

    export default app;
    ```
  </Step>

  <Step>
    #### Add Convex auth config

    Add a `convex/auth.config.ts` file to configure Better Auth as an authentication provider.

    ```ts title="convex/auth.config.ts"
    export default {
        providers: [
            {
                domain: process.env.CONVEX_SITE_URL,
                applicationID: "convex",
            },
        ],
    };
    ```
  </Step>

  <Step>
    #### Set environment variables

    Generate a secret for encryption and generating hashes. Use the command below if you have openssl installed,
    or use the button to generate a random value instead. Or generate your own however you like.

    <CodeBlockTabs defaultValue="npm">
      <CodeBlockTabsList>
        <CodeBlockTabsTrigger value="npm">
          npm
        </CodeBlockTabsTrigger>
~~~

~~~
        <CodeBlockTabsTrigger value="pnpm">
          pnpm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="yarn">
          yarn
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="bun">
          bun
        </CodeBlockTabsTrigger>
      </CodeBlockTabsList>

      <CodeBlockTab value="npm">
        ```bash
        npx convex env set BETTER_AUTH_SECRET=$(openssl rand -base64 32)
        ```
      </CodeBlockTab>

      <CodeBlockTab value="pnpm">
        ```bash
        pnpm dlx convex env set BETTER_AUTH_SECRET=$(openssl rand -base64 32)
        ```
      </CodeBlockTab>

      <CodeBlockTab value="yarn">
        ```bash
        yarn dlx convex env set BETTER_AUTH_SECRET=$(openssl rand -base64 32)
        ```
      </CodeBlockTab>

      <CodeBlockTab value="bun">
        ```bash
        bun x convex env set BETTER_AUTH_SECRET=$(openssl rand -base64 32)
        ```
      </CodeBlockTab>
    </CodeBlockTabs>

    Add your site URL to your Convex deployment.

    <CodeBlockTabs defaultValue="npm">
      <CodeBlockTabsList>
        <CodeBlockTabsTrigger value="npm">
          npm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="pnpm">
          pnpm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="yarn">
          yarn
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="bun">
          bun
        </CodeBlockTabsTrigger>
      </CodeBlockTabsList>

      <CodeBlockTab value="npm">
        ```bash
        npx convex env set SITE_URL http://localhost:3000
        ```
      </CodeBlockTab>

      <CodeBlockTab value="pnpm">
        ```bash
        pnpm dlx convex env set SITE_URL http://localhost:3000
        ```
      </CodeBlockTab>

      <CodeBlockTab value="yarn">
        ```bash
        yarn dlx convex env set SITE_URL http://localhost:3000
        ```
      </CodeBlockTab>

      <CodeBlockTab value="bun">
        ```bash
        bun x convex env set SITE_URL http://localhost:3000
        ```
      </CodeBlockTab>
    </CodeBlockTabs>

    Add environment variables to the `.env.local` file created by `npx convex dev`.
    It will be picked up by your framework dev server.

    ```shell title=".env.local" tab="Cloud"
    # Deployment used by \`npx convex dev\`
    CONVEX_DEPLOYMENT=dev:adjective-animal-123 # team: team-name, project: project-name

    NEXT_PUBLIC_CONVEX_URL=https://adjective-animal-123.convex.cloud

    # Same as NEXT_PUBLIC_CONVEX_URL but ends in .site // [!code ++]
    NEXT_PUBLIC_CONVEX_SITE_URL=https://adjective-animal-123.convex.site # [!code ++]

    # Your local site URL // [!code ++]
    SITE_URL=http://localhost:3000 # [!code ++]
    ```

    ```shell title=".env.local" tab="Self hosted"
    # Deployment used by \`npx convex dev\`
    CONVEX_DEPLOYMENT=dev:adjective-animal-123 # team: team-name, project: project-name

    NEXT_PUBLIC_CONVEX_URL=http://127.0.0.1:3210

    # Will generally be one number higher than NEXT_PUBLIC_CONVEX_URL,
    # so if your convex url is :3212, your site url will be :3213
    NEXT_PUBLIC_CONVEX_SITE_URL=http://127.0.0.1:3211 # [!code ++]

    # Your local site URL // [!code ++]
    SITE_URL=http://localhost:3000 # [!code ++]
    ```
  </Step>

  <Step>
    ### Create a Better Auth instance

    Create a Better Auth instance and initialize the component.

    <Callout>Some TypeScript errors will show until you save the file.</Callout>

    ```ts title="convex/auth.ts"
    import { createClient, type GenericCtx } from "@convex-dev/better-auth";
    import { convex } from "@convex-dev/better-auth/plugins";
    import { components } from "./_generated/api";
    import { DataModel } from "./_generated/dataModel";
    import { query } from "./_generated/server";
    import { betterAuth } from "better-auth";

    const siteUrl = process.env.SITE_URL!;
~~~

~~~
    // The component client has methods needed for integrating Convex with Better Auth,
    // as well as helper methods for general use.
    export const authComponent = createClient<DataModel>(components.betterAuth);

    export const createAuth = (
        ctx: GenericCtx<DataModel>,
            { optionsOnly } = { optionsOnly: false },
        ) => {
          return betterAuth({
            // disable logging when createAuth is called just to generate options.
            // this is not required, but there's a lot of noise in logs without it.
            logger: {
                disabled: optionsOnly,
            },
            baseURL: siteUrl,
            database: authComponent.adapter(ctx),
            // Configure simple, non-verified email/password to get started
            emailAndPassword: {
                enabled: true,
                requireEmailVerification: false,
            },
            plugins: [
                // The Convex plugin is required for Convex compatibility
                convex(),
            ],
        });
    };

    // Example function for getting the current user
    // Feel free to edit, omit, etc.
    export const getCurrentUser = query({
        args: {},
        handler: async (ctx) => {
            return authComponent.getAuthUser(ctx);
        },
    });
    ```
  </Step>

  <Step>
    ### Create a Better Auth client instance

    Create a Better Auth client instance for interacting with the Better Auth server from your client.

    ```ts title="src/lib/auth-client.ts"
    import { createAuthClient } from "better-auth/react";
    import { convexClient } from "@convex-dev/better-auth/client/plugins";

    export const authClient = createAuthClient({
        plugins: [convexClient()],
    });
    ```
  </Step>

  <Step>
    ### Mount handlers

    Register Better Auth route handlers on your Convex deployment.

    ```ts title="convex/http.ts"
    import { httpRouter } from "convex/server";
    import { authComponent, createAuth } from "./auth";

    const http = httpRouter();

    authComponent.registerRoutes(http, createAuth);

    export default http;
    ```

    Set up route handlers to proxy auth requests from your framework server to your Convex deployment.

    ```ts title="app/api/auth/[...all]/route.ts"
    import { nextJsHandler } from "@convex-dev/better-auth/nextjs";

    export const { GET, POST } = nextJsHandler();
    ```
  </Step>

  <Step>
    ### Set up Convex client provider

    Wrap your app with the `ConvexBetterAuthProvider` component.

    ```ts title="app/ConvexClientProvider.tsx"
    "use client";

    import { ReactNode } from "react";
    import { ConvexReactClient } from "convex/react";
    import { authClient } from "@/lib/auth-client"; // [!code ++]
    import { ConvexBetterAuthProvider } from "@convex-dev/better-auth/react"; // [!code ++]

    const convex = new ConvexReactClient(process.env.NEXT_PUBLIC_CONVEX_URL!, {
      // Optionally pause queries until the user is authenticated // [!code ++]
      expectAuth: true, // [!code ++]
    });

    export function ConvexClientProvider({ children }: { children: ReactNode }) {
      return (
        <ConvexBetterAuthProvider client={convex} authClient={authClient}> // [!code ++]
          {children}
        </ConvexBetterAuthProvider> // [!code ++]
      );
    }
    ```
  </Step>
</Steps>

### You're done!

You're now ready to start using Better Auth with Convex.

## Usage

### Using Better Auth from the server

To use Better Auth's [server
methods](https://www.better-auth.com/docs/concepts/api) in server rendering,
server functions, or any other Next.js server code, use Convex functions
and call the function from your server code.

First, a token helper for calling Convex functions from your server code.

```ts title="src/lib/auth-server.ts"
import { createAuth } from "@/convex/auth";
import { getToken as getTokenNextjs } from "@convex-dev/better-auth/nextjs";

export const getToken = () => {
  return getTokenNextjs(createAuth);
};
```
~~~

Here's an example Convex function that uses Better Auth's server methods, and
a server action that calls the Convex function.

```ts title="convex/users.ts"
import { mutation } from "./_generated/server";
import { v } from "convex/values";
import { createAuth, authComponent } from "./auth";

export const updateUserPassword = mutation({
  args: {
    currentPassword: v.string(),
    newPassword: v.string(),
  },
  handler: async (ctx, args) => {
    await createAuth(ctx).api.changePassword({
      body: {
        currentPassword: args.currentPassword,
        newPassword: args.newPassword,
      },
      headers: await authComponent.getHeaders(ctx),
    });
  },
});
```

```ts title="app/actions.ts"
"use server";

import { fetchMutation } from "convex/nextjs";
import { api } from "../convex/_generated/api";
import { getToken } from "../lib/auth-server";

// Authenticated mutation via server function
export async function updatePassword({
  currentPassword,
  newPassword,
}: {
  currentPassword: string;
  newPassword: string;
}) {
  const token = await getToken();
  await fetchMutation(
    api.users.updatePassword,
    { currentPassword, newPassword },
    { token }
  );
}
```

<Callout>
  This documentation comes from the [Convex documentation](https://convex-better-auth.netlify.app/),
  for more information, please refer to their documentation.
</Callout>

# integrations: Elysia Integration
URL: /docs/integrations/elysia
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/integrations/elysia.mdx

Integrate Better Auth with Elysia.

***

title: Elysia Integration
description: Integrate Better Auth with Elysia.

This integration guide is assuming you are using Elysia with bun server.

Before you start, make sure you have a Better Auth instance configured. If you haven't done that yet, check out the [installation](/docs/installation).

### Mount the handler

We need to mount the handler to Elysia endpoint.

```ts
import { Elysia } from "elysia";
import { auth } from "./auth";

const app = new Elysia().mount(auth.handler).listen(3000);

console.log(
  `ğŸ¦Š Elysia is running at ${app.server?.hostname}:${app.server?.port}`,
);
```

### CORS

To configure cors, you can use the `cors` plugin from `@elysiajs/cors`.

```ts
import { Elysia } from "elysia";
import { cors } from "@elysiajs/cors";

import { auth } from "./auth";

const app = new Elysia()
  .use(
    cors({
      origin: "http://localhost:3001",
      methods: ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
      credentials: true,
      allowedHeaders: ["Content-Type", "Authorization"],
    }),
  )
  .mount(auth.handler)
  .listen(3000);

console.log(
  `ğŸ¦Š Elysia is running at ${app.server?.hostname}:${app.server?.port}`,
);
```

### Macro

You can use [macro](https://elysiajs.com/patterns/macro.html#macro) with [resolve](https://elysiajs.com/essential/handler.html#resolve) to provide session and user information before pass to view.

```ts
import { Elysia } from "elysia";
import { auth } from "./auth";
```

~~~
// user middleware (compute user and session and pass to routes)
const betterAuth = new Elysia({ name: "better-auth" })
  .mount(auth.handler)
  .macro({
    auth: {
      async resolve({ status, request: { headers } }) {
        const session = await auth.api.getSession({
          headers,
        });

        if (!session) return status(401);

        return {
          user: session.user,
          session: session.session,
        };
      },
    },
  });

const app = new Elysia()
  .use(betterAuth)
  .get("/user", ({ user }) => user, {
    auth: true,
  })
  .listen(3000);

console.log(
  `ğŸ¦Š Elysia is running at ${app.server?.hostname}:${app.server?.port}`,
);
```

This will allow you to access the `user` and `session` object in all of your routes.



# integrations: Expo Integration
URL: /docs/integrations/expo
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/integrations/expo.mdx

Integrate Better Auth with Expo.

***

title: Expo Integration
description: Integrate Better Auth with Expo.
---------------------------------------------

Expo is a popular framework for building cross-platform apps with React Native. Better Auth supports both Expo native and web apps.

## Installation

<Steps>
  <Step>
    ## Configure A Better Auth Backend

    Before using Better Auth with Expo, make sure you have a Better Auth backend set up. You can either use a separate server or leverage Expo's new [API Routes](https://docs.expo.dev/router/reference/api-routes) feature to host your Better Auth instance.

    To get started, check out our [installation](/docs/installation) guide for setting up Better Auth on your server. If you prefer to check out the full example, you can find it [here](https://github.com/better-auth/examples/tree/main/expo-example).

    To use the new API routes feature in Expo to host your Better Auth instance you can create a new API route in your Expo app and mount the Better Auth handler.

    ```ts title="app/api/auth/[...auth]+api.ts"
    import { auth } from "@/lib/auth"; // import Better Auth handler

    const handler = auth.handler;
    export { handler as GET, handler as POST }; // export handler for both GET and POST requests
    ```
  </Step>

  <Step>
    ## Install Server Dependencies

    Install both the Better Auth package and Expo plugin into your server application.

    <CodeBlockTabs defaultValue="npm">
      <CodeBlockTabsList>
        <CodeBlockTabsTrigger value="npm">
          npm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="pnpm">
          pnpm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="yarn">
          yarn
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="bun">
          bun
        </CodeBlockTabsTrigger>
      </CodeBlockTabsList>

      <CodeBlockTab value="npm">
        ```bash
        npm install better-auth @better-auth/expo
        ```
      </CodeBlockTab>

      <CodeBlockTab value="pnpm">
        ```bash
        pnpm add better-auth @better-auth/expo
        ```
      </CodeBlockTab>

      <CodeBlockTab value="yarn">
        ```bash
        yarn add better-auth @better-auth/expo
        ```
      </CodeBlockTab>

      <CodeBlockTab value="bun">
        ```bash
        bun add better-auth @better-auth/expo
        ```
      </CodeBlockTab>
    </CodeBlockTabs>
  </Step>

  <Step>
    ## Install Client Dependencies

    You also need to install both the Better Auth package and Expo plugin into your Expo application.

    <CodeBlockTabs defaultValue="npm">
      <CodeBlockTabsList>
        <CodeBlockTabsTrigger value="npm">
~~~

~~~
          npm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="pnpm">
          pnpm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="yarn">
          yarn
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="bun">
          bun
        </CodeBlockTabsTrigger>
      </CodeBlockTabsList>

      <CodeBlockTab value="npm">
        ```bash
        npm install better-auth @better-auth/expo
        ```
      </CodeBlockTab>

      <CodeBlockTab value="pnpm">
        ```bash
        pnpm add better-auth @better-auth/expo
        ```
      </CodeBlockTab>

      <CodeBlockTab value="yarn">
        ```bash
        yarn add better-auth @better-auth/expo
        ```
      </CodeBlockTab>

      <CodeBlockTab value="bun">
        ```bash
        bun add better-auth @better-auth/expo
        ```
      </CodeBlockTab>
    </CodeBlockTabs>

    If you plan on using our social integrations (Google, Apple etc.) then there are a few more dependencies that are required in your Expo app. In the default Expo template these are already installed so you may be able to skip this step if you have these dependencies already.

    <CodeBlockTabs defaultValue="npm">
      <CodeBlockTabsList>
        <CodeBlockTabsTrigger value="npm">
          npm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="pnpm">
          pnpm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="yarn">
          yarn
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="bun">
          bun
        </CodeBlockTabsTrigger>
      </CodeBlockTabsList>

      <CodeBlockTab value="npm">
        ```bash
        npm install expo-linking expo-web-browser expo-constants

        ```
      </CodeBlockTab>

      <CodeBlockTab value="pnpm">
        ```bash
        pnpm add expo-linking expo-web-browser expo-constants

        ```
      </CodeBlockTab>

      <CodeBlockTab value="yarn">
        ```bash
        yarn add expo-linking expo-web-browser expo-constants

        ```
      </CodeBlockTab>

      <CodeBlockTab value="bun">
        ```bash
        bun add expo-linking expo-web-browser expo-constants

        ```
      </CodeBlockTab>
    </CodeBlockTabs>
  </Step>

  <Step>
    ## Add the Expo Plugin on Your Server

    Add the Expo plugin to your Better Auth server.

    ```ts title="lib/auth.ts"
    import { betterAuth } from "better-auth";
    import { expo } from "@better-auth/expo";

    export const auth = betterAuth({
        plugins: [expo()],
        emailAndPassword: {
            enabled: true, // Enable authentication using email and password.
          },
    });
    ```
  </Step>

  <Step>
    ## Initialize Better Auth Client

    To initialize Better Auth in your Expo app, you need to call `createAuthClient` with the base URL of your Better Auth backend. Make sure to import the client from `/react`.

    Make sure you install the `expo-secure-store` package into your Expo app. This is used to store the session data and cookies securely.

    <CodeBlockTabs defaultValue="npm">
      <CodeBlockTabsList>
        <CodeBlockTabsTrigger value="npm">
          npm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="pnpm">
          pnpm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="yarn">
          yarn
        </CodeBlockTabsTrigger>
~~~

~~~
        <CodeBlockTabsTrigger value="bun">
          bun
        </CodeBlockTabsTrigger>
      </CodeBlockTabsList>

      <CodeBlockTab value="npm">
        ```bash
        npm install expo-secure-store
        ```
      </CodeBlockTab>

      <CodeBlockTab value="pnpm">
        ```bash
        pnpm add expo-secure-store
        ```
      </CodeBlockTab>

      <CodeBlockTab value="yarn">
        ```bash
        yarn add expo-secure-store
        ```
      </CodeBlockTab>

      <CodeBlockTab value="bun">
        ```bash
        bun add expo-secure-store
        ```
      </CodeBlockTab>
    </CodeBlockTabs>

    You need to also import client plugin from `@better-auth/expo/client` and pass it to the `plugins` array when initializing the auth client.

    This is important because:

    * **Social Authentication Support:** enables social auth flows by handling authorization URLs and callbacks within the Expo web browser.
    * **Secure Cookie Management:** stores cookies securely and automatically adds them to the headers of your auth requests.

    ```ts title="lib/auth-client.ts"
    import { createAuthClient } from "better-auth/react";
    import { expoClient } from "@better-auth/expo/client";
    import * as SecureStore from "expo-secure-store";

    export const authClient = createAuthClient({
        baseURL: "http://localhost:8081", // Base URL of your Better Auth backend.
        plugins: [
            expoClient({
                scheme: "myapp",
                storagePrefix: "myapp",
                storage: SecureStore,
            })
        ]
    });
    ```

    <Callout>
      Be sure to include the full URL, including the path, if you've changed the default path from `/api/auth`.
    </Callout>
  </Step>

  <Step>
    ## Scheme and Trusted Origins

    Better Auth uses deep links to redirect users back to your app after authentication. To enable this, you need to add your app's scheme to the `trustedOrigins` list in your Better Auth config.

    First, make sure you have a scheme defined in your `app.json` file.

    ```json title="app.json"
    {
        "expo": {
            "scheme": "myapp"
        }
    }
    ```

    Then, update your Better Auth config to include the scheme in the `trustedOrigins` list.

    ```ts title="auth.ts"
    export const auth = betterAuth({
        trustedOrigins: ["myapp://"]
    })
    ```

    If you have multiple schemes or need to support deep linking with various paths, you can use specific patterns or wildcards:

    ```ts title="auth.ts"
    export const auth = betterAuth({
        trustedOrigins: [
            // Basic scheme
            "myapp://",

            // Production & staging schemes
            "myapp-prod://",
            "myapp-staging://",

            // Wildcard support for all paths following the scheme
            "myapp://*"
        ]
    })
    ```

    <Callout>
      The wildcard pattern can be particularly useful if your app uses different URL formats for deep linking based on features or screens.
    </Callout>
  </Step>

  <Step>
    ## Configure Metro Bundler

    To resolve Better Auth exports you'll need to enable `unstable_enablePackageExports` in your metro config.

    ```js title="metro.config.js"
    const { getDefaultConfig } = require("expo/metro-config");

    const config = getDefaultConfig(__dirname)

    config.resolver.unstable_enablePackageExports = true; // [!code highlight]

    module.exports = config;
    ```

    <Callout>In case you don't have a `metro.config.js` file in your project run `npx expo customize metro.config.js`.</Callout>

    If you can't enable `unstable_enablePackageExports` option, you can use [babel-plugin-module-resolver](https://github.com/tleunen/babel-plugin-module-resolver) to manually resolve the paths.
~~~

~~~mdx
    ```ts title="babel.config.js"
    module.exports = function (api) {
        api.cache(true);
        return {
            presets: ["babel-preset-expo"],
            plugins: [
                [
                    "module-resolver",
                    {
                        alias: {
                            "better-auth/react": "./node_modules/better-auth/dist/client/react/index.cjs",
                            "better-auth/client/plugins": "./node_modules/better-auth/dist/client/plugins/index.cjs",
                            "@better-auth/expo/client": "./node_modules/@better-auth/expo/dist/client.cjs",
                        },
                    },
                ],
            ],
        }
    }
    ```

    <Callout>In case you don't have a `babel.config.js` file in your project run `npx expo customize babel.config.js`.</Callout>

    Don't forget to clear the cache after making changes.

    ```bash
    npx expo start --clear
    ```
  </Step>
</Steps>

## Usage

### Authenticating Users

With Better Auth initialized, you can now use the `authClient` to authenticate users in your Expo app.

<Tabs items={["sign-in", "sign-up"]}>
  <Tab value="sign-in">
    ```tsx title="app/sign-in.tsx"
    import { useState } from "react";
    import { View, TextInput, Button } from "react-native";
    import { authClient } from "@/lib/auth-client";

    export default function SignIn() {
        const [email, setEmail] = useState("");
        const [password, setPassword] = useState("");

        const handleLogin = async () => {
            await authClient.signIn.email({
                email,
                password,
            })
        };

        return (
            <View>
                <TextInput
                    placeholder="Email"
                    value={email}
                    onChangeText={setEmail}
                />
                <TextInput
                    placeholder="Password"
                    value={password}
                    onChangeText={setPassword}
                />
                <Button title="Login" onPress={handleLogin} />
            </View>
        );
    }
    ```
  </Tab>

  <Tab value="sign-up">
    ```tsx title="app/sign-up.tsx"
    import { useState } from "react";
    import { View, TextInput, Button } from "react-native";
    import { authClient } from "@/lib/auth-client";

    export default function SignUp() {
        const [email, setEmail] = useState("");
        const [name, setName] = useState("");
        const [password, setPassword] = useState("");

        const handleLogin = async () => {
            await authClient.signUp.email({
                    email,
                    password,
                    name
            })
        };

        return (
            <View>
                <TextInput
                    placeholder="Name"
                    value={name}
                    onChangeText={setName}
                />
                <TextInput
                    placeholder="Email"
                    value={email}
                    onChangeText={setEmail}
                />
                <TextInput
                    placeholder="Password"
                    value={password}
                    onChangeText={setPassword}
                />
                <Button title="Login" onPress={handleLogin} />
            </View>
        );
    }
    ```
  </Tab>
</Tabs>

#### Social Sign-In

For social sign-in, you can use the `authClient.signIn.social` method with the provider name and a callback URL.

```tsx title="app/social-sign-in.tsx"
import { Button } from "react-native";
~~~

```
export default function SocialSignIn() {
    const handleLogin = async () => {
        await authClient.signIn.social({
            provider: "google",
            callbackURL: "/dashboard" // this will be converted to a deep link (eg. `myapp://dashboard`) on native
        })
    };
    return <Button title="Login with Google" onPress={handleLogin} />;
}
```

#### IdToken Sign-In

If you want to make provider request on the mobile device and then verify the ID token on the server, you can use the `authClient.signIn.social` method with the `idToken` option.

```tsx title="app/social-sign-in.tsx"
import { Button } from "react-native";

export default function SocialSignIn() {
    const handleLogin = async () => {
        await authClient.signIn.social({
            provider: "google", // only google, apple and facebook are supported for idToken signIn
            idToken: {
                token: "...", // ID token from provider
                nonce: "...", // nonce from provider (optional)
            }
            callbackURL: "/dashboard" // this will be converted to a deep link (eg. `myapp://dashboard`) on native
        })
    };
    return <Button title="Login with Google" onPress={handleLogin} />;
}
```

### Session

Better Auth provides a `useSession` hook to access the current user's session in your app.

```tsx title="app/index.tsx"
import { Text } from "react-native";
import { authClient } from "@/lib/auth-client";

export default function Index() {
    const { data: session } = authClient.useSession();

    return <Text>Welcome, {session?.user.name}</Text>;
}
```

On native, the session data will be cached in SecureStore. This will allow you to remove the need for a loading spinner when the app is reloaded. You can disable this behavior by passing the `disableCache` option to the client.

### Making Authenticated Requests to Your Server

To make authenticated requests to your server that require the user's session, you have to retrieve the session cookie from `SecureStore` and manually add it to your request headers.

```tsx
import { authClient } from "@/lib/auth-client";

const makeAuthenticatedRequest = async () => {
  const cookies = authClient.getCookie(); // [!code highlight]
  const headers = {
    "Cookie": cookies, // [!code highlight]
  };
  const response = await fetch("http://localhost:8081/api/secure-endpoint", {
    headers,
    // 'include' can interfere with the cookies we just set manually in the headers
    credentials: "omit" // [!code highlight]
  });
  const data = await response.json();
  return data;
};
```

Example: Usage With TRPC

```tsx title="lib/trpc-provider.tsx"
//...other imports
import { authClient } from "@/lib/auth-client"; // [!code highlight]

export const api = createTRPCReact<AppRouter>();

export function TRPCProvider(props: { children: React.ReactNode }) {
  const [queryClient] = useState(() => new QueryClient());
  const [trpcClient] = useState(() =>
    api.createClient({
      links: [
        httpBatchLink({
          //...your other options
          headers() {
            const headers = new Map<string, string>(); // [!code highlight]
            const cookies = authClient.getCookie(); // [!code highlight]
            if (cookies) { // [!code highlight]
              headers.set("Cookie", cookies); // [!code highlight]
            } // [!code highlight]
            return Object.fromEntries(headers); // [!code highlight]
          },
        }),
      ],
    }),
  );

  return (
    <api.Provider client={trpcClient} queryClient={queryClient}>
      <QueryClientProvider client={queryClient}>
        {props.children}
      </QueryClientProvider>
    </api.Provider>
  );
}
```

## Options

### Expo Client

storage: the storage mechanism used to cache the session data and cookies.

```ts title="lib/auth-client.ts"
import { createAuthClient } from "better-auth/react";
import SecureStorage from "expo-secure-store";

const authClient = createAuthClient({
    baseURL: "http://localhost:8081",
    storage: SecureStorage
});
```

scheme: scheme is used to deep link back to your app after a user has authenticated using oAuth providers. By default, Better Auth tries to read the scheme from the `app.json` file. If you need to override this, you can pass the scheme option to the client.

```ts title="lib/auth-client.ts"
import { createAuthClient } from "better-auth/react";

const authClient = createAuthClient({
    baseURL: "http://localhost:8081",
    scheme: "myapp"
});
```

disableCache: By default, the client will cache the session data in SecureStore. You can disable this behavior by passing the `disableCache` option to the client.

```ts title="lib/auth-client.ts"
import { createAuthClient } from "better-auth/react";

const authClient = createAuthClient({
    baseURL: "http://localhost:8081",
    disableCache: true
});
```

### Expo Servers

Server plugin options:

overrideOrigin: Override the origin for Expo API routes (default: false). Enable this if you're facing cors origin issues with Expo API routes.

# integrations: Express Integration
URL: /docs/integrations/express
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/integrations/express.mdx

Integrate Better Auth with Express.

***

title: Express Integration
description: Integrate Better Auth with Express.

This guide will show you how to integrate Better Auth with [express.js](https://expressjs.com/).

Before you start, make sure you have a Better Auth instance configured. If you haven't done that yet, check out the [installation](/docs/installation).

<Callout>
  Note that CommonJS (cjs) isn't supported. Use ECMAScript Modules (ESM) by setting `"type": "module"` in your `package.json` or configuring your `tsconfig.json` to use ES modules.
</Callout>

### Mount the handler

To enable Better Auth to handle requests, we need to mount the handler to an API route. Create a catch-all route to manage all requests to `/api/auth/*` in case of ExpressJS v4 or `/api/auth/*splat` in case of ExpressJS v5 (or any other path specified in your Better Auth options).

<Callout type="warn">
  Donâ€™t use `express.json()` before the Better Auth handler. Use it only for other routes, or the client API will get stuck on "pending".
</Callout>

```ts title="server.ts"
import express from "express";
import { toNodeHandler } from "better-auth/node";
import { auth } from "./auth";

const app = express();
const port = 3005;

app.all("/api/auth/*", toNodeHandler(auth)); // For ExpressJS v4
// app.all("/api/auth/*splat", toNodeHandler(auth)); For ExpressJS v5

// Mount express json middleware after Better Auth handler
// or only apply it to routes that don't interact with Better Auth
app.use(express.json());

app.listen(port, () => {
        console.log(`Example app listening on port ${port}`);
});
```

After completing the setup, start your server. Better Auth will be ready to use. You can send a `GET` request to the `/ok` endpoint (`/api/auth/ok`) to verify that the server is running.

### Cors Configuration

To add CORS (Cross-Origin Resource Sharing) support to your Express server when integrating Better Auth, you can use the `cors` middleware. Below is an updated example showing how to configure CORS for your server:

```ts
import express from "express";
import cors from "cors"; // Import the CORS middleware
import { toNodeHandler, fromNodeHeaders } from "better-auth/node";
import { auth } from "./auth";

const app = express();
const port = 3005;

// Configure CORS middleware
app.use(
  cors({
    origin: "http://your-frontend-domain.com", // Replace with your frontend's origin
    methods: ["GET", "POST", "PUT", "DELETE"], // Specify allowed HTTP methods
    credentials: true, // Allow credentials (cookies, authorization headers, etc.)
  })
);
```

### Getting the User Session

To retrieve the user's session, you can use the `getSession` method provided by the `auth` object. This method requires the request headers to be passed in a specific format. To simplify this process, Better Auth provides a `fromNodeHeaders` helper function that converts Node.js request headers to the format expected by Better Auth (a `Headers` object).

Here's an example of how to use `getSession` in an Express route:

```ts title="server.ts"
import { fromNodeHeaders } from "better-auth/node";
import { auth } from "./auth"; // Your Better Auth instance

app.get("/api/me", async (req, res) => {
        const session = await auth.api.getSession({
      headers: fromNodeHeaders(req.headers),
    });
        return res.json(session);
});
```

# integrations: Better Auth Fastify Integration Guide
URL: /docs/integrations/fastify
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/integrations/fastify.mdx

Learn how to seamlessly integrate Better Auth with your Fastify application.

***

title: Better Auth Fastify Integration Guide
description: Learn how to seamlessly integrate Better Auth with your Fastify application.

This guide provides step-by-step instructions for configuring both essential handlers and CORS settings.

<Callout type="important">
  A configured Better Auth instance is required before proceeding. If you haven't set this up yet, please consult our [Installation Guide](/docs/installation).
</Callout>

### Prerequisites

Verify the following requirements before integration:

* **Node.js Environment**: v16 or later installed
* **ES Module Support**: Enable ES modules in either:
  * `package.json`: `{ "type": "module" }`
  * TypeScript `tsconfig.json`: `{ "module": "ESNext" }`
* **Fastify Dependencies**:
  <CodeBlockTabs defaultValue="npm">
    <CodeBlockTabsList>
      <CodeBlockTabsTrigger value="npm">
        npm
      </CodeBlockTabsTrigger>

      <CodeBlockTabsTrigger value="pnpm">
        pnpm
      </CodeBlockTabsTrigger>

      <CodeBlockTabsTrigger value="yarn">
        yarn
      </CodeBlockTabsTrigger>

      <CodeBlockTabsTrigger value="bun">
        bun
      </CodeBlockTabsTrigger>
    </CodeBlockTabsList>

    <CodeBlockTab value="npm">
      ```bash
      npm install fastify @fastify/cors
      ```
    </CodeBlockTab>

    <CodeBlockTab value="pnpm">
      ```bash
      pnpm add fastify @fastify/cors
      ```
    </CodeBlockTab>

    <CodeBlockTab value="yarn">
      ```bash
      yarn add fastify @fastify/cors
      ```
    </CodeBlockTab>

    <CodeBlockTab value="bun">
      ```bash
      bun add fastify @fastify/cors
      ```
    </CodeBlockTab>
  </CodeBlockTabs>

<Callout type="tip"> For TypeScript: Ensure your `tsconfig.json` includes `"esModuleInterop": true` for optimal compatibility. </Callout>

### Authentication Handler Setup

Configure Better Auth to process authentication requests by creating a catch-all route:

```ts title="server.ts"
import Fastify from "fastify";
import { auth } from "./auth"; // Your configured Better Auth instance

const fastify = Fastify({ logger: true });

// Register authentication endpoint
fastify.route({
  method: ["GET", "POST"],
  url: "/api/auth/*",
  async handler(request, reply) {
    try {
      // Construct request URL
      const url = new URL(request.url, `http://${request.headers.host}`);

      // Convert Fastify headers to standard Headers object
      const headers = new Headers();
      Object.entries(request.headers).forEach(([key, value]) => {
        if (value) headers.append(key, value.toString());
      });

      // Create Fetch API-compatible request
      const req = new Request(url.toString(), {
        method: request.method,
        headers,
        body: request.body ? JSON.stringify(request.body) : undefined,
      });

      // Process authentication request
      const response = await auth.handler(req);

      // Forward response to client
      reply.status(response.status);
      response.headers.forEach((value, key) => reply.header(key, value));
      reply.send(response.body ? await response.text() : null);

    } catch (error) {
      fastify.log.error("Authentication Error:", error);
      reply.status(500).send({
        error: "Internal authentication error",
        code: "AUTH_FAILURE"
      });
    }
  }
});

// Initialize server
fastify.listen({ port: 4000 }, (err) => {
  if (err) {
    fastify.log.error(err);
    process.exit(1);
  }
  console.log("Server running on port 4000");
});
```

### Trusted origins

When a request is made from a different origin, the request will be blocked by default. You can add trusted origins to the `auth` instance.

```ts
export const auth = betterAuth({
  trustedOrigins: ["http://localhost:3000", "https://example.com"],
});
```

### Configuring CORS

Secure your API endpoints with proper CORS configuration:

```ts
import fastifyCors from "@fastify/cors";

// Configure CORS policies
fastify.register(fastifyCors, {
  origin: process.env.CLIENT_ORIGIN || "http://localhost:3000",
  methods: ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
  allowedHeaders: [
    "Content-Type",
    "Authorization",
    "X-Requested-With"
  ],
  credentials: true,
  maxAge: 86400
});

// Mount authentication handler after CORS registration
// (Use previous handler configuration here)
```

<Callout type="warning"> Always restrict CORS origins in production environments. Use environment variables for dynamic configuration. </Callout>

# integrations: Hono Integration
URL: /docs/integrations/hono
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/integrations/hono.mdx

Integrate Better Auth with Hono.

***

title: Hono Integration
description: Integrate Better Auth with Hono.

Before you start, make sure you have a Better Auth instance configured. If you haven't done that yet, check out the [installation](/docs/installation).

### Mount the handler

We need to mount the handler to Hono endpoint.

```ts
import { Hono } from "hono";
import { auth } from "./auth";
import { serve } from "@hono/node-server";

const app = new Hono();

app.on(["POST", "GET"], "/api/auth/*", (c) => {
        return auth.handler(c.req.raw);
});

serve(app);
```

### Cors

To configure cors, you need to use the `cors` plugin from `hono/cors`.

```ts
import { Hono } from "hono";
import { auth } from "./auth";
import { serve } from "@hono/node-server";
import { cors } from "hono/cors";

const app = new Hono();

app.use(
        "/api/auth/*", // or replace with "*" to enable cors for all routes
        cors({
                origin: "http://localhost:3001", // replace with your origin
                allowHeaders: ["Content-Type", "Authorization"],
                allowMethods: ["POST", "GET", "OPTIONS"],
                exposeHeaders: ["Content-Length"],
                maxAge: 600,
                credentials: true,
        }),
);

app.on(["POST", "GET"], "/api/auth/*", (c) => {
        return auth.handler(c.req.raw);
});

serve(app);
```

> **Important:** CORS middleware must be registered before your routes. This ensures that cross-origin requests are properly handled before they reach your authentication endpoints.

### Middleware

You can add a middleware to save the `session` and `user` in a `context` and also add validations for every route.

```ts
import { Hono } from "hono";
import { auth } from "./auth";
import { serve } from "@hono/node-server";
import { cors } from "hono/cors";

const app = new Hono<{
        Variables: {
                user: typeof auth.$Infer.Session.user | null;
                session: typeof auth.$Infer.Session.session | null
        }
}>();

app.use("*", async (c, next) => {
        const session = await auth.api.getSession({ headers: c.req.raw.headers });

        if (!session) {
        c.set("user", null);
        c.set("session", null);
        return next();
        }

        c.set("user", session.user);
        c.set("session", session.session);
        return next();
});

```
app.on(["POST", "GET"], "/api/auth/*", (c) => {
        return auth.handler(c.req.raw);
});

serve(app);

This will allow you to access the `user` and `session` object in all of your routes.

```ts
app.get("/session", (c) => {
        const session = c.get("session")
        const user = c.get("user")

        if(!user) return c.body(null, 401);

        return c.json({
          session,
          user
        });
});
```

### Cross-Domain Cookies

By default, all Better Auth cookies are set with `SameSite=Lax`. If you need to use cookies across different domains, youâ€™ll need to set `SameSite=None` and `Secure=true`. However, we recommend using subdomains whenever possible, as this allows you to keep `SameSite=Lax`. To enable cross-subdomain cookies, simply turn on `crossSubDomainCookies` in your auth config.

```ts title="auth.ts"
export const auth = createAuth({
  advanced: {
    crossSubDomainCookies: {
      enabled: true
    }
  }
})
```

If you still need to set `SameSite=None` and `Secure=true`, you can adjust these attributes globally through `cookieOptions` in the `createAuth` configuration.

```ts title="auth.ts"
export const auth = createAuth({
  advanced: {
    defaultCookieAttributes: {
      sameSite: "none",
      secure: true,
      partitioned: true // New browser standards will mandate this for foreign cookies
    }
  }
})
```

You can also customize cookie attributes individually by setting them within `cookies` in your auth config.

```ts title="auth.ts"
export const auth = createAuth({
  advanced: {
    cookies: {
      sessionToken: {
        attributes: {
          sameSite: "none",
          secure: true,
          partitioned: true // New browser standards will mandate this for foreign cookies
        }
      }
    }
  }
})
```

### Client-Side Configuration

When using the Hono client (`@hono/client`) to make requests to your Better Auth-protected endpoints, you need to configure it to send credentials (cookies) with cross-origin requests.

```ts title="api.ts"
import { hc } from "hono/client";
import type { AppType } from "./server"; // Your Hono app type

const client = hc<AppType>("http://localhost:8787/", {
  init: {
    credentials: "include", // Required for sending cookies cross-origin
  },
});

// Now your client requests will include credentials
const response = await client.someProtectedEndpoint.$get();
```

This configuration is necessary when:

* Your client and server are on different domains/ports during development
* You're making cross-origin requests in production
* You need to send authentication cookies with your requests

The `credentials: "include"` option tells the fetch client to send cookies even for cross-origin requests. This works in conjunction with the CORS configuration on your server that has `credentials: true`.

> **Note:** Make sure your CORS configuration on the server matches your client's domain, and that `credentials: true` is set in both the server's CORS config and the client's fetch config.

# integrations: Lynx Integration
URL: /docs/integrations/lynx
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/integrations/lynx.mdx

Integrate Better Auth with Lynx cross-platform framework.

***

title: Lynx Integration
description: Integrate Better Auth with Lynx cross-platform framework.

This integration guide is for using Better Auth with [Lynx](https://lynxjs.org), a cross-platform rendering framework that enables developers to build applications for Android, iOS, and Web platforms with native rendering performance.

Before you start, make sure you have a Better Auth instance configured. If you haven't done that yet, check out the [installation](/docs/installation).

## Installation

Install Better Auth and the Lynx React dependency:

<CodeBlockTabs defaultValue="npm">
  <CodeBlockTabsList>
    <CodeBlockTabsTrigger value="npm">
      npm
    </CodeBlockTabsTrigger>

~~~
    <CodeBlockTabsTrigger value="pnpm">
      pnpm
    </CodeBlockTabsTrigger>

    <CodeBlockTabsTrigger value="yarn">
      yarn
    </CodeBlockTabsTrigger>

    <CodeBlockTabsTrigger value="bun">
      bun
    </CodeBlockTabsTrigger>
  </CodeBlockTabsList>

  <CodeBlockTab value="npm">
    ```bash
    npm install better-auth @lynx-js/react
    ```
  </CodeBlockTab>

  <CodeBlockTab value="pnpm">
    ```bash
    pnpm add better-auth @lynx-js/react
    ```
  </CodeBlockTab>

  <CodeBlockTab value="yarn">
    ```bash
    yarn add better-auth @lynx-js/react
    ```
  </CodeBlockTab>

  <CodeBlockTab value="bun">
    ```bash
    bun add better-auth @lynx-js/react
    ```
  </CodeBlockTab>
</CodeBlockTabs>

## Create Client Instance

Import `createAuthClient` from `better-auth/lynx` to create your client instance:

```ts title="lib/auth-client.ts"
import { createAuthClient } from "better-auth/lynx"

export const authClient = createAuthClient({
    baseURL: "http://localhost:3000" // The base URL of your auth server
})
```

## Usage

The Lynx client provides the same API as other Better Auth clients, with optimized integration for Lynx's reactive system.

### Authentication Methods

```ts
import { authClient } from "./lib/auth-client"

// Sign in with email and password
await authClient.signIn.email({
    email: "test@user.com",
    password: "password1234"
})

// Sign up
await authClient.signUp.email({
    email: "test@user.com",
    password: "password1234",
    name: "John Doe"
})

// Sign out
await authClient.signOut()
```

### Hooks

The Lynx client includes reactive hooks that integrate seamlessly with Lynx's component system:

#### useSession

```tsx title="components/user.tsx"
import { authClient } from "../lib/auth-client"

export function User() {
    const {
        data: session,
        isPending, // loading state
        error // error object
    } = authClient.useSession()

    if (isPending) return <div>Loading...</div>
    if (error) return <div>Error: {error.message}</div>

    return (
        <div>
            {session ? (
                <div>
                    <p>Welcome, {session.user.name}!</p>
                    <button onClick={() => authClient.signOut()}>
                        Sign Out
                    </button>
                </div>
            ) : (
                <button onClick={() => authClient.signIn.social({
                    provider: 'github'
                })}>
                    Sign In with GitHub
                </button>
            )}
        </div>
    )
}
```

### Store Integration

The Lynx client uses [nanostores](https://github.com/nanostores/nanostores) for state management and provides a `useStore` hook for accessing reactive state:

```tsx title="components/session-info.tsx"
import { useStore } from "better-auth/lynx"
import { authClient } from "../lib/auth-client"

export function SessionInfo() {
    // Access the session store directly
    const session = useStore(authClient.$store.session)
~~~

    return (
        <div>
            {session && (
                <pre>{JSON.stringify(session, null, 2)}</pre>
)}
        </div>
)
}

### Advanced Store Usage

You can use the store with selective key watching for optimized re-renders:

```tsx title="components/optimized-user.tsx"
import { useStore } from "better-auth/lynx"
import { authClient } from "../lib/auth-client"

export function OptimizedUser() {
    // Only re-render when specific keys change
    const session = useStore(authClient.$store.session, {
        keys: ['user.name', 'user.email'] // Only watch these specific keys
    })

    return (
        <div>
            {session?.user && (
                <div>
                    <h2>{session.user.name}</h2>
                    <p>{session.user.email}</p>
                </div>
            )}
        </div>
    )
}
```

## Plugin Support

The Lynx client supports all Better Auth plugins:

```ts title="lib/auth-client.ts"
import { createAuthClient } from "better-auth/lynx"
import { magicLinkClient } from "better-auth/client/plugins"

const authClient = createAuthClient({
    plugins: [
        magicLinkClient()
    ]
})

// Use plugin methods
await authClient.signIn.magicLink({
    email: "test@email.com"
})
```

## Error Handling

Error handling works the same as other Better Auth clients:

```tsx title="components/login-form.tsx"
import { authClient } from "../lib/auth-client"

export function LoginForm() {
    const signIn = async (email: string, password: string) => {
        const { data, error } = await authClient.signIn.email({
            email,
            password
        })

        if (error) {
            console.error('Login failed:', error.message)
            return
        }

        console.log('Login successful:', data)
    }

    return (
        <form onSubmit={(e) => {
            e.preventDefault()
            const formData = new FormData(e.target)
            signIn(formData.get('email'), formData.get('password'))
        }}>
            <input name="email" type="email" placeholder="Email" />
            <input name="password" type="password" placeholder="Password" />
            <button type="submit">Sign In</button>
        </form>
    )
}
```

## Features

The Lynx client provides:

* **Cross-Platform Support**: Works across Android, iOS, and Web platforms
* **Optimized Performance**: Built specifically for Lynx's reactive system
* **Nanostores Integration**: Uses nanostores for efficient state management
* **Selective Re-rendering**: Watch specific store keys to minimize unnecessary updates
* **Full API Compatibility**: All Better Auth methods and plugins work seamlessly
* **TypeScript Support**: Full type safety with TypeScript inference

The Lynx integration maintains all the features and benefits of Better Auth while providing optimal performance and developer experience within Lynx's cross-platform ecosystem.

# integrations: NestJS Integration
URL: /docs/integrations/nestjs
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/integrations/nestjs.mdx

Integrate Better Auth with NestJS.

***

title: NestJS Integration
description: Integrate Better Auth with NestJS.

This guide will show you how to integrate Better Auth with [NestJS](https://nestjs.com/).

Before you start, make sure you have a Better Auth instance configured. If you haven't done that yet, check out the [installation](/docs/installation).

<Callout type="info">
  The NestJS integration is **community maintained**. If you encounter any issues, please open them at [nestjs-better-auth](https://github.com/ThallesP/nestjs-better-auth).
</Callout>

## Installation

Install the NestJS integration library:

<CodeBlockTabs defaultValue="npm">
  <CodeBlockTabsList>
    <CodeBlockTabsTrigger value="npm">
      npm
    </CodeBlockTabsTrigger>

    <CodeBlockTabsTrigger value="pnpm">
      pnpm
    </CodeBlockTabsTrigger>

    <CodeBlockTabsTrigger value="yarn">
      yarn
    </CodeBlockTabsTrigger>

    <CodeBlockTabsTrigger value="bun">
      bun
    </CodeBlockTabsTrigger>
  </CodeBlockTabsList>

  <CodeBlockTab value="npm">
    ```bash
    npm install @thallesp/nestjs-better-auth
    ```
  </CodeBlockTab>

  <CodeBlockTab value="pnpm">
    ```bash
    pnpm add @thallesp/nestjs-better-auth
    ```
  </CodeBlockTab>

  <CodeBlockTab value="yarn">
    ```bash
    yarn add @thallesp/nestjs-better-auth
    ```
  </CodeBlockTab>

  <CodeBlockTab value="bun">
    ```bash
    bun add @thallesp/nestjs-better-auth
    ```
  </CodeBlockTab>
</CodeBlockTabs>

## Basic Setup

<Callout type="warn">
  Currently the library has beta support for Fastify, if you experience any issues with it, please open an issue at [nestjs-better-auth](https://github.com/ThallesP/nestjs-better-auth).
</Callout>

### 1. Disable Body Parser

Disable NestJS's built-in body parser to allow Better Auth to handle the raw request body:

```ts title="main.ts"
import { NestFactory } from "@nestjs/core";
import { AppModule } from "./app.module";

async function bootstrap() {
  const app = await NestFactory.create(AppModule, {
    bodyParser: false, // Required for Better Auth
  });
  await app.listen(process.env.PORT ?? 3000);
}
bootstrap();
```

### 2. Import AuthModule

Import the `AuthModule` in your root module:

```ts title="app.module.ts"
import { Module } from '@nestjs/common';
import { AuthModule } from '@thallesp/nestjs-better-auth';
import { auth } from "./auth"; // Your Better Auth instance

@Module({
  imports: [
    AuthModule.forRoot({ auth }),
  ],
})
export class AppModule {}
```

### 3. Route Protection

Global by default: An `AuthGuard` is registered globally by this module. All routes are protected unless you explicitly allow access.

Use the `Session` decorator to access the user session:

```ts title="user.controller.ts"
import { Controller, Get } from '@nestjs/common';
import { Session, UserSession, AllowAnonymous, OptionalAuth } from '@thallesp/nestjs-better-auth';

@Controller('users')
export class UserController {
  @Get('me')
  async getProfile(@Session() session: UserSession) {
    return { user: session.user };
  }

  @Get('public')
  @AllowAnonymous() // Allow anonymous access
  async getPublic() {
    return { message: 'Public route' };
  }

  @Get('optional')
  @OptionalAuth() // Authentication is optional
  async getOptional(@Session() session: UserSession) {
    return { authenticated: !!session };
  }
}
```

## Full Documentation

For comprehensive documentation including decorators, hooks, global guards, and advanced configuration, visit the [NestJS Better Auth repository](https://github.com/thallesp/nestjs-better-auth).

# integrations: Next.js integration
URL: /docs/integrations/next
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/integrations/next.mdx

Integrate Better Auth with Next.js.

***

~~~
title: Next.js integration
description: Integrate Better Auth with Next.js.
------------------------------------------------

Better Auth can be easily integrated with Next.js. Before you start, make sure you have a Better Auth instance configured. If you haven't done that yet, check out the
[installation](/docs/installation).

### Create API Route

We need to mount the handler to an API route. Create a route file inside `/api/auth/[...all]` directory. And add the following code:

```ts title="api/auth/[...all]/route.ts"
import { auth } from "@/lib/auth";
import { toNextJsHandler } from "better-auth/next-js";

export const { GET, POST } = toNextJsHandler(auth.handler);
```

<Callout type="info">
  You can change the path on your better-auth configuration but it's recommended to keep it as `/api/auth/[...all]`
</Callout>

For `pages` route, you need to use `toNodeHandler` instead of `toNextJsHandler` and set `bodyParser` to `false` in the `config` object. Here is an example:

```ts title="pages/api/auth/[...all].ts"
import { toNodeHandler } from "better-auth/node"
import { auth } from "@/lib/auth"

// Disallow body parsing, we will parse it manually
export const config = { api: { bodyParser: false } }

export default toNodeHandler(auth.handler)
```

## Create a client

Create a client instance. You can name the file anything you want. Here we are creating `client.ts` file inside the `lib/` directory.

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/react" // make sure to import from better-auth/react

export const authClient =  createAuthClient({
    //you can pass client configuration here
})
```

Once you have created the client, you can use it to sign up, sign in, and perform other actions.
Some of the actions are reactive. The client uses [nano-store](https://github.com/nanostores/nanostores) to store the state and re-render the components when the state changes.

The client also uses [better-fetch](https://github.com/bekacru/better-fetch) to make the requests. You can pass the fetch configuration to the client.

## RSC and Server actions

The `api` object exported from the auth instance contains all the actions that you can perform on the server. Every endpoint made inside Better Auth is a invocable as a function.
Including plugins endpoints.

**Example: Getting Session on a server action**

```tsx title="server.ts"
import { auth } from "@/lib/auth"
import { headers } from "next/headers"

const someAuthenticatedAction = async () => {
    "use server";
    const session = await auth.api.getSession({
        headers: await headers()
    })
};
```

**Example: Getting Session on a RSC**

```tsx
import { auth } from "@/lib/auth"
import { headers } from "next/headers"

export async function ServerComponent() {
    const session = await auth.api.getSession({
        headers: await headers()
    })
    if(!session) {
        return <div>Not authenticated</div>
    }
    return (
        <div>
            <h1>Welcome {session.user.name}</h1>
        </div>
    )
}
```

<Callout type="warn">As RSCs cannot set cookies, the [cookie cache](/docs/concepts/session-management#cookie-cache) will not be refreshed until the server is interacted with from
the client via Server Actions or Route Handlers.</Callout>

### Server Action Cookies

When you call a function that needs to set cookies, like `signInEmail` or `signUpEmail` in a server action, cookies wonâ€™t be set. This is because server actions need to use the
`cookies` helper from Next.js to set cookies.

To simplify this, you can use the `nextCookies` plugin, which will automatically set cookies for you whenever a `Set-Cookie` header is present in the response.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { nextCookies } from "better-auth/next-js";

export const auth = betterAuth({
    //...your config
    plugins: [nextCookies()] // make sure this is the last plugin in the array // [!code highlight]
})
```

Now, when you call functions that set cookies, they will be automatically set.

```ts
"use server";
import { auth } from "@/lib/auth"

const signIn = async () => {
    await auth.api.signInEmail({
        body: {
            email: "user@email.com",
            password: "password",
        }
    })
}
```

## Middleware

In Next.js middleware, it's recommended to only check for the existence of a session cookie to handle redirection. To avoid blocking requests by making API or database calls.
~~~

You can use the `getSessionCookie` helper from Better Auth for this purpose:

<Callout type="warn">
  The <code>getSessionCookie()</code> function does not automatically reference the auth config specified in <code>auth.ts</code>. Therefore, if you customized the cookie name or prefix, you need to ensure that the configuration in <code>getSessionCookie()</code> matches the config defined in your <code>auth.ts</code>.
</Callout>

```ts
import { NextRequest, NextResponse } from "next/server";
import { getSessionCookie } from "better-auth/cookies";

export async function middleware(request: NextRequest) {
        const sessionCookie = getSessionCookie(request);

    // THIS IS NOT SECURE!
    // This is the recommended approach to optimistically redirect users
    // We recommend handling auth checks in each page/route
        if (!sessionCookie) {
                return NextResponse.redirect(new URL("/", request.url));
        }

        return NextResponse.next();
}

export const config = {
        matcher: ["/dashboard"], // Specify the routes the middleware applies to
};
```

<Callout type="warn">
  Security Warning: The `getSessionCookie` function only checks for the
  existence of a session cookie; it does **not** validate it. Relying solely
  on this check for security is dangerous, as anyone can manually create a
  cookie to bypass it. You must always validate the session on your server for
  any protected actions or pages.
</Callout>

<Callout type="info">
  If you have a custom cookie name or prefix, you can pass it to the `getSessionCookie` function.

  ```ts
  const sessionCookie = getSessionCookie(request, {
      cookieName: "my_session_cookie",
      cookiePrefix: "my_prefix"
  });
  ```
</Callout>

Alternatively, you can use the `getCookieCache` helper to get the session object from the cookie cache.

```ts
import { getCookieCache } from "better-auth/cookies";

export async function middleware(request: NextRequest) {
        const session = await getCookieCache(request);
        if (!session) {
                return NextResponse.redirect(new URL("/sign-in", request.url));
        }
        return NextResponse.next();
}
```

### How to handle auth checks in each page/route

In this example, we are using the `auth.api.getSession` function within a server component to get the session object,
then we are checking if the session is valid. If it's not, we are redirecting the user to the sign-in page.

```tsx title="app/dashboard/page.tsx"
import { auth } from "@/lib/auth";
import { headers } from "next/headers";
import { redirect } from "next/navigation";

export default async function DashboardPage() {
    const session = await auth.api.getSession({
        headers: await headers()
    })

    if(!session) {
        redirect("/sign-in")
    }

    return (
        <div>
            <h1>Welcome {session.user.name}</h1>
        </div>
    )
}
```

### For Next.js release `15.1.7` and below

If you need the full session object, you'll have to fetch it from the `/get-session` API route. Since Next.js middleware doesn't support running Node.js APIs directly, you must make an HTTP request.

<Callout>
  The example uses [better-fetch](https://better-fetch.vercel.app), but you can use any fetch library.
</Callout>

```ts
import { betterFetch } from "@better-fetch/fetch";
import type { auth } from "@/lib/auth";
import { NextRequest, NextResponse } from "next/server";

type Session = typeof auth.$Infer.Session;

export async function middleware(request: NextRequest) {
        const { data: session } = await betterFetch<Session>("/api/auth/get-session", {
                baseURL: request.nextUrl.origin,
                headers: {
                        cookie: request.headers.get("cookie") || "", // Forward the cookies from the request
                },
        });

        if (!session) {
                return NextResponse.redirect(new URL("/sign-in", request.url));
        }

        return NextResponse.next();
}

export const config = {
        matcher: ["/dashboard"], // Apply middleware to specific routes
};
```

### For Next.js release `15.2.0` and above

From the version 15.2.0, Next.js allows you to use the `Node.js` runtime in middleware. This means you can use the `auth.api` object directly in middleware.

<Callout type="warn">
  You may refer to the [Next.js documentation](https://nextjs.org/docs/app/building-your-application/routing/middleware#runtime) for more information about runtime configuration, and how to enable it.
  Be careful when using the new runtime. It's an experimental feature and it may be subject to breaking changes.
</Callout>

```ts
import { NextRequest, NextResponse } from "next/server";
import { headers } from "next/headers";
import { auth } from "@/lib/auth";

export async function middleware(request: NextRequest) {
    const session = await auth.api.getSession({
        headers: await headers()
    })

    if(!session) {
        return NextResponse.redirect(new URL("/sign-in", request.url));
    }

    return NextResponse.next();
}

export const config = {
  runtime: "nodejs",
  matcher: ["/dashboard"], // Apply middleware to specific routes
};
```

# integrations: Nitro Integration
URL: /docs/integrations/nitro
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/integrations/nitro.mdx

Integrate Better Auth with Nitro.

***

title: Nitro Integration
description: Integrate Better Auth with Nitro.

Better Auth can be integrated with your [Nitro Application](https://nitro.build/) (an open source framework to build web servers).

This guide aims to help you integrate Better Auth with your Nitro application in a few simple steps.

## Create a new Nitro Application

Start by scaffolding a new Nitro application using the following command:

```bash title="Terminal"
npx giget@latest nitro nitro-app --install
```

This will create the `nitro-app` directory and install all the dependencies. You can now open the `nitro-app` directory in your code editor.

### Prisma Adapter Setup

<Callout>
  This guide assumes that you have a basic understanding of Prisma. If you are new to Prisma, you can check out the [Prisma documentation](https://www.prisma.io/docs/getting-started).

  The `sqlite` database used in this guide will not work in a production environment. You should replace it with a production-ready database like `PostgreSQL`.
</Callout>

For this guide, we will be using the Prisma adapter. You can install prisma client by running the following command:

<CodeBlockTabs defaultValue="npm">
  <CodeBlockTabsList>
    <CodeBlockTabsTrigger value="npm">
      npm
    </CodeBlockTabsTrigger>

    <CodeBlockTabsTrigger value="pnpm">
      pnpm
    </CodeBlockTabsTrigger>

    <CodeBlockTabsTrigger value="yarn">
      yarn
    </CodeBlockTabsTrigger>

    <CodeBlockTabsTrigger value="bun">
      bun
    </CodeBlockTabsTrigger>
  </CodeBlockTabsList>

  <CodeBlockTab value="npm">
    ```bash
    npm install @prisma/client
    ```
  </CodeBlockTab>

  <CodeBlockTab value="pnpm">
    ```bash
    pnpm add @prisma/client
    ```
  </CodeBlockTab>

  <CodeBlockTab value="yarn">
    ```bash
    yarn add @prisma/client
    ```
  </CodeBlockTab>

  <CodeBlockTab value="bun">
    ```bash
    bun add @prisma/client
    ```
  </CodeBlockTab>
</CodeBlockTabs>

`prisma` can be installed as a dev dependency using the following command:

<CodeBlockTabs defaultValue="npm">
  <CodeBlockTabsList>
    <CodeBlockTabsTrigger value="npm">
      npm
    </CodeBlockTabsTrigger>

    <CodeBlockTabsTrigger value="pnpm">
      pnpm
    </CodeBlockTabsTrigger>

    <CodeBlockTabsTrigger value="yarn">
      yarn
    </CodeBlockTabsTrigger>

    <CodeBlockTabsTrigger value="bun">
      bun

~~~
    </CodeBlockTabsTrigger>
  </CodeBlockTabsList>

  <CodeBlockTab value="npm">
    ```bash
    npm install -D prisma
    ```
  </CodeBlockTab>

  <CodeBlockTab value="pnpm">
    ```bash
    pnpm add -D prisma
    ```
  </CodeBlockTab>

  <CodeBlockTab value="yarn">
    ```bash
    yarn add --dev prisma
    ```
  </CodeBlockTab>

  <CodeBlockTab value="bun">
    ```bash
    bun add --dev prisma
    ```
  </CodeBlockTab>
</CodeBlockTabs>

Generate a `schema.prisma` file in the `prisma` directory by running the following command:

```bash title="Terminal"
npx prisma init
```

You can now replace the contents of the `schema.prisma` file with the following:

```prisma title="prisma/schema.prisma"
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

// Will be deleted. Just need it to generate the prisma client
model Test {
  id   Int    @id @default(autoincrement())
  name String
}
```

Ensure that you update the `DATABASE_URL` in your `.env` file to point to the location of your database.

```txt title=".env"
DATABASE_URL="file:./dev.db"
```

Run the following command to generate the Prisma client & sync the database:

```bash title="Terminal"
npx prisma db push
```

### Install & Configure Better Auth

Follow steps 1 & 2 from the [installation guide](/docs/installation) to install Better Auth in your Nitro application & set up the environment variables.

Once that is done, create your Better Auth instance within the `server/utils/auth.ts` file.

```ts title="server/utils/auth.ts"
import { betterAuth } from "better-auth";
import { prismaAdapter } from "better-auth/adapters/prisma";
import { PrismaClient } from "@prisma/client";

const prisma = new PrismaClient();
export const auth = betterAuth({
  database: prismaAdapter(prisma, { provider: "sqlite" }),
  emailAndPassword: { enabled: true },
});
```

### Update Prisma Schema

Use the Better Auth CLI to update your Prisma schema with the required models by running the following command:

```bash title="Terminal"
npx @better-auth/cli generate --config server/utils/auth.ts
```

<Callout>
  The `--config` flag is used to specify the path to the file where you have created your Better Auth instance.
</Callout>

Head over to the `prisma/schema.prisma` file & save the file to trigger the format on save.

After saving the file, you can run the `npx prisma db push` command to update the database schema.

## Mount The Handler

You can now mount the Better Auth handler in your Nitro application. You can do this by adding the following code to your `server/routes/api/auth/[...all].ts` file:

```ts title="server/routes/api/auth/[...all].ts"
export default defineEventHandler((event) => {
  return auth.handler(toWebRequest(event));
});
```

<Callout>
  This is a [catch-all](https://nitro.build/guide/routing#catch-all-route) route that will handle all requests to `/api/auth/*`.
</Callout>

### CORS

You can configure CORS for your Nitro app by creating a plugin.

Start by installing the cors package:

<CodeBlockTabs defaultValue="npm">
  <CodeBlockTabsList>
    <CodeBlockTabsTrigger value="npm">
      npm
    </CodeBlockTabsTrigger>

    <CodeBlockTabsTrigger value="pnpm">
      pnpm
    </CodeBlockTabsTrigger>

    <CodeBlockTabsTrigger value="yarn">
~~~

      yarn
    </CodeBlockTabsTrigger>

    <CodeBlockTabsTrigger value="bun">
      bun
    </CodeBlockTabsTrigger>
  </CodeBlockTabsList>

  <CodeBlockTab value="npm">
    ```bash
    npm install cors
    ```
  </CodeBlockTab>

  <CodeBlockTab value="pnpm">
    ```bash
    pnpm add cors
    ```
  </CodeBlockTab>

  <CodeBlockTab value="yarn">
    ```bash
    yarn add cors
    ```
  </CodeBlockTab>

  <CodeBlockTab value="bun">
    ```bash
    bun add cors
    ```
  </CodeBlockTab>
</CodeBlockTabs>

You can now create a new file `server/plugins/cors.ts` and add the following code:

```ts title="server/plugins/cors.ts"
import cors from "cors";
export default defineNitroPlugin((plugin) => {
  plugin.h3App.use(
    fromNodeMiddleware(
      cors({
        origin: "*",
      }),
    ),
  );
});
```

<Callout>
  This will enable CORS for all routes. You can customize the `origin` property to allow requests from specific domains. Ensure that the config is in sync with your frontend application.
</Callout>

### Auth Guard/Middleware

You can add an auth guard to your Nitro application to protect routes that require authentication. You can do this by creating a new file `server/utils/require-auth.ts` and adding the following code:

```ts title="server/utils/require-auth.ts"
import { EventHandler, H3Event } from "h3";
import { fromNodeHeaders } from "better-auth/node";

/**
 * Middleware used to require authentication for a route.
 *
 * Can be extended to check for specific roles or permissions.
 */
export const requireAuth: EventHandler = async (event: H3Event) => {
  const headers = event.headers;

  const session = await auth.api.getSession({
    headers: headers,
  });
  if (!session)
    throw createError({
      statusCode: 401,
      statusMessage: "Unauthorized",
    });
  // You can save the session to the event context for later use
  event.context.auth = session;
};
```

You can now use this event handler/middleware in your routes to protect them:

```ts title="server/routes/api/secret.get.ts"
// Object syntax of the route handler
export default defineEventHandler({
  // The user has to be logged in to access this route
  onRequest: [requireAuth],
  handler: async (event) => {
    setResponseStatus(event, 201, "Secret data");
    return { message: "Secret data" };
  },
});
```

### Example

You can find an example of a Nitro application integrated with Better Auth & Prisma [here](https://github.com/BayBreezy/nitrojs-better-auth-prisma).

# integrations: Nuxt Integration
URL: /docs/integrations/nuxt
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/integrations/nuxt.mdx

Integrate Better Auth with Nuxt.

***

title: Nuxt Integration
description: Integrate Better Auth with Nuxt.

Before you start, make sure you have a Better Auth instance configured. If you haven't done that yet, check out the [installation](/docs/installation).

### Create API Route

We need to mount the handler to an API route. Create a file inside `/server/api/auth` called `[...all].ts` and add the following code:

```ts title="server/api/auth/[...all].ts"
import { auth } from "~/lib/auth"; // import your auth config

export default defineEventHandler((event) => {
	return auth.handler(toWebRequest(event));
});
```

<Callout type="info">
  You can change the path on your better-auth configuration but it's recommended to keep it as `/api/auth/[...all]`
</Callout>

### Migrate the database

Run the following command to create the necessary tables in your database:

```bash
npx @better-auth/cli migrate
```

## Create a client

Create a client instance. You can name the file anything you want. Here we are creating `client.ts` file inside the `lib/` directory.

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/vue" // make sure to import from better-auth/vue

export const authClient = createAuthClient({
    //you can pass client configuration here
})
```

Once you have created the client, you can use it to sign up, sign in, and perform other actions.
Some of the actions are reactive.

### Example usage

```vue title="index.vue"
<script setup lang="ts">
import { authClient } from "~/lib/client"
const session = authClient.useSession()
</script>

<template>
    <div>
        <button v-if="!session?.data" @click="() => authClient.signIn.social({
            provider: 'github'
        })">
            Continue with GitHub
        </button>
        <div>
            <pre>{{ session.data }}</pre>
            <button v-if="session.data" @click="authClient.signOut()">
                Sign out
            </button>
        </div>
    </div>
</template>
```

### Server Usage

The `api` object exported from the auth instance contains all the actions that you can perform on the server. Every endpoint made inside Better Auth is a invocable as a function.
Including plugins endpoints.

Example: Getting Session on a server API route

```tsx title="server/api/example.ts"
import { auth } from "~/lib/auth";

export default defineEventHandler((event) => {
    const session = await auth.api.getSession({
      headers: event.headers
    });

   if(session) {
     // access the session.session && session.user
   }
});
```

### SSR Usage

If you are using Nuxt with SSR, you can use the `useSession` function in the `setup` function of your page component and pass `useFetch` to make it work with SSR.

```vue title="index.vue"
<script setup lang="ts">
import { authClient } from "~/lib/auth-client";

const { data: session } = await authClient.useSession(useFetch);
</script>

<template>
    <p>
        {{ session }}
    </p>
</template>
```

### Middleware

To add middleware to your Nuxt project, you can use the `useSession` method from the client.

```ts title="middleware/auth.global.ts"
import { authClient } from "~/lib/auth-client";
export default defineNuxtRouteMiddleware(async (to, from) => {
        const { data: session } = await authClient.useSession(useFetch);
        if (!session.value) {
                if (to.path === "/dashboard") {
                        return navigateTo("/");
                }
        }
});
```

### Resources & Examples

* [Nuxt and Nuxt Hub example](https://github.com/atinux/nuxthub-better-auth) on GitHub.
* [NuxtZzle is Nuxt,Drizzle ORM example](https://github.com/leamsigc/nuxt-better-auth-drizzle) on GitHub [preview](https://nuxt-better-auth.giessen.dev/)
* [Nuxt example](https://stackblitz.com/github/better-auth/examples/tree/main/nuxt-example) on StackBlitz.
* [NuxSaaS (Github)](https://github.com/NuxSaaS/NuxSaaS) is a full-stack SaaS Starter Kit that leverages Better Auth for secure and efficient user authentication. [Demo](https://nuxsaas.com/)
* [NuxtOne (Github)](https://github.com/nuxtone/nuxt-one) is a Nuxt-based starter template for building AIaaS (AI-as-a-Service) applications [preview](https://www.one.devv.zone)

# integrations: Remix Integration

URL: /docs/integrations/remix
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/integrations/remix.mdx

Integrate Better Auth with Remix.

***

title: Remix Integration
description: Integrate Better Auth with Remix.

Better Auth can be easily integrated with Remix. This guide will show you how to integrate Better Auth with Remix.

You can follow the steps from [installation](/docs/installation) to get started or you can follow this guide to make it the Remix-way.

If you have followed the installation steps, you can skip the first step.

## Create auth instance

Create a file named `auth.server.ts` in one of these locations:

* Project root
* `lib/` folder
* `utils/` folder

You can also nest any of these folders under `app/` folder. (e.g. `app/lib/auth.server.ts`)

And in this file, import Better Auth and create your instance.

<Callout type="warn">
  Make sure to export the auth instance with the variable name `auth` or as a `default` export.
</Callout>

```ts title="app/lib/auth.server.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    database: {
        provider: "postgres", //change this to your database provider
        url: process.env.DATABASE_URL, // path to your database or connection string
    }
})
```

## Create API Route

We need to mount the handler to a API route. Create a resource route file `api.auth.$.ts` inside `app/routes/` directory. And add the following code:

```ts title="app/routes/api.auth.$.ts"
import { auth } from '~/lib/auth.server' // Adjust the path as necessary
import type { LoaderFunctionArgs, ActionFunctionArgs } from "@remix-run/node"

export async function loader({ request }: LoaderFunctionArgs) {
    return auth.handler(request)
}

export async function action({ request }: ActionFunctionArgs) {
    return auth.handler(request)
}
```

<Callout type="info">
  You can change the path on your better-auth configuration but it's recommended to keep it as `routes/api.auth.$.ts`
</Callout>

## Create a client

Create a client instance. Here we are creating `auth-client.ts` file inside the `lib/` directory.

```ts title="app/lib/auth-client.ts"
import { createAuthClient } from "better-auth/react" // make sure to import from better-auth/react

export const authClient = createAuthClient({
    //you can pass client configuration here
})
```

Once you have created the client, you can use it to sign up, sign in, and perform other actions.

### Example usage

#### Sign Up

```ts title="app/routes/signup.tsx"
import { Form } from "@remix-run/react"
import { useState } from "react"
import { authClient } from "~/lib/auth-client"

export default function SignUp() {
  const [email, setEmail] = useState("")
  const [name, setName] = useState("")
  const [password, setPassword] = useState("")

  const signUp = async () => {
    await authClient.signUp.email(
      {
        email,
        password,
        name,
      },
      {
        onRequest: (ctx) => {
          // show loading state
        },
        onSuccess: (ctx) => {
          // redirect to home
        },
        onError: (ctx) => {
          alert(ctx.error)
        },
      },
    )
  }

  return (
    <div>
      <h2>
        Sign Up
      </h2>
      <Form
        onSubmit={signUp}
      >
        <input
          type="text"
          value={name}
          onChange={(e) => setName(e.target.value)}
          placeholder="Name"
        />
        <input
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          placeholder="Email"
        />
        <input
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          placeholder="Password"
        />
        <button
          type="submit"
        >
```

```
          Sign Up
        </button>
      </Form>
    </div>
  )
}

```

#### Sign In

```ts title="app/routes/signin.tsx"
import { Form } from "@remix-run/react"
import { useState } from "react"
import { authClient } from "~/services/auth-client"

export default function SignIn() {
  const [email, setEmail] = useState("")
  const [password, setPassword] = useState("")

  const signIn = async () => {
    await authClient.signIn.email(
      {
        email,
        password,
      },
      {
        onRequest: (ctx) => {
          // show loading state
        },
        onSuccess: (ctx) => {
          // redirect to home
        },
        onError: (ctx) => {
          alert(ctx.error)
        },
      },
    )
  }

  return (
    <div>
      <h2>
        Sign In
      </h2>
      <Form onSubmit={signIn}>
        <input
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
        />
        <input
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
        />
        <button
          type="submit"
        >
          Sign In
        </button>
      </Form>
    </div>
  )
}
```

# integrations: SolidStart Integration

URL: /docs/integrations/solid-start
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/integrations/solid-start.mdx

Integrate Better Auth with SolidStart.

***

title: SolidStart Integration
description: Integrate Better Auth with SolidStart.

Before you start, make sure you have a Better Auth instance configured. If you haven't done that yet, check out the [installation](/docs/installation).

### Mount the handler

We need to mount the handler to SolidStart server. Put the following code in your `*auth.ts` file inside `/routes/api/auth` folder.

```ts title="*auth.ts"
import { auth } from "~/lib/auth";
import { toSolidStartHandler } from "better-auth/solid-start";

export const { GET, POST } = toSolidStartHandler(auth);
```

# integrations: SvelteKit Integration

URL: /docs/integrations/svelte-kit
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/integrations/svelte-kit.mdx

Integrate Better Auth with SvelteKit.

***

title: SvelteKit Integration
description: Integrate Better Auth with SvelteKit.

Before you start, make sure you have a Better Auth instance configured. If you haven't done that yet, check out the [installation](/docs/installation).

### Mount the handler

We need to mount the handler to SvelteKit server hook.

```ts title="hooks.server.ts"
import { auth } from "$lib/auth";
import { svelteKitHandler } from "better-auth/svelte-kit";
import { building } from "$app/environment";

export async function handle({ event, resolve }) {
  return svelteKitHandler({ event, resolve, auth, building });
}
```

### Populate session data in the event (`event.locals`)

The `svelteKitHandler` does not automatically populate `event.locals.user` or `event.locals.session`. If you want to access the current session in your server code (e.g., in `+layout.server.ts`, actions, or endpoints), populate `event.locals` in your `handle` hook:

```ts title="hooks.server.ts"
import { auth } from "$lib/auth";
import { svelteKitHandler } from "better-auth/svelte-kit";
import { building } from "$app/environment";

export async function handle({ event, resolve }) {
  // Fetch current session from Better Auth
  const session = await auth.api.getSession({
    headers: event.request.headers,
  });

  // Make session and user available on server
  if (session) {
    event.locals.session = session.session;
    event.locals.user = session.user;
  }

  return svelteKitHandler({ event, resolve, auth, building });
}
```

### Server Action Cookies

To ensure cookies are properly set when you call functions like `signInEmail` or `signUpEmail` in a server action, you should use the `sveltekitCookies` plugin. This plugin will automatically handle setting cookies for you in SvelteKit.

You need to add it as a plugin to your Better Auth instance.

<Callout>
  The `getRequestEvent` function is available in SvelteKit `2.20.0` and later.
  Make sure you are using a compatible version.
</Callout>

```ts title="lib/auth.ts"
import { betterAuth } from "better-auth";
import { sveltekitCookies } from "better-auth/svelte-kit";
import { getRequestEvent } from "$app/server";

export const auth = betterAuth({
  // ... your config
  plugins: [sveltekitCookies(getRequestEvent)], // make sure this is the last plugin in the array
});
```

## Create a client

Create a client instance. You can name the file anything you want. Here we are creating `client.ts` file inside the `lib/` directory.

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/svelte"; // make sure to import from better-auth/svelte

export const authClient = createAuthClient({
  // you can pass client configuration here
});
```

Once you have created the client, you can use it to sign up, sign in, and perform other actions.
Some of the actions are reactive. The client use [nano-store](https://github.com/nanostores/nanostores) to store the state and reflect changes when there is a change like a user signing in or out affecting the session state.

### Example usage

```svelte
<script lang="ts">
  import { authClient } from "$lib/client";
  const session = authClient.useSession();
</script>
    <div>
      {#if $session.data}
        <div>
          <p>
            {$session.data.user.name}
          </p>
          <button
            on:click={async () => {
              await authClient.signOut();
            }}
          >
            Sign Out
          </button>
        </div>
      {:else}
        <button
          on:click={async () => {
            await authClient.signIn.social({
              provider: "github",
            });
          }}
        >
          Continue with GitHub
        </button>
      {/if}
    </div>
```

# integrations: TanStack Start Integration
URL: /docs/integrations/tanstack
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/integrations/tanstack.mdx

Integrate Better Auth with TanStack Start.

***

title: TanStack Start Integration
description: Integrate Better Auth with TanStack Start.

This integration guide is assuming you are using TanStack Start.

Before you start, make sure you have a Better Auth instance configured. If you haven't done that yet, check out the [installation](/docs/installation).

### Mount the handler

We need to mount the handler to a TanStack API endpoint/Server Route.
Create a new file: `/src/routes/api/auth/$.ts`

```ts title="src/routes/api/auth/$.ts"
import { auth } from '@/lib/auth'
import { createFileRoute } from '@tanstack/react-router'
```

```
export const Route = createFileRoute('/api/auth/$')({
  server: {
    handlers: {
      GET: ({ request }) => {
        return auth.handler(request)
      },
      POST: ({ request }) => {
        return auth.handler(request)
      },
    },
  },
})
```

If you haven't created your server route handler yet, you can do so by creating a file: `/src/server.ts`

```ts title="src/server.ts"
import {
  createStartHandler,
  defaultStreamHandler,
} from '@tanstack/react-start/server'
import { createRouter } from './router'

export default createStartHandler({
  createRouter,
})(defaultStreamHandler)
```

### Usage tips

* We recommend using the client SDK or `authClient` to handle authentication, rather than server actions with `auth.api`.
* When you call functions that need to set cookies (like `signInEmail` or `signUpEmail`), you'll need to handle cookie setting for TanStack Start. Better Auth provides a `reactStartCookies` plugin to automatically handle this for you.

```ts title="src/lib/auth.ts"
import { betterAuth } from "better-auth";
import { reactStartCookies } from "better-auth/react-start";

export const auth = betterAuth({
    //...your config
    plugins: [reactStartCookies()] // make sure this is the last plugin in the array
})
```

Now, when you call functions that set cookies, they will be automatically set using TanStack Start's cookie handling system.

```ts
import { auth } from "@/lib/auth"

const signIn = async () => {
    await auth.api.signInEmail({
        body: {
            email: "user@email.com",
            password: "password",
        }
    })
}
```

# integrations: Waku Integration

URL: /docs/integrations/waku
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/integrations/waku.mdx

Integrate Better Auth with Waku.

***

title: Waku Integration
description: Integrate Better Auth with Waku.

Better Auth can be easily integrated with Waku. Before you start, make sure you have a Better Auth instance configured. If you haven't done that yet, check out the [installation](/docs/installation).

## Create auth instance

Create a file named `auth.ts` in your application. Import Better Auth and create your instance.

<Callout type="warn">
  Make sure to export the auth instance with the variable name `auth` or as a `default` export.
</Callout>

```ts title="src/auth.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    database: {
        provider: "postgres", //change this to your database provider
        url: process.env.DATABASE_URL, // path to your database or connection string
    }
})
```

## Create API Route

We need to mount the handler to a API route. Create a directory for Waku's file system router at `src/pages/api/auth`. Create a catch-all route file `[...route].ts` inside the `src/pages/api/auth` directory. And add the following code:

```ts title="src/pages/api/auth/[...route].ts"
import { auth } from "../../../auth" // Adjust the path as necessary

export const GET = async (request: Request): Promise<Response> => {
  return auth.handler(request)
}

export const POST = async (request: Request): Promise<Response> => {
  return auth.handler(request)
}
```

<Callout type="info">
  You can change the path on your better-auth configuration but it's recommended to keep it as `src/pages/api/auth/[...route].ts`
</Callout>

## Create a client

Create a client instance. Here we are creating `auth-client.ts` file inside the `lib/` directory.

```ts title="src/lib/auth-client.ts"
import { createAuthClient } from "better-auth/react" // make sure to import from better-auth/react

export const authClient = createAuthClient({
    //you can pass client configuration here
})

export type Session = typeof authClient.$Infer.Session // you can infer typescript types from the authClient
```

Once you have created the client, you can use it to sign up, sign in, and perform other actions.
Some of the actions are reactive. The client uses [nano-store](https://github.com/nanostores/nanostores) to store the state and re-render the components when the state changes.

The client also uses [better-fetch](https://github.com/bekacru/better-fetch) to make the requests. You can pass the fetch configuration to the client.

## RSC and Server actions

The `api` object exported from the auth instance contains all the actions that you can perform on the server. Every endpoint made inside Better Auth is a invocable as a function. Including plugins endpoints.

Example: Getting Session on a server action

```tsx title="server.ts"
"use server" // Waku currently only supports file-level "use server"

import { auth } from "./auth"
import { getContext } from "waku/middleware/context"

export const someAuthenticatedAction = async () => {
  "use server"
  const session = await auth.api.getSession({
    headers: new Headers(getContext().req.headers),
  })
};
```

Example: Getting Session on a RSC

```tsx
import { auth } from "../auth"
import { getContext } from "waku/middleware/context"

export async function ServerComponent() {
    const session = await auth.api.getSession({
        headers: new Headers(getContext().req.headers),
    })
    if(!session) {
        return <div>Not authenticated</div>
    }
    return (
        <div>
            <h1>Welcome {session.user.name}</h1>
        </div>
    )
}
```

<Callout type="warn">RSCs that run after the response has started streaming cannot set cookies. The [cookie cache](/docs/concepts/session-management#cookie-cache) will not be refreshed until the server is interacted with from the client via Server Actions or Route Handlers.</Callout>

### Server Action Cookies

When you call a function that needs to set cookies, like `signInEmail` or `signUpEmail` in a server action, cookies wonâ€™t be set.

We can create a plugin that works together with our middleware to set cookies.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { wakuCookies } from "better-auth/waku";
import { getContextData } from "waku/middleware/context";

export const auth = betterAuth({
    //...your config
    plugins: [wakuCookies()] // make sure this is the last plugin in the array // [!code highlight]
})

function wakuCookies() {
  return {
    id: "waku-cookies",
    hooks: {
      after: [
        {
          matcher(ctx) {
            return true;
          },
          handler: createAuthMiddleware(async (ctx) => {
            const returned = ctx.context.responseHeaders;
            if ("_flag" in ctx && ctx._flag === "router") {
              return;
            }
            if (returned instanceof Headers) {
              const setCookieHeader = returned?.get("set-cookie");
              if (!setCookieHeader) return;
              const contextData = getContextData();
              contextData.betterAuthSetCookie = setCookieHeader;
            }
          }),
        },
      ],
    },
  } satisfies BetterAuthPlugin;
}
```

See below for the middleware to create to add the `contextData.betterAuthSetCookie` cookies to the response.
Now, when you call functions that set cookies, they will be automatically set.

```ts
"use server";
import { auth } from "../auth"

const signIn = async () => {
    await auth.api.signInEmail({
        body: {
            email: "user@email.com",
            password: "password",
        }
    })
}
```

### Middleware

In Waku middleware, it's recommended to only check for the existence of a session cookie to handle redirection. This avoids blocking requests by making API or database calls.

You can use the `getSessionCookie` helper from Better Auth for this purpose:

<Callout type="warn">
  The <code>getSessionCookie()</code> function does not automatically reference the auth config specified in <code>auth.ts</code>. Therefore, if you customized the cookie name or prefix, you need to ensure that the configuration in <code>getSessionCookie()</code> matches the config defined in your <code>auth.ts</code>.
</Callout>

```ts title="src/middleware/auth.ts"
import type { Middleware } from "waku/config"
import { getSession } from "../auth"
import { getSessionCookie } from "better-auth/cookies"
```

~~~
const authMiddleware: Middleware = () => {
    return async (ctx, next) => {
        const sessionCookie = getSessionCookie(
            new Request(ctx.req.url, {
                body: ctx.req.body,
                headers: ctx.req.headers,
                method: ctx.req.method,
            })
        )
        // THIS IS NOT SECURE!
        // This is the recommended approach to optimistically redirect users
        // We recommend handling auth checks in each page/route
        if (!sessionCookie && ctx.req.url.pathname !== "/") {
            if (!ctx.req.url.pathname.endsWith(".txt")) {
                // Currently RSC requests end in .txt and don't handle redirect responses
                // The redirect needs to be encoded in the React flight stream somehow
                // There is some functionality in Waku to do this from a server component
                // but not from middleware.
                ctx.res.status = 302;
                ctx.res.headers = {
                  Location: new URL("/", ctx.req.url).toString(),
                };
            }
        }

        // TODO possible to inspect ctx.req.url and not do this on every request
        // Or skip starting the promise here and just invoke from server components and functions
        getSession()
        await next()
        if (ctx.data.betterAuthSetCookie) {
            ctx.res.headers ||= {}
            let origSetCookie = ctx.res.headers["set-cookie"] || ([] as string[])
            if (typeof origSetCookie === "string") {
                origSetCookie = [origSetCookie]
            }
            ctx.res.headers["set-cookie"] = [
                ...origSetCookie,
                ctx.data.betterAuthSetCookie as string,
            ]
        }
    }
};

export default authMiddleware;
```

<Callout type="warn">
  **Security Warning:** The `getSessionCookie` function only checks for the
  existence of a session cookie; it does **not** validate it. Relying solely
  on this check for security is dangerous, as anyone can manually create a
  cookie to bypass it. You must always validate the session on your server for
  any protected actions or pages.
</Callout>

<Callout type="info">
  If you have a custom cookie name or prefix, you can pass it to the `getSessionCookie` function.

  ```ts
  const sessionCookie = getSessionCookie(request, {
      cookieName: "my_session_cookie",
      cookiePrefix: "my_prefix"
  })
  ```
</Callout>

Alternatively, you can use the `getCookieCache` helper to get the session object from the cookie cache.

```ts
import { getCookieCache } from "better-auth/cookies"

const authMiddleware: Middleware = () => {
    return async (ctx, next) => {
        const session = await getCookieCache(ctx.req)
        if (!session && ctx.req.url.pathname !== "/") {
            if (!ctx.req.url.pathname.endsWith(".txt")) {
                ctx.res.status = 302
                ctx.res.headers = {
                    Location: new URL("/", ctx.req.url).toString(),
                }
            }
        }
    }
    await next();
  }
}

export default authMiddleware;
```

Note that your middleware will need to be added to a waku.config.ts file (create this file if it doesn't already exist in your project):

```ts title="waku.config.ts"
import { defineConfig } from "waku/config";

export default defineConfig({
  middleware: [
    "waku/middleware/context",
    "waku/middleware/dev-server",
    "./src/middleware/auth.ts",
    "waku/middleware/handler",
  ],
});
```

### How to handle auth checks in each page/route

In this example, we are using the `auth.api.getSession` function within a server component to get the session object,
then we are checking if the session is valid. If it's not, we are redirecting the user to the sign-in page.
Waku has `getContext` to get the request headers and `getContextData()` to store data per request. We can use this
to avoid fetching the session more than once per request.

```ts title="auth.ts"
import { getContext, getContextData } from "waku/middleware/context";

// Code from above to create the server auth config
// export const auth = ...
~~~

~~~
export function getSession(): Promise<Session | null> {
  const contextData = getContextData();
  const ctx = getContext();
  const existingSessionPromise = contextData.sessionPromise as
    | Promise<Session | null>
    | undefined;
  if (existingSessionPromise) {
    return existingSessionPromise;
  }
  const sessionPromise = auth.api.getSession({
    headers: new Headers(ctx.req.headers),
  });
  contextData.sessionPromise = sessionPromise;
  return sessionPromise;
}
```

```tsx title="src/pages/dashboard.tsx"
import { getSession } from "../auth";
import { unstable_redirect as redirect } from 'waku/router/server';

export default async function DashboardPage() {
    const session = await getSession()

    if (!session) {
        redirect("/sign-in")
    }

    return (
        <div>
            <h1>Welcome {session.user.name}</h1>
        </div>
    )
}
```

### Example usage

#### Sign Up

```ts title="src/components/signup.tsx"
"use client"

import { useState } from "react"
import { authClient } from "../lib/auth-client"

export default function SignUp() {
  const [email, setEmail] = useState("")
  const [name, setName] = useState("")
  const [password, setPassword] = useState("")

  const signUp = async () => {
    await authClient.signUp.email(
      {
        email,
        password,
        name,
      },
      {
        onRequest: (ctx) => {
          // show loading state
        },
        onSuccess: (ctx) => {
          // redirect to home
        },
        onError: (ctx) => {
          alert(ctx.error)
        },
      },
    )
  }

  return (
    <div>
      <h2>
        Sign Up
      </h2>
      <form
        onSubmit={signUp}
      >
        <input
          type="text"
          value={name}
          onChange={(e) => setName(e.target.value)}
          placeholder="Name"
        />
        <input
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          placeholder="Email"
        />
        <input
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          placeholder="Password"
        />
        <button
          type="submit"
        >
          Sign Up
        </button>
      </form>
    </div>
  )
}

```

#### Sign In

```ts title="src/components/signin.tsx"
"use client"

import { useState } from "react"
import { authClient } from "../lib/auth-client"

export default function SignIn() {
  const [email, setEmail] = useState("")
  const [password, setPassword] = useState("")
~~~

~~~~
  const signIn = async () => {
    await authClient.signIn.email(
      {
        email,
        password,
      },
      {
        onRequest: (ctx) => {
          // show loading state
        },
        onSuccess: (ctx) => {
          // redirect to home
        },
        onError: (ctx) => {
          alert(ctx.error)
        },
      },
    )
  }

  return (
    <div>
      <h2>
        Sign In
      </h2>
      <form onSubmit={signIn}>
        <input
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
        />
        <input
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
        />
        <button
          type="submit"
        >
          Sign In
        </button>
      </form>
    </div>
  )
}
```



# plugins: Two-Factor Authentication (2FA)
URL: /docs/plugins/2fa
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/2fa.mdx

Enhance your app's security with two-factor authentication.

***

title: Two-Factor Authentication (2FA)
description: Enhance your app's security with two-factor authentication.
------------------------------------------------------------------------

`OTP` `TOTP` `Backup Codes` `Trusted Devices`

Two-Factor Authentication (2FA) adds an extra security step when users log in. Instead of just using a password, they'll need to provide a second form of verification. This makes it much harder for unauthorized people to access accounts, even if they've somehow gotten the password.

This plugin offers two main methods to do a second factor verification:

1. **OTP (One-Time Password)**: A temporary code sent to the user's email or phone.
2. **TOTP (Time-based One-Time Password)**: A code generated by an app on the user's device.

**Additional features include:**

* Generating backup codes for account recovery
* Enabling/disabling 2FA
* Managing trusted devices

## Installation

<Steps>
  <Step>
    ### Add the plugin to your auth config

    Add the two-factor plugin to your auth configuration and specify your app name as the issuer.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { twoFactor } from "better-auth/plugins" // [!code highlight]

    export const auth = betterAuth({
        // ... other config options
        appName: "My App", // provide your app name. It'll be used as an issuer. // [!code highlight]
        plugins: [
            twoFactor() // [!code highlight]
        ]
    })
    ```
  </Step>

  <Step>
    ### Migrate the database

    Run the migration or generate the schema to add the necessary fields and tables to the database.

    <Tabs items={["migrate", "generate"]}>
      <Tab value="migrate">
        ```bash
        npx @better-auth/cli migrate
        ```
      </Tab>

      <Tab value="generate">
        ```bash
        npx @better-auth/cli generate
        ```
      </Tab>
    </Tabs>

    See the [Schema](#schema) section to add the fields manually.
  </Step>

  <Step>
    ### Add the client plugin

    Add the client plugin and Specify where the user should be redirected if they need to verify 2nd factor

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    import { twoFactorClient } from "better-auth/client/plugins"
~~~~

    export const authClient = createAuthClient({
        plugins: [
            twoFactorClient()
        ]
    })
    ~~~
  </Step>
</Steps>

## Usage

### Enabling 2FA

To enable two-factor authentication, call `twoFactor.enable` with the user's password and issuer (optional):

### Client Side

```ts
const { data, error } = await authClient.twoFactor.enable({
    password: secure-password,
    issuer: my-app-name, // required
});
~~~

### Server Side

```ts
const data = await auth.api.enableTwoFactor({
    body: {
        password: secure-password,
        issuer: my-app-name, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type enableTwoFactor = {
    /**
     * The user's password
     */
    password: string = "secure-password"
    /**
     * An optional custom issuer for the TOTP URI. Defaults to app-name defined in your auth config.
     */
    issuer?: string = "my-app-name"

}
```

When 2FA is enabled:

* An encrypted `secret` and `backupCodes` are generated.
* `enable` returns `totpURI` and `backupCodes`.

Note: `twoFactorEnabled` wonâ€™t be set to `true` until the user verifies their TOTP code. Learn more about veryifying TOTP [here](#totp). You can skip verification by setting `skipVerificationOnEnable` to true in your plugin config.

<Callout type="warn">
  Two Factor can only be enabled for credential accounts at the moment. For social accounts, it's assumed the provider already handles 2FA.
</Callout>

### Sign In with 2FA

When a user with 2FA enabled tries to sign in via email, the response object will contain `twoFactorRedirect` set to `true`. This indicates that the user needs to verify their 2FA code.

You can handle this in the `onSuccess` callback or by providing a `onTwoFactorRedirect` callback in the plugin config.

```ts title="sign-in.tsx"
await authClient.signIn.email({
        email: "user@example.com",
        password: "password123",
    },
    {
        async onSuccess(context) {
            if (context.data.twoFactorRedirect) {
                // Handle the 2FA verification in place
            }
        },
    }
)
```

Using the `onTwoFactorRedirect` config:

```ts title="sign-in.ts"
import { createAuthClient } from "better-auth/client";
import { twoFactorClient } from "better-auth/client/plugins";

const authClient = createAuthClient({
    plugins: [
        twoFactorClient({
            onTwoFactorRedirect(){
                // Handle the 2FA verification globally
            },
        }),
    ],
});
```

<Callout type="warn">
  With `auth.api`

  When you call `auth.api.signInEmail` on the server, and the user has 2FA enabled, it will return an object where `twoFactorRedirect` is set to `true`. This behavior isnâ€™t inferred in TypeScript, which can be misleading. You can check using `in` instead to check if `twoFactorRedirect` is set to `true`.

  ```ts
  const response = await auth.api.signInEmail({
        body: {
                email: "test@test.com",
                password: "test",
        },
  });

  if ("twoFactorRedirect" in response) {
        // Handle the 2FA verification in place
  }
  ```
</Callout>

### Disabling 2FA

To disable two-factor authentication, call `twoFactor.disable` with the user's password:

### Client Side

```ts
const { data, error } = await authClient.twoFactor.disable({
    password,
});
```

### Server Side

```ts
const data = await auth.api.disableTwoFactor({
    body: {
        password,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type disableTwoFactor = {
    /**
     * The user's password
     */
    password: string

}
```

### TOTP

TOTP (Time-Based One-Time Password) is an algorithm that generates a unique password for each login attempt using time as a counter. Every fixed interval (Better Auth defaults to 30 seconds), a new password is generated. This addresses several issues with traditional passwords: they can be forgotten, stolen, or guessed. OTPs solve some of these problems, but their delivery via SMS or email can be unreliable (or even risky, considering it opens new attack vectors).

TOTP, however, generates codes offline, making it both secure and convenient. You just need an authenticator app on your phone.

#### Getting TOTP URI

After enabling 2FA, you can get the TOTP URI to display to the user. This URI is generated by the server using the `secret` and `issuer` and can be used to generate a QR code for the user to scan with their authenticator app.

### Client Side

```ts
const { data, error } = await authClient.twoFactor.getTotpUri({
    password,
});
```

### Server Side

```ts
const data = await auth.api.getTOTPURI({
    body: {
        password,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type getTOTPURI = {
    /**
     * The user's password
     */
    password: string

}
```

Example: Using React

Once you have the TOTP URI, you can use it to generate a QR code for the user to scan with their authenticator app.

```tsx title="user-card.tsx"
import QRCode from "react-qr-code";

export default function UserCard({ password }: { password: string }){
    const { data: session } = client.useSession();
        const { data: qr } = useQuery({
                queryKey: ["two-factor-qr"],
                queryFn: async () => {
                        const res = await authClient.twoFactor.getTotpUri({ password });
                        return res.data;
                },
                enabled: !!session?.user.twoFactorEnabled,
        });
    return (
        <QRCode value={qr?.totpURI || ""} />
   )
}
```

<Callout>
  By default the issuer for TOTP is set to the app name provided in the auth config or if not provided it will be set to `Better Auth`. You can override this by passing `issuer` to the plugin config.
</Callout>

#### Verifying TOTP

After the user has entered their 2FA code, you can verify it using `twoFactor.verifyTotp` method. `Better Auth` follows standard practice by accepting TOTP codes from one period before and one after the current code, ensuring users can authenticate even with minor time delays on their end.

### Client Side

```ts
const { data, error } = await authClient.twoFactor.verifyTotp({
    code: 012345,
    trustDevice, // required
});
```

### Server Side

```ts
const data = await auth.api.verifyTOTP({
    body: {
        code: 012345,
        trustDevice, // required
    }
});
```

### Type Definition

```ts
type verifyTOTP = {
    /**
     * The otp code to verify.
     */
    code: string = "012345"
    /**
     * If true, the device will be trusted for 30 days. It'll be refreshed on every sign in request within this time.
     */
    trustDevice?: boolean = true

}
```

### OTP

OTP (One-Time Password) is similar to TOTP but a random code is generated and sent to the user's email or phone.

Before using OTP to verify the second factor, you need to configure `sendOTP` in your Better Auth instance. This function is responsible for sending the OTP to the user's email, phone, or any other method supported by your application.

```ts title="auth.ts"
import { betterAuth } from "better-auth"
import { twoFactor } from "better-auth/plugins"

export const auth = betterAuth({
    plugins: [
        twoFactor({
                otpOptions: {
                                async sendOTP({ user, otp }, request) {
                    // send otp to user
                                },
                        },
        })
    ]
})
```

#### Sending OTP

Sending an OTP is done by calling the `twoFactor.sendOtp` function. This function will trigger your sendOTP implementation that you provided in the Better Auth configuration.

### Client Side

```ts
const { data, error } = await authClient.twoFactor.sendOtp({
    trustDevice, // required
});
```

### Server Side

```ts
const data = await auth.api.send2FaOTP({
    body: {
        trustDevice, // required
    }
});
```

### Type Definition

```ts
type send2FaOTP = {
    /**
     * If true, the device will be trusted for 30 days. It'll be refreshed on every sign in request within this time.
     */
    trustDevice?: boolean = true

}
```

#### Verifying OTP

After the user has entered their OTP code, you can verify it

### Client Side

```ts
const { data, error } = await authClient.twoFactor.verifyOtp({
    code: 012345,
    trustDevice, // required
});
```

### Server Side

```ts
const data = await auth.api.verifyOTP({
    body: {
        code: 012345,
        trustDevice, // required
    }
});
```

### Type Definition

```ts
type verifyOTP = {
    /**
     * The otp code to verify.
     */
    code: string = "012345"
    /**
     * If true, the device will be trusted for 30 days. It'll be refreshed on every sign in request within this time.
     */
    trustDevice?: boolean = true

}
```

### Backup Codes

Backup codes are generated and stored in the database. This can be used to recover access to the account if the user loses access to their phone or email.

#### Generating Backup Codes

Generate backup codes for account recovery:

### Client Side

```ts
const { data, error } = await authClient.twoFactor.generateBackupCodes({
    password,
});
```

### Server Side

```ts
const data = await auth.api.generateBackupCodes({
    body: {
        password,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type generateBackupCodes = {
    /**
     * The users password.
     */
    password: string

}
```

<Callout type="warn">
  When you generate backup codes, the old backup codes will be deleted and new ones will be generated.
</Callout>

#### Using Backup Codes

You can now allow users to provider backup code as account recover method.

### Client Side

```ts
const { data, error } = await authClient.twoFactor.verifyBackupCode({
    code: 123456,
    disableSession, // required
    trustDevice, // required
});
```

### Server Side

```ts
const data = await auth.api.verifyBackupCode({
    body: {
        code: 123456,
        disableSession, // required
        trustDevice, // required
    }
});
```

### Type Definition

```ts
type verifyBackupCode = {
    /**
     * A backup code to verify.
     */
    code: string = "123456"
    /**
     * If true, the session cookie will not be set.
     */
    disableSession?: boolean = false
    /**
     * If true, the device will be trusted for 30 days. It'll be refreshed on every sign in request within this time.
     */
    trustDevice?: boolean = true

}
```

<Callout>
  Once a backup code is used, it will be removed from the database and can't be used again.
</Callout>

#### Viewing Backup Codes

To display the backup codes to the user, you can call `viewBackupCodes` on the server. This will return the backup codes in the response. You should only this if the user has a fresh session - a session that was just created.

### Client Side

```ts
const { data, error } = await authClient.twoFactor.viewBackupCodes({
    userId: user-id, // required
});
```

### Server Side

```ts
const data = await auth.api.viewBackupCodes({
    body: {
        userId: user-id, // required
    }
});
```

### Type Definition

```ts
type viewBackupCodes = {
    /**
     * The user ID to view all backup codes.
     */
    userId?: string | null = "user-id"

}
```

### Trusted Devices

You can mark a device as trusted by passing `trustDevice` to `verifyTotp` or `verifyOtp`.

```ts
const verify2FA = async (code: string) => {
    const { data, error } = await authClient.twoFactor.verifyTotp({
        code,
        callbackURL: "/dashboard",
        trustDevice: true // Mark this device as trusted
    })
    if (data) {
        // 2FA verified and device trusted
    }
}
```

When `trustDevice` is set to `true`, the current device will be remembered for 60 days. During this period, the user won't be prompted for 2FA on subsequent sign-ins from this device. The trust period is refreshed each time the user signs in successfully.

~~~
### Issuer

By adding an `issuer` you can set your application name for the 2fa application.

For example, if your user uses Google Auth, the default appName will show up as `Better Auth`. However, by using the following code, it will show up as `my-app-name`.

```ts
twoFactor({
    issuer: "my-app-name" // [!code highlight]
})
```

***

## Schema

The plugin requires 1 additional fields in the `user` table and 1 additional table to store the two factor authentication data.

Table: `user`

<DatabaseTable
  fields={[
      { name: "twoFactorEnabled", type: "boolean", description: "Whether two factor authentication is enabled for the user.", isOptional: true },
  ]}
/>

Table: `twoFactor`

<DatabaseTable
  fields={[
      { name: "id", type: "string", description: "The ID of the two factor authentication.", isPrimaryKey: true },
      { name: "userId", type: "string", description: "The ID of the user", isForeignKey: true },
      { name: "secret", type: "string", description: "The secret used to generate the TOTP code.", isOptional: true },
      { name: "backupCodes", type: "string", description: "The backup codes used to recover access to the account if the user loses access to their phone or email.", isOptional: true },
  ]}
/>

## Options

### Server

**twoFactorTable**: The name of the table that stores the two factor authentication data. Default: `twoFactor`.

**skipVerificationOnEnable**: Skip the verification process before enabling two factor for a user.

**Issuer**: The issuer is the name of your application. It's used to generate TOTP codes. It'll be displayed in the authenticator apps.

**TOTP options**

these are options for TOTP.

<TypeTable
  type={{
  digits:{
      description: "The number of digits the otp to be",
      type: "number",
      default: 6,
  },
  period: {
      description: "The period for totp in seconds.",
      type: "number",
      default: 30,
  },
}}
/>

**OTP options**

these are options for OTP.

<TypeTable
  type={{
  sendOTP: {
      description: "a function that sends the otp to the user's email or phone number. It takes two parameters: user and otp",
      type: "function",
  },
  period: {
      description: "The period for otp in minutes.",
      type: "number",
      default: 3,
  },
  storeOTP: {
      description: "How to store the otp in the database. Whether to store it as plain text, encrypted or hashed. You can also provide a custom encryptor or hasher.",
      type: "string",
      default: "plain",
  },
}}
/>

**Backup Code Options**

backup codes are generated and stored in the database when the user enabled two factor authentication. This can be used to recover access to the account if the user loses access to their phone or email.

<TypeTable
  type={{
      amount: {
          description: "The amount of backup codes to generate",
          type: "number",
          default: 10,
      },
      length: {
          description: "The length of the backup codes",
          type: "number",
          default: 10,
      },
      customBackupCodesGenerate: {
          description: "A function that generates custom backup codes. It takes no parameters and returns an array of strings.",
          type: "function",
      },
      storeBackupCodes: {
          description: "How to store the backup codes in the database. Whether to store it as plain text or encrypted. You can also provide a custom encryptor.",
          type: "string",
          default: "plain",
      },
  }}
/>

### Client

To use the two factor plugin in the client, you need to add it on your plugins list.

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client"
import { twoFactorClient } from "better-auth/client/plugins"
~~~

~~~
const authClient =  createAuthClient({
    plugins: [
        twoFactorClient({ // [!code highlight]
            onTwoFactorRedirect(){ // [!code highlight]
                window.location.href = "/2fa" // Handle the 2FA verification redirect // [!code highlight]
            } // [!code highlight]
        }) // [!code highlight]
    ]
})
```

**Options**

`onTwoFactorRedirect`: A callback that will be called when the user needs to verify their 2FA code. This can be used to redirect the user to the 2FA page.



# plugins: Admin
URL: /docs/plugins/admin
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/admin.mdx

Admin plugin for Better Auth

***

title: Admin
description: Admin plugin for Better Auth
-----------------------------------------

The Admin plugin provides a set of administrative functions for user management in your application. It allows administrators to perform various operations such as creating
users, managing user roles, banning/unbanning users, impersonating users, and more.

## Installation

<Steps>
  <Step>
    ### Add the plugin to your auth config

    To use the Admin plugin, add it to your auth config.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { admin } from "better-auth/plugins" // [!code highlight]

    export const auth = betterAuth({
        // ... other config options
        plugins: [
            admin() // [!code highlight]
        ]
    })
    ```
  </Step>

  <Step>
    ### Migrate the database

    Run the migration or generate the schema to add the necessary fields and tables to the database.

    <Tabs items={["migrate", "generate"]}>
      <Tab value="migrate">
        ```bash
        npx @better-auth/cli migrate
        ```
      </Tab>

      <Tab value="generate">
        ```bash
        npx @better-auth/cli generate
        ```
      </Tab>
    </Tabs>

    See the [Schema](#schema) section to add the fields manually.
  </Step>

  <Step>
    ### Add the client plugin

    Next, include the admin client plugin in your authentication client instance.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    import { adminClient } from "better-auth/client/plugins"

    export const authClient = createAuthClient({
        plugins: [
            adminClient()
        ]
    })
    ```
  </Step>
</Steps>

## Usage

Before performing any admin operations, the user must be authenticated with an admin account. An admin is any user assigned the `admin` role or any user whose ID is included in
the `adminUserIds` option.

### Create User

Allows an admin to create a new user.

### Client Side

```ts
const { data, error } = await authClient.admin.createUser({
    email: user@example.com,
    password: some-secure-password,
    name: James Smith,
    role: user, // required
    data, // required
});
```

### Server Side

```ts
const newUser = await auth.api.createUser({
    body: {
        email: user@example.com,
        password: some-secure-password,
        name: James Smith,
        role: user, // required
        data, // required
    }
});
```

### Type Definition
~~~

```ts
type createUser = {
    /**
     * The email of the user.
     */
    email: string = "user@example.com"
    /**
     * The password of the user.
     */
    password: string = "some-secure-password"
    /**
     * The name of the user.
     */
    name: string = "James Smith"
    /**
     * A string or array of strings representing the roles to apply to the new user.
     */
    role?: string | string[] = "user"
    /**
     * Extra fields for the user. Including custom additional fields.
     */
    data?: Record<string, any> = { customField: "customValue"
}
```

### List Users

Allows an admin to list all users in the database.

### Client Side

```ts
const { data, error } = await authClient.admin.listUsers({
    searchValue: some name, // required
    searchField: name, // required
    searchOperator: contains, // required
    limit, // required
    offset, // required
    sortBy: name, // required
    sortDirection: desc, // required
    filterField: email, // required
    filterValue: hello@example.com, // required
    filterOperator: eq, // required
});
```

### Server Side

```ts
const data = await auth.api.listUsers({
    query: {
        searchValue: some name, // required
        searchField: name, // required
        searchOperator: contains, // required
        limit, // required
        offset, // required
        sortBy: name, // required
        sortDirection: desc, // required
        filterField: email, // required
        filterValue: hello@example.com, // required
        filterOperator: eq, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type listUsers = {
  /**
   * The value to search for.
   */
  searchValue?: string = "some name"
  /**
   * The field to search in, defaults to email. Can be `email` or `name`.
   */
  searchField?: "email" | "name" = "name"
  /**
   * The operator to use for the search. Can be `contains`, `starts_with` or `ends_with`.
   */
  searchOperator?: "contains" | "starts_with" | "ends_with" = "contains"
  /**
   * The number of users to return. Defaults to 100.
   */
  limit?: string | number = 100
  /**
   * The offset to start from.
   */
  offset?: string | number = 100
  /**
   * The field to sort by.
   */
  sortBy?: string = "name"
  /**
   * The direction to sort by.
   */
  sortDirection?: "asc" | "desc" = "desc"
  /**
   * The field to filter by.
   */
  filterField?: string = "email"
  /**
   * The value to filter by.
   */
  filterValue?: string | number | boolean = "hello@example.com"
  /**
   * The operator to use for the filter.
   */
  filterOperator?: "eq" | "ne" | "lt" | "lte" | "gt" | "gte" = "eq"

}
```

#### Query Filtering

The `listUsers` function supports various filter operators including `eq`, `contains`, `starts_with`, and `ends_with`.

#### Pagination

The `listUsers` function supports pagination by returning metadata alongside the user list. The response includes the following fields:

```ts
{
  users: User[],   // Array of returned users
  total: number,   // Total number of users after filters and search queries
  limit: number | undefined,   // The limit provided in the query
  offset: number | undefined   // The offset provided in the query
}
```

##### How to Implement Pagination

To paginate results, use the `total`, `limit`, and `offset` values to calculate:

* **Total pages:** `Math.ceil(total / limit)`
* **Current page:** `(offset / limit) + 1`
* **Next page offset:** `Math.min(offset + limit, (total - 1))` â€“ The value to use as `offset` for the next page, ensuring it does not exceed the total number of pages.
* **Previous page offset:** `Math.max(0, offset - limit)` â€“ The value to use as `offset` for the previous page (ensuring it doesnâ€™t go below zero).

##### Example Usage

Fetching the second page with 10 users per page:

```ts title="admin.ts"
const pageSize = 10;
const currentPage = 2;

const users = await authClient.admin.listUsers({
    query: {
        limit: pageSize,
        offset: (currentPage - 1) * pageSize
    }
});

const totalUsers = users.total;
const totalPages = Math.ceil(totalUsers / pageSize)
```

### Set User Role

Changes the role of a user.

### Client Side

```ts
const { data, error } = await authClient.admin.setRole({
    userId: user-id, // required
    role: admin,
});
```

### Server Side

```ts
const data = await auth.api.setRole({
    body: {
        userId: user-id, // required
        role: admin,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type setRole = {
    /**
     * The user id which you want to set the role for.
     */
    userId?: string = "user-id"
    /**
     * The role to set, this can be a string or an array of strings.
     */
    role: string | string[] = "admin"

}
```

### Set User Password

Changes the password of a user.

### Client Side

```ts
const { data, error } = await authClient.admin.setUserPassword({
    newPassword: new-password,
    userId: user-id,
});
```

### Server Side

```ts
const data = await auth.api.setUserPassword({
    body: {
        newPassword: new-password,
        userId: user-id,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type setUserPassword = {
    /**
     * The new password.
     */
    newPassword: string = 'new-password'
    /**
     * The user id which you want to set the password for.
     */
    userId: string = 'user-id'

}
```

### Update user

Update a user's details.

### Client Side

```ts
const { data, error } = await authClient.admin.updateUser({
    userId: user-id,
    data,
});
```

### Server Side

```ts
const data = await auth.api.adminUpdateUser({
    body: {
        userId: user-id,
        data,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type adminUpdateUser = {
    /**
     * The user id which you want to update.
     */
    userId: string = "user-id"
    /**
     * The data to update.
     */
    data: Record<string, any> = { name: "John Doe"
}
```

### Ban User

Bans a user, preventing them from signing in and revokes all of their existing sessions.

### Client Side

```ts
const { data, error } = await authClient.admin.banUser({
    userId: user-id,
    banReason: Spamming, // required
    banExpiresIn, // required
});
```

### Server Side

```ts
await auth.api.banUser({
    body: {
        userId: user-id,
        banReason: Spamming, // required
        banExpiresIn, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type banUser = {
    /**
     * The user id which you want to ban.
     */
    userId: string = "user-id"
    /**
     * The reason for the ban.
     */
    banReason?: string = "Spamming"
    /**
     * The number of seconds until the ban expires. If not provided, the ban will never expire.
     */
    banExpiresIn?: number = 60 * 60 * 24 * 7

}
```

### Unban User

Removes the ban from a user, allowing them to sign in again.

### Client Side

```ts
const { data, error } = await authClient.admin.unbanUser({
    userId: user-id,
});
```

### Server Side

```ts
await auth.api.unbanUser({
    body: {
        userId: user-id,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type unbanUser = {
    /**
     * The user id which you want to unban.
     */
    userId: string = "user-id"

}
```

### List User Sessions

Lists all sessions for a user.

### Client Side

```ts
const { data, error } = await authClient.admin.listUserSessions({
    userId: user-id,
});
```

### Server Side

```ts
const data = await auth.api.listUserSessions({
    body: {
        userId: user-id,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type listUserSessions = {
    /**
     * The user id.
     */
    userId: string = "user-id"

}
```

### Revoke User Session

Revokes a specific session for a user.

### Client Side

```ts
const { data, error } = await authClient.admin.revokeUserSession({
    sessionToken: session_token_here,
});
```

### Server Side

```ts
const data = await auth.api.revokeUserSession({
    body: {
        sessionToken: session_token_here,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type revokeUserSession = {
    /**
     * The session token which you want to revoke.
     */
    sessionToken: string = "session_token_here"

}
```

### Revoke All Sessions for a User

Revokes all sessions for a user.

### Client Side

```ts
const { data, error } = await authClient.admin.revokeUserSessions({
    userId: user-id,
});
```

### Server Side

```ts
const data = await auth.api.revokeUserSessions({
    body: {
        userId: user-id,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type revokeUserSessions = {
    /**
     * The user id which you want to revoke all sessions for.
     */
    userId: string = "user-id"

}
```

### Impersonate User

This feature allows an admin to create a session that mimics the specified user. The session will remain active until either the browser session ends or it reaches 1 hour. You can change this duration by setting the `impersonationSessionDuration` option.

### Client Side

```ts
const { data, error } = await authClient.admin.impersonateUser({
    userId: user-id,
});
```

### Server Side

```ts
const data = await auth.api.impersonateUser({
    body: {
        userId: user-id,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type impersonateUser = {
    /**
     * The user id which you want to impersonate.
     */
    userId: string = "user-id"

}
```

### Stop Impersonating User

To stop impersonating a user and continue with the admin account, you can use `stopImpersonating`

### Client Side

```ts
const { data, error } = await authClient.admin.stopImpersonating({});
```

### Server Side

```ts
await auth.api.stopImpersonating({

    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type stopImpersonating = {

}
```

### Remove User

Hard deletes a user from the database.

### Client Side

```ts
const { data, error } = await authClient.admin.removeUser({
    userId: user-id,
});
```

### Server Side

```ts
const deletedUser = await auth.api.removeUser({
    body: {
        userId: user-id,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type removeUser = {
    /**
     * The user id which you want to remove.
     */
    userId: string = "user-id"

}
```

## Access Control

The admin plugin offers a highly flexible access control system, allowing you to manage user permissions based on their role. You can define custom permission sets to fit your needs.

### Roles

By default, there are two roles:

`admin`: Users with the admin role have full control over other users.

`user`: Users with the user role have no control over other users.

<Callout>
  A user can have multiple roles. Multiple roles are stored as string separated by comma (",").
</Callout>

### Permissions

By default, there are two resources with up to six permissions.

user:
`create` `list` `set-role` `ban` `impersonate` `delete` `set-password`

session:
`list` `revoke` `delete`

Users with the admin role have full control over all the resources and actions. Users with the user role have no control over any of those actions.

### Custom Permissions

The plugin provides an easy way to define your own set of permissions for each role.

<Steps>
  <Step>
    #### Create Access Control

    You first need to create an access controller by calling the `createAccessControl` function and passing the statement object. The statement object should have the resource name as the key and the array of actions as the value.

    ```ts title="permissions.ts"
    import { createAccessControl } from "better-auth/plugins/access";

    /**
     * make sure to use `as const` so typescript can infer the type correctly
     */
    const statement = { // [!code highlight]
        project: ["create", "share", "update", "delete"], // [!code highlight]
    } as const; // [!code highlight]

    const ac = createAccessControl(statement); // [!code highlight]
    ```
  </Step>

  <Step>
    #### Create Roles

    Once you have created the access controller you can create roles with the permissions you have defined.

    ```ts title="permissions.ts"
    import { createAccessControl } from "better-auth/plugins/access";

    export const statement = {
        project: ["create", "share", "update", "delete"], // <-- Permissions available for created roles
    } as const;

    const ac = createAccessControl(statement);

````
    export const user = ac.newRole({ // [!code highlight]
        project: ["create"], // [!code highlight]
    }); // [!code highlight]

    export const admin = ac.newRole({ // [!code highlight]
        project: ["create", "update"], // [!code highlight]
    }); // [!code highlight]

    export const myCustomRole = ac.newRole({ // [!code highlight]
        project: ["create", "update", "delete"], // [!code highlight]
        user: ["ban"], // [!code highlight]
    }); // [!code highlight]

    When you create custom roles for existing roles, the predefined permissions for those roles will be overridden. To add the existing permissions to the custom role, you need to import `defaultStatements` and merge it with your new statement, plus merge the roles' permissions set with the default roles.

    ```ts title="permissions.ts"
    import { createAccessControl } from "better-auth/plugins/access";
    import { defaultStatements, adminAc } from "better-auth/plugins/admin/access";

    const statement = {
        ...defaultStatements, // [!code highlight]
        project: ["create", "share", "update", "delete"],
    } as const;

    const ac = createAccessControl(statement);

    const admin = ac.newRole({
        project: ["create", "update"],
        ...adminAc.statements, // [!code highlight]
    });
    ```
  </Step>

  <Step>
    #### Pass Roles to the Plugin

    Once you have created the roles you can pass them to the admin plugin both on the client and the server.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { admin as adminPlugin } from "better-auth/plugins"
    import { ac, admin, user } from "@/auth/permissions"

    export const auth = betterAuth({
        plugins: [
            adminPlugin({
                ac,
                roles: {
                    admin,
                    user,
                    myCustomRole
                }
            }),
        ],
    });
    ```

    You also need to pass the access controller and the roles to the client plugin.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    import { adminClient } from "better-auth/client/plugins"
    import { ac, admin, user, myCustomRole } from "@/auth/permissions"

    export const client = createAuthClient({
        plugins: [
            adminClient({
                ac,
                roles: {
                    admin,
                    user,
                    myCustomRole
                }
            })
        ]
    })
    ```
  </Step>
</Steps>

### Access Control Usage

Has Permission:

To check a user's permissions, you can use the `hasPermission` function provided by the client.

### Client Side

```ts
const { data, error } = await authClient.admin.hasPermission({
    userId: user-id, // required
    role: admin, // required
    permission, // required
});
```

### Server Side

```ts
const data = await auth.api.userHasPermission({
    body: {
        userId: user-id, // required
        role: admin, // required
        permission, // required
    }
});
```

### Type Definition

```ts
type userHasPermission = {
    /**
     * The user id which you want to check the permissions for.
     */
    userId?: string = "user-id"
    /**
     * Check role permissions.
     * @serverOnly
     */
    role?: string = "admin"
    /**
     * Optionally check if a single permission is granted. Must use this, or permissions.
     */
    permission?: Record<string, string[]> = { "project": ["create", "update"]
}
```
~~~

Example usage:

```ts title="auth-client.ts"
const canCreateProject = await authClient.admin.hasPermission({
  permissions: {
    project: ["create"],
  },
});

// You can also check multiple resource permissions at the same time
const canCreateProjectAndCreateSale = await authClient.admin.hasPermission({
  permissions: {
    project: ["create"],
    sale: ["create"]
  },
});
```

If you want to check a user's permissions server-side, you can use the `userHasPermission` action provided by the `api` to check the user's permissions.

```ts title="api.ts"
import { auth } from "@/auth";

await auth.api.userHasPermission({
  body: {
    userId: 'id', //the user id
    permissions: {
      project: ["create"], // This must match the structure in your access control
    },
  },
});

// You can also just pass the role directly
await auth.api.userHasPermission({
  body: {
   role: "admin",
    permissions: {
      project: ["create"], // This must match the structure in your access control
    },
  },
});

// You can also check multiple resource permissions at the same time
await auth.api.userHasPermission({
  body: {
   role: "admin",
    permissions: {
      project: ["create"], // This must match the structure in your access control
      sale: ["create"]
    },
  },
});
```

Check Role Permission:

Use the `checkRolePermission` function on the client side to verify whether a given **role** has a specific **permission**. This is helpful after defining roles and their permissions, as it allows you to perform permission checks without needing to contact the server.

Note that this function does **not** check the permissions of the currently logged-in user directly. Instead, it checks what permissions are assigned to a specified role. The function is synchronous, so you don't need to use `await` when calling it.

```ts title="auth-client.ts"
const canCreateProject = authClient.admin.checkRolePermission({
  permissions: {
    user: ["delete"],
  },
  role: "admin",
});

// You can also check multiple resource permissions at the same time
const canDeleteUserAndRevokeSession = authClient.admin.checkRolePermission({
  permissions: {
    user: ["delete"],
    session: ["revoke"]
  },
  role: "admin",
});
```

## Schema

This plugin adds the following fields to the `user` table:

```
<DatabaseTable
  fields={[
  {
    name: "role",
    type: "string",
    description:
      "The user's role. Defaults to `user`. Admins will have the `admin` role.",
    isOptional: true,
  },
  {
    name: "banned",
    type: "boolean",
    description: "Indicates whether the user is banned.",
    isOptional: true,
  },
  {
    name: "banReason",
    type: "string",
    description: "The reason for the user's ban.",
    isOptional: true,
  },
  {
    name: "banExpires",
    type: "date",
    description: "The date when the user's ban will expire.",
    isOptional: true,
  },
]}
/>
```

And adds one field in the `session` table:

```
<DatabaseTable
  fields={[
  {
    name: "impersonatedBy",
    type: "string",
    description: "The ID of the admin that is impersonating this session.",
    isOptional: true,
  },
]}
/>
```

## Options

### Default Role

The default role for a user. Defaults to `user`.

```ts title="auth.ts"
admin({
  defaultRole: "regular",
});
```

### Admin Roles

The roles that are considered admin roles. Defaults to `["admin"]`.

```ts title="auth.ts"
admin({
  adminRoles: ["admin", "superadmin"],
});
```

<Callout type="warning">
  Any role that isn't in the `adminRoles` list, even if they have the permission,
  will not be considered an admin.
</Callout>

### Admin userIds

You can pass an array of userIds that should be considered as admin. Default to `[]`

```ts title="auth.ts"
admin({
    adminUserIds: ["user_id_1", "user_id_2"]
})
```

If a user is in the `adminUserIds` list, they will be able to perform any admin operation.

### impersonationSessionDuration

The duration of the impersonation session in seconds. Defaults to 1 hour.

```ts title="auth.ts"
admin({
  impersonationSessionDuration: 60 * 60 * 24, // 1 day
});
```

### Default Ban Reason

The default ban reason for a user created by the admin. Defaults to `No reason`.

```ts title="auth.ts"
admin({
  defaultBanReason: "Spamming",
});
```

### Default Ban Expires In

The default ban expires in for a user created by the admin in seconds. Defaults to `undefined` (meaning the ban never expires).

```ts title="auth.ts"
admin({
  defaultBanExpiresIn: 60 * 60 * 24, // 1 day
});
```

### bannedUserMessage

The message to show when a banned user tries to sign in. Defaults to "You have been banned from this application. Please contact support if you believe this is an error."

```ts title="auth.ts"
admin({
  bannedUserMessage: "Custom banned user message",
});
```

# plugins: Anonymous
URL: /docs/plugins/anonymous
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/anonymous.mdx

Anonymous plugin for Better Auth.

***

title: Anonymous\
description: Anonymous plugin for Better Auth.

The Anonymous plugin allows users to have an authenticated experience without requiring them to provide an email address, password, OAuth provider, or any other Personally Identifiable Information (PII). Users can later link an authentication method to their account when ready.

## Installation

<Steps>
  <Step>
    ### Add the plugin to your auth config

    To enable anonymous authentication, add the anonymous plugin to your authentication configuration.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { anonymous } from "better-auth/plugins" // [!code highlight]

    export const auth = betterAuth({
        // ... other config options
        plugins: [
            anonymous() // [!code highlight]
        ]
    })
    ```
  </Step>

  <Step>
    ### Migrate the database

    Run the migration or generate the schema to add the necessary fields and tables to the database.

    <Tabs items={["migrate", "generate"]}>
      <Tab value="migrate">
        ```bash
        npx @better-auth/cli migrate
        ```
      </Tab>

      <Tab value="generate">
        ```bash
        npx @better-auth/cli generate
        ```
      </Tab>
    </Tabs>

    See the [Schema](#schema) section to add the fields manually.
  </Step>

  <Step>
    ### Add the client plugin

    Next, include the anonymous client plugin in your authentication client instance.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    import { anonymousClient } from "better-auth/client/plugins"

    export const authClient = createAuthClient({
        plugins: [
            anonymousClient()
        ]
    })
    ```
  </Step>
</Steps>

## Usage

### Sign In

To sign in a user anonymously, use the `signIn.anonymous()` method.

```ts title="example.ts"
const user = await authClient.signIn.anonymous()
```

### Link Account

If a user is already signed in anonymously and tries to `signIn` or `signUp` with another method, their anonymous activities can be linked to the new account.

To do that you first need to provide `onLinkAccount` callback to the plugin.

```ts title="auth.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    plugins: [
        anonymous({
            onLinkAccount: async ({ anonymousUser, newUser }) => {
               // perform actions like moving the cart items from anonymous user to the new user
            }
        })
    ]
```

Then when you call `signIn` or `signUp` with another method, the `onLinkAccount` callback will be called. And the `anonymousUser` will be deleted by default.

```ts title="example.ts"
const user = await authClient.signIn.email({
    email,
})
```

## Options

* `emailDomainName`: The domain name to use when generating an email address for anonymous users. Defaults to the domain name of the current site.

```ts title="auth.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    plugins: [
        anonymous({
            emailDomainName: "example.com"
        })
    ]
})
```

* `onLinkAccount`: A callback function that is called when an anonymous user links their account to a new authentication method. The callback receives an object with the `anonymousUser` and the `newUser`.

* `disableDeleteAnonymousUser`: By default, the anonymous user is deleted when the account is linked to a new authentication method. Set this option to `true` to disable this behavior.

* `generateName`: A callback function that is called to generate a name for the anonymous user. Useful if you want to have random names for anonymous users, or if `name` is unique in your database.

## Schema

The anonymous plugin requires an additional field in the user table:

<DatabaseTable
  fields={[
      { name: "isAnonymous", type: "boolean", description: "Indicates whether the user is anonymous.", isOptional: true },
  ]}
/>

# plugins: API Key
URL: /docs/plugins/api-key
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/api-key.mdx

API Key plugin for Better Auth.

***

title: API Key
description: API Key plugin for Better Auth.

The API Key plugin allows you to create and manage API keys for your application. It provides a way to authenticate and authorize API requests by verifying API keys.

## Features

* Create, manage, and verify API keys
* [Built-in rate limiting](/docs/plugins/api-key#rate-limiting)
* [Custom expiration times, remaining count, and refill systems](/docs/plugins/api-key#remaining-refill-and-expiration)
* [metadata for API keys](/docs/plugins/api-key#metadata)
* Custom prefix
* [Sessions from API keys](/docs/plugins/api-key#sessions-from-api-keys)

## Installation

<Steps>
  <Step>
    ### Add Plugin to the server

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { apiKey } from "better-auth/plugins"

~~~
    export const auth = betterAuth({
        plugins: [ // [!code highlight]
            apiKey() // [!code highlight]
        ] // [!code highlight]
    })
    ~~~
  </Step>

  <Step>
    ### Migrate the database

    Run the migration or generate the schema to add the necessary fields and tables to the database.

    <Tabs items={["migrate", "generate"]}>
      <Tab value="migrate">
        ```bash
        npx @better-auth/cli migrate
        ~~~
      </Tab>

      <Tab value="generate">
        ```bash
        npx @better-auth/cli generate
        ```
      </Tab>
    </Tabs>

    See the [Schema](#schema) section to add the fields manually.
  </Step>

  <Step>
    ### Add the client plugin

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    import { apiKeyClient } from "better-auth/client/plugins"

    export const authClient = createAuthClient({
        plugins: [ // [!code highlight]
            apiKeyClient() // [!code highlight]
        ] // [!code highlight]
    })
    ```
  </Step>
</Steps>

## Usage

You can view the list of API Key plugin options [here](/docs/plugins/api-key#api-key-plugin-options).

### Create an API key

### Client Side

```ts
const { data, error } = await authClient.apiKey.create({
    name: project-api-key, // required
    expiresIn, // required
    userId: user-id, // required
    prefix: project-api-key, // required
    remaining, // required
    metadata, // required
});
```

### Server Side

```ts
const data = await auth.api.createApiKey({
    body: {
        name: project-api-key, // required
        expiresIn, // required
        userId: user-id, // required
        prefix: project-api-key, // required
        remaining, // required
        metadata, // required
    }
});
```

### Type Definition

```ts
type createApiKey = {
    /**
     * Name of the Api Key.
     */
    name?: string = 'project-api-key'
    /**
     * Expiration time of the Api Key in seconds.
     */
    expiresIn?: number = 60 * 60 * 24 * 7
    /**
     * User Id of the user that the Api Key belongs to. server-only.
     * @serverOnly
     */
    userId?: string = "user-id"
    /**
     * Prefix of the Api Key.
     */
    prefix?: string = 'project-api-key'
    /**
     * Remaining number of requests. server-only.
     * @serverOnly
     */
    remaining?: number = 100
    /**
     * Metadata of the Api Key.
     */
    metadata?: any | null = { someKey: 'someValue'
}
```

<Callout>API keys are assigned to a user.</Callout>

#### Result

It'll return the `ApiKey` object which includes the `key` value for you to use.
Otherwise if it throws, it will throw an `APIError`.

***

### Verify an API key

### Client Side
````

```ts
const { data, error } = await authClient.apiKey.verify({
    key: your_api_key_here,
    permissions, // required
});
```

### Server Side

```ts
const data = await auth.api.verifyApiKey({
    body: {
        key: your_api_key_here,
        permissions, // required
    }
});
```

### Type Definition

```ts
type verifyApiKey = {
    /**
     * The key to verify.
     */
    key: string = "your_api_key_here"
    /**
     * The permissions to verify. Optional.
     */
    permissions?: Record<string, string[]>

}
```

#### Result

```ts
type Result = {
  valid: boolean;
  error: { message: string; code: string } | null;
  key: Omit<ApiKey, "key"> | null;
};
```

***

### Get an API key

### Client Side

```ts
const { data, error } = await authClient.apiKey.get({
    id: some-api-key-id,
});
```

### Server Side

```ts
const data = await auth.api.getApiKey({
    query: {
        id: some-api-key-id,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type getApiKey = {
    /**
     * The id of the Api Key.
     */
    id: string = "some-api-key-id"

}
```

#### Result

You'll receive everything about the API key details, except for the `key` value itself.
If it fails, it will throw an `APIError`.

```ts
type Result = Omit<ApiKey, "key">;
```

***

### Update an API key

### Client Side

```ts
const { data, error } = await authClient.apiKey.update({
    keyId: some-api-key-id,
    userId: some-user-id, // required
    name: some-api-key-name, // required
    enabled, // required
    remaining, // required
    refillAmount, // required
    refillInterval, // required
    metadata, // required
});
```

### Server Side

```ts
const data = await auth.api.updateApiKey({
    body: {
        keyId: some-api-key-id,
        userId: some-user-id, // required
        name: some-api-key-name, // required
        enabled, // required
        remaining, // required
        refillAmount, // required
        refillInterval, // required
        metadata, // required
    }
});
```

### Type Definition

```ts
type updateApiKey = {
    /**
     * The id of the Api Key to update.
     */
    keyId: string = "some-api-key-id"
    /**
     * The id of the user which the api key belongs to. server-only.
     * @serverOnly
     */
    userId?: string = "some-user-id"
    /**
     * The name of the key.
     */
    name?: string = "some-api-key-name"
    /**
     * Whether the Api Key is enabled or not. server-only.
     * @serverOnly
     */
    enabled?: boolean = true
    /**
     * The number of remaining requests. server-only.
     * @serverOnly
     */
    remaining?: number = 100
    /**
     * The refill amount. server-only.
     * @serverOnly
     */
    refillAmount?: number = 100
    /**
     * The refill interval in milliseconds. server-only.
     * @serverOnly
     */
    refillInterval?: number = 1000
    /**
     * The metadata of the Api Key. server-only.
     * @serverOnly
     */
    metadata?: any | null = { "key": "value"
}
```

#### Result

If fails, throws `APIError`.
Otherwise, you'll receive the API Key details, except for the `key` value itself.

***

### Delete an API Key

### Client Side

```ts
const { data, error } = await authClient.apiKey.delete({
    keyId: some-api-key-id,
});
```

### Server Side

```ts
const data = await auth.api.deleteApiKey({
    body: {
        keyId: some-api-key-id,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type deleteApiKey = {
    /**
     * The id of the Api Key to delete.
     */
    keyId: string = "some-api-key-id"

}
```

#### Result

If fails, throws `APIError`.
Otherwise, you'll receive:

```ts
type Result = {
  success: boolean;
};
```

***

### List API keys

### Client Side

```ts
const { data, error } = await authClient.apiKey.list({});
```

### Server Side

```ts
const data = await auth.api.listApiKeys({

    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type listApiKeys = {

}
```

#### Result

If fails, throws `APIError`.
Otherwise, you'll receive:

```ts
type Result = ApiKey[];
```

***

### Delete all expired API keys

This function will delete all API keys that have an expired expiration date.

### Client Side

```ts
const { data, error } = await authClient.apiKey.deleteAllExpiredApiKeys({});
```

### Server Side

```ts
const data = await auth.api.deleteAllExpiredApiKeys({});
```

### Type Definition

```ts
type deleteAllExpiredApiKeys = {

}
```

<Callout>
  We automatically delete expired API keys every time any apiKey plugin
  endpoints were called, however they are rate-limited to a 10 second cool down
  each call to prevent multiple calls to the database.
</Callout>

***

## Sessions from API keys

Any time an endpoint in Better Auth is called that has a valid API key in the headers, you can automatically create a mock session to represent the user by enabling `sessionForAPIKeys` option.

<Callout type="warn">
  This is generally not recommended, as it can lead to security issues if not used carefully. A leaked api key can be used to impersonate a user.
</Callout>

```ts
export const auth = betterAuth({
  plugins: [
    apiKey({
      enableSessionForAPIKeys: true,
    }),
  ],
});
```

<Tabs items={['Server']}>
  <Tab value="Server">
    ```ts
    const session = await auth.api.getSession({
          headers: new Headers({
                'x-api-key': apiKey,
          }),
    });
    ```
  </Tab>
</Tabs>

The default header key is `x-api-key`, but this can be changed by setting the `apiKeyHeaders` option in the plugin options.

```ts
export const auth = betterAuth({
  plugins: [
    apiKey({
      apiKeyHeaders: ["x-api-key", "xyz-api-key"], // or you can pass just a string, eg: "x-api-key"
    }),
  ],
});
```

Or optionally, you can pass an `apiKeyGetter` function to the plugin options, which will be called with the `GenericEndpointContext`, and from there, you should return the API key, or `null` if the request is invalid.

```ts
export const auth = betterAuth({
  plugins: [
    apiKey({
      apiKeyGetter: (ctx) => {
        const has = ctx.request.headers.has("x-api-key");
        if (!has) return null;
        return ctx.request.headers.get("x-api-key");
      },
    }),
  ],
});
```

## Rate Limiting

Every API key can have its own rate limit settings, however, the built-in rate-limiting only applies to the verification process for a given API key.
For every other endpoint/method, you should utilize Better Auth's [built-in rate-limiting](/docs/concepts/rate-limit).

You can refer to the rate-limit default configurations below in the API Key plugin options.

An example default value:

```ts
export const auth = betterAuth({
  plugins: [
    apiKey({
      rateLimit: {
        enabled: true,
        timeWindow: 1000 * 60 * 60 * 24, // 1 day
        maxRequests: 10, // 10 requests per day
      },
    }),
  ],
});
```

For each API key, you can customize the rate-limit options on create.

<Callout>
  You can only customize the rate-limit options on the server auth instance.
</Callout>

```ts
const apiKey = await auth.api.createApiKey({
  body: {
    rateLimitEnabled: true,
    rateLimitTimeWindow: 1000 * 60 * 60 * 24, // 1 day
    rateLimitMax: 10, // 10 requests per day
  },
  headers: user_headers,
});
```

### How does it work?

For each request, a counter (internally called `requestCount`) is incremented.
If the `rateLimitMax` is reached, the request will be rejected until the `timeWindow` has passed, at which point the `timeWindow` will be reset.

## Remaining, refill, and expiration

The remaining count is the number of requests left before the API key is disabled.
The refill interval is the interval in milliseconds where the `remaining` count is refilled by day.
The expiration time is the expiration date of the API key.

### How does it work?

#### Remaining:

Whenever an API key is used, the `remaining` count is updated.
If the `remaining` count is `null`, then there is no cap to key usage.
Otherwise, the `remaining` count is decremented by 1.
If the `remaining` count is 0, then the API key is disabled & removed.

#### refillInterval & refillAmount:

Whenever an API key is created, the `refillInterval` and `refillAmount` are set to `null`.
This means that the API key will not be refilled automatically.
However, if `refillInterval` & `refillAmount` are set, then the API key will be refilled accordingly.

#### Expiration:

Whenever an API key is created, the `expiresAt` is set to `null`.
This means that the API key will never expire.
However, if the `expiresIn` is set, then the API key will expire after the `expiresIn` time.

## Custom Key generation & verification

You can customize the key generation and verification process straight from the plugin options.

Here's an example:

```ts
export const auth = betterAuth({
  plugins: [
    apiKey({
      customKeyGenerator: (options: {
        length: number;
        prefix: string | undefined;
      }) => {
        const apiKey = mySuperSecretApiKeyGenerator(
          options.length,
          options.prefix
        );
        return apiKey;
      },
      customAPIKeyValidator: async ({ ctx, key }) => {
        const res = await keyService.verify(key)
        return res.valid
      },
    }),
  ],
});
```

<Callout>
  If you're **not** using the `length` property provided by `customKeyGenerator`, you **must** set the `defaultKeyLength` property to how long generated keys will be.

  ```ts
  export const auth = betterAuth({
    plugins: [
      apiKey({
        customKeyGenerator: () => {
          return crypto.randomUUID();
        },
        defaultKeyLength: 36, // Or whatever the length is
      }),
    ],
  });
  ```
</Callout>

If an API key is validated from your `customAPIKeyValidator`, we still must match that against the database's key.
However, by providing this custom function, you can improve the performance of the API key verification process,
as all failed keys can be invalidated without having to query your database.

## Metadata

We allow you to store metadata alongside your API keys. This is useful for storing information about the key, such as a subscription plan for example.

To store metadata, make sure you haven't disabled the metadata feature in the plugin options.

```ts
export const auth = betterAuth({
  plugins: [
    apiKey({
      enableMetadata: true,
    }),
  ],
});
```

Then, you can store metadata in the `metadata` field of the API key object.

```ts
const apiKey = await auth.api.createApiKey({
  body: {
    metadata: {
      plan: "premium",
    },
  },
});
```

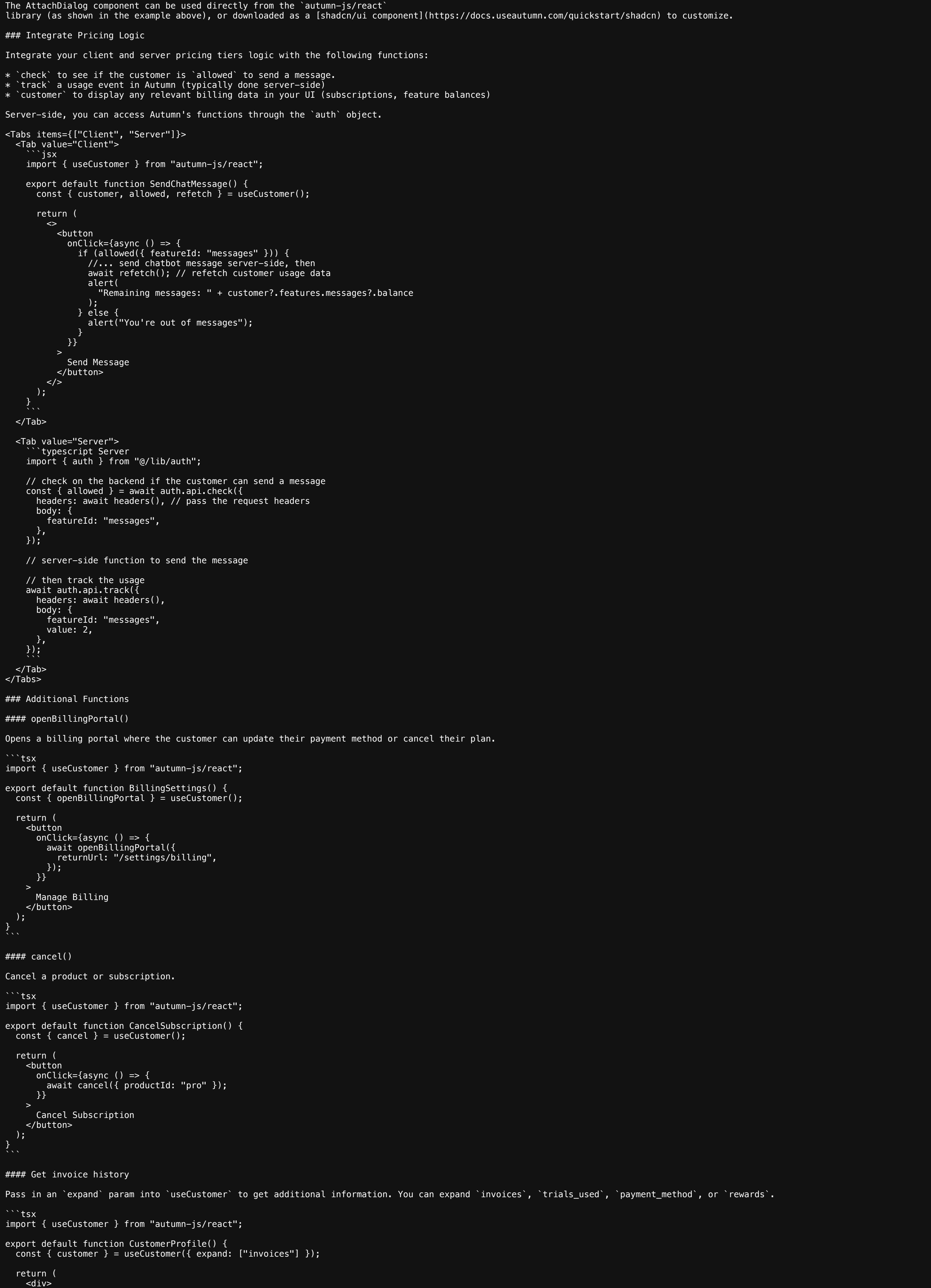
You can then retrieve the metadata from the API key object.

~~~ts
const apiKey = await auth.api.getApiKey({
  body: {
    keyId: "your_api_key_id_here",
  },
});

````mdx
console.log(apiKey.metadata.plan); // "premium"
~~~

## API Key plugin options

`apiKeyHeaders` `string | string[];`

The header name to check for API key. Default is `x-api-key`.

`customAPIKeyGetter` `(ctx: GenericEndpointContext) => string | null`

A custom function to get the API key from the context.

`customAPIKeyValidator` `(options: { ctx: GenericEndpointContext; key: string; }) => boolean | Promise<boolean>`

A custom function to validate the API key.

`customKeyGenerator` `(options: { length: number; prefix: string | undefined; }) => string | Promise<string>`

A custom function to generate the API key.

`startingCharactersConfig` `{ shouldStore?: boolean; charactersLength?: number; }`

Customize the starting characters configuration.

<Accordions>
  <Accordion title="startingCharactersConfig Options">
    `shouldStore` `boolean`

    Wether to store the starting characters in the database.
    If false, we will set `start` to `null`.
    Default is `true`.

    `charactersLength` `number`

    The length of the starting characters to store in the database.
    This includes the prefix length.
    Default is `6`.
  </Accordion>
</Accordions>

`defaultKeyLength` `number`

The length of the API key. Longer is better. Default is 64. (Doesn't include the prefix length)

`defaultPrefix` `string`

The prefix of the API key.

Note: We recommend you append an underscore to the prefix to make the prefix more identifiable. (eg `hello_`)

`maximumPrefixLength` `number`

The maximum length of the prefix.

`minimumPrefixLength` `number`

The minimum length of the prefix.

`requireName` `boolean`

Whether to require a name for the API key. Default is `false`.

`maximumNameLength` `number`

The maximum length of the name.

`minimumNameLength` `number`

The minimum length of the name.

`enableMetadata` `boolean`

Whether to enable metadata for an API key.

`keyExpiration` `{ defaultExpiresIn?: number | null; disableCustomExpiresTime?: boolean; minExpiresIn?: number; maxExpiresIn?: number; }`

Customize the key expiration.

<Accordions>
  <Accordion title="keyExpiration options">
    `defaultExpiresIn` `number | null`

    The default expires time in milliseconds.
    If `null`, then there will be no expiration time.
    Default is `null`.

    `disableCustomExpiresTime` `boolean`

    Wether to disable the expires time passed from the client.
    If `true`, the expires time will be based on the default values.
    Default is `false`.

    `minExpiresIn` `number`

    The minimum expiresIn value allowed to be set from the client. in days.
    Default is `1`.

    `maxExpiresIn` `number`

    The maximum expiresIn value allowed to be set from the client. in days.
    Default is `365`.
  </Accordion>
</Accordions>

`rateLimit` `{ enabled?: boolean; timeWindow?: number; maxRequests?: number; }`

Customize the rate-limiting.

<Accordions>
  <Accordion title="rateLimit options">
    `enabled` `boolean`

    Whether to enable rate limiting. (Default true)

    `timeWindow` `number`

    The duration in milliseconds where each request is counted.
    Once the `maxRequests` is reached, the request will be rejected until the `timeWindow` has passed, at which point the `timeWindow` will be reset.

    `maxRequests` `number`

    Maximum amount of requests allowed within a window.
    Once the `maxRequests` is reached, the request will be rejected until the `timeWindow` has passed, at which point the `timeWindow` will be reset.
  </Accordion>
</Accordions>

`schema` `InferOptionSchema<ReturnType<typeof apiKeySchema>>`

Custom schema for the API key plugin.
~~~

`enableSessionForAPIKeys` `boolean`

An API Key can represent a valid session, so we can mock a session for the user if we find a valid API key in the request headers. Default is `false`.

`permissions` `{ defaultPermissions?: Statements | ((userId: string, ctx: GenericEndpointContext) => Statements | Promise<Statements>) }`

Permissions for the API key.

Read more about permissions [here](/docs/plugins/api-key#permissions).

<Accordions>
  <Accordion title="permissions Options">
    `defaultPermissions` `Statements | ((userId: string, ctx: GenericEndpointContext) => Statements | Promise<Statements>)`

    The default permissions for the API key.
  </Accordion>
</Accordions>

`disableKeyHashing` `boolean`

Disable hashing of the API key.

âš ï¸ Security Warning: It's strongly recommended to not disable hashing.
Storing API keys in plaintext makes them vulnerable to database breaches, potentially exposing all your users' API keys.

***

## Permissions

API keys can have permissions associated with them, allowing you to control access at a granular level. Permissions are structured as a record of resource types to arrays of allowed actions.

### Setting Default Permissions

You can configure default permissions that will be applied to all newly created API keys:

```ts
export const auth = betterAuth({
  plugins: [
    apiKey({
      permissions: {
        defaultPermissions: {
          files: ["read"],
          users: ["read"],
        },
      },
    }),
  ],
});
```

You can also provide a function that returns permissions dynamically:

```ts
export const auth = betterAuth({
  plugins: [
    apiKey({
      permissions: {
        defaultPermissions: async (userId, ctx) => {
          // Fetch user role or other data to determine permissions
          return {
            files: ["read"],
            users: ["read"],
          };
        },
      },
    }),
  ],
});
```

### Creating API Keys with Permissions

When creating an API key, you can specify custom permissions:

```ts
const apiKey = await auth.api.createApiKey({
  body: {
    name: "My API Key",
    permissions: {
      files: ["read", "write"],
      users: ["read"],
    },
    userId: "userId",
  },
});
```

### Verifying API Keys with Required Permissions

When verifying an API key, you can check if it has the required permissions:

```ts
const result = await auth.api.verifyApiKey({
  body: {
    key: "your_api_key_here",
    permissions: {
      files: ["read"],
    },
  },
});

if (result.valid) {
  // API key is valid and has the required permissions
} else {
  // API key is invalid or doesn't have the required permissions
}
```

### Updating API Key Permissions

You can update the permissions of an existing API key:

```ts
const apiKey = await auth.api.updateApiKey({
  body: {
    keyId: existingApiKeyId,
    permissions: {
      files: ["read", "write", "delete"],
      users: ["read", "write"],
    },
  },
  headers: user_headers,
});
```

### Permissions Structure

Permissions follow a resource-based structure:

~~~
```ts
type Permissions = {
  [resourceType: string]: string[];
};

// Example:
const permissions = {
  files: ["read", "write", "delete"],
  users: ["read"],
  projects: ["read", "write"],
};
```

When verifying an API key, all required permissions must be present in the API key's permissions for validation to succeed.

Schema

Table: `apiKey`

<DatabaseTable
  fields={[
  {
    name: "id",
    type: "string",
    description: "The ID of the API key.",
    isUnique: true,
    isPrimaryKey: true,
  },
  {
    name: "name",
    type: "string",
    description: "The name of the API key.",
    isOptional: true,
  },
  {
    name: "start",
    type: "string",
    description:
      "The starting characters of the API key. Useful for showing the first few characters of the API key in the UI for the users to easily identify.",
    isOptional: true,
  },
  {
    name: "prefix",
    type: "string",
    description: "The API Key prefix. Stored as plain text.",
    isOptional: true,
  },
  {
    name: "key",
    type: "string",
    description: "The hashed API key itself.",
  },
  {
    name: "userId",
    type: "string",
    description: "The ID of the user associated with the API key.",
    isForeignKey: true,
  },
  {
    name: "refillInterval",
    type: "number",
    description: "The interval to refill the key in milliseconds.",
    isOptional: true,
  },
  {
    name: "refillAmount",
    type: "number",
    description: "The amount to refill the remaining count of the key.",
    isOptional: true,
  },
  {
    name: "lastRefillAt",
    type: "Date",
    description: "The date and time when the key was last refilled.",
    isOptional: true,
  },
  {
    name: "enabled",
    type: "boolean",
    description: "Whether the API key is enabled.",
  },
  {
    name: "rateLimitEnabled",
    type: "boolean",
    description: "Whether the API key has rate limiting enabled.",
  },
  {
    name: "rateLimitTimeWindow",
    type: "number",
    description: "The time window in milliseconds for the rate limit.",
    isOptional: true,
  },
  {
    name: "rateLimitMax",
    type: "number",
    description:
      "The maximum number of requests allowed within the `rateLimitTimeWindow`.",
    isOptional: true,
  },
  {
    name: "requestCount",
    type: "number",
    description:
      "The number of requests made within the rate limit time window.",
  },
  {
    name: "remaining",
    type: "number",
    description: "The number of requests remaining.",
    isOptional: true,
  },
  {
    name: "lastRequest",
    type: "Date",
    description: "The date and time of the last request made to the key.",
    isOptional: true,
  },
  {
    name: "expiresAt",
    type: "Date",
    description: "The date and time when the key will expire.",
    isOptional: true,
  },
  {
    name: "createdAt",
    type: "Date",
    description: "The date and time the API key was created.",
  },
  {
````

````
    name: "updatedAt",
    type: "Date",
    description: "The date and time the API key was updated.",
  },
  {
    name: "permissions",
    type: "string",
    description: "The permissions of the key.",
    isOptional: true,
  },
  {
    name: "metadata",
    type: "Object",
    isOptional: true,
    description: "Any additional metadata you want to store with the key.",
  },
]}
/>

# plugins: Autumn Billing
URL: /docs/plugins/autumn
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/autumn.mdx

Better Auth Plugin for Autumn Billing

***

title: Autumn Billing
description: Better Auth Plugin for Autumn Billing
--------------------------------------------------

import { HomeIcon } from "lucide-react";
import { Accordion, Accordions } from "fumadocs-ui/components/accordion";

[Autumn](https://useautumn.com) is open source infrastructure to run SaaS pricing plans. It sits between your app and Stripe, and acts as the database for your customers' subscription status, usage metering and feature permissions.

<Card href="https://discord.gg/STqxY92zuS" title="Get help on Autumn's Discord">
  We're online to help you with any questions you have.
</Card>

## Features

* One function for all checkout, subscription and payment flows
* No webhooks required: query Autumn for the data you need
* Manages your application's free and paid plans
* Usage tracking for usage billing and periodic limits
* Custom plans and pricing changes through Autumn's dashboard

<Steps>
  <Step>
    ### Setup Autumn Account

    First, create your pricing plans in Autumn's [dashboard](https://app.useautumn.com), where you define what each plan and product gets access to and how it should be billed. In this example, we're handling the free and pro plans for an AI chatbot, which comes with a number of `messages` per month.
  </Step>

  <Step>
    ### Install Autumn SDK

    <CodeBlockTabs defaultValue="npm">
      <CodeBlockTabsList>
        <CodeBlockTabsTrigger value="npm">
          npm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="pnpm">
          pnpm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="yarn">
          yarn
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="bun">
          bun
        </CodeBlockTabsTrigger>
      </CodeBlockTabsList>

      <CodeBlockTab value="npm">
        ```bash
        npm install autumn-js
        ```
      </CodeBlockTab>

      <CodeBlockTab value="pnpm">
        ```bash
        pnpm add autumn-js
        ```
      </CodeBlockTab>

      <CodeBlockTab value="yarn">
        ```bash
        yarn add autumn-js
        ```
      </CodeBlockTab>

      <CodeBlockTab value="bun">
        ```bash
        bun add autumn-js
        ```
      </CodeBlockTab>
    </CodeBlockTabs>

    <Callout>
      If you're using a separate client and server setup, make sure to install the plugin in both parts of your project.
    </Callout>
  </Step>

  <Step>
    ### Add `AUTUMN_SECRET_KEY` to your environment variables

    You can find it in Autumn's dashboard under "[Developer](https://app.useautumn.com/sandbox/onboarding)".

    ```bash title=".env"
    AUTUMN_SECRET_KEY=am_sk_xxxxxxxxxx
    ```
  </Step>

  <Step>
    ### Add the Autumn plugin to your `auth` config

    <Tabs items={["User", "Organization", "User & Organization", "Custom"]}>
      <Tab value="User">
        ```ts title="auth.ts"
        import { autumn } from "autumn-js/better-auth";
````

~~~mdx
        export const auth = betterAuth({
          // ...
          plugins: [autumn()],
        });
        ```
      </Tab>

      <Tab value="Organization">
        ```ts title="auth.ts"
        import { autumn } from "autumn-js/better-auth";
        import { organization } from "better-auth/plugins";

        export const auth = betterAuth({
          // ...
          plugins: [organization(), autumn({ customerScope: "organization" })],
        });
        ```
      </Tab>

      <Tab value="User & Organization">
        ```ts title="auth.ts"
        import { autumn } from "autumn-js/better-auth";
        import { organization } from "better-auth/plugins";

        export const auth = betterAuth({
          // ...
          plugins: [
            organization(),
            autumn({ customerScope: "user_and_organization" })
          ],
        });
        ```
      </Tab>

      <Tab value="Custom">
        ```ts title="auth.ts"
        import { autumn } from "autumn-js/better-auth";
        import { organization } from "better-auth/plugins";

        export const auth = betterAuth({
          // ...
          plugins: [
            organization(),
            autumn({
              identify: async ({ session, organization }) => {
                return {
                  customerId: "your_customer_id",
                  customerData: {
                    name: "Customer Name",
                    email: "customer@gmail.com",
                  },
                };
              },
            }),
          ],
        });
        ```
      </Tab>
    </Tabs>

    <Callout>
      Autumn will auto-create your customers when they sign up, and assign them any
      default plans you created (eg your Free plan). You can choose who becomes a customer: individual users, organizations, both, or something custom like workspaces.
    </Callout>
  </Step>

  <Step>
    ### Add `<AutumnProvider />`

    Client side, wrap your application with the AutumnProvider component, and pass in the `baseUrl` that you define within better-auth's `authClient`.

    ```tsx title="app/layout.tsx"
    import { AutumnProvider } from "autumn-js/react";

    export default function RootLayout({
      children,
    }: {
      children: React.ReactNode;
    }) {
      return (
        <html>
          <body>
            {/* or meta.env.BETTER_AUTH_URL for vite */}
            <AutumnProvider betterAuthUrl={process.env.NEXT_PUBLIC_BETTER_AUTH_URL}>
              {children}
            </AutumnProvider>
          </body>
        </html>
      );
    }
    ```
  </Step>
</Steps>

## Usage

### Handle payments

Call `attach` to redirect the customer to a Stripe checkout page when they want to purchase the Pro plan.

If their payment method is already on file, `AttachDialog` will open instead to let the customer confirm their new subscription or purchase, and handle the payment.

<Callout type="warn">
  {" "}

  Make sure you've pasted in your [Stripe test secret
  key](https://dashboard.stripe.com/test/apikeys) in the [Autumn
  dashboard](https://app.useautumn.com/integrations/stripe).
</Callout>

```tsx
import { useCustomer, AttachDialog } from "autumn-js/react";

export default function PurchaseButton() {
  const { attach } = useCustomer();

  return (
    <button
      onClick={async () => {
        await attach({
          productId: "pro",
          dialog: AttachDialog,
        });
      }}
    >
      Upgrade to Pro
    </button>
  );
}
```
~~~



The AttachDialog component can be used directly from the `autumn-js/react`
library (as shown in the example above), or downloaded as a [shadcn/ui component](https://docs.useautumn.com/quickstart/shadcn) to customize.

### Integrate Pricing Logic

Integrate your client and server pricing tiers logic with the following functions:

* `check` to see if the customer is `allowed` to send a message.
* `track` a usage event in Autumn (typically done server-side)
* `customer` to display any relevant billing data in your UI (subscriptions, feature balances)

Server-side, you can access Autumn's functions through the `auth` object.

<Tabs items={["Client", "Server"]}>
  <Tab value="Client">
    ```jsx
    import { useCustomer } from "autumn-js/react";

    export default function SendChatMessage() {
      const { customer, allowed, refetch } = useCustomer();

      return (
        <>
          <button
            onClick={async () => {
              if (allowed({ featureId: "messages" })) {
                //... send chatbot message server-side, then
                await refetch(); // refetch customer usage data
                alert(
                  "Remaining messages: " + customer?.features.messages?.balance
                );
              } else {
                alert("You're out of messages");
              }
            }}
          >
            Send Message
          </button>
        </>
      );
    }
    ```
  </Tab>

  <Tab value="Server">
    ```typescript Server
    import { auth } from "@/lib/auth";

    // check on the backend if the customer can send a message
    const { allowed } = await auth.api.check({
      headers: await headers(), // pass the request headers
      body: {
        featureId: "messages",
      },
    });

    // server-side function to send the message

    // then track the usage
    await auth.api.track({
      headers: await headers(),
      body: {
        featureId: "messages",
        value: 2,
      },
    });
    ```
  </Tab>
</Tabs>

### Additional Functions

#### openBillingPortal()

Opens a billing portal where the customer can update their payment method or cancel their plan.

```tsx
import { useCustomer } from "autumn-js/react";

export default function BillingSettings() {
  const { openBillingPortal } = useCustomer();

  return (
    <button
      onClick={async () => {
        await openBillingPortal({
          returnUrl: "/settings/billing",
        });
      }}
    >
      Manage Billing
    </button>
  );
}
```

#### cancel()

Cancel a product or subscription.

```tsx
import { useCustomer } from "autumn-js/react";

export default function CancelSubscription() {
  const { cancel } = useCustomer();

  return (
    <button
      onClick={async () => {
        await cancel({ productId: "pro" });
      }}
    >
      Cancel Subscription
    </button>
  );
}
```

#### Get invoice history

Pass in an `expand` param into `useCustomer` to get additional information. You can expand `invoices`, `trials_used`, `payment_method`, or `rewards`.

```tsx
import { useCustomer } from "autumn-js/react";

export default function CustomerProfile() {
  const { customer } = useCustomer({ expand: ["invoices"] });

  return (
    <div>
```

```
      <h2>Customer Profile</h2>
      <p>Name: {customer?.name}</p>
      <p>Email: {customer?.email}</p>
      <p>Balance: {customer?.features.chat_messages?.balance}</p>
    </div>
  );
}
```

# plugins: Bearer Token Authentication

URL: /docs/plugins/bearer
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/bearer.mdx

Authenticate API requests using Bearer tokens instead of browser cookies

***

title: Bearer Token Authentication
description: Authenticate API requests using Bearer tokens instead of browser cookies
----

The Bearer plugin enables authentication using Bearer tokens as an alternative to browser cookies. It intercepts requests, adding the Bearer token to the Authorization header before forwarding them to your API.

<Callout type="warn">
  Use this cautiously; it is intended only for APIs that don't support cookies or require Bearer tokens for authentication. Improper implementation could easily lead to security vulnerabilities.
</Callout>

## Installing the Bearer Plugin

Add the Bearer plugin to your authentication setup:

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { bearer } from "better-auth/plugins";

export const auth = betterAuth({
    plugins: [bearer()]
});
```

## How to Use Bearer Tokens

### 1. Obtain the Bearer Token

After a successful sign-in, you'll receive a session token in the response headers. Store this token securely (e.g., in `localStorage`):

```ts title="auth-client.ts"
const { data } = await authClient.signIn.email({
    email: "user@example.com",
    password: "securepassword"
}, {
  onSuccess: (ctx)=>{
    const authToken = ctx.response.headers.get("set-auth-token") // get the token from the response headers
    // Store the token securely (e.g., in localStorage)
    localStorage.setItem("bearer_token", authToken);
  }
});
```

You can also set this up globally in your auth client:

```ts title="auth-client.ts"
export const authClient = createAuthClient({
    fetchOptions: {
        onSuccess: (ctx) => {
            const authToken = ctx.response.headers.get("set-auth-token") // get the token from the response headers
            // Store the token securely (e.g., in localStorage)
            if(authToken){
              localStorage.setItem("bearer_token", authToken);
            }
        }
    }
});
```

You may want to clear the token based on the response status code or other conditions:

### 2. Configure the Auth Client

Set up your auth client to include the Bearer token in all requests:

```ts title="auth-client.ts"
export const authClient = createAuthClient({
    fetchOptions: {
        auth: {
           type:"Bearer",
           token: () => localStorage.getItem("bearer_token") || "" // get the token from localStorage
        }
    }
});
```

### 3. Make Authenticated Requests

Now you can make authenticated API calls:

```ts title="auth-client.ts"
// This request is automatically authenticated
const { data } = await authClient.listSessions();
```

### 4. Per-Request Token (Optional)

You can also provide the token for individual requests:

```ts title="auth-client.ts"
const { data } = await authClient.listSessions({
    fetchOptions: {
        headers: {
            Authorization: `Bearer ${token}`
        }
    }
});
```

### 5. Using Bearer Tokens Outside the Auth Client

The Bearer token can be used to authenticate any request to your API, even when not using the auth client:

```ts title="api-call.ts"
const token = localStorage.getItem("bearer_token");

const response = await fetch("https://api.example.com/data", {
  headers: {
    Authorization: `Bearer ${token}`
  }
});
```

const data = await response.json();

And in the server, you can use the `auth.api.getSession` function to authenticate requests:

```ts title="server.ts"
import { auth } from "@/auth";

export async function handler(req, res) {
  const session = await auth.api.getSession({
    headers: req.headers
  });

  if (!session) {
    return res.status(401).json({ error: "Unauthorized" });
  }

  // Process authenticated request
  // ...
}
```

## Options

requireSignature (boolean): Require the token to be signed. Default: `false`.

# plugins: Captcha
URL: /docs/plugins/captcha
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/captcha.mdx

Captcha plugin

***

title: Captcha
description: Captcha plugin

The **Captcha Plugin** integrates bot protection into your Better Auth system by adding captcha verification for key endpoints. This plugin ensures that only human users can perform actions like signing up, signing in, or resetting passwords. The following providers are currently supported:

* [Google reCAPTCHA](https://developers.google.com/recaptcha)
* [Cloudflare Turnstile](https://www.cloudflare.com/application-services/products/turnstile/)
* [hCaptcha](https://www.hcaptcha.com/)
* [CaptchaFox](https://captchafox.com/)

<Callout type="info">
  This plugin works out of the box with <Link href="/docs/authentication/email-password">Email & Password</Link> authentication. To use it with other authentication methods, you will need to configure the <Link href="/docs/plugins/captcha#plugin-options">endpoints</Link> array in the plugin options.
</Callout>

## Installation

<Steps>
  <Step>
    ### Add the plugin to your **auth** config

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth";
    import { captcha } from "better-auth/plugins";

    export const auth = betterAuth({
        plugins: [ // [!code highlight]
            captcha({ // [!code highlight]
                provider: "cloudflare-turnstile", // or google-recaptcha, hcaptcha, captchafox // [!code highlight]
                secretKey: process.env.TURNSTILE_SECRET_KEY!, // [!code highlight]
            }), // [!code highlight]
        ], // [!code highlight]
    });
    ```
  </Step>

  <Step>
    ### Add the captcha token to your request headers

    Add the captcha token to your request headers for all protected endpoints. This example shows how to include it in a `signIn` request:

    ```ts
    await authClient.signIn.email({
        email: "user@example.com",
        password: "secure-password",
        fetchOptions: { // [!code highlight]
            headers: { // [!code highlight]
                "x-captcha-response": turnstileToken, // [!code highlight]
                "x-captcha-user-remote-ip": userIp, // optional: forwards the user's IP address to the captcha service // [!code highlight]
            }, // [!code highlight]
        }, // [!code highlight]
    });
    ```

    * To implement Cloudflare Turnstile on the client side, follow the official [Cloudflare Turnstile documentation](https://developers.cloudflare.com/turnstile/) or use a library like [react-turnstile](https://www.npmjs.com/package/@marsidev/react-turnstile).
    * To implement Google reCAPTCHA on the client side, follow the official [Google reCAPTCHA documentation](https://developers.google.com/recaptcha/intro) or use libraries like [react-google-recaptcha](https://www.npmjs.com/package/react-google-recaptcha) (v2) and [react-google-recaptcha-v3](https://www.npmjs.com/package/react-google-recaptcha-v3) (v3).
    * To implement hCaptcha on the client side, follow the official [hCaptcha documentation](https://docs.hcaptcha.com/#add-the-hcaptcha-widget-to-your-webpage) or use libraries like [@hcaptcha/react-hcaptcha](https://www.npmjs.com/package/@hcaptcha/react-hcaptcha)
    * To implement CaptchaFox on the client side, follow the official [CaptchaFox documentation](https://docs.captchafox.com/getting-started) or use libraries like [@captchafox/react](https://www.npmjs.com/package/@captchafox/react)
  </Step>
</Steps>

## How it works

<Steps>
  <Step>
    The plugin acts as a middleware: it intercepts all `POST` requests to configured endpoints (see `endpoints`
    in the [Plugin Options](#plugin-options) section).
  </Step>

  <Step>
    it validates the captcha token on the server, by calling the captcha provider's `/siteverify`.
  </Step>

  <Step>
    * if the token is missing, gets rejected by the captcha provider, or if the `/siteverify` endpoint is
      unavailable, the plugin returns an error and interrupts the request.
    * if the token is accepted by the captcha provider, the middleware returns `undefined`, meaning the request is allowed to proceed.
  </Step>
</Steps>

## Plugin Options

* **`provider` (required)**: your captcha provider.
* **`secretKey` (required)**: your provider's secret key used for the server-side validation.
* `endpoints` (optional): overrides the default array of paths where captcha validation is enforced. Default is: `["/sign-up/email", "/sign-in/email", "/forget-password",]`.
* `minScore` (optional - only *Google ReCAPTCHA v3*): minimum score threshold. Default is `0.5`.
* `siteKey` (optional - only *hCaptcha* and *CaptchaFox*): prevents tokens issued on one sitekey from being redeemed elsewhere.
* `siteVerifyURLOverride` (optional): overrides endpoint URL for the captcha verification request.

# plugins: Community Plugins
URL: /docs/plugins/community-plugins
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/community-plugins.mdx

A list of recommended community plugins.

***

title: Community Plugins
description: A list of recommended community plugins.

This page showcases a list of recommended community made plugins.

We encourage you to create custom plugins and maybe get added to the list!

To create your own custom plugin, get started by reading our [plugins documentation](/docs/concepts/plugins). And if you want to share your plugin with the community, please open a pull request to add it to this list.

<div className="w-[200px]">Plugin</div>	Description	<div className="w-[150px]">Author</div>
[@dymo-api/better-auth](https://github.com/TPEOficial/dymo-api-better-auth)	Sign Up Protection and validation of disposable emails (the world's largest database with nearly 14 million entries).	 [TPEOficial](https://github.com/TPEOficial)
[better-auth-harmony](https://github.com/gekorm/better-auth-harmony/)	Email & phone normalization and additional validation, blocking over 55,000 temporary email domains.	 [GeKorm](https://github.com/GeKorm)
[validation-better-auth](https://github.com/Daanish2003/validation-better-auth)	Validate API request using any validation library (e.g., Zod, Yup)	 [Daanish2003](https://github.com/Daanish2003)
[better-auth-localization](https://github.com/marcellosso/better-auth-localization)	Localize and customize better-auth messages with easy translation and message override support.	 [marcellosso](https://github.com/marcellosso)
[better-auth-attio-plugin](https://github.com/tobimori/better-auth-attio-plugin)	Sync your products Better Auth users & workspaces with Attio	 [tobimori](https://github.com/tobimori)
[better-auth-cloudflare](https://github.com/zpg6/better-auth-cloudflare)	Seamlessly integrate with Cloudflare Workers, D1, Hyperdrive, KV, R2, and geolocation services. Includes CLI for project generation, automated resource provisioning on Cloudflare, and database migrations. Supports Next.js, Hono, and more!	 [zpg6](https://github.com/zpg6)
[expo-better-auth-passkey](https://github.com/kevcube/expo-better-auth-passkey)	Better-auth client plugin for using passkeys on mobile platforms in expo apps. Supports iOS, macOS, Android (and web!) by wrapping the existing better-auth passkey client plugin.	 [kevcube](https://github.com/kevcube)
[better-auth-credentials-plugin](https://github.com/erickweil/better-auth-credentials-plugin)	LDAP authentication plugin for Better Auth.	 [erickweil](https://github.com/erickweil)

# plugins: Device Authorization
URL: /docs/plugins/device-authorization
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/device-authorization.mdx

OAuth 2.0 Device Authorization Grant for limited-input devices

***

title: Device Authorization
description: OAuth 2.0 Device Authorization Grant for limited-input devices

---

`RFC 8628` `CLI` `Smart TV` `IoT`

The Device Authorization plugin implements the OAuth 2.0 Device Authorization Grant ([RFC 8628](https://datatracker.ietf.org/doc/html/rfc8628)), enabling authentication for devices with limited input capabilities such as smart TVs, CLI applications, IoT devices, and gaming consoles.

## Try It Out

You can test the device authorization flow right now using the Better Auth CLI:

```bash
npx @better-auth/cli login
```

This will demonstrate the complete device authorization flow by:

1. Requesting a device code from the Better Auth demo server
2. Displaying a user code for you to enter
3. Opening your browser to the verification page
4. Polling for authorization completion

<Callout type="info">
  The CLI login command is a demo feature that connects to the Better Auth demo server to showcase the device authorization flow in action.
</Callout>

## Installation

<Steps>
  <Step>
    ### Add the plugin to your auth config

    Add the device authorization plugin to your server configuration.

```ts title="auth.ts"
    import { betterAuth } from "better-auth";
    import { deviceAuthorization } from "better-auth/plugins"; // [!code highlight]

    export const auth = betterAuth({
      // ... other config
      plugins: [ // [!code highlight]
        deviceAuthorization({ // [!code highlight]
          // Optional configuration
          expiresIn: "30m", // Device code expiration time // [!code highlight]
          interval: "5s",    // Minimum polling interval // [!code highlight]
        }), // [!code highlight]
      ], // [!code highlight]
    });
```
  </Step>

  <Step>
    ### Migrate the database

    Run the migration or generate the schema to add the necessary tables to the database.

    <Tabs items={["migrate", "generate"]}>
      <Tab value="migrate">
```bash
        npx @better-auth/cli migrate
```
      </Tab>

      <Tab value="generate">
```bash
        npx @better-auth/cli generate
```
      </Tab>
    </Tabs>

~~~
    See the [Schema](#schema) section to add the fields manually.
  </Step>

  <Step>
    ### Add the client plugin

    Add the device authorization plugin to your client.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client";
    import { deviceAuthorizationClient } from "better-auth/client/plugins"; // [!code highlight]

    export const authClient = createAuthClient({
      plugins: [ // [!code highlight]
        deviceAuthorizationClient(), // [!code highlight]
      ], // [!code highlight]
    });
    ```
  </Step>
</Steps>

## How It Works

The device flow follows these steps:

1. **Device requests codes**: The device requests a device code and user code from the authorization server
2. **User authorizes**: The user visits a verification URL and enters the user code
3. **Device polls for token**: The device polls the server until the user completes authorization
4. **Access granted**: Once authorized, the device receives an access token

## Basic Usage

### Requesting Device Authorization

To initiate device authorization, call `device.code` with the client ID:

### Client Side

```ts
const { data, error } = await authClient.device.code({
    client_id,
    scope, // required
});
```

### Server Side

```ts
const data = await auth.api.deviceCode({
    body: {
        client_id,
        scope, // required
    }
});
```

### Type Definition

```ts
type deviceCode = {
    /**
     * The OAuth client identifier
     */
    client_id: string;
    /**
     * Space-separated list of requested scopes (optional)
     */
    scope?: string;

}
```

Example usage:

```ts
const { data } = await authClient.device.code({
  client_id: "your-client-id",
  scope: "openid profile email",
});

if (data) {
  console.log(`Please visit: ${data.verification_uri}`);
  console.log(`And enter code: ${data.user_code}`);
}
```

### Polling for Token

After displaying the user code, poll for the access token:

### Client Side

```ts
const { data, error } = await authClient.device.token({
    grant_type,
    device_code,
    client_id,
});
```

### Server Side

```ts
const data = await auth.api.deviceToken({
    body: {
        grant_type,
        device_code,
        client_id,
    }
});
```

### Type Definition

```ts
type deviceToken = {
    /**
     * Must be "urn:ietf:params:oauth:grant-type:device_code"
     */
    grant_type: string;
    /**
     * The device code from the initial request
     */
    device_code: string;
    /**
     * The OAuth client identifier
     */
    client_id: string;

}
```
~~~

Example polling implementation:

```ts
let pollingInterval = 5; // Start with 5 seconds
const pollForToken = async () => {
  const { data, error } = await authClient.device.token({
    grant_type: "urn:ietf:params:oauth:grant-type:device_code",
    device_code,
    client_id: yourClientId,
    fetchOptions: {
      headers: {
        "user-agent": `My CLI`,
      },
    },
  });

  if (data?.access_token) {
    console.log("Authorization successful!");
  } else if (error) {
    switch (error.error) {
      case "authorization_pending":
        // Continue polling
        break;
      case "slow_down":
        pollingInterval += 5;
        break;
      case "access_denied":
        console.error("Access was denied by the user");
        return;
      case "expired_token":
        console.error("The device code has expired. Please try again.");
        return;
      default:
        console.error(`Error: ${error.error_description}`);
        return;
    }
    setTimeout(pollForToken, pollingInterval * 1000);
  }
};

pollForToken();
```

### User Authorization Flow

The user authorization flow requires two steps:

1. **Code Verification**: Check if the entered user code is valid
2. **Authorization**: User must be authenticated to approve/deny the device

<Callout type="warn">
  Users must be authenticated before they can approve or deny device authorization requests. If not authenticated, redirect them to the login page with a return URL.
</Callout>

Create a page where users can enter their code:

```tsx title="app/device/page.tsx"
export default function DeviceAuthorizationPage() {
  const [userCode, setUserCode] = useState("");
  const [error, setError] = useState(null);

  const handleSubmit = async (e) => {
    e.preventDefault();

    try {
      // Format the code: remove dashes and convert to uppercase
      const formattedCode = userCode.trim().replace(/-/g, "").toUpperCase();

      // Check if the code is valid using GET /device endpoint
      const response = await authClient.device({
        query: { user_code: formattedCode },
      });

      if (response.data) {
        // Redirect to approval page
        window.location.href = `/device/approve?user_code=${formattedCode}`;
      }
    } catch (err) {
      setError("Invalid or expired code");
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        value={userCode}
        onChange={(e) => setUserCode(e.target.value)}
        placeholder="Enter device code (e.g., ABCD-1234)"
        maxLength={12}
      />
      <button type="submit">Continue</button>
      {error && <p>{error}</p>}
    </form>
  );
}
```

### Approving or Denying Device

Users must be authenticated to approve or deny device authorization requests:

#### Approve Device

### Client Side

```ts
const { data, error } = await authClient.device.approve({
    userCode,
});
```

### Server Side

```ts
const data = await auth.api.deviceApprove({
    body: {
        userCode,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type deviceApprove = {
    /**
     * The user code to approve
     */
    userCode: string;

}
```

#### Deny Device

### Client Side

```ts
const { data, error } = await authClient.device.deny({
    userCode,
});
```

### Server Side

```ts
const data = await auth.api.deviceDeny({
    body: {
        userCode,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type deviceDeny = {
    /**
     * The user code to deny
     */
    userCode: string;

}
```

#### Example Approval Page

```tsx title="app/device/approve/page.tsx"
export default function DeviceApprovalPage() {
  const { user } = useAuth(); // Must be authenticated
  const searchParams = useSearchParams();
  const userCode = searchParams.get("userCode");
  const [isProcessing, setIsProcessing] = useState(false);

  const handleApprove = async () => {
    setIsProcessing(true);
    try {
      await authClient.device.approve({
        userCode: userCode,
      });
      // Show success message
      alert("Device approved successfully!");
      window.location.href = "/";
    } catch (error) {
      alert("Failed to approve device");
    }
    setIsProcessing(false);
  };

  const handleDeny = async () => {
    setIsProcessing(true);
    try {
      await authClient.device.deny({
        userCode: userCode,
      });
      alert("Device denied");
      window.location.href = "/";
    } catch (error) {
      alert("Failed to deny device");
    }
    setIsProcessing(false);
  };

  if (!user) {
    // Redirect to login if not authenticated
    window.location.href = `/login?redirect=/device/approve?user_code=${userCode}`;
    return null;
  }

  return (
    <div>
      <h2>Device Authorization Request</h2>
      <p>A device is requesting access to your account.</p>
      <p>Code: {userCode}</p>

      <button onClick={handleApprove} disabled={isProcessing}>
        Approve
      </button>
      <button onClick={handleDeny} disabled={isProcessing}>
        Deny
      </button>
    </div>
  );
}
```

## Advanced Configuration

### Client Validation

You can validate client IDs to ensure only authorized applications can use the device flow:

```ts
deviceAuthorization({
  validateClient: async (clientId) => {
    // Check if client is authorized
    const client = await db.oauth_clients.findOne({ id: clientId });
    return client && client.allowDeviceFlow;
  },

  onDeviceAuthRequest: async (clientId, scope) => {
    // Log device authorization requests
    await logDeviceAuthRequest(clientId, scope);
  },
})
```

### Custom Code Generation

Customize how device and user codes are generated:

```ts
deviceAuthorization({
  generateDeviceCode: async () => {
    // Custom device code generation
    return crypto.randomBytes(32).toString("hex");
  },

  generateUserCode: async () => {
    // Custom user code generation
    // Default uses: ABCDEFGHJKLMNPQRSTUVWXYZ23456789
    // (excludes 0, O, 1, I to avoid confusion)
    const charset = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789";
    let code = "";
    for (let i = 0; i < 8; i++) {
      code += charset[Math.floor(Math.random() * charset.length)];
    }
    return code;
  },
})
```

## Error Handling

The device flow defines specific error codes:

Error Code	Description
`authorization_pending`	User hasn't approved yet (continue polling)
`slow_down`	Polling too frequently (increase interval)
`expired_token`	Device code has expired
`access_denied`	User denied the authorization
`invalid_grant`	Invalid device code or client ID

## Example: CLI Application

Here's a complete example for a CLI application based on the actual demo:

```ts title="cli-auth.ts"
import { createAuthClient } from "better-auth/client";
import { deviceAuthorizationClient } from "better-auth/client/plugins";
import open from "open";

const authClient = createAuthClient({
  baseURL: "http://localhost:3000",
  plugins: [deviceAuthorizationClient()],
});

async function authenticateCLI() {
  console.log("ğŸ”  Better Auth Device Authorization Demo");
  console.log("â ³ Requesting device authorization...");

  try {
    // Request device code
    const { data, error } = await authClient.device.code({
      client_id: "demo-cli",
      scope: "openid profile email",
    });

    if (error || !data) {
      console.error("â Œ Error:", error?.error_description);
      process.exit(1);
    }

    const {
      device_code,
      user_code,
      verification_uri,
      verification_uri_complete,
      interval = 5,
    } = data;

    console.log("\nğŸ“± Device Authorization in Progress");
    console.log(`Please visit: ${verification_uri}`);
    console.log(`Enter code: ${user_code}\n`);

    // Open browser with the complete URL
    const urlToOpen = verification_uri_complete || verification_uri;
    if (urlToOpen) {
      console.log("ğŸŒ  Opening browser...");
      await open(urlToOpen);
    }

    console.log(`â ³ Waiting for authorization... (polling every ${interval}s)`);

    // Poll for token
    await pollForToken(device_code, interval);
  } catch (err) {
    console.error("â Œ Error:", err.message);
    process.exit(1);
  }
}

async function pollForToken(deviceCode: string, interval: number) {
  let pollingInterval = interval;

  return new Promise<void>((resolve) => {
    const poll = async () => {
      try {
        const { data, error } = await authClient.device.token({
          grant_type: "urn:ietf:params:oauth:grant-type:device_code",
          device_code: deviceCode,
          client_id: "demo-cli",
        });

        if (data?.access_token) {
          console.log("\nAuthorization Successful!");
          console.log("Access token received!");

          // Get user session
          const { data: session } = await authClient.getSession({
            fetchOptions: {
              headers: {
                Authorization: `Bearer ${data.access_token}`,
              },
            },
          });
```

~~~~
          console.log(`Hello, ${session?.user?.name || "User"}!`);
          resolve();
          process.exit(0);
        } else if (error) {
          switch (error.error) {
            case "authorization_pending":
              // Continue polling silently
              break;
            case "slow_down":
              pollingInterval += 5;
              console.log(`âš ï¸    Slowing down polling to ${pollingInterval}s`);
              break;
            case "access_denied":
              console.error("â  Œ Access was denied by the user");
              process.exit(1);
              break;
            case "expired_token":
              console.error("â  Œ The device code has expired. Please try again.");
              process.exit(1);
              break;
            default:
              console.error("â  Œ Error:", error.error_description);
              process.exit(1);
          }
        }
      } catch (err) {
        console.error("â  Œ Network error:", err.message);
        process.exit(1);
      }

      // Schedule next poll
      setTimeout(poll, pollingInterval * 1000);
    };

    // Start polling
    setTimeout(poll, pollingInterval * 1000);
  });
}

// Run the authentication flow
authenticateCLI().catch((err) => {
  console.error("â  Œ Fatal error:", err);
  process.exit(1);
});
```

## Security Considerations

1. **Rate Limiting**: The plugin enforces polling intervals to prevent abuse
2. **Code Expiration**: Device and user codes expire after the configured time (default: 30 minutes)
3. **Client Validation**: Always validate client IDs in production to prevent unauthorized access
4. **HTTPS Only**: Always use HTTPS in production for device authorization
5. **User Code Format**: User codes use a limited character set (excluding similar-looking characters like 0/O, 1/I) to reduce typing errors
6. **Authentication Required**: Users must be authenticated before they can approve or deny device requests

## Options

### Server

**expiresIn**: The expiration time for device codes. Default: `"30m"` (30 minutes).

**interval**: The minimum polling interval. Default: `"5s"` (5 seconds).

**userCodeLength**: The length of the user code. Default: `8`.

**deviceCodeLength**: The length of the device code. Default: `40`.

**generateDeviceCode**: Custom function to generate device codes. Returns a string or `Promise<string>`.

**generateUserCode**: Custom function to generate user codes. Returns a string or `Promise<string>`.

**validateClient**: Function to validate client IDs. Takes a clientId and returns boolean or `Promise<boolean>`.

**onDeviceAuthRequest**: Hook called when device authorization is requested. Takes clientId and optional scope.

### Client

No client-specific configuration options. The plugin adds the following methods:

* **device()**: Verify user code validity
* **device.code()**: Request device and user codes
* **device.token()**: Poll for access token
* **device.approve()**: Approve device (requires authentication)
* **device.deny()**: Deny device (requires authentication)

## Schema

The plugin requires a new table to store device authorization data.

Table Name: `deviceCode`

<DatabaseTable
  fields={[
      {
          name: "id",
          type: "string",
          description: "Unique identifier for the device authorization request",
          isPrimaryKey: true
      },
      {
          name: "deviceCode",
          type: "string",
          description: "The device verification code",
      },
      {
          name: "userCode",
          type: "string",
          description: "The user-friendly code for verification",
      },
      {
          name: "userId",
          type: "string",
          description: "The ID of the user who approved/denied",
          isOptional: true,
          isForeignKey: true
      },
      {
          name: "clientId",
          type: "string",
          description: "The OAuth client identifier",
          isOptional: true
      },
      {
          name: "scope",
          type: "string",
          description: "Requested OAuth scopes",
          isOptional: true
      },
      {
~~~~

~~~~
          name: "status",
          type: "string",
          description: "Current status: pending, approved, or denied",
      },
      {
          name: "expiresAt",
          type: "Date",
          description: "When the device code expires",
      },
      {
          name: "lastPolledAt",
          type: "Date",
          description: "Last time the device polled for status",
          isOptional: true
      },
      {
          name: "pollingInterval",
          type: "number",
          description: "Minimum seconds between polls",
          isOptional: true
      },
      {
          name: "createdAt",
          type: "Date",
          description: "When the request was created",
      },
      {
          name: "updatedAt",
          type: "Date",
          description: "When the request was last updated",
      }
  ]}
/>



# plugins: Dodo Payments
URL: /docs/plugins/dodopayments
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/dodopayments.mdx

Better Auth Plugin for Dodo Payments

***

title: Dodo Payments
description: Better Auth Plugin for Dodo Payments
-------------------------------------------------

[Dodo Payments](https://dodopayments.com) is a global Merchant-of-Record platform that lets AI, SaaS and digital businesses sell in 150+ countries without touching tax, fraud, or
compliance. A single, developer-friendly API powers checkout, billing, and payouts so you can launch worldwide in minutes.

<Card href="https://discord.gg/bYqAp4ayYh" title="Get support on Dodo Payments' Discord">
  This plugin is maintained by the Dodo Payments team.<br />
  Have questions? Our team is available on Discord to assist you anytime.
</Card>

## Features

* Automatic customer creation on sign-up
* Type-safe checkout flows with product slug mapping
* Self-service customer portal
* Real-time webhook event processing with signature verification

<Card href="https://app.dodopayments.com" title="Get started with Dodo Payments">
  You need a Dodo Payments account and API keys to use this integration.
</Card>

## Installation

<Steps>
  <Step title="Install dependencies">
    Run the following command in your project root:

    ```bash
    npm install @dodopayments/better-auth dodopayments better-auth zod
    ```
  </Step>

  <Step title="Configure environment variables">
    Add these to your `.env` file:

    ```txt
    DODO_PAYMENTS_API_KEY=your_api_key_here
    DODO_PAYMENTS_WEBHOOK_SECRET=your_webhook_secret_here
    ```
  </Step>

  <Step title="Set up server-side integration">
    Create or update `src/lib/auth.ts`:

    ```typescript
    import { betterAuth } from "better-auth";
    import {
      dodopayments,
      checkout,
      portal,
      webhooks,
    } from "@dodopayments/better-auth";
    import DodoPayments from "dodopayments";

    export const dodoPayments = new DodoPayments({
      bearerToken: process.env.DODO_PAYMENTS_API_KEY!,
      environment: "test_mode"
    });

    export const auth = betterAuth({
      plugins: [
        dodopayments({
          client: dodoPayments,
          createCustomerOnSignUp: true,
          use: [
            checkout({
              products: [
                {
~~~~

~~~
                  productId: "pdt_xxxxxxxxxxxxxxxxxxxxx",
                  slug: "premium-plan",
                },
              ],
              successUrl: "/dashboard/success",
              authenticatedUsersOnly: true,
            }),
            portal(),
            webhooks({
              webhookKey: process.env.DODO_PAYMENTS_WEBHOOK_SECRET!,
              onPayload: async (payload) => {
                console.log("Received webhook:", payload.event_type);
              },
            }),
          ],
        }),
      ],
    });
    ```

    <Card>
      Set `environment` to `live_mode` for production.
    </Card>
  </Step>

  <Step title="Set up client-side integration">
    Create or update `src/lib/auth-client.ts`:

    ```typescript
    import { dodopaymentsClient } from "@dodopayments/better-auth";

    export const authClient = createAuthClient({
      baseURL: process.env.BETTER_AUTH_URL || "http://localhost:3000",
      plugins: [dodopaymentsClient()],
    });
    ```
  </Step>
</Steps>

## Usage

### Creating a Checkout Session

```typescript
const { data: checkout, error } = await authClient.dodopayments.checkout({
  slug: "premium-plan",
  customer: {
    email: "customer@example.com",
    name: "John Doe",
  },
  billing: {
    city: "San Francisco",
    country: "US",
    state: "CA",
    street: "123 Market St",
    zipcode: "94103",
  },
  referenceId: "order_123",
});

if (checkout) {
  window.location.href = checkout.url;
}
```

### Accessing the Customer Portal

```typescript
const { data: customerPortal, error } = await authClient.dodopayments.customer.portal();
if (customerPortal && customerPortal.redirect) {
  window.location.href = customerPortal.url;
}
```

### Listing Customer Data

```typescript
// Get subscriptions
const { data: subscriptions, error } =
  await authClient.dodopayments.customer.subscriptions.list({
    query: {
      limit: 10,
      page: 1,
      active: true,
    },
  });

// Get payment history
const { data: payments, error } = await authClient.dodopayments.customer.payments.list({
  query: {
    limit: 10,
    page: 1,
    status: "succeeded",
  },
});
```

### Webhooks

<Card>
  The webhooks plugin processes real-time payment events from Dodo Payments with secure signature verification. The default endpoint is `/api/auth/dodopayments/webhooks`.
</Card>

<Steps>
  <Step title="Generate and set webhook secret">
    Generate a webhook secret for your endpoint URL (e.g., `https://your-domain.com/api/auth/dodopayments/webhooks`) in the Dodo Payments Dashboard and set it in your .env file:

    ```txt
    DODO_PAYMENTS_WEBHOOK_SECRET=your_webhook_secret_here
    ```
  </Step>

  <Step title="Handle webhook events">
    Example handler:

    ```typescript
    webhooks({
      webhookKey: process.env.DODO_PAYMENTS_WEBHOOK_SECRET!,
      onPayload: async (payload) => {
        console.log("Received webhook:", payload.event_type);
      },
    });
    ```
  </Step>
</Steps>

## Configuration Reference

### Plugin Options
~~~

~~~
* **client** (required): DodoPayments client instance
* **createCustomerOnSignUp** (optional): Auto-create customers on user signup
* **use** (required): Array of plugins to enable (checkout, portal, webhooks)

### Checkout Plugin Options

* **products**: Array of products or async function returning products
* **successUrl**: URL to redirect after successful payment
* **authenticatedUsersOnly**: Require user authentication (default: false)

If you encounter any issues, please refer to the [Dodo Payments documentation](https://docs.dodopayments.com) for troubleshooting steps.



# plugins: Dub
URL: /docs/plugins/dub
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/dub.mdx

Better Auth Plugin for Lead Tracking using Dub links and OAuth Linking

***

title: Dub
description: Better Auth Plugin for Lead Tracking using Dub links and OAuth Linking
-----------------------------------------------------------------------------------

[Dub](https://dub.co/) is an open source modern link management platform for entrepreneurs, creators, and growth teams.

This plugins allows you to track leads when a user signs up using a Dub link. It also adds OAuth linking support to allow you to build integrations extending Dub's linking
management infrastructure.

## Installation

<Steps>
  <Step>
    ### Install the plugin

    First, install the plugin:

    <CodeBlockTabs defaultValue="npm">
      <CodeBlockTabsList>
        <CodeBlockTabsTrigger value="npm">
          npm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="pnpm">
          pnpm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="yarn">
          yarn
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="bun">
          bun
        </CodeBlockTabsTrigger>
      </CodeBlockTabsList>

      <CodeBlockTab value="npm">
        ```bash
        npm install @dub/better-auth
        ```
      </CodeBlockTab>

      <CodeBlockTab value="pnpm">
        ```bash
        pnpm add @dub/better-auth
        ```
      </CodeBlockTab>

      <CodeBlockTab value="yarn">
        ```bash
        yarn add @dub/better-auth
        ```
      </CodeBlockTab>

      <CodeBlockTab value="bun">
        ```bash
        bun add @dub/better-auth
        ```
      </CodeBlockTab>
    </CodeBlockTabs>
  </Step>

  <Step>
    ### Install the Dub SDK

    Next, install the Dub SDK on your server:

    <CodeBlockTabs defaultValue="npm">
      <CodeBlockTabsList>
        <CodeBlockTabsTrigger value="npm">
          npm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="pnpm">
          pnpm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="yarn">
          yarn
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="bun">
          bun
        </CodeBlockTabsTrigger>
      </CodeBlockTabsList>

      <CodeBlockTab value="npm">
        ```bash
        npm install dub
        ```
      </CodeBlockTab>

      <CodeBlockTab value="pnpm">
        ```bash
        pnpm add dub
        ```
      </CodeBlockTab>

      <CodeBlockTab value="yarn">
        ```bash
        yarn add dub
        ```
      </CodeBlockTab>

      <CodeBlockTab value="bun">
        ```bash
        bun add dub
        ```
~~~

~~~mdx
      </CodeBlockTab>
    </CodeBlockTabs>
  </Step>

  <Step>
    ### Configure the plugin

    Add the plugin to your auth config:

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { dubAnalytics } from "@dub/better-auth"
    import { dub } from "dub"

    export const auth = betterAuth({
        plugins: [
            dubAnalytics({
                dubClient: new Dub()
            })
        ]
    })
    ```
  </Step>
</Steps>

## Usage

### Lead Tracking

By default, the plugin will track sign up events as leads. You can disable this by setting `disableLeadTracking` to `true`.

```ts
import { dubAnalytics } from "@dub/better-auth";
import { betterAuth } from "better-auth";
import { Dub } from "dub";

const dub = new Dub();

const betterAuth = betterAuth({
  plugins: [
    dubAnalytics({
      dubClient: dub,
      disableLeadTracking: true, // Disable lead tracking
    }),
  ],
});
```

### OAuth Linking

The plugin supports OAuth for account linking.

First, you need to setup OAuth app in Dub. Dub supports OAuth 2.0 authentication, which is recommended if you build integrations extending Dubâ€™s functionality [Learn more about OAuth](https://dub.co/docs/integrations/quickstart#integrating-via-oauth-2-0-recommended).

Once you get the client ID and client secret, you can configure the plugin.

```ts
dubAnalytics({
  dubClient: dub,
  oauth: {
    clientId: "your-client-id",
    clientSecret: "your-client-secret",
  },
});
```

And in the client, you need to use the `dubAnalyticsClient` plugin.

```ts
import { createAuthClient } from "better-auth/client";
import { dubAnalyticsClient } from "@dub/better-auth/client";

const authClient = createAuthClient({
  plugins: [dubAnalyticsClient()],
});
```

To link account with Dub, you need to use the `dub.link`.

### Client Side

```ts
const { data, error } = await authClient.dub.link({
    callbackURL: /dashboard,
});
```

### Server Side

```ts
const data = await auth.api.dubLink({
    body: {
        callbackURL: /dashboard,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type dubLink = {
  /**
   * URL to redirect to after linking
   * @clientOnly
  */
  callbackURL: string = "/dashboard"

}
```

## Options

You can pass the following options to the plugin:

### `dubClient`

The Dub client instance.

### `disableLeadTracking`

Disable lead tracking for sign up events.

### `leadEventName`

Event name for sign up leads.

### `customLeadTrack`
~~~

Custom lead track function.

### `oauth`

Dub OAuth configuration.

### `oauth.clientId`

Client ID for Dub OAuth.

### `oauth.clientSecret`

Client secret for Dub OAuth.

### `oauth.pkce`

Enable PKCE for Dub OAuth.

# plugins: Email OTP

URL: /docs/plugins/email-otp
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/email-otp.mdx

Email OTP plugin for Better Auth.

***

title: Email OTP
description: Email OTP plugin for Better Auth.

The Email OTP plugin allows user to sign in, verify their email, or reset their password using a one-time password (OTP) sent to their email address.

## Installation

<Steps>
  <Step>

### Add the plugin to your auth config

Add the `emailOTP` plugin to your auth config and implement the `sendVerificationOTP()` method.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { emailOTP } from "better-auth/plugins" // [!code highlight]

    export const auth = betterAuth({
        // ... other config options
        plugins: [
            emailOTP({ // [!code highlight]
                async sendVerificationOTP({ email, otp, type }) { // [!code highlight]
                    if (type === "sign-in") { // [!code highlight]
                        // Send the OTP for sign in // [!code highlight]
                    } else if (type === "email-verification") { // [!code highlight]
                        // Send the OTP for email verification // [!code highlight]
                    } else { // [!code highlight]
                        // Send the OTP for password reset // [!code highlight]
                    } // [!code highlight]
                }, // [!code highlight]
            }) // [!code highlight]
        ]
    })
    ```
  </Step>

  <Step>

### Add the client plugin

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    import { emailOTPClient } from "better-auth/client/plugins"

    export const authClient = createAuthClient({
        plugins: [
            emailOTPClient()
        ]
    })
    ```
  </Step>
</Steps>

## Usage

### Send an OTP

Use the `sendVerificationOtp()` method to send an OTP to the user's email address.

### Client Side

```ts
const { data, error } = await authClient.emailOtp.sendVerificationOtp({
    email: user@example.com,
    type: sign-in,
});
```

### Server Side

```ts
const data = await auth.api.sendVerificationOTP({
    body: {
        email: user@example.com,
        type: sign-in,
    }
});
```

### Type Definition

```ts
type sendVerificationOTP = {
    /**
     * Email address to send the OTP.
     */
    email: string = "user@example.com"
    /**
     * Type of the OTP. `sign-in`, `email-verification`, or `forget-password`.
     */
    type: "email-verification" | "sign-in" | "forget-password" = "sign-in"

}
```

### Check an OTP (optional)

Use the `checkVerificationOtp()` method to check if an OTP is valid.

### Client Side

```ts
const { data, error } = await authClient.emailOtp.checkVerificationOtp({
    email: user@example.com,
    type: sign-in,
    otp: 123456,
});
```

### Server Side

```ts
const data = await auth.api.checkVerificationOTP({
    body: {
        email: user@example.com,
        type: sign-in,
        otp: 123456,
    }
});
```

### Type Definition

```ts
type checkVerificationOTP = {
    /**
     * Email address to send the OTP.
     */
    email: string = "user@example.com"
    /**
     * Type of the OTP. `sign-in`, `email-verification`, or `forget-password`.
     */
    type: "email-verification" | "sign-in" | "forget-password" = "sign-in"
    /**
     * OTP sent to the email.
     */
    otp: string = "123456"

}
```

### Sign In with OTP

To sign in with OTP, use the `sendVerificationOtp()` method to send a "sign-in" OTP to the user's email address.

### Client Side

```ts
const { data, error } = await authClient.emailOtp.sendVerificationOtp({
    email: user@example.com,
    type: sign-in,
});
```

### Server Side

```ts
const data = await auth.api.sendVerificationOTP({
    body: {
        email: user@example.com,
        type: sign-in,
    }
});
```

### Type Definition

```ts
type sendVerificationOTP = {
    /**
     * Email address to send the OTP.
     */
    email: string = "user@example.com"
    /**
     * Type of the OTP.
     */
    type: "sign-in" = "sign-in"

}
```

Once the user provides the OTP, you can sign in the user using the `signIn.emailOtp()` method.

### Client Side

```ts
const { data, error } = await authClient.signIn.emailOtp({
    email: user@example.com,
    otp: 123456,
});
```

### Server Side

```ts
const data = await auth.api.signInEmailOTP({
    body: {
        email: user@example.com,
        otp: 123456,
    }
});
```

### Type Definition

```ts
type signInEmailOTP = {
    /**
     * Email address to sign in.
     */
    email: string = "user@example.com"
    /**
     * OTP sent to the email.
     */
    otp: string = "123456"

}
```

<Callout>
  If the user is not registered, they'll be automatically registered. If you want to prevent this, you can pass `disableSignUp` as `true` in the [options](#options).
</Callout>

### Verify Email with OTP

To verify the user's email address with OTP, use the `sendVerificationOtp()` method to send an "email-verification" OTP to the user's email address.

### Client Side

```ts
const { data, error } = await authClient.emailOtp.sendVerificationOtp({
    email: user@example.com,
    type: email-verification,
});
```

### Server Side

```ts
const data = await auth.api.sendVerificationOTP({
    body: {
        email: user@example.com,
        type: email-verification,
    }
});
```

### Type Definition

```ts
type sendVerificationOTP = {
    /**
     * Email address to send the OTP.
     */
    email: string = "user@example.com"
    /**
     * Type of the OTP.
     */
    type: "email-verification" = "email-verification"

}
```

Once the user provides the OTP, use the `verifyEmail()` method to complete email verification.

### Client Side

```ts
const { data, error } = await authClient.emailOtp.verifyEmail({
    email: user@example.com,
    otp: 123456,
});
```

### Server Side

```ts
const data = await auth.api.verifyEmailOTP({
    body: {
        email: user@example.com,
        otp: 123456,
    }
});
```

### Type Definition

```ts
type verifyEmailOTP = {
    /**
     * Email address to verify.
     */
    email: string = "user@example.com"
    /**
     * OTP to verify.
     */
    otp: string = "123456"

}
```

### Reset Password with OTP

To reset the user's password with OTP, use the `forgetPassword.emailOTP()` method to send a "forget-password" OTP to the user's email address.

### Client Side

```ts
const { data, error } = await authClient.forgetPassword.emailOtp({
    email: user@example.com,
});
```

### Server Side

```ts
const data = await auth.api.forgetPasswordEmailOTP({
    body: {
        email: user@example.com,
    }
});
```

### Type Definition

```ts
type forgetPasswordEmailOTP = {
    /**
     * Email address to send the OTP.
     */
    email: string = "user@example.com"

}
```

Once the user provides the OTP, use the `checkVerificationOtp()` method to check if it's valid (optional).

### Client Side

```ts
const { data, error } = await authClient.emailOtp.checkVerificationOtp({
    email: user@example.com,
    type: forget-password,
    otp: 123456,
});
```

### Server Side

```ts
const data = await auth.api.checkVerificationOTP({
    body: {
        email: user@example.com,
        type: forget-password,
        otp: 123456,
    }
});
```

### Type Definition

```ts
type checkVerificationOTP = {
    /**
     * Email address to send the OTP.
     */
    email: string = "user@example.com"
    /**
     * Type of the OTP.
     */
    type: "forget-password" = "forget-password"
    /**
     * OTP sent to the email.
     */
    otp: string = "123456"

}
```

Then, use the `resetPassword()` method to reset the user's password.

### Client Side

```ts
const { data, error } = await authClient.emailOtp.resetPassword({
    email: user@example.com,
    otp: 123456,
    password: new-secure-password,
});
```

### Server Side

```ts
const data = await auth.api.resetPasswordEmailOTP({
    body: {
        email: user@example.com,
        otp: 123456,
        password: new-secure-password,
    }
});
```

### Type Definition

```ts
type resetPasswordEmailOTP = {
    /**
     * Email address to reset the password.
     */
    email: string = "user@example.com"
    /**
     * OTP sent to the email.
     */
    otp: string = "123456"
    /**
     * New password.
     */
    password: string = "new-secure-password"

}
```

### Override Default Email Verification

To override the default email verification, pass `overrideDefaultEmailVerification: true` in the options. This will make the system use an email OTP instead of the default verification link whenever email verification is triggered. In other words, the user will verify their email using an OTP rather than clicking a link.

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
  plugins: [
    emailOTP({
      overrideDefaultEmailVerification: true, // [!code highlight]
      async sendVerificationOTP({ email, otp, type }) {
        // Implement the sendVerificationOTP method to send the OTP to the user's email address
      },
    }),
  ],
});
```

## Options

* `sendVerificationOTP`: A function that sends the OTP to the user's email address. The function receives an object with the following properties:
  * `email`: The user's email address.
  * `otp`: The OTP to send.
  * `type`: The type of OTP to send. Can be "sign-in", "email-verification", or "forget-password".

* `otpLength`: The length of the OTP. Defaults to `6`.

* `expiresIn`: The expiry time of the OTP in seconds. Defaults to `300` seconds.

```ts title="auth.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    plugins: [
        emailOTP({
            otpLength: 8,
            expiresIn: 600
        })
    ]
})
```

* `sendVerificationOnSignUp`: A boolean value that determines whether to send the OTP when a user signs up. Defaults to `false`.

* `disableSignUp`: A boolean value that determines whether to prevent automatic sign-up when the user is not registered. Defaults to `false`.

* `generateOTP`: A function that generates the OTP. Defaults to a random 6-digit number.

* `allowedAttempts`: The maximum number of attempts allowed for verifying an OTP. Defaults to `3`. After exceeding this limit, the OTP becomes invalid and the user needs to request a new one.

```ts title="auth.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    plugins: [
        emailOTP({
            allowedAttempts: 5, // Allow 5 attempts before invalidating the OTP
            expiresIn: 300
        })
    ]
})
```

When the maximum attempts are exceeded, the `verifyOTP`, `signIn.emailOtp`, `verifyEmail`, and `resetPassword` methods will return an error with code `TOO_MANY_ATTEMPTS`.

* `storeOTP`: The method to store the OTP in your database, wether `encrypted`, `hashed` or `plain` text. Default is `plain` text.

<Callout>
  Note: This will not affect the OTP sent to the user, it will only affect the OTP stored in your database.
</Callout>

Alternatively, you can pass a custom encryptor or hasher to store the OTP in your database.

Custom encryptor

```ts title="auth.ts"
emailOTP({
    storeOTP: {
        encrypt: async (otp) => {
            return myCustomEncryptor(otp);
        },
        decrypt: async (otp) => {
            return myCustomDecryptor(otp);
        },
    }
})
```

Custom hasher

```ts title="auth.ts"
emailOTP({
    storeOTP: {
        hash: async (otp) => {
            return myCustomHasher(otp);
        },
    }
})
```

# plugins: Generic OAuth

URL: /docs/plugins/generic-oauth
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/generic-oauth.mdx

Authenticate users with any OAuth provider

***

title: Generic OAuth
description: Authenticate users with any OAuth provider

The Generic OAuth plugin provides a flexible way to integrate authentication with any OAuth provider. It supports both OAuth 2.0 and OpenID Connect (OIDC) flows, allowing you to easily add social login or custom OAuth authentication to your application.

## Installation

<Steps>
  <Step>
    ### Add the plugin to your auth config

    To use the Generic OAuth plugin, add it to your auth config.

```ts title="auth.ts"
import { betterAuth } from "better-auth"
import { genericOAuth } from "better-auth/plugins" // [!code highlight]

export const auth = betterAuth({
    // ... other config options
    plugins: [
        genericOAuth({ // [!code highlight]
            config: [ // [!code highlight]
                { // [!code highlight]
                    providerId: "provider-id", // [!code highlight]
                    clientId: "test-client-id", // [!code highlight]
                    clientSecret: "test-client-secret", // [!code highlight]
                    discoveryUrl: "https://auth.example.com/.well-known/openid-configuration", // [!code highlight]
                    // ... other config options // [!code highlight]
                }, // [!code highlight]
                // Add more providers as needed // [!code highlight]
            ] // [!code highlight]
        }) // [!code highlight]
    ]
})
```
  </Step>

  <Step>
    ### Add the client plugin

    Include the Generic OAuth client plugin in your authentication client instance.

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client"
import { genericOAuthClient } from "better-auth/client/plugins"

export const authClient = createAuthClient({
    plugins: [
        genericOAuthClient()
    ]
})
```
  </Step>
</Steps>

## Usage

The Generic OAuth plugin provides endpoints for initiating the OAuth flow and handling the callback. Here's how to use them:

### Initiate OAuth Sign-In

To start the OAuth sign-in process:

### Client Side

```ts
const { data, error } = await authClient.signIn.oauth2({
    providerId: provider-id,
    callbackURL: /dashboard, // required
    errorCallbackURL: /error-page, // required
    newUserCallbackURL: /welcome, // required
    disableRedirect, // required
    scopes, // required
    requestSignUp, // required
});
```

### Server Side

```ts
const data = await auth.api.signInWithOAuth2({
    body: {
        providerId: provider-id,
        callbackURL: /dashboard, // required
        errorCallbackURL: /error-page, // required
        newUserCallbackURL: /welcome, // required
        disableRedirect, // required
        scopes, // required
        requestSignUp, // required
    }
});
```

### Type Definition

```ts
type signInWithOAuth2 = {
    /**
     * The provider ID for the OAuth provider.
     */
    providerId: string = "provider-id"
    /**
     * The URL to redirect to after sign in.
     */
    callbackURL?: string = "/dashboard"
    /**
     * The URL to redirect to if an error occurs.
     */
    errorCallbackURL?: string = "/error-page"
    /**
     * The URL to redirect to after login if the user is new.
     */
    newUserCallbackURL?: string = "/welcome"
    /**
     * Disable redirect.
     */
    disableRedirect?: boolean = false
    /**
     * Scopes to be passed to the provider authorization request.
     */
    scopes?: string[] = ["my-scope"]
    /**
     * Explicitly request sign-up. Useful when disableImplicitSignUp is true for this provider.
     */
    requestSignUp?: boolean = false

}
```

### Linking OAuth Accounts

To link an OAuth account to an existing user:

### Client Side

```ts
const { data, error } = await authClient.oauth2.link({
    providerId: my-provider-id,
    callbackURL: /successful-link,
});
```

### Server Side

```ts
const data = await auth.api.oAuth2LinkAccount({
    body: {
        providerId: my-provider-id,
        callbackURL: /successful-link,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type oAuth2LinkAccount = {
    /**
     * The OAuth provider ID.
     */
    providerId: string = "my-provider-id"
    /**
     * The URL to redirect to once the account linking was complete.
     */
    callbackURL: string = "/successful-link"

}
```

### Handle OAuth Callback

The plugin mounts a route to handle the OAuth callback `/oauth2/callback/:providerId`. This means by default `${baseURL}/api/auth/oauth2/callback/:providerId` will be used as the callback URL. Make sure your OAuth provider is configured to use this URL.

## Configuration

When adding the plugin to your auth config, you can configure multiple OAuth providers. Each provider configuration object supports the following options:

```ts
interface GenericOAuthConfig {
  providerId: string;
  discoveryUrl?: string;
  authorizationUrl?: string;
  tokenUrl?: string;
  userInfoUrl?: string;
  clientId: string;
  clientSecret: string;
  scopes?: string[];
  redirectURI?: string;
  responseType?: string;
  prompt?: string;
  pkce?: boolean;
  accessType?: string;
  getUserInfo?: (tokens: OAuth2Tokens) => Promise<User | null>;
}
```

### Other Provider Configurations

providerId: A unique string to identify the OAuth provider configuration.

discoveryUrl: (Optional) URL to fetch the provider's OAuth 2.0/OIDC configuration. If provided, endpoints like `authorizationUrl`, `tokenUrl`, and `userInfoUrl` can be auto-discovered.

authorizationUrl: (Optional) The OAuth provider's authorization endpoint. Not required if using `discoveryUrl`.

tokenUrl: (Optional) The OAuth provider's token endpoint. Not required if using `discoveryUrl`.

userInfoUrl: (Optional) The endpoint to fetch user profile information. Not required if using `discoveryUrl`.

clientId: The OAuth client ID issued by your provider.

clientSecret: The OAuth client secret issued by your provider.

scopes: (Optional) An array of scopes to request from the provider (e.g., `["openid", "email", "profile"]`).

redirectURI: (Optional) The redirect URI to use for the OAuth flow. If not set, a default is constructed based on your app's base URL.

responseType: (Optional) The OAuth response type. Defaults to `"code"` for authorization code flow.

responseMode: (Optional) The response mode for the authorization code request, such as `"query"` or `"form_post"`.

prompt: (Optional) Controls the authentication experience (e.g., force login, consent, etc.).

pkce: (Optional) If true, enables PKCE (Proof Key for Code Exchange) for enhanced security. Defaults to `false`.

accessType: (Optional) The access type for the authorization request. Use `"offline"` to request a refresh token.

getUserInfo: (Optional) A custom function to fetch user info from the provider, given the OAuth tokens. If not provided, a default fetch is used.

mapProfileToUser: (Optional) A function to map the provider's user profile to your app's user object. Useful for custom field mapping or transformations.

authorizationUrlParams: (Optional) Additional query parameters to add to the authorization URL. These can override default parameters. You can also provide a function that returns the parameters.

tokenUrlParams: (Optional) Additional query parameters to add to the token URL. These can override default parameters. You can also provide a function that returns the parameters.

disableImplicitSignUp: (Optional) If true, disables automatic sign-up for new users. Sign-in must be explicitly requested with sign-up intent.

disableSignUp: (Optional) If true, disables sign-up for new users entirely. Only existing users can sign in.

authentication: (Optional) The authentication method for token requests. Can be `'basic'` or `'post'`. Defaults to `'post'`.

discoveryHeaders: (Optional) Custom headers to include in the discovery request. Useful for providers that require special headers.

authorizationHeaders: (Optional) Custom headers to include in the authorization request. Useful for providers that require special headers.

overrideUserInfo: (Optional) If true, the user's info in your database will be updated with the provider's info every time they sign in. Defaults to `false`.

## Advanced Usage

### Custom User Info Fetching

You can provide a custom `getUserInfo` function to handle specific provider requirements:

```ts
genericOAuth({
  config: [
    {
      providerId: "custom-provider",
      // ... other config options
      getUserInfo: async (tokens) => {
        // Custom logic to fetch and return user info
        const userInfo = await fetchUserInfoFromCustomProvider(tokens);
        return {
          id: userInfo.sub,
          email: userInfo.email,
          name: userInfo.name,
          // ... map other fields as needed
        };
      }
    }
  ]
})
```

### Map User Info Fields

If the user info returned by the provider does not match the expected format, or you need to map additional fields, you can use the `mapProfileToUser`:

```ts
genericOAuth({
  config: [
    {
      providerId: "custom-provider",
      // ... other config options
      mapProfileToUser: async (profile) => {
        return {
          firstName: profile.given_name,
          // ... map other fields as needed
        };
      }
    }
  ]
})
```

### Error Handling

The plugin includes built-in error handling for common OAuth issues. Errors are typically redirected to your application's error page with an appropriate error message in the URL parameters. If the callback URL is not provided, the user will be redirected to Better Auth's default error page.

# plugins: Have I Been Pwned
URL: /docs/plugins/have-i-been-pwned
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/have-i-been-pwned.mdx

A plugin to check if a password has been compromised

***

title: Have I Been Pwned
description: A plugin to check if a password has been compromised
----

The Have I Been Pwned plugin helps protect user accounts by preventing the use of passwords that have been exposed in known data breaches. It uses the [Have I Been Pwned](https://haveibeenpwned.com/) API to check if a password has been compromised.

## Installation

### Add the plugin to your **auth** config

```ts title="auth.ts"
import { betterAuth } from "better-auth"
import { haveIBeenPwned } from "better-auth/plugins" // [!code highlight]

export const auth = betterAuth({
    plugins: [
        haveIBeenPwned()
    ]
})
```

## Usage

When a user attempts to create an account or update their password with a compromised password, they'll receive the following default error:

```json
{
  "code": "PASSWORD_COMPROMISED",
  "message": "Password is compromised"
}
```

## Config

You can customize the error message:

```ts
haveIBeenPwned({
    customPasswordCompromisedMessage: "Please choose a more secure password."
})
```

## Security Notes

* Only the first 5 characters of the password hash are sent to the API
* The full password is never transmitted
* Provides an additional layer of account security

# plugins: JWT
URL: /docs/plugins/jwt
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/jwt.mdx

Authenticate users with JWT tokens in services that can't use the session

***

title: JWT
description: Authenticate users with JWT tokens in services that can't use the session
----

The JWT plugin provides endpoints to retrieve a JWT token and a JWKS endpoint to verify the token.

<Callout type="info">
  This plugin is not meant as a replacement for the session. It's meant to be used for services that require JWT tokens. If you're looking to use JWT tokens for authentication, check out the [Bearer Plugin](/docs/plugins/bearer).
</Callout>

## Installation

<Steps>
  <Step>
    ### Add the plugin to your **auth** config

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { jwt } from "better-auth/plugins"

    export const auth = betterAuth({
        plugins: [ // [!code highlight]
            jwt(), // [!code highlight]
        ] // [!code highlight]
    })
    ```
  </Step>

  <Step>
    ### Migrate the database

    Run the migration or generate the schema to add the necessary fields and tables to the database.

    <Tabs items={["migrate", "generate"]}>
      <Tab value="migrate">
        ```bash
        npx @better-auth/cli migrate
        ```
      </Tab>

      <Tab value="generate">
        ```bash
        npx @better-auth/cli generate
        ```
      </Tab>
    </Tabs>

    See the [Schema](#schema) section to add the fields manually.
  </Step>
</Steps>

## Usage

Once you've installed the plugin, you can start using the JWT & JWKS plugin to get the token and the JWKS through their respective endpoints.

## JWT

### Retrieve the token

There are multiple ways to retrieve JWT tokens:

1. **Using the client plugin (recommended)**

Add the `jwtClient` plugin to your auth client configuration:

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client"
import { jwtClient } from "better-auth/client/plugins" // [!code highlight]

export const authClient = createAuthClient({
  plugins: [
    jwtClient() // [!code highlight]
  ]
})
```

Then use the client to get JWT tokens:

```ts
const { data, error } = await authClient.token()
if (error) {
  // handle error
}
if (data) {
  const jwtToken = data.token
  // Use this token for authenticated requests to external services
}
```

This is the recommended approach for client applications that need JWT tokens for external API authentication.

2. **Using your session token**

To get the token, call the `/token` endpoint. This will return the following:

```json
  {
    "token": "ey..."
  }
```

Make sure to include the token in the `Authorization` header of your requests if the `bearer` plugin is added in your auth configuration.

```ts
await fetch("/api/auth/token", {
  headers: {
    "Authorization": `Bearer ${token}`
  },
})
```

3. **From `set-auth-jwt` header**

When you call `getSession` method, a JWT is returned in the `set-auth-jwt` header, which you can use to send to your services directly.

```ts
await authClient.getSession({
  fetchOptions: {
    onSuccess: (ctx)=>{
      const jwt = ctx.response.headers.get("set-auth-jwt")
    }
  }
})
```

### Verifying the token

The token can be verified in your own service, without the need for an additional verify call or database check.
For this JWKS is used. The public key can be fetched from the `/api/auth/jwks` endpoint.

Since this key is not subject to frequent changes, it can be cached indefinitely.
The key ID (`kid`) that was used to sign a JWT is included in the header of the token.
In case a JWT with a different `kid` is received, it is recommended to fetch the JWKS again.

```json
  {
    "keys": [
        {
            "crv": "Ed25519",
            "x": "bDHiLTt7u-VIU7rfmcltcFhaHKLVvWFy-_csKZARUEU",
            "kty": "OKP",
            "kid": "c5c7995d-0037-4553-8aee-b5b620b89b23"
        }
    ]
  }
```

### OAuth Provider Mode

If you are making your system oAuth compliant (such as when utilizing the OIDC or MCP plugins), you **MUST** disable the `/token` endpoint (oAuth equivalent `/oauth2/token`) and disable setting the jwt header (oAuth equivalent `/oauth2/userinfo`).

```ts title="auth.ts"
betterAuth({
  disabledPaths: [
    "/token",
  ],
  plugins: [jwt({
    disableSettingJwtHeader: true,
  })]
})
```

#### Example using jose with remote JWKS

```ts
import { jwtVerify, createRemoteJWKSet } from 'jose'

async function validateToken(token: string) {
  try {
    const JWKS = createRemoteJWKSet(
      new URL('http://localhost:3000/api/auth/jwks')
    )
    const { payload } = await jwtVerify(token, JWKS, {
      issuer: 'http://localhost:3000', // Should match your JWT issuer, which is the BASE_URL
      audience: 'http://localhost:3000', // Should match your JWT audience, which is the BASE_URL by default
    })
    return payload
  } catch (error) {
    console.error('Token validation failed:', error)
    throw error
  }
}

// Usage example
const token = 'your.jwt.token' // this is the token you get from the /api/auth/token endpoint
const payload = await validateToken(token)
```

#### Example with local JWKS

```ts
import { jwtVerify, createLocalJWKSet } from 'jose'


async function validateToken(token: string) {
  try {
    /**
     * This is the JWKS that you get from the /api/auth/
     * jwks endpoint
     */
    const storedJWKS = {
      keys: [{
        //...
      }]
    };
    const JWKS = createLocalJWKSet({
      keys: storedJWKS.data?.keys!,
    })
    const { payload } = await jwtVerify(token, JWKS, {
      issuer: 'http://localhost:3000', // Should match your JWT issuer, which is the BASE_URL
      audience: 'http://localhost:3000', // Should match your JWT audience, which is the BASE_URL by default
    })
    return payload
  } catch (error) {
    console.error('Token validation failed:', error)
    throw error
  }
}

// Usage example
const token = 'your.jwt.token' // this is the token you get from the /api/auth/token endpoint
const payload = await validateToken(token)
```

### Remote JWKS Url

Disables the `/jwks` endpoint and uses this endpoint in any discovery such as OIDC.

Useful if your JWKS are not managed at `/jwks` or if your jwks are signed with a certificate and placed on your CDN.

NOTE: you **MUST** specify which asymmetric algorithm is used for signing.

```ts title="auth.ts"
jwt({
  jwks: {
    remoteUrl: "https://example.com/.well-known/jwks.json",
    keyPairConfig: {
      alg: 'ES256',
    },
  }
})
```

### Custom Signing

This is an advanced feature. Configuration outside of this plugin **MUST** be provided.

Implementers:

* `remoteUrl` must be defined if using the `sign` function. This shall store all active keys, not just the current one.
* If using localized approach, ensure server uses the latest private key when rotated. Depending on deployment, the server may need to be restarted.
* When using remote approach, verify the payload is unchanged after transit. Use integrity validation like CRC32 or SHA256 checks if available.

#### Localized Signing

```ts title="auth.ts"
jwt({
  jwks: {
    remoteUrl: "https://example.com/.well-known/jwks.json",
    keyPairConfig: {
      alg: 'EdDSA',
    },
  },
  jwt: {
    sign: async (jwtPayload: JWTPayload) => {
      // this is pseudocode
      return await new SignJWT(jwtPayload)
        .setProtectedHeader({
          alg: "EdDSA",
          kid: process.env.currentKid,
          typ: "JWT",
        })
        .sign(process.env.clientPrivateKey);
    },
  },
})
```

#### Remote Signing

Useful if you are using a remote Key Management Service such as [Google KMS](https://cloud.google.com/kms/docs/encrypt-decrypt-rsa#kms-encrypt-asymmetric-nodejs), [Amazon KMS](https://docs.aws.amazon.com/kms/latest/APIReference/API_Sign.html), or [Azure Key Vault](https://learn.microsoft.com/en-us/rest/api/keyvault/keys/sign/sign?view=rest-keyvault-keys-7.4\&tabs=HTTP).

```ts title="auth.ts"
jwt({
  jwks: {
    remoteUrl: "https://example.com/.well-known/jwks.json",
    keyPairConfig: {
      alg: 'ES256',
    },
  },
  jwt: {
    sign: async (jwtPayload: JWTPayload) => {
      // this is pseudocode
      const headers = JSON.stringify({ kid: '123', alg: 'ES256', typ: 'JWT' })
      const payload = JSON.stringify(jwtPayload)
      const encodedHeaders = Buffer.from(headers).toString('base64url')
      const encodedPayload = Buffer.from(payload).toString('base64url')
      const hash = createHash('sha256')
      const data = `${encodedHeaders}.${encodedPayload}`
      hash.update(Buffer.from(data))
      const digest = hash.digest()
      const sig = await remoteSign(digest)
      // integrityCheck(sig)
      const jwt = `${data}.${sig}`
      // verifyJwt(jwt)
      return jwt
    },
  },
})
```

## Schema

The JWT plugin adds the following tables to the database:

### JWKS

Table Name: `jwks`

```
<DatabaseTable
  fields={[
  {
    name: "id",
    type: "string",
    description: "Unique identifier for each web key",
    isPrimaryKey: true
  },
  {
    name: "publicKey",
    type: "string",
    description: "The public part of the web key"
  },
  {
    name: "privateKey",
    type: "string",
    description: "The private part of the web key"
  },
  {
    name: "createdAt",
    type: "Date",
    description: "Timestamp of when the web key was created"
  },
]}
/>
```

<Callout>
  You can customize the table name and fields for the `jwks` table. See the [Database concept documentation](/docs/concepts/database#custom-table-names) for more information on how to customize plugin schema.
</Callout>

## Options

### Algorithm of the Key Pair

The algorithm used for the generation of the key pair. The default is **EdDSA** with the **Ed25519** curve. Below are the available options:

```ts title="auth.ts"
jwt({
  jwks: {
    keyPairConfig: {
      alg: "EdDSA",
      crv: "Ed25519"
    }
  }
})
```

#### EdDSA

* **Default Curve**: `Ed25519`
* **Optional Property**: `crv`
  * Available options: `Ed25519`, `Ed448`
  * Default: `Ed25519`

#### ES256

* No additional properties

#### RSA256

* **Optional Property**: `modulusLength`
  * Expects a number
  * Default: `2048`

#### PS256

* **Optional Property**: `modulusLength`
  * Expects a number
  * Default: `2048`

#### ECDH-ES

* **Optional Property**: `crv`
  * Available options: `P-256`, `P-384`, `P-521`
  * Default: `P-256`

#### ES512

* No additional properties

### Disable private key encryption

By default, the private key is encrypted using AES256 GCM. You can disable this by setting the `disablePrivateKeyEncryption` option to `true`.

For security reasons, it's recommended to keep the private key encrypted.

```ts title="auth.ts"
jwt({
  jwks: {
    disablePrivateKeyEncryption: true
  }
})
```

### Modify JWT payload

By default the entire user object is added to the JWT payload. You can modify the payload by providing a function to the `definePayload` option.

```ts title="auth.ts"
jwt({
  jwt: {
    definePayload: ({user}) => {
      return {
        id: user.id,
        email: user.email,
        role: user.role
      }
    }
  }
})
```

### Modify Issuer, Audience, Subject or Expiration time

If none is given, the `BASE_URL` is used as the issuer and the audience is set to the `BASE_URL`. The expiration time is set to 15 minutes.

```ts title="auth.ts"
jwt({
  jwt: {
    issuer: "https://example.com",
    audience: "https://example.com",
    expirationTime: "1h",
    getSubject: (session) => {
      // by default the subject is the user id
      return session.user.email
    }
  }
})
```

# plugins: Last Login Method
URL: /docs/plugins/last-login-method
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/last-login-method.mdx

Track and display the last authentication method used by users

***

title: Last Login Method
description: Track and display the last authentication method used by users
----

The last login method plugin tracks the most recent authentication method used by users (email, OAuth providers, etc.). This enables you to display helpful indicators on login pages, such as "Last signed in with Google" or prioritize certain login methods based on user preferences.

## Installation

<Steps>
  <Step>
    ### Add the plugin to your auth config

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { lastLoginMethod } from "better-auth/plugins" // [!code highlight]

    export const auth = betterAuth({
        // ... other config options
        plugins: [
            lastLoginMethod() // [!code highlight]
        ]
    })
    ```
  </Step>

~~~mdx
  <Step>
    ### Add the client plugin to your auth client

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    import { lastLoginMethodClient } from "better-auth/client/plugins" // [!code highlight]

    export const authClient = createAuthClient({
        plugins: [
            lastLoginMethodClient() // [!code highlight]
        ]
    })
    ```
  </Step>
</Steps>

## Usage

Once installed, the plugin automatically tracks the last authentication method used by users. You can then retrieve and display this information in your application.

### Getting the Last Used Method

The client plugin provides several methods to work with the last login method:

```ts title="app.tsx"
import { authClient } from "@/lib/auth-client"

// Get the last used login method
const lastMethod = authClient.getLastUsedLoginMethod()
console.log(lastMethod) // "google", "email", "github", etc.

// Check if a specific method was last used
const wasGoogle = authClient.isLastUsedLoginMethod("google")

// Clear the stored method
authClient.clearLastUsedLoginMethod()
```

### UI Integration Example

Here's how to use the plugin to enhance your login page:

```tsx title="sign-in.tsx"
import { authClient } from "@/lib/auth-client"
import { Button } from "@/components/ui/button"
import { Badge } from "@/components/ui/badge"

export function SignInPage() {
    const lastMethod = authClient.getLastUsedLoginMethod()

    return (
        <div className="space-y-4">
            <h1>Sign In</h1>

            {/* Email sign in */}
            <div className="relative">
                <Button
                    onClick={() => authClient.signIn.email({...})}
                    variant={lastMethod === "email" ? "default" : "outline"}
                    className="w-full"
                >
                    Sign in with Email
                    {lastMethod === "email" && (
                        <Badge className="ml-2">Last used</Badge>
                    )}
                </Button>
            </div>

            {/* OAuth providers */}
            <div className="relative">
                <Button
                    onClick={() => authClient.signIn.social({ provider: "google" })}
                    variant={lastMethod === "google" ? "default" : "outline"}
                    className="w-full"
                >
                    Continue with Google
                    {lastMethod === "google" && (
                        <Badge className="ml-2">Last used</Badge>
                    )}
                </Button>
            </div>

            <div className="relative">
                <Button
                    onClick={() => authClient.signIn.social({ provider: "github" })}
                    variant={lastMethod === "github" ? "default" : "outline"}
                    className="w-full"
                >
                    Continue with GitHub
                    {lastMethod === "github" && (
                        <Badge className="ml-2">Last used</Badge>
                    )}
                </Button>
            </div>
        </div>
    )
}
```

## Database Persistence

By default, the last login method is stored only in cookies. For more persistent tracking and analytics, you can enable database storage.

<Steps>
  <Step>
    ### Enable database storage

    Set `storeInDatabase` to `true` in your plugin configuration:

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { lastLoginMethod } from "better-auth/plugins"

    export const auth = betterAuth({
        plugins: [
            lastLoginMethod({
                storeInDatabase: true // [!code highlight]
            })
        ]
    })
    ```
  </Step>

  <Step>
    ### Run database migration

    The plugin will automatically add a `lastLoginMethod` field to your user table. Run the migration to apply the changes:
~~~

~~~mdx
    <Tabs items={["migrate", "generate"]}>
      <Tab value="migrate">
        ```bash
        npx @better-auth/cli migrate
        ```
      </Tab>

      <Tab value="generate">
        ```bash
        npx @better-auth/cli generate
        ```
      </Tab>
    </Tabs>
  </Step>

  <Step>
    ### Access database field

    When database storage is enabled, the `lastLoginMethod` field becomes available in user objects:

    ```ts title="user-profile.tsx"
    import { auth } from "@/lib/auth"

    // Server-side access
    const session = await auth.api.getSession({ headers })
    console.log(session?.user.lastLoginMethod) // "google", "email", etc.

    // Client-side access via session
    const { data: session } = authClient.useSession()
    console.log(session?.user.lastLoginMethod)
    ```
  </Step>
</Steps>

### Database Schema

When `storeInDatabase` is enabled, the plugin adds the following field to the `user` table:

Table: `user`

<DatabaseTable
  fields={[
      { name: "lastLoginMethod", type: "string", description: "The last authentication method used by the user", isOptional: true },
  ]}
/>

### Custom Schema Configuration

You can customize the database field name:

```ts title="auth.ts"
import { betterAuth } from "better-auth"
import { lastLoginMethod } from "better-auth/plugins"

export const auth = betterAuth({
    plugins: [
        lastLoginMethod({
            storeInDatabase: true,
            schema: {
                user: {
                    lastLoginMethod: "last_auth_method" // Custom field name
                }
            }
        })
    ]
})
```

## Configuration Options

The last login method plugin accepts the following options:

### Server Options

```ts title="auth.ts"
import { betterAuth } from "better-auth"
import { lastLoginMethod } from "better-auth/plugins"

export const auth = betterAuth({
    plugins: [
        lastLoginMethod({
            // Cookie configuration
            cookieName: "better-auth.last_used_login_method", // Default: "better-auth.last_used_login_method"
            maxAge: 60 * 60 * 24 * 30, // Default: 30 days in seconds

            // Database persistence
            storeInDatabase: false, // Default: false

            // Custom method resolution
            customResolveMethod: (ctx) => {
                // Custom logic to determine the login method
                if (ctx.path === "/oauth/callback/custom-provider") {
                    return "custom-provider"
                }
                // Return null to use default resolution
                return null
            },

            // Schema customization (when storeInDatabase is true)
            schema: {
                user: {
                    lastLoginMethod: "custom_field_name"
                }
            }
        })
    ]
})
```

**cookieName**: `string`

* The name of the cookie used to store the last login method
* Default: `"better-auth.last_used_login_method"`
* **Note**: This cookie is `httpOnly: false` to allow client-side JavaScript access for UI features

**maxAge**: `number`

* Cookie expiration time in seconds
* Default: `2592000` (30 days)

**storeInDatabase**: `boolean`

* Whether to store the last login method in the database
* Default: `false`
* When enabled, adds a `lastLoginMethod` field to the user table

**customResolveMethod**: `(ctx: GenericEndpointContext) => string | null`

* Custom function to determine the login method from the request context
* Return `null` to use the default resolution logic
* Useful for custom OAuth providers or authentication flows
~~~

schema: `object`

* Customize database field names when `storeInDatabase` is enabled
* Allows mapping the `lastLoginMethod` field to a custom column name

### Client Options

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client"
import { lastLoginMethodClient } from "better-auth/client/plugins"

export const authClient = createAuthClient({
    plugins: [
        lastLoginMethodClient({
            cookieName: "better-auth.last_used_login_method" // Default: "better-auth.last_used_login_method"
        })
    ]
})
```

cookieName: `string`

* The name of the cookie to read the last login method from
* Must match the server-side `cookieName` configuration
* Default: `"better-auth.last_used_login_method"`

### Default Method Resolution

By default, the plugin tracks these authentication methods:

* **Email authentication**: `"email"`
* **OAuth providers**: Provider ID (e.g., `"google"`, `"github"`, `"discord"`)
* **OAuth2 callbacks**: Provider ID from URL path
* **Sign up methods**: Tracked the same as sign in methods

The plugin automatically detects the method from these endpoints:

* `/callback/:id` - OAuth callback with provider ID
* `/oauth2/callback/:id` - OAuth2 callback with provider ID
* `/sign-in/email` - Email sign in
* `/sign-up/email` - Email sign up

## Cross-Domain Support

The plugin automatically inherits cookie settings from Better Auth's centralized cookie system. This solves the problem where the last login method wouldn't persist across:

* **Cross-subdomain setups**: `auth.example.com` â†’ `app.example.com`
* **Cross-origin setups**: `api.company.com` â†’ `app.different.com`

When you enable `crossSubDomainCookies` or `crossOriginCookies` in your Better Auth config, the plugin will automatically use the same domain, secure, and sameSite settings as your session cookies, ensuring consistent behavior across your application.

## Advanced Examples

### Custom Provider Tracking

If you have custom OAuth providers or authentication methods, you can use the `customResolveMethod` option:

```ts title="auth.ts"
import { betterAuth } from "better-auth"
import { lastLoginMethod } from "better-auth/plugins"

export const auth = betterAuth({
    plugins: [
        lastLoginMethod({
            customResolveMethod: (ctx) => {
                // Track custom SAML provider
                if (ctx.path === "/saml/callback") {
                    return "saml"
                }

                // Track magic link authentication
                if (ctx.path === "/magic-link/verify") {
                    return "magic-link"
                }

                // Track phone authentication
                if (ctx.path === "/sign-in/phone") {
                    return "phone"
                }

                // Return null to use default logic
                return null
            }
        })
    ]
})
```

# plugins: Magic link
URL: /docs/plugins/magic-link
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/magic-link.mdx

Magic link plugin

***

title: Magic link
description: Magic link plugin

Magic link or email link is a way to authenticate users without a password. When a user enters their email, a link is sent to their email. When the user clicks on the link, they are authenticated.

## Installation

<Steps>
  <Step>
    ### Add the server Plugin

    Add the magic link plugin to your server:

    ```ts title="server.ts"
    import { betterAuth } from "better-auth";
    import { magicLink } from "better-auth/plugins";

    export const auth = betterAuth({
        plugins: [
            magicLink({
                sendMagicLink: async ({ email, token, url }, request) => {
                    // send email to user
                }
            })
        ]
    })
    ```
  </Step>

~~~
  <Step>
    ### Add the client Plugin

    Add the magic link plugin to your client:

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client";
    import { magicLinkClient } from "better-auth/client/plugins";
    export const authClient = createAuthClient({
        plugins: [
            magicLinkClient()
        ]
    });
    ```
  </Step>
</Steps>

## Usage

### Sign In with Magic Link

To sign in with a magic link, you need to call `signIn.magicLink` with the user's email address. The `sendMagicLink` function is called to send the magic link to the user's
email.

### Client Side

```ts
const { data, error } = await authClient.signIn.magicLink({
    email: user@email.com,
    name: my-name, // required
    callbackURL: /dashboard, // required
    newUserCallbackURL: /welcome, // required
    errorCallbackURL: /error, // required
});
```

### Server Side

```ts
const data = await auth.api.signInMagicLink({
    body: {
        email: user@email.com,
        name: my-name, // required
        callbackURL: /dashboard, // required
        newUserCallbackURL: /welcome, // required
        errorCallbackURL: /error, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type signInMagicLink = {
    /**
     * Email address to send the magic link.
     */
    email: string = "user@email.com"
    /**
     * User display name. Only used if the user is registering for the first time.
     */
    name?: string = "my-name"
    /**
     * URL to redirect after magic link verification.
     */
    callbackURL?: string = "/dashboard"
    /**
     * URL to redirect after new user signup
     */
    newUserCallbackURL?: string = "/welcome"
    /**
     * URL to redirect if an error happen on verification
     * If only callbackURL is provided but without an `errorCallbackURL` then they will be
     * redirected to the callbackURL with an `error` query parameter.
     */
    errorCallbackURL?: string = "/error"

}
```

<Callout>
  If the user has not signed up, unless `disableSignUp` is set to `true`, the user will be signed up automatically.
</Callout>

### Verify Magic Link

When you send the URL generated by the `sendMagicLink` function to a user, clicking the link will authenticate them and redirect them to the `callbackURL` specified in the
`signIn.magicLink` function. If an error occurs, the user will be redirected to the `callbackURL` with an error query parameter.

<Callout type="warn">
  If no `callbackURL` is provided, the user will be redirected to the root URL.
</Callout>

If you want to handle the verification manually, (e.g, if you send the user a different URL), you can use the `verify` function.

### Client Side

```ts
const { data, error } = await authClient.magicLink.verify({
    token: 123456,
    callbackURL: /dashboard, // required
});
```

### Server Side

```ts
const data = await auth.api.magicLinkVerify({
    query: {
        token: 123456,
        callbackURL: /dashboard, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type magicLinkVerify = {
    /**
     * Verification token.
     */
    token: string = "123456"
    /**
     * URL to redirect after magic link verification, if not provided will return the session.
     */
    callbackURL?: string = "/dashboard"
~~~

}

## Configuration Options

sendMagicLink: The `sendMagicLink` function is called when a user requests a magic link. It takes an object with the following properties:

* `email`: The email address of the user.
* `url`: The URL to be sent to the user. This URL contains the token.
* `token`: The token if you want to send the token with custom URL.

and a `request` object as the second parameter.

expiresIn: specifies the time in seconds after which the magic link will expire. The default value is `300` seconds (5 minutes).

disableSignUp: If set to `true`, the user will not be able to sign up using the magic link. The default value is `false`.

generateToken: The `generateToken` function is called to generate a token which is used to uniquely identify the user. The default value is a random string. There is one parameter:

* `email`: The email address of the user.

<Callout type="warn">
  When using `generateToken`, ensure that the returned string is hard to guess
  because it is used to verify who someone actually is in a confidential way. By
  default, we return a long and cryptographically secure string.
</Callout>

storeToken: The `storeToken` function is called to store the magic link token in the database. The default value is `"plain"`.

The `storeToken` function can be one of the following:

* `"plain"`: The token is stored in plain text.
* `"hashed"`: The token is hashed using the default hasher.
* `{ type: "custom-hasher", hash: (token: string) => Promise<string> }`: The token is hashed using a custom hasher.

# plugins: MCP
URL: /docs/plugins/mcp
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/mcp.mdx

MCP provider plugin for Better Auth

***

title: MCP
description: MCP provider plugin for Better Auth

`OAuth` `MCP`

The **MCP** plugin lets your app act as an OAuth provider for MCP clients. It handles authentication and makes it easy to issue and manage access tokens for MCP applications.

## Installation

<Steps>
  <Step>
    ### Add the Plugin

    Add the MCP plugin to your auth configuration and specify the login page path.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth";
    import { mcp } from "better-auth/plugins";

    export const auth = betterAuth({
        plugins: [
            mcp({
                loginPage: "/sign-in" // path to your login page
            })
        ]
    });
    ```

    <Callout>
      This doesn't have a client plugin, so you don't need to make any changes to your authClient.
    </Callout>
  </Step>

  <Step>
    ### Generate Schema

    Run the migration or generate the schema to add the necessary fields and tables to the database.

    <Tabs items={["migrate", "generate"]}>
      <Tab value="migrate">
        ```bash
        npx @better-auth/cli migrate
        ```
      </Tab>

      <Tab value="generate">
        ```bash
        npx @better-auth/cli generate
        ```
      </Tab>
    </Tabs>

    The MCP plugin uses the same schema as the OIDC Provider plugin. See the [OIDC Provider Schema](#schema) section for details.
  </Step>
</Steps>

## Usage

### OAuth Discovery Metadata

Better Auth already handles the `/api/auth/.well-known/oauth-authorization-server` route automatically but some client may fail to parse the `WWW-Authenticate` header and default to `/.well-known/oauth-authorization-server` (this can happen, for example, if your CORS configuration doesn't expose the `WWW-Authenticate`). For this reason it's better to add a route to expose OAuth metadata for MCP clients:

```ts title=".well-known/oauth-authorization-server/route.ts"
import { oAuthDiscoveryMetadata } from "better-auth/plugins";
import { auth } from "../../../lib/auth";

export const GET = oAuthDiscoveryMetadata(auth);
```

### OAuth Protected Resource Metadata

Better Auth already handles the `/api/auth/.well-known/oauth-protected-resource` route automatically but some client may fail to parse the `WWW-Authenticate` header and default to `/.well-known/oauth-protected-resource` (this can happen, for example, if your CORS configuration doesn't expose the `WWW-Authenticate`). For this reason it's better to add a route to expose OAuth metadata for MCP clients:

```ts title="/.well-known/oauth-protected-resource/route.ts"
import { oAuthProtectedResourceMetadata } from "better-auth/plugins";
import { auth } from "@/lib/auth";

export const GET = oAuthProtectedResourceMetadata(auth);
```

### MCP Session Handling

You can use the helper function `withMcpAuth` to get the session and handle unauthenticated calls automatically.

```ts title="api/[transport]/route.ts"
import { auth } from "@/lib/auth";
import { createMcpHandler } from "@vercel/mcp-adapter";
import { withMcpAuth } from "better-auth/plugins";
import { z } from "zod";

const handler = withMcpAuth(auth, (req, session) => {
    // session contains the access token record with scopes and user ID
    return createMcpHandler(
        (server) => {
            server.tool(
                "echo",
                "Echo a message",
                { message: z.string() },
                async ({ message }) => {
                    return {
                        content: [{ type: "text", text: `Tool echo: ${message}` }],
                    };
                },
            );
        },
        {
            capabilities: {
                tools: {
                    echo: {
                        description: "Echo a message",
                    },
                },
            },
        },
        {
            redisUrl: process.env.REDIS_URL,
            basePath: "/api",
            verboseLogs: true,
            maxDuration: 60,
        },
    )(req);
});

export { handler as GET, handler as POST, handler as DELETE };
```

You can also use `auth.api.getMcpSession` to get the session using the access token sent from the MCP client:

```ts title="api/[transport]/route.ts"
import { auth } from "@/lib/auth";
import { createMcpHandler } from "@vercel/mcp-adapter";
import { z } from "zod";

const handler = async (req: Request) => {
     // session contains the access token record with scopes and user ID
    const session = await auth.api.getMcpSession({
        headers: req.headers
    })
    if(!session){
        //this is important and you must return 401
        return new Response(null, {
            status: 401
        })
    }
    return createMcpHandler(
        (server) => {
            server.tool(
                "echo",
                "Echo a message",
                { message: z.string() },
                async ({ message }) => {
                    return {
                        content: [{ type: "text", text: `Tool echo: ${message}` }],
                    };
                },
            );
        },
        {
            capabilities: {
                tools: {
                    echo: {
                        description: "Echo a message",
                    },
                },
            },
        },
        {
            redisUrl: process.env.REDIS_URL,
            basePath: "/api",
            verboseLogs: true,
            maxDuration: 60,
        },
    )(req);
}

export { handler as GET, handler as POST, handler as DELETE };
```

## Configuration

The MCP plugin accepts the following configuration options:

```
<TypeTable
  type={{
  loginPage: {
      description: "Path to the login page where users will be redirected for authentication",
      type: "string",
      required: true
  },
  resource: {
      description: "The resource that should be returned by the protected resource metadata endpoint",
      type: "string",
      required: false
  },
  oidcConfig: {
      description: "Optional OIDC configuration options",
      type: "object",
      required: false
  }
}}
/>
```

### OIDC Configuration

The plugin supports additional OIDC configuration options through the `oidcConfig` parameter:

~~~
<TypeTable
  type={{
  codeExpiresIn: {
      description: "Expiration time for authorization codes in seconds",
      type: "number",
      default: 600
  },
  accessTokenExpiresIn: {
      description: "Expiration time for access tokens in seconds",
      type: "number",
      default: 3600
  },
  refreshTokenExpiresIn: {
      description: "Expiration time for refresh tokens in seconds",
      type: "number",
      default: 604800
  },
  defaultScope: {
      description: "Default scope for OAuth requests",
      type: "string",
      default: "openid"
  },
  scopes: {
      description: "Additional scopes to support",
      type: "string[]",
      default: '["openid", "profile", "email", "offline_access"]'
  }
}}
/>

## Schema

The MCP plugin uses the same schema as the OIDC Provider plugin. See the [OIDC Provider Schema](#schema) section for details.



# plugins: Multi Session
URL: /docs/plugins/multi-session
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/multi-session.mdx

Learn how to use multi-session plugin in Better Auth.

***

title: Multi Session
description: Learn how to use multi-session plugin in Better Auth.
------------------------------------------------------------------

The multi-session plugin allows users to maintain multiple active sessions across different accounts in the same browser. This plugin is useful for applications that require
users to switch between multiple accounts without logging out.

## Installation

<Steps>
  <Step>
    ### Add the plugin to your **auth** config

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { multiSession } from "better-auth/plugins"

    export const auth = betterAuth({
        plugins: [ // [!code highlight]
            multiSession(), // [!code highlight]
        ] // [!code highlight]
    })
    ```
  </Step>

  <Step>
    ### Add the client Plugin

    Add the client plugin and Specify where the user should be redirected if they need to verify 2nd factor

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    import { multiSessionClient } from "better-auth/client/plugins"

    export const authClient = createAuthClient({
        plugins: [
            multiSessionClient()
        ]
    })
    ```
  </Step>
</Steps>

## Usage

Whenever a user logs in, the plugin will add additional cookie to the browser. This cookie will be used to maintain multiple sessions across different accounts.

### List all device sessions

To list all active sessions for the current user, you can call the `listDeviceSessions` method.

### Client Side

```ts
const { data, error } = await authClient.multiSession.listDeviceSessions({});
```

### Server Side

```ts
const data = await auth.api.listDeviceSessions({

    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type listDeviceSessions = {

}
```

### Set active session

To set the active session, you can call the `setActive` method.

### Client Side

```ts
const { data, error } = await authClient.multiSession.setActive({
    sessionToken: some-session-token,
});
```
~~~

~~~
### Server Side

```ts
const data = await auth.api.setActiveSession({
    body: {
        sessionToken: some-session-token,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type setActiveSession = {
    /**
     * The session token to set as active.
     */
    sessionToken: string = "some-session-token"

}
```

### Revoke a session

To revoke a session, you can call the `revoke` method.

### Client Side

```ts
const { data, error } = await authClient.multiSession.revoke({
    sessionToken: some-session-token,
});
```

### Server Side

```ts
const data = await auth.api.revokeDeviceSession({
    body: {
        sessionToken: some-session-token,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type revokeDeviceSession = {
    /**
     * The session token to revoke.
     */
    sessionToken: string = "some-session-token"

}
```

### Signout and Revoke all sessions

When a user logs out, the plugin will revoke all active sessions for the user. You can do this by calling the existing `signOut` method, which handles revoking all sessions
automatically.

### Max Sessions

You can specify the maximum number of sessions a user can have by passing the `maximumSessions` option to the plugin. By default, the plugin allows 5 sessions per device.

```ts title="auth.ts"
import { betterAuth } from "better-auth"

export const auth = betterAuth({
    plugins: [
        multiSession({
            maximumSessions: 3
        })
    ]
})
```



# plugins: OAuth Proxy
URL: /docs/plugins/oauth-proxy
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/oauth-proxy.mdx

OAuth Proxy plugin for Better Auth

***

title: OAuth Proxy
description: OAuth Proxy plugin for Better Auth
-----------------------------------------------

A proxy plugin, that allows you to proxy OAuth requests. Useful for development and preview deployments where the redirect URL can't be known in advance to add to the OAuth
provider.

## Installation

<Steps>
  <Step>
    ### Add the plugin to your **auth** config

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { oAuthProxy } from "better-auth/plugins"

    export const auth = betterAuth({
        plugins: [ // [!code highlight]
            oAuthProxy({ // [!code highlight]
                productionURL: "https://my-main-app.com", // Optional - if the URL isn't inferred correctly // [!code highlight]
                currentURL: "http://localhost:3000", // Optional - if the URL isn't inferred correctly // [!code highlight]
            }), // [!code highlight]
        ] // [!code highlight]
    })
    ```
  </Step>

  <Step>
    ### Add redirect URL to your OAuth provider

    For the proxy server to work properly, youâ€™ll need to pass the redirect URL of your main production app registered with the OAuth provider in your social provider config.
This needs to be done for each social provider you want to proxy requests for.
~~~

    ```ts
    export const auth = betterAuth({
       plugins: [
           oAuthProxy(),
       ],
       socialProviders: {
            github: {
                clientId: "your-client-id",
                clientSecret: "your-client-secret",
                redirectURI: "https://my-main-app.com/api/auth/callback/github" // [!code highlight]
            }
       }
    })
    ```
  </Step>
</Steps>

## How it works

The plugin adds an endpoint to your server that proxies OAuth requests. When you initiate a social sign-in, it sets the redirect URL to this proxy endpoint. After the OAuth provider redirects back to your server, the plugin then forwards the user to the original callback URL.

```ts
await authClient.signIn.social({
    provider: "github",
    callbackURL: "/dashboard" // the plugin will override this to something like "http://localhost:3000/api/auth/oauth-proxy?callbackURL=/dashboard"
})
```

When the OAuth provider returns the user to your server, the plugin automatically redirects them to the intended callback URL.

To share cookies between the proxy server and your main server it uses URL query parameters to pass the cookies encrypted in the URL. This is secure as the cookies are encrypted and can only be decrypted by the server.

<Callout type="warn">
  This plugin requires skipping the state cookie check. This has security implications and should only be used in dev or staging environments. If `baseURL` and `productionURL` are the same, the plugin will not proxy the request.
</Callout>

## Options

currentURL: The application's current URL is automatically determined by the plugin. It first checks for the request URL if invoked by a client, then it checks the base URL from popular hosting providers, and finally falls back to the `baseURL` in your auth config. If the URL isnâ€™t inferred correctly, you can specify it manually here.

productionURL: If this value matches the `baseURL` in your auth config, requests will not be proxied. Defaults to the `BETTER_AUTH_URL` environment variable.

# plugins: OIDC Provider
URL: /docs/plugins/oidc-provider
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/oidc-provider.mdx

Open ID Connect plugin for Better Auth that allows you to have your own OIDC provider.

***

title: OIDC Provider
description: Open ID Connect plugin for Better Auth that allows you to have your own OIDC provider.

The **OIDC Provider Plugin** enables you to build and manage your own OpenID Connect (OIDC) provider, granting full control over user authentication without relying on third-party services like Okta or Azure AD. It also allows other services to authenticate users through your OIDC provider.

Key Features:

* **Client Registration**: Register clients to authenticate with your OIDC provider.
* **Dynamic Client Registration**: Allow clients to register dynamically.
* **Trusted Clients**: Configure hard-coded trusted clients with optional consent bypass.
* **Authorization Code Flow**: Support the Authorization Code Flow.
* **Public Clients**: Support public clients for SPA, mobile apps, CLI tools, etc.
* **JWKS Endpoint**: Publish a JWKS endpoint to allow clients to verify tokens. (Not fully implemented)
* **Refresh Tokens**: Issue refresh tokens and handle access token renewal using the `refresh_token` grant.
* **OAuth Consent**: Implement OAuth consent screens for user authorization, with an option to bypass consent for trusted applications.
* **UserInfo Endpoint**: Provide a UserInfo endpoint for clients to retrieve user details.

<Callout type="warn">
  This plugin is in active development and may not be suitable for production use. Please report any issues or bugs on [GitHub](https://github.com/better-auth/better-auth).
</Callout>

## Installation

<Steps>
  <Step>
    ### Mount the Plugin

    Add the OIDC plugin to your auth config. See [OIDC Configuration](#oidc-configuration) on how to configure the plugin.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth";
    import { oidcProvider } from "better-auth/plugins";

    const auth = betterAuth({
        plugins: [oidcProvider({
            loginPage: "/sign-in", // path to the login page
            // ...other options
        })]
    })
    ```
  </Step>

  <Step>
    ### Migrate the Database

    Run the migration or generate the schema to add the necessary fields and tables to the database.

    <Tabs items={["migrate", "generate"]}>
      <Tab value="migrate">
        ```bash
        npx @better-auth/cli migrate
        ```
      </Tab>

      <Tab value="generate">
        ```bash
        npx @better-auth/cli generate
        ```
      </Tab>
    </Tabs>

    See the [Schema](#schema) section to add the fields manually.
  </Step>

  <Step>
    ### Add the Client Plugin

    Add the OIDC client plugin to your auth client config.

    ```ts
    import { createAuthClient } from "better-auth/client";
    import { oidcClient } from "better-auth/client/plugins"
    const authClient = createAuthClient({
        plugins: [oidcClient({
            // Your OIDC configuration
        })]
    })
    ```
  </Step>
</Steps>

## Usage

Once installed, you can utilize the OIDC Provider to manage authentication flows within your application.

### Register a New Client

To register a new OIDC client, use the `oauth2.register` method.

#### Simple Example

```ts
const application = await client.oauth2.register({
    client_name: "My Client",
    redirect_uris: ["https://client.example.com/callback"],
});
```

#### Full Method

### Client Side

```ts
const { data, error } = await authClient.oauth2.register({
    redirect_uris, // client.example.com/callback"]
    token_endpoint_auth_method: client_secret_basic, // required
    grant_types, // required
    response_types, // required
    client_name: My App, // required
    client_uri: https://client.example.com, // required
    logo_uri: https://client.example.com/logo.png, // required
    scope: profile email, // required
    contacts, // required
    tos_uri: https://client.example.com/tos, // required
    policy_uri: https://client.example.com/policy, // required
    jwks_uri: https://client.example.com/jwks, // required
    jwks, // required
});
```

### Server Side

```ts
const data = await auth.api.registerOAuthApplication({
    body: {
        redirect_uris, // client.example.com/callback"]
        token_endpoint_auth_method: client_secret_basic, // required
        grant_types, // required
        response_types, // required
        client_name: My App, // required
        client_uri: https://client.example.com, // required
        logo_uri: https://client.example.com/logo.png, // required
        scope: profile email, // required
        contacts, // required
        tos_uri: https://client.example.com/tos, // required
        policy_uri: https://client.example.com/policy, // required
        jwks_uri: https://client.example.com/jwks, // required
        jwks, // required
    }
});
```

### Type Definition

```ts
type registerOAuthApplication = {
    /**
     * A list of redirect URIs.
     */
    redirect_uris: string[] = ["https://client.example.com/callback"]
    /**
     * The authentication method for the token endpoint.
     */
    token_endpoint_auth_method?: "none" | "client_secret_basic" | "client_secret_post" = "client_secret_basic"
    /**
     * The grant types supported by the application.
     */
    grant_types?: ("authorization_code" | "implicit" | "password" | "client_credentials" | "refresh_token" | "urn:ietf:params:oauth:grant-type:jwt-bearer" | "urn:ietf:params:oauth:grant-type:saml2-bearer")[] = ["authorization_code"]
    /**
     * The response types supported by the application.
     */
    response_types?: ("code" | "token")[] = ["code"]
    /**
     * The name of the application.
     */
    client_name?: string = "My App"
    /**
     * The URI of the application.
     */
    client_uri?: string = "https://client.example.com"
    /**
     * The URI of the application logo.
     */
    logo_uri?: string = "https://client.example.com/logo.png"
    /**
     * The scopes supported by the application. Separated by spaces.
     */
    scope?: string = "profile email"
    /**
     * The contact information for the application.
     */
    contacts?: string[] = ["admin@example.com"]
    /**
     * The URI of the application terms of service.
     */
    tos_uri?: string = "https://client.example.com/tos"
    /**
     * The URI of the application privacy policy.
     */
    policy_uri?: string = "https://client.example.com/policy"
    /**
     * The URI of the application JWKS.
     */
    jwks_uri?: string = "https://client.example.com/jwks"
    /**

     * The JWKS of the application.
     */
    jwks?: Record<string, any> = {"keys": [{"kty": "RSA", "alg": "RS256", "use": "sig", "n": "...", "e": "..."
}
```

<Callout>
  This endpoint supports [RFC7591](https://datatracker.ietf.org/doc/html/rfc7591) compliant client registration.
</Callout>

Once the application is created, you will receive a `client_id` and `client_secret` that you can display to the user.

### Trusted Clients

For first-party applications and internal services, you can configure trusted clients directly in your OIDC provider configuration. Trusted clients bypass database lookups for better performance and can optionally skip consent screens for improved user experience.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { oidcProvider } from "better-auth/plugins";

const auth = betterAuth({
    plugins: [
      oidcProvider({
        loginPage: "/sign-in",
        trustedClients: [
            {
                clientId: "internal-dashboard",
                clientSecret: "secure-secret-here",
                name: "Internal Dashboard",
                type: "web",
                redirectURLs: ["https://dashboard.company.com/auth/callback"],
                disabled: false,
                skipConsent: true, // Skip consent for this trusted client
                metadata: { internal: true }
            },
            {
                clientId: "mobile-app",
                clientSecret: "mobile-secret",
                name: "Company Mobile App",
                type: "native",
                redirectURLs: ["com.company.app://auth"],
                disabled: false,
                skipConsent: false, // Still require consent if needed
                metadata: {}
            }
        ]
    })]
})
```

### UserInfo Endpoint

The OIDC Provider includes a UserInfo endpoint that allows clients to retrieve information about the authenticated user. This endpoint is available at `/oauth2/userinfo` and requires a valid access token.

<Endpoint path="/oauth2/userinfo" method="GET" />

```ts title="client-app.ts"
// Example of how a client would use the UserInfo endpoint
const response = await fetch('https://your-domain.com/api/auth/oauth2/userinfo', {
  headers: {
    'Authorization': 'Bearer ACCESS_TOKEN'
  }
});

const userInfo = await response.json();
// userInfo contains user details based on the scopes granted
```

The UserInfo endpoint returns different claims based on the scopes that were granted during authorization:

* With `openid` scope: Returns the user's ID (`sub` claim)
* With `profile` scope: Returns name, picture, given_name, family_name
* With `email` scope: Returns email and email_verified

The `getAdditionalUserInfoClaim` function receives the user object, requested scopes array, and the client, allowing you to conditionally include claims based on the scopes granted during authorization. These additional claims will be included in both the UserInfo endpoint response and the ID token.

### Consent Screen

When a user is redirected to the OIDC provider for authentication, they may be prompted to authorize the application to access their data. This is known as the consent screen. By default, Better Auth will display a sample consent screen. You can customize the consent screen by providing a `consentPage` option during initialization.

Note: Trusted clients with `skipConsent: true` will bypass the consent screen entirely, providing a seamless experience for first-party applications.

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
    plugins: [oidcProvider({
        consentPage: "/path/to/consent/page"
    })]
})
```

The plugin will redirect the user to the specified path with `consent_code`, `client_id` and `scope` query parameters. You can use this information to display a custom consent screen. Once the user consents, you can call `oauth2.consent` to complete the authorization.

<Endpoint path="/oauth2/consent" method="POST" />

The consent endpoint supports two methods for passing the consent code:

Method 1: URL Parameter

```ts title="consent-page.ts"
// Get the consent code from the URL
const params = new URLSearchParams(window.location.search);

// Submit consent with the code in the request body
const consentCode = params.get('consent_code');
if (!consentCode) {
        throw new Error('Consent code not found in URL parameters');
}

const res = await client.oauth2.consent({
        accept: true, // or false to deny
        consent_code: consentCode,
});
```

Method 2: Cookie-Based

```ts title="consent-page.ts"
// The consent code is automatically stored in a signed cookie
// Just submit the consent decision
const res = await client.oauth2.consent({
        accept: true, // or false to deny
        // consent_code not needed when using cookie-based flow
});
```

Both methods are fully supported. The URL parameter method works well with mobile apps and third-party contexts, while the cookie-based method provides a simpler implementation for web applications.

### Handling Login

When a user is redirected to the OIDC provider for authentication, if they are not already logged in, they will be redirected to the login page. You can customize the login page by providing a `loginPage` option during initialization.

```ts title="auth.ts"
import { betterAuth } from "better-auth";

export const auth = betterAuth({
    plugins: [oidcProvider({
        loginPage: "/sign-in"
    })]
})
```

You don't need to handle anything from your side; when a new session is created, the plugin will handle continuing the authorization flow.

## Configuration

### OIDC Metadata

Customize the OIDC metadata by providing a configuration object during initialization.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { oidcProvider } from "better-auth/plugins";

export const auth = betterAuth({
    plugins: [oidcProvider({
        metadata: {
            issuer: "https://your-domain.com",
            authorization_endpoint: "/custom/oauth2/authorize",
            token_endpoint: "/custom/oauth2/token",
            // ...other custom metadata
        }
    })]
})
```

### JWKS Endpoint

The OIDC Provider plugin can integrate with the JWT plugin to provide asymmetric key signing for ID tokens verifiable at a JWKS endpoint.

To make your plugin OIDC compliant, you **MUST** disable the `/token` endpoint, the OAuth equivalent is located at `/oauth2/token` instead.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { oidcProvider } from "better-auth/plugins";
import { jwt } from "better-auth/plugins";

export const auth = betterAuth({
    disabledPaths: [
        "/token",
    ],
    plugins: [
        jwt(), // Make sure to add the JWT plugin
        oidcProvider({
            useJWTPlugin: true, // Enable JWT plugin integration
            loginPage: "/sign-in",
            // ... other options
        })
    ]
})
```

<Callout type="info">
  When `useJWTPlugin: false` (default), ID tokens are signed with the application secret.
</Callout>

### Dynamic Client Registration

If you want to allow clients to register dynamically, you can enable this feature by setting the `allowDynamicClientRegistration` option to `true`.

```ts title="auth.ts"
const auth = betterAuth({
    plugins: [oidcProvider({
        allowDynamicClientRegistration: true,
    })]
})
```

This will allow clients to register using the `/register` endpoint to be publicly available.

## Schema

The OIDC Provider plugin adds the following tables to the database:

### OAuth Application

Table Name: `oauthApplication`

```
<DatabaseTable
  fields={[
 {
    name: "id",
    type: "string",
    description: "Database ID of the OAuth client",
    isPrimaryKey: true
 },
  {
    name: "clientId",
    type: "string",
    description: "Unique identifier for each OAuth client",
    isPrimaryKey: true
  },
  {
    name: "clientSecret",
    type: "string",
    description: "Secret key for the OAuth client. Optional for public clients using PKCE.",
    isOptional: true
  },
  {
    name: "name",
    type: "string",
    description: "Name of the OAuth client",
    isRequired: true
  },
  {
    name: "redirectURLs",
    type: "string",
    description: "Comma-separated list of redirect URLs",
    isRequired: true
  },
  {
```

```
    name: "metadata",
    type: "string",
    description: "Additional metadata for the OAuth client",
    isOptional: true
  },
  {
    name: "type",
    type: "string",
    description: "Type of OAuth client (e.g., web, mobile)",
    isRequired: true
  },
  {
    name: "disabled",
    type: "boolean",
    description: "Indicates if the client is disabled",
    isRequired: true
  },
  {
    name: "userId",
    type: "string",
    description: "ID of the user who owns the client. (optional)",
    isOptional: true,
    references: { model: "user", field: "id" }
  },
  {
    name: "createdAt",
    type: "Date",
    description: "Timestamp of when the OAuth client was created"
  },
 {
    name: "updatedAt",
    type: "Date",
    description: "Timestamp of when the OAuth client was last updated"
 }
]}
/>

### OAuth Access Token

Table Name: `oauthAccessToken`

<DatabaseTable
  fields={[
  {
    name: "id",
    type: "string",
    description: "Database ID of the access token",
    isPrimaryKey: true
 },
  {
    name: "accessToken",
    type: "string",
    description: "Access token issued to the client",
  },
  {
    name: "refreshToken",
    type: "string",
    description: "Refresh token issued to the client",
    isRequired: true
  },
  {
    name: "accessTokenExpiresAt",
    type: "Date",
    description: "Expiration date of the access token",
    isRequired: true
  },
  {
    name: "refreshTokenExpiresAt",
    type: "Date",
    description: "Expiration date of the refresh token",
    isRequired: true
  },
  {
    name: "clientId",
    type: "string",
    description: "ID of the OAuth client",
    isForeignKey: true,
    references: { model: "oauthApplication", field: "clientId" }
  },
  {
    name: "userId",
    type: "string",
    description: "ID of the user associated with the token",
    isForeignKey: true,
    references: { model: "user", field: "id" }
  },
  {
    name: "scopes",
    type: "string",
    description: "Comma-separated list of scopes granted",
    isRequired: true
  },
  {
    name: "createdAt",
    type: "Date",
    description: "Timestamp of when the access token was created"
  },
  {
    name: "updatedAt",
    type: "Date",
    description: "Timestamp of when the access token was last updated"
  }
]}
/>

### OAuth Consent

Table Name: `oauthConsent`
```

<DatabaseTable
  fields={[
  {
    name: "id",
    type: "string",
    description: "Database ID of the consent",
    isPrimaryKey: true
  },
  {
    name: "userId",
    type: "string",
    description: "ID of the user who gave consent",
    isForeignKey: true,
    references: { model: "user", field: "id" }
  },
  {
    name: "clientId",
    type: "string",
    description: "ID of the OAuth client",
    isForeignKey: true,
    references: { model: "oauthApplication", field: "clientId" }
  },
  {
    name: "scopes",
    type: "string",
    description: "Comma-separated list of scopes consented to",
    isRequired: true
  },
  {
    name: "consentGiven",
    type: "boolean",
    description: "Indicates if consent was given",
    isRequired: true
  },
  {
    name: "createdAt",
    type: "Date",
    description: "Timestamp of when the consent was given"
  },
  {
    name: "updatedAt",
    type: "Date",
    description: "Timestamp of when the consent was last updated"
  }
]}
/>

## Options

allowDynamicClientRegistration: `boolean` - Enable or disable dynamic client registration.

metadata: `OIDCMetadata` - Customize the OIDC provider metadata.

loginPage: `string` - Path to the custom login page.

consentPage: `string` - Path to the custom consent page.

trustedClients: `(Client & { skipConsent?: boolean })[]` - Array of trusted clients that are configured directly in the provider options. These clients bypass database lookups and can optionally skip consent screens.

getAdditionalUserInfoClaim: `(user: User, scopes: string[], client: Client) => Record<string, any>` - Function to get additional user info claims.

useJWTPlugin: `boolean` - When `true`, ID tokens are signed using the JWT plugin's asymmetric keys. When `false` (default), ID tokens are signed with HMAC-SHA256 using the application secret.

schema: `AuthPluginSchema` - Customize the OIDC provider schema.

# plugins: One Tap
URL: /docs/plugins/one-tap
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/one-tap.mdx

One Tap plugin for Better Auth

***

title: One Tap
description: One Tap plugin for Better Auth

The One Tap plugin allows users to log in with a single tap using Google's One Tap API. The plugin provides a simple way to integrate One Tap into your application, handling the client-side and server-side logic for you.

## Installation

### Add the Server Plugin

Add the One Tap plugin to your auth configuration:

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { oneTap } from "better-auth/plugins"; // [!code highlight]

export const auth = betterAuth({
    plugins: [ // [!code highlight]
        oneTap(), // Add the One Tap server plugin  // [!code highlight]
    ] // [!code highlight]
});
```

### Add the Client Plugin

Add the client plugin and specify where the user should be redirected after sign-in or if additional verification (like 2FA) is needed.

```ts
import { createAuthClient } from "better-auth/client";
import { oneTapClient } from "better-auth/client/plugins";

export const authClient = createAuthClient({
  plugins: [
    oneTapClient({
      clientId: "YOUR_CLIENT_ID",
      // Optional client configuration:
      autoSelect: false,
      cancelOnTapOutside: true,
      context: "signin",
      additionalOptions: {
        // Any extra options for the Google initialize method
      },
      // Configure prompt behavior and exponential backoff:
      promptOptions: {
        baseDelay: 1000,   // Base delay in ms (default: 1000)
        maxAttempts: 5     // Maximum number of attempts before triggering onPromptNotification (default: 5)
      }
    })
  ]
});
```

### Usage

To display the One Tap popup, simply call the oneTap method on your auth client:

```ts
await authClient.oneTap();
```

### Customizing Redirect Behavior

By default, after a successful login the plugin will hard redirect the user to `/`. You can customize this behavior as follows:

#### Avoiding a Hard Redirect

Pass fetchOptions with an onSuccess callback to handle the login response without a page reload:

```ts
await authClient.oneTap({
  fetchOptions: {
    onSuccess: () => {
      // For example, use a router to navigate without a full reload:
      router.push("/dashboard");
    }
  }
});
```

#### Specifying a Custom Callback URL

To perform a hard redirect to a different page after login, use the callbackURL option:

```ts
await authClient.oneTap({
  callbackURL: "/dashboard"
});
```

#### Handling Prompt Dismissals with Exponential Backoff

If the user dismisses or skips the prompt, the plugin will retry showing the One Tap prompt using exponential backoff based on your configured promptOptions.

If the maximum number of attempts is reached without a successful sign-in, you can use the onPromptNotification callback to be notifiedâ€”allowing you to render an alternative UI (e.g., a traditional Google Sign-In button) so users can restart the process manually:

```ts
await authClient.oneTap({
  onPromptNotification: (notification) => {
    console.warn("Prompt was dismissed or skipped. Consider displaying an alternative sign-in option.", notification);
    // Render your alternative UI here
  }
});
```

### Client Options

* **clientId**: The client ID for your Google One Tap API.
* **autoSelect**: Automatically select the account if the user is already signed in. Default is false.
* **context**: The context in which the One Tap API should be used (e.g., "signin"). Default is "signin".
* **cancelOnTapOutside**: Cancel the One Tap popup when the user taps outside it. Default is true.
* additionalOptions: Extra options to pass to Google's initialize method as per the [Google Identity Services docs](https://developers.google.com/identity/gsi/web/reference/js-reference#google.accounts.id.prompt).
* **promptOptions**: Configuration for the prompt behavior and exponential backoff:
* **baseDelay**: Base delay in milliseconds for retries. Default is 1000.
* **maxAttempts**: Maximum number of prompt attempts before invoking the onPromptNotification callback. Default is 5.

### Server Options

* **disableSignUp**: Disable the sign-up option, allowing only existing users to sign in. Default is `false`.
* **ClientId**: Optionally, pass a client ID here if it is not provided in your social provider configuration.

# plugins: One-Time Token Plugin
URL: /docs/plugins/one-time-token
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/one-time-token.mdx

Generate and verify single-use token

***

title: One-Time Token Plugin
description: Generate and verify single-use token

The One-Time Token (OTT) plugin provides functionality to generate and verify secure, single-use session tokens. These are commonly used for across domains authentication.

## Installation

<Steps>
  <Step>
    ### Add the plugin to your auth config

    To use the One-Time Token plugin, add it to your auth config.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth";
    import { oneTimeToken } from "better-auth/plugins/one-time-token";

    export const auth = betterAuth({
        plugins: [
          oneTimeToken()
        ]
        // ... other auth config
    });
    ```
  </Step>

  <Step>
    ### Add the client plugin

    Next, include the one-time-token client plugin in your authentication client instance.

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    import { oneTimeTokenClient } from "better-auth/client/plugins"

    export const authClient = createAuthClient({
        plugins: [
            oneTimeTokenClient()
        ]
    })
    ```
  </Step>
</Steps>

## Usage

### 1. Generate a Token

Generate a token using `auth.api.generateOneTimeToken` or `authClient.oneTimeToken.generate`

### Client Side

```ts
const { data, error } = await authClient.oneTimeToken.generate({});
```

### Server Side

```ts
const data = await auth.api.generateOneTimeToken({

    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type generateOneTimeToken = {

}
```

This will return a `token` that is attached to the current session which can be used to verify the one-time token. By default, the token will expire in 3 minutes.

### 2. Verify the Token

When the user clicks the link or submits the token, use the `auth.api.verifyOneTimeToken` or `authClient.oneTimeToken.verify` method in another API route to validate it.

### Client Side

```ts
const { data, error } = await authClient.oneTimeToken.verify({
    token: some-token,
});
```

### Server Side

```ts
const data = await auth.api.verifyOneTimeToken({
    body: {
        token: some-token,
    }
});
```

### Type Definition

```ts
type verifyOneTimeToken = {
    /**
     * The token to verify.
     */
    token: string = "some-token"

}
```

This will return the session that was attached to the token.

## Options

These options can be configured when adding the `oneTimeToken` plugin:

* **`disableClientRequest`** (boolean): Optional. If `true`, the token will only be generated on the server side. Default: `false`.
* **`expiresIn`** (number): Optional. The duration for which the token is valid in minutes. Default: `3`.

```ts
oneTimeToken({
    expiresIn: 10 // 10 minutes
})
```

* **`generateToken`**: A custom token generator function that takes `session` object and a `ctx` as paramters.

* **`storeToken`**: Optional. This option allows you to configure how the token is stored in your database.

  * **`plain`**: The token is stored in plain text. (Default)
  * **`hashed`**: The token is hashed using the default hasher.
  * **`custom-hasher`**: A custom hasher function that takes a token and returns a hashed token.

<Callout type="info">
  Note: It will not affect the token that's sent, it will only affect the token stored in your database.
</Callout>

Examples:

```ts title="No hashing (default)"
oneTimeToken({
    storeToken: "plain"
})
```

```ts title="built-in hasher"
oneTimeToken({
    storeToken: "hashed"
})
```

```ts title="custom hasher"
oneTimeToken({
    storeToken: {
        type: "custom-hasher",
        hash: async (token) => {
            return myCustomHasher(token);
        }
    }
})
```

# plugins: Open API

URL: /docs/plugins/open-api
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/open-api.mdx

Open API reference for Better Auth.

***

title: Open API
description: Open API reference for Better Auth.

This is a plugin that provides an Open API reference for Better Auth. It shows all endpoints added by plugins and the core. It also provides a way to test the endpoints. It uses [Scalar](https://scalar.com/) to display the Open API reference.

<Callout>
  This plugin is still in the early stages of development. We are working on adding more features to it and filling in the gaps.
</Callout>

## Installation

<Steps>
  <Step>
    ### Add the plugin to your **auth** config

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { openAPI } from "better-auth/plugins"

    export const auth = betterAuth({
        plugins: [ // [!code highlight]
            openAPI(), // [!code highlight]
        ] // [!code highlight]
    })
    ```
  </Step>

  <Step>
    ### Navigate to `/api/auth/reference` to view the Open API reference

    Each plugin endpoints are grouped by the plugin name. The core endpoints are grouped under the `Default` group. And Model schemas are grouped under the `Models` group.

    ![Open API reference](/open-api-reference.png)
  </Step>
</Steps>

## Usage

The Open API reference is generated using the [OpenAPI 3.0](https://swagger.io/specification/) specification. You can use the reference to generate client libraries, documentation, and more.

The reference is generated using the [Scalar](https://scalar.com/) library. Scalar provides a way to view and test the endpoints. You can test the endpoints by clicking on the `Try it out` button and providing the required parameters.

![Open API reference](/open-api-reference.png)

### Generated Schema

To get the generated Open API schema directly as JSON, you can do `auth.api.generateOpenAPISchema()`. This will return the Open API schema as a JSON object.

```ts
import { auth } from "~/lib/auth"

const openAPISchema = await auth.api.generateOpenAPISchema()
console.log(openAPISchema)
```

### Using Scalar with Multiple Sources

If you're using Scalar for your API documentation, you can add Better Auth as an additional source alongside your main API:

When using Hono with Scalar for OpenAPI documentation, you can integrate Better Auth by adding it as a source:

```ts
app.get("/docs", Scalar({
  pageTitle: "API Documentation",
  sources: [
    { url: "/api/open-api", title: "API" },
    // Better Auth schema generation endpoint
    { url: "/api/auth/open-api/generate-schema", title: "Auth" },
  ],
}));
```

## Configuration

`path` - The path where the Open API reference is served. Default is `/api/auth/reference`. You can change it to any path you like, but keep in mind that it will be appended to the base path of your auth server.

`disableDefaultReference` - If set to `true`, the default Open API reference UI by Scalar will be disabled. Default is `false`.

This allows you to display both your application's API and Better Auth's authentication endpoints in a unified documentation interface.

`theme` - Allows you to change the theme of the OpenAPI reference page. Default is `default`.

# plugins: Organization
URL: /docs/plugins/organization
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/organization.mdx

The organization plugin allows you to manage your organization's members and teams.

***

title: Organization
description: The organization plugin allows you to manage your organization's members and teams.

Organizations simplifies user access and permissions management. Assign roles and permissions to streamline project management, team coordination, and partnerships.

## Installation

<Steps>
  <Step>
    ### Add the plugin to your **auth** config

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { organization } from "better-auth/plugins"

    export const auth = betterAuth({
        plugins: [ // [!code highlight]
            organization() // [!code highlight]
        ] // [!code highlight]
    })
    ```
  </Step>

  <Step>
    ### Migrate the database

    Run the migration or generate the schema to add the necessary fields and tables to the database.

    <Tabs items={["migrate", "generate"]}>
      <Tab value="migrate">
        ```bash
        npx @better-auth/cli migrate
        ```
      </Tab>

      <Tab value="generate">
        ```bash
        npx @better-auth/cli generate
        ```
      </Tab>
    </Tabs>

    See the [Schema](#schema) section to add the fields manually.
  </Step>

  <Step>
    ### Add the client plugin

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    import { organizationClient } from "better-auth/client/plugins"

    export const authClient = createAuthClient({
        plugins: [ // [!code highlight]
            organizationClient() // [!code highlight]
        ] // [!code highlight]
    })
    ```
  </Step>
</Steps>

## Usage

Once you've installed the plugin, you can start using the organization plugin to manage your organization's members and teams. The client plugin will provide you with methods under the `organization` namespace, and the server `api` will provide you with the necessary endpoints to manage your organization and give you an easier way to call the functions on your own backend.

## Organization

### Create an organization

### Client Side

```ts
const { data, error } = await authClient.organization.create({
    name: My Organization,
    slug: my-org,
    logo: https://example.com/logo.png, // required
    metadata, // required
    userId: some_user_id, // required
    keepCurrentActiveOrganization, // required
});
```

### Server Side

```ts
const data = await auth.api.createOrganization({
    body: {
        name: My Organization,
        slug: my-org,
        logo: https://example.com/logo.png, // required
        metadata, // required
        userId: some_user_id, // required
        keepCurrentActiveOrganization, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type createOrganization = {
  /**
  * The organization name.
  */
  name: string = "My Organization"
  /**
  * The organization slug.
  */
  slug: string = "my-org"
  /**
  * The organization logo.
  */
  logo?: string = "https://example.com/logo.png"
  /**
  * The metadata of the organization.
  */
  metadata?: Record<string, any>
  /**
  * The user ID of the organization creator. If not provided, the current user will be used. Should only be used by admins or when called by the server.
  * @serverOnly
  */
  userId?: string = "some_user_id"
  /**
  * Whether to keep the current active organization active after creating a new one.
  */
  keepCurrentActiveOrganization?: boolean = false

}
```

#### Restrict who can create an organization

By default, any user can create an organization. To restrict this, set the `allowUserToCreateOrganization` option to a function that returns a boolean, or directly to `true` or `false`.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { organization } from "better-auth/plugins";

const auth = betterAuth({
  //...
  plugins: [
    organization({
      allowUserToCreateOrganization: async (user) => {
        // [!code highlight]
        const subscription = await getSubscription(user.id); // [!code highlight]
        return subscription.plan === "pro"; // [!code highlight]
      }, // [!code highlight]
    }),
  ],
});
```

#### Check if organization slug is taken

To check if an organization slug is taken or not you can use the `checkSlug` function provided by the client. The function takes an object with the following properties:

### Client Side

```ts
const { data, error } = await authClient.organization.checkSlug({
    slug: my-org,
});
```

### Server Side

```ts
const data = await auth.api.checkOrganizationSlug({
    body: {
        slug: my-org,
    }
});
```

### Type Definition

```ts
type checkOrganizationSlug = {
    /**
     * The organization slug to check.
     */
    slug: string = "my-org"

}
```

### Organization Hooks

You can customize organization operations using hooks that run before and after various organization-related activities. Better Auth provides two ways to configure hooks:

1. **Legacy organizationCreation hooks** (deprecated, use `organizationHooks` instead)
2. **Modern organizationHooks** (recommended) - provides comprehensive control over all organization-related activities

#### Organization Creation and Management Hooks

Control organization lifecycle operations:

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { organization } from "better-auth/plugins";

export const auth = betterAuth({
  plugins: [
    organization({
      organizationHooks: {
        // Organization creation hooks
        beforeCreateOrganization: async ({ organization, user }) => {
          // Run custom logic before organization is created
          // Optionally modify the organization data
          return {
            data: {
              ...organization,
              metadata: {
                customField: "value",
              },
            },
          };
        },

        afterCreateOrganization: async ({ organization, member, user }) => {
          // Run custom logic after organization is created
          // e.g., create default resources, send notifications
          await setupDefaultResources(organization.id);
        },

        // Organization update hooks
        beforeUpdateOrganization: async ({ organization, user, member }) => {
          // Validate updates, apply business rules
          return {
            data: {
              ...organization,
              name: organization.name?.toLowerCase(),
            },
          };
        },

        afterUpdateOrganization: async ({ organization, user, member }) => {
          // Sync changes to external systems
          await syncOrganizationToExternalSystems(organization);
        },
      },
    }),
  ],
});
```

<Callout type="info">
  The legacy `organizationCreation` hooks are still supported but deprecated.
  Use `organizationHooks.beforeCreateOrganization` and
  `organizationHooks.afterCreateOrganization` instead for new projects.
</Callout>

#### Member Hooks

Control member operations within organizations:

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { organization } from "better-auth/plugins";

export const auth = betterAuth({
  plugins: [
    organization({
      organizationHooks: {
        // Before a member is added to an organization
        beforeAddMember: async ({ member, user, organization }) => {
          // Custom validation or modification
          console.log(`Adding ${user.email} to ${organization.name}`);

          // Optionally modify member data
          return {
            data: {
              ...member,
              role: "custom-role", // Override the role
            },
          };
        },

        // After a member is added
        afterAddMember: async ({ member, user, organization }) => {
          // Send welcome email, create default resources, etc.
          await sendWelcomeEmail(user.email, organization.name);
        },

        // Before a member is removed
        beforeRemoveMember: async ({ member, user, organization }) => {
          // Cleanup user's resources, send notification, etc.
          await cleanupUserResources(user.id, organization.id);
        },

        // After a member is removed
        afterRemoveMember: async ({ member, user, organization }) => {
          await logMemberRemoval(user.id, organization.id);
        },
```

```ts
        // Before updating a member's role
        beforeUpdateMemberRole: async ({
          member,
          newRole,
          user,
          organization,
        }) => {
          // Validate role change permissions
          if (newRole === "owner" && !hasOwnerUpgradePermission(user)) {
            throw new Error("Cannot upgrade to owner role");
          }

          // Optionally modify the role
          return {
            data: {
              role: newRole,
            },
          };
        },

        // After updating a member's role
        afterUpdateMemberRole: async ({
          member,
          previousRole,
          user,
          organization,
        }) => {
          await logRoleChange(user.id, previousRole, member.role);
        },
      },
    }),
  ],
});
```

#### Invitation Hooks

Control invitation lifecycle:

```ts title="auth.ts"
export const auth = betterAuth({
  plugins: [
    organization({
      organizationHooks: {
        // Before creating an invitation
        beforeCreateInvitation: async ({
          invitation,
          inviter,
          organization,
        }) => {
          // Custom validation or expiration logic
          const customExpiration = new Date(
            Date.now() + 1000 * 60 * 60 * 24 * 7
          ); // 7 days

          return {
            data: {
              ...invitation,
              expiresAt: customExpiration,
            },
          };
        },

        // After creating an invitation
        afterCreateInvitation: async ({
          invitation,
          inviter,
          organization,
        }) => {
          // Send custom invitation email, track metrics, etc.
          await sendCustomInvitationEmail(invitation, organization);
        },

        // Before accepting an invitation
        beforeAcceptInvitation: async ({ invitation, user, organization }) => {
          // Additional validation before acceptance
          await validateUserEligibility(user, organization);
        },

        // After accepting an invitation
        afterAcceptInvitation: async ({
          invitation,
          member,
          user,
          organization,
        }) => {
          // Setup user account, assign default resources
          await setupNewMemberResources(user, organization);
        },

        // Before/after rejecting invitations
        beforeRejectInvitation: async ({ invitation, user, organization }) => {
          // Log rejection reason, send notification to inviter
        },

        afterRejectInvitation: async ({ invitation, user, organization }) => {
          await notifyInviterOfRejection(invitation.inviterId, user.email);
        },

        // Before/after cancelling invitations
        beforeCancelInvitation: async ({
          invitation,
          cancelledBy,
          organization,
        }) => {
          // Verify cancellation permissions
        },

        afterCancelInvitation: async ({
          invitation,
          cancelledBy,
          organization,
        }) => {
          await logInvitationCancellation(invitation.id, cancelledBy.id);
        },
      },
    }),
  ],
});
```

#### Team Hooks

Control team operations (when teams are enabled):

```ts title="auth.ts"
export const auth = betterAuth({
  plugins: [
    organization({
      teams: { enabled: true },
      organizationHooks: {
        // Before creating a team
        beforeCreateTeam: async ({ team, user, organization }) => {
          // Validate team name, apply naming conventions
          return {
            data: {
              ...team,
              name: team.name.toLowerCase().replace(/\s+/g, "-"),
            },
          };
        },

        // After creating a team
        afterCreateTeam: async ({ team, user, organization }) => {
          // Create default team resources, channels, etc.
          await createDefaultTeamResources(team.id);
        },

        // Before updating a team
        beforeUpdateTeam: async ({ team, updates, user, organization }) => {
          // Validate updates, apply business rules
          return {
            data: {
              ...updates,
              name: updates.name?.toLowerCase(),
            },
          };
        },

        // After updating a team
        afterUpdateTeam: async ({ team, user, organization }) => {
          await syncTeamChangesToExternalSystems(team);
        },

        // Before deleting a team
        beforeDeleteTeam: async ({ team, user, organization }) => {
          // Backup team data, notify members
          await backupTeamData(team.id);
        },

        // After deleting a team
        afterDeleteTeam: async ({ team, user, organization }) => {
          await cleanupTeamResources(team.id);
        },

        // Team member operations
        beforeAddTeamMember: async ({
          teamMember,
          team,
          user,
          organization,
        }) => {
          // Validate team membership limits, permissions
          const memberCount = await getTeamMemberCount(team.id);
          if (memberCount >= 10) {
            throw new Error("Team is full");
          }
        },

        afterAddTeamMember: async ({
          teamMember,
          team,
          user,
          organization,
        }) => {
          await grantTeamAccess(user.id, team.id);
        },

        beforeRemoveTeamMember: async ({
          teamMember,
          team,
          user,
          organization,
        }) => {
          // Backup user's team-specific data
          await backupTeamMemberData(user.id, team.id);
        },

        afterRemoveTeamMember: async ({
          teamMember,
          team,
          user,
          organization,
        }) => {
          await revokeTeamAccess(user.id, team.id);
        },
      },
    }),
  ],
});
```

#### Hook Error Handling

All hooks support error handling. Throwing an error in a `before` hook will prevent the operation from proceeding:

```ts title="auth.ts"
import { APIError } from "better-auth/api";

export const auth = betterAuth({
  plugins: [
    organization({
      organizationHooks: {
        beforeAddMember: async ({ member, user, organization }) => {
          // Check if user has pending violations
          const violations = await checkUserViolations(user.id);
          if (violations.length > 0) {
            throw new APIError("BAD_REQUEST", {
              message:
                "User has pending violations and cannot join organizations",
            });
          }
        },
```

~~~
        beforeCreateTeam: async ({ team, user, organization }) => {
          // Validate team name uniqueness
          const existingTeam = await findTeamByName(team.name, organization.id);
          if (existingTeam) {
            throw new APIError("BAD_REQUEST", {
              message: "Team name already exists in this organization",
            });
          }
        },
      },
    }),
  ],
});
```

### List User's Organizations

To list the organizations that a user is a member of, you can use `useListOrganizations` hook. It implements a reactive way to get the organizations that the user is a member of.

<Tabs items={["React", "Vue", "Svelte"]} default="React">
  <Tab value="React">
    ```tsx title="client.tsx"
    import { authClient } from "@/lib/auth-client"

    function App(){
    const { data: organizations } = authClient.useListOrganizations()
    return (
      <div>
        {organizations.map((org) => (
          <p>{org.name}</p>
        ))}
      </div>)
    }
    ```
  </Tab>

  <Tab value="Svelte">
    ```svelte title="page.svelte"
    <script lang="ts">
      import { authClient } from "$lib/auth-client";
      const organizations = authClient.useListOrganizations();
    </script>

    <h1>Organizations</h1>

    {#if $organizations.isPending}

      <p>Loading...</p>
    {:else if !$organizations.data?.length}
      <p>No organizations found.</p>
    {:else}
      <ul>
        {#each $organizations.data as organization}
          <li>{organization.name}</li>
        {/each}
      </ul>
    {/if}
    ```
  </Tab>

  <Tab value="Vue">
    ```vue title="organization.vue"
    <script lang="ts">;
    export default {
        setup() {
            const organizations = authClient.useListOrganizations()
            return { organizations };
        }
    };
    </script>

    <template>
        <div>
            <h1>Organizations</h1>
            <div v-if="organizations.isPending">Loading...</div>
            <div v-else-if="organizations.data === null">No organizations found.</div>
            <ul v-else>
                <li v-for="organization in organizations.data" :key="organization.id">
                    {{ organization.name }}
                </li>
            </ul>
        </div>
    </template>
    ```
  </Tab>
</Tabs>

Or alternatively, you can call `organization.list` if you don't want to use a hook.

### Client Side

```ts
const { data, error } = await authClient.organization.list({});
```

### Server Side

```ts
const data = await auth.api.listOrganizations({

    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type listOrganizations = {

}
```

### Active Organization

Active organization is the workspace the user is currently working on. By default when the user is signed in the active organization is set to `null`. You can set the active organization to the user session.

<Callout type="info">
  It's not always you want to persist the active organization in the session.
  You can manage the active organization in the client side only. For example,
  multiple tabs can have different active organizations.
</Callout>

#### Set Active Organization

You can set the active organization by calling the `organization.setActive` function. It'll set the active organization for the user session.

<Callout>
~~~

~~~mdx
  In some applications, you may want the ability to unset an active
  organization. In this case, you can call this endpoint with `organizationId`
  set to `null`.
</Callout>

### Client Side

```ts
const { data, error } = await authClient.organization.setActive({
    organizationId: org-id, // required
    organizationSlug: org-slug, // required
});
```

### Server Side

```ts
const data = await auth.api.setActiveOrganization({
    body: {
        organizationId: org-id, // required
        organizationSlug: org-slug, // required
    }
});
```

### Type Definition

```ts
type setActiveOrganization = {
    /**
     * The organization ID to set as active. It can be null to unset the active organization.
     */
    organizationId?: string | null = "org-id"
    /**
     * The organization slug to set as active. It can be null to unset the active organization if organizationId is not provided.
     */
    organizationSlug?: string = "org-slug"

}
```

To set active organization when a session is created you can use [database hooks](/docs/concepts/database#database-hooks).

```ts title="auth.ts"
export const auth = betterAuth({
  databaseHooks: {
    session: {
      create: {
        before: async (session) => {
          const organization = await getActiveOrganization(session.userId);
          return {
            data: {
              ...session,
              activeOrganizationId: organization.id,
            },
          };
        },
      },
    },
  },
});
```

#### Use Active Organization

To retrieve the active organization for the user, you can call the `useActiveOrganization` hook. It returns the active organization for the user. Whenever the active organization changes, the hook will re-evaluate and return the new active organization.

<Tabs items={['React', 'Vue', 'Svelte']}>
  <Tab value="React">
    ```tsx title="client.tsx"
    import { authClient } from "@/lib/auth-client"

    function App(){
        const { data: activeOrganization } = authClient.useActiveOrganization()
        return (
            <div>
                {activeOrganization ? <p>{activeOrganization.name}</p> : null}
            </div>
        )
    }
    ```
  </Tab>

  <Tab value="Svelte">
    ```tsx title="client.tsx"
    <script lang="ts">
    import { authClient } from "$lib/auth-client";
    const activeOrganization = authClient.useActiveOrganization();
    </script>

    <h2>Active Organization</h2>

    {#if $activeOrganization.isPending}
    <p>Loading...</p>
    {:else if $activeOrganization.data === null}
    <p>No active organization found.</p>
    {:else}
    <p>{$activeOrganization.data.name}</p>
    {/if}
    ```
  </Tab>

  <Tab value="Vue">
    ```vue title="organization.vue"
    <script lang="ts">;
    export default {
        setup() {
            const activeOrganization = authClient.useActiveOrganization();
            return { activeOrganization };
        }
    };
    </script>

    <template>
        <div>
            <h2>Active organization</h2>
            <div v-if="activeOrganization.isPending">Loading...</div>
            <div v-else-if="activeOrganization.data === null">No active organization.</div>
            <div v-else>
                {{ activeOrganization.data.name }}
            </div>
        </div>
    </template>
    ```
  </Tab>
</Tabs>

### Get Full Organization
~~~

To get the full details of an organization, you can use the `getFullOrganization` function.
By default, if you don't pass any properties, it will use the active organization.

### Client Side

```ts
const { data, error } = await authClient.organization.getFullOrganization({
    organizationId: org-id, // required
    organizationSlug: org-slug, // required
    membersLimit, // required
});
```

### Server Side

```ts
const data = await auth.api.getFullOrganization({
    query: {
        organizationId: org-id, // required
        organizationSlug: org-slug, // required
        membersLimit, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type getFullOrganization = {
    /**
     * The organization ID to get. By default, it will use the active organization.
     */
    organizationId?: string = "org-id"
    /**
     * The organization slug to get.
     */
    organizationSlug?: string = "org-slug"
    /**
     * The limit of members to get. By default, it uses the membershipLimit option which defaults to 100.
     */
    membersLimit?: number = 100

}
```

### Update Organization

To update organization info, you can use `organization.update`

### Client Side

```ts
const { data, error } = await authClient.organization.update({
    data,
    name: updated-name, // required
    slug: updated-slug, // required
    logo: new-logo.url, // required
    metadata, // required
});
```

### Server Side

```ts
const data = await auth.api.updateOrganization({
    body: {
        data,
        name: updated-name, // required
        slug: updated-slug, // required
        logo: new-logo.url, // required
        metadata, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type updateOrganization = {
    /**
     * A partial list of data to update the organization.
     */
    data: {
        /**
         * The name of the organization.
         */
        name?: string = "updated-name"
        /**
         * The slug of the organization.
         */
        slug?: string = "updated-slug"
        /**
         * The logo of the organization.
         */
        logo?: string = "new-logo.url"
        /**
         * The metadata of the organization.
         */
        metadata?: Record<string, any> | null = { customerId: "test"
}
```

### Delete Organization

To remove user owned organization, you can use `organization.delete`

### Client Side

```ts
const { data, error } = await authClient.organization.delete({
    organizationId: org-id,
});
```

### Server Side

```ts
const data = await auth.api.deleteOrganization({
    body: {
        organizationId: org-id,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type deleteOrganization = {
    /*
    * The organization ID to delete.
    */
    organizationId: string = "org-id"

}
```

If the user has the necessary permissions (by default: role is owner) in the specified organization, all members, invitations and organization information will be removed.

You can configure how organization deletion is handled through `organizationDeletion` option:

```ts
const auth = betterAuth({
  plugins: [
    organization({
      disableOrganizationDeletion: true, //to disable it altogether
      organizationHooks: {
        beforeDeleteOrganization: async (data, request) => {
          // a callback to run before deleting org
        },
        afterDeleteOrganization: async (data, request) => {
          // a callback to run after deleting org
        },
      },
    }),
  ],
});
```

## Invitations

To add a member to an organization, we first need to send an invitation to the user. The user will receive an email/sms with the invitation link. Once the user accepts the invitation, they will be added to the organization.

### Setup Invitation Email

For member invitation to work we first need to provide `sendInvitationEmail` to the `better-auth` instance. This function is responsible for sending the invitation email to the user.

You'll need to construct and send the invitation link to the user. The link should include the invitation ID, which will be used with the acceptInvitation function when the user clicks on it.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { organization } from "better-auth/plugins";
import { sendOrganizationInvitation } from "./email";
export const auth = betterAuth({
  plugins: [
    organization({
      async sendInvitationEmail(data) {
        const inviteLink = `https://example.com/accept-invitation/${data.id}`;
        sendOrganizationInvitation({
          email: data.email,
          invitedByUsername: data.inviter.user.name,
          invitedByEmail: data.inviter.user.email,
          teamName: data.organization.name,
          inviteLink,
        });
      },
    }),
  ],
});
```

### Send Invitation

To invite users to an organization, you can use the `invite` function provided by the client. The `invite` function takes an object with the following properties:

### Client Side

```ts
const { data, error } = await authClient.organization.inviteMember({
    email: example@gmail.com,
    role: member,
    organizationId: org-id, // required
    resend, // required
    teamId: team-id, // required
});
```

### Server Side

```ts
const data = await auth.api.createInvitation({
    body: {
        email: example@gmail.com,
        role: member,
        organizationId: org-id, // required
        resend, // required
        teamId: team-id, // required
    }
});
```

### Type Definition

```ts
type createInvitation = {
    /**
     * The email address of the user to invite.
     */
    email: string = "example@gmail.com"
    /**
     * The role(s) to assign to the user. It can be `admin`, `member`, or `guest`.
     */
    role: string | string[] = "member"
    /**
     * The organization ID to invite the user to. Defaults to the active organization.
     */
    organizationId?: string = "org-id"
    /**
     * Resend the invitation email, if the user is already invited.
     */
    resend?: boolean = true
    /**
     * The team ID to invite the user to.
     */
    teamId?: string = "team-id"

}
```

<Callout>

  * If the user is already a member of the organization, the invitation will be
    canceled. - If the user is already invited to the organization, unless
    `resend` is set to `true`, the invitation will not be sent again. - If
    `cancelPendingInvitationsOnReInvite` is set to `true`, the invitation will be
    canceled if the user is already invited to the organization and a new
    invitation is sent.
</Callout>

### Accept Invitation

When a user receives an invitation email, they can click on the invitation link to accept the invitation. The invitation link should include the invitation ID, which will be used to accept the invitation.

Make sure to call the `acceptInvitation` function after the user is logged in.

### Client Side

```ts
const { data, error } = await authClient.organization.acceptInvitation({
    invitationId: invitation-id,
});
```

### Server Side

```ts
const data = await auth.api.acceptInvitation({
    body: {
        invitationId: invitation-id,
    }
});
```

### Type Definition

```ts
type acceptInvitation = {
    /**
     * The ID of the invitation to accept.
     */
    invitationId: string = "invitation-id"

}
```

#### Email Verification Requirement

If the `requireEmailVerificationOnInvitation` option is enabled in your organization configuration, users must verify their email address before they can accept invitations. This adds an extra security layer to ensure that only verified users can join your organization.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { organization } from "better-auth/plugins";

export const auth = betterAuth({
  plugins: [
    organization({
      requireEmailVerificationOnInvitation: true, // [!code highlight]
      async sendInvitationEmail(data) {
        // ... your email sending logic
      },
    }),
  ],
});
```

### Invitation Accepted Callback

You can configure Better Auth to execute a callback function when an invitation is accepted. This is useful for logging events, updating analytics, sending notifications, or any other custom logic you need to run when someone joins your organization.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { organization } from "better-auth/plugins";

export const auth = betterAuth({
  plugins: [
    organization({
      async sendInvitationEmail(data) {
        // ... your invitation email logic
      },
      async onInvitationAccepted(data) {
        // This callback gets triggered when an invitation is accepted
      },
    }),
  ],
});
```

The callback receives the following data:

* `id`: The invitation ID
* `role`: The role assigned to the user
* `organization`: The organization the user joined
* `invitation`: The invitation object
* `inviter`: The member who sent the invitation (including user details)
* `acceptedUser`: The user who accepted the invitation

### Cancel Invitation

If a user has sent out an invitation, you can use this method to cancel it.

If you're looking for how a user can reject an invitation, you can find that [here](#reject-invitation).

### Client Side

```ts
const { data, error } = await authClient.organization.cancelInvitation({
    invitationId: invitation-id,
});
```

### Server Side

```ts
await auth.api.cancelInvitation({
    body: {
        invitationId: invitation-id,
    }
});
```

### Type Definition

```ts
type cancelInvitation = {
    /**
     * The ID of the invitation to cancel.
     */
    invitationId: string = "invitation-id"

}
```

### Reject Invitation

If this user has received an invitation, but wants to decline it, this method will allow you to do so by rejecting it.

### Client Side

```ts
const { data, error } = await authClient.organization.rejectInvitation({
    invitationId: invitation-id,
});
```

### Server Side

```ts
await auth.api.rejectInvitation({
    body: {
        invitationId: invitation-id,
    }
});
```

### Type Definition

```ts
type rejectInvitation = {
    /**
     * The ID of the invitation to reject.
     */
    invitationId: string = "invitation-id"

}
```

<Callout type="info">
  Like accepting invitations, rejecting invitations also requires email
  verification when the `requireEmailVerificationOnInvitation` option is
  enabled. Users with unverified emails will receive an error when attempting to
  reject invitations.
</Callout>

### Get Invitation

To get an invitation you can use the `organization.getInvitation` function provided by the client. You need to provide the invitation id as a query parameter.

### Client Side

```ts
const { data, error } = await authClient.organization.getInvitation({
    id: invitation-id,
});
```

### Server Side

```ts
const data = await auth.api.getInvitation({
    query: {
        id: invitation-id,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type getInvitation = {
    /**
     * The ID of the invitation to get.
     */
    id: string = "invitation-id"

}
```

### List Invitations

To list all invitations for a given organization you can use the `listInvitations` function provided by the client.

### Client Side

```ts
const { data, error } = await authClient.organization.listInvitations({
    organizationId: organization-id, // required
});
```

### Server Side

```ts
const data = await auth.api.listInvitations({
    query: {
        organizationId: organization-id, // required
    }
});
```

### Type Definition

```ts
type listInvitations = {
    /**
     * An optional ID of the organization to list invitations for. If not provided, will default to the user's active organization.
     */
    organizationId?: string = "organization-id"

}
```

### List user invitations

To list all invitations for a given user you can use the `listUserInvitations` function provided by the client.

```ts title="auth-client.ts"
const invitations = await authClient.organization.listUserInvitations();
```

On the server, you can pass the user ID as a query parameter.

```ts title="api.ts"
const invitations = await auth.api.listUserInvitations({
  query: {
    email: "user@example.com",
  },
});
```

<Callout type="warn">
  The `email` query parameter is only available on the server to query for
  invitations for a specific user.
</Callout>

## Members

### List Members

To list all members of an organization you can use the `listMembers` function.

### Client Side

```ts
const { data, error } = await authClient.organization.listMembers({
    organizationId: organization-id, // required
    limit, // required
    offset, // required
    sortBy: createdAt, // required
    sortDirection: desc, // required
    filterField: createdAt, // required
    filterOperator: eq, // required
    filterValue: value, // required
});
```

### Server Side

```ts
const data = await auth.api.listMembers({
    query: {
        organizationId: organization-id, // required
        limit, // required
        offset, // required
        sortBy: createdAt, // required
        sortDirection: desc, // required
        filterField: createdAt, // required
        filterOperator: eq, // required
        filterValue: value, // required
    }
});
```

### Type Definition

```ts
type listMembers = {
    /**
     * An optional organization ID to list members for. If not provided, will default to the user's active organization.
     */
    organizationId?: string = "organization-id"
    /**
     * The limit of members to return.
     */
    limit?: number = 100
    /**
     * The offset to start from.
     */
    offset?: number = 0
    /**
     * The field to sort by.
     */
    sortBy?: string = "createdAt"
    /**
     * The direction to sort by.
     */
    sortDirection?: "asc" | "desc" = "desc"
    /**
     * The field to filter by.
     */
    filterField?: string = "createdAt"
    /**
     * The operator to filter by.
     */
    filterOperator?: "eq" | "ne" | "gt" | "gte" | "lt" | "lte" | "in" | "nin" | "contains" = "eq"
    /**
     * The value to filter by.
     */
    filterValue?: string = "value"

}
```

### Remove Member

To remove you can use `organization.removeMember`

### Client Side

```ts
const { data, error } = await authClient.organization.removeMember({
    memberIdOrEmail: user@example.com,
    organizationId: org-id, // required
});
```

### Server Side

```ts
const data = await auth.api.removeMember({
    body: {
        memberIdOrEmail: user@example.com,
        organizationId: org-id, // required
    }
});
```

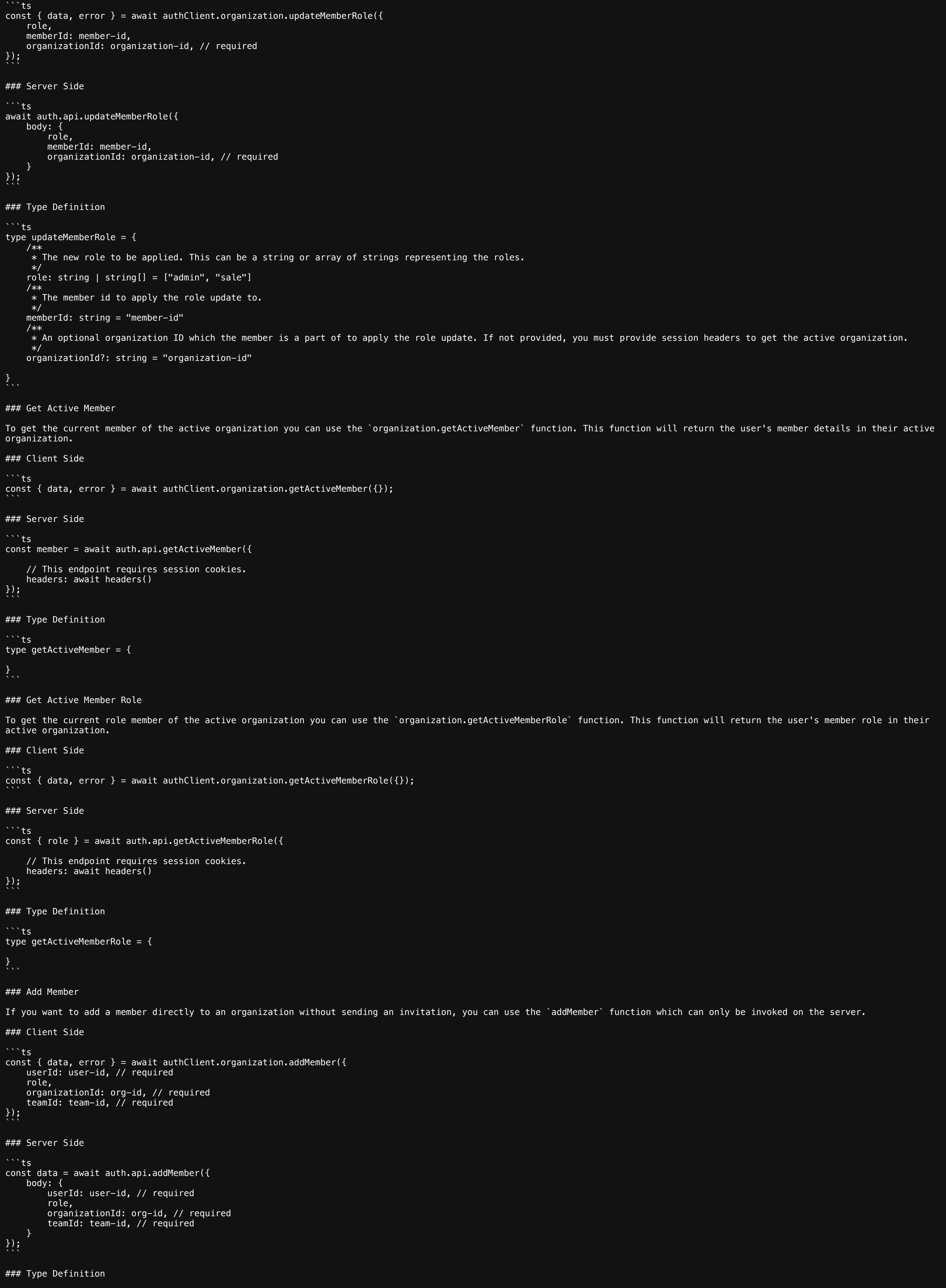
### Type Definition

```ts
type removeMember = {
    /**
     * The ID or email of the member to remove.
     */
    memberIdOrEmail: string = "user@example.com"
    /**
     * The ID of the organization to remove the member from. If not provided, the active organization will be used.
     */
    organizationId?: string = "org-id"

}
```

### Update Member Role

To update the role of a member in an organization, you can use the `organization.updateMemberRole`. If the user has the permission to update the role of the member, the role will be updated.

### Client Side



```ts
const { data, error } = await authClient.organization.updateMemberRole({
    role,
    memberId: member-id,
    organizationId: organization-id, // required
});
```

### Server Side

```ts
await auth.api.updateMemberRole({
    body: {
        role,
        memberId: member-id,
        organizationId: organization-id, // required
    }
});
```

### Type Definition

```ts
type updateMemberRole = {
    /**
     * The new role to be applied. This can be a string or array of strings representing the roles.
     */
    role: string | string[] = ["admin", "sale"]
    /**
     * The member id to apply the role update to.
     */
    memberId: string = "member-id"
    /**
     * An optional organization ID which the member is a part of to apply the role update. If not provided, you must provide session headers to get the active organization.
     */
    organizationId?: string = "organization-id"

}
```

### Get Active Member

To get the current member of the active organization you can use the `organization.getActiveMember` function. This function will return the user's member details in their active organization.

### Client Side

```ts
const { data, error } = await authClient.organization.getActiveMember({});
```

### Server Side

```ts
const member = await auth.api.getActiveMember({

    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type getActiveMember = {

}
```

### Get Active Member Role

To get the current role member of the active organization you can use the `organization.getActiveMemberRole` function. This function will return the user's member role in their active organization.

### Client Side

```ts
const { data, error } = await authClient.organization.getActiveMemberRole({});
```

### Server Side

```ts
const { role } = await auth.api.getActiveMemberRole({

    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type getActiveMemberRole = {

}
```

### Add Member

If you want to add a member directly to an organization without sending an invitation, you can use the `addMember` function which can only be invoked on the server.

### Client Side

```ts
const { data, error } = await authClient.organization.addMember({
    userId: user-id, // required
    role,
    organizationId: org-id, // required
    teamId: team-id, // required
});
```

### Server Side

```ts
const data = await auth.api.addMember({
    body: {
        userId: user-id, // required
        role,
        organizationId: org-id, // required
        teamId: team-id, // required
    }
});
```

### Type Definition

```ts
type addMember = {
    /**
     * The user ID which represents the user to be added as a member. If `null` is provided, then it's expected to provide session headers.
     */
    userId?: string | null = "user-id"
    /**
     * The role(s) to assign to the new member.
     */
    role: string | string[] = ["admin", "sale"]
    /**
     * An optional organization ID to pass. If not provided, will default to the user's active organization.
     */
    organizationId?: string = "org-id"
    /**
     * An optional team ID to add the member to.
     */
    teamId?: string = "team-id"

}
```

### Leave Organization

To leave organization you can use `organization.leave` function. This function will remove the current user from the organization.

### Client Side

```ts
const { data, error } = await authClient.organization.leave({
    organizationId: organization-id,
});
```

### Server Side

```ts
await auth.api.leaveOrganization({
    body: {
        organizationId: organization-id,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type leaveOrganization = {
    /**
     * The organization ID for the member to leave.
     */
    organizationId: string = "organization-id"

}
```

## Access Control

The organization plugin provides a very flexible access control system. You can control the access of the user based on the role they have in the organization. You can define your own set of permissions based on the role of the user.

### Roles

By default, there are three roles in the organization:

`owner`: The user who created the organization by default. The owner has full control over the organization and can perform any action.

`admin`: Users with the admin role have full control over the organization except for deleting the organization or changing the owner.

`member`: Users with the member role have limited control over the organization. They can create projects, invite users, and manage projects they have created.

<Callout>
  A user can have multiple roles. Multiple roles are stored as string separated
  by comma (",").
</Callout>

### Permissions

By default, there are three resources, and these have two to three actions.

organization:

`update` `delete`

member:

`create` `update` `delete`

invitation:

`create` `cancel`

The owner has full control over all the resources and actions. The admin has full control over all the resources except for deleting the organization or changing the owner. The member has no control over any of those actions other than reading the data.

### Custom Permissions

The plugin provides an easy way to define your own set of permissions for each role.

<Steps>
  <Step>
    #### Create Access Control

    You first need to create access controller by calling `createAccessControl` function and passing the statement object. The statement object should have the resource name as the key and the array of actions as the value.

    ```ts title="permissions.ts"
    import { createAccessControl } from "better-auth/plugins/access";

    /**
     * make sure to use `as const` so typescript can infer the type correctly
     */
    const statement = { // [!code highlight]
        project: ["create", "share", "update", "delete"], // [!code highlight]
    } as const; // [!code highlight]

    const ac = createAccessControl(statement); // [!code highlight]
    ```
  </Step>

  <Step>
    #### Create Roles

    Once you have created the access controller you can create roles with the permissions you have defined.

    ```ts title="permissions.ts"
    import { createAccessControl } from "better-auth/plugins/access";

````
    const statement = {
        project: ["create", "share", "update", "delete"],
    } as const;

    const ac = createAccessControl(statement);

    const member = ac.newRole({ // [!code highlight]
        project: ["create"], // [!code highlight]
    }); // [!code highlight]

    const admin = ac.newRole({ // [!code highlight]
        project: ["create", "update"], // [!code highlight]
    }); // [!code highlight]

    const owner = ac.newRole({ // [!code highlight]
        project: ["create", "update", "delete"], // [!code highlight]
    }); // [!code highlight]

    const myCustomRole = ac.newRole({ // [!code highlight]
        project: ["create", "update", "delete"], // [!code highlight]
        organization: ["update"], // [!code highlight]
    }); // [!code highlight]

    When you create custom roles for existing roles, the predefined permissions for those roles will be overridden. To add the existing permissions to the custom role, you need to import `defaultStatements` and merge it with your new statement, plus merge the roles' permissions set with the default roles.

    ```ts title="permissions.ts"
    import { createAccessControl } from "better-auth/plugins/access";
    import { defaultStatements, adminAc } from 'better-auth/plugins/organization/access'

    const statement = {
        ...defaultStatements, // [!code highlight]
        project: ["create", "share", "update", "delete"],
    } as const;

    const ac = createAccessControl(statement);

    const admin = ac.newRole({
        project: ["create", "update"],
        ...adminAc.statements, // [!code highlight]
    });
    ```
  </Step>

  <Step>
    #### Pass Roles to the Plugin

    Once you have created the roles you can pass them to the organization plugin both on the client and the server.

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { organization } from "better-auth/plugins"
    import { ac, owner, admin, member } from "@/auth/permissions"

    export const auth = betterAuth({
        plugins: [
            organization({
                ac,
                roles: {
                    owner,
                    admin,
                    member,
                    myCustomRole
                }
            }),
        ],
    });
    ```

    You also need to pass the access controller and the roles to the client plugin.

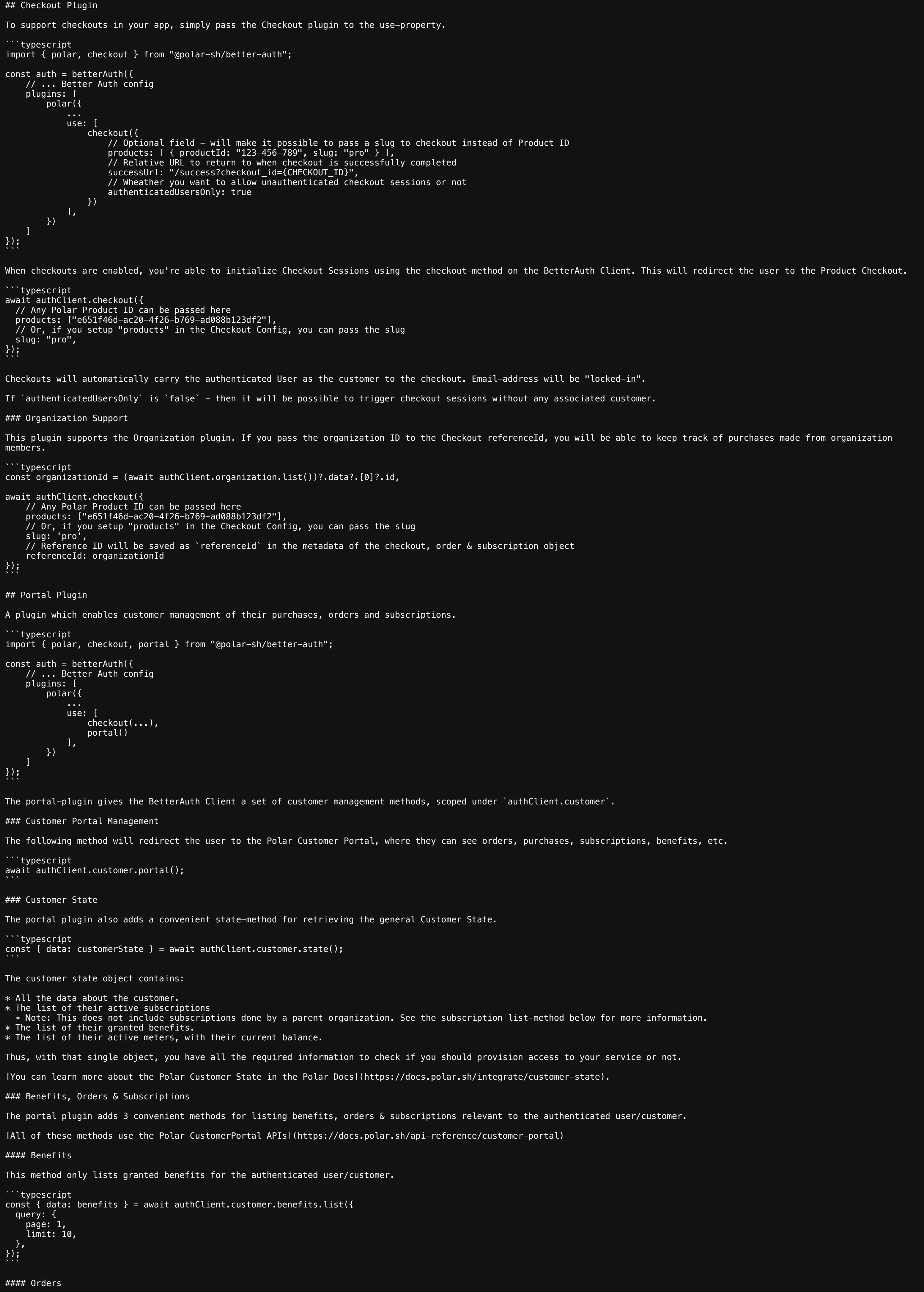
    ```ts title="auth-client"
    import { createAuthClient } from "better-auth/client"
    import { organizationClient } from "better-auth/client/plugins"
    import { ac, owner, admin, member, myCustomRole } from "@/auth/permissions"

    export const authClient = createAuthClient({
        plugins: [
            organizationClient({
                ac,
                roles: {
                    owner,
                    admin,
                    member,
                    myCustomRole
                }
            })
      ]
    })
    ```
  </Step>
</Steps>

### Access Control Usage

Has Permission:

You can use the `hasPermission` action provided by the `api` to check the permission of the user.

```ts title="api.ts"
import { auth } from "@/auth";

await auth.api.hasPermission({
  headers: await headers(),
  body: {
    permissions: {
      project: ["create"], // This must match the structure in your access control
    },
  },
});

// You can also check multiple resource permissions at the same time
await auth.api.hasPermission({
  headers: await headers(),
  body: {
    permissions: {
      project: ["create"], // This must match the structure in your access control
      sale: ["create"],
    },
  },
});
```

If you want to check the permission of the user on the client from the server you can use the `hasPermission` function provided by the client.
````

```ts title="auth-client.ts"
const canCreateProject = await authClient.organization.hasPermission({
  permissions: {
    project: ["create"],
  },
});

// You can also check multiple resource permissions at the same time
const canCreateProjectAndCreateSale =
  await authClient.organization.hasPermission({
    permissions: {
      project: ["create"],
      sale: ["create"],
    },
  });
```

Check Role Permission:

Once you have defined the roles and permissions to avoid checking the permission from the server you can use the `checkRolePermission` function provided by the client.

```ts title="auth-client.ts"
const canCreateProject = authClient.organization.checkRolePermission({
  permissions: {
    organization: ["delete"],
  },
  role: "admin",
});

// You can also check multiple resource permissions at the same time
const canCreateProjectAndCreateSale =
  authClient.organization.checkRolePermission({
    permissions: {
      organization: ["delete"],
      member: ["delete"],
    },
    role: "admin",
  });
```

<Callout type="warn">
  This will not include any dynamic roles as everything is ran syncronously on the client side.
  Please use the [hasPermission](#access-control-usage) APIs to include checks for any dynamic roles & permissions.
</Callout>

***

## Dynamic Access Control

Dynamic access control allows you to create roles at runtime for organizations. This is achieved by storing the
created roles and permissions associated with an organization in a database table.

### Enabling Dynamic Access Control

To enable dynamic access control, pass the `dynamicAccessControl` configuration option with `enabled` set to `true` to both server and client plugins.

Ensure you have pre-defined an `ac` instance on the server auth plugin.
This is important as this is how we can infer the permissions that are available for use.

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { organization } from "better-auth/plugins";
import { ac } from "@/auth/permissions";

export const auth = betterAuth({
    plugins: [ // [!code highlight]
        organization({ // [!code highlight]
            ac, // Must be defined in order for dynamic access control to work // [!code highlight]
            dynamicAccessControl: { // [!code highlight]
              enabled: true, // [!code highlight]
            }, // [!code highlight]
        }) // [!code highlight]
    ] // [!code highlight]
})
```

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client";
import { organizationClient } from "better-auth/client/plugins";

export const authClient = createAuthClient({
    plugins: [ // [!code highlight]
        organizationClient({ // [!code highlight]
            dynamicAccessControl: { // [!code highlight]
              enabled: true, // [!code highlight]
            }, // [!code highlight]
        }) // [!code highlight]
    ] // [!code highlight]
})
```

<Callout>
  This will require you to run migrations to add the new `organizationRole` table to the database.
</Callout>

<Callout type="warn">
  The `authClient.organization.checkRolePermission` function will not include any dynamic roles as everything is ran syncronously on the client side.
  Please use the [hasPermission](#access-control-usage) APIs to include checks for any dynamic roles.
</Callout>

### Creating a role

To create a new role for an organization at runtime, you can use the `createRole` function.

Only users with roles which contain the `ac` resource with the `create` permission can create a new role.
By default, only the `admin` and `owner` roles have this permission. You also cannot add permissions that your
current role in that organization can't already access.

<Callout>
  TIP: You can validate role names by using the `dynamicAccessControl.validateRoleName` option in the organization plugin config.
  Learn more [here](#validaterolename).
</Callout>

### Client Side

```ts
const { data, error } = await authClient.organization.createRole({
    role: my-unique-role,
    permission, // required
    organizationId: organization-id, // required
});
```

### Server Side

```ts
await auth.api.createOrgRole({
    body: {
        role: my-unique-role,
        permission, // required
        organizationId: organization-id, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type createOrgRole = {
    /**
     * A unique name of the role to create.
     */
    role: string = "my-unique-role"
    /**
     * The permissions to assign to the role.
     */
    permission?: Record<string, string[]> = permission,
    /**
     * The organization ID which the role will be created in. Defaults to the active organization.
     */
    organizationId?: string = "organization-id"

}
```

Now you can freely call [`updateMemberRole`](#updating-a-member-role) to update the role of a member with your newly created role!

### Deleting a role

To delete a role, you can use the `deleteRole` function, then provide either a `roleName` or `roleId` parameter along with the `organizationId` parameter.

### Client Side

```ts
const { data, error } = await authClient.organization.deleteRole({
    roleName: my-role, // required
    roleId: role-id, // required
    organizationId: organization-id, // required
});
```

### Server Side

```ts
await auth.api.deleteOrgRole({
    body: {
        roleName: my-role, // required
        roleId: role-id, // required
        organizationId: organization-id, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type deleteOrgRole = {
    /**
     * The name of the role to delete. Alternatively, you can pass a `roleId` parameter instead.
     */
    roleName?: string = "my-role"
    /**
     * The id of the role to delete. Alternatively, you can pass a `roleName` parameter instead.
     */
    roleId?: string = "role-id"
    /**
     * The organization ID which the role will be deleted in. Defaults to the active organization.
     */
    organizationId?: string = "organization-id"

}
```

### Listing roles

To list roles, you can use the `listOrgRoles` function.
This requires the `ac` resource with the `read` permission for the member to be able to list roles.

### Client Side

```ts
const { data, error } = await authClient.organization.listRoles({
    organizationId: organization-id, // required
});
```

### Server Side

```ts
const roles = await auth.api.listOrgRoles({
    query: {
        organizationId: organization-id, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type listOrgRoles = {
    /**
     * The organization ID which the roles are under to list. Defaults to the user's active organization.
     */
    organizationId?: string = "organization-id"

}
```

### Getting a specific role

To get a specific role, you can use the `getOrgRole` function and pass either a `roleName` or `roleId` parameter.
This requires the `ac` resource with the `read` permission for the member to be able to get a role.

### Client Side

```ts
const { data, error } = await authClient.organization.getRole({
    roleName: my-role, // required
    roleId: role-id, // required
    organizationId: organization-id, // required
});
```

### Server Side

```ts
const role = await auth.api.getOrgRole({
    query: {
        roleName: my-role, // required
        roleId: role-id, // required
        organizationId: organization-id, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type getOrgRole = {
    /**
     * The name of the role to get. Alternatively, you can pass a `roleId` parameter instead.
     */
    roleName?: string = "my-role"
    /**
     * The id of the role to get. Alternatively, you can pass a `roleName` parameter instead.
     */
    roleId?: string = "role-id"
    /**
     * The organization ID which the role will be deleted in. Defaults to the active organization.
     */
    organizationId?: string = "organization-id"

}
```

### Updating a role

To update a role, you can use the `updateOrgRole` function and pass either a `roleName` or `roleId` parameter.

### Client Side

```ts
const { data, error } = await authClient.organization.updateRole({
    roleName: my-role, // required
    roleId: role-id, // required
    organizationId: organization-id, // required
    data,
    permission, // required
});
```

### Server Side

```ts
const updatedRole = await auth.api.updateOrgRole({
    body: {
        roleName: my-role, // required
        roleId: role-id, // required
        organizationId: organization-id, // required
        data,
        permission, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type updateOrgRole = {
    /**
     * The name of the role to update. Alternatively, you can pass a `roleId` parameter instead.
     */
    roleName?: string = "my-role"
    /**
     * The id of the role to update. Alternatively, you can pass a `roleName` parameter instead.
     */
    roleId?: string = "role-id"
    /**
     * The organization ID which the role will be updated in. Defaults to the active organization.
     */
    organizationId?: string = "organization-id"
    /**
     * The data which will be updated
    */
    data: {
      /**
       * Optionally update the permissions of the role.
       */
      permission?: Record<string, string[]> = { project: ["create", "update", "delete"]
}
```

### Configuration Options

Below is a list of options that can be passed to the `dynamicAccessControl` object.

`enabled`

This option is used to enable or disable dynamic access control. By default, it is disabled.

```ts
organization({
  dynamicAccessControl: {
    enabled: true // [!code highlight]
  }
})
```

#### `maximumRolesPerOrganization`

This option is used to limit the number of roles that can be created for an organization.

By default, the maximum number of roles that can be created for an organization is infinite.

```ts
organization({
  dynamicAccessControl: {
    maximumRolesPerOrganization: 10 // [!code highlight]
  }
})
```

You can also pass a function that returns a number.

```ts
organization({
  dynamicAccessControl: {
    maximumRolesPerOrganization: async (organizationId) => { // [!code highlight]
      const organization = await getOrganization(organizationId); // [!code highlight]
      return organization.plan === "pro" ? 100 : 10; // [!code highlight]
    } // [!code highlight]
  }
})
```

### Additional Fields

To add additional fields to the `organizationRole` table, you can pass the `additionalFields` configuration option to the `organization` plugin.

```ts
organization({
  schema: {
    organizationRole: {
      additionalFields: {
        // Role colors!
        color: {
          type: "string",
          defaultValue: "#ffffff",
        },
        //... other fields
      },
    },
  },
})
```

Then, if you don't already use `inferOrgAdditionalFields` to infer the additional fields, you can use it to infer the additional fields.

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client"
import { organizationClient, inferOrgAdditionalFields } from "better-auth/client/plugins"
import type { auth } from "./auth"

export const authClient = createAuthClient({
    plugins: [
        organizationClient({
            schema: inferOrgAdditionalFields<typeof auth>()
        })
    ]
})
```

Otherwise, you can pass the schema values directly, the same way you do on the org plugin in the server.

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client"
import { organizationClient } from "better-auth/client/plugins"

export const authClient = createAuthClient({
    plugins: [
        organizationClient({
            schema: {
                organizationRole: {
                    additionalFields: {
                        color: {
                            type: "string",
                            defaultValue: "#ffffff",
                        }
                    }
                }
            }
        })
    ]
})
```

***

## Teams

Teams allow you to group members within an organization. The teams feature provides additional organization structure and can be used to manage permissions at a more granular level.

### Enabling Teams

To enable teams, pass the `teams` configuration option to both server and client plugins:

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { organization } from "better-auth/plugins";

export const auth = betterAuth({
  plugins: [
    organization({
      teams: {
        enabled: true,
        maximumTeams: 10, // Optional: limit teams per organization
        allowRemovingAllTeams: false, // Optional: prevent removing the last team
      },
    }),
  ],
});
```

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client";
import { organizationClient } from "better-auth/client/plugins";

export const authClient = createAuthClient({
  plugins: [
    organizationClient({
      teams: {
        enabled: true,
      },
    }),
  ],
});
```

### Managing Teams

#### Create Team

Create a new team within an organization:

### Client Side

```ts
const { data, error } = await authClient.organization.createTeam({
    name: my-team,
    organizationId: organization-id, // required
});
```

### Server Side

```ts
const data = await auth.api.createTeam({
    body: {
        name: my-team,
        organizationId: organization-id, // required
    }
});
```

### Type Definition

```ts
type createTeam = {
    /**
     * The name of the team.
     */
    name: string = "my-team"
    /**
     * The organization ID which the team will be created in. Defaults to the active organization.
     */
    organizationId?: string = "organization-id"

}
```

#### List Teams

Get all teams in an organization:

### Client Side

```ts
const { data, error } = await authClient.organization.listTeams({
    organizationId: organization-id, // required
});
```

### Server Side

```ts
const data = await auth.api.listOrganizationTeams({
    query: {
        organizationId: organization-id, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type listOrganizationTeams = {
    /**
    * The organization ID which the teams are under to list. Defaults to the user's active organization.
    */
    organizationId?: string = "organization-id"

}
```

#### Update Team

Update a team's details:

### Client Side

```ts
const { data, error } = await authClient.organization.updateTeam({
    teamId: team-id,
    data,
    name: My new team name, // required
    organizationId: My new organization ID for this team, // required
    createdAt, // required
    updatedAt, // required
});
```

### Server Side

```ts
const data = await auth.api.updateTeam({
    body: {
        teamId: team-id,
        data,
        name: My new team name, // required
        organizationId: My new organization ID for this team, // required
        createdAt, // required
        updatedAt, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type updateTeam = {
    /**
     * The ID of the team to be updated.
     */
    teamId: string = "team-id"
    /**
     * A partial object containing options for you to update.
     */
    data: {
        /**
         * The name of the team to be updated.
         */
        name?: string = "My new team name"
        /**
         * The organization ID which the team falls under.
         */
        organizationId?: string = "My new organization ID for this team"
        /**
         * The timestamp of when the team was created.
         */
        createdAt?: Date = new Date()
        /**
         * The timestamp of when the team was last updated.
         */
        updatedAt?: Date = new Date()

}
```

#### Remove Team

Delete a team from an organization:

### Client Side

```ts
const { data, error } = await authClient.organization.removeTeam({
    teamId: team-id,
    organizationId: organization-id, // required
});
```

### Server Side

```ts
const data = await auth.api.removeTeam({
    body: {
        teamId: team-id,
        organizationId: organization-id, // required
    }
});
```

### Type Definition

```ts
type removeTeam = {
    /**
     * The team ID of the team to remove.
     */
    teamId: string = "team-id"
    /**
     * The organization ID which the team falls under. If not provided, it will default to the user's active organization.
     */
    organizationId?: string = "organization-id"

}
```

#### Set Active Team

Sets the given team as the current active team. If `teamId` is `null` the current active team is unset.

### Client Side

```ts
const { data, error } = await authClient.organization.setActiveTeam({
    teamId: team-id, // required
});
```

### Server Side

```ts
const data = await auth.api.setActiveTeam({
    body: {
        teamId: team-id, // required
    }
});
```

### Type Definition

```ts
type setActiveTeam = {
    /**
     * The team ID of the team to set as the current active team.
     */
    teamId?: string = "team-id"

}
```

#### List User Teams

List all teams that the current user is a part of.

### Client Side

```ts
const { data, error } = await authClient.organization.listUserTeams({});
```

### Server Side

```ts
const data = await auth.api.listUserTeams({});
```

### Type Definition

```ts
type listUserTeams = {

}
```

#### List Team Members

List the members of the given team.

### Client Side

```ts
const { data, error } = await authClient.organization.listTeamMembers({
    teamId: team-id, // required
});
```

### Server Side

```ts
const data = await auth.api.listTeamMembers({
    body: {
        teamId: team-id, // required
    }
});
```

### Type Definition

```ts
type listTeamMembers = {
    /**
     * The team whose members we should return. If this is not provided the members of the current active team get returned.
     */
    teamId?: string = "team-id"

}
```

#### Add Team Member

Add a member to a team.

### Client Side

```ts
const { data, error } = await authClient.organization.addTeamMember({
    teamId: team-id,
    userId: user-id,
});
```

### Server Side

```ts
const data = await auth.api.addTeamMember({
    body: {
        teamId: team-id,
        userId: user-id,
    }
});
```

### Type Definition

```ts
type addTeamMember = {
    /**
     * The team the user should be a member of.
     */
    teamId: string = "team-id"
    /**
     * The user ID which represents the user to be added as a member.
     */
    userId: string = "user-id"

}
```

#### Remove Team Member

Remove a member from a team.

### Client Side

```ts
const { data, error } = await authClient.organization.removeTeamMember({
    teamId: team-id,
    userId: user-id,
});
```

### Server Side

```ts
const data = await auth.api.removeTeamMember({
    body: {
        teamId: team-id,
        userId: user-id,
    }
});
```

### Type Definition

```ts
type removeTeamMember = {
    /**
     * The team the user should be removed from.
     */
    teamId: string = "team-id"
    /**
     * The user which should be removed from the team.
     */
    userId: string = "user-id"

}
```

### Team Permissions

Teams follow the organization's permission system. To manage teams, users need the following permissions:

* `team:create` - Create new teams
* `team:update` - Update team details
* `team:delete` - Remove teams

By default:

* Organization owners and admins can manage teams
* Regular members cannot create, update, or delete teams

### Team Configuration Options

The teams feature supports several configuration options:

* `maximumTeams`: Limit the number of teams per organization

  ```ts
  teams: {
    enabled: true,
    maximumTeams: 10 // Fixed number
    // OR
    maximumTeams: async ({ organizationId, session }, request) => {
      // Dynamic limit based on organization plan
      const plan = await getPlan(organizationId)
      return plan === 'pro' ? 20 : 5
    },
    maximumMembersPerTeam: 10 // Fixed number
    // OR
    maximumMembersPerTeam: async ({ teamId, session, organizationId }, request) => {
      // Dynamic limit based on team plan
      const plan = await getPlan(organizationId, teamId)
      return plan === 'pro' ? 50 : 10
    },
  }
  ```

* `allowRemovingAllTeams`: Control whether the last team can be removed
  ```ts
  teams: {
    enabled: true,
    allowRemovingAllTeams: false // Prevent removing the last team
  }
  ```

### Team Members

When inviting members to an organization, you can specify a team:

```ts
await authClient.organization.inviteMember({
  email: "user@example.com",
  role: "member",
  teamId: "team-id",
});
```

The invited member will be added to the specified team upon accepting the invitation.

### Database Schema

When teams are enabled, new `team` and `teamMember` tables are added to the database.

Table Name: `team`

```
<DatabaseTable
  fields={[
  {
    name: "id",
    type: "string",
    description: "Unique identifier for each team",
    isPrimaryKey: true,
  },
  {
    name: "name",
    type: "string",
    description: "The name of the team",
  },
  {
    name: "organizationId",
    type: "string",
    description: "The ID of the organization",
    isForeignKey: true,
  },
  {
    name: "createdAt",
    type: "Date",
    description: "Timestamp of when the team was created",
  },
  {
    name: "updatedAt",
    type: "Date",
    isOptional: true,
    description: "Timestamp of when the team was created",
  },
]}
/>
```

Table Name: `teamMember`

```
<DatabaseTable
  fields={[
  {
    name: "id",
    type: "string",
    description: "Unique identifier for each team member",
    isPrimaryKey: true,
  },
  {
    name: "teamId",
    type: "string",
    description: "Unique identifier for each team",
    isForeignKey: true,
  },
  {
    name: "userId",
    type: "string",
    description: "The ID of the user",
    isForeignKey: true,
  },
  {
    name: "createdAt",
    type: "Date",
    description: "Timestamp of when the team member was created",
  },
]}
/>
```

## Schema

The organization plugin adds the following tables to the database:

### Organization

Table Name: `organization`

```
<DatabaseTable
  fields={[
  {
    name: "id",
    type: "string",
    description: "Unique identifier for each organization",
    isPrimaryKey: true,
  },
  {
    name: "name",
    type: "string",
    description: "The name of the organization",
  },
  {
    name: "slug",
    type: "string",
    description: "The slug of the organization",
  },
  {
    name: "logo",
    type: "string",
    description: "The logo of the organization",
    isOptional: true,
  },
  {
    name: "metadata",
    type: "string",
    description: "Additional metadata for the organization",
    isOptional: true,
  },
  {
    name: "createdAt",
    type: "Date",
    description: "Timestamp of when the organization was created",
  },
]}
/>
```

### Member

Table Name: `member`

```
<DatabaseTable
  fields={[
  {
    name: "id",
    type: "string",
    description: "Unique identifier for each member",
    isPrimaryKey: true,
  },
  {
    name: "userId",
    type: "string",
    description: "The ID of the user",
    isForeignKey: true,
  },
  {
    name: "organizationId",
    type: "string",
    description: "The ID of the organization",
    isForeignKey: true,
  },
  {
    name: "role",
    type: "string",
    description: "The role of the user in the organization",
  },
  {
    name: "createdAt",
    type: "Date",
    description: "Timestamp of when the member was added to the organization",
  },
]}
/>

### Invitation

Table Name: `invitation`

<DatabaseTable
  fields={[
  {
    name: "id",
    type: "string",
    description: "Unique identifier for each invitation",
    isPrimaryKey: true,
  },
  {
    name: "email",
    type: "string",
    description: "The email address of the user",
  },
  {
    name: "inviterId",
    type: "string",
    description: "The ID of the inviter",
    isForeignKey: true,
  },
  {
    name: "organizationId",
    type: "string",
    description: "The ID of the organization",
    isForeignKey: true,
  },
  {
    name: "role",
    type: "string",
    description: "The role of the user in the organization",
  },
  {
    name: "status",
    type: "string",
    description: "The status of the invitation",
  },
  {
    name: "expiresAt",
    type: "Date",
    description: "Timestamp of when the invitation expires",
  },
]}
/>

If teams are enabled, you need to add the following fields to the invitation table:

<DatabaseTable
  fields={[
  {
    name: "teamId",
    type: "string",
    description: "The ID of the team",
    isOptional: true,
  },
]}
/>

### Session

Table Name: `session`

You need to add two more fields to the session table to store the active organization ID and the active team ID.

<DatabaseTable
  fields={[
  {
    name: "activeOrganizationId",
    type: "string",
    description: "The ID of the active organization",
    isOptional: true,
  },
  {
    name: "activeTeamId",
    type: "string",
    description: "The ID of the active team",
    isOptional: true,
  },
]}
/>

### Teams (optional)

Table Name: `team`
```

~~~mdx
<DatabaseTable
  fields={[
  {
    name: "id",
    type: "string",
    description: "Unique identifier for each team",
    isPrimaryKey: true,
  },
  {
    name: "name",
    type: "string",
    description: "The name of the team",
  },
  {
    name: "organizationId",
    type: "string",
    description: "The ID of the organization",
    isForeignKey: true,
  },
  {
    name: "createdAt",
    type: "Date",
    description: "Timestamp of when the team was created",
  },
  {
    name: "updatedAt",
    type: "Date",
    isOptional: true,
    description: "Timestamp of when the team was created",
  },
]}
/>

Table Name: `teamMember`

<DatabaseTable
  fields={[
  {
    name: "id",
    type: "string",
    description: "Unique identifier for each team member",
    isPrimaryKey: true,
  },
  {
    name: "teamId",
    type: "string",
    description: "Unique identifier for each team",
    isForeignKey: true,
  },
  {
    name: "userId",
    type: "string",
    description: "The ID of the user",
    isForeignKey: true,
  },
  {
    name: "createdAt",
    type: "Date",
    description: "Timestamp of when the team member was created",
  },
]}
/>

Table Name: `invitation`

<DatabaseTable
  fields={[
  {
    name: "teamId",
    type: "string",
    description: "The ID of the team",
    isOptional: true,
  },
]}
/>

### Customizing the Schema

To change the schema table name or fields, you can pass `schema` option to the organization plugin.

```ts title="auth.ts"
const auth = betterAuth({
  plugins: [
    organization({
      schema: {
        organization: {
          modelName: "organizations", //map the organization table to organizations
          fields: {
            name: "title", //map the name field to title
          },
          additionalFields: {
            // Add a new field to the organization table
            myCustomField: {
              type: "string",
              input: true,
              required: false,
            },
          },
        },
      },
    }),
  ],
});
```

#### Additional Fields

Starting with [Better Auth v1.3](https://github.com/better-auth/better-auth/releases/tag/v1.3.0), you can easily add custom fields to the `organization`, `invitation`, `member`, and `team` tables.

When you add extra fields to a model, the relevant API endpoints will automatically accept and return these new properties. For instance, if you add a custom field to the `organization` table, the `createOrganization` endpoint will include this field in its request and response payloads as needed.
~~~

```ts title="auth.ts"
const auth = betterAuth({
  plugins: [
    organization({
      schema: {
        organization: {
          additionalFields: {
            myCustomField: {
              // [!code highlight]
              type: "string", // [!code highlight]
              input: true, // [!code highlight]
              required: false, // [!code highlight]
            }, // [!code highlight]
          },
        },
      },
    }),
  ],
});
```

For inferring the additional fields, you can use the `inferOrgAdditionalFields` function. This function will infer the additional fields from the auth object type.

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client";
import {
  inferOrgAdditionalFields,
  organizationClient,
} from "better-auth/client/plugins";
import type { auth } from "@/auth"; // import the auth object type only

const client = createAuthClient({
  plugins: [
    organizationClient({
      schema: inferOrgAdditionalFields<typeof auth>(),
    }),
  ],
});
```

if you can't import the auth object type, you can use the `inferOrgAdditionalFields` function without the generic. This function will infer the additional fields from the schema object.

```ts title="auth-client.ts"
const client = createAuthClient({
  plugins: [
    organizationClient({
      schema: inferOrgAdditionalFields({
        organization: {
          // [!code highlight]
          additionalFields: {
            newField: {
              // [!code highlight]
              type: "string", // [!code highlight]
            }, // [!code highlight]
          },
        },
      }),
    }),
  ],
});

//example usage
await client.organization.create({
  name: "Test",
  slug: "test",
  newField: "123", //this should be allowed
  //@ts-expect-error - this field is not available
  unavalibleField: "123", //this should be not allowed
});
```

## Options

allowUserToCreateOrganization: `boolean` | `((user: User) => Promise<boolean> | boolean)` - A function that determines whether a user can create an organization. By default, it's `true`. You can set it to `false` to restrict users from creating organizations.

organizationLimit: `number` | `((user: User) => Promise<boolean> | boolean)` - The maximum number of organizations allowed for a user. By default, it's `5`. You can set it to any number you want or a function that returns a boolean.

creatorRole: `admin | owner` - The role of the user who creates the organization. By default, it's `owner`. You can set it to `admin`.

membershipLimit: `number` - The maximum number of members allowed in an organization. By default, it's `100`. You can set it to any number you want.

sendInvitationEmail: `async (data) => Promise<void>` - A function that sends an invitation email to the user.

invitationExpiresIn : `number` - How long the invitation link is valid for in seconds. By default, it's 48 hours (2 days).

cancelPendingInvitationsOnReInvite: `boolean` - Whether to cancel pending invitations if the user is already invited to the organization. By default, it's `false`.

invitationLimit: `number` | `((user: User) => Promise<boolean> | boolean)` - The maximum number of invitations allowed for a user. By default, it's `100`. You can set it to any number you want or a function that returns a boolean.

requireEmailVerificationOnInvitation: `boolean` - Whether to require email verification before accepting or rejecting invitations. By default, it's `false`. When enabled, users must have verified their email address before they can accept or reject organization invitations.

# plugins: Passkey
URL: /docs/plugins/passkey
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/passkey.mdx

Passkey

***

title: Passkey
description: Passkey

Passkeys are a secure, passwordless authentication method using cryptographic key pairs, supported by WebAuthn and FIDO2 standards in web browsers. They replace passwords with unique key pairs: a private key stored on the user's device and a public key shared with the website. Users can log in using biometrics, PINs, or security keys, providing strong, phishing-resistant authentication without traditional passwords.

The passkey plugin implementation is powered by [SimpleWebAuthn](https://simplewebauthn.dev/) behind the scenes.

## Installation

<Steps>
  <Step>
    ### Add the plugin to your auth config

    To add the passkey plugin to your auth config, you need to import the plugin and pass it to the `plugins` option of the auth instance.

    Options

    `rpID`: A unique identifier for your website. 'localhost' is okay for local dev

    `rpName`: Human-readable title for your website

    `origin`: The URL at which registrations and authentications should occur. `http://localhost` and `http://localhost:PORT` are also valid. Do **NOT** include any trailing /

~~~
    `authenticatorSelection`: Allows customization of WebAuthn authenticator selection criteria. Leave unspecified for default settings.

    * `authenticatorAttachment`: Specifies the type of authenticator
      * `platform`: Authenticator is attached to the platform (e.g., fingerprint reader)
      * `cross-platform`: Authenticator is not attached to the platform (e.g., security key)
      * Default: `not set` (both platform and cross-platform allowed, with platform preferred)
    * `residentKey`: Determines credential storage behavior.
      * `required`: User MUST store credentials on the authenticator (highest security)
      * `preferred`: Encourages credential storage but not mandatory
      * `discouraged`: No credential storage required (fastest experience)
      * Default: `preferred`
    * `userVerification`: Controls biometric/PIN verification during authentication:
      * `required`: User MUST verify identity (highest security)
      * `preferred`: Verification encouraged but not mandatory
      * `discouraged`: No verification required (fastest experience)
      * Default: `preferred`

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { passkey } from "better-auth/plugins/passkey" // [!code highlight]

    export const auth = betterAuth({
        plugins: [ // [!code highlight]
            passkey(), // [!code highlight]
        ], // [!code highlight]
    })
    ```
  </Step>

  <Step>
    ### Migrate the database

    Run the migration or generate the schema to add the necessary fields and tables to the database.

    <Tabs items={["migrate", "generate"]}>
      <Tab value="migrate">
        ```bash
        npx @better-auth/cli migrate
        ```
      </Tab>

      <Tab value="generate">
        ```bash
        npx @better-auth/cli generate
        ```
      </Tab>
    </Tabs>

    See the [Schema](#schema) section to add the fields manually.
  </Step>

  <Step>
    ### Add the client plugin

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    import { passkeyClient } from "better-auth/client/plugins"

    export const authClient = createAuthClient({
        plugins: [ // [!code highlight]
            passkeyClient() // [!code highlight]
        ] // [!code highlight]
    })
    ```
  </Step>
</Steps>

## Usage

### Add/Register a passkey

To add or register a passkey make sure a user is authenticated and then call the `passkey.addPasskey` function provided by the client.

### Client Side

```ts
const { data, error } = await authClient.passkey.addPasskey({
    name: example-passkey-name, // required
    authenticatorAttachment: cross-platform, // required
});
```

### Server Side

```ts
const data = await auth.api.addPasskey({
    body: {
        name: example-passkey-name, // required
        authenticatorAttachment: cross-platform, // required
    }
});
```

### Type Definition

```ts
type addPasskey = {
    /**
     * An optional name to label the authenticator account being registered. If not provided, it will default to the user's email address or user ID
    */
    name?: string = "example-passkey-name"
    /**
     * You can also specify the type of authenticator you want to register. Default behavior allows both platform and cross-platform passkeys
    */
    authenticatorAttachment?: "platform" | "cross-platform" = "cross-platform"

}
```

<Callout>
  Setting `throw: true` in the fetch options has no effect for the register and sign-in passkey responses â€” they will always return a data object containing the error object.
</Callout>

### Sign in with a passkey

To sign in with a passkey you can use the `signIn.passkey` method. This will prompt the user to sign in with their passkey.

### Client Side

```ts
const { data, error } = await authClient.signIn.passkey({
    autoFill, // required
});
```

### Server Side
~~~

```ts
const data = await auth.api.signInPasskey({
    body: {
        autoFill, // required
    }
});
```

### Type Definition

```ts
type signInPasskey = {
    /**
     * Browser autofill, a.k.a. Conditional UI. Read more: https://simplewebauthn.dev/docs/packages/browser#browser-autofill-aka-conditional-ui
    */
    autoFill?: boolean = true

}
```

#### Example Usage

```ts
// With post authentication redirect
await authClient.signIn.passkey({
    autoFill: true,
    fetchOptions: {
        onSuccess(context) {
            // Redirect to dashboard after successful authentication
            window.location.href = "/dashboard";
        },
        onError(context) {
            // Handle authentication errors
            console.error("Authentication failed:", context.error.message);
        }
    }
});
```

### List passkeys

You can list all of the passkeys for the authenticated user by calling `passkey.listUserPasskeys`:

### Client Side

```ts
const { data, error } = await authClient.passkey.listUserPasskeys({});
```

### Server Side

```ts
const passkeys = await auth.api.listPasskeys({

    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type listPasskeys = {

}
```

### Deleting passkeys

You can delete a passkey by calling `passkey.delete` and providing the passkey ID.

### Client Side

```ts
const { data, error } = await authClient.passkey.deletePasskey({
    id: some-passkey-id,
});
```

### Server Side

```ts
const data = await auth.api.deletePasskey({
    body: {
        id: some-passkey-id,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type deletePasskey = {
    /**
     * The ID of the passkey to delete.
     */
    id: string = "some-passkey-id"

}
```

### Updating passkey names

### Client Side

```ts
const { data, error } = await authClient.passkey.updatePasskey({
    id: id of passkey,
    name: my-new-passkey-name,
});
```

### Server Side

```ts
const data = await auth.api.updatePasskey({
    body: {
        id: id of passkey,
        name: my-new-passkey-name,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

~~~
```ts
type updatePasskey = {
    /**
     * The ID of the passkey which you want to update.
     */
    id: string = "id of passkey"
    /**
     * The new name which the passkey will be updated to.
     */
    name: string = "my-new-passkey-name"

}
```

### Conditional UI

The plugin supports conditional UI, which allows the browser to autofill the passkey if the user has already registered a passkey.

There are two requirements for conditional UI to work:

<Steps>
  <Step>
    #### Update input fields

    Add the `autocomplete` attribute with the value `webauthn` to your input fields. You can add this attribute to multiple input fields, but at least one is required for conditional UI to work.

    The `webauthn` value should also be the last entry of the `autocomplete` attribute.

    ```html
    <label for="name">Username:</label>
    <input type="text" name="name" autocomplete="username webauthn">
    <label for="password">Password:</label>
    <input type="password" name="password" autocomplete="current-password webauthn">
    ```
  </Step>

  <Step>
    #### Preload the passkeys

    When your component mounts, you can preload the user's passkeys by calling the `authClient.signIn.passkey` method with the `autoFill` option set to `true`.

    To prevent unnecessary calls, we will also add a check to see if the browser supports conditional UI.

    <Tabs items={["React"]}>
      <Tab value="React">
        ```ts
        useEffect(() => {
           if (!PublicKeyCredential.isConditionalMediationAvailable ||
               !PublicKeyCredential.isConditionalMediationAvailable()) {
             return;
           }

          void authClient.signIn.passkey({ autoFill: true })
        }, [])
        ```
      </Tab>
    </Tabs>
  </Step>
</Steps>

Depending on the browser, a prompt will appear to autofill the passkey. If the user has multiple passkeys, they can select the one they want to use.

Some browsers also require the user to first interact with the input field before the autofill prompt appears.

### Debugging

To test your passkey implementation you can use [emulated authenticators](https://developer.chrome.com/docs/devtools/webauthn). This way you can test the registration and sign-in process without even owning a physical device.

## Schema

The plugin require a new table in the database to store passkey data.

Table Name: `passkey`

<DatabaseTable
  fields={[
      {
          name: "id",
          type: "string",
          description: "Unique identifier for each passkey",
          isPrimaryKey: true
      },
      {
          name: "name",
          type: "string",
          description: "The name of the passkey",
          isOptional: true
      },
      {
          name: "publicKey",
          type: "string",
          description: "The public key of the passkey",
      },
      {
          name: "userId",
          type: "string",
          description: "The ID of the user",
          isForeignKey: true
      },
      {
          name: "credentialID",
          type: "string",
          description: "The unique identifier of the registered credential",
      },
      {
          name: "counter",
          type: "number",
          description: "The counter of the passkey",
      },
      {
          name: "deviceType",
          type: "string",
          description: "The type of device used to register the passkey",
      },
      {
          name: "backedUp",
          type: "boolean",
          description: "Whether the passkey is backed up",
      },
      {
          name: "transports",
          type: "string",
          description: "The transports used to register the passkey",
      },
      {
~~~

~~~
          name: "createdAt",
          type: "Date",
          description: "The time when the passkey was created",
      },
      {
              name: "aaguid",
              type: "string",
              description: "Authenticator's Attestation GUID indicating the type of the authenticator",
              isOptional: true
      },
  ]}
/>

## Options

**rpID**: A unique identifier for your website. 'localhost' is okay for local dev.

**rpName**: Human-readable title for your website.

**origin**: The URL at which registrations and authentications should occur. `http://localhost` and `http://localhost:PORT` are also valid. Do NOT include any trailing /.

**authenticatorSelection**: Allows customization of WebAuthn authenticator selection criteria. When unspecified, both platform and cross-platform authenticators are allowed with
`preferred` settings for `residentKey` and `userVerification`.

**aaguid**: (optional) Authenticator Attestation GUID. This is a unique identifier for the passkey provider (device or authenticator type) and can be used to identify the type of
passkey device used during registration or authentication.



# plugins: Phone Number
URL: /docs/plugins/phone-number
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/phone-number.mdx

Phone number plugin

***

title: Phone Number
description: Phone number plugin
--------------------------------

The phone number plugin extends the authentication system by allowing users to sign in and sign up using their phone number. It includes OTP (One-Time Password) functionality to
verify phone numbers.

## Installation

<Steps>
  <Step>
    ### Add Plugin to the server

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { phoneNumber } from "better-auth/plugins"

    const auth = betterAuth({
        plugins: [
            phoneNumber({  // [!code highlight]
                sendOTP: ({ phoneNumber, code }, request) => { // [!code highlight]
                    // Implement sending OTP code via SMS // [!code highlight]
                } // [!code highlight]
            }) // [!code highlight]
        ]
    })
    ```
  </Step>

  <Step>
    ### Migrate the database

    Run the migration or generate the schema to add the necessary fields and tables to the database.

    <Tabs items={["migrate", "generate"]}>
      <Tab value="migrate">
        ```bash
        npx @better-auth/cli migrate
        ```
      </Tab>

      <Tab value="generate">
        ```bash
        npx @better-auth/cli generate
        ```
      </Tab>
    </Tabs>

    See the [Schema](#schema) section to add the fields manually.
  </Step>

  <Step>
    ### Add the client plugin

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    import { phoneNumberClient } from "better-auth/client/plugins"

    const authClient =  createAuthClient({
        plugins: [ // [!code highlight]
            phoneNumberClient() // [!code highlight]
        ] // [!code highlight]
    })
    ```
  </Step>
</Steps>

## Usage

### Send OTP for Verification

To send an OTP to a user's phone number for verification, you can use the `sendVerificationCode` endpoint.

### Client Side

```ts
const { data, error } = await authClient.phoneNumber.sendOtp({
    phoneNumber: +1234567890,
});
```

### Server Side

```ts
const data = await auth.api.sendPhoneNumberOTP({
    body: {
        phoneNumber: +1234567890,
    }
});
```

### Type Definition
~~~

```ts
type sendPhoneNumberOTP = {
    /**
     * Phone number to send OTP.
     */
    phoneNumber: string = "+1234567890"

}
```

### Verify Phone Number

After the OTP is sent, users can verify their phone number by providing the code.

### Client Side

```ts
const { data, error } = await authClient.phoneNumber.verify({
    phoneNumber: +1234567890,
    code: 123456,
    disableSession, // required
    updatePhoneNumber, // required
});
```

### Server Side

```ts
const data = await auth.api.verifyPhoneNumber({
    body: {
        phoneNumber: +1234567890,
        code: 123456,
        disableSession, // required
        updatePhoneNumber, // required
    }
});
```

### Type Definition

```ts
type verifyPhoneNumber = {
    /**
     * Phone number to verify.
     */
    phoneNumber: string = "+1234567890"
    /**
     * OTP code.
     */
    code: string = "123456"
    /**
     * Disable session creation after verification.
     */
    disableSession?: boolean = false
    /**
     * Check if there is a session and update the phone number.
     */
    updatePhoneNumber?: boolean = true

}
```

<Callout>
  When the phone number is verified, the `phoneNumberVerified` field in the user table is set to `true`. If `disableSession` is not set to `true`, a session is created for the user. Additionally, if `callbackOnVerification` is provided, it will be called.
</Callout>

### Allow Sign-Up with Phone Number

To allow users to sign up using their phone number, you can pass `signUpOnVerification` option to your plugin configuration. It requires you to pass `getTempEmail` function to generate a temporary email for the user.

```ts title="auth.ts"
export const auth = betterAuth({
    plugins: [
        phoneNumber({
            sendOTP: ({ phoneNumber, code }, request) => {
                // Implement sending OTP code via SMS
            },
            signUpOnVerification: {
                getTempEmail: (phoneNumber) => {
                    return `${phoneNumber}@my-site.com`
                },
                //optionally, you can also pass `getTempName` function to generate a temporary name for the user
                getTempName: (phoneNumber) => {
                    return phoneNumber //by default, it will use the phone number as the name
                }
            }
        })
    ]
})
```

### Sign In with Phone Number

In addition to signing in a user using send-verify flow, you can also use phone number as an identifier and sign in a user using phone number and password.

### Client Side

```ts
const { data, error } = await authClient.signIn.phoneNumber({
    phoneNumber: +1234567890,
    password,
    rememberMe, // required
});
```

### Server Side

```ts
const data = await auth.api.signInPhoneNumber({
    body: {
        phoneNumber: +1234567890,
        password,
        rememberMe, // required
    }
});
```

### Type Definition

```ts
type signInPhoneNumber = {
    /**
     * Phone number to sign in.
     */
    phoneNumber: string = "+1234567890"
    /**
     * Password to use for sign in.
     */
    password: string
    /**
     * Remember the session.
     */
    rememberMe?: boolean = true

}
```

### Update Phone Number

Updating phone number uses the same process as verifying a phone number. The user will receive an OTP code to verify the new phone number.

```ts title="auth-client.ts"
await authClient.phoneNumber.sendOtp({
    phoneNumber: "+1234567890" // New phone number
})
```

Then verify the new phone number with the OTP code.

```ts title="auth-client.ts"
const isVerified = await authClient.phoneNumber.verify({
    phoneNumber: "+1234567890",
    code: "123456",
    updatePhoneNumber: true // Set to true to update the phone number [!code highlight]
})
```

If a user session exist the phone number will be updated automatically.

### Disable Session Creation

By default, the plugin creates a session for the user after verifying the phone number. You can disable this behavior by passing `disableSession: true` to the `verify` method.

```ts title="auth-client.ts"
const isVerified = await authClient.phoneNumber.verify({
    phoneNumber: "+1234567890",
    code: "123456",
    disableSession: true // [!code highlight]
})
```

### Request Password Reset

To initiate a request password reset flow using `phoneNumber`, you can start by calling `requestPasswordReset` on the client to send an OTP code to the user's phone number.

### Client Side

```ts
const { data, error } = await authClient.phoneNumber.requestPasswordReset({
    phoneNumber: +1234567890,
});
```

### Server Side

```ts
const data = await auth.api.requestPasswordResetPhoneNumber({
    body: {
        phoneNumber: +1234567890,
    }
});
```

### Type Definition

```ts
type requestPasswordResetPhoneNumber = {
    /**
     * The phone number which is associated with the user.
     */
    phoneNumber: string = "+1234567890"

}
```

Then, you can reset the password by calling `resetPassword` on the client with the OTP code and the new password.

### Client Side

```ts
const { data, error } = await authClient.phoneNumber.resetPassword({
    otp: 123456,
    phoneNumber: +1234567890,
    newPassword: new-and-secure-password,
});
```

### Server Side

```ts
const data = await auth.api.resetPasswordPhoneNumber({
    body: {
        otp: 123456,
        phoneNumber: +1234567890,
        newPassword: new-and-secure-password,
    }
});
```

### Type Definition

```ts
type resetPasswordPhoneNumber = {
    /**
     * The one time password to reset the password.
     */
    otp: string = "123456"
    /**
     * The phone number to the account which intends to reset the password for.
     */
    phoneNumber: string = "+1234567890"
    /**
     * The new password.
     */
    newPassword: string = "new-and-secure-password"

}
```

## Options

* `otpLength`: The length of the OTP code to be generated. Default is `6`.
* `sendOTP`: A function that sends the OTP code to the user's phone number. It takes the phone number and the OTP code as arguments.
* `expiresIn`: The time in seconds after which the OTP code expires. Default is `300` seconds.
* `callbackOnVerification`: A function that is called after the phone number is verified. It takes the phone number and the user object as the first argument and a request object as the second argument.

```ts
export const auth = betterAuth({
    plugins: [
        phoneNumber({
            sendOTP: ({ phoneNumber, code }, request) => {
                // Implement sending OTP code via SMS
            },
            callbackOnVerification: async ({ phoneNumber, user }, request) => {
                // Implement callback after phone number verification
            }
        })
    ]
})
```

* `sendPasswordResetOTP`: A function that sends the OTP code to the user's phone number for password reset. It takes the phone number and the OTP code as arguments.

* `phoneNumberValidator`: A custom function to validate the phone number. It takes the phone number as an argument and returns a boolean indicating whether the phone number is valid.

* `signUpOnVerification`: An object with the following properties:
  * `getTempEmail`: A function that generates a temporary email for the user. It takes the phone number as an argument and returns the temporary email.
  * `getTempName`: A function that generates a temporary name for the user. It takes the phone number as an argument and returns the temporary name.

* `requireVerification`: When enabled, users cannot sign in with their phone number until it has been verified. If an unverified user attempts to sign in, the server will respond with a 401 error (PHONE_NUMBER_NOT_VERIFIED) and automatically trigger an OTP send to start the verification process.

## Schema

The plugin requires 2 fields to be added to the user table

### User Table

```
<DatabaseTable
  fields={[
      {
          name: "phoneNumber",
          type: "string",
          description: "The phone number of the user",
          isUnique: true,
          isOptional: true
      },
      {
          name: "phoneNumberVerified",
          type: "boolean",
          description: "Whether the phone number is verified or not",
          defaultValue: false,
          isOptional: true
      },
  ]}
/>
```

### OTP Verification Attempts

The phone number plugin includes a built-in protection against brute force attacks by limiting the number of verification attempts for each OTP code.

```typescript
phoneNumber({
  allowedAttempts: 3, // default is 3
  // ... other options
})
```

When a user exceeds the allowed number of verification attempts:

* The OTP code is automatically deleted
* Further verification attempts will return a 403 (Forbidden) status with "Too many attempts" message
* The user will need to request a new OTP code to continue

Example error response after exceeding attempts:

```json
{
  "error": {
    "status": 403,
    "message": "Too many attempts"
  }
}
```

<Callout type="warning">
  When receiving a 403 status, prompt the user to request a new OTP code
</Callout>

# plugins: Polar
URL: /docs/plugins/polar
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/polar.mdx

Better Auth Plugin for Payment and Checkouts using Polar

***

title: Polar
description: Better Auth Plugin for Payment and Checkouts using Polar
----

[Polar](https://polar.sh) is a developer first payment infrastructure. Out of the box it provides a lot of developer first integrations for payments, checkouts and more. This plugin helps you integrate Polar with Better Auth to make your auth + payments flow seamless.

<Callout>
  This plugin is maintained by Polar team. For bugs, issues or feature requests,
  please visit the [Polar GitHub
  repo](https://github.com/polarsource/polar-adapters).
</Callout>

## Features

* Checkout Integration
* Customer Portal
* Automatic Customer creation on signup
* Event Ingestion & Customer Meters for flexible Usage Based Billing
* Handle Polar Webhooks securely with signature verification
* Reference System to associate purchases with organizations

## Installation

```bash
pnpm add better-auth @polar-sh/better-auth @polar-sh/sdk
```

## Preparation

Go to your Polar Organization Settings, and create an Organization Access Token. Add it to your environment.

```bash
# .env
POLAR_ACCESS_TOKEN=...
```

### Configuring BetterAuth Server

The Polar plugin comes with a handful additional plugins which adds functionality to your stack.

- Checkout - Enables a seamless checkout integration
- Portal - Makes it possible for your customers to manage their orders, subscriptions & granted benefits
- Usage - Simple extension for listing customer meters & ingesting events for Usage Based Billing
- Webhooks - Listen for relevant Polar webhooks

```typescript
import { betterAuth } from "better-auth";
import { polar, checkout, portal, usage, webhooks } from "@polar-sh/better-auth";
import { Polar } from "@polar-sh/sdk";

const polarClient = new Polar({
    accessToken: process.env.POLAR_ACCESS_TOKEN,
    // Use 'sandbox' if you're using the Polar Sandbox environment
    // Remember that access tokens, products, etc. are completely separated between environments.
    // Access tokens obtained in Production are for instance not usable in the Sandbox environment.
    server: 'sandbox'
});

const auth = betterAuth({
    // ... Better Auth config
    plugins: [
        polar({
            client: polarClient,
            createCustomerOnSignUp: true,
            use: [
                checkout({
                    products: [
                        {
                            productId: "123-456-789", // ID of Product from Polar Dashboard
                            slug: "pro" // Custom slug for easy reference in Checkout URL, e.g. /checkout/pro
                        }
                    ],
                    successUrl: "/success?checkout_id={CHECKOUT_ID}",
                    authenticatedUsersOnly: true
                }),
                portal(),
                usage(),
                webhooks({
                    secret: process.env.POLAR_WEBHOOK_SECRET,
                    onCustomerStateChanged: (payload) => // Triggered when anything regarding a customer changes
                    onOrderPaid: (payload) => // Triggered when an order was paid (purchase, subscription renewal, etc.)
                    ...  // Over 25 granular webhook handlers
                    onPayload: (payload) => // Catch-all for all events
                })
            ],
        })
    ]
});
```

### Configuring BetterAuth Client

You will be using the BetterAuth Client to interact with the Polar functionalities.

```typescript
import { createAuthClient } from "better-auth/react";
import { polarClient } from "@polar-sh/better-auth";

// This is all that is needed
// All Polar plugins, etc. should be attached to the server-side BetterAuth config
export const authClient = createAuthClient({
  plugins: [polarClient()],
});
```

## Configuration Options

```typescript
import { betterAuth } from "better-auth";
import {
  polar,
  checkout,
  portal,
  usage,
  webhooks,
} from "@polar-sh/better-auth";
import { Polar } from "@polar-sh/sdk";

const polarClient = new Polar({
  accessToken: process.env.POLAR_ACCESS_TOKEN,
  // Use 'sandbox' if you're using the Polar Sandbox environment
  // Remember that access tokens, products, etc. are completely separated between environments.
  // Access tokens obtained in Production are for instance not usable in the Sandbox environment.
  server: "sandbox",
});

const auth = betterAuth({
  // ... Better Auth config
  plugins: [
    polar({
      client: polarClient,
      createCustomerOnSignUp: true,
      getCustomerCreateParams: ({ user }, request) => ({
        metadata: {
          myCustomProperty: 123,
        },
      }),
      use: [
        // This is where you add Polar plugins
      ],
    }),
  ],
});
```

### Required Options

- `client`: Polar SDK client instance

### Optional Options

- `createCustomerOnSignUp`: Automatically create a Polar customer when a user signs up
- `getCustomerCreateParams`: Custom function to provide additional customer creation metadata

### Customers

When `createCustomerOnSignUp` is enabled, a new Polar Customer is automatically created when a new User is added in the Better-Auth Database.

All new customers are created with an associated `externalId`, which is the ID of your User in the Database. This allows us to skip any Polar to User mapping in your Database.



## Checkout Plugin

To support checkouts in your app, simply pass the Checkout plugin to the use-property.

```typescript
import { polar, checkout } from "@polar-sh/better-auth";

const auth = betterAuth({
    // ... Better Auth config
    plugins: [
        polar({
            ...
            use: [
                checkout({
                    // Optional field - will make it possible to pass a slug to checkout instead of Product ID
                    products: [ { productId: "123-456-789", slug: "pro" } ],
                    // Relative URL to return to when checkout is successfully completed
                    successUrl: "/success?checkout_id={CHECKOUT_ID}",
                    // Wheather you want to allow unauthenticated checkout sessions or not
                    authenticatedUsersOnly: true
                })
            ],
        })
    ]
});
```

When checkouts are enabled, you're able to initialize Checkout Sessions using the checkout-method on the BetterAuth Client. This will redirect the user to the Product Checkout.

```typescript
await authClient.checkout({
  // Any Polar Product ID can be passed here
  products: ["e651f46d-ac20-4f26-b769-ad088b123df2"],
  // Or, if you setup "products" in the Checkout Config, you can pass the slug
  slug: "pro",
});
```

Checkouts will automatically carry the authenticated User as the customer to the checkout. Email-address will be "locked-in".

If `authenticatedUsersOnly` is `false` - then it will be possible to trigger checkout sessions without any associated customer.

### Organization Support

This plugin supports the Organization plugin. If you pass the organization ID to the Checkout referenceId, you will be able to keep track of purchases made from organization members.

```typescript
const organizationId = (await authClient.organization.list())?.data?.[0]?.id,

await authClient.checkout({
    // Any Polar Product ID can be passed here
    products: ["e651f46d-ac20-4f26-b769-ad088b123df2"],
    // Or, if you setup "products" in the Checkout Config, you can pass the slug
    slug: 'pro',
    // Reference ID will be saved as `referenceId` in the metadata of the checkout, order & subscription object
    referenceId: organizationId
});
```

## Portal Plugin

A plugin which enables customer management of their purchases, orders and subscriptions.

```typescript
import { polar, checkout, portal } from "@polar-sh/better-auth";

const auth = betterAuth({
    // ... Better Auth config
    plugins: [
        polar({
            ...
            use: [
                checkout(...),
                portal()
            ],
        })
    ]
});
```

The portal-plugin gives the BetterAuth Client a set of customer management methods, scoped under `authClient.customer`.

### Customer Portal Management

The following method will redirect the user to the Polar Customer Portal, where they can see orders, purchases, subscriptions, benefits, etc.

```typescript
await authClient.customer.portal();
```

### Customer State

The portal plugin also adds a convenient state-method for retrieving the general Customer State.

```typescript
const { data: customerState } = await authClient.customer.state();
```

The customer state object contains:

* All the data about the customer.
* The list of their active subscriptions
  * Note: This does not include subscriptions done by a parent organization. See the subscription list-method below for more information.
* The list of their granted benefits.
* The list of their active meters, with their current balance.

Thus, with that single object, you have all the required information to check if you should provision access to your service or not.

[You can learn more about the Polar Customer State in the Polar Docs](https://docs.polar.sh/integrate/customer-state).

### Benefits, Orders & Subscriptions

The portal plugin adds 3 convenient methods for listing benefits, orders & subscriptions relevant to the authenticated user/customer.

[All of these methods use the Polar CustomerPortal APIs](https://docs.polar.sh/api-reference/customer-portal)

#### Benefits

This method only lists granted benefits for the authenticated user/customer.

```typescript
const { data: benefits } = await authClient.customer.benefits.list({
  query: {
    page: 1,
    limit: 10,
  },
});
```

#### Orders

This method lists orders like purchases and subscription renewals for the authenticated user/customer.

```typescript
const { data: orders } = await authClient.customer.orders.list({
  query: {
    page: 1,
    limit: 10,
    productBillingType: "one_time", // or 'recurring'
  },
});
```

#### Subscriptions

This method lists the subscriptions associated with authenticated user/customer.

```typescript
const { data: subscriptions } = await authClient.customer.subscriptions.list({
  query: {
    page: 1,
    limit: 10,
    active: true,
  },
});
```

Important - Organization Support

This will **not** return subscriptions made by a parent organization to the authenticated user.

However, you can pass a `referenceId` to this method. This will return all subscriptions associated with that referenceId instead of subscriptions associated with the user.

So in order to figure out if a user should have access, pass the user's organization ID to see if there is an active subscription for that organization.

```typescript
const organizationId = (await authClient.organization.list())?.data?.[0]?.id,

const { data: subscriptions } = await authClient.customer.orders.list({
    query: {
            page: 1,
                limit: 10,
                active: true,
        referenceId: organizationId
    },
});

const userShouldHaveAccess = subscriptions.some(
    sub => // Your logic to check subscription product or whatever.
)
```

## Usage Plugin

A simple plugin for Usage Based Billing.

```typescript
import { polar, checkout, portal, usage } from "@polar-sh/better-auth";

const auth = betterAuth({
    // ... Better Auth config
    plugins: [
        polar({
            ...
            use: [
                checkout(...),
                portal(),
                usage()
            ],
        })
    ]
});
```

### Event Ingestion

Polar's Usage Based Billing builds entirely on event ingestion. Ingest events from your application, create Meters to represent that usage, and add metered prices to Products to charge for it.

[Learn more about Usage Based Billing in the Polar Docs.](https://docs.polar.sh/features/usage-based-billing/introduction)

```typescript
const { data: ingested } = await authClient.usage.ingest({
  event: "file-uploads",
  metadata: {
    uploadedFiles: 12,
  },
});
```

The authenticated user is automatically associated with the ingested event.

### Customer Meters

A simple method for listing the authenticated user's Usage Meters, or as we call them, Customer Meters.

Customer Meter's contains all information about their consumtion on your defined meters.

* Customer Information
* Meter Information
* Customer Meter Information
  * Consumed Units
  * Credited Units
  * Balance

```typescript
const { data: customerMeters } = await authClient.usage.meters.list({
  query: {
    page: 1,
    limit: 10,
  },
});
```

## Webhooks Plugin

The Webhooks plugin can be used to capture incoming events from your Polar organization.

```typescript
import { polar, webhooks } from "@polar-sh/better-auth";

const auth = betterAuth({
    // ... Better Auth config
    plugins: [
        polar({
            ...

````
            use: [
                webhooks({
                    secret: process.env.POLAR_WEBHOOK_SECRET,
                    onCustomerStateChanged: (payload) => // Triggered when anything regarding a customer changes
                    onOrderPaid: (payload) => // Triggered when an order was paid (purchase, subscription renewal, etc.)
                    ... // Over 25 granular webhook handlers
                    onPayload: (payload) => // Catch-all for all events
                })
            ],
        })
    ]
});

Configure a Webhook endpoint in your Polar Organization Settings page. Webhook endpoint is configured at /polar/webhooks.

Add the secret to your environment.

```bash
# .env
POLAR_WEBHOOK_SECRET=...
```

The plugin supports handlers for all Polar webhook events:

* `onPayload` - Catch-all handler for any incoming Webhook event
* `onCheckoutCreated` - Triggered when a checkout is created
* `onCheckoutUpdated` - Triggered when a checkout is updated
* `onOrderCreated` - Triggered when an order is created
* `onOrderPaid` - Triggered when an order is paid
* `onOrderRefunded` - Triggered when an order is refunded
* `onRefundCreated` - Triggered when a refund is created
* `onRefundUpdated` - Triggered when a refund is updated
* `onSubscriptionCreated` - Triggered when a subscription is created
* `onSubscriptionUpdated` - Triggered when a subscription is updated
* `onSubscriptionActive` - Triggered when a subscription becomes active
* `onSubscriptionCanceled` - Triggered when a subscription is canceled
* `onSubscriptionRevoked` - Triggered when a subscription is revoked
* `onSubscriptionUncanceled` - Triggered when a subscription cancellation is reversed
* `onProductCreated` - Triggered when a product is created
* `onProductUpdated` - Triggered when a product is updated
* `onOrganizationUpdated` - Triggered when an organization is updated
* `onBenefitCreated` - Triggered when a benefit is created
* `onBenefitUpdated` - Triggered when a benefit is updated
* `onBenefitGrantCreated` - Triggered when a benefit grant is created
* `onBenefitGrantUpdated` - Triggered when a benefit grant is updated
* `onBenefitGrantRevoked` - Triggered when a benefit grant is revoked
* `onCustomerCreated` - Triggered when a customer is created
* `onCustomerUpdated` - Triggered when a customer is updated
* `onCustomerDeleted` - Triggered when a customer is deleted
* `onCustomerStateChanged` - Triggered when a customer is created

# plugins: Sign In With Ethereum (SIWE)
URL: /docs/plugins/siwe
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/siwe.mdx

Sign in with Ethereum plugin for Better Auth

***

title: Sign In With Ethereum (SIWE)
description: Sign in with Ethereum plugin for Better Auth

The Sign in with Ethereum (SIWE) plugin allows users to authenticate using their Ethereum wallets following the [ERC-4361 standard](https://eips.ethereum.org/EIPS/eip-4361). This
plugin provides flexibility by allowing you to implement your own message verification and nonce generation logic.

## Installation

<Steps>
  <Step>
    ### Add the Server Plugin

    Add the SIWE plugin to your auth configuration:

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth";
    import { siwe } from "better-auth/plugins";

    export const auth = betterAuth({
        plugins: [
            siwe({
                domain: "example.com",
                emailDomainName: "example.com", // optional
                anonymous: false, // optional, default is true
                getNonce: async () => {
                    // Implement your nonce generation logic here
                    return "your-secure-random-nonce";
                },
                verifyMessage: async (args) => {
                    // Implement your SIWE message verification logic here
                    // This should verify the signature against the message
                    return true; // return true if signature is valid
                },
                ensLookup: async (args) => {
                    // Optional: Implement ENS lookup for user names and avatars
                    return {
                        name: "user.eth",
                        avatar: "https://example.com/avatar.png"
                    };
                },
            }),
        ],
    });
    ```
  </Step>

  <Step>
    ### Migrate the database

    Run the migration or generate the schema to add the necessary fields and tables to the database.

    <Tabs items={["migrate", "generate"]}>
      <Tab value="migrate">
        ```bash
        npx @better-auth/cli migrate
        ```
      </Tab>

      <Tab value="generate">
        ```bash
        npx @better-auth/cli generate
        ```
      </Tab>
    </Tabs>

    See the [Schema](#schema) section to add the fields manually.
  </Step>
````

````
  <Step>
    ### Add the Client Plugin

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client";
    import { siweClient } from "better-auth/client/plugins";

    export const authClient = createAuthClient({
        plugins: [siweClient()],
    });
    ```
  </Step>
</Steps>

## Usage

### Generate a Nonce

Before signing a SIWE message, you need to generate a nonce for the wallet address:

```ts title="generate-nonce.ts"
const { data, error } = await authClient.siwe.nonce({
  walletAddress: "0x1234567890abcdef1234567890abcdef12345678",
  chainId: 1, // optional for Ethereum mainnet, required for other chains. Defaults to 1
});

if (data) {
  console.log("Nonce:", data.nonce);
}
```

### Sign In with Ethereum

After generating a nonce and creating a SIWE message, verify the signature to authenticate:

```ts title="sign-in-siwe.ts"
const { data, error } = await authClient.siwe.verify({
  message: "Your SIWE message string",
  signature: "0x...", // The signature from the user's wallet
  walletAddress: "0x1234567890abcdef1234567890abcdef12345678",
  chainId: 1, // optional for Ethereum mainnet, required for other chains. Must match Chain ID in SIWE message
  email: "user@example.com", // optional, required if anonymous is false
});

if (data) {
  console.log("Authentication successful:", data.user);
}
```

### Chain-Specific Examples

Here are examples for different blockchain networks:

```ts title="ethereum-mainnet.ts"
// Ethereum Mainnet (chainId can be omitted, defaults to 1)
const { data, error } = await authClient.siwe.verify({
  message,
  signature,
  walletAddress,
  // chainId: 1 (default)
});
```

```ts title="polygon.ts"
// Polygon (chainId REQUIRED)
const { data, error } = await authClient.siwe.verify({
  message,
  signature,
  walletAddress,
  chainId: 137, // Required for Polygon
});
```

```ts title="arbitrum.ts"
// Arbitrum (chainId REQUIRED)
const { data, error } = await authClient.siwe.verify({
  message,
  signature,
  walletAddress,
  chainId: 42161, // Required for Arbitrum
});
```

```ts title="base.ts"
// Base (chainId REQUIRED)
const { data, error } = await authClient.siwe.verify({
  message,
  signature,
  walletAddress,
  chainId: 8453, // Required for Base
});
```

<Callout type="warning">
  The `chainId` must match the Chain ID specified in your SIWE message. Verification will fail with a 401 error if there's a mismatch between the message's Chain ID and the `chainId` parameter.
</Callout>

## Configuration Options

### Server Options

The SIWE plugin accepts the following configuration options:

* **domain**: The domain name of your application (required for SIWE message generation)
* **emailDomainName**: The email domain name for creating user accounts when not using anonymous mode. Defaults to the domain from your base URL
* **anonymous**: Whether to allow anonymous sign-ins without requiring an email. Default is `true`
* **getNonce**: Function to generate a unique nonce for each sign-in attempt. You must implement this function to return a cryptographically secure random string. Must return a `Promise<string>`
* **verifyMessage**: Function to verify the signed SIWE message. Receives message details and should return `Promise<boolean>`
* **ensLookup**: Optional function to lookup ENS names and avatars for Ethereum addresses

### Client Options

The SIWE client plugin doesn't require any configuration options, but you can pass them if needed for future extensibility:

```ts title="auth-client.ts"
import { createAuthClient } from "better-auth/client";
import { siweClient } from "better-auth/client/plugins";

export const authClient = createAuthClient({
  plugins: [
    siweClient({
      // Optional client configuration can go here
    }),
  ],
});
```

## Schema
~~~

The SIWE plugin adds a `walletAddress` table to store user wallet associations:

Field	Type	Description
id	string	Primary key
userId	string	Reference to user.id
address	string	Ethereum wallet address
chainId	number	Chain ID (e.g., 1 for Ethereum mainnet)
isPrimary	boolean	Whether this is the user's primary wallet
createdAt	date	Creation timestamp

Example Implementation

Here's a complete example showing how to implement SIWE authentication:

```ts title="auth.ts"
import { betterAuth } from "better-auth";
import { siwe } from "better-auth/plugins";
import { generateRandomString } from "better-auth/crypto";
import { verifyMessage, createPublicClient, http } from "viem";
import { mainnet } from "viem/chains";

export const auth = betterAuth({
  database: {
    // your database configuration
  },
  plugins: [
    siwe({
      domain: "myapp.com",
      emailDomainName: "myapp.com",
      anonymous: false,
      getNonce: async () => {
        // Generate a cryptographically secure random nonce
        return generateRandomString(32);
      },
      verifyMessage: async ({ message, signature, address }) => {
        try {
          // Verify the signature using viem (recommended)
          const isValid = await verifyMessage({
            address: address as `0x${string}`,
            message,
            signature: signature as `0x${string}`,
          });
          return isValid;
        } catch (error) {
          console.error("SIWE verification failed:", error);
          return false;
        }
      },
      ensLookup: async ({ walletAddress }) => {
        try {
          // Optional: lookup ENS name and avatar using viem
          // You can use viem's ENS utilities here
          const client = createPublicClient({
            chain: mainnet,
            transport: http(),
          });

          const ensName = await client.getEnsName({
            address: walletAddress as `0x${string}`,
          });

          const ensAvatar = ensName
            ? await client.getEnsAvatar({
                name: ensName,
              })
            : null;

          return {
            name: ensName || walletAddress,
            avatar: ensAvatar || "",
          };
        } catch {
          return {
            name: walletAddress,
            avatar: "",
          };
        }
      },
    }),
  ],
});
```

plugins: Single Sign-On (SSO)

URL: /docs/plugins/sso
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/sso.mdx

Integrate Single Sign-On (SSO) with your application.

***

title: Single Sign-On (SSO)
description: Integrate Single Sign-On (SSO) with your application.

`OIDC` `OAuth2` `SSO` `SAML`

Single Sign-On (SSO) allows users to authenticate with multiple applications using a single set of credentials. This plugin supports OpenID Connect (OIDC), OAuth2 providers, and SAML 2.0.

Installation

<Steps>
  <Step>

Install the plugin

```bash
npm install @better-auth/sso
```

  </Step>

  <Step>

Add Plugin to the server

```ts title="auth.ts"
import { betterAuth } from "better-auth"
import { sso } from "@better-auth/sso";

const auth = betterAuth({
    plugins: [ // [!code highlight]
        sso() // [!code highlight]
    ] // [!code highlight]
})
```

  </Step>

  <Step>

Migrate the database

~~~
    Run the migration or generate the schema to add the necessary fields and tables to the database.

    <Tabs items={["migrate", "generate"]}>
      <Tab value="migrate">
        ```bash
        npx @better-auth/cli migrate
        ```
      </Tab>

      <Tab value="generate">
        ```bash
        npx @better-auth/cli generate
        ```
      </Tab>
    </Tabs>

    See the [Schema](#schema) section to add the fields manually.
  </Step>

  <Step>
    ### Add the client plugin

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    import { ssoClient } from "@better-auth/sso/client"

    const authClient = createAuthClient({
        plugins: [ // [!code highlight]
            ssoClient() // [!code highlight]
        ] // [!code highlight]
    })
    ```
  </Step>
</Steps>

## Usage

### Register an OIDC Provider

To register an OIDC provider, use the `registerSSOProvider` endpoint and provide the necessary configuration details for the provider.

A redirect URL will be automatically generated using the provider ID. For instance, if the provider ID is `hydra`, the redirect URL would be
`{baseURL}/api/auth/sso/callback/hydra`. Note that `/api/auth` may vary depending on your base path configuration.

#### Example

<Tabs items={["client", "server"]}>
  <Tab value="client">
    ```ts title="register-oidc-provider.ts"
    import { authClient } from "@/lib/auth-client";

    // Register with OIDC configuration
    await authClient.sso.register({
        providerId: "example-provider",
        issuer: "https://idp.example.com",
        domain: "example.com",
        oidcConfig: {
            clientId: "client-id",
            clientSecret: "client-secret",
            authorizationEndpoint: "https://idp.example.com/authorize",
            tokenEndpoint: "https://idp.example.com/token",
            jwksEndpoint: "https://idp.example.com/jwks",
            discoveryEndpoint: "https://idp.example.com/.well-known/openid-configuration",
            scopes: ["openid", "email", "profile"],
            pkce: true,
            mapping: {
                id: "sub",
                email: "email",
                emailVerified: "email_verified",
                name: "name",
                image: "picture",
                extraFields: {
                    department: "department",
                    role: "role"
                }
            }
        }
    });
    ```
  </Tab>

  <Tab value="server">
    ```ts title="register-oidc-provider.ts"
    const { headers } = await signInWithTestUser();
    await auth.api.registerSSOProvider({
        body: {
            providerId: "example-provider",
            issuer: "https://idp.example.com",
            domain: "example.com",
            oidcConfig: {
                clientId: "your-client-id",
                clientSecret: "your-client-secret",
                authorizationEndpoint: "https://idp.example.com/authorize",
                tokenEndpoint: "https://idp.example.com/token",
                jwksEndpoint: "https://idp.example.com/jwks",
                discoveryEndpoint: "https://idp.example.com/.well-known/openid-configuration",
                scopes: ["openid", "email", "profile"],
                pkce: true,
                mapping: {
                    id: "sub",
                    email: "email",
                    emailVerified: "email_verified",
                    name: "name",
                    image: "picture",
                    extraFields: {
                        department: "department",
                        role: "role"
                    }
                }
            }
        },
        headers,
    });
    ```
  </Tab>
</Tabs>

### Register a SAML Provider

To register a SAML provider, use the `registerSSOProvider` endpoint with SAML configuration details. The provider will act as a Service Provider (SP) and integrate with your
Identity Provider (IdP).

<Tabs items={["client", "server"]}>
  <Tab value="client">
    ```ts title="register-saml-provider.ts"
    import { authClient } from "@/lib/auth-client";
````

~~~~
    await authClient.sso.register({
        providerId: "saml-provider",
        issuer: "https://idp.example.com",
        domain: "example.com",
        samlConfig: {
            entryPoint: "https://idp.example.com/sso",
            cert: "-----BEGIN CERTIFICATE-----\n...\n-----END CERTIFICATE-----",
            callbackUrl: "https://yourapp.com/api/auth/sso/saml2/callback/saml-provider",
            audience: "https://yourapp.com",
            wantAssertionsSigned: true,
            signatureAlgorithm: "sha256",
            digestAlgorithm: "sha256",
            identifierFormat: "urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress",
            idpMetadata: {
                metadata: "<!-- IdP Metadata XML -->",
                privateKey: "-----BEGIN RSA PRIVATE KEY-----\n...\n-----END RSA PRIVATE KEY-----",
                privateKeyPass: "your-private-key-password",
                isAssertionEncrypted: true,
                encPrivateKey: "-----BEGIN RSA PRIVATE KEY-----\n...\n-----END RSA PRIVATE KEY-----",
                encPrivateKeyPass: "your-encryption-key-password"
            },
            spMetadata: {
                metadata: "<!-- SP Metadata XML -->",
                binding: "post",
                privateKey: "-----BEGIN RSA PRIVATE KEY-----\n...\n-----END RSA PRIVATE KEY-----",
                privateKeyPass: "your-sp-private-key-password",
                isAssertionEncrypted: true,
                encPrivateKey: "-----BEGIN RSA PRIVATE KEY-----\n...\n-----END RSA PRIVATE KEY-----",
                encPrivateKeyPass: "your-sp-encryption-key-password"
            },
            mapping: {
                id: "nameID",
                email: "email",
                name: "displayName",
                firstName: "givenName",
                lastName: "surname",
                emailVerified: "email_verified",
                extraFields: {
                    department: "department",
                    role: "role"
                }
            }
        }
    });
    ```
  </Tab>

  <Tab value="server">
    ```ts title="register-saml-provider.ts"
    const { headers } = await signInWithTestUser();
    await auth.api.registerSSOProvider({
        body: {
            providerId: "saml-provider",
            issuer: "https://idp.example.com",
            domain: "example.com",
            samlConfig: {
                entryPoint: "https://idp.example.com/sso",
                cert: "-----BEGIN CERTIFICATE-----\n...\n-----END CERTIFICATE-----",
                callbackUrl: "https://yourapp.com/api/auth/sso/saml2/callback/saml-provider",
                audience: "https://yourapp.com",
                wantAssertionsSigned: true,
                signatureAlgorithm: "sha256",
                digestAlgorithm: "sha256",
                identifierFormat: "urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress",
                idpMetadata: {
                    metadata: "<!-- IdP Metadata XML -->",
                    privateKey: "-----BEGIN RSA PRIVATE KEY-----\n...\n-----END RSA PRIVATE KEY-----",
                    privateKeyPass: "your-private-key-password",
                    isAssertionEncrypted: true,
                    encPrivateKey: "-----BEGIN RSA PRIVATE KEY-----\n...\n-----END RSA PRIVATE KEY-----",
                    encPrivateKeyPass: "your-encryption-key-password"
                },
                spMetadata: {
                    metadata: "<!-- SP Metadata XML -->",
                    binding: "post",
                    privateKey: "-----BEGIN RSA PRIVATE KEY-----\n...\n-----END RSA PRIVATE KEY-----",
                    privateKeyPass: "your-sp-private-key-password",
                    isAssertionEncrypted: true,
                    encPrivateKey: "-----BEGIN RSA PRIVATE KEY-----\n...\n-----END RSA PRIVATE KEY-----",
                    encPrivateKeyPass: "your-sp-encryption-key-password"
                },
                mapping: {
                    id: "nameID",
                    email: "email",
                    name: "displayName",
                    firstName: "givenName",
                    lastName: "surname",
                    emailVerified: "email_verified",
                    extraFields: {
                        department: "department",
                        role: "role"
                    }
                }
            }
        },
        headers,
    });
    ```
  </Tab>
</Tabs>

Get Service Provider Metadata

For SAML providers, you can retrieve the Service Provider metadata XML that needs to be configured in your Identity Provider:

```ts title="get-sp-metadata.ts"
const response = await auth.api.spMetadata({
    query: {
        providerId: "saml-provider",
        format: "xml" // or "json"
    }
});

const metadataXML = await response.text();
console.log(metadataXML);
```

Sign In with SSO

To sign in with an SSO provider, you can call `signIn.sso`

You can sign in using the email with domain matching:

```ts title="sign-in.ts"
const res = await authClient.signIn.sso({
    email: "user@example.com",
    callbackURL: "/dashboard",
});
```

or you can specify the domain:
~~~~

```ts title="sign-in-domain.ts"
const res = await authClient.signIn.sso({
    domain: "example.com",
    callbackURL: "/dashboard",
});
```

You can also sign in using the organization slug if a provider is associated with an organization:

```ts title="sign-in-org.ts"
const res = await authClient.signIn.sso({
    organizationSlug: "example-org",
    callbackURL: "/dashboard",
});
```

Alternatively, you can sign in using the provider's ID:

```ts title="sign-in-provider-id.ts"
const res = await authClient.signIn.sso({
    providerId: "example-provider-id",
    callbackURL: "/dashboard",
});
```

To use the server API you can use `signInSSO`

```ts title="sign-in-org.ts"
const res = await auth.api.signInSSO({
    body: {
        organizationSlug: "example-org",
        callbackURL: "/dashboard",
    }
});
```

#### Full method

### Client Side

```ts
const { data, error } = await authClient.signIn.sso({
    email: john@example.com, // required
    organizationSlug: example-org, // required
    providerId: example-provider, // required
    domain: example.com, // required
    callbackURL: https://example.com/callback,
    errorCallbackURL: https://example.com/callback, // required
    newUserCallbackURL: https://example.com/new-user, // required
    scopes, // required
    requestSignUp, // required
});
```

### Server Side

```ts
const data = await auth.api.signInSSO({
    body: {
        email: john@example.com, // required
        organizationSlug: example-org, // required
        providerId: example-provider, // required
        domain: example.com, // required
        callbackURL: https://example.com/callback,
        errorCallbackURL: https://example.com/callback, // required
        newUserCallbackURL: https://example.com/new-user, // required
        scopes, // required
        requestSignUp, // required
    }
});
```

### Type Definition

```ts
type signInSSO = {
    /**
     * The email address to sign in with. This is used to identify the issuer to sign in with. It's optional if the issuer is provided.
     */
    email?: string = "john@example.com"
    /**
     * The slug of the organization to sign in with.
     */
    organizationSlug?: string = "example-org"
    /**
     * The ID of the provider to sign in with. This can be provided instead of email or issuer.
     */
    providerId?: string = "example-provider"
    /**
     * The domain of the provider.
     */
    domain?: string = "example.com"
    /**
     * The URL to redirect to after login.
     */
    callbackURL: string = "https://example.com/callback"
    /**
     * The URL to redirect to after login.
     */
    errorCallbackURL?: string = "https://example.com/callback"
    /**
     * The URL to redirect to after login if the user is new.
     */
    newUserCallbackURL?: string = "https://example.com/new-user"
    /**
     * Scopes to request from the provider.
     */
    scopes?: string[] = ["openid", "email", "profile", "offline_access"]
    /**
     * Explicitly request sign-up. Useful when disableImplicitSignUp is true for this provider.
     */
    requestSignUp?: boolean = true

}
```

When a user is authenticated, if the user does not exist, the user will be provisioned using the `provisionUser` function. If the organization provisioning is enabled and a provider is associated with an organization, the user will be added to the organization.

```ts title="auth.ts"
const auth = betterAuth({
    plugins: [
        sso({
            provisionUser: async (user) => {
                // provision user
            },
            organizationProvisioning: {
                disabled: false,
                defaultRole: "member",
                getRole: async (user) => {
                    // get role if needed
                },
            },
        }),
    ],
});
```

## Provisioning

The SSO plugin provides powerful provisioning capabilities to automatically set up users and manage their organization memberships when they sign in through SSO providers.

### User Provisioning

User provisioning allows you to run custom logic whenever a user signs in through an SSO provider. This is useful for:

* Setting up user profiles with additional data from the SSO provider
* Synchronizing user attributes with external systems
* Creating user-specific resources
* Logging SSO sign-ins
* Updating user information from the SSO provider

```ts title="auth.ts"
const auth = betterAuth({
    plugins: [
        sso({
            provisionUser: async ({ user, userInfo, token, provider }) => {
                // Update user profile with SSO data
                await updateUserProfile(user.id, {
                    department: userInfo.attributes?.department,
                    jobTitle: userInfo.attributes?.jobTitle,
                    manager: userInfo.attributes?.manager,
                    lastSSOLogin: new Date(),
                });

                // Create user-specific resources
                await createUserWorkspace(user.id);

                // Sync with external systems
                await syncUserWithCRM(user.id, userInfo);

                // Log the SSO sign-in
                await auditLog.create({
                    userId: user.id,
                    action: 'sso_signin',
                    provider: provider.providerId,
                    metadata: {
                        email: userInfo.email,
                        ssoProvider: provider.issuer,
                    },
                });
            },
        }),
    ],
});
```

The `provisionUser` function receives:

* **user**: The user object from the database
* **userInfo**: User information from the SSO provider (includes attributes, email, name, etc.)
* **token**: OAuth2 tokens (for OIDC providers) - may be undefined for SAML
* **provider**: The SSO provider configuration

### Organization Provisioning

Organization provisioning automatically manages user memberships in organizations when SSO providers are linked to specific organizations. This is particularly useful for:

* Enterprise SSO where each company/domain maps to an organization
* Automatic role assignment based on SSO attributes
* Managing team memberships through SSO

#### Basic Organization Provisioning

```ts title="auth.ts"
const auth = betterAuth({
    plugins: [
        sso({
            organizationProvisioning: {
                disabled: false,           // Enable org provisioning
                defaultRole: "member",     // Default role for new members
            },
        }),
    ],
});
```

#### Advanced Organization Provisioning with Custom Roles

```ts title="auth.ts"
const auth = betterAuth({
    plugins: [
        sso({
            organizationProvisioning: {
                disabled: false,
                defaultRole: "member",
                getRole: async ({ user, userInfo, provider }) => {
                    // Assign roles based on SSO attributes
                    const department = userInfo.attributes?.department;
                    const jobTitle = userInfo.attributes?.jobTitle;

                    // Admins based on job title
                    if (jobTitle?.toLowerCase().includes('manager') ||
                        jobTitle?.toLowerCase().includes('director') ||
                        jobTitle?.toLowerCase().includes('vp')) {
                        return "admin";
                    }

                    // Special roles for IT department
                    if (department?.toLowerCase() === 'it') {
                        return "admin";
                    }
```

```
                    // Default to member for everyone else
                    return "member";
                },
            },
        }),
    ],
});
```

#### Linking SSO Providers to Organizations

When registering an SSO provider, you can link it to a specific organization:

```ts title="register-org-provider.ts"
await auth.api.registerSSOProvider({
    body: {
        providerId: "acme-corp-saml",
        issuer: "https://acme-corp.okta.com",
        domain: "acmecorp.com",
        organizationId: "org_acme_corp_id", // Link to organization
        samlConfig: {
            // SAML configuration...
        },
    },
    headers,
});
```

Now when users from `acmecorp.com` sign in through this provider, they'll automatically be added to the "Acme Corp" organization with the appropriate role.

#### Multiple Organizations Example

You can set up multiple SSO providers for different organizations:

```ts title="multi-org-setup.ts"
// Acme Corp SAML provider
await auth.api.registerSSOProvider({
    body: {
        providerId: "acme-corp",
        issuer: "https://acme.okta.com",
        domain: "acmecorp.com",
        organizationId: "org_acme_id",
        samlConfig: { /* ... */ },
    },
    headers,
});

// TechStart OIDC provider
await auth.api.registerSSOProvider({
    body: {
        providerId: "techstart-google",
        issuer: "https://accounts.google.com",
        domain: "techstart.io",
        organizationId: "org_techstart_id",
        oidcConfig: { /* ... */ },
    },
    headers,
});
```

#### Organization Provisioning Flow

1. **User signs in** through an SSO provider linked to an organization
2. **User is authenticated** and either found or created in the database
3. **Organization membership is checked** - if the user isn't already a member of the linked organization
4. **Role is determined** using either the `defaultRole` or `getRole` function
5. **User is added** to the organization with the determined role
6. **User provisioning runs** (if configured) for additional setup

### Provisioning Best Practices

#### 1. Idempotent Operations

Make sure your provisioning functions can be safely run multiple times:

```ts
provisionUser: async ({ user, userInfo }) => {
    // Check if already provisioned
    const existingProfile = await getUserProfile(user.id);
    if (!existingProfile.ssoProvisioned) {
        await createUserResources(user.id);
        await markAsProvisioned(user.id);
    }

    // Always update attributes (they might change)
    await updateUserAttributes(user.id, userInfo.attributes);
},
```

#### 2. Error Handling

Handle errors gracefully to avoid blocking user sign-in:

```ts
provisionUser: async ({ user, userInfo }) => {
    try {
        await syncWithExternalSystem(user, userInfo);
    } catch (error) {
        // Log error but don't throw - user can still sign in
        console.error('Failed to sync user with external system:', error);
        await logProvisioningError(user.id, error);
    }
},
```

#### 3. Conditional Provisioning

Only run certain provisioning steps when needed:

```ts
organizationProvisioning: {
    disabled: false,
    getRole: async ({ user, userInfo, provider }) => {
        // Only process role assignment for certain providers
        if (provider.providerId.includes('enterprise')) {
            return determineEnterpriseRole(userInfo);
        }
        return "member";
    },
},
```

## SAML Configuration

### Default SSO Provider

~~~markdown
```ts title="auth.ts"
const auth = betterAuth({
    plugins: [
        sso({
            defaultSSO: {
                providerId: "default-saml", // Provider ID for the default provider
                samlConfig: {
                    issuer: "https://your-app.com",
                    entryPoint: "https://idp.example.com/sso",
                    cert: "-----BEGIN CERTIFICATE-----\n...\n-----END CERTIFICATE-----",
                    callbackUrl: "http://localhost:3000/api/auth/sso/saml2/sp/acs",
                    spMetadata: {
                        entityID: "http://localhost:3000/api/auth/sso/saml2/sp/metadata",
                        metadata: "<!-- Your SP Metadata XML -->",
                    }
                }
            }
        })
    ]
});
```

The defaultSSO provider will be used when:

1. No matching provider is found in the database

This allows you to test SAML authentication without setting up providers in the database. The defaultSSO provider supports all the same configuration options as regular SAML providers.

### Service Provider Configuration

When registering a SAML provider, you need to provide Service Provider (SP) metadata configuration:

* **metadata**: XML metadata for the Service Provider
* **binding**: The binding method, typically "post" or "redirect"
* **privateKey**: Private key for signing (optional)
* **privateKeyPass**: Password for the private key (if encrypted)
* **isAssertionEncrypted**: Whether assertions should be encrypted
* **encPrivateKey**: Private key for decryption (if encryption is enabled)
* **encPrivateKeyPass**: Password for the encryption private key

### Identity Provider Configuration

You also need to provide Identity Provider (IdP) configuration:

* **metadata**: XML metadata from your Identity Provider
* **privateKey**: Private key for the IdP communication (optional)
* **privateKeyPass**: Password for the IdP private key (if encrypted)
* **isAssertionEncrypted**: Whether assertions from IdP are encrypted
* **encPrivateKey**: Private key for IdP assertion decryption
* **encPrivateKeyPass**: Password for the IdP decryption key

### SAML Attribute Mapping

Configure how SAML attributes map to user fields:

```ts
mapping: {
    id: "nameID",           // Default: "nameID"
    email: "email",         // Default: "email" or "nameID"
    name: "displayName",    // Default: "displayName"
    firstName: "givenName", // Default: "givenName"
    lastName: "surname",    // Default: "surname"
    extraFields: {
        department: "department",
        role: "jobTitle",
        phone: "telephoneNumber"
    }
}
```

### SAML Endpoints

The plugin automatically creates the following SAML endpoints:

* **SP Metadata**: `/api/auth/sso/saml2/sp/metadata?providerId={providerId}`
* **SAML Callback**: `/api/auth/sso/saml2/callback/{providerId}`

## Schema

The plugin requires additional fields in the `ssoProvider` table to store the provider's configuration.

<DatabaseTable
  fields={[
      {
          name: "id", type: "string", description: "A database identifier", isRequired: true, isPrimaryKey: true,
      },
      { name: "issuer", type: "string", description: "The issuer identifier", isRequired: true },
      { name: "domain", type: "string", description: "The domain of the provider", isRequired: true },
      { name: "oidcConfig", type: "string", description: "The OIDC configuration (JSON string)", isRequired: false },
      { name: "samlConfig", type: "string", description: "The SAML configuration (JSON string)", isRequired: false },
      { name: "userId", type: "string", description: "The user ID", isRequired: true, references: { model: "user", field: "id" } },
      { name: "providerId", type: "string", description: "The provider ID. Used to identify a provider and to generate a redirect URL.", isRequired: true, isUnique: true },
      { name: "organizationId", type: "string", description: "The organization Id. If provider is linked to an organization.", isRequired: false },
  ]}
/>

For a detailed guide on setting up SAML SSO with examples for Okta and testing with DummyIDP, see our [SAML SSO with Okta](/docs/guides/saml-sso-with-okta).

## Options

### Server

**provisionUser**: A custom function to provision a user when they sign in with an SSO provider.

**organizationProvisioning**: Options for provisioning users to an organization.

**defaultOverrideUserInfo**: Override user info with the provider info by default.

**disableImplicitSignUp**: Disable implicit sign up for new users.

**trustEmailVerified**: Trust the email verified flag from the provider.
~~~

~~~
<TypeTable
  type={{
  provisionUser: {
      description: "A custom function to provision a user when they sign in with an SSO provider.",
      type: "function",
  },
  organizationProvisioning: {
      description: "Options for provisioning users to an organization.",
      type: "object",
      properties: {
          disabled: {
              description: "Disable organization provisioning.",
              type: "boolean",
              default: false,
          },
          defaultRole: {
              description: "The default role for new users.",
              type: "string",
              enum: ["member", "admin"],
              default: "member",
          },
          getRole: {
              description: "A custom function to determine the role for new users.",
              type: "function",
          },
      },
  },
  defaultOverrideUserInfo: {
      description: "Override user info with the provider info by default.",
      type: "boolean",
      default: false,
  },
  disableImplicitSignUp: {
      description: "Disable implicit sign up for new users. When set to true, sign-in needs to be called with requestSignUp as true to create new users.",
      type: "boolean",
      default: false,
  },
  providersLimit: {
      description: "Configure the maximum number of SSO providers a user can register. Set to 0 to disable SSO provider registration.",
      type: "number | function",
      default: 10,
  },
  defaultSSO: {
      description: "Configure a default SSO provider for testing and development. This provider will be used when no matching provider is found in the database.",
      type: "object",
      properties: {
          domain: {
              description: "The domain to match for this default provider.",
              type: "string",
              required: true,
          },
          providerId: {
              description: "The provider ID to use for the default provider.",
              type: "string",
              required: true,
          },
          samlConfig: {
              description: "SAML configuration for the default provider.",
              type: "SAMLConfig",
              required: false,
          },
          oidcConfig: {
              description: "OIDC configuration for the default provider.",
              type: "OIDCConfig",
              required: false,
          },
      },
  },
}}
/>



# plugins: Stripe
URL: /docs/plugins/stripe
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/stripe.mdx

Stripe plugin for Better Auth to manage subscriptions and payments.

***

title: Stripe
description: Stripe plugin for Better Auth to manage subscriptions and payments.
--------------------------------------------------------------------------------

The Stripe plugin integrates Stripe's payment and subscription functionality with Better Auth. Since payment and authentication are often tightly coupled, this plugin simplifies
the integration of Stripe into your application, handling customer creation, subscription management, and webhook processing.

## Features

* Create Stripe Customers automatically when users sign up
* Manage subscription plans and pricing
* Process subscription lifecycle events (creation, updates, cancellations)
* Handle Stripe webhooks securely with signature verification
* Expose subscription data to your application
* Support for trial periods and subscription upgrades
* **Automatic trial abuse prevention** - Users can only get one trial per account across all plans
* Flexible reference system to associate subscriptions with users or organizations
* Team subscription support with seats management

## Installation

<Steps>
  <Step>
    ### Install the plugin

    First, install the plugin:

    <CodeBlockTabs defaultValue="npm">
      <CodeBlockTabsList>
        <CodeBlockTabsTrigger value="npm">
          npm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="pnpm">
          pnpm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="yarn">
          yarn
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="bun">
          bun
        </CodeBlockTabsTrigger>
      </CodeBlockTabsList>

      <CodeBlockTab value="npm">
        ```bash
        npm install @better-auth/stripe
        ```
~~~

~~~
      </CodeBlockTab>

      <CodeBlockTab value="pnpm">
        ```bash
        pnpm add @better-auth/stripe
        ```
      </CodeBlockTab>

      <CodeBlockTab value="yarn">
        ```bash
        yarn add @better-auth/stripe
        ```
      </CodeBlockTab>

      <CodeBlockTab value="bun">
        ```bash
        bun add @better-auth/stripe
        ```
      </CodeBlockTab>
    </CodeBlockTabs>

    <Callout>
      If you're using a separate client and server setup, make sure to install the plugin in both parts of your project.
    </Callout>
  </Step>

  <Step>
    ### Install the Stripe SDK

    Next, install the Stripe SDK on your server:

    <CodeBlockTabs defaultValue="npm">
      <CodeBlockTabsList>
        <CodeBlockTabsTrigger value="npm">
          npm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="pnpm">
          pnpm
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="yarn">
          yarn
        </CodeBlockTabsTrigger>

        <CodeBlockTabsTrigger value="bun">
          bun
        </CodeBlockTabsTrigger>
      </CodeBlockTabsList>

      <CodeBlockTab value="npm">
        ```bash
        npm install stripe@^19.1.0
        ```
      </CodeBlockTab>

      <CodeBlockTab value="pnpm">
        ```bash
        pnpm add stripe@^19.1.0
        ```
      </CodeBlockTab>

      <CodeBlockTab value="yarn">
        ```bash
        yarn add stripe@^19.1.0
        ```
      </CodeBlockTab>

      <CodeBlockTab value="bun">
        ```bash
        bun add stripe@^19.1.0
        ```
      </CodeBlockTab>
    </CodeBlockTabs>
  </Step>

  <Step>
    ### Add the plugin to your auth config

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { stripe } from "@better-auth/stripe"
    import Stripe from "stripe"

    const stripeClient = new Stripe(process.env.STRIPE_SECRET_KEY!, {
        apiVersion: "2025-09-30.clover", // Latest API version as of Stripe SDK v19
    })

    export const auth = betterAuth({
        // ... your existing config
        plugins: [
            stripe({
                stripeClient,
                stripeWebhookSecret: process.env.STRIPE_WEBHOOK_SECRET!,
                createCustomerOnSignUp: true,
            })
        ]
    })
    ```

    <Callout type="info">
      **Upgrading from Stripe v18?** Version 19 uses async webhook signature verification (`constructEventAsync`) which is handled internally by the plugin. No code changes required on your end!
    </Callout>
  </Step>

  <Step>
    ### Add the client plugin

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    import { stripeClient } from "@better-auth/stripe/client"

    export const client = createAuthClient({
        // ... your existing config
        plugins: [
            stripeClient({
                subscription: true //if you want to enable subscription management
            })
        ]
    })
    ```
  </Step>

  <Step>
    ### Migrate the database

    Run the migration or generate the schema to add the necessary tables to the database.
~~~

~~~mdx
    <Tabs items={["migrate", "generate"]}>
      <Tab value="migrate">
        ```bash
        npx @better-auth/cli migrate
        ```
      </Tab>

      <Tab value="generate">
        ```bash
        npx @better-auth/cli generate
        ```
      </Tab>
    </Tabs>

    See the [Schema](#schema) section to add the tables manually.
  </Step>

  <Step>
    ### Set up Stripe webhooks

    Create a webhook endpoint in your Stripe dashboard pointing to:

    ```
    https://your-domain.com/api/auth/stripe/webhook
    ```

    `/api/auth` is the default path for the auth server.

    Make sure to select at least these events:

    * `checkout.session.completed`
    * `customer.subscription.updated`
    * `customer.subscription.deleted`

    Save the webhook signing secret provided by Stripe and add it to your environment variables as `STRIPE_WEBHOOK_SECRET`.
  </Step>
</Steps>

## Usage

### Customer Management

You can use this plugin solely for customer management without enabling subscriptions. This is useful if you just want to link Stripe customers to your users.

By default, when a user signs up, a Stripe customer is automatically created if you set `createCustomerOnSignUp: true`. This customer is linked to the user in your database.
You can customize the customer creation process:

```ts title="auth.ts"
stripe({
    // ... other options
    createCustomerOnSignUp: true,
    onCustomerCreate: async ({ customer, stripeCustomer, user }, request) => {
        // Do something with the newly created customer
        console.log(`Customer ${customer.id} created for user ${user.id}`);
    },
    getCustomerCreateParams: async ({ user, session }, request) => {
        // Customize the Stripe customer creation parameters
        return {
            metadata: {
                referralSource: user.metadata?.referralSource
            }
        };
    }
})
```

### Subscription Management

#### Defining Plans

You can define your subscription plans either statically or dynamically:

```ts title="auth.ts"
// Static plans
subscription: {
    enabled: true,
    plans: [
        {
            name: "basic", // the name of the plan, it'll be automatically lower cased when stored in the database
            priceId: "price_1234567890", // the price ID from stripe
            annualDiscountPriceId: "price_1234567890", // (optional) the price ID for annual billing with a discount
            limits: {
                projects: 5,
                storage: 10
            }
        },
        {
            name: "pro",
            priceId: "price_0987654321",
            limits: {
                projects: 20,
                storage: 50
            },
            freeTrial: {
                days: 14,
            }
        }
    ]
}

// Dynamic plans (fetched from database or API)
subscription: {
    enabled: true,
    plans: async () => {
        const plans = await db.query("SELECT * FROM plans");
        return plans.map(plan => ({
            name: plan.name,
            priceId: plan.stripe_price_id,
            limits: JSON.parse(plan.limits)
        }));
    }
}
```

see [plan configuration](#plan-configuration) for more.

#### Creating a Subscription

To create a subscription, use the `subscription.upgrade` method:

### Client Side
~~~

```ts
const { data, error } = await authClient.subscription.upgrade({
    plan: pro,
    annual, // required
    referenceId: 123, // required
    subscriptionId: sub_123, // required
    metadata, // required
    seats, // required
    successUrl,
    cancelUrl,
    returnUrl, // required
    disableRedirect,
});
```

### Server Side

```ts
const data = await auth.api.upgradeSubscription({
    body: {
        plan: pro,
        annual, // required
        referenceId: 123, // required
        subscriptionId: sub_123, // required
        metadata, // required
        seats, // required
        successUrl,
        cancelUrl,
        returnUrl, // required
        disableRedirect,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type upgradeSubscription = {
    /**
     * The name of the plan to upgrade to.
     */
    plan: string = "pro"
    /**
     * Whether to upgrade to an annual plan.
     */
    annual?: boolean = true
    /**
     * Reference id of the subscription to upgrade.
     */
    referenceId?: string = "123"
    /**
     * The id of the subscription to upgrade.
     */
    subscriptionId?: string = "sub_123"
    metadata?: Record<string, any>
    /**
     * Number of seats to upgrade to (if applicable).
     */
    seats?: number = 1
    /**
     * Callback URL to redirect back after successful subscription.
     */
    successUrl: string
    /**
     * If set, checkout shows a back button and customers will be directed here if they cancel payment.
     */
    cancelUrl: string
    /**
     * URL to take customers to when they click on the billing portalâ€™s link to return to your website.
     */
    returnUrl?: string
    /**
     * Disable redirect after successful subscription.
     */
    disableRedirect: boolean = true

}
```

Simple Example:

```ts title="client.ts"
await client.subscription.upgrade({
    plan: "pro",
    successUrl: "/dashboard",
    cancelUrl: "/pricing",
    annual: true, // Optional: upgrade to an annual plan
    referenceId: "org_123", // Optional: defaults to the current logged in user ID
    seats: 5 // Optional: for team plans
});
```

This will create a Checkout Session and redirect the user to the Stripe Checkout page.

<Callout type="warn">
  If the user already has an active subscription, you *must* provide the `subscriptionId` parameter. Otherwise, the user will be subscribed to (and pay for) both plans.
</Callout>

> **Important:** The `successUrl` parameter will be internally modified to handle race conditions between checkout completion and webhook processing. The plugin creates an intermediate redirect that ensures subscription status is properly updated before redirecting to your success page.

```ts
const { error } = await client.subscription.upgrade({
    plan: "pro",
    successUrl: "/dashboard",
    cancelUrl: "/pricing",
});
if(error) {
    alert(error.message);
}
```

<Callout type="warn">
  For each reference ID (user or organization), only one active or trialing subscription is supported at a time. The plugin doesn't currently support multiple concurrent active subscriptions for the same reference ID.
</Callout>

#### Switching Plans

To switch a subscription to a different plan, use the `subscription.upgrade` method:

```ts title="client.ts"
await client.subscription.upgrade({
    plan: "pro",
    successUrl: "/dashboard",
    cancelUrl: "/pricing",
    subscriptionId: "sub_123", // the Stripe subscription ID of the user's current plan
});
```

This ensures that the user only pays for the new plan, and not both.

#### Listing Active Subscriptions

To get the user's active subscriptions:

### Client Side

```ts
const { data, error } = await authClient.subscription.list({
    referenceId: 123, // required
});
```

### Server Side

```ts
const subscriptions = await auth.api.listActiveSubscriptions({
    query: {
        referenceId: 123, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type listActiveSubscriptions = {
    /**
     * Reference id of the subscription to list.
     */
    referenceId?: string = '123'

}
```

#### Canceling a Subscription

To cancel a subscription:

### Client Side

```ts
const { data, error } = await authClient.subscription.cancel({
    referenceId: org_123, // required
    subscriptionId: sub_123, // required
    returnUrl: /account,
});
```

### Server Side

```ts
const data = await auth.api.cancelSubscription({
    body: {
        referenceId: org_123, // required
        subscriptionId: sub_123, // required
        returnUrl: /account,
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type cancelSubscription = {
    /**
     * Reference id of the subscription to cancel. Defaults to the userId.
     */
    referenceId?: string = 'org_123'
    /**
     * The id of the subscription to cancel.
     */
    subscriptionId?: string = 'sub_123'
    /**
     * URL to take customers to when they click on the billing portalâ€™s link to return to your website.
     */
    returnUrl: string = '/account'

}
```

This will redirect the user to the Stripe Billing Portal where they can cancel their subscription.

#### Restoring a Canceled Subscription

If a user changes their mind after canceling a subscription (but before the subscription period ends), you can restore the subscription:

### Client Side

```ts
const { data, error } = await authClient.subscription.restore({
    referenceId: 123, // required
    subscriptionId: sub_123, // required
});
```

### Server Side

```ts
const data = await auth.api.restoreSubscription({
    body: {
        referenceId: 123, // required
        subscriptionId: sub_123, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type restoreSubscription = {
    /**
     * Reference id of the subscription to restore. Defaults to the userId.
     */
    referenceId?: string = '123'
    /**
     * The id of the subscription to restore.
     */
    subscriptionId?: string = 'sub_123'

}
```

This will reactivate a subscription that was previously set to cancel at the end of the billing period (`cancelAtPeriodEnd: true`). The subscription will continue to renew automatically.

> **Note:** This only works for subscriptions that are still active but marked to cancel at the end of the period. It cannot restore subscriptions that have already ended.

#### Creating Billing Portal Sessions

To create a [Stripe billing portal session](https://docs.stripe.com/api/customer_portal/sessions/create) where customers can manage their subscriptions, update payment methods, and view billing history:

### Client Side

```ts
const { data, error } = await authClient.subscription.billingPortal({
    locale, // required
    referenceId: 123, // required
    returnUrl, // required
});
```

### Server Side

```ts
const data = await auth.api.createBillingPortal({
    body: {
        locale, // required
        referenceId: 123, // required
        returnUrl, // required
    },
    // This endpoint requires session cookies.
    headers: await headers()
});
```

### Type Definition

```ts
type createBillingPortal = {
    /**
    * The IETF language tag of the locale customer portal is displayed in. If blank or auto, browser's locale is used.
    */
    locale?: string
    /**
     * Reference id of the subscription to upgrade.
     */
    referenceId?: string = "123"
    /**
     * Return URL to redirect back after successful subscription.
     */
    returnUrl?: string

}
```

<Callout type="info">
  For supported locales, see the [IETF language tag documentation](https://docs.stripe.com/js/appendix/supported_locales).
</Callout>

This endpoint creates a Stripe billing portal session and returns a URL in the response as `data.url`. You can redirect users to this URL to allow them to manage their subscription, payment methods, and billing history.

### Reference System

By default, subscriptions are associated with the user ID. However, you can use a custom reference ID to associate subscriptions with other entities, such as organizations:

```ts title="client.ts"
// Create a subscription for an organization
await client.subscription.upgrade({
    plan: "pro",
    referenceId: "org_123456",
    successUrl: "/dashboard",
    cancelUrl: "/pricing",
    seats: 5 // Number of seats for team plans
});

// List subscriptions for an organization
const { data: subscriptions } = await client.subscription.list({
    query: {
        referenceId: "org_123456"
    }
});
```

#### Team Subscriptions with Seats

For team or organization plans, you can specify the number of seats:

```ts
await client.subscription.upgrade({
    plan: "team",
    referenceId: "org_123456",
    seats: 10, // 10 team members
    successUrl: "/org/billing/success",
    cancelUrl: "/org/billing"
});
```

The `seats` parameter is passed to Stripe as the quantity for the subscription item. You can use this value in your application logic to limit the number of members in a team or organization.

To authorize reference IDs, implement the `authorizeReference` function:

```ts title="auth.ts"
subscription: {
    // ... other options
    authorizeReference: async ({ user, session, referenceId, action }) => {
        // Check if the user has permission to manage subscriptions for this reference
        if (action === "upgrade-subscription" || action === "cancel-subscription" || action === "restore-subscription") {
            const org = await db.member.findFirst({
                where: {
                    organizationId: referenceId,
                    userId: user.id
                }
            });
            return org?.role === "owner"
        }
        return true;
    }
}
```

### Webhook Handling

The plugin automatically handles common webhook events:

* `checkout.session.completed`: Updates subscription status after checkout
* `customer.subscription.updated`: Updates subscription details when changed
* `customer.subscription.deleted`: Marks subscription as canceled

You can also handle custom events:

```ts title="auth.ts"
stripe({
    // ... other options
    onEvent: async (event) => {
        // Handle any Stripe event
        switch (event.type) {
            case "invoice.paid":
                // Handle paid invoice
                break;
            case "payment_intent.succeeded":
                // Handle successful payment
                break;
        }
    }
})
```

### Subscription Lifecycle Hooks

You can hook into various subscription lifecycle events:

```ts title="auth.ts"
subscription: {
    // ... other options
    onSubscriptionComplete: async ({ event, subscription, stripeSubscription, plan }) => {
        // Called when a subscription is successfully created
        await sendWelcomeEmail(subscription.referenceId, plan.name);
    },
    onSubscriptionUpdate: async ({ event, subscription }) => {
        // Called when a subscription is updated
        console.log(`Subscription ${subscription.id} updated`);
    },
    onSubscriptionCancel: async ({ event, subscription, stripeSubscription, cancellationDetails }) => {
        // Called when a subscription is canceled
        await sendCancellationEmail(subscription.referenceId);
    },
    onSubscriptionDeleted: async ({ event, subscription, stripeSubscription }) => {
        // Called when a subscription is deleted
        console.log(`Subscription ${subscription.id} deleted`);
    }
}
```

### Trial Periods

You can configure trial periods for your plans:

```ts title="auth.ts"
{
    name: "pro",
    priceId: "price_0987654321",
    freeTrial: {
        days: 14,
        onTrialStart: async (subscription) => {
            // Called when a trial starts
            await sendTrialStartEmail(subscription.referenceId);
        },
        onTrialEnd: async ({ subscription, user }, request) => {
            // Called when a trial ends
            await sendTrialEndEmail(user.email);
        },
        onTrialExpired: async (subscription) => {
            // Called when a trial expires without conversion
            await sendTrialExpiredEmail(subscription.referenceId);
        }
    }
}
```

## Schema

The Stripe plugin adds the following tables to your database:

### User

Table Name: `user`

```
<DatabaseTable
  fields={[
  {
    name: "stripeCustomerId",
    type: "string",
    description: "The Stripe customer ID",
    isOptional: true
  },
]}
/>
```

### Subscription

Table Name: `subscription`

```
<DatabaseTable
  fields={[
  {
    name: "id",
    type: "string",
    description: "Unique identifier for each subscription",
    isPrimaryKey: true
  },
  {
    name: "plan",
    type: "string",
    description: "The name of the subscription plan"
  },
  {
    name: "referenceId",
    type: "string",
    description: "The ID this subscription is associated with (user ID by default)",
    isUnique: true
  },
  {
    name: "stripeCustomerId",
    type: "string",
    description: "The Stripe customer ID",
    isOptional: true
  },
  {
    name: "stripeSubscriptionId",
    type: "string",
    description: "The Stripe subscription ID",
    isOptional: true
  },
  {
    name: "status",
    type: "string",
    description: "The status of the subscription (active, canceled, etc.)",
    defaultValue: "incomplete"
  },
  {
```

    name: "periodStart",
    type: "Date",
    description: "Start date of the current billing period",
    isOptional: true
  },
  {
    name: "periodEnd",
    type: "Date",
    description: "End date of the current billing period",
    isOptional: true
  },
  {
    name: "cancelAtPeriodEnd",
    type: "boolean",
    description: "Whether the subscription will be canceled at the end of the period",
    defaultValue: false,
    isOptional: true
  },
  {
    name: "seats",
    type: "number",
    description: "Number of seats for team plans",
    isOptional: true
  },
  {
    name: "trialStart",
    type: "Date",
    description: "Start date of the trial period",
    isOptional: true
  },
  {
    name: "trialEnd",
    type: "Date",
    description: "End date of the trial period",
    isOptional: true
  }
]}
/>

### Customizing the Schema

To change the schema table names or fields, you can pass a `schema` option to the Stripe plugin:

```ts title="auth.ts"
stripe({
    // ... other options
    schema: {
        subscription: {
            modelName: "stripeSubscriptions", // map the subscription table to stripeSubscriptions
            fields: {
                plan: "planName" // map the plan field to planName
            }
        }
    }
})
```

## Options

### Main Options

stripeClient: `Stripe` - The Stripe client instance. Required.

stripeWebhookSecret: `string` - The webhook signing secret from Stripe. Required.

createCustomerOnSignUp: `boolean` - Whether to automatically create a Stripe customer when a user signs up. Default: `false`.

onCustomerCreate: `(data: { customer: Customer, stripeCustomer: Stripe.Customer, user: User }, request?: Request) => Promise<void>` - A function called after a customer is created.

getCustomerCreateParams: `(data: { user: User, session: Session }, request?: Request) => Promise<{}>` - A function to customize the Stripe customer creation parameters.

onEvent: `(event: Stripe.Event) => Promise<void>` - A function called for any Stripe webhook event.

### Subscription Options

enabled: `boolean` - Whether to enable subscription functionality. Required.

plans: `Plan[] | (() => Promise<Plan[]>)` - An array of subscription plans or a function that returns plans. Required if subscriptions are enabled.

requireEmailVerification: `boolean` - Whether to require email verification before allowing subscription upgrades. Default: `false`.

authorizeReference: `(data: { user: User, session: Session, referenceId: string, action: "upgrade-subscription" | "list-subscription" | "cancel-subscription" | "restore-subscription"}, request?: Request) => Promise<boolean>` - A function to authorize reference IDs.

### Plan Configuration

Each plan can have the following properties:

name: `string` - The name of the plan. Required.

priceId: `string` - The Stripe price ID. Required unless using `lookupKey`.

lookupKey: `string` - The Stripe price lookup key. Alternative to `priceId`.

annualDiscountPriceId: `string` - A price ID for annual billing.

annualDiscountLookupKey: `string` - The Stripe price lookup key for annual billing. Alternative to `annualDiscountPriceId`.

limits: `Record<string, number>` - Limits associated with the plan (e.g., `{ projects: 10, storage: 5 }`).

group: `string` - A group name for the plan, useful for categorizing plans.

freeTrial: Object containing trial configuration:

* **days**: `number` - Number of trial days.
* **onTrialStart**: `(subscription: Subscription) => Promise<void>` - Called when a trial starts.
* **onTrialEnd**: `(data: { subscription: Subscription, user: User }, request?: Request) => Promise<void>` - Called when a trial ends.
* **onTrialExpired**: `(subscription: Subscription) => Promise<void>` - Called when a trial expires without conversion.

## Advanced Usage

### Using with Organizations

The Stripe plugin works well with the organization plugin. You can associate subscriptions with organizations instead of individual users:

```ts title="client.ts"
// Get the active organization
const { data: activeOrg } = client.useActiveOrganization();

// Create a subscription for the organization
await client.subscription.upgrade({
    plan: "team",
    referenceId: activeOrg.id,
    seats: 10,
    annual: true, // upgrade to an annual plan (optional)
    successUrl: "/org/billing/success",
    cancelUrl: "/org/billing"
});
```

Make sure to implement the `authorizeReference` function to verify that the user has permission to manage subscriptions for the organization:

```ts title="auth.ts"
authorizeReference: async ({ user, referenceId, action }) => {
    const member = await db.members.findFirst({
        where: {
            userId: user.id,
            organizationId: referenceId
        }
    });

    return member?.role === "owner" || member?.role === "admin";
}
```

### Custom Checkout Session Parameters

You can customize the Stripe Checkout session with additional parameters:

```ts title="auth.ts"
getCheckoutSessionParams: async ({ user, session, plan, subscription }, request) => {
    return {
        params: {
            allow_promotion_codes: true,
            tax_id_collection: {
                enabled: true
            },
            billing_address_collection: "required",
            custom_text: {
                submit: {
                    message: "We'll start your subscription right away"
                }
            },
            metadata: {
                planType: "business",
                referralCode: user.metadata?.referralCode
            }
        },
        options: {
            idempotencyKey: `sub_${user.id}_${plan.name}_${Date.now()}`
        }
    };
}
```

### Tax Collection

To collect tax IDs from the customer, set `tax_id_collection` to true:

```ts title="auth.ts"
subscription: {
    // ... other options
    getCheckoutSessionParams: async ({ user, session, plan, subscription }, request) => {
        return {
            params: {
                tax_id_collection: {
                    enabled: true
                }
            }
        };
    }
}
```

### Automatic Tax Calculation

To enable automatic tax calculation using the customer's location, set `automatic_tax` to true. Enabling this parameter causes Checkout to collect any billing address information necessary for tax calculation. You need to have tax registration setup and configured in the Stripe dashboard first for this to work.

```ts title="auth.ts"
subscription: {
    // ... other options
    getCheckoutSessionParams: async ({ user, session, plan, subscription }, request) => {
        return {
            params: {
                automatic_tax: {
                    enabled: true
                }
            }
        };
    }
}
```

### Trial Period Management

The Stripe plugin automatically prevents users from getting multiple free trials. Once a user has used a trial period (regardless of which plan), they will not be eligible for additional trials on any plan.

How it works:

* The system tracks trial usage across all plans for each user
* When a user subscribes to a plan with a trial, the system checks their subscription history
* If the user has ever had a trial (indicated by `trialStart`/`trialEnd` fields or `trialing` status), no new trial will be offered
* This prevents abuse where users cancel subscriptions and resubscribe to get multiple free trials

Example scenario:

1. User subscribes to "Starter" plan with 7-day trial
2. User cancels the subscription after the trial
3. User tries to subscribe to "Premium" plan - no trial will be offered
4. User will be charged immediately for the Premium plan

This behavior is automatic and requires no additional configuration. The trial eligibility is determined at the time of subscription creation and cannot be overridden through configuration.

## Troubleshooting

### Webhook Issues

If webhooks aren't being processed correctly:

1. Check that your webhook URL is correctly configured in the Stripe dashboard
2. Verify that the webhook signing secret is correct
3. Ensure you've selected all the necessary events in the Stripe dashboard
4. Check your server logs for any errors during webhook processing

### Subscription Status Issues

If subscription statuses aren't updating correctly:

1. Make sure the webhook events are being received and processed
2. Check that the `stripeCustomerId` and `stripeSubscriptionId` fields are correctly populated
3. Verify that the reference IDs match between your application and Stripe

### Testing Webhooks Locally

For local development, you can use the Stripe CLI to forward webhooks to your local environment:

```bash

````
stripe listen --forward-to localhost:3000/api/auth/stripe/webhook

This will provide you with a webhook signing secret that you can use in your local environment.

# plugins: Username
URL: /docs/plugins/username
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/plugins/username.mdx

Username plugin

***

title: Username
description: Username plugin

The username plugin is a lightweight plugin that adds username support to the email and password authenticator. This allows users to sign in and sign up with their username
instead of their email.

## Installation

<Steps>
  <Step>
    ### Add Plugin to the server

    ```ts title="auth.ts"
    import { betterAuth } from "better-auth"
    import { username } from "better-auth/plugins"

    export const auth = betterAuth({
        plugins: [ // [!code highlight]
            username() // [!code highlight]
        ] // [!code highlight]
    })
    ```
  </Step>

  <Step>
    ### Migrate the database

    Run the migration or generate the schema to add the necessary fields and tables to the database.

    <Tabs items={["migrate", "generate"]}>
      <Tab value="migrate">
        ```bash
        npx @better-auth/cli migrate
        ```
      </Tab>

      <Tab value="generate">
        ```bash
        npx @better-auth/cli generate
        ```
      </Tab>
    </Tabs>

    See the [Schema](#schema) section to add the fields manually.
  </Step>

  <Step>
    ### Add the client plugin

    ```ts title="auth-client.ts"
    import { createAuthClient } from "better-auth/client"
    import { usernameClient } from "better-auth/client/plugins"

    export const authClient = createAuthClient({
        plugins: [ // [!code highlight]
            usernameClient() // [!code highlight]
        ] // [!code highlight]
    })
    ```
  </Step>
</Steps>

## Usage

### Sign up

To sign up a user with username, you can use the existing `signUp.email` function provided by the client.
The `signUp` function should take a new `username` property in the object.

### Client Side

```ts
const { data, error } = await authClient.signUp.email({
    email: email@domain.com,
    name: Test User,
    password: password1234,
    username: test,
    displayUsername: Test User123, // required
});
```

### Server Side

```ts
const data = await auth.api.signUpEmail({
    body: {
        email: email@domain.com,
        name: Test User,
        password: password1234,
        username: test,
        displayUsername: Test User123, // required
    }
});
```

### Type Definition

```ts
type signUpEmail = {
    /**
     * The email of the user.
    */
    email: string = "email@domain.com"
    /**
     * The name of the user.
    */
    name: string = "Test User"
    /**
     * The password of the user.
    */
    password: string = "password1234"
    /**
     * The username of the user.
````

~~~
    */
    username: string = "test"
    /**
     * An optional display username of the user.
    */
    displayUsername?: string = "Test User123"

}
```

<Callout type="info">
  If only `username` is provided, the `displayUsername` will be set to the pre normalized version of the `username`. You can see the [Username Normalization](#username-normalization) and [Display Username Normalization](#display-username-normalization) sections for more details.
</Callout>

### Sign in

To sign in a user with username, you can use the `signIn.username` function provided by the client.

### Client Side

```ts
const { data, error } = await authClient.signIn.username({
    username: test,
    password: password1234,
});
```

### Server Side

```ts
const data = await auth.api.signInUsername({
    body: {
        username: test,
        password: password1234,
    }
});
```

### Type Definition

```ts
type signInUsername = {
    /**
     * The username of the user.
    */
    username: string = "test"
    /**
     * The password of the user.
    */
    password: string = "password1234"

}
```

### Update username

To update the username of a user, you can use the `updateUser` function provided by the client.

### Client Side

```ts
const { data, error } = await authClient.updateUser({
    username: new-username, // required
});
```

### Server Side

```ts
const data = await auth.api.updateUser({
    body: {
        username: new-username, // required
    }
});
```

### Type Definition

```ts
type updateUser = {
    /**
     * The username to update.
    */
    username?: string = "new-username"

}
```

### Check if username is available

To check if a username is available, you can use the `isUsernameAvailable` function provided by the client.

### Client Side

```ts
const { data, error } = await authClient.isUsernameAvailable({
    username: new-username,
});
```

### Server Side

```ts
const response = await auth.api.isUsernameAvailable({
    body: {
        username: new-username,
    }
});
```

### Type Definition

```ts
type isUsernameAvailable = {
    /**
     * The username to check.
    */
    username: string = "new-username"

}
```

## Options

### Min Username Length

The minimum length of the username. Default is `3`.
~~~

```ts title="auth.ts"
import { betterAuth } from "better-auth"
import { username } from "better-auth/plugins"

const auth = betterAuth({
    plugins: [
        username({
            minUsernameLength: 5
        })
    ]
})
```

### Max Username Length

The maximum length of the username. Default is `30`.

```ts title="auth.ts"
import { betterAuth } from "better-auth"
import { username } from "better-auth/plugins"

const auth = betterAuth({
    plugins: [
        username({
            maxUsernameLength: 100
        })
    ]
})
```

### Username Validator

A function that validates the username. The function should return false if the username is invalid. By default, the username should only contain alphanumeric characters, underscores, and dots.

```ts title="auth.ts"
import { betterAuth } from "better-auth"
import { username } from "better-auth/plugins"

const auth = betterAuth({
    plugins: [
        username({
            usernameValidator: (username) => {
                if (username === "admin") {
                    return false
                }
                return true
            }
        })
    ]
})
```

### Display Username Validator

A function that validates the display username. The function should return false if the display username is invalid. By default, no validation is applied to display username.

```ts title="auth.ts"
import { betterAuth } from "better-auth"
import { username } from "better-auth/plugins"

const auth = betterAuth({
    plugins: [
        username({
            displayUsernameValidator: (displayUsername) => {
                // Allow only alphanumeric characters, underscores, and hyphens
                return /^[a-zA-Z0-9_-]+$/.test(displayUsername)
            }
        })
    ]
})
```

### Username Normalization

A function that normalizes the username, or `false` if you want to disable normalization.

By default, usernames are normalized to lowercase, so "TestUser" and "testuser", for example, are considered the same username. The `username` field will contain the normalized (lower case) username, while `displayUsername` will contain the original `username`.

```ts title="auth.ts"
import { betterAuth } from "better-auth"
import { username } from "better-auth/plugins"

const auth = betterAuth({
    plugins: [
        username({
            usernameNormalization: (username) => {
                return username.toLowerCase()
                    .replaceAll("0", "o")
                    .replaceAll("3", "e")
                    .replaceAll("4", "a");
            }
        })
    ]
})
```

### Display Username Normalization

A function that normalizes the display username, or `false` to disable normalization.

By default, display usernames are not normalized. When only `username` is provided during signup or update, the `displayUsername` will be set to match the original `username` value (before normalization). You can also explicitly set a `displayUsername` which will be preserved as-is. For custom normalization, provide a function that takes the display username as input and returns the normalized version.

```ts title="auth.ts"
import { betterAuth } from "better-auth"
import { username } from "better-auth/plugins"

const auth = betterAuth({
    plugins: [
        username({
            displayUsernameNormalization: (displayUsername) => displayUsername.toLowerCase(),
        })
    ]
})
```

### Validation Order

By default, username and display username are validated before normalization. You can change this behavior by setting `validationOrder` to `post-normalization`.

```ts title="auth.ts"
import { betterAuth } from "better-auth"
import { username } from "better-auth/plugins"

````
const auth = betterAuth({
    plugins: [
        username({
            validationOrder: {
                username: "post-normalization",
                displayUsername: "post-normalization",
            }
        })
    ]
})

## Schema

The plugin requires 2 fields to be added to the user table:

<DatabaseTable
  fields={[
      {
          name: "username",
          type: "string",
          description: "The username of the user",
          isUnique: true
      },
      {
          name: "displayUsername",
          type: "string",
          description: "Non normalized username of the user",
          isUnique: true
      },
  ]}
/>

# reference: Contributing to BetterAuth
URL: /docs/reference/contributing
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/reference/contributing.mdx

A concise guide to contributing to BetterAuth

***

title: Contributing to BetterAuth
description: A concise guide to contributing to BetterAuth

Thank you for your interest in contributing to Better Auth! This guide is a concise guide to contributing to Better Auth.

## Getting Started

Before diving in, here are a few important resources:

* Take a look at our existing <Link href="https://github.com/better-auth/better-auth/issues">issues</Link> and <Link href="https://github.com/better-auth/better-auth/pulls">pull requests</Link>
* Join our community discussions in <Link href="https://discord.gg/better-auth">Discord</Link>

## Development Setup

To get started with development:

<Callout type="warn">
  Make sure you have <Link href="https://nodejs.org/en/download">Node.JS</Link>{" "}
  installed, preferably on LTS.
</Callout>

<Steps>
  <Step>
    ### 1. Fork the repository

    Visit https://github.com/better-auth/better-auth

    Click the "Fork" button in the top right.
  </Step>

  <Step>
    ### 2. Clone your fork

    ```bash
    # Replace YOUR-USERNAME with your GitHub username
    git clone https://github.com/YOUR-USERNAME/better-auth.git
    cd better-auth
    ```
  </Step>

  <Step>
    ### 3. Install dependencies

    Make sure you have <Link href="https://pnpm.io/installation">pnpm</Link> installed!

    ```bash
    pnpm install
    ```
  </Step>

  <Step>
    ### 4. Prepare ENV files

    Copy the example env file to create your new `.env` file.

    ```bash
    cp -n ./docs/.env.example ./docs/.env
    ```
  </Step>
</Steps>

## Making changes

Once you have an idea of what you want to contribute, you can start making changes. Here are some steps to get started:

<Steps>
  <Step>
    ### 1. Create a new branch

    ```bash
    # Make sure you're on main
    git checkout main

    # Pull latest changes
    git pull upstream main

    # Create and switch to a new branch
    git checkout -b feature/your-feature-name
    ```
  </Step>

  <Step>
    ### 2. Start development server

    Start the development server:
~~~

    ```bash
    pnpm dev
    ```

    To start the docs server:

    ```bash
    pnpm -F docs dev
    ```
  </Step>

  <Step>
    ### 3. Make Your Changes

    * Make your changes to the codebase.

    * Write tests if needed. (Read more about testing <Link href="/docs/reference/contributing#testing">here</Link>)

    * Update documentation. (Read more about documenting <Link href="/docs/reference/contributing#documentation">here</Link>)
  </Step>
</Steps>

Issues and Bug Fixes

* Check our [GitHub issues](https://github.com/better-auth/better-auth/issues) for tasks labeled `good first issue`
* When reporting bugs, include steps to reproduce and expected behavior
* Comment on issues you'd like to work on to avoid duplicate efforts

Framework Integrations

We welcome contributions to support more frameworks:

* Focus on framework-agnostic solutions where possible
* Keep integrations minimal and maintainable
* All integrations currently live in the main package

Plugin Development

* For core plugins: Open an issue first to discuss your idea
* For community plugins: Feel free to develop independently
* Follow our plugin architecture guidelines

Documentation

* Fix typos and errors
* Add examples and clarify existing content
* Ensure documentation is up to date with code changes

Testing

We use Vitest for testing. Place test files next to the source files they test:

```ts
import { describe, it, expect } from "vitest";
import { getTestInstance } from "./test-utils/test-instance";

describe("Feature", () => {
    it("should work as expected", async () => {
        const { client } = await getTestInstance();
        // Test code here
        expect(result).toBeDefined();
    });
});
```

Using the Test Instance Helper

The test instance helper now includes improved async context support for managing user sessions:

```ts
const { client, runWithUser, signInWithTestUser } = await getTestInstance();

// Run tests with a specific user context
await runWithUser("user@example.com", "password", async (headers) => {
    // All client calls within this block will use the user's session
    const response = await client.getSession();
    // headers are automatically applied
});

// Or use the test user with async context
const { runWithDefaultUser } = await signInWithTestUser();
await runWithDefaultUser(async (headers) => {
    // Code here runs with the test user's session context
});
```

Testing Best Practices

* Write clear commit messages
* Update documentation to reflect your changes
* Add tests for new features
* Follow our coding standards
* Keep pull requests focused on a single change

Need Help?

Don't hesitate to ask for help! You can:

* Open an <Link href="https://github.com/better-auth/better-auth/issues">issue</Link> with questions
* Join our <Link href="https://discord.gg/better-auth">community discussions</Link>
* Reach out to project maintainers

Thank you for contributing to Better Auth!

reference: FAQ
URL: /docs/reference/faq
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/reference/faq.mdx

Frequently asked questions about Better Auth.

***

title: FAQ
description: Frequently asked questions about Better Auth.

This page contains frequently asked questions, common issues, and other helpful information about Better Auth.

<Accordions>
  <Accordion title="Auth client not working">
    When encountering `createAuthClient` related errors, make sure to have the correct import path as it varies based on environment.

    If you're using the auth client on react front-end, you'll need to import it from `/react`:

    ```ts title="component.ts"
    import { createAuthClient } from "better-auth/react";
    ```

    Where as if you're using the auth client in Next.js middleware, server-actions, server-components or anything server-related, you'll likely need to import it from `/client`:

````mdx
    ```ts title="server.ts"
    import { createAuthClient } from "better-auth/client";
    ```
  </Accordion>

  <Accordion title="getSession not working">
    If you try to call `authClient.getSession` on a server environment (e.g, a Next.js server component), it doesn't work since it can't access the cookies. You can use the `auth.api.getSession` instead and pass the request headers to it.

    ```tsx title="server.tsx"
    import { auth } from "./auth";
    import { headers } from "next/headers";

    const session = await auth.api.getSession({
        headers: await headers()
    })
    ```

    if you need to use the auth client on the server for different purposes, you still can pass the request headers to it:

    ```tsx title="server.tsx"
    import { authClient } from "./auth-client";
    import { headers } from "next/headers";

    const session = await authClient.getSession({
        fetchOptions:{
          headers: await headers()
        }
    })
    ```
  </Accordion>

  <Accordion title="Adding custom fields to the users table">
    Better Auth provides a type-safe way to extend the user and session schemas, take a look at our docs on <Link href="/docs/concepts/database#extending-core-schema">extending core schema</Link>.
  </Accordion>

  <Accordion title="Difference between getSession and useSession">
    Both `useSession` and `getSession` instances are used fundamentally different based on the situation.

    `useSession` is a hook, meaning it can trigger re-renders whenever session data changes.

    If you have UI you need to change based on user or session data, you can use this hook.

    <Callout type="warn">
      For performance reasons, do not use this hook on your `layout.tsx` file. We
      recommend using RSC and use your server auth instance to get the session data
      via `auth.api.getSession`.
    </Callout>

    `getSession` returns a promise containing data and error.

    For all other situations where you shouldn't use `useSession`, is when you should be using `getSession`.

    <Callout type="info">
      `getSession` is available on both server and client auth instances.
      Not just the latter.
    </Callout>
  </Accordion>

  <Accordion title="Common TypeScript Errors">
    If you're facing typescript errors, make sure your tsconfig has `strict` set to `true`:

    ```json title="tsconfig.json"
    {
      "compilerOptions": {
        "strict": true,
      }
    }
    ```

    if you can't set strict to true, you can enable strictNullChecks:

    ```json title="tsconfig.json"
    {
      "compilerOptions": {
        "strictNullChecks": true,
      }
    }
    ```

    You can learn more in our <Link href="/docs/concepts/typescript#typescript-config">TypeScript docs</Link>.
  </Accordion>

  <Accordion title="Can I remove `name`, `image`, or `email` fields from the user table?">
    At this time, you can't remove the `name`, `image`, or `email` fields from the user table.

    We do plan to have more customizability in the future in this regard, but for now, you can't remove these fields.
  </Accordion>
</Accordions>



# reference: Options
URL: /docs/reference/options
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/reference/options.mdx

Better Auth configuration options reference.

***

title: Options
description: Better Auth configuration options reference.
----------------------------------------------------------

List of all the available options for configuring Better Auth. See [Better Auth Options](https://github.com/better-auth/better-auth/blob/main/packages/better-auth/src/types/options.ts#L13).

## `appName`

The name of the application.

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
        appName: "My App",
})
```

## `baseURL`

Base URL for Better Auth. This is typically the root URL where your application server is hosted. Note: If you include a path in the baseURL, it will take precedence over the default path.

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
        baseURL: "https://example.com",
})
```
~~~

If not explicitly set, the system will check for the environment variable `process.env.BETTER_AUTH_URL`

## `basePath`

Base path for Better Auth. This is typically the path where the Better Auth routes are mounted. It will be overridden if there is a path component within `baseURL`.

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
        basePath: "/api/auth",
})
```

Default: `/api/auth`

## `trustedOrigins`

List of trusted origins. You can provide a static array of origins, a function that returns origins dynamically, or use wildcard patterns to match multiple domains.

### Static Origins

You can provide a static array of origins:

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
        trustedOrigins: ["http://localhost:3000", "https://example.com"],
})
```

### Dynamic Origins

You can provide a function that returns origins dynamically:

```ts
export const auth = betterAuth({
        trustedOrigins: async (request: Request) => {
                // Return an array of trusted origins based on the request
                return ["https://dynamic-origin.com"];
        }
})
```

### Wildcard Support

You can use wildcard patterns in trusted origins:

```ts
export const auth = betterAuth({
        trustedOrigins: [
                "*.example.com",             // Trust all subdomains of example.com
                "https://*.example.com",     // Trust only HTTPS subdomains
                "http://*.dev.example.com"   // Trust HTTP subdomains of dev.example.com
        ]
})
```

## `secret`

The secret used for encryption, signing, and hashing.

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
        secret: "your-secret-key",
})
```

By default, Better Auth will look for the following environment variables:

* `process.env.BETTER_AUTH_SECRET`
* `process.env.AUTH_SECRET`

If none of these environment variables are set, it will default to `"better-auth-secret-123456789"`. In production, if it's not set, it will throw an error.

You can generate a good secret using the following command:

```bash
openssl rand -base64 32
```

## `database`

Database configuration for Better Auth.

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
        database: {
                dialect: "postgres",
                type: "postgres",
                casing: "camel"
        },
})
```

Better Auth supports various database configurations including [PostgreSQL](/docs/adapters/postgresql), [MySQL](/docs/adapters/mysql), and [SQLite](/docs/adapters/sqlite).

Read more about databases [here](/docs/concepts/database).

## `secondaryStorage`

Secondary storage configuration used to store session and rate limit data.

```ts
import { betterAuth } from "better-auth";

export const auth = betterAuth({
        // ... other options
    secondaryStorage: {
        // Your implementation here
    },
})
```

Read more about secondary storage [here](/docs/concepts/database#secondary-storage).

## `emailVerification`

Email verification configuration.

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
	emailVerification: {
		sendVerificationEmail: async ({ user, url, token }) => {
			// Send verification email to user
		},
		sendOnSignUp: true,
		autoSignInAfterVerification: true,
		expiresIn: 3600 // 1 hour
	},
})
```

* `sendVerificationEmail`: Function to send verification email
* `sendOnSignUp`: Send verification email automatically after sign up (default: `false`)
* `sendOnSignIn`: Send verification email automatically on sign in when the user's email is not verified (default: `false`)
* `autoSignInAfterVerification`: Auto sign in the user after they verify their email
* `expiresIn`: Number of seconds the verification token is valid for (default: `3600` seconds)

## `emailAndPassword`

Email and password authentication configuration.

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
	emailAndPassword: {
		enabled: true,
		disableSignUp: false,
		requireEmailVerification: true,
		minPasswordLength: 8,
		maxPasswordLength: 128,
		autoSignIn: true,
		sendResetPassword: async ({ user, url, token }) => {
			// Send reset password email
		},
		resetPasswordTokenExpiresIn: 3600, // 1 hour
		password: {
			hash: async (password) => {
				// Custom password hashing
				return hashedPassword;
			},
			verify: async ({ hash, password }) => {
				// Custom password verification
				return isValid;
			}
		}
	},
})
```

* `enabled`: Enable email and password authentication (default: `false`)
* `disableSignUp`: Disable email and password sign up (default: `false`)
* `requireEmailVerification`: Require email verification before a session can be created
* `minPasswordLength`: Minimum password length (default: `8`)
* `maxPasswordLength`: Maximum password length (default: `128`)
* `autoSignIn`: Automatically sign in the user after sign up
* `sendResetPassword`: Function to send reset password email
* `resetPasswordTokenExpiresIn`: Number of seconds the reset password token is valid for (default: `3600` seconds)
* `password`: Custom password hashing and verification functions

## `socialProviders`

Configure social login providers.

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
	socialProviders: {
		google: {
			clientId: "your-client-id",
			clientSecret: "your-client-secret",
			redirectURI: "https://example.com/api/auth/callback/google"
		},
		github: {
			clientId: "your-client-id",
			clientSecret: "your-client-secret",
			redirectURI: "https://example.com/api/auth/callback/github"
		}
	},
})
```

## `plugins`

List of Better Auth plugins.

```ts
import { betterAuth } from "better-auth";
import { emailOTP } from "better-auth/plugins";

export const auth = betterAuth({
	plugins: [
		emailOTP({
			sendVerificationOTP: async ({ email, otp, type }) => {
				// Send OTP to user's email
			}
		})
	],
})
```

## `user`

User configuration options.

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
	user: {
		modelName: "users",
		fields: {
			email: "emailAddress",
			name: "fullName"
		},
		additionalFields: {
			customField: {
				type: "string",
			}
		},
		changeEmail: {
			enabled: true,
			sendChangeEmailVerification: async ({ user, newEmail, url, token }) => {
				// Send change email verification
			}
		},
		deleteUser: {
			enabled: true,
			sendDeleteAccountVerification: async ({ user, url, token }) => {
				// Send delete account verification
			},
			beforeDelete: async (user) => {
				// Perform actions before user deletion
			},
			afterDelete: async (user) => {
				// Perform cleanup after user deletion
			}
		}
	},
})
```

* `modelName`: The model name for the user (default: `"user"`)
* `fields`: Map fields to different column names
* `additionalFields`: Additional fields for the user table
* `changeEmail`: Configuration for changing email
* `deleteUser`: Configuration for user deletion

## `session`

Session configuration options.

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
	session: {
		modelName: "sessions",
		fields: {
			userId: "user_id"
		},
		expiresIn: 604800, // 7 days
		updateAge: 86400, // 1 day
		disableSessionRefresh: true, // Disable session refresh so that the session is not updated regardless of the `updateAge` option. (default: `false`)
		additionalFields: { // Additional fields for the session table
			customField: {
				type: "string",
			}
		},
		storeSessionInDatabase: true, // Store session in database when secondary storage is provided (default: `false`)
		preserveSessionInDatabase: false, // Preserve session records in database when deleted from secondary storage (default: `false`)
		cookieCache: {
			enabled: true, // Enable caching session in cookie (default: `false`)
			maxAge: 300 // 5 minutes
		}
	},
})
```

* `modelName`: The model name for the session (default: `"session"`)
* `fields`: Map fields to different column names
* `expiresIn`: Expiration time for the session token in seconds (default: `604800` - 7 days)
* `updateAge`: How often the session should be refreshed in seconds (default: `86400` - 1 day)
* `additionalFields`: Additional fields for the session table
* `storeSessionInDatabase`: Store session in database when secondary storage is provided (default: `false`)
* `preserveSessionInDatabase`: Preserve session records in database when deleted from secondary storage (default: `false`)
* `cookieCache`: Enable caching session in cookie

## `account`

Account configuration options.

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
	account: {
		modelName: "accounts",
		fields: {
			userId: "user_id"
		},
		encryptOAuthTokens: true, // Encrypt OAuth tokens before storing them in the database
		accountLinking: {
			enabled: true,
			trustedProviders: ["google", "github", "email-password"],
			allowDifferentEmails: false
		}
	},
})
```

* `modelName`: The model name for the account
* `fields`: Map fields to different column names

### `encryptOAuthTokens`

Encrypt OAuth tokens before storing them in the database. Default: `false`.

### `updateAccountOnSignIn`

If enabled (true), the user account data (accessToken, idToken, refreshToken, etc.) will be updated on sign in with the latest data from the provider.

### `accountLinking`

Configuration for account linking.

* `enabled`: Enable account linking (default: `false`)
* `trustedProviders`: List of trusted providers
* `allowDifferentEmails`: Allow users to link accounts with different email addresses
* `allowUnlinkingAll`: Allow users to unlink all accounts

## `verification`

Verification configuration options.

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
	verification: {
		modelName: "verifications",
		fields: {
			userId: "user_id"
		},
		disableCleanup: false
	},
})
```

* `modelName`: The model name for the verification table
* `fields`: Map fields to different column names
* `disableCleanup`: Disable cleaning up expired values when a verification value is fetched

## `rateLimit`

Rate limiting configuration.

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
	rateLimit: {
		enabled: true,
		window: 10,
		max: 100,
		customRules: {
			"/example/path": {
				window: 10,
				max: 100
			}
		},
		storage: "memory",
		modelName: "rateLimit"
	}
})
```

* `enabled`: Enable rate limiting (defaults: `true` in production, `false` in development)
* `window`: Time window to use for rate limiting. The value should be in seconds. (default: `10`)
* `max`: The default maximum number of requests allowed within the window. (default: `100`)
* `customRules`: Custom rate limit rules to apply to specific paths.
* `storage`: Storage configuration. If you passed a secondary storage, rate limiting will be stored in the secondary storage. (options: `"memory", "database", "secondary-storage"`, default: `"memory"`)
* `modelName`: The name of the table to use for rate limiting if database is used as storage. (default: `"rateLimit"`)

## `advanced`

Advanced configuration options.

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
	advanced: {
		ipAddress: {
			ipAddressHeaders: ["x-client-ip", "x-forwarded-for"],
			disableIpTracking: false
		},
		useSecureCookies: true,
		disableCSRFCheck: false,
		crossSubDomainCookies: {
			enabled: true,
			additionalCookies: ["custom_cookie"],
			domain: "example.com"
		},
		cookies: {
			session_token: {
				name: "custom_session_token",
				attributes: {
					httpOnly: true,
					secure: true
				}
			}
		},
		defaultCookieAttributes: {
			httpOnly: true,
			secure: true
		},
		cookiePrefix: "myapp",
		database: {
			// If your DB is using auto-incrementing IDs, set this to true.
			useNumberId: false,
			// Use your own custom ID generator, or disable generating IDs as a whole.
			generateId: (((options: {
				model: LiteralUnion<Models, string>;
				size?: number;
			}) => {
				return "my-super-unique-id";
			})) | false,
			defaultFindManyLimit: 100,
		}
	},
})
```

* `ipAddress`: IP address configuration for rate limiting and session tracking
* `useSecureCookies`: Use secure cookies (default: `false`)
* `disableCSRFCheck`: Disable trusted origins check (âš ï¸  security risk)
* `crossSubDomainCookies`: Configure cookies to be shared across subdomains
* `cookies`: Customize cookie names and attributes
* `defaultCookieAttributes`: Default attributes for all cookies
* `cookiePrefix`: Prefix for cookies
* `generateId`: Function to generate a unique ID for a model

## `logger`

Logger configuration for Better Auth.

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
	logger: {
		disabled: false,
		disableColors: false,
		level: "error",
		log: (level, message, ...args) => {
			// Custom logging implementation
			console.log(`[${level}] ${message}`, ...args);
		}
	}
})
```

The logger configuration allows you to customize how Better Auth handles logging. It supports the following options:

* `disabled`: Disable all logging when set to `true` (default: `false`)
* `disableColors`: Disable colors in the default logger implementation (default: determined by the terminal's color support)
* `level`: Set the minimum log level to display. Available levels are:
  * `"info"`: Show all logs
  * `"warn"`: Show warnings and errors
  * `"error"`: Show only errors
  * `"debug"`: Show all logs including debug information
* `log`: Custom logging function that receives:
  * `level`: The log level (`"info"`, `"warn"`, `"error"`, or `"debug"`)
  * `message`: The log message
  * `...args`: Additional arguments passed to the logger

Example with custom logging:

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
	logger: {
		level: "info",
		log: (level, message, ...args) => {
			// Send logs to a custom logging service
			myLoggingService.log({
				level,
				message,
				metadata: args,
				timestamp: new Date().toISOString()
			});
		}
	}
})
```

## `databaseHooks`

Database lifecycle hooks for core operations.

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
	databaseHooks: {
		user: {
			create: {
				before: async (user) => {
					// Modify user data before creation
					return { data: { ...user, customField: "value" } };
				},
				after: async (user) => {
					// Perform actions after user creation
				}
			},
			update: {
				before: async (userData) => {
					// Modify user data before update
					return { data: { ...userData, updatedAt: new Date() } };
				},
				after: async (user) => {
					// Perform actions after user update
				}
			}
		},
		session: {
			// Session hooks
		},
		account: {
			// Account hooks
		},
		verification: {
			// Verification hooks
		}
	},
})
```

## `onAPIError`

API error handling configuration.

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
	onAPIError: {
		throw: true,
		onError: (error, ctx) => {
			// Custom error handling
			console.error("Auth error:", error);
		},
		errorURL: "/auth/error"
	},
})
```

* `throw`: Throw an error on API error (default: `false`)
* `onError`: Custom error handler
* `errorURL`: URL to redirect to on error (default: `/api/auth/error`)

## `hooks`

Request lifecycle hooks.

```ts
import { betterAuth } from "better-auth";
import { createAuthMiddleware } from "better-auth/api";

export const auth = betterAuth({
	hooks: {
		before: createAuthMiddleware(async (ctx) => {
			// Execute before processing the request
			console.log("Request path:", ctx.path);
		}),
		after: createAuthMiddleware(async (ctx) => {
			// Execute after processing the request
			console.log("Response:", ctx.context.returned);
		})
	},
})
```

For more details and examples, see the [Hooks documentation](/docs/concepts/hooks).

## `disabledPaths`

Disable specific auth paths.

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
	disabledPaths: ["/sign-up/email", "/sign-in/email"],
})
```

## `telemetry`

Enable or disable Better Auth's telemetry collection. (default: `false`)

```ts
import { betterAuth } from "better-auth";
export const auth = betterAuth({
  telemetry: {
    enabled: false,
  }
})
```

# reference: Resources
URL: /docs/reference/resources
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/reference/resources.mdx

A curated collection of resources to help you learn and master Better Auth.

***

title: Resources
description: A curated collection of resources to help you learn and master Better Auth.
---------------------------------------------------------------------------------------

import { Resource } from "@/components/resource-section";

A curated collection of resources to help you learn and master Better Auth. From blog posts to video tutorials, find everything you need to get started.

## Video tutorials

```
<Resource
  resources={
  [
        {
                title: "The State of Authentication",
                description:
                        "<strong>Theo(t3.gg)</strong> explores the current landscape of authentication, discussing trends, challenges, and where the industry is heading.",
                href: "https://www.youtube.com/watch?v=lxslnp-ZEMw",
                tags: ["trends", "showcase", "review"],
        },
        {
                title: "Last Authentication You Will Ever Need",
                description:
                        "A comprehensive tutorial demonstrating why Better Auth could be the final authentication solution you'll need for your projects.",
                href: "https://www.youtube.com/watch?v=hFtufpaMcLM",
                tags: ["implementation", "showcase"],
        },
        {
                title: "This Might Be My New Favourite Auth Library",
                description:
                        "<strong>developedbyed</strong> explores the features and capabilities of Better Auth, explaining why it stands out among authentication libraries.",
                href: "https://www.youtube.com/watch?v=Hjs3zM7o7NE",
                tags: ["review", "showcase"],
        },
        {
                title: "8 Reasons To Try Better Auth",
                description:
                        "<strong>CJ</strong> presents 8 compelling reasons why Better Auth is the BEST auth framework he's ever used, demonstrating its superior features and ease of implementation.",
                href: "https://www.youtube.com/watch?v=_OApmLmex14",
                tags: ["review", "showcase", "implementation"],
        },
        {
                title: "Better Auth is so good that I almost switched programming languages",
                description:
                        "<strong>Dreams of Code</strong> reviews Better Auth's features that nearly made them switch languages.",
                href: "https://www.youtube.com/watch?v=dNY4FKXwTsM",
                tags: ["review", "showcase", "implementation"],
        },
        {
                title: "Best authentication framework for next.js",
                description:
                        "A detailed comparison of authentication frameworks for Next.js, highlighting why Better Auth might be your best choice.",
                href: "https://www.youtube.com/watch?v=V--T0q9FrEw",
                tags: ["nextjs", "comparison"],
        },
        {
                title: "Better-Auth: A First Look",
                description:
                        "An introductory overview and demonstration of Better Auth's core features and capabilities.",
                href: "https://www.youtube.com/watch?v=2cQTV6NYxis",
                tags: ["implementation", "showcase"],
        },
        {
                title: "Stripe was never so easy (with better auth)",
                description: "A tutorial on how to integrate Stripe with Better Auth.",
                href: "https://www.youtube.com/watch?v=g-RIrzBEX6M",
                tags: ["implementation"],
        },
        {
                title: "Nextjs 15 Authentication Made EASY with Better Auth",
                description:
                        "A practical guide showing how to seamlessly integrate Better Auth with Next.js 15 for robust authentication.",
                href: "https://www.youtube.com/watch?v=lxslnp-ZEMw",
                tags: ["nextjs", "implementation", "tutorial"],
        },
        {
                title: "Better Auth: Headless Authentication for Your TanStack Start App",
                description: "<strong>Jack</strong> demonstrates how to implement headless authentication in your TanStack Start application using Better Auth, providing a modern approach to auth.",
                href: "https://www.youtube.com/watch?v=Atev8Nxpw7c",
                tags: ["tanstack", "implementation"],
        },
        {
                title: "Goodbye Clerk, Hello Better Auth â€“ Full Migration Guide!",
                description: "A comprehensive guide showing how to migrate your authentication from Clerk to Better Auth, with step-by-step instructions and best practices.",
                href: "https://www.youtube.com/watch?v=Za_QihbDSuk",
                tags: ["migration", "clerk", "tutorial"],
        },
  ]
}
/>
```

## Blog posts

```
<Resource
  resources={
  [
      {
                title: "Better Auth with Hono, Bun, TypeScript, React and Vite",
                description:
                        "You'll learn how to implement authentication with Better Auth in a client - server architecture, where the frontend is separate from the backend.",
                href: "https://catalins.tech/better-auth-with-hono-bun-typescript-react-vite",
                tags: ["typescript", "react", "bun", "vite"],
        },
        {
                title: "Polar.sh + BetterAuth for Organizations",
                description:
                        "Polar.sh is a platform for building payment integrations. This article will show you how to use Better Auth to authenticate your users.",
                href: "https://dev.to/phumudzosly/polarsh-betterauth-for-organizations-1j1b",
                tags: ["organizations", "integration", "payments"],
        },
        {
                title: "Authenticating users in Astro with Better Auth",
                description:
                        "Step by step guide on how to authenticate users in Astro with Better Auth.",
                href: "https://www.launchfa.st/blog/astro-better-auth",
                tags: ["astro", "integration", "tutorial"],
        },
        {
                title: "Building Multi-Tenant Apps With Better-Auth and ZenStack",
                description:
                        "Learn how to build multi-tenant apps with Better-Auth and ZenStack.",
                href: "https://zenstack.dev/blog/better-auth",
                tags: ["multi-tenant", "zenstack", "architecture"],
        },
  ]
}
/>
```

# reference: Security

URL: /docs/reference/security
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/reference/security.mdx

Better Auth security features.

***

title: Security
description: Better Auth security features.

---

This page contains information about security features of Better Auth.

## Password Hashing

Better Auth uses the `scrypt` algorithm to hash passwords by default. This algorithm is designed to be memory-hard and CPU-intensive, making it resistant to brute-force attacks. You can customize the password hashing function by setting the `password` option in the configuration. This option should include a `hash` function to hash passwords and a `verify` function to verify them.

## Session Management

### Session Expiration

Better Auth uses secure session management to protect user data. Sessions are stored in the database or a secondary storage, if configured, to prevent unauthorized access. By default, sessions expire after 7 days, but you can customize this value in the configuration. Additionally, each time a session is used, if it reaches the `updateAge` threshold, the expiration date is extended, which by default is set to 1 day.

### Session Revocation

Better Auth allows you to revoke sessions to enhance security. When a session is revoked, the user is logged out and can no longer access the application. A logged in user can also revoke their own sessions to log out from different devices or browsers.

See the [session management](/docs/concepts/session-management) for more details.

## CSRF Protection

Better Auth includes multiple safeguards to prevent Cross-Site Request Forgery (CSRF) attacks:

1. **Non-Simple Headers**
   POST requests must either have a non-simple header or a `Content-Type` header of `application/json`. Non-simple headers are headers that are not included in the [simple headers](https://developer.mozilla.org/en-US/docs/Glossary/Simple_header) list.

2. **Origin Validation**
   Each requestâ€™s `Origin` header is verified to confirm it comes from your application or another explicitly trusted source. Requests from untrusted origins are rejected. By default, Better Auth trusts the base URL of your app, but you can specify additional trusted origins via the `trustedOrigins` configuration option.

3. **Secure Cookie Settings**
   Session cookies use the `SameSite=Lax` attribute by default, preventing browsers from sending cookies with most cross-site requests. You can override this behavior using the `defaultCookieAttributes` option.

4. **No Mutations on GET Requests (with additional safeguards)**
   `GET` requests are assumed to be read-only and should not alter the applicationâ€™s state. In cases where a `GET` request must perform a mutationâ€"such as during OAuth callbacks - Better Auth applies extra security measures, including validating `nonce` and `state` parameters to ensure the requestâ€™s authenticity.

You can skip the CSRF check for all requests by setting the `disableCSRFCheck` option to `true` in the configuration.

```typescript
{
  advanced: {
    disableCSRFCheck: true
  }
}
```

You can skip the origin check for all requests by setting the `disableOriginCheck` option to `true` in the configuration.

```typescript
{
  advanced: {
    disableOriginCheck: true
  }
}
```

<Callout type="warning">
  Skipping csrf check will open your application to CSRF attacks. And skipping origin check may open up your application to other security vulnerabilities including open redirects.
</Callout>

## OAuth State and PKCE

To secure OAuth flows, Better Auth stores the OAuth state and PKCE (Proof Key for Code Exchange) in the database. The state helps prevent CSRF attacks, while PKCE protects against code injection threats. Once the OAuth process completes, these values are removed from the database.

## Cookies

Better Auth assigns secure cookies by default when the base URL uses `https`. These secure cookies are encrypted and only sent over secure connections, adding an extra layer of protection. They are also set with the `sameSite` attribute to `lax` by default to prevent cross-site request forgery attacks. And the `httpOnly` attribute is enabled to prevent client-side JavaScript from accessing the cookie.

For Cross-Subdomain Cookies, you can set the `crossSubDomainCookies` option in the configuration. This option allows cookies to be shared across subdomains, enabling seamless authentication across multiple subdomains.

### Customizing Cookies

You can customize cookie names to minimize the risk of fingerprinting attacks and set specific cookie options as needed for additional control. For more information, refer to the [cookie options](/docs/concepts/cookies).

Plugins can also set custom cookie options to align with specific security needs. If you're using Better Auth in non-browser environments, plugins offer ways to manage cookies securely in those contexts as well.

## Rate Limiting

Better Auth includes built-in rate limiting to safeguard against brute-force attacks. Rate limits are applied across all routes by default, with specific routes subject to stricter limits based on potential risk.

## IP Address Headers

Better Auth uses client IP addresses for rate limiting and security monitoring. By default, it reads the IP address from the standard `X-Forwarded-For` header. However, you can configure a specific trusted header to ensure accurate IP address detection and prevent IP spoofing attacks.

You can configure the IP address header in your Better Auth configuration:

```typescript
{
  advanced: {
    ipAddress: {
      ipAddressHeaders: ['cf-connecting-ip'] // or any other custom header
    }
  }
}
```

This ensures that Better Auth only accepts IP addresses from your trusted proxy's header, making it more difficult for attackers to bypass rate limiting or other IP-based security measures by spoofing headers.

> **Important**: When setting a custom IP address header, ensure that your proxy or load balancer is properly configured to set this header, and that it cannot be set by end users directly.

## Trusted Origins

Trusted origins prevent CSRF attacks and block open redirects. You can set a list of trusted origins in the `trustedOrigins` configuration option. Requests from origins not on this list are automatically blocked.

### Basic Usage

The most basic usage is to specify exact origins:

```typescript
{
  trustedOrigins: [
    "https://example.com",
    "https://app.example.com",
    "http://localhost:3000"
  ]
}
```

### Wildcard Domains

Better Auth supports wildcard patterns in trusted origins, which allows you to trust multiple subdomains with a single entry:

```typescript
{
  trustedOrigins: [
    "*.example.com",             // Trust all subdomains of example.com (any protocol)
    "https://*.example.com",     // Trust only HTTPS subdomains of example.com
    "http://*.dev.example.com"   // Trust all HTTP subdomains of dev.example.com
  ]
}
```

#### Protocol-specific wildcards

When using a wildcard pattern with a protocol prefix (like `https://`):

* The protocol must match exactly
* The domain can have any subdomain in place of the `*`
* Requests using a different protocol will be rejected, even if the domain matches

#### Protocol-agnostic wildcards

When using a wildcard pattern without a protocol prefix (like `*.example.com`):

* Any protocol (http, https, etc.) will be accepted
* The domain must match the wildcard pattern

### Custom Schemes

Trusted origins also support custom schemes for mobile apps and browser extensions:

```typescript
{
  trustedOrigins: [
    "myapp://",                               // Mobile app scheme
    "chrome-extension://YOUR_EXTENSION_ID"    // Browser extension
  ]
}
```

## Reporting Vulnerabilities

If you discover a security vulnerability in Better Auth, please report it to us at [security@better-auth.com](mailto:security@better-auth.com). We address all reports promptly, and credits will be given for validated discoveries.

---

# reference: Telemetry
URL: /docs/reference/telemetry
Source: https://raw.githubusercontent.com/better-auth/better-auth/refs/heads/main/docs/content/docs/reference/telemetry.mdx

Better Auth now collects anonymous telemetry data about general usage.

***

title: Telemetry
description: Better Auth now collects anonymous telemetry data about general usage.
-------------------------------------------------------------------------------------

Better Auth collects anonymous usage data to help us improve the project. This is optional, transparent, and disabled by default.

## Why is telemetry collected?

Since v1.3.5, Better Auth collects anonymous telemetry data about general usage if enabled.

Telemetry data helps us understand how Better Auth is being used across different environments so we can improve performance, prioritize features, and fix issues more effectively. It guides our decisions on performance optimizations, feature development, and bug fixes. All data is collected completely anonymously and with privacy in mind, and users can opt out at any time. We strive to keep what we collect as transparent as possible.

## What is being collected?

The following data points may be reported. Everything is anonymous and intended for aggregate insights only.

* **Anonymous identifier**: A non-reversible hash derived from your project (`package.json` name and optionally `baseURL`). This lets us deâ€‘duplicate events per project without knowing who you are.
* **Runtime**: `{ name: "node" | "bun" | "deno", version }`.
* **Environment**: one of `development`, `production`, `test`, or `ci`.
* **Framework (if detected)**: `{ name, version }` for frameworks like Next.js, Nuxt, Remix, Astro, SvelteKit, etc.
* **Database (if detected)**: `{ name, version }` for integrations like PostgreSQL, MySQL, SQLite, Prisma, Drizzle, MongoDB, etc.
* **System info**: platform, OS release, architecture, CPU count/model/speed, total memory, and flags like `isDocker`, `isWSL`, `isTTY`.
* **Package manager**: `{ name, version }` derived from the npm user agent.
* **Redacted auth config snapshot**: A minimized, privacyâ€‘preserving view of your `betterAuth` options produced by `getTelemetryAuthConfig`.

We also collect anonymous telemetry from the CLI:

* **CLI generate (`cli_generate`)**: outcome `generated | overwritten | appended | no_changes | aborted` plus redacted config.
* **CLI migrate (`cli_migrate`)**: outcome `migrated | no_changes | aborted | unsupported_adapter` plus adapter id (when relevant) and redacted config.

<Callout type="info">
  You can audit telemetry locally by setting the `BETTER_AUTH_TELEMETRY_DEBUG=1` environment variable when running your project or by setting `telemetry: { debug: true }` in your auth config. In this debug mode, telemetry events are logged only to the console.

  ```ts title="auth.ts"
  export const auth = betterAuth({
    // [!code highlight]
    telemetry: { // [!code highlight]
      debug: true // [!code highlight]
    } // [!code highlight]
  });
  ```
</Callout>

## How is my data protected?

All collected data is fully anonymous and only useful in aggregate. It cannot be traced back to any individual source and is accessible only to a small group of core Better Auth maintainers to guide roadmap decisions.

* **No PII or secrets**: We do not collect emails, usernames, tokens, secrets, client IDs, client secrets, or database URLs.
* **No full config**: We never send your full `betterAuth` configuration. Instead we send a reduced, redacted snapshot of nonâ€‘sensitive toggles and counts.
* **Redaction by design**: See [detect-auth-config.ts](https://github.com/better-auth/better-auth/blob/main/packages/better-auth/src/telemetry/detectors/detect-auth-config.ts) in the Better Auth source for the exact shape of what is included. It purposely converts sensitive values to booleans, counts, or generic identifiers.

## How can I enable it?

You can enable telemetry collection in your auth config or by setting an environment variable.

* Via your auth config.

  ```ts title="auth.ts"
  export const auth = betterAuth({
    // [!code highlight]
    telemetry: { // [!code highlight]
      enabled: true// [!code highlight]
    } // [!code highlight]
  });
  ```

* Via an environment variable.

  ```txt title=".env"
  # Enable telemetry
  BETTER_AUTH_TELEMETRY=1

  # Disable telemetry
  BETTER_AUTH_TELEMETRY=0
  ```

### When is telemetry sent?

* On `betterAuth` initialization (`type: "init"`).
* On CLI actions: `generate` and `migrate` as described above.

Telemetry is disabled automatically in tests (`NODE_ENV=test`) unless explicitly overridden by internal tooling.
~~~